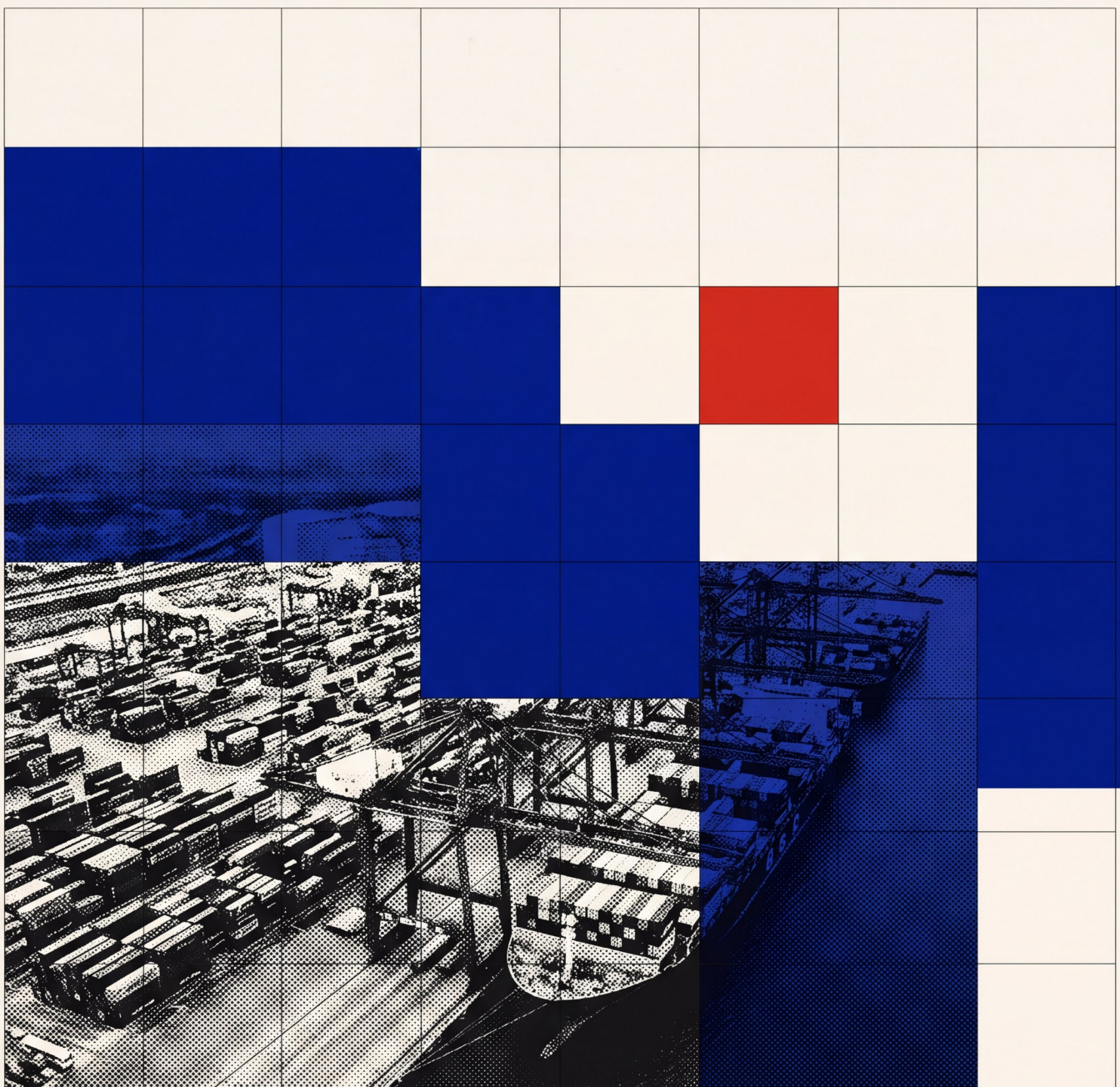


The Harbor, the Person, and the Economy

A textbook of accountable autonomous work

Erich Owens · Curiositech · Textbook Edition



The Harbor, the Person, and the Economy

A textbook of accountable autonomous work

Autonomy scales only when authority, evidence, and consequence remain coupled at every effect boundary.

This book follows that claim from one machine and one operator to work shared across principals, economic interests, and mutually distrustful systems. Its eight chapters are ordered by what they stand on; each proof chapter follows the chapter whose promises it keeps.

Erich Owens · Curiositech LLC · engineering@portdaddy.dev
Curiositech · Version 4.0 (Textbook Edition) · September 2026 ·
<https://portdaddy.dev/whitepaper>



Abstract

Agentic development fails in a predictable way when concurrency grows faster than accountability. More processes produce more work, but they also create more unread evidence, more overlapping effects, more disposable identities, and more ways for a confident report to outrun what actually happened. Better models do not remove this institutional problem.

This book makes one claim: *autonomy scales only when authority, evidence, and consequence remain coupled at every effect boundary*. Eight chapters establish that claim in the order the argument requires. They place one serial authority at the local write boundary; prove that delegated capability only narrows on its way there; seal the room in which two distrustful owners compute one answer, and price what its only slot can leak; derive the information cost of legible supervision; bind work to a durable principal and outcome history; separate payment, collateral, evidence, and settlement; connect witnessed acts to audit and bounded remedy; and state what may cross between mutually distrustful machines without moving either machine's sovereignty. The result is not a plea to trust agents. It is a design for making trust unnecessary where structure can suffice, measurable where uncertainty remains, and contestable everywhere else.

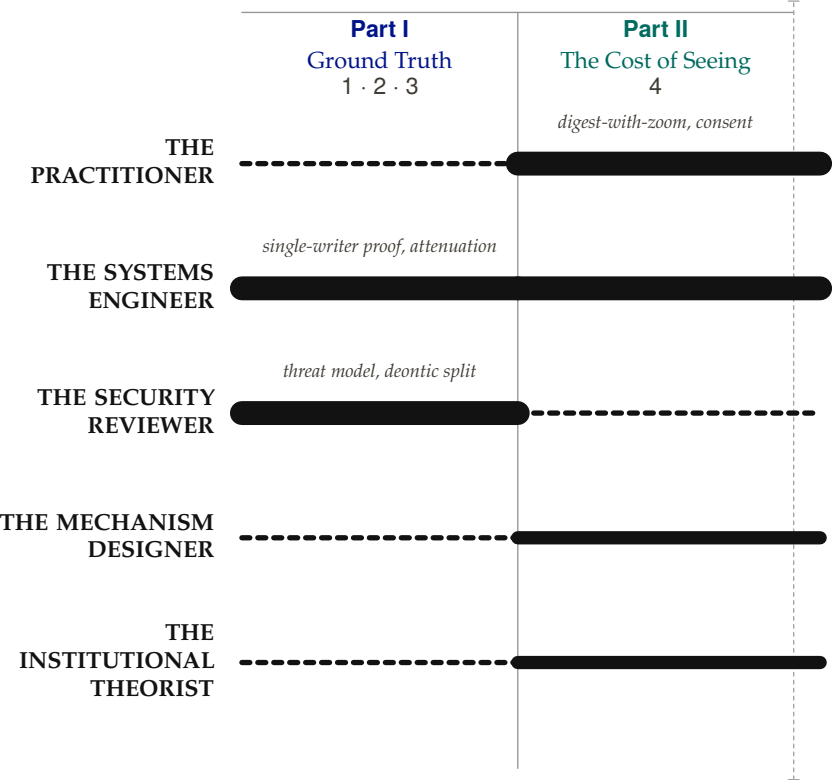
What is built, what is modelled, what is proposed

This book is ahead of the product, and it says so here rather than in a footnote. What runs today is a local daemon — a single-writer SQLite/WAL path for sessions, claims, messages, commitments, and coordination state — together with the operator projections derived from those records (each with a route back to the artifact it summarizes), durable work and obligation records that outlive any one process, a Rust card verifier whose harnesses are bounded proofs rather than full ones, and a relay that carries authenticated events outbound. Those parts are built and regression-tested, and each of them is graded *built, weakly* in the implementation boundary, because filesystem and network effects outside the mediated paths lie outside the transaction, and the grade is honest about that. Most of the rest is modelled and machine-checked but not running: the sealed room's noninterference and its release ledger, attenuation at every hop, the incentive to signal claims truthfully, conservation of the bond ledger, and the seventeen executed results whose numbers the chapters work by hand. And some of it is proposed, neither built nor modelled — the confined mode itself (an enforcement point below the agent: a per-agent machine with brokered system calls and metered egress), custody and settlement across authorities, and the federation's witness log.

Two consequences follow for how to read the chapters. First, the kernel chapter's supervisory-control result says that an interceptor at the tool level can observe but not regiment; the git shim and the coordination guard that ship today are such interceptors, and the book treats them as interim. The confinement the book prescribes requires the enforcement point below the agent. Second, whether the single-writer rail is the right *collaboration* model, and not only the right confinement model, is an open empirical question, and a pre-registered study kept separate from the product (studies/substrate-study in the repository) is testing it against per-agent worktrees with a merge queue. Until that study reports, the rail stands as a design invariant; the book does not yet know whether it is also a good model of collaboration, and it would rather say so than guess. Every important statement in the chapters is labelled by kind (theorem, design invariant, model-checked property, or empirical hypothesis), every runtime claim names its assurance mode, and the appendix keeps the ledger of both.

How to use this textbook

The unit of account in this book is the work, not the agent. A work unit is a case file: who authorized it, what it may touch, what it did, what evidence it left, what it still owes, and who pays if it fails. Every chapter builds or defends one part of that file. The chapters are ordered so that each stands on the ones before it, and each proving chapter follows the chapter whose promises it discharges. Read in order for the argument, or enter at the question you need: where a rule can be made real (*The Single-Writer Kernel*); how authority travels without growing (*The Anchor Protocol*); what two owners who distrust each other may compute together, and what its only exit may leak (*The Sealed Harbor*); what an operator must be able to see (*The Legible Swarm*); what remains long enough to owe anything (*From Spawn to Person*); what makes exchange rational (*The Harbor Economy*); who may decide, on what evidence (*The Bonded Commons*); and what may cross a trust boundary (*The Federated Harbor*). No chapter carries its own abstract or reader's map, in this edition or any other; each chapter simply opens on the question it answers and begins with its argument. What follows is the one reader's map this book keeps: not a table repeated once per chapter, but a single figure tracing five kinds of reader through the book's four parts, drawn once, in front, where a route can be checked against the whole argument instead of guessed one chapter at a time.



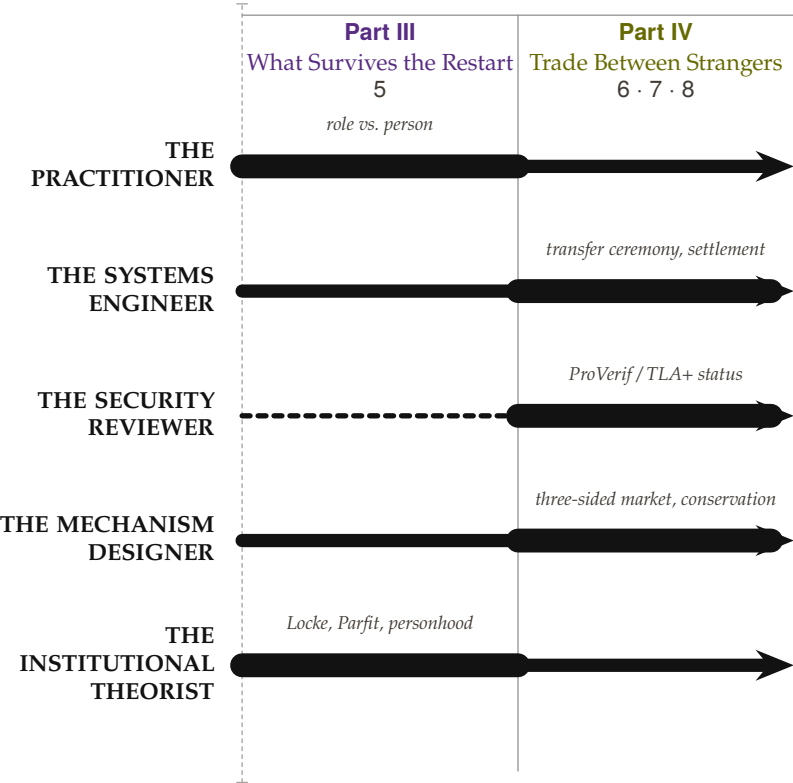


Figure 1: One reader’s map for the whole book, spread across two pages. Five archetypes, each a single flowing path left to right through Parts I–IV, continuing across the facing page at the dotted seam; line width is the encoding that matters — fat where that reader should read closely, thin and dashed where the part is safe to skim without losing the argument — and every path is named on the page, on both halves of the spread, rather than in a legend. Position and width carry the distinction; colour marks only the four parts themselves (the same per-part hue used throughout this edition), never one archetype against another, so the figure does not depend on the categorical palette currently being re-stepped for full-colour legibility (claude/figure-palette-separation). A sixth reader, the instructor checking exercises, is deliberately not drawn as a sixth lane: every chapter’s own closing EXERCISES section is that reader’s route, in every part at once, and a sixth flat line spanning the full width would say nothing a reader could not already infer. Five paths is close to the legibility ceiling for one spread; a design that needed a sixth or seventh would be better split into three pages than crowded onto two.

There are two ways through every result. Each opens with an express lane, the claim in one breath and a pointer to the box that states it precisely; readers who know the field read the lane and the box and move on, and everyone else reads straight through the scene, the analogy, the box, the numbers worked by hand, what the result buys, and where it stops. Worked numbers carry a tag: [verified] when a named

script in the repository regenerates them, [internal] when only the chapter's own derivation does. Exercises come in three kinds, CHECK, TRACE, and OPEN, rated one to three; their solutions are collected at the back of the book, and each exercise and solution links to the other.

Every important statement is labeled by what backs it. A *Theorem* follows from a stated formal model with explicit assumptions. A *Design invariant* is intended to hold and is backed by the implementation and its tests. A *Model-checked property* holds for a bounded model that a checker exhausted. An *Empirical hypothesis* awaits measurement. Run-time claims name one of five assurance modes, *Observed*, *Coordinated*, *Brokered*, *Confined*, and *Attested*, in increasing order of what the harbor itself enforces. The implementation status vocabulary is equally strict: **IMPLEMENTED** means running code exercised by tests; **PARTIAL** means a running slice that does not yet satisfy its full contract; **SPECIFIED** means a concrete design without a shipping realization; *proposed* is a research direction. Mathematical proofs, imported frameworks, finite model checks, repository experiments, and running code remain separate kinds of evidence throughout, and none is silently promoted into another.

Four ways to teach from this book. The eight chapters are written to be read in order and they do not have to be. Each of the paths below is a coherent semester that ends somewhere, rather than a subset that merely omits things, and each names the chapter that carries the weight of that discipline's own question.

Two warnings that apply to all four. Every path starts at the chapter that grounds it rather than at the chapter that sounds most like the course title, which is why three of the four begin with the kernel. And no path is self-contained on the formal side: results are cross-cited across parts on purpose, so a path's chapters will reach for boxes outside it, and the Result Atlas at the back is the map for that rather than a reason to widen the path.

How a claim about the harness earns its grade. The vocabulary above grades claims about mechanisms. Claims about the harness itself — that a compaction preserved the agent's intent, that a conflict was settled rather than papered over, that a resumed body actually continued the work — are graded the same way, and for them the book borrows the vocabulary its own experiments use. A *Genesis* is a reproducible initial world and a schedule of perturbations: a repository, some agents, and the failures, messages, and context pressure they will meet. A *Voyage* is one execution of a Genesis under a particular model and harness. The *Wake* is the behaviour it leaves; the *Logbook* is the structured record of that behaviour; the *Receipt* is the set of claims we are willing to assert afterwards, adjudicated against the Logbook. The unit of evidence is the provable transition: a before-state, a stimulus, what the harness

Table 1: Course paths. A path’s chapters are read in the order given, which is always the book’s own order; the rest of the book becomes reference.

Course	Chapters	What the path is about
Distributed Systems and Security	1, 2, 3, 8	One writer and one history; authority that can only narrow; a room whose exit is metered; and what survives crossing a trust boundary between parties who share no daemon.
Multi-Agent Economics and Mechanism Design	5, 6, 7	What has to persist before anything can owe anything, what makes exchange rational between strangers, and who decides on what evidence when the bond is called.
Human Supervisory Control and Interface Design	1, 4, 7	The kernel as the thing being supervised, the price of seeing it in bits, and the governance surface an operator actually acts through.
Formal Methods and Software Verification	1, 2, 7, 8	Supervisory control and decidability, mechanized protocol proofs and what their harnesses do not cover, TLA ⁺ over a coordination lifecycle, and a sheaf-theoretic detector with a stated blind spot.

observed and put in front of the model, which authority permitted or refused the next act, what the agent did, the after-state, and the receipt that binds them. That is the work unit’s receipt chain of *The Single-Writer Kernel* applied to the harness’s own conduct. The invariant is the one this book applies to everything else. Watching a feature run once and behave establishes nothing (one of us has watched a harness behave beautifully for an evening and fall over at breakfast); what establishes a harness claim is a controlled Genesis that produces a Voyage whose consequential transitions are supported by durable evidence and an adjudication receipt. The first Genesis the book cites is the substrate study named above, and the same Genesis replayed under different harness conditions is how the questions in this book that are still empirical hypotheses become results.

Evidence is typed as well. When a chapter reports what a harbor observed, it names the channel that produced the observation, and each channel has a blind spot that no amount of volume removes. A *deterministic verifier* (a test, a type checker, a model checker) shows that a stated property holds of the artifact it examined, and nothing about properties it did not state. A *provenance witness* (a signature, a hash, a Merkle inclusion, an append-only log) shows attribution and tamper-evidence relative to a commitment, never that every event was logged or that a description is true. A *protected meter* (a counter its subject can-

not rewrite) shows a quantity, not its cause. *Expert judgment* shows an assessment under a named rubric and carries the assessor's error rate. *Buyer preference* shows what was chosen, not that the choice was right. *A delayed outcome* shows what happened, and arrives too late to prevent it. The reader should ask the same question of any dashboard: which channel, and what can it not show.

Cross-references are live. Chapter titles, section numbers, theorems, figures, tables, exercises, page numbers, and citations are links, and every bibliography entry ends with the pages that cite it. The standalone research papers and earlier printings cite chapters by first-edition numerals; the concordance on the next page translates.

The argument, chapter by chapter

Each chapter stands on the ones before it. A chapter marked as a proof discharges a promise that an earlier chapter had to make and could not yet keep.

PART I · GROUND TRUTH

1 The Single-Writer Kernel

One writer, one durable file, decides what is true so nothing above it has to guess.

2 The Anchor Protocol *Proves what The Single-Writer Kernel assumes*

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

3 The Sealed Harbor *Proves what The Single-Writer Kernel assumes*

A work order runs in a room with two fences and two gates, the only thing that leaves is what the slot lets out, and the budget of what can leak is a ledger that conserves.

PART II · THE COST OF SEEING

4 The Legible Swarm

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.

PART III · WHAT SURVIVES THE RESTART

5 From Spawn to Person

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

PART IV · TRADE BETWEEN STRANGERS

6 The Harbor Economy

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.

7 The Bonded Commons *Proves what The Harbor Economy assumes*

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

8 The Federated Harbor *Proves what The Harbor Economy assumes*

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

First-edition numbering. The standalone research papers and earlier printings cite these chapters by the Roman numerals of the first edition, which followed the order the chapters were written in. This edition orders them by what they stand on. The concordance:

First edition	This edition	Chapter
I	4	<i>The Legible Swarm</i>
II	1	<i>The Single-Writer Kernel</i>
III	5	<i>From Spawn to Person</i>
IV	6	<i>The Harbor Economy</i>
V	2	<i>The Anchor Protocol</i>
VI	7	<i>The Bonded Commons</i>
VII	8	<i>The Federated Harbor</i>

Contents

What is built, what is modelled, what is proposed i

How to use this textbook ii

The argument, chapter by chapter viii

1 Introduction: the unit of account is the work xiv

PART I GROUND TRUTH.....1

1 The Single-Writer Kernel 3

1.1 Where this paper sits, and what it promises 4

1.2 The kernel as seven subsystems 7

1.3 The substrate subsystem: one writer, one file 9

1.4 Durability, split by fault class 12

1.5 The resource subsystem: claims, locks, and fair exclusion . . . 16

1.6 The communication subsystem: the bus, stigmergy, and two delegation chains 23

1.7 The obligation & enforcement subsystem: the sovereign’s two levers 27

1.8 The continuity subsystem: memory, checkpoint, and the foundation of the economy 46

1.9 The self-attestation subsystem: honest green 48

1.10 The invariants, stated as theorems 49

1.11 Cross-subsystem atomicity: a buildable defect, not a frontier . 57

1.12 Threat model and the limits of mediation 58

1.13 Adjacency contract: what the kernel assumes and provides . . 66

Review of the key ideas 68

1.14 Exercises 69

1.15 Related work 73

1.16 Open problems 75

1.17 Conclusion: solid where local, provisional where it must grow 79

1. Chapter Appendices 80

1.A Implementation & status 80

2 The Anchor Protocol 83

2.1 What problem this solves 84

2.2 The Anchor Protocol Architecture 85

2.3 How we know it is correct 100

2.4 The verified crate is the deployed crate 107

2.5 What the adversary can do, and what it cannot 110

2.6 Proofs are leverage, not decoration 114

2. Chapter Appendices 116

2.A Formal Verification Status 116

2.B Full ProVerif Models 118

2.C Phase 2: Asymmetric Ed25519 119

2.D Phase 3: Multi-hop Delegation 120

2.E Limitations & Boundaries 126

2.F Further reading and prior art 126

Review of the key ideas 128

2.G Exercises 129

3	The Sealed Harbor	131
3.1	Two owners, one answer	132
3.2	One work order, two fences, two gates	135
3.3	Silence except through the slot	143
3.4	The gate sits on the only enforceable boundary	148
3.5	The budget is a ledger, and the ledger conserves	151
3.6	Canaries with a power curve and a clock	156
3.7	The leakage budget: $q \cdot b$ bits, before timing	160
3.8	Limitations and boundaries	162
	Review of the key ideas	164
3.9	Exercises	165
3.10	History and references	166

PART II THE COST OF SEEING 170

4	The Legible Swarm	172
4.1	Introduction: the swarm is a state of nature	174
4.2	Consent to the Leviathan (the normative claim)	178
4.3	The mechanism: legibility-with-zoom	183
4.4	The Leviathan rules, it does not just watch (the authority half)	188
4.5	The operator as a modeled entity	200
4.6	Read-poverty: the binding constraint at scale	208
4.7	Discovery: the directory substrate and the split ranker	218
4.8	Tokens: the swarm's COGS and its legibility engine	228
4.9	Reconciling the canon: roles, consent, and local knowledge	237
	Review of the key ideas	240
4.10	Exercises	241
4.11	Related work and prior art	246
4.12	The time axis and the bridge to the economy	249
4.	Chapter Appendices	252
4.A	Implementation and status	252
4.B	Notation and the nomenclature key	255

PART III WHAT SURVIVES THE RESTART 256

5	From Spawn to Person	258
5.1	Introduction: the spawn that cannot be paid	259
5.2	Prior art, and where this paper departs from it	261
5.3	The role / person distinction, made precise	263
5.4	Continuity is a personal-identity problem	266
5.5	The three subsystems of continuity	272
5.6	Identity: the root the whole chain hangs from	276
5.7	The keystone split: local identity vs. cross-operator attestation	292
5.8	Reputation as a substrate, not a score	294
5.9	Why reputation is <i>not</i> a bandit problem	296
5.10	The multi-dimensional reputation protocol	298
5.11	The grading oracle's incentive-compatibility	304
5.12	Revocation, tombstones, and bounded memory	311
5.13	What this chapter hands the market	314
	Review of the key ideas	316
5.14	Exercises	317

5.15	Open problems (the starred exercises, collected)	321
5.16	Conclusion	323
5.	Chapter Appendices	324
5.A	Implementation status of the reference system	324

PART IV TRADE BETWEEN STRANGERS 326

6 The Harbor Economy 328

6.1	The thesis: a market, not a payment rail	330
6.2	What a “side” is, and why there are three	331
6.3	The float plan: the built floor	338
6.4	Three sides on one escrow	341
6.5	The keystone the market rests on — and does not yet have	351
6.6	Conservation under composition	353
6.7	The Anchor Protocol proper: cross-harbor capability transfer	356
6.8	Federation: trading across machines you don’t own	358
6.9	Reputation: monotone, but revocable	360
6.10	Hosted trust is the product	362
6.11	Cold start, and the auction question	363
6.12	A gluing analogy for the open federation problem	366
6.13	Gaps the design must still close	366
	Review of the key ideas	369
6.14	Exercises	369
6.15	Related work	373
6.16	One further market: adversarial testing and purchased assurance	377
6.17	Conclusion	382
6.	Chapter Appendices	383
6.A	Implementation & status	383
6.B	Proof sketches	385

7 The Bonded Commons 387

7.1	Introduction: The Trust Problem	388
7.2	The Commons Authority	395
7.3	Layer 1: Structural Prevention	398
7.4	Layer 2: Immutable Attribution	403
7.5	The Limits of Decisive Allocation	407
7.6	Crash Recovery as Social Continuation	413
7.7	Layer 3: Economic Alignment	415
7.8	Coordination as Substrate: An Expressive-Act Taxonomy	436
7.9	Semantic Cadence, Agentic Psychosis, and Observability	437
7.10	Formal Model	439
7.11	Related Work	446
7.12	Discussion and Limitations	449
	Review of the key ideas	453
7.13	Exercises	453
7.14	Conclusion	456
7.	Chapter Appendices	458
7.A	Formal Verification Status	458
7.B	Worked Example: One Agent, All Layers	460
7.C	Scope Boundary Checklist	463

8 The Federated Harbor **465**

8.1 Introduction: The Two-Machine Problem 466

8.2 The Federated Authority 469

8.3 Threat Model 473

8.4 Cross-Harbor Capability Transfer 478

8.5 Federated Revocation 481

8.6 Cross-Harbor Settlement 495

8.7 Federation Admission 501

8.8 Worked Example 504

8.9 Formal Model 509

8.10 Failure Modes 511

8.11 Related Work 512

8.12 Limitations and Open Questions 516

Review of the key ideas 518

8.13 Exercises 519

8.14 Conclusion 521

8. Chapter Appendices 522

8.A Verification Status 522

8.B ProVerif Models (draft) 523

8.C TLA+ Specification (draft) 524

SOLUTIONS TO THE EXERCISES **525**

APPENDICES **580**

A Implementation boundary 580

B Mechanized claims 582

C Result atlas 598

D Notation and cross-chapter concordance 600

References 603

1 Introduction: the unit of account is the work

Agentic software is usually measured by the agent: capability, runtime, tool access, and parallel copies. That is the wrong unit of account. A process may disappear, rename itself, summarize its mistakes, or hand work off. The durable object must be the work: its authorization, permitted effects, evidence, open obligations, and liability for the outcome.

This book develops that claim in eight propositions, one per chapter, in the order the book is read.

1. **Prevention begins at the effect boundary.** Rules can be enforced before an act only when the system controls the channel through which the act occurs. Local effects therefore pass through one serial authority; semantic failures that no gate can decide remain matters of detection and remedy.
2. **Delegation only narrows.** Every child capability is a strict subset of its parent's authority and lifetime. Authentication says who signed; attenuation says what the signature may authorize. Both are needed at every hop.
3. **Confinement is a property of the room, not of the tenant.** Two owners who trust neither each other nor the venue can still take one answer out of a sealed room when its only exit is a metered slot: silence except through the slot, a gate on the only enforceable boundary, a budget that conserves, and canaries for what the gate cannot prove. What the room cannot promise is stated as a limit, at the same prominence as the claims.
4. **Supervision has an information cost.** A digest cannot guarantee that a human will notice any one of many relevant events unless it spends enough bits, attention, or artifact openings to distinguish them. A summary is therefore an index into evidence, not a substitute for it.
5. **Accountability must outlive the process.** A task acquires a durable principal, capability history, evidence record, obligations, and outcome. Restarting an agent or choosing a new name must not erase that case file or refill the resources it already consumed.
6. **Exchange requires separated instruments.** Payment rewards the requested work. Collateral prices a bounded failure. Evidence supports a claim. Settlement closes it exactly once. Combining those functions in one score or token makes each of them less trustworthy.
7. **Uncertainty needs an institution.** Some harms cannot be prevented and some outcomes cannot be judged mechanically. Witnesses, randomized audits, bounded bonds, appeals, and conservation rules turn that residual uncertainty into a process that can be inspected and contested.

8. **Federation relays evidence, not sovereignty.** Mutually distrustful machines may exchange signed capabilities, revocations, witness roots, and settlement claims without sharing a database or accepting a global ruler. Transport can carry evidence; it cannot decide what either machine is permitted to do with it.

Together they form one causal chain. The kernel marks where effects become real; the protocol keeps delegated authority from growing on its way there; legibility states what a human must recover; continuity gives the record a durable subject; the economy bounds consequence and prices it; the commons connects evidence to remedy; federation carries those objects without pretending that transport creates consensus.

1.1 What counts as knowing

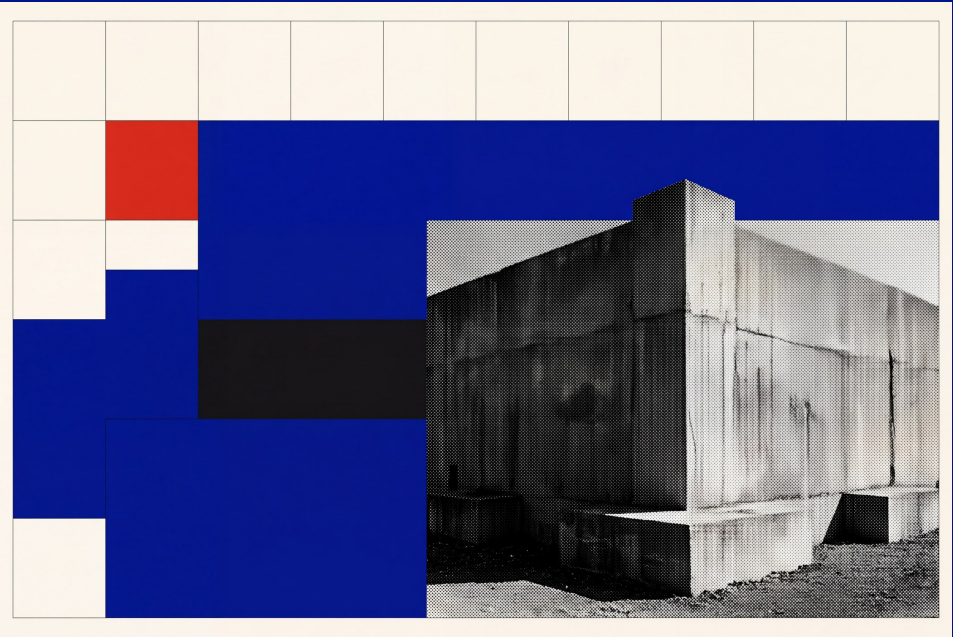
Every consequential statement has a status and a boundary. Derivations establish relations under named assumptions. Finite checks exhaust a stated state space, not every deployment. Simulations estimate behavior under a stated generator; repository tests exercise named paths; running code proves availability, not universal mediation. Counterexamples remain in the record because they mark claims the research killed rather than conclusions it merely failed to prove.

Appendix A gives the implementation boundary; Appendix C records the results and their limits. The companion docs/harbor-research/mega-volume-epistemic-manifest.yaml preserves each result's source, status, conditions, boundary, chapter home, and verification artifact. The chapters stand alone; the book's claim is the chain they make.



Ground Truth

One writer decides what is real, every request that reaches it arrives with authority that can only have narrowed, and what runs on its behalf runs in a sealed room. Three chapters build the machine every later chapter answers to.



1 The Single-Writer Kernel

One writer, one durable file, decides what is true so nothing above it has to guess.

2 The Anchor Protocol

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

3 The Sealed Harbor

A work order runs in a room with two fences and two gates, the only thing that leaves is what the slot lets out, and the budget of what can leak is a ledger that conserves.

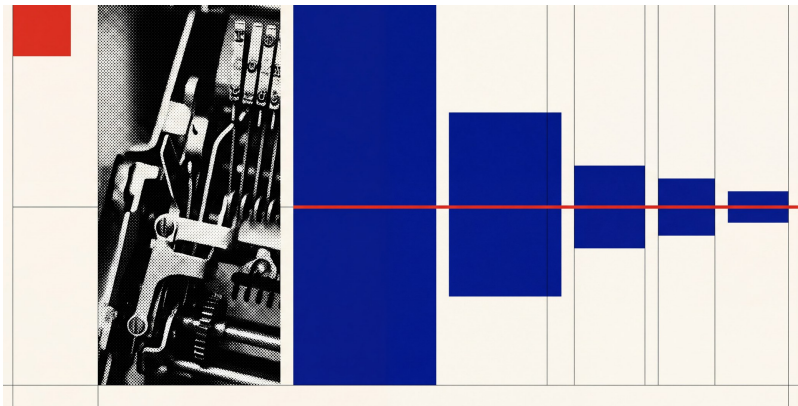
1

The Single-Writer Kernel

The question this chapter answers

Where can a rule be made real?

One writer, one durable file, decides what is true so nothing above it has to guess.



A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

Leslie Lamport, message to a DEC SRC bulletin board, 28 May 1987

WHERE CAN A RULE BE MADE REAL? Six agents may agree about a file and still overwrite one another. A log can report the collision after the fact; only a mediated writer can refuse the second effect before it lands. This chapter identifies that local writer, separates preventable rules from detect-only claims, and states the work-unit invariants that every later institution assumes. R5 and R8 set the boundary.

1.1 Where this paper sits, and what it promises

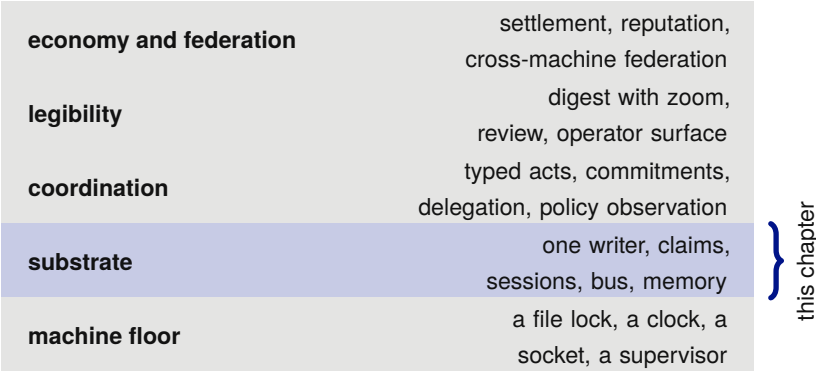


Figure 1.1: An architectural section through the four-layer coordination stack. Each band names a layer and what that layer hands the one above it; the shaded band is this chapter’s, the layer that rests directly on the machine floor. The dashed floor band is not a layer of the system but the narrow dependency boundary the substrate assumes: a file lock, a clock, a socket and a supervisor. [internal]

We model an agent-coordination system as a four-layer stack (Figure 1.1), each layer serving a different *whom*.

Substrate (the machine).

A single always-on daemon over one local database: durable storage and serialized mutation. *This paper’s core.*

Coordination (the agents).

The protocol agents speak over the substrate: ownership claims, a message bus, stigmergic markers, obligations, and policy enforcement. This paper specifies the implemented half of it.

Legibility (the operator).

The human-facing view that renders the swarm intelligible. Treated in an accompanying chapter (*The Legible Swarm*).

Economy (the market between operators).

The cross-machine market in which operators trade work and reputation. Treated in an accompanying chapter (*The Harbor Economy*).

You climb the stack in order, and you never buy a layer before you need it. This paper is the structural floor. The legibility layer reads the kernel's claims, bus, and policy monitor to render the swarm; the personhood argument leans the entire notion of a *durable agent identity* on the kernel's continuity machinery; the economy's settlement ledger is, mechanically, another set of tables under the same single writer, and its cross-machine capability transfer rides the kernel's authorization chain. If this floor is unsound, everything above it inherits the fault. So the discipline of this paper is to state each promise precisely and, where the promise has an edge, to draw the edge in ink.

1.1.1 The one-sentence thesis

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database — the one process that mediates contended resources, and the one place where “true” is decided rather than asserted.

THESIS

That is the whole claim, and it has two halves, which are the kernel's entire job; everything else is convenience.

1. **Durability (with an edge):** a write that returned success is durable across the death of any agent — and, we will be careful, across the death of the *daemon process*, though *not* unconditionally across power loss.
2. **Mediation (with a correction):** a state the kernel *regiments* cannot come to exist; a state the kernel merely *enforces* is detected and compensated within a bounded window. These are different guarantees and we will never conflate them.

1.1.2 Why “reference monitor,” and why *not* consensus

The right mental model is the **reference monitor** of Anderson and Lamson [10, 143]: a mediator that is *always invoked* (complete mediation), *tamper-evident*, and *small enough to verify*. The kernel instantiates this not as an abstract security kernel but as a concrete local daemon over one SQLite file. Complete mediation holds *by construction over the kernel's own state*: because every mutation of *coordination state* routes through one writer holding one file lock, the operating system's serialization is the mediation primitive — Saltzer and Schroeder's “economy of mechanism” [224] done with one moving part instead of *N*

hand-rolled critical sections. We are deliberate about the scope of the word *complete*: it is complete over writes that *enter* the daemon, not over what a same-user agent can do *beside* it. A classical reference monitor earns “always invoked” from the hardware/operating-system boundary that forces every access through it; this kernel has no such boundary at the substrate layer, so a same-user process can mutate shared state the daemon never sees — edit a file under an advisory claim, run `git` directly, or write the SQLite file itself — without routing through the writer. Mediation is therefore complete over the daemon’s *application-program interface* and advisory everywhere else; making it complete over the agent’s host capabilities is an out-of-band enforcement problem we scope to the threat model (§1.12) and credit, when it lands, to a separate-user kernel boundary that does not yet exist (§1.12.2).

The swarm *looks* like a distributed-systems problem — many agents, contention, failover — so the trained reflex is Raft, Paxos, CRDTs. The kernel’s defining move is to refuse that problem. One writer, one machine, one file: there is nothing to *agree* on, so the consensus impossibility of Fischer, Lynch, and Paterson [81] does not bite. It is sidestepped, not solved. Distribution is a cross-machine concern that belongs to the economy layer; pushing a consensus protocol down into the local substrate would be the wrong layer.

This is the single most important architectural decision a local-first coordination layer can make, and the field is littered with systems that got it wrong by manufacturing a consensus problem they did not have [134]. We will state the formal payoff of the choice as a conditional theorem in §1.10: when every read and write is routed through the sole daemon connection and responses follow commit, transactions are *serializable* and claim/lock operations are *linearizable* — the property that lets the coordination layer treat “who holds this” as ground truth without any agreement protocol at all.

1.1.3 A research-maturity scale, and why the grades matter

A systems paper that mixes “we proved this” with “we hope to build this” is unfalsifiable. So every consequential claim in this paper carries an explicit *maturity grade* on a four-point scale, plus two refinements introduced here because a single grade cannot formally carry a claim that spans fault classes or interception points. The grade is a property of the *claim*, not a marketing adjective: it says how far the claim is from a verified, running artifact. The concrete artifacts behind each grade are catalogued in Appendix 1.A; the body argues at the level of mechanisms.

The grades are not decoration. The two most consequential corrections in this paper — the durability split and the regimentation/detection



Lamport’s 1971 Protection gave operating systems the access-matrix model this kernel still enforces, one gate at a time.

KEY IDEA

PITFALL

Table 1.1: The research-maturity scale used throughout. The last two rows are refinements this paper introduces (see §1.4 and §1.7).

Grade	Meaning
IMPLEMENTED	Realized, exercised, and true under the production runtime.
PARTIAL	Realized but partial: holds under a narrower condition than the prose might invite, or detects rather than prevents.
SPECIFIED	A concrete design exists, but no running artifact realizes it.
<i>proposed</i>	Named as a direction; no design yet.
I1a / I1b	Durability <i>split by fault class</i> : I1a (process-crash) is IMPLEMENTED ; I1b (power-loss) is <i>not guaranteed</i> under the default WAL configuration. A claim that conflates fault classes cannot carry one grade.
regimented / enforced	Prohibition split by interception point: <i>regimented</i> = rejected pre-commit, truly unreachable (a uniqueness constraint / boot-admission gate); <i>enforced</i> = detected post-commit, halted or compensated within a bounded window (the policy monitor). “Physically unreachable” is reserved for the regimented set.

split — are exactly the places where a single optimistic grade would let the kernel’s foundational promise be read as stronger than the artifact delivers. When a system claims “the database survives every agent’s death,” a reader hears “power-loss durable.” It is not, by default. Saying so is the difference between a systems paper and a brochure.

1.2 The kernel as seven subsystems

We organize the kernel into seven *subsystems*, each a contract the daemon must hold for the layers above to be coherent (Table 1.2). The temptation is to describe such a kernel as a flat noun-list of features — ports, claims, sessions, a bus, markers, commitments, a monitor, a memory store — which is the symptom of treating the kernel as a database. It is not a database. It is a system of record *plus* a mediator, and naming the subsystems surfaces the dependencies a noun-list hides.

The **substrate subsystem** underlies the other six; mechanically, they are tables *with discipline* over the same file. We treat the substrate and the two subsystems that carry the paper’s sharpest corrections (resource/exclusion and obligation/enforcement) in full, and the rest with the depth their novelty warrants.

Table 1.2: The seven subsystems, each the contract the daemon holds for the layer above it, with the table that is its home where the chapter names one and its maturity grade. All seven are disciplines over one SQLite/WAL file: one commit history, not seven stores. The grades come from the chapter’s own status table (§1.A).

Subsystem	Contract held for the layer above	Maturity
transactional substrate	one writer, one durable file; every other subsystem is a table with discipline over the same commit history	IMPLEMENTED
resource / exclusion	at most one compatible holder per conflict domain; leases expire and are swept lazily (home: <code>claims</code>)	IMPLEMENTED
actor continuity	a stable actor identity across sessions, mechanically present but asserted by the actor, not proven	PARTIAL self-asserted
message carriage	an ordered, cursor-addressable, durable envelope delivered at least once (home: <code>messages</code>)	IMPLEMENTED
obligation & enforcement	no <i>done</i> without a receipt the daemon can re-check; the policy monitor detects and cannot always prevent (home: <code>commitments</code>)	PARTIAL partial
session memory	notes and events survive process death; execution state does not	IMPLEMENTED (memory)
self-attestation	the daemon reports its own coverage as enforced, degraded or stubbed, never silently	IMPLEMENTED

1.3 The substrate subsystem: one writer, one file

The substrate underlies everything else, mechanically nothing more than a table with two promises to keep — durability and serialized mutation — and to keep them formally. We cover the single-writer discipline here and durability in §1.4, because durability is where the most consequential correction lives.

1.3.1 The single-writer discipline

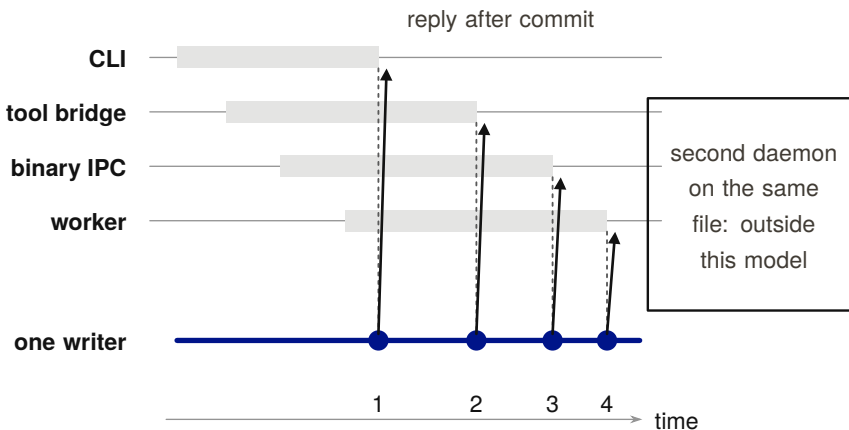


Figure 1.2: Four concurrent callers and one writer. Each band is a request in flight; its right edge is where the daemon commits it, and the rail shows the resulting order 1–4. A reply is sent only after the commit. A second daemon opening the same file is outside the model, not another lane. [internal]

The kernel’s concurrency story (Figure 1.2) is: dozens of command-line invocations and agent connections funnel through one writer — the daemon, the sole holder of a read–write connection to the database file — and SQLite’s operating-system file lock plus a bounded busy-wait serialize all mutations. We must be precise about *what* serializes writers, because there are two stories and only one is true at a time.

Definition 1.3.1 (Single-writer discipline). All state-mutating operations route through a single daemon process holding one SQLite write connection. Concurrency among the many clients (command-line invocations, agents over the message transports, background workers) is resolved by the daemon’s request queue; the SQLite file lock is the *backstop* that preserves correctness if the discipline is ever violated by a second writer (for example a stray direct write), not the primary serialization mechanism.

“Single-writer” is an architectural *discipline* (everything goes through the daemon), backstopped — not replaced — by SQLite’s file lock. This distinction matters: SQLite permits multiple writer connections (it serializes them, it does not reject them), so the property holds because of where writes are routed, not because the database forbids a second writer. The standing risk is therefore a second *copy* of the daemon coming up against the same file — which is why the substrate also runs self-checks that detect a divergent or stale second instance (§1.9).

KEY IDEA

1.3.2 The configuration is the durability claim

The substrate’s storage configuration is short and decisive. In WAL mode with the *normal* synchronization level (the WAL is fsync’d at checkpoints rather than on every commit), a bounded auto-checkpoint interval, a bounded busy-wait, foreign-key enforcement on, and a clean-shutdown checkpoint-and-truncate, the kernel inherits exactly the crash semantics that configuration defines. The claim “a write that returned success survives a crash” reduces *entirely* to what WAL with normal synchronization guarantees — which is the subject of §1.4, and which is not what an unguarded reading assumes.

1.3.3 Schema evolution: an ordered boot ledger, not yet a sequencer (OP-7)

The initial schema-by-union approach (CREATE TABLE IF NOT EXISTS) created vulnerabilities around unsequenced renames and multi-column backfills. The kernel’s answer so far is a partial one, and the honest way to report it is to name the trade it makes rather than the property it saves.

What ships (PARTIAL) is a **boot-time migration ledger**: the daemon applies a numbered, totally ordered set of migrations before it serves any request, and then *verifies the schema it is about to serve from* — probing the actual tables and column sentinels rather than trusting that the statements ran without raising — and refuses to serve on a mismatch. That is a real fail-safe default, and it is the reason a half-applied migration cannot quietly become coordination truth.

What does *not* yet ship is the sequencer that would make that order authoritative. Subsystem modules still self-initialize their own tables over the shared database handle (Appendix 1.A), so the schema is still, in part, a union. A strict linear ledger managed exclusively by the daemon via SQLite’s pragma `user_version`, in which modules *declare* migrations rather than applying them, is SPECIFIED. OP-7 therefore stands PARTIAL: this is Table 1.1’s second question — “is a version table unavoidable?” — answered *probably yes*, at the cost of the “any module, any order” flexibility the schema-by-union design offered, and not yet paid for in full. Table 1.3 states the trade in two rows.

Table 1.3: Schema evolution (OP-7): what ships today against the sequencer the design would still need. [internal]

State	What governs migration order
ships (PARTIAL)	a numbered, totally ordered boot-time migration ledger, applied before any request is served, with post-apply verification against the actual tables; organ modules still self-initialize their own tables, so the schema is still, in part, a union
designed (SPECIFIED)	a strict linear ledger managed exclusively by the daemon via <code>pragma user_version</code> , in which modules <i>declare</i> migrations rather than applying them themselves

1.3.4 Path and ownership invariants

The database resolves to a single canonical path with owner-only file permissions (historically the file also carried the system’s signing key in plaintext; that secret is being migrated to the operating system’s keychain, see §1.12). A fail-closed guard refuses to open the live registry from a test context — the analogue of a framework’s protected-environment check. This is Saltzer and Schroeder’s *fail-safe defaults* [224] applied at the one place a test could corrupt production truth.

1.3.5 The second writer: institutional truth above the local harbor

A team runs several harbors and wants one plan across them: which objectives are authorized, who owns what, which transitions were accepted. The product calls that surface **Chartroom**, under the umbrella name Grand Harbor, and its design record (ADR-0137, Proposed, 2026-09-01) is the first place this chapter’s invariant meets a second writer. The record exists on an unmerged branch; no Chartroom code is on the main line, and the book grades it `SPECIFIED` throughout this section.

The design rule is short enough to state before the mechanism. *A second writer above the local harbor is either a projection of the local record or the writer of it, never both, and the switch between the two is an epoch.* A projection may be copied, searched, and rendered anywhere; it carries no authority, and a client that writes to it has written nothing. The writer is the one process whose accepted transitions define the record, and there is one per record per epoch. The design record draws this line itself: the harbor’s own signing key is the sole signer of Chartroom intents, the relay verifies and applies an intent but never authors one, and the record rejects reusing the existing roadmap-snapshot route precisely because that route is a full-replace mirror whose contract keeps the daemon authoritative. That is a projection, and the record declines to let a projection masquerade as a writer. This is a *design invariant*:

intended to hold, backed by the single-writer implementation for the local case, and not yet exercised across harbors by any code.

Moving the writer is the part no product yet does, and it wants a procedure rather than a setting — six steps, in this order and no other. (1) Freeze: the local harbor stops accepting writes and records the freeze as its last transition. (2) Export: the ledger and its root hash leave the harbor as one artifact. (3) Verified import: the receiving authority imports the artifact and proves, against the root, that what it holds is what left. (4) Check: projections and permissions are re-derived on the new side and compared with the old. (5) Epoch: the new writer's epoch is signed and published, and every client refuses a write under any older epoch from that moment. (6) Rollback only by another epoch: there is no return to the old writer except by running the same procedure in reverse, so the record never has two writers, even for the duration of a mistake. The reason the switch must be an epoch and not a flag is the same reason revocation must be an epoch in *The Anchor Protocol*: the model-checked attack there restores a backup taken before a revocation and thereby resurrects a dead credential, and it is refuted only when state carries the epoch it was valid under. A restored harbor database that predates the cutover is the same attack against authority, and the same defence closes it. That transfer is a *model-checked property* for credentials and a *design invariant* for writers; nothing in the repository yet checks the writer case.

The alternative the procedure exists to forbid is dual writing with reconciliation: both authorities accept writes for a while and receipts reconcile them afterwards. That is two truths, the condition every result in this chapter assumes away, and no receipt repairs it after the fact, because the receipt itself would need a writer. The design record's own deployment plan is consistent with this section: a staged migration, a fixture receipt, a rollback probe, and a root-authorized production write, with no import of existing plans or documents in the first step. Until that plan runs, a remote Grand Harbor is a projection over local authorities, which is exactly what the relay is today.

WHERE THIS STOPS

Exercises 1.4–1.6, p. 69.

1.4 Durability, split by fault class

This is the most important correction in the paper, so it gets its own section. The kernel's foundational promise — “a write that returned success survives” — is true, but only for the fault class a careful reader expects and not for the one the pitch most invites.

Table 1.4 draws the split this section defends: the same write, judged against two different fault classes with two different answers.

Table 1.4: Durability by fault class under the two synchronization levels. A classification of three fault classes against two configurations, not a quantity, so it is set as a table rather than drawn. The reference implementation ships `synchronous=NORMAL`: a process crash never loses a returned write, but an OS crash or power loss can roll back the most recent commits. `synchronous=FULL` buys the power-loss cell too, at the cost of an `fsync` on every commit. [internal]

synchronization level	process crash	OS crash	power loss
NORMAL (default)	durable	may lose last commits	may lose last commits
FULL	durable	durable	durable

The default trades the power-loss cell for one `fsync` per commit.

1.4.1 Why one label is a lie here

Gray and Reuter [94] draw the canonical line between *atomicity/consistency* (a transaction is all-or-nothing and never leaves the store corrupt) and *durability* (the “D” in ACID: a committed transaction survives subsequent failures). Under SQLite’s WAL with the *normal* synchronization level, the log is *not* `fsync`’d on every commit — it is synced at checkpoint [111]. Per SQLite’s own documentation, in WAL mode the normal level means “commits might lose durability following a power loss or hard reset,” though the database remains uncorrupted. So the honest statement splits in two.

Property 1.4.1 (Durability by fault class). Let a write return success under the default WAL configuration.

- **I1a (process-crash durability).** If the *daemon process* dies (a forced kill, a panic, an out-of-memory termination), the write survives: the data is in the operating system’s page cache, which outlives the process. **IMPLEMENTED.**
- **I1b (power-loss durability).** If the *operating system* crashes or power is lost *before the next checkpoint*, the most recent committed writes may roll back. *not guaranteed* under the normal synchronization level; it would require full synchronization or a checkpoint on the commit path.

Worked dramatization: Alice crashes mid-write. Alice is an agent editing a file. At t_0 she commits a claim on a port and a note recording her edit; the daemon returns success. At $t_0 + 5$ ms her host operating system kills the daemon process outright — a forced termination, no clean shutdown. *What survives?* Both writes. The committed pages live in the operating system’s page cache, which the daemon process does not own and does not take with it when it dies; when the supervisor restarts the daemon, SQLite finds a dirty log, replays it, and Alice’s

claim and note are intact. This is I1a, and it is the fault class the design actually targets: agents die constantly, and the record must outlive them. Now change one detail. At $t_0 + 5$ ms, instead of the daemon dying, *the power fails*. The committed pages were in the page cache but had not yet reached stable storage, because under the normal synchronization level the log is fsync'd only at the next checkpoint. On reboot, the database is uncorrupted — but Alice's last claim and note may simply be gone, rolled back as if never written. This is I1b, and it is *not guaranteed*. The two faults differ only in whether the page cache survived; conflating them is the single most common over-claim about WAL-backed storage.

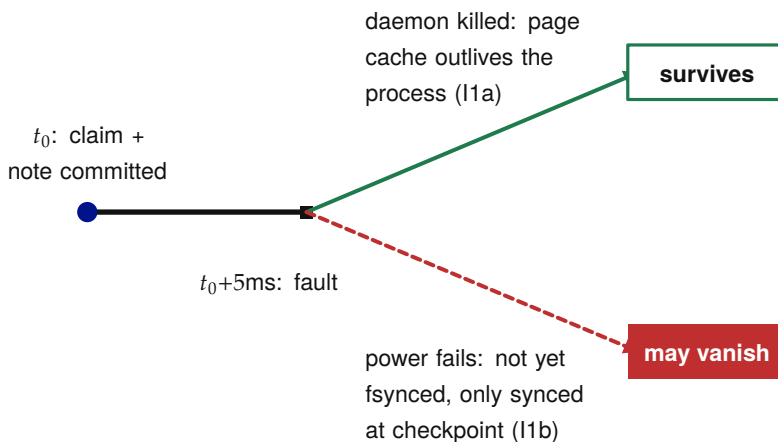


Figure 1.3: The same commit, the same 5 ms-later fault instant, two outcomes. A daemon kill and a power loss are equidistant from the commit at t_0 ; the only thing that differs is whether the committed pages had left the operating system's page cache. I1a (process-crash durability) is **IMPLEMENTED** because the cache outlives the process; I1b (power-loss durability) is *not guaranteed* because the normal synchronization level fsyncs only at the next checkpoint. [internal]

Figure 1.3 draws the two outcomes from one fork, so the equidistance is visible rather than merely asserted.

The seductive misconception is “the normal sync level is safe in WAL mode because WAL already guarantees crash safety.” This conflates crash-consistency (true: the database is never corrupt) with *durability* (false for power loss). The two are different ACID guarantees, and only the first is unconditional here. A systems paper cannot ship the unqualified claim. We split it.

PITFALL

1.4.2 Why the trade is defensible — and where it stops being so

Trading power-loss durability for throughput is a reasonable default for a local-first developer tool: the failure (lose the last few hundred milliseconds of claims after a power cut) is benign and self-correcting — agents re-claim, heartbeats resume. What is *not* defensible is letting the prose imply the stronger guarantee — and for the few paths that genuinely need I1b, the right fix is a per-write-path selector rather than a global change of synchronization level.

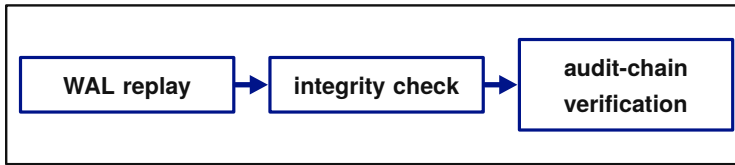
The design, **Selective Checkpointing** (SPECIFIED, OP-10), is small: a high-stakes, money-bearing write path — a settlement-ledger entry, say — appends `PRAGMA wal_checkpoint(FULL)`; to its commit path, forcing a synchronous flush to disk, while claim, marker, and note writes keep the fast default. Two things must be said plainly about it. First, *no write path in the reference implementation opts in today*: the only checkpoints it issues are the bounded auto-checkpoint, a passive checkpoint on the maintenance path, and the clean-shutdown truncate (Appendix 1.A). Second, even once a path opts in, the guarantee it buys is *per path*: Property 1.4.1 continues to govern every path that does not, so I1b remains *not guaranteed* for the kernel as a whole, and a table row or a figure that says otherwise is wrong. OP-10 stays *open* until both the selector and an interface that stops a *new* write path from silently defaulting into the fast lane exist. Table 1.5 states the selector’s shape, not a status report.

Table 1.5: Selective Checkpointing (OP-10, SPECIFIED): the write-path selector it specifies, not a status report. *No path opts in today* — this is the design’s shape, not the reference implementation’s current behavior. [internal]

write-path kind	sync level it would opt into
claim, marker, note (ordinary)	NORMAL (the fast default; unchanged)
settlement-ledger entry (money-bearing)	FULL on that commit (<code>PRAGMA wal_checkpoint(FULL)</code>)

1.4.3 A recovery story, not just a durability story

Durability is about what survives; recovery is about what happens next. On daemon restart with a dirty WAL, SQLite replays the log to a consistent state (I1a’s mechanism). But the kernel-level questions are larger: who validates the audit chain on boot, and what becomes of a half-applied cross-subsystem operation (§1.11) after a crash? The hook exists — the boot-admission gate that refuses to serve when a critical self-check fails (§1.9) — but boot-time WAL recovery plus integrity check plus chain verification should be a single named invariant, and today it is closer to **PARTIAL** than **IMPLEMENTED**.



not yet one named invariant: three
mechanisms, closer to BUILTWEAK than BUILT

Figure 1.4: Boot recovery today: WAL replay, an integrity check, and audit-chain verification run in that order, gated by the boot-admission check that refuses to serve on a failed critical self-check. The three should be one named invariant with one failure semantics; today they are three separate mechanisms, which is why the whole is closer to BUILTWEAK than BUILT. [internal]

Figure 1.4 draws the three mechanisms in the order they run, with the dashed panel naming what does not yet unify them.

Exercises 1.7–1.9, p. 69.

1.5 The resource subsystem: claims, locks, and fair exclusion

Network ports are the namesake and the original sin: two agents that each start a development server on the same port collide, and one silently fails. The resource subsystem generalizes that problem into a single exclusion primitive — over network ports, over named mutex locks, and over edit-surface claims on regions of source files — to which every higher layer’s notion of ownership reduces.

1.5.1 Claim granularity is richer than “claims”

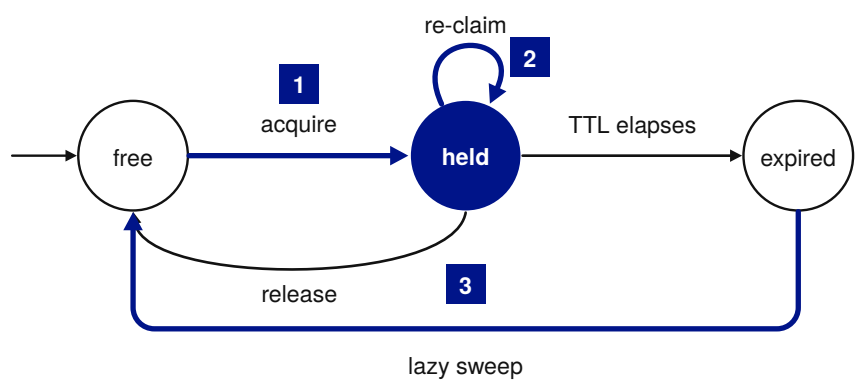
It is tempting to collapse all three of these into one word. The kernel instead distinguishes **whole-file**, **line-range**, and **symbol-path** claims, each keyed by the file path together with, respectively, nothing, a line interval, or a syntactic symbol identifier. This is the substrate for a “reserve regions, not whole files” coordination policy and for semantic-conflict prediction over a syntax-aware symbol index — but at the substrate layer it is simply exclusion at three granularities, listed in Table 1.6.

1.5.2 The claim lifecycle as a finite-state machine

Figure 1.5 states, as a state machine, the four properties on which the coordination layer builds typed ownership. The acquire path is small enough to state as an algorithm (Listing 1.1); we then give the central property as a theorem.

Table 1.6: The three claim granularities: what varies is the key, not the exclusion primitive underneath it. [internal]

granularity	key components	example
whole-file	file path alone	<code>claim(lib/webhooks.ts)</code>
line-range	file path + a line interval	<code>claim(lib/webhooks.ts, 40--72)</code>
symbol-path	file path + a syntactic symbol identifier	<code>claim(lib/webhooks.ts#handleEvent)</code>



there is no queued state: a losing acquire is simply denied, and nothing schedules it a turn.

what the badge adds to the transition name	
1	one atomic uniqueness decision; a losing caller is told the current holder, not handed a bare error
2	idempotent: the same holder renews its TTL, it does not create a second claim
3	expiry is removed lazily, on the next read that touches the row, not by a background sweep

Figure 1.5: The guarded claim lifecycle as a state machine. Acquire is a single insert against a uniqueness constraint; re-claim and release are idempotent; expiry moves a claim to *expired*, from which the next read lazily sweeps it back to *free*. There is no *queued* state and therefore no fairness or bounded-wait guarantee: a losing caller must retry, and nothing schedules it a turn. [internal]


```

1 function acquire(key, requester, ttl):
2   // single atomic statement; the uniqueness constraint is
   ↪ the mutex
3   try:
4     INSERT row (key, holder=requester, expires_at = now()+ttl
   ↪ )
5     return GRANTED(holder=requester)           // we won
6   catch UNIQUE_CONSTRAINT_VIOLATION:
7     existing = SELECT row WHERE row.key = key
8     if existing.holder == requester:
9       UPDATE row SET expires_at = now()+ttl // idempotent re
   ↪ -claim
10    return GRANTED(holder=requester)
11    if existing.expires_at < now():             // stale holder
12      DELETE row WHERE key = key AND expires_at < now()
13      return acquire(key, requester, ttl)       // one bounded
   ↪ retry
14    return DENIED(holder=existing.holder)       // loser learns
   ↪ the winner

```

Listing 1.1: Claim acquisition. The whole acquire is a single insert against a uniqueness constraint; the loser is handed the current holder, not an error to retry blindly.

The second half of the theorem below is the **fencing token**, and it should be named before it is proved, because it is the part of the design that was not invented here. Kleppmann’s rule is the whole of it — “a number that increases every time a client acquires the lock,” handed to the resource along with every write, so that the resource itself and not the lock service can refuse a holder who paused past its lease and woke up still believing it held it [135] — and Chubby shipped it first, as the *sequencer* a lock holder passes downstream so a storage server can tell a current handle from a stale predecessor’s [39]. This chapter calls the number an *epoch* because it is kept per resource and survives reallocation, but the protocol shape is the same one: acquire, receive a monotone number, present it with every effect, get refused when it is stale.

Theorem 1.5.1 (Atomic Mutual Exclusion and Fenced Leases). *Within a canonical conflict domain (exact normalized key, transactional interval overlap $[a, b] \cap [c, d] \neq \emptyset$, or path prefix hierarchy), at most one holder can hold an active lease at any logical instant. Furthermore, every granted lease carries a strictly monotonic fencing epoch $e \in \mathbb{N}$; downstream effect brokers reject any operation bearing an expired epoch $e' < e_{\text{current}}$.*

Proof sketch. Within an immediate SQLite transaction (BEGIN IMMEDIATE), the daemon evaluates the conflict predicate against all currently unexpired leases. For exact keys, this is enforced by primary-key

uniqueness; for intervals and hierarchies, by an indexed range check within the same transaction. Because all writes funnel through the single connection (Def. 1.3.1), predicate evaluation and insert are atomic. Upon grant or reallocation, the daemon increments the resource’s fencing epoch e . Downstream effect brokers validate that e matches the active lease at execution time, preventing stale or paused holders from producing side effects after lease expiration. $\square$

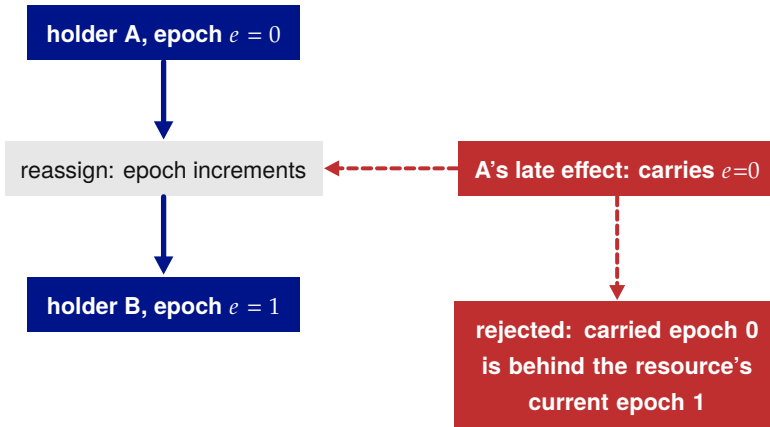


Figure 1.6: The epoch, not the lock, rejects the paused holder. A acquires a lease at epoch $e = 0$; reassignment increments the resource’s fencing epoch to $e = 1$ and grants it to B. A’s late effect still names a real lease it once held, but it carries the epoch $e = 0$ it was issued – the effect broker compares that carried epoch to the resource’s current epoch and rejects it, independent of any deadline. This is guard 1 of the work-unit machine (admit-effect: current epoch only), whose shortest violating trace without it is exactly this four-step sequence [internal, Thm. 1.5.1, `c0_workunit.py`].

Figure 1.6 isolates that guard as a four-step sequence: the epoch, not the lock, is what a paused holder’s late effect cannot get past.

What differs from Chubby is the setting, and it is worth being exact about how much smaller our problem is. Chubby’s sequencer and Raft’s leader term fence across machines that cannot otherwise agree on who is alive; the danger they answer is a paused JVM or a partitioned node still acting on a lease the rest of the world has moved past. This rail fences one process’s own asynchronous handlers against each other, so the race the epoch defends against is logical rather than physical, and the whole apparatus fits inside one SQLite transaction. That is a good reason not to need Raft (the related-work section makes the argument by way of Fischer, Lynch and Paterson); it is not a reason to let the token pass as new.

Worked dramatization: Alice and Bob race for one port. Alice and Bob are two agents, started a millisecond apart, that both want to bind

a development server to port 7421. Each issues `acquire(7421)` (Algorithm 1.1). Both requests reach the single daemon, which serializes them: one insert lands first. Say Alice’s does. Her insert succeeds and she is granted the port. Bob’s insert, arriving an instant later, hits the uniqueness constraint on key 7421 and is rejected *atomically* — there is no moment at which both rows exist. Crucially, Bob does not get a bare error. The catch path reads back the row and hands Bob the *identity of the holder*: “port 7421 is held by Alice.” Bob can now pick another port, wait, or message Alice, but he can never double-bind. The serialization that makes this clean is not a hand-rolled critical section; it is the operating system’s file lock funnelling both inserts through one writer, and the database’s primary-key uniqueness deciding the winner. One writer, one constraint, one holder.

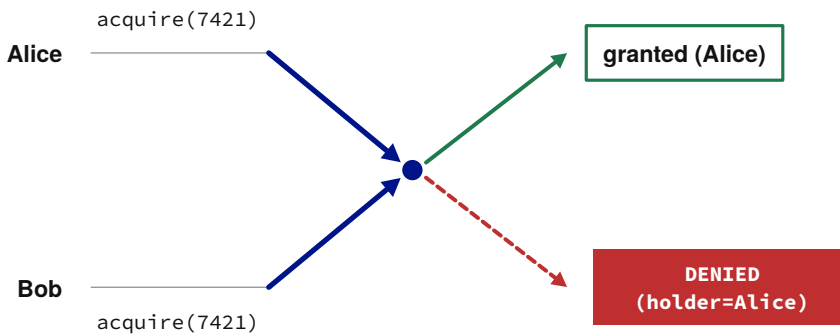


Figure 1.7: Two racing `acquire(7421)` calls converge on one insert-commit point in the single daemon; the uniqueness constraint decides the winner. Alice’s insert lands first and she is granted the port. Bob’s insert is rejected atomically — there is no moment at which both rows exist — and the catch path reads the row back, handing Bob a named `DENIED(holder=Alice)` rather than a bare error, so he can pick another port, wait, or message Alice. [internal, alg:acquire]

Figure 1.7 draws the two lanes converging on that one commit point and the two outcomes.

Theorem 1.5.1 is sound for the *fresh-contention* case above. A subtler hazard hides in the *stale-holder* branch: reclaiming an expired lock runs a delete (of the expired row) and *then* an insert as two separate statements rather than one transaction. On the single daemon connection this is serialized by the connection itself and is safe. But if a second writer connection ever existed — the very scenario the single-writer discipline exists to prevent — a deferred transaction would open a window for a transient “database busy” error mid-sequence rather than clean serialization; the fix is to begin the transaction in immediate mode. So Theorem 1.5.1 holds because all writes route through the one daemon connection (Def. 1.3.1), *not* because the expire-then-acquire sequence is itself atomic. State which story you mean; do not tell both.

PITFALL

What the discipline looks like from two terminals: one agent holds a lock on a file, a second asks for the same name and is told who holds it and for how long, and only after the first releases does the second acquire. Nothing here is advisory — the daemon is the single writer for the locks row, so the second request cannot race the first.

two agents, one lock, one holder at a time.

AT THE TERMINAL

```
$ export PORT_DADDY_AGENT_ID=alice
$ pd lock lib.webhooks.ts --ttl 600000
SUCCESS: Acquired lock: lib.webhooks.ts
  TTL: 600s

$ export PORT_DADDY_AGENT_ID=bob
$ pd lock lib.webhooks.ts --wait --timeout 2000
Lock 'lib.webhooks.ts' is held by <actor-id>
  Held since: <timestamp>
  Expires in: <n>s

$ pd locks

NAME                                OWNER                                EXPIRES
-----
lib.webhooks.ts                     <actor-id> <timestamp>

Total: 1 lock(s)

$ export PORT_DADDY_AGENT_ID=alice
$ pd unlock lib.webhooks.ts
SUCCESS: Released lock: lib.webhooks.ts

$ export PORT_DADDY_AGENT_ID=bob
$ pd lock lib.webhooks.ts
SUCCESS: Acquired lock: lib.webhooks.ts
  TTL: 300s
```

1.5.3 Fairness without a scheduler: the Stigmergic Ticket-Lock (OP-1)

The conspicuous gap in the claim lifecycle (Figure 1.5) is the absence of a QUEUED state. Locks have a time-to-live, but acquisition is racy first-come, so a fast agent churning a hot file can starve a slow agent indefinitely.

The design that would close OP-1 without dragging a background scheduler into the kernel is a **Stigmergic Ticket-Lock**. It is SPECIFIED — specified here in enough detail to build and to attack, but not realized: no `claim_tickets` table exists in the reference implementation, and

the lifecycle above is still the one the kernel runs. It would introduce a `claim_tickets` table:

```
1 CREATE TABLE claim_tickets (
2   resource_id TEXT NOT NULL,
3   agent_id TEXT NOT NULL,
4   seq_num INTEGER PRIMARY KEY AUTOINCREMENT
5 );
```

When Agent *B*'s claim acquisition failed against the primary-key UNIQUE constraint, the error handler would insert an entry into `claim_tickets`. When Agent *A* then called `release('auth.ts')`, the daemon would execute a single atomic transaction:

1. Delete Agent *A*'s active claim row: `DELETE FROM claims WHERE resource = :r;`

2. Read the lowest ticket:

```
1 SELECT agent_id, seq_num FROM claim_tickets
2 WHERE resource_id = :r
3 ORDER BY seq_num ASC LIMIT 1;
```

3. Atomically re-grant the claim: `INSERT INTO claims (resource, holder, ttl) VALUES (:r, :next_agent, :ttl);`

4. Remove the claimed ticket: `DELETE FROM claim_tickets WHERE seq_num = :next_seq;`

Because the state transition would be driven entirely *reactively* by the releasing agent inside one transaction, FIFO ordering among waiters follows from the ticket table's own monotonic sequence, with no background timer or polling thread in the kernel — which is the property that makes the design worth stating.

Its honest edge is the path it does *not* cover. A holder that never calls `release` — it crashed, or its lease simply expired — returns the resource through the lazy TTL sweep (I4), and the sweep consults no ticket queue. So bounded wait would hold for cooperative release and fail on exactly the failure path the kernel was built to expect. OP-1 stays *open* until the sweep and the queue are the same code path, and until the ticket table exists at all.

Figure 1.8 draws that split as a delta on the shipped machine: one exit closed, one still open.

*Exercises 1.10–1.12,
p. 70.*

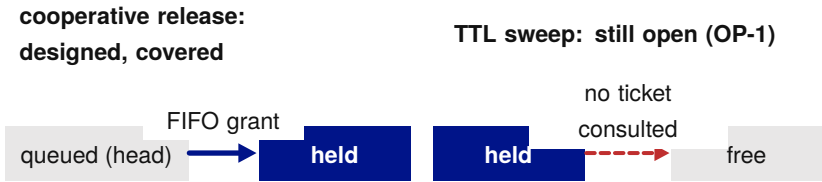


Figure 1.8: The Stigmergic Ticket-Lock’s coverage, drawn as a delta on the shipped claim lifecycle (Figure 1.5), not a second machine. Left: a cooperative release grants FIFO to the head of the ticket queue in one atomic transaction, closing the starvation gap. Right: a holder that never releases – crashed, or simply expired – still returns through the lazy TTL sweep, which consults no ticket queue. Bounded wait would hold on the left exit and fail on the right one, so OP-1 stays *open* until the sweep and the queue are the same code path. [internal, sec:op1]

1.6 The communication subsystem: the bus, stigmergy, and two delegation chains

Table 1.7: The communication subsystem: one durable carrier and two provenance traces that must never be merged. The envelope fields are what the bus stores; the two traces answer two different questions and are checked by two different mechanisms.

Piece	What it is	Status
the bus envelope	version tag, time-to-live, message kind, typed act with its conversation id, actor, reason and event reference; ordered by a durable history cursor	IMPLEMENTED
authorization chain	the cryptographic object: who authorized whom, hop-bound, attenuation-monotonic, anti-replay	IMPLEMENTED (ProVerif)
coordination lineage	the coordination object: what work is already in flight, so loops and upward blocks can be detected	SPECIFIED

The communication subsystem (Table 1.7) is where the substrate stops being a lockbox and becomes a medium for conversation. The carrier is **IMPLEMENTED**; the *semantics* that ride on top of it (typed speech acts, protocol state machines) belong to the coordination layer. Three pieces matter here.

1.6.1 The bus: a durable carrier envelope

The bus is a durable, ordered, *at-least-once* publish/subscribe channel over a messages table, with a versioned envelope (a version tag, a message kind, a body, and an in-reply-to pointer), a history cursor for replay, and a per-message time-to-live. The coordination layer’s typed speech act — carrying a performative, a conversation identifier, a coordination lineage, and a reply deadline — is layered *inside* this envelope. The kernel ships the carrier; the coordination layer ships what it carries.

At-least-once delivery means a *healthy* bus routinely redelivers and reorders. This collides with a loud-fail honesty discipline: if “every anomaly is loud,” benign redelivery drowns the operator in false alarms. The resolution is that benign transport non-determinism must be *deduplicated silently*, and only genuine protocol-state-machine violations surface loudly. That requires an idempotency key in the envelope — currently absent — which is the cheapest high-leverage fix at the coordination layer. The kernel provides the bus; making every speech-act effect idempotent is the coordination layer’s debt, flagged here so the reader does not mistake silent deduplication for dishonesty.

KEY IDEA

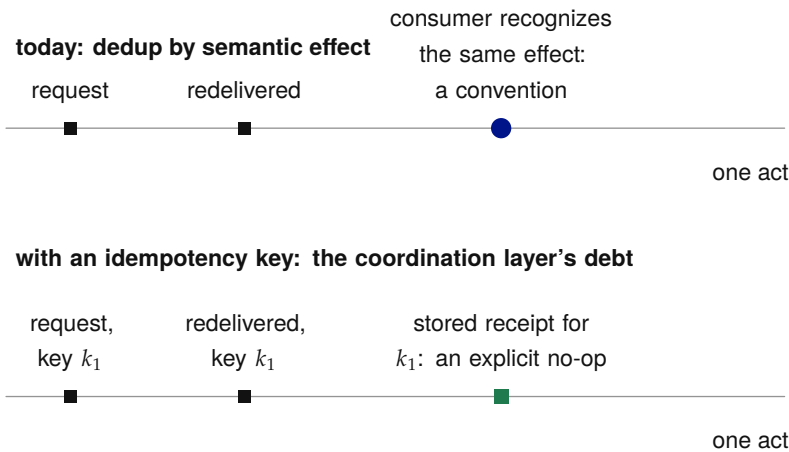


Figure 1.9: The idempotency gap, drawn as one redelivery on two lanes. Today (top), the consumer deduplicates by recognizing it already applied the same semantic effect – silent, but a convention. With an envelope idempotency key (bottom, not yet shipped – the coordination layer’s debt), the consumer instead checks a stored receipt for that key: the same operator-visible outcome, but retry safety is now explicit rather than conventional. Exactly-once is still not guaranteed for an unbrokered external effect. [internal, Exercise 1.14]

Figure 1.9 draws the one redelivery on two lanes: today’s convention and the still-absent key’s explicit receipt.

1.6.2 Stigmergic markers: decay as a provided guarantee

The kernel also carries *stigmergic markers* — numeric coordination signals deposited in shared state, after the manner of Grassé’s stigmergy in social insects [93] and the generative-communication tuple spaces of Gelernter’s Linda [88], and used in ant-colony optimization [65]. Each marker carries per-kind exponential decay $w \cdot r^{\Delta t}$, computed both on a background evaporation tick *and* at read time (so a reader sees the correctly decayed value without waiting for the tick), and pruned below a small threshold. The decay is the structural defense against “black-board rot”: stale coordination signals evaporate on their own. The calibration of the decay rate is genuinely open (OP-8): too slow and signals rot, too fast and they evaporate before they coordinate.

Numbers by hand — The decay arithmetic a calibration will use. Take $w_0 = 1$ and $r = 0.9$ per tick, with a prune threshold of 0.05. Then $w(5) = 0.9^5 = 0.590$, $w(10) = 0.349$, $w(20) = 0.122$, and the marker is pruned at the first tick where $0.9^t < 0.05$: $0.9^{28} = 0.052$ survives and $0.9^{29} = 0.047$ does not, so the marker lives 29 ticks [internal]. Moving the rate half the distance to one ($r = 0.95$) more than doubles that life, to 59 ticks, because $\ln 0.05 / \ln 0.95 = 58.4$. The calibration question of OP-8 is which of these lifetimes matches the time a signal stays true; the arithmetic is the same at every answer.

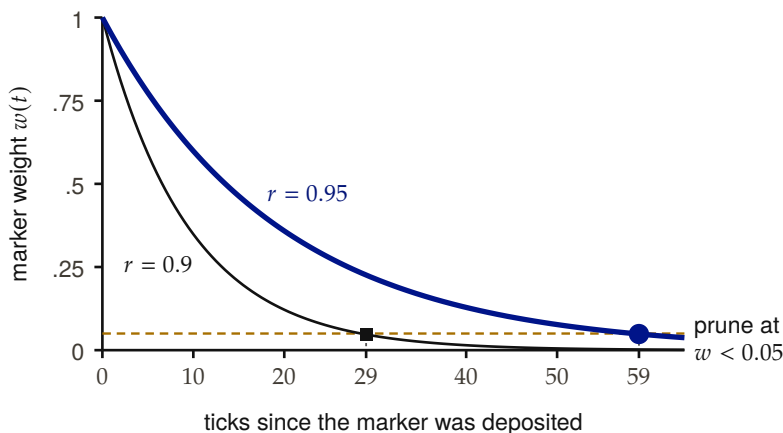


Figure 1.10: Marker weight $w(t) = r^t$ for two decay rates against the prune threshold of 0.05 (dashed). At $r = 0.9$ the marker survives 28 ticks and is pruned at tick 29; at $r = 0.95$ it lives until tick 59, since $\ln 0.05 / \ln 0.95 = 58.4$. Both curves are nearly flat where they meet the threshold, which is why moving the rate half the distance to one more than doubles the lifetime: that is the whole content of the calibration question. [internal; analytic]

1.6.3 Two delegation chains, one dangerous name

A dangerous naming collision lives in this subsystem. The phrase “delegation chain” names *two different objects*, which we keep rigorously distinct:

- the **authorization chain** — the *cryptographic* object (**IMPLEMENTED**, mechanically verified in a protocol-verification tool): a hop-bound, attenuation-monotonic, anti-replay and anti-splice chain of digital signatures that proves *who authorized whom*. This is what cross-machine capability transfer in the economy layer rides on.
- the **coordination lineage** — the *coordination* object (**SPECIFIED**): the task-lineage over which loop detection and upward-block run, preventing two agents from delegating a task back and forth forever.

These are distinct objects, and the relationship is that the coordination lineage is carried *inside* the authorization chain — loop detection runs over a lineage whose authenticity the cryptographic chain guarantees. We never again write bare “delegation chain.”

The “lineage inside chain” relationship is a slogan until the task-shape field is in the *signed* payload at each hop — and that collides with a rule against keyword-based text classification, because loop detection needs a *semantic* task-shape equivalence (a learned embedding or a structural hash), which is not deterministically signable. This tension is a real open problem; we flag it rather than assert it solved.

PITFALL

Figure 1.11 draws the containment: the coordination lineage rides inside the authorization chain, never beside it.

1.6.4 A second transport

The kernel ships *two* transports, not one: a request/response protocol over a Unix domain socket *and* a binary framed transport over a Unix domain socket, the latter with peer-credential authentication and back-pressure/draining for high-frequency, tight-loop coordination. We note in §1.12 that the peer-credential check is a software handshake — the connecting process *states* an agent identity and the daemon checks it against the registration table, backed by owner-only (0600) permissions on the socket file — and *not* the kernel-enforced socket-level credential check (SO_PEERCREC) it resembles and is named after. That is a real trust-boundary caveat, it holds for both transports, and it is unchanged by the hardening design of §1.12.4.

Exercises 1.13–1.15,
p. 70.

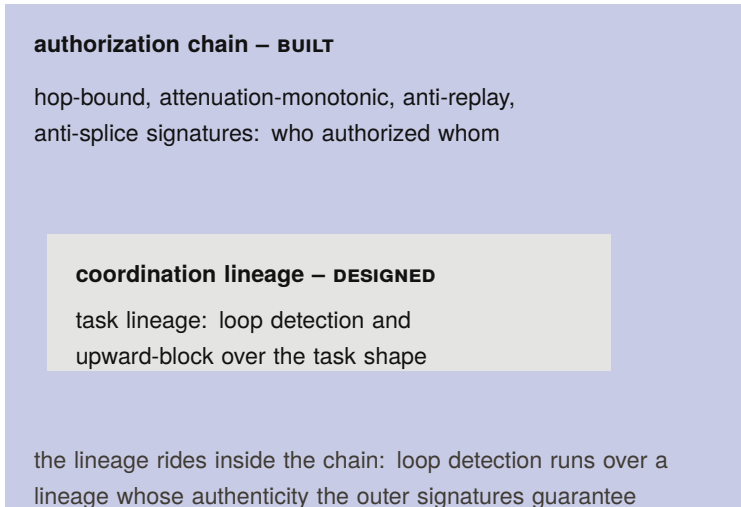


Figure 1.11: Two objects, never merged: the cryptographic authorization chain (BUILT, mechanically verified) and the coordination lineage (DESIGNED) it carries. The nesting is the claim – loop detection runs over a task lineage whose authenticity the outer chain of signatures guarantees, so the chapter never again writes bare “delegation chain.” Making the inner task-shape field itself signable at each hop, given that loop detection needs a semantic equivalence rather than a keyword match, is an open tension the chapter flags rather than claims solved. [internal]

1.7 The obligation & enforcement subsystem: the sovereign’s two levers

This subsystem is the deontic core, and it is genuinely two distinct mechanisms — in the vocabulary of deontic logic for computer systems (Jones and Sergot [124]): *prohibition* (a policy monitor) and *obligation* (commitments). Getting the distinction right — and being honest about how strong each lever actually is — is the security core of the paper.

Table 1.8 lays out the three modalities and the mechanism behind each.

1.7.1 The deontic split

Definition 1.7.1 (The kernel’s deontic split). The kernel realizes three deontic modalities with three mechanisms: *prohibition* is regimented (rejected pre-commit) or enforced (detected post-commit); *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability. The coordination layer’s deontic binding is a direct lift of this split.

We keep **capability** (*ability* — what the agent’s sandbox makes possible) distinct from **permission** (*normative status* — what the deontic layer

Table 1.8: The deontic split: three modalities, three mechanisms. Capability (what the sandbox makes possible) is kept apart from permission (what the deontic layer allows) so that “able but forbidden” stays expressible.

Modality	Trigger	Mechanism	Holds / fails when	Example
prohibition	an act the policy forbids is attempted	regimented before commit by a gate when the event is controllable; otherwise detected after commit and compensated	holds when the effect never commits; fails as detect-and-compensate when the gate could not see the trigger	a tool’s emergency override flag; the agent is <i>able</i> to call it and <i>forbidden</i> to
obligation	a commitment is recorded	an oracle-bound commitment: closure only through a receipt the daemon can re-check, before a daemon-set deadline	holds when a typed receipt closes it in time; fails by staying open and escalating, never by free-text assertion	“release the claim on lib/webhooks.ts within thirty minutes”
permission	a grant is issued	a recorded capability row, attenuable, never widened downstream	holds while the grant is live; an act without a grant is a prohibition case	“may read the audit log until the session ends”

allows), so that “able but forbidden” stays expressible. The canonical example is a dangerous override flag that a tool exposes for emergencies: it *exists* (an agent is *capable* of invoking it) but *must not be used* (it is *forbidden*). Collapsing capability into permission makes that state inexpressible — which is exactly the bug a discipline against silently by-passing guardrails cannot afford.

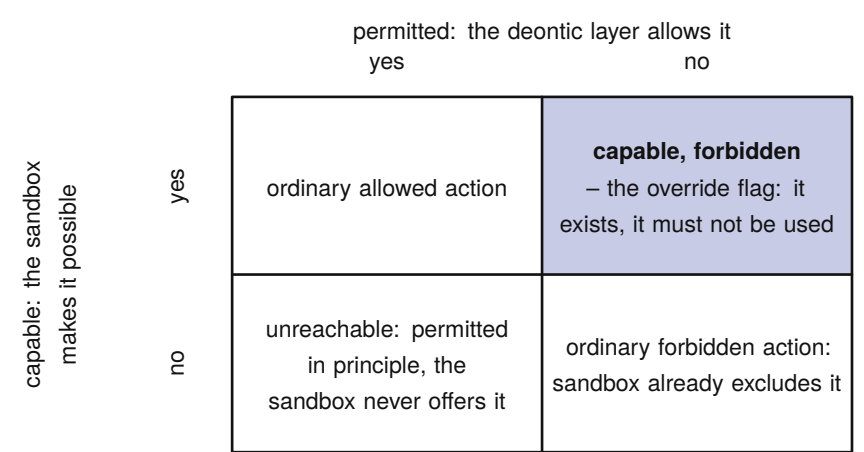


Figure 1.12: Capability and permission are independent axes, not one word. The shaded cell – capable but forbidden – is the one collapsing them makes inexpressible: a dangerous override flag a tool exposes for emergencies exists (the agent is capable of invoking it) but must not be used (it is forbidden). The other three cells are ordinary; only this one is why the chapter keeps the two distinct. [internal]

Figure 1.12 puts both axes on one grid; the override flag names the one cell that must stay expressible.

1.7.2 The policy monitor is a detector, not a regimenter

Figure 1.13 draws the correction this section makes precise. Here is the correction that the rest of this stack must adopt. The most seductive claim one can make about a coordination policy monitor — that it “makes forbidden states physically unreachable” — is false as stated. The monitor is a *subscriber to an append-only event stream of operations that have already committed*. By the time it observes a violation, the offending write is already durable in the log. Even in its strictest mode, its strongest response is a *compensating undo*, not a prevention. This is not an implementation accident; it is a theorem.

Theorem 1.7.1 (A prevention bound for post-commit monitoring). *A monitor invoked only after a transition commits cannot prevent a safety violation whose bad prefix ends at that transition. It can detect the prefix, halt later actions, and enforce a separate recovery or bounded-response property.*

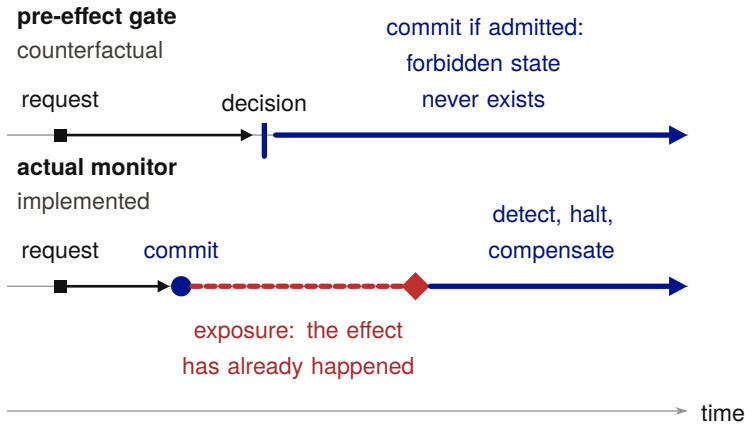


Figure 1.13: The reference-monitor test as a causal counterfactual, both traces drawn on the same time axis. A true mediator (top) decides before the commit point, so a forbidden state never exists. The present policy monitor (bottom) instead subscribes after commit: the breach segment is the exposure interval in which the effect has already happened, before the monitor can observe, halt subsequent work, and compensate – the past effect stays in the history. Only uniqueness constraints and fail-closed boot gates belong to the upper trace; subscriber rules belong to the lower one. [internal]

Therefore a post-commit subscription must not be credited with making the triggering state unreachable.

Proof sketch. Schneider’s execution-monitor model [228] recognizes safety properties through finite bad prefixes and enforces them by suppressing an action before that prefix enters the target execution. A subscriber invoked after commit cannot suppress the committing action. Compensation may establish a different eventual or bounded-recovery property, but it does not retroactively remove the bad prefix. $\square$

Worked dramatization: Bob attempts a forbidden action. Bob is an agent under a policy that forbids editing files outside his assigned task scope. He nonetheless issues a write that records an out-of-scope edit. *What happens?* The write commits. The single writer serializes it, the uniqueness constraint has no objection (it is a well-formed row), and the operation lands durably in the log. Only *then* does the policy monitor, subscribed to that log, observe the out-of-scope edit and raise a violation. Its response is compensating: it can flag Bob’s session, revoke his claims, alert the operator, and trigger a salvage of the affected surface — but it cannot pretend the write never happened, because it did. Contrast this with Bob trying to claim a port Alice already holds (the previous dramatization): *that* attempt never commits, because the uniqueness constraint rejects it before the commit line. The two cases are the whole deontic story in miniature: a uniqueness constraint and

a boot-time admission gate *regiment* (the bad state is unreachable); the policy monitor *enforces* (the bad state is reached, then detected and compensated within a bounded window).

The only genuine pre-commit regimentation is credited to the right mechanism: the uniqueness constraint (no two holders of one resource key; no duplicate process-identity squat) and the fail-closed boot-admission gate (no serving from a test database; refuse to serve when a critical self-check fails). Those reject *before* the commit line and are *truly* unreachable. The policy monitor notices everything else *afterward* and compensates. So: “physically unreachable” for the regimented set; “detected within a bounded window, then halted or compensated” for the rest. Crediting double-claim prevention to the monitor *double-counts* — the uniqueness constraint already did it; the monitor merely logs it.

KEY IDEA

Theorem 1.7.2 (Synchronous State Decidability — closing OP-2). *A safety invariant ϕ is **regimentable** (strictly preventable pre-commit, making violation states physically unreachable) if and only if $\phi(S, \Delta)$ is a pure, deterministic boolean predicate decidable solely over the daemon’s local, synchronous, committed database state S and the incoming transaction payload Δ .*

*Conversely, any invariant requiring external I/O, asynchronous wall-clock verification, distributed consensus, or non-deterministic grading oracles is strictly **enforceable** (post-commit, detect-and-compensate).*

Proof sketch. In SQLite WAL mode under single-writer serialization, transaction commits occur synchronously on the database connection thread. A predicate $\phi(S, \Delta)$ computable in bounded CPU steps without yielding the event loop can be embedded directly into SQLite constraints (UNIQUE, CHECK) or transaction guard clauses; any transition yielding $\neg\phi$ rolls back atomically before state mutation. If ϕ depends on external network I/O or asynchronous delays, holding the exclusive transaction lock during evaluation starves the single-writer queue; hence evaluation must be deferred post-commit, converting the mechanism from a synchronous gate into an asynchronous event subscriber governed by Theorem 1.7.1. $\square$

1.7.3 The general boundary: regimentation is controllability

Express lane: a governance rule can be enforced by prevention exactly when no event the runtime cannot refuse can carry a legal history out of the rule; that is Ramadge–Wonham controllability with the model’s own steps as the



uncontrollable alphabet. The formal statement is the box below; the machine-checked policy table is Table 1.9.¹

The scene. The operations lead reads the governance document aloud. *No force-push to main. No network egress from sandboxed jobs. No writes outside the granted scope. And then: no confident falsehoods in status reports.* She asks the question every practitioner senses the answer to: which of these does the runtime *guarantee*, and which does it merely *watch for*? The first three feel airtight. The daemon holds the remote credential, so a push that never happens needs no forgiveness. The fourth is not airtight and never will be: the falsehood is composed inside the model's forward pass, and by the time any mediator sees tokens the falsehood already exists. Theorem 1.7.2 settled this for one door, the database commit. This section settles it for every door a runtime can stand at, and the instinct turns out to be an instance of a theorem from 1987.

The idea in one breath. *Split the agent's events into what the runtime can refuse and what it cannot; a safety rule is preventable if and only if no unrefusable event can carry a legal history out of the rule.*

Intuition: the bouncer at the one door. A club has one door and a bouncer. The bouncer can refuse *entry*: entry is mediated by the door he stands at. He cannot refuse a patron's *thought*: nothing the club owns mediates it. A door policy ("no entry after two") is enforceable by prevention; a thought policy ("no ill intent inside") is enforceable only by watching and, if it comes to it, walking someone out. The daemon's effect boundary is the door. Writes, egress, tool execution, pushes, spawns, and API calls cross it, so each can be refused; token emission, in-context reads, plan formation, and hidden activations complete inside the model, so none can. The analogy maps relations rather than surface features, which is why it earns a prediction: a *compound* rule that mentions thought but gates only the door ("no entry for anyone who arrived after the fight started") should still be preventable, because the bouncer need only *observe* the trigger and *refuse* the door. Section 1.7.3.1 confirms exactly this. The predictable misread, preempted: uncontrollable does not mean invisible. Tokens are eminently loggable after emission. Uncontrollable means *unrefusable before the fact*, not unobservable after it; when an event is also unobservable a second, stricter theory applies (§1.7.3.2).

¹A standalone, submission-form version of this section is *Regimented or Enforced: The Controllability Boundary for Agent Governance* (research paper 2), whose policy table and verdicts regenerate from `skills/harbor-results/scripts/b3_controllability.py`.

The vocabulary, at the point of use. Everything below is a statement about strings and sets of strings. An *alphabet* Σ is the finite set of event types the system can emit: `fs_write`, `net_egress`, `model_emit_token`, and the rest. A *string* over Σ is one possible history of an execution. A set of strings K is *prefix-closed* when every prefix of a member is a member: if the first ten events of a run are allowed, so are the first nine. Prefix-closed languages are exactly Lamport’s safety properties: a violation has a finite witness, and once you have violated one you can never un-violate it. The *plant* L is the set of histories that are physically possible before any policy is imposed; we take $L = \Sigma^*$, since nothing about the hardware forbids an interleaving. A *policy* is a prefix-closed $K \subseteq L$. A *supervisor* is a function from the history so far to a set of events to disable next, and the whole theorem turns on its type: it may disable members of Σ_c , the *controllable* events that pass through a mediation boundary before taking effect, and nothing else. The *closed loop* is the set of histories that occur once the supervisor is attached. A supervisor *regiments* K when the closed loop equals K exactly: nothing forbidden happens and nothing permitted is lost. That second half is half the value. A supervisor that achieves safety by disabling everything is not a solution.

THEOREM 1.7.3 Regimentation is controllability. Let $L \subseteq \Sigma^*$ be a prefix-closed plant over $\Sigma = \Sigma_c \uplus \Sigma_u$ and let $K \subseteq L$ be a nonempty prefix-closed safety policy. Assume *full observation*: the supervisor sees every event it does not disable, so the observable subalphabet Σ_o is all of Σ . There exists a regimentation mechanism for K , a supervisor disabling only controllable events whose closed loop is exactly $\bar{K}$, if and only if K is *controllable* with respect to L and Σ_u :

Proved in §2.3.

$$\bar{K}\Sigma_u \cap \bar{L} \subseteq \bar{K},$$

that is, no uncontrollable event, physically enabled after a legal history, exits the policy. When the condition fails, no runtime confined to disabling Σ_c can prevent the violation; the best achievable enforcement is detect-and-compensate, and the largest preventable weakening of the policy is the supremal controllable sublanguage $\sup C(K)$, which exists and is computable for regular K and L [verified framework: Ramadge and Wonham [207]]. Relaxing full observation does not weaken this conclusion, it adds a second and independent condition: see Theorem 1.7.4.

Proof idea. If the policy is controllable, build the laziest possible supervisor: after any legal history, disable exactly the controllable events that would step outside the policy. It never has to disable an uncontrollable event, because controllability says none of those can step outside.

If the policy is not controllable, there is a legal history after which the plant can fire an uncontrollable event that leaves the policy; any mechanism that keeps every legal history reachable must let that history happen, and then cannot stop the event.

Proof. The proof is structured; each numbered step can be checked on its own.

⟨1⟩1.

ASSUME K controllable. DEFINE the supervisor $f(s) = \{a \in \Sigma_c : sa \in \bar{L} \setminus \bar{K}\}$ for $s \in \bar{K}$. PROVE the closed loop equals $\bar{K}$.

⟨2⟩1.

The closed loop is contained in $\bar{K}$, by induction on history length. The empty history lies in $\bar{K}$ because K is nonempty and prefix-closed. If the current history s lies in $\bar{K}$ and the loop executes a , then $sa \in \bar{L}$ and $a \notin f(s)$. If $a \in \Sigma_c$, then $a \notin f(s)$ means $sa \in \bar{K}$ by the definition of f . If $a \in \Sigma_u$, then $sa \in \bar{K}\Sigma_u \cap \bar{L} \subseteq \bar{K}$ by controllability.

⟨2⟩2.

Every $sa \in \bar{K}$ is reachable: an uncontrollable a is never disabled, and a controllable a with $sa \in \bar{K}$ is not in $f(s)$.

⟨2⟩3.

Q.E.D. Containment both ways is equality, so f regiments K .

⟨1⟩2.

ASSUME K not controllable: there are $s \in \bar{K}$ and $\sigma \in \Sigma_u$ with $s\sigma \in \bar{L}$ and $s\sigma \notin \bar{K}$. PROVE no regimentation mechanism exists.

⟨2⟩1.

Let M be any regimentation mechanism for K . Because M realizes exactly $\bar{K}$, the legal history s is reachable under M .

⟨2⟩2.

At s the plant enables σ , and M cannot disable it: $\sigma \notin \Sigma_c$, so disabling it is outside M 's type. Physically, the event completes before the mediation boundary sees it.

⟨2⟩3.

Q.E.D. The closed loop under M reaches $s\sigma \notin \bar{K}$, contradicting the assumption that M 's closed loop is contained in $\bar{K}$. Any enforcement of K must therefore tolerate the reachable violation $s\sigma$ and respond to it afterwards, which is detect-and-compensate by definition.

⟨1⟩3.

The existence, uniqueness, and computability of $\sup C(K)$ are im-

ported from Ramadge and Wonham [207]: controllable sublanguages of K are closed under arbitrary union, so a supremal element exists, and for regular languages a fixpoint iteration on the product automaton computes it.

⟨1⟩4.

Q.E.D.

□

Two remarks on what the proof used. The forward direction is constructive and *maximally permissive*: the supervisor disables nothing except the controllable exits, so regimentation costs no legal behavior. The governance rule and the agent's freedom are not in tension when the rule is controllable at all. The reverse direction used nothing about the mechanism except its type. No cleverness inside the runtime, no better prompt, faster monitor, or smarter policy engine, escapes the theorem, because the obstruction is that σ finishes before any decision point the runtime owns. Theorem 1.7.2 is the special case $\Sigma_c = \{\text{synchronous database commits}\}$: when the only mediated channel is the commit, the regimentable policies are exactly those expressible over committed state. The general statement strictly dominates it. The slogan cannot say why a force-push is preventable by a runtime that also owns the credential channel, nor why a confident falsehood stays unpreventable however much state is committed.

Numbers by hand — *The policy table, checked by hand.* The controllability test is decidable by a product-automaton search: walk the synchronous product of plant and policy automaton and flag any reachable state where the plant enables an uncontrollable event that the policy disables. Run over the reference alphabet, the checker returns Table 1.9 [internal, b3_controllability.py; deterministic, no seed]. The eight single-event rows also follow by hand, which is the right way to first believe the table. Take “forbid git_push.” The policy automaton has one state with every event except git_push self-looping; git_push lies in Σ_c , so the only event the policy ever disables is controllable, the premise of a controllability violation is unsatisfiable, and K is controllable: regimentable. Now take “forbid internal_plan.” The same one-state automaton disables internal_plan, which lies in Σ_u ; the universal plant enables it at the start state, so the empty history and $\sigma = \text{internal_plan}$ witness $\bar{K}\Sigma_u \cap \bar{L} \not\subseteq \bar{K}$: detect-only, for any runtime with this alphabet split. *Now you try*: classify “forbid api_call after spawn_child” and “forbid internal_plan after fs_write” (Exercise 1.19).

Table 1.9: Nine governance rules classified by the product-automaton checker. The verdicts are the checker’s output; the eight single-event rows also follow by hand from Theorem 1.7.3 [internal, b3_controllability.py].

Policy	Event class	Regimentable?
forbid net_egress	controllable	yes (regiment)
forbid git_push (incl. force-push)	controllable	yes (regiment)
forbid fs_write (out-of-scope write)	controllable	yes (regiment)
forbid exec_tool	controllable	yes (regiment)
forbid spawn_child	controllable	yes (regiment)
forbid model_emit_token	uncontrollable	no (detect only)
forbid in_context_read	uncontrollable	no (detect only)
forbid internal_plan (“confident lie”)	uncontrollable	no (detect only)
compound: no net_egress after a secret in_context_read	mixed	yes (regiment)

1.7.3.1 The compound case: gate the channel, never the token

The last row of Table 1.9 is the one an entire product line rests on. The policy *no network egress after an in-context read of a secret* mentions an uncontrollable event, so a naive reading of the single-event rows predicts detect-only. The checker says otherwise, and the mechanism is visible in the policy automaton (Figure 1.14): two states, `ok` and `tainted`. The uncontrollable read is enabled in both. The policy never tries to forbid it; it lets the read happen and records the taint transition. What the tainted state disables is `net_egress` alone, and `egress` is controllable. Every uncontrollable event the plant enables is enabled by the policy in every reachable state, so the criterion holds and the constructive direction of the theorem hands us the supervisor: track taint, gate the channel. This automaton is Schneider’s own Figure 1, a two-state security automaton for “no Send after a FileRead” [228], with his states relabelled. What the alphabet split adds is not the construction but the grading: under complete mediation the figure is one more enforceable safety policy; once $\Sigma_u \neq \emptyset$ the same two states become the canonical witness that a policy may reference an unrefusable event freely so long as it never tries to refuse it.

Three consequences, in increasing order of consequence.

1. **The design rule.** A compound trigger-to-effect policy is regimentable if and only if its *effect* events are controllable; the trigger may be as uncontrollable as it likes, provided the policy observes rather than forbids it. Uncontrollable events in the policy’s condition are free; uncontrollable events in its prohibition are fatal.
2. **The sealed room.** Put an agent alone in a room with a customer’s secret and promise that the secret does not leave. It is impossible to prevent the model from *reading* the secret (the read is uncontrollable, and useless to forbid, since reading is the job) and impossible

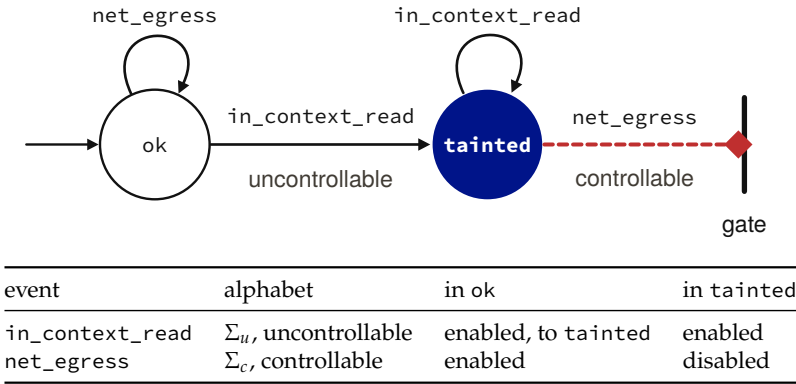


Figure 1.14: The taint automaton behind the sealed room: Schneider’s two-state security automaton for “no egress after an in-context read of a secret”, with his states relabelled. The read is uncontrollable, and the policy never tries to refuse it: it is enabled in both states and only records the taint. The one event the tainted state disables is `net_egress`, which the daemon mediates, so every uncontrollable event stays enabled and the policy is regimentable. Gate the channel, never the token [internal, `b3_controllability.py`].

to prevent it from *incorporating* the secret into tokens and plans (uncontrollable again). What is possible, exactly and only, is to gate every controllable egress channel out of the room, with taint recorded from the moment of exposure. Gate the channel, never the token. The companion noninterference result for the sealed room proves from the other direction that the gate suffices; controllability says the gate is the only place a guarantee *can* live. This is why “all spend is recorded” in the economy chapters is precisely complete mediation of Σ_c .

3. **The two dials.** Holding the plant fixed, widening Σ_c (owning more channels) is the only way to grow the regimentable set; no policy-language cleverness does. The plant is the other dial: $\bar{L}$ sits on the left of the containment, so shrinking it weakens the antecedent and strictly more policies pass. Both detect-only verdicts above turn on “the universal plant enables `internal_plan`,” and are conditional on the modelling choice $L = \Sigma^*$. An event that never becomes physically possible needs no supervisor, which is why data minimization and capability removal are governance moves and not merely hygiene: a secret never loaded into the room regiments the compound policy with no gate at all.

Numbers by hand — Synthesis by hand: the sandboxed review job. The theorem’s constructive direction is the synthesis algorithm the Supremica and TCT tool families implement, and it is small enough to run on

paper. A sandboxed code-review job carries three rules: P_1 , no `net_egress`, ever; P_2 , no `git_push`, ever; P_3 , after any `in_context_read` of customer material, no `fs_write`. P_1 and P_2 are one-state automata and P_3 is the two-state taint automaton, so the product specification has $1 \times 1 \times 2 = 2$ states, `ok` and `tainted`. The controllability check visits both states and tests each of the four uncontrollable events in each: all eight (state, event) pairs are enabled by the specification, so there are 0 violations and the composite is controllable [verified: eight cases, checkable by eye]. The maximally permissive supervisor falls out of the proof as a two-row disablement map: in `ok`, disable `{net_egress, git_push}`; in `tainted`, disable `{net_egress, git_push, fs_write}`; one observed transition flips `ok` to `tainted`. That table is exactly the policy engine a competent engineer would have written by hand. The value of synthesis is not surprise; it is that the answer arrives with a maximal-permissiveness certificate and a controllability certificate, neither of which hand-written engines carry.

Synthesis is equally informative when it fails. Ask for the best preventable approximation of “forbid `internal_plan`”: any nonempty controllable sublanguage K' contains the empty history, the plant enables `internal_plan` there, so controllability forces `internal_plan` $\in \overline{K'} \subseteq \overline{K}$, contradicting that K forbids it. Hence $\sup C(K) = \emptyset$ [verified: three lines]. There is no nontrivial preventable weakening of a thought ban, and a governance document that claims to “partially enforce” such a rule by prevention is describing detection with optimistic vocabulary. *Now you try:* add P_4 , “no `spawn_child` after `git_push`,” to the review job and write the new disablement map (Exercise 1.20).

Table 1.10 states the two-row disablement map the proof falls out as.

Table 1.10: The sandboxed review job’s maximally permissive supervisor: a two-row disablement map, checkable by eye against the three rules P_1 – P_3 . [verified: eight cases]

state	disabled events
ok	net_egress, git_push
tainted	net_egress, git_push, fs_write

1.7.3.2 Partial observation: the second boundary

Theorem 1.7.3 assumes the supervisor *observes* every event, even the ones it cannot disable: the daemon logs tokens after emission and reads after completion. Realistic deployments break this in a second place. Some events are not only unrefusable but unobservable: activations never surfaced, state internal to a third-party tool, another principal’s context. Supervisory control under partial observation is also solved

territory, and the refinement is worth stating with the same precision as the theorem it refines — this is a *local* correction in Lakatos’s sense, a hypothesis that was doing work without being named, so the classification survives once the hypothesis is written down, and nothing the chapter concluded under full observation is withdrawn.

THEOREM 1.7.4 Regimentation under partial observation. Let L , K , Σ_c and Σ_u be as in Theorem 1.7.3, but let the supervisor observe only $\Sigma_o \subseteq \Sigma$, through the projection $P : \Sigma^* \rightarrow \Sigma_o^*$ that erases every unobservable event. A regimentation mechanism for K whose decisions depend only on what it observed — a supervisor that factors through P — exists if and only if K is controllable with respect to L and Σ_u and K is *observable* with respect to L , P and Σ_c : for all $s, s' \in \bar{K}$ with $P(s) = P(s')$ and every $\sigma \in \Sigma_c$,

$$s\sigma \in \bar{K} \wedge s'\sigma \in \bar{L} \implies s'\sigma \in \bar{K},$$

that is, no two histories the runtime cannot tell apart may demand opposite decisions about the same controllable event [verified framework: Lin and Wonham [152]].

Proof idea. A supervisor that only sees $P(s)$ has one decision to spend on the whole class of histories that share a projection. If the policy ever wants σ enabled after s and disabled after an indistinguishable s' , no such supervisor can serve both, and the failure is not a matter of trying harder: it is a counting argument about how many decisions the observation map leaves available. If the policy never asks for that, the laziest supervisor of Theorem 1.7.3 can be defined on projections instead of histories, and the same closed-loop argument goes through unchanged.

Observability strictly shrinks the regimentable set, and $\Sigma_o = \Sigma$ collapses the condition to a tautology, which is why the first theorem never had to mention it. “No egress after the model *considered* exfiltration” has a controllable prohibition and an unwitnessed trigger, so no supervisor can implement its gate condition — two runs, identical in everything the daemon logged, differing only in a deliberation nobody saw, are exactly the s and s' the condition forbids. The practical reading: a compound policy is deployable only if its trigger is witnessed at the mediation boundary. Taint must be an event the daemon sees, not a state it infers. Figure 1.15 puts both boundaries on one grid.

What it buys. The chapter’s own vocabulary. *Regimented* and *enforced* (Definition 1.7.1) are now the two sides of one inequality rather than two adjectives; every row of Table 1.12 marked “detected, not regimented” is a row whose prohibition names an uncontrollable event or

		the trigger is witnessed at the mediation boundary	
		no	yes
the event is controllable: it passes through the daemon	yes	detect-only a plan stated in the model's context	preventable ★ git_push, net egress, spawn child
	no	detect-only a thought	detect-only an effect whose guard reads unwitnessed state

Figure 1.15: Preventable is one cell, not a spectrum. An event is preventable exactly when it is controllable (top row) *and* its trigger is witnessed at the mediation boundary (right column): the shaded cell, marked ★ and tied to Theorem 1.7.3. Every other combination is detect-only. Column membership reproduces from the checker; row membership is by inspection of each trigger. [internal, b3_controllability.py]

an unwitnessed trigger. Downstream it buys the assurance modes the rest of the book uses to describe what a runtime can see and control, the design rule behind the sealed room, and the pricing stance of the market chapters: detect-only policies are enforced by audit, reputation, and bond-slashing, which must be sized for violations that *will* occur.

The ladder these modes form has two obvious public neighbours, and since the resemblance is real it's worth saying exactly what each of the three grades before somebody assumes they're the same instrument in different clothes. Sheridan and Verplank rank a system on a ten-point scale by how much of the decide-and-act loop the computer holds rather than the human [236]; Parasuraman, Sheridan and Wickens later split that single scale across four stages of processing — acquisition, analysis, decision selection, action implementation — each carrying its own level [193]. Both grade the *allocation of the work*: who does it. The Common Criteria's evaluation assurance levels, EAL1 through EAL7, rank a security claim by how much independent evaluation effort was spent before deployment and against how deep a body of design evidence, from functionally tested at the bottom to formally verified design at the top [120]. EAL grades *how hard somebody looked*, once, at evaluation time, for a whole product. This ladder grades neither, and the question it does ask is narrower than both: of one runtime claim about one mechanism, *what would have to be defeated for the claim to be false*? Nothing at all at Observed — the act may have happened somewhere else entirely and the record was written afterwards — then a client's

willingness to declare at Coordinated, a daemon-owned broker at Brokered, the operating system's own confinement at Confined, and a measured environment plus a conditional key release at Attested. That is an adversary's work factor, per mechanism, at run time, and neither neighbour measures it.

DESIGN INVARIANT 1.7.5 The three gradings are independent axes. No mechanism's rung in this book is assigned from its automation level or from its evaluation depth, and the corners are all occupiable. A fully autonomous component (Sheridan level 10) whose actions are merely logged is OBSERVED; a component that stops for human approval at every step (level 3) but routes its one effect through an attested broker is ATTESTED. An EAL7 product can contain OBSERVED mechanisms and an unevaluated one can contain ATTESTED mechanisms, because EAL scores the evaluation and this ladder scores the mechanism.

The boundary on that is worth stating in the same breath: the ladder is a design vocabulary, not a measured scale. No procedure assigns a rung by measurement, no inter-rater agreement stands behind the assignments, and the five rungs are not claimed to be evenly spaced or even totally ordered — Brokered and Confined defeat different adversaries rather than the same adversary harder, which is why the appendix says these modes are not a maturity staircase. What the ladder is for is naming which part of the system a claim rests on, and it earns that only by being applied consistently.

(i) The theorem does not say detect-only policies are hopeless. Detection feeding audit, reputation, and slashing is real enforcement; the theorem says only that prevention is off the table. (ii) It certifies no implementation. Its Σ_c is the set of events *actually* mediated, and every unenumerated side channel (raw sockets, DNS, inherited file descriptors, /proc, credential helpers, package managers, provider callbacks) silently moves events from Σ_c to Σ_u and shrinks the regimentable set behind the operator's back. Complete mediation of the claimed alphabet is an architectural obligation (Anderson; Rushby) that a red team, not this proof, discharges. (iii) The classification is relative to the alphabet split: a runtime that gates model steps changes Σ_u and re-grades the table. The theorem is a functor from architectures to boundaries, not one fixed verdict. (iv) Full observation is the assumption this classification rests on: policies whose triggers are unwitnessed internal state fall to the Lin-Wonham refinement even when their prohibitions are controllable. (v) The checker verifies the stated automata over the stated alphabet; it is a faithful implementa-

WHERE THIS STOPS

tion of the test, not a mechanized proof of the theorem. An Isabelle or Coq formalization remains open. (vi) Prevention governs effects, never semantics. For “no confident falsehoods in status reports” the obstruction is observability: a report leaves through emission and then through `api_call`, `fs_write`, or `net_egress`, all controllable, so the prohibition can be made controllable; but “this report is false” is not a predicate over the event alphabet, so the gate condition is unwitnessed at every alphabet split. The rule falls to (iv), not to the box’s inequality.

The monitor is honest about its own coverage, which is itself a genuine security property: it reports each rule as *enforced*, *degraded*, or *stuffed*, so a reader can never mistake a stuffed rule for an active one. We note in §1.12 that one rule’s enforcement (capability escalation) is backed by a separate native component whose evidence is the weakest in the subsystem, carrying a time-of-check-to-time-of-use surface.

1.7.4 Commitments: obligation with oracle-bound closure

Commitments are durable, *violable* promises — in the sense of Cohen and Levesque’s “intention as a persistent goal” [50], with both single-minded and open-minded strategies for when a promise may be dropped. Two hardening laws are implemented, and they are the ones that determine safety (Figure 1.16).

Property 1.7.1 (Metric-control separation, Law 1). A commitment’s deadline is derived by the daemon from a scope policy and is *never* agent-supplied. The agent picks the work; the daemon picks the deadline. (Invariant I5.) The point is to deny the agent control of the very metric it is judged against. This removes one direct manipulation channel; it is not a general proof of resistance to proxy gaming or misspecified scope policy.

Property 1.7.2 (Oracle-bound closure, Law 2). Closing a commitment as “done” is refused unless the agent supplies a non-empty reference drawn from a *finite oracle vocabulary*: a released claim, a merged commit hash, a passing test identifier, or a satisfied policy sub-check. Free-text notes cannot close a commitment. (Invariant I6.)

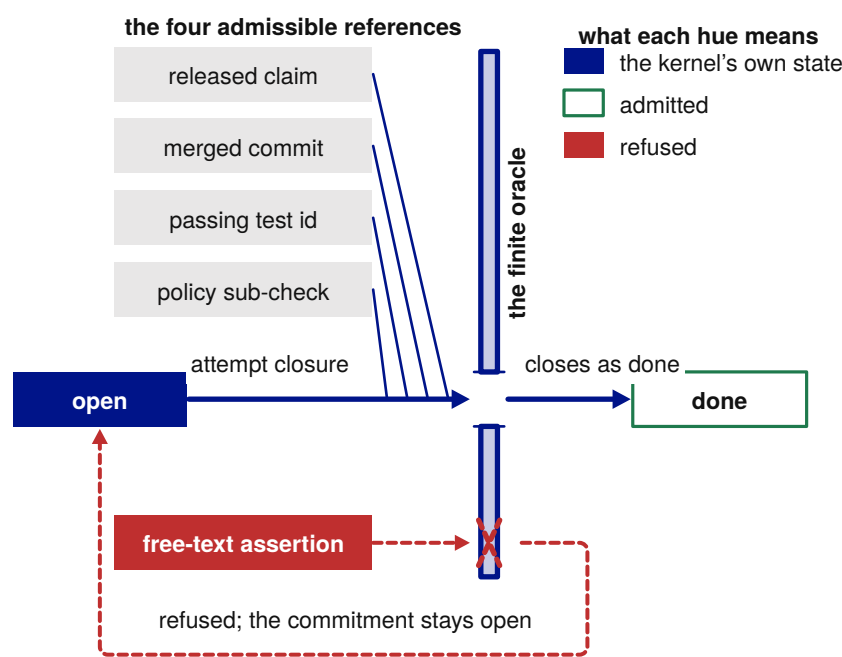
The closure gate is small enough to state as a protocol (Algorithm 1.2). The crux is the final clause: the supplied reference must not only be drawn from the vocabulary but must *currently hold* when the daemon re-checks it, so an agent cannot name a test that passed yesterday and has since regressed.

That finite oracle vocabulary *is* the kernel’s honesty boundary. It can mechanically verify “the test you named passed,” never “the refactor is good.” Naming the oracle set is naming the limit of what the substrate

RECALL

1. Without looking: which two sets does the controllability criterion mention besides the policy, and which of them is a dial the operator can turn?
2. State the design rule for compound policies in one sentence.
3. Why does the confident-falsehood rule stay detect-only even for a runtime that gates every effect channel?

KEY IDEA



the deadline is set by the daemon, never by the agent

Figure 1.16: Typed evidence passes a narrow closure aperture. A commitment closes as done only when the agent names a reference from the finite oracle vocabulary — a released claim, a merged commit, a passing test identifier, or a satisfied policy sub-check — and the daemon re-checks that the reference still holds at closure time. A free-text assertion (red, dashed) meets solid wall: it is refused and the commitment remains open. The deadline the commitment is judged against is derived by the daemon from scope policy and is never agent-supplied. [internal]

```

1 function close(commitment, status, oracle_ref):
2   if status != 'done':
3     return apply_drop_strategy(commitment, status)  //
      ↪ abandon / supersede
4   if oracle_ref is empty:
5     return REFUSED("done requires a verifiable oracle
      ↪ reference")
6   kind = classify(oracle_ref)  //
      ↪ structured field, not NLP
7   // the four kinds below are the SHIPPED vocabulary. State-
      ↪ Machine Assertions
8   // (OP-5, specified only) would add DELTA_ORACLE and
      ↪ AST_ASSERTION here.
9   if kind not in {RELEASED_CLAIM, MERGED_COMMIT, PASSING_TEST
      ↪ , POLICY_SUBCHECK}:
10    return REFUSED("oracle reference outside the finite
      ↪ vocabulary")
11   if not verify_holds_now(kind, oracle_ref):  // re-
      ↪ check at closure time
12    return REFUSED("named oracle does not currently hold")
13   transition(commitment, DONE, witnessed_by = oracle_ref)
14   return CLOSED

```

Listing 1.2: The commitment-closure oracle. Closure requires a reference from a finite vocabulary that the daemon re-verifies at closure time; free text is rejected by construction, not by inspection.

layer can verify — and open problem OP-5 asks which real “done” conditions it fails to capture, and whether the set can be extended without re-opening the Goodhart hole that free-text closure would.

1.7.5 Extending the oracle vocabulary: State-Machine Assertions (OP-5)

The obvious way to widen what a commitment can close against is to widen the vocabulary, and there is a concrete design for doing so without re-admitting the Goodhart hole. **State-Machine Assertions** (SPECIFIED) would add two kinds to Listing 1.2’s finite set: *delta oracles*, which close against a machine-checkable before/after measurement (a named benchmark improving past a declared threshold), and *AST assertions*, which use a parser such as *tree-sitter* to state a structural property of the resulting code (“no call site of *f* remains”) that the daemon can re-evaluate at closure time.

Both kinds satisfy the property that matters — the daemon can re-check them itself, so they are oracles and not testimony. Neither is in the shipped set: Listing 1.2’s four kinds are the vocabulary the kernel actually enforces, and the listing marks the two proposed additions as exactly that. Two questions also stand between the design and the claim, which is why we do not make it. A delta oracle inherits the noise of whatever it measures, so “improved by a threshold” is a statistical statement dressed as a boolean; and an AST assertion is a proxy for a structural intent, which is the shape of every target that has ever been gamed. *Oracle completeness* — the claim that some finite vocabulary captures the “done” conditions agents actually need — is not established by adding two kinds, and OP-5 stays *open*.

A reader who has got this far usually asks a different question — not “is the vocabulary complete?” but “who keeps the oracle honest?” — and that one has an answer, in another chapter. *From Spawn to Person* builds an inspection tower over graders and proves a contraction: escalation through C disjoint cliques drives the probability that a bad verdict survives down geometrically, so no single grader is trusted and none has to be. Note what that does and does not settle, because the two are easy to run together. It is a result about the oracle’s *incentives* — whether a grader who would rather lie can profit by it — and OP-5 is a question about the oracle’s *expressiveness*, whether the vocabulary can state the closure condition at all. A perfectly incentivised grader still cannot close a commitment the vocabulary cannot phrase, which is why the tower does not move this status and OP-5 stays where it is.

*Exercises 1.16–1.21,
p. 71.*

1.8 The continuity subsystem: memory, checkpoint, and the foundation of the economy

The through-line of the whole series — memory → checkpoint → continuity → durable identity → reputation → market — *bottoms out here*. If the substrate’s continuity is weak, the cross-machine economy is built on sand. This is the section the personhood and market papers lean on hardest, so it carries the canonical statement of the layer’s own softest link.

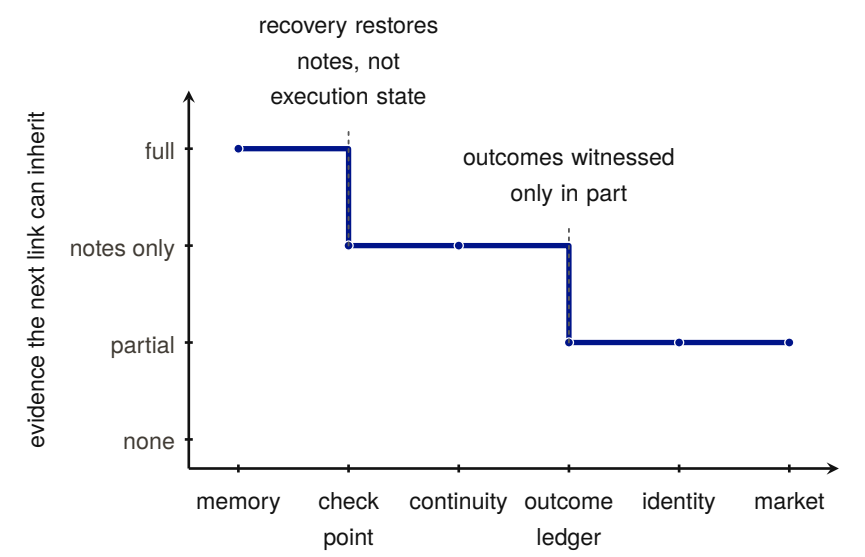


Figure 1.17: The continuity chain as an evidence-support profile, computed from the chapter’s own status grades. What checkpoint inherits from memory already drops from full to notes only; what the outcome ledger inherits from continuity drops again, to partial; identity and market inherit only that partial residue, not an independent grade of their own. Downstream confidence cannot exceed the weakest upstream link. [internal]

1.8.1 Three subsystems, one weak link

Figure 1.17 names the three continuity subsystems and states the canonical sentence the rest of the series depends on:

The economy rests on three continuity subsystems — memory (IMPLEMENTED), checkpoint (PARTIAL: notes, not execution state), and a witnessed-outcome ledger (PARTIAL: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third subsystem, which does not yet exist; the checkpoint subsystem is the weakest continuity link, and a checkpoint with teeth (a real

THESIS

execution-state snapshot) is the literal foundation of the cross-machine economy.

Memory — session records, durable notes, and the operation log — is **IMPLEMENTED**, and the kernel even forbids amnesia: a monitor rule guards that an active session’s durable note count never regresses (detect-and-compensate, per Theorem 1.7.1, but real).

Checkpoint is **PARTIAL**, and the weakness is precise. The recovery pipeline runs a heartbeat watchdog → stale/dead detection → a recovery queue → a salvage handoff, and it *passes notes*; it does not checkpoint execution state. A checkpoint with teeth — the gap between passing notes and restoring work — is open problem OP-4, the most important unbuilt thing at this layer because it is the literal foundation of any reputation economy.

The most promising direction we have for it is worth naming, at exactly its strength. **Event-sourced rehydration** (**SPECIFIED**) would treat the successor’s starting context as a replay rather than a summary: pin the working tree to the predecessor’s Git commit, truncate the durable message log to the recorded failure point, and replay that log into a fresh agent so its prompt prefix is reconstructed from the record instead of paraphrased from it. What this restores is the *durable* half — a commit identifier, a message log, an open-claim and commitment set — and that half is genuinely worth having. What it does not establish is the half OP-4 is named for. A language model’s inference state is bound to a particular model, sampler, and session; a replayed prefix reconstructs the *inputs* to that state, not the state, and the reference implementation ships none of this today. Whether replaying the inputs is close enough to count as “restoring work,” and against what test, is precisely what remains open — so we report the mechanism as a direction with an unestablished completeness claim, not as a closure of OP-4. The companion personhood chapter reaches the same boundary from the identity side: a continuity witness attests that the ledger followed the right body, never that the successor inherited the predecessor’s experience.

Figure 1.18 draws the pipeline with its own exclusion: what replay restores, and the one thing it does not.

The witnessed-outcome ledger — the durable record of what an agent actually delivered — is the subsystem on which reputation keys. A **PARTIAL** substrate now ships: durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI surfaces, and overdue detection. Neutral graded outcome events, sanctions, identity binding, and reputation updates remain absent. OP-4 (here) and that richer outcome-ledger gap (the personhood paper) are the *same* finding viewed from two ends.

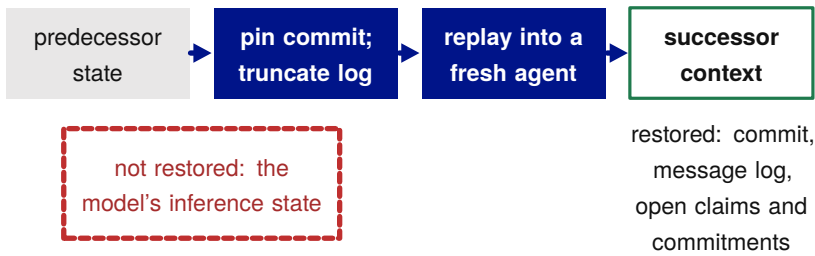


Figure 1.18: Event-sourced rehydration (DESIGNED), as a pipeline with its own exclusion drawn in. Pinning the predecessor’s commit and truncating the message log to the recorded failure point, then replaying it into a fresh agent, restores the durable half: a commit identifier, a message log, an open-claim and commitment set. The dashed box names what this does not restore – a language model’s inference state is bound to a particular model, sampler and session, so a replayed prefix reconstructs the inputs to that state, not the state itself. This is a direction for OP-4 with an unestablished completeness claim, not a closure of it. [internal]

1.8.2 The operation log and tamper-evidence

The append-only operation log is the kernel’s audit subsystem — the stream the policy monitor subscribes to and the thing that makes “what actually happened” answerable. Hash-chained tamper-evidence over it — the digital-timestamping pattern of Haber and Stornetta [99] that predates blockchains, itself building on Merkle’s hash trees [170] — exists but is re-scoped (see §1.12.4).

Exercises 1.22–1.24, p. 71.

1.9 The self-attestation subsystem: honest green

Honest self-attestation belongs to the substrate layer, because the kernel is the one component that can formally report its own integrity: it is always-on and owns the only writable truth. The attestation routine evaluates a registry of invariants, each returning *pass*, *fail*, *skipped-with-reason*, or *unknown*. The report is *scoped and conjunctive*: “all good” is printed only when every checked invariant passes, and the report always enumerates what was *not* checkable.

Property 1.9.1 (Honest self-report, I10). The attestation reports “all good” only if every checked critical invariant passed, and it always lists the unchecked set. There are three triggers: a boot-admission gate (refuse to serve on a critical failure), a command pre-flight (loud refusal that names the fix), and a continuous watchdog (a pass-to-fail transition deposits a coordination marker and a notification). **IMPLEMENTED.**

Table 1.11 lines the three triggers up against when each fires and what it does when a checked invariant fails; a row is silent about *which*

class of failure is most likely there because the registry is the same one at every trigger, only the moment and the response differ.

Table 1.11: The three attestation triggers, all evaluating the same invariant registry (*pass*, *fail*, *skipped-with-reason*, or *unknown*), differing in when they run and what a failure does. [internal, I10]

trigger	when it fires	on a critical failure
boot-admission gate	at daemon start, before serving any request	refuses to serve
command pre-flight	before a command runs	a loud refusal that names the fix
continuous watchdog	on every check cycle while running	a pass-to-fail transition deposits a co-ordination marker and a notification

This is the honesty discipline of the rest of the series turned on the kernel itself, and it is the right posture for a trusted computing base: when integrity is unknown, deny. The drift and liveness self-checks exist precisely because a second copy of the daemon *can* come up against the same file — a packaged install lagging the development tree is a recurring hazard — and the kernel must know which copy of itself is the real one.

1.10 The invariants, stated as theorems

A completionist treatment names the kernel's properties as falsifiable claims, each with its mechanism and its maturity grade. Table 1.12 is the formal spine of the layer; the grade column is the maturity discipline turned on the theorems themselves.

1.10.1 The work unit, model-checked

Express lane: the kernel's promises are properties of one object, an evidence-bearing work unit; on a two-principal instance every one of its 536 reachable states satisfies six invariants, and removing any single guard lets a checker construct the crime in at most seven steps. The machine is Figure 1.19 and the box below; the numbers are Table 1.13.

The scene. An agent dies mid-task. Its successor, possibly a different model from a different vendor, picks up the work and later says “done.” Who checks the paperwork? Where is the record that the port it held was released, that the payment moved once and not twice, that “verified” meant an inspection actually happened rather than a mood? Every subsystem in this chapter answers a piece of that question. The

Table 1.12: The kernel invariants. I1 is split by fault class (I1a/I1b) per §1.4; I7–I9 carry the regimentation/detection correction per §1.7.

#	Invariant (theorem)	Mechanism	Maturity
I1a	Process-crash durability	WAL + OS page cache	IMPLEMENTED
I1b	Power-loss durability	(needs full synchronization)	<i>not guaranteed</i>
I2	Mutual exclusion (1 holder/key)	uniqueness constraint + busy-wait	IMPLEMENTED
I3	Idempotence of re-claim/re-release	upsert / delete-if-exists	IMPLEMENTED
I4	Liveness reclamation (TTL sweep)	lazy expiry on read + ticks	IMPLEMENTED
I5	Metric-control separation (Law 1)	daemon-derived deadline	IMPLEMENTED
I6	Oracle-bound closure (Law 2)	finite oracle vocabulary	IMPLEMENTED
I7	No-amnesia (note monotonicity)	monitor: note-monotonicity rule	PARTIAL (detected, not regimented)
I8	Forbidden-state handling	constraint/boot (regimented) + monitor (detected)	PARTIAL (mostly detect-and-compensate)
I9	Tamper-evidence of the audit log	hash chain	PARTIAL (re-scoped to the economy layer, §1.12.4)
I10	Honest self-report	scoped attestation report	IMPLEMENTED
I11	Runtime parity (interpreter $\equiv$ test bindings)	runtime-shim layer	PARTIAL (OP-3)
I12	Non-forgeable identity	actor-soul id + lookup credential	PARTIAL (bounded gate ships; universal write gating absent)

work unit is the object that holds the pieces together: a case file that outlives every clerk who touches it.

The idea in one breath. *Make the unit of trust a work unit whose every consequential step passes a guard, then prove, by exhausting the machine's reachable states, that its six promises can never be broken, and that every guard is essential, because removing any one lets the checker construct the crime.*

Intuition: the notarized escrow folder. Nothing enters the folder without the right stamp: an active grant carrying the *current* role epoch, so a clerk who was replaced yesterday finds that today's stamp rejects him. No payment leaves twice: each external operation carries an idempotency key, honored at most once. And "verified" is not a feeling: it means the acceptance policy and the inspector's receipt are physically in the folder. Then we do something a paper folder cannot: enumerate every state the machine can ever reach and check all six promises in each. The misread to preempt: a model checker does not prove the daemon correct. It proves the *specification* correct at these parameters; the daemon must refine it, and that refinement is a separate obligation named in the boundary below.

The checker that explores the machine is short enough to run while reading. Its output below is the evidence behind the state count and the shortest-crime table: with every guard on, no reachable state violates an invariant; with any one guard off, the checker finds the violating run and prints it.

the work-unit machine: 536 states, every guard essential.

AT THE TERMINAL

\$ python3 skills/harbor-results/scripts/c0_workunit.py

```
BASELINE (all guards on): 536 reachable states explored, violations: NONE -- all six
    ↪ invariants hold in every state [ok]
```

```
MUTATION SUITE -- disable each guard; the checker must find a violating trace:
```

```
guard OFF [epoch    ] -> caught I1/I5: effect admitted with stale epoch
    shortest trace (4 steps): propose->authorize -> start_exec -> reassign_role ->
    ↪ admit_effect(STALE)
guard OFF [idem     ] -> caught I2: idempotency key settled twice
    shortest trace (2 steps): ext_op(k1) -> ext_op(k1)
guard OFF [verify   ] -> caught I3: verified without policy+receipt
    shortest trace (4 steps): propose->authorize -> start_exec -> witness -> verify
guard OFF [attenuate] -> caught I4: child grant exceeds parent scope
    shortest trace (1 steps): delegate(ESCALATED)
guard OFF [settle   ] -> caught I6: settlement unfunded/unreceipted/wrong principal
    shortest trace (7 steps): propose->authorize -> start_exec -> set_policy ->
    ↪ admit_verifier_receipt -> witness -> verify -> settle(p2!)
```

```
Guards restored: 536 states, violations: none [ok]
```

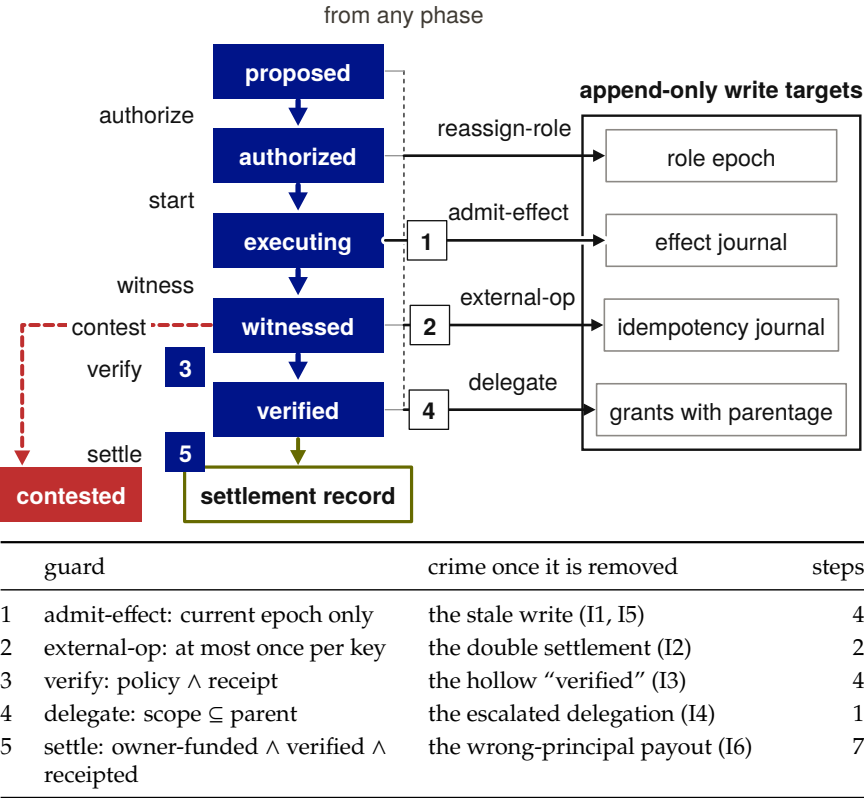


Figure 1.19: The work-unit machine. Five phases run down the spine to a settlement record; a witnessed unit may instead be contested. Admit-effect is the one write only an executing unit may make; external-op, delegate and reassign-role may happen in any phase. Five guards, numbered on the edge they guard and keyed in the table, are each essential: with all five on, all 536 reachable states of the two-principal instance satisfy the six invariants; with any one off, the checker finds the crime in the number of steps shown [internal, c0_workunit.py].

MODEL-CHECKED PROPERTY 1.10.1 Six invariants of the work-unit machine. **State.** A tuple of the work-unit phase (proposed, authorized, executing, witnessed, verified, contested), the acceptance policy and verifier receipt flags, the role epoch with its current holder and any stale token retained by a previous holder, the grants with their parentage, the effect journal, the idempotency journal, and the settlement record. **Transitions.** Propose, authorize, start, admit-effect (guarded on the current epoch), reassign-role (epoch increments; the old holder keeps a stale token), delegate (guarded on $\text{scope} \subseteq \text{parent}$), external-op (guarded at-most-once per key), witness, admit-receipt, verify (guarded on $\text{policy} \wedge \text{receipt}$), contest, settle (guarded on $\text{owner-funded} \wedge \text{verified} \wedge \text{receipted}$). **Invariants.** I1, every admitted effect carried the current role epoch at admit time; I2, each idempotency key settles at most once; I3, a verified completion has an acceptance policy and an admitted verifier receipt; I4, no delegated grant is more permissive than its immediate parent; I5, role reassignment invalidates stale epochs at the effect boundary; I6, every settlement is funded by the owning principal and references a receipt. **Result.** On the two-principal instance (one work unit, one exclusive role with fencing epochs, a root grant with one delegation, one idempotency key, at most two effects), all 536 reachable states satisfy all six invariants; disabling any single guard yields a violation, found automatically as a shortest counterexample trace, and restoring the guard restores safety [internal, `co_workunit.py`].

I2's guarantee is what the literature calls **effectively-once** processing: a deduplication discipline enforced at the boundary (an idempotency key plus a unique receipt row), not a promise the network itself keeps, since true exactly-once delivery across an unreliable channel is impossible [105].

Proof idea. There is no proof to read; there is a search to rerun. The checker performs a breadth-first enumeration of the reachable states from the initial state, evaluates the six predicates in each, and stops at the first violation. The mutation suite then disables one guard at a time and reports the shortest violating trace it finds. A checker that has never caught a seeded bug is an unvalidated checker; the five traces below are the evidence that this one has teeth.

Numbers by hand — The crimes, and their shortest scripts. Table 1.13 lists the five guards and the shortest trace the checker finds when each is

■

disabled [internal, c0_workunit.py]. Two of them you can reconstruct by hand. *The stale write in four steps*: authorize the work unit; start execution; reassign the role, so the epoch becomes 1 and the old holder retains token 0; now the old holder admits an effect with token 0. With the epoch guard on, that last transition does not exist, so the state is unreachable; with the guard off, the effect lands carrying yesterday's epoch and I1 fails. This is exactly the "paused holder wakes after expiry" scenario the fenced-lease theorem (Theorem 1.5.1) exists to exclude, and it is the shortest possible: no shorter history contains both a reassignment and a subsequent effect. *The double settlement in two steps*: run the external operation under key k_1 , then run it again. With the idempotency guard off the second run settles the same key twice and I2 fails; no history of length one can settle anything twice. *Now you try*: the escalated delegation takes one step. Which transition is it, and which invariant does it break? (Exercise 1.25.)

Table 1.13: Every guard is essential. Disabling one guard at a time, the checker's shortest violating trace, in transitions from the initial state [internal, c0_workunit.py].

Guard disabled	Crime	Steps	Invariant broken
epoch check at admit-effect	the stale write	4	I1, I5
at-most-once per idempotency key	the double settlement	2	I2
policy $\wedge$ receipt at verify	the hollow "verified"	4	I3
scope $\subseteq$ parent at delegate	the escalated delegation	1	I4
owner-funded $\wedge$ verified $\wedge$ receipt at settle	the wrong-principal payout	7	I6

The seventh row deserves a sentence. To reach the wrong-principal payout the checker had to walk the entire legitimate path first: set the policy, admit the receipt, witness, verify, and only then settle from the wrong purse. That is exactly how the fraud would look in production, which is the point of asking a machine rather than an author to find it.

What it buys. Every earlier theorem in this chapter now has a substrate to be a property *of*. Table 1.12 states the kernel's invariants as claims about subsystems; the work unit is where they meet: I2 and I3 of the machine are the resource subsystem's fenced leases and idempotent re-claim, I4 is the authorization chain's monotone narrowing that the Anchor chapter mechanizes across principals, I3 and I6 are the

obligation subsystem’s oracle-bound closure carried through to settlement. The continuity chapter’s checkpoint and the market chapters’ settlement inherit the same object, which is why the book can name the work unit as its primitive in the front matter without a definition per chapter.

Bounded model checking on a small instance proves the specification at these parameters, not the deployed daemon: deployment must *re-fine* this machine, mapping every real code path to a transition, and that mapping is a separate obligation. Safety only: liveness (“the work eventually settles”) needs fairness assumptions this check does not make. And an invariant list is only as good as its coverage; the mutation suite certifies that these six have teeth, not that they are complete. The unbounded-parameter version, in TLA⁺ with Apache, is planned and not done.

1.10.2 The consistency-model theorem

The strongest words a single-writer design earns are “serializable” and “linearizable.” They are the formal payoff of the architecture, and they are what make the coordination layer’s typed ownership trustworthy. Figure 1.20 makes the two guarantees concrete on one history; the five assumptions that buy them are the theorem’s own hypotheses.

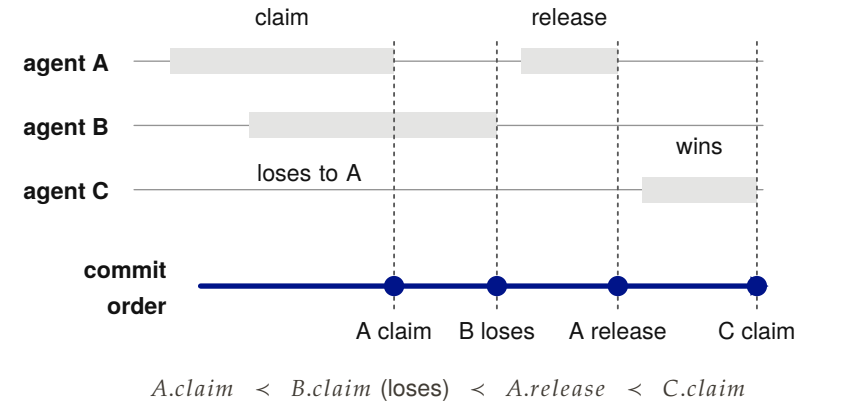


Figure 1.20: A real-time history and its extracted linearization. Operations overlap in caller time, but each receives one commit point on the single writer’s rail, and the linearization reads: A claims, B loses, A releases, C claims. The claim is conditional on one writer, no uncommitted reads, and replies after commit [internal, thm:consistency].

Theorem 1.10.2 (Conditional consistency model). Assume the single-writer discipline holds without violation (Def. 1.3.1), transactions never enable `read_uncommitted`, each successful response is sent only after commit,

WHERE THIS STOPS
RECALL

1. Without looking: name the three journals the work-unit state carries, and say which invariant each one exists to make checkable.
2. Why is the wrong-principal payout the longest crime?
3. What does the mutation suite certify, and what does it not certify?

Exercises 1.25–1.26, p. 72.

no out-of-band writer mutates the tables, and — the fifth assumption, which the first four were quietly leaning on — the execution is crash-free, or else every operation in it was routed at synchronous=FULL. Then committed transactions admit a serial order, and the externally observed claim/lock operations are linearizable with commit as the linearization point.

Proof sketch. Under the stated configuration there is one stream of write transactions, totally ordered by commit, and reads observe a consistent snapshot. For linearizability (Herlihy & Wing [107]) we need a single point per operation, between its invocation and response, at which it appears to take effect, with these points consistent with real time. The commit of each claim/lock operation is exactly such a point: it is atomic (Theorem 1.5.1), it lies between the client's request and the daemon's response. Non-overlapping operations respect real-time order because a later invocation occurs after the earlier response and therefore after its commit; overlapping operations may be ordered by commit. Hence the observable history is linearizable. $\square$

The fifth assumption is where this theorem and Property 1.4.1 finally meet, and until now they sat in two sections that never spoke. A history that spans a power loss is not merely a history with a gap in it: under synchronous=NORMAL the operating system may not have flushed the write-ahead log when the daemon answered, so the crash can take back a commit whose response a client already holds and acted on. That is not a reordering and the proof above does not break — every step of it is still valid for the operations that survive — it is a *durability* failure, and the property that rules it out is *durable linearizability* (Izraelevitz, Mendes and Scott [121]), which asks that the operations persisting across a crash themselves form a linearizable history. The kernel claims it only on the synchronous=FULL path, which is exactly the scope of I1b and the reason OP-10 (§1.4) is still *open*: ordinary claim writes ride NORMAL and buy latency with a guarantee they do not make. Naming the hypothesis re-scopes the theorem; it withdraws nothing the theorem proved.

WHERE THIS STOPS

This is why the kernel needs no agreement protocol inside one fault domain. Linearizability follows from the routing, transaction, and response-order conditions above; it is lost if a second writer or an out-of-band read bypasses those conditions. One decider, one order, one auditable truth.

KEY IDEA

1.10.3 The dual-runtime hazard, and what would make I11 a theorem

Invariant I11 (runtime parity) is the one place the formal spine is held together by a compatibility shim rather than a proof. In deployment, truth is *computed* under the SQLite bindings of one runtime (the compiled single-binary daemon), but in the test suite it is *verified* under a different runtime's bindings; a thin shim normalizes their pragma and option interfaces. An invariant that is green under test can fail in production if the two bindings diverge in lock or pragma semantics — the classic “green in the test runtime, broken in the deployment runtime” failure. A continuous-integration pipeline must therefore *boot the deployment runtime* and exercise its endpoints, not merely build it.

The harness that would turn I11 into a theorem is **differential fuzzing** (SPECIFIED, OP-3): drive a large randomized stream of concurrent operations — 10^6 is the order of magnitude the design targets — against both bindings from the same seed, and assert hash-equivalence of the resulting database state. It does not run in the reference implementation's pipeline today, and it is worth being clear about what it would and would not buy even when it does. Hash-equivalence over random operation streams is a *test*, not a proof: it would raise the cost of a divergence surviving to deployment, but a divergence that needs a specific interleaving, a specific pragma set, or a filesystem the fuzzer never exercises can pass 10^6 operations and still be there. I11 accordingly stays PARTIAL in Table 1.12, and OP-3 stays *open*.

Exercises 1.27–1.29,
p. 72.

1.11 Cross-subsystem atomicity: a buildable defect, not a frontier

“Claim this port *and* open this session *and* record this commitment” is three writes. Nothing at the kernel's interface boundary wraps them in one transaction, so partial-failure semantics across subsystems are undefined. It is tempting to file this under “research”; that is wrong, and the correction matters because it changes the engineering posture from “research” to “do it.”

This is the Saga problem (Garcia-Molina & Salem [87]) — but with a *cheap local fix*. These are all local writes to one file, and the database already provides a transaction primitive (used elsewhere in the system). Wrapping a multi-subsystem operation in one local transaction (begun in immediate mode) gives all-or-nothing for free. Filing this under “research” understates how cheap the correct answer is. It is a **buildable defect**: the durable primitive is present; what is missing is the wrapping of the multi-subsystem endpoints in it.

KEY IDEA

This is the kind of finding the maturity discipline exists to surface: a gap mislabeled as a research frontier hides a one-line fix behind a

grander word. Sagas matter when the steps are remote and a single transaction is impossible; here, where every step is a local write to one file, the single transaction is strictly simpler and strictly stronger.

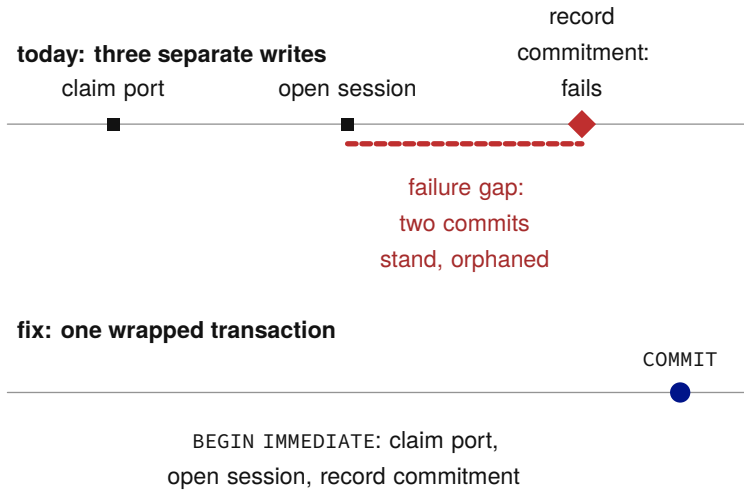


Figure 1.21: The same three writes, one local fix. Today (top), each write commits on its own; if the third fails, the first two stand, orphaned — a real failure gap. Wrapped in one local transaction, begun in immediate mode (bottom), all three commit together or not at all: no remote step, no Saga, because the durable transaction primitive already exists and every step is a local write to one file. [internal, OP-11]

Figure 1.21 draws the same three writes twice: today’s failure gap, and the fix’s single commit point.

*Exercises 1.30–1.31,
p. 72.*

1.12 Threat model and the limits of mediation

A systems-security paper lives or dies on its threat model. The kernel deliberately shrinks its trusted computing base (one writer, one file), which is good security hygiene — but the shrinking pushes several adversaries explicitly out of scope, and the honest move is to draw the boundary, not gesture at it.

1.12.1 Advisory claims are not sandboxing

The sharpest category error to avoid: edit-surface claims are *advisory coordination locks between cooperating agents*, not OS-enforced filesystem isolation. There is no syscall filter, no `pledge/unveil`, no macOS Seatbelt, no Landlock. A malicious agent ignores a claim row and writes the file. The kernel provides *advisory* coordination; real OS sandboxing of agent processes is out of scope at this layer (it belongs to the out-of-band “jail” — the separate-user kernel boundary of §1.12.2 — which

Table 1.14: The substrate-layer threat model. The same-user adversary is *out of scope* by design; several “security” subsystems become necessary only against the cross-machine adversary, which belongs to the economy layer.

Adversary	In/out of scope	What defends (or does not)
Buggy cooperating agent	in	Claims, commitments, post-commit monitoring, oracle-bound closure. The primary design target.
Misbehaving agent (ignores advisory claims)	partial	Edit-surface claims are <i>advisory coordination</i> , not OS-enforced isolation. A malicious agent can write the file anyway (no kernel-level sandbox such as Landlock, <i>unveil</i> , or <i>Seatbelt</i>).
Same-user process (reads/edits the DB)	out	The database is owner-read/write only; the signing key is being migrated to the OS keychain. Hash-chain tamper-evidence defends against <i>nobody</i> under this exclusion (§1.12.4).
Different-uid process	in (partial)	Owner-only permissions on the database and both socket files; but the socket peer-credential check is a software handshake (a self-reported agent identity checked against the registration table), <i>not</i> the kernel-enforced socket credential (SO_PEERCREDS) — a real authentication caveat. This row is the document’s operative statement: no other section supersedes it (§1.12.4 specifies hardening that would, but does not ship).
Cross-machine / cross-operator	out (economy layer)	Needs the complete local identity contract (PARTIAL) and cross-operator attestation (the economy paper).

is *proposed* and would need to be grounded in a real operating-system primitive). We say so plainly rather than let advisory claims be read as isolation.

1.12.2 The enforcement gate proves; it does not yet compel

The same boundary problem reappears one level up, at the kernel’s *enforcement* ambition, and it is the paper’s most consequential “*specified, not implemented*” caveat. The kernel ships a real cryptographic gate — a **macaroon** [30] discharge primitive (a bearer credential whose authority can only ever *narrow*, since each caveat is folded into a chained keyed hash so that tampering breaks the signature). Its reference implementation is `core/kernel/pd-anchor/src/macaroon.rs`: the root key never leaves the daemon, the third-party “rent-paid” caveat is an HMAC commitment to a discharge the daemon alone can issue, and the discharge construction is machine-checked in a protocol-verification tool (`analyses/macaroon_discharge_v1.pv` proves no push is authorized without a daemon-issued discharge bound to *that* grant). The cryptography is **IMPLEMENTED** and sound *as cryptography*.

A gate that is cryptographically unforgeable is not, for that reason, *compulsory*. The macaroon proves that a given push was authorized; it does not yet *stop* an unauthorized one, because the agent runs as the same user and holds its own host credentials.

KEY IDEA

What it does *not* yet do is bind that gate to the agent’s actual capability to act. Today’s coordination enforcement is the **Coordination Guard** (`cli/commands/guard.ts`) and a **git shim** (`cli/utils/git-shim.ts`) that intercept destructive verbs — and both run *in-band*, as the same user, through a PATH the agent controls. By construction that arrangement is advisory: an agent that hits the wall has at least five escapes that defeat no cryptography (a documented bypass environment variable, an absolute path to the real `git`, a PATH rewrite, in-process `libgit2`/editor bindings, or simply never attaching a session). The design record for this is **ADR-0053** (`docs/adr/0053-out-of-band-enforcement.md`), which states the ceiling plainly: *you cannot out-trick a same-user process from inside that same process*. The cryptographic version of the same limit is ADR-0050’s concession — *a secret a process can use, it can copy* — and it applies to enforcement verbs no less than to keys: a guard a process can decline to invoke is advisory.

The third instrument of the same kind is the **hook harness** (ADR-0108, which absorbed the earlier ADR-0091; both still marked Proposed). It rides the lifecycle events a vendor’s agent loop already exposes (the prompt submission, the moments before and after a tool call, the compaction, the stop — under whichever names the vendor happens to give them), and at each event it may do one of three things: inject a bounded advisory text into the model’s context, block the act,

or charge it against a budget. Two dials set how hard it leans, from advisory through warn to enforce, with a one-second deadline per hook and a breaker that stands the whole harness down when hooks start failing. Of the three instruments it is the most useful, because it is the one that actually *shapes* the cooperative case, and the ADR grades it with the same sentence this chapter would: blast-radius reduction for the cooperative case, not a vault — a same-user agent that means to can disable the hook, or simply reach the effect directly. Guard, shim, and harness are therefore one thing seen three times: the instruments of the OBSERVED and COORDINATED modes, valuable for the transcript they leave and for the cooperative agents they steer, and silent about the agent that declines them.

Closing this gap is a layered program, and only the top layer is a real wall. A server-side branch-protection-plus-push-broker layer (SPECIFIED) removes the most painful direct-to-protected-ref incidents but not same-user trickery; a credential broker that meters and injects credentials per request (*proposed*) narrows the ambient-token surface; and only a **separate-user (or separate-VM) kernel process** with forced egress (*proposed*) actually confines a malicious same-user holder, because it is the first layer that removes the capability rather than refusing the verb. That separate-user boundary is the analogue of the hardware/operating-system boundary a classical reference monitor relies on for “always invoked,” and *it does not yet exist*. Until it does, the honest grade of kernel enforcement against a non-cooperating same-user agent is *not guaranteed*: the gate is a verifiable transcript and a loud tripwire, not a wall. We state this here so no reader carries the (legitimate) “mechanically verified macaroon” result forward into the (false) belief that the kernel can presently *compel* a same-user agent through it. Figure 1.22 lays out the four layers of the program and names, one clause each, what every layer actually stops.

That separate boundary is therefore the substrate this chapter prescribes, rather than one more option on the menu of layers. The guard and the shim are the interim instruments of an observed mode — kept because they leave a transcript, which is worth having — and the book does not build its argument on them. What the boundary does *not* settle is whether the single-writer rail is also the right way for several agents to share a repository, as against a worktree per agent with a merge queue. That is an empirical question (an honest one; the authors do not know the answer), and a pre-registered study outside the product (studies/substrate-study) is running it. Until it reports, the rail is a design invariant for confinement and a hypothesis for collaboration.

only the shaded layer removes the capability rather than refusing the verb

separate-user (or separate-VM) kernel, forced egress	Vision removes the capability itself: the only real wall
credential broker: meters and injects credentials per request	Vision narrows the ambient-token surface
server-side branch-protection-plus-push-broker	Designed most direct-push incidents, not same-user tricks
advisory guard, shim, hook harness – in-band, same user	Built nothing from an agent that declines them: advisory only

Figure 1.22: Enforcement closes as a layered program, and only the top layer is a real wall. Today’s advisory guard, shim, and hook harness (BUILT) run in-band as the same user: a same-user agent has documented escapes that defeat no cryptography. A server-side broker (DESIGNED) and a credential broker (VISION) each narrow the surface further, but only a separate-user or separate-VM kernel process (VISION, the shaded band) confines a malicious same-user holder, because it is the first layer that removes the capability rather than refusing the verb. [internal]

1.12.3 Partial actor-soul identity bounds every other invariant

The reference system now mints an opaque actor-soul id and lookup credential inside the daemon, and the budget guard assigns fresh actors to a shared bounded newcomer pool (I12 is PARTIAL). Legacy session paths and security-relevant writes that do not yet require that credential remain impersonable. This still bounds the security weight of I7, I8, and I9: lock-owner validity, capability escalation, note monotonicity, and owner-scoped release are only as strong as their least-protected identity path. This dependency — complete I12 enforcement gates the soundness of I7, I8, and I9 — is a hard precondition, stated here so the reader does not over-trust the enforcement subsystem. It is also the central reason the cross-machine layer, where identities genuinely conflict, must make identity cryptographic. Figure 1.23 draws the one-root, three-dependent shape of that precondition.

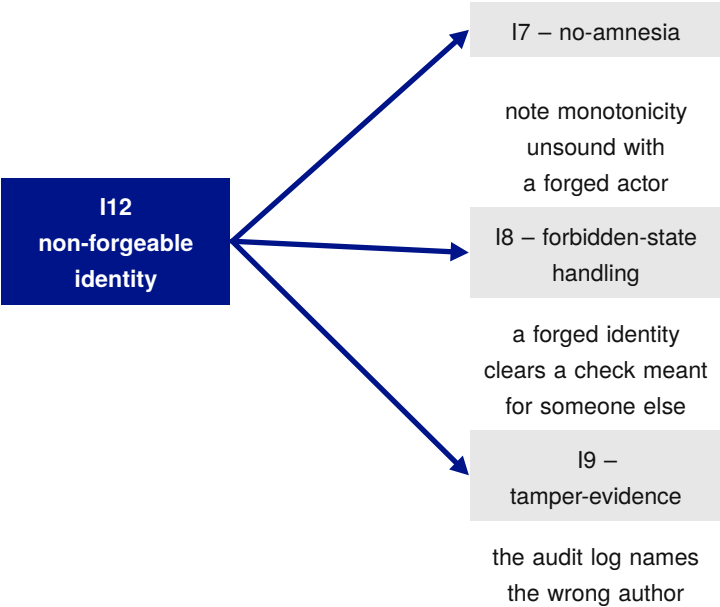


Figure 1.23: Complete I12 enforcement is a hard precondition for I7, I8 and I9: each is only as strong as its least-protected identity path. With I12 BUILTWEAK (a bounded actor-soul gate, not yet universal write gating), a legacy path with a self-asserted identity lets a forged release clear a lock-owner check meant for someone else – the monitor rule is correct relative to its input; I12 failed to authenticate the input. [internal]

1.12.4 Hash-chain tamper-evidence, and the hardening OP-9 would need

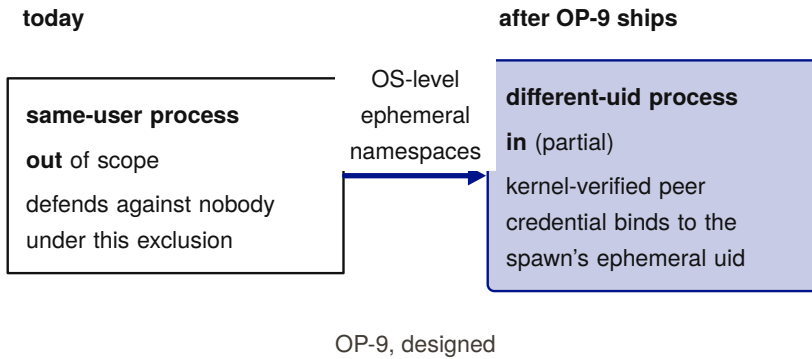
The hash chain (I9) is built, but under the initial threat model it defended against nobody: its only adversary was someone who could edit the audit log, which was exactly the same-user process the model excluded. Tamper-evidence that nothing on the read path verifies is a data structure, not a control. We therefore re-scope I9 as **cross-machine provisioning**: it matters only against cross-machine synchronization or a tamperer who is not the same user.

The design that would let the kernel defend against the same-machine adversary directly — and so close OP-9 — is **OS-level ephemeral namespaces** (SPECIFIED). The daemon keeps its database and sockets owner-only (0600), and spawns each agent inside a bwrap or unshare sandbox under an ephemeral UID of its own. The agent then ceases to be a same-user process, which is the only move that changes the answer: the filesystem bypasses of §1.12 stop being available because the capability is removed, not because a verb was refused. Paired with it, the daemon reads the *kernel-verified peer credential* of each connection — on Linux the struct `ucred` (pid, uid, gid) that the kernel records at `connect(2)` and `SO_PEERCREDS` returns via `getsockopt(2)`; on macOS the equivalent `LOCAL_PEERCREDS` — and binds each request to the ephemeral UID of the spawn instance it must have come from.

Three things about that paragraph determine the result, and none of them is a claim that OP-9 is closed.

- **It is not what the kernel does today.** The reference implementation does not read the kernel-verified peer credential: it relies on the owner-only socket permission plus a self-reported agent identity checked against the registration table, which is precisely the software handshake Table 1.14 records. Nor does it spawn agents into ephemeral-UID sandboxes. The threat-model row, not this section, states the kernel's current authentication posture.
- **The peer credential is not a cryptographic binding.** It is a kernel-recorded uid/gid/pid triple attached to a socket connection — strong precisely because the kernel, not the peer, supplies it, and *weak* in the way any identity is when the adversary can obtain the same uid. Nothing about it is signed, and calling it a cryptographic bind confuses the guarantee with the authorization chain's, which is a different mechanism (§1.6).
- **Even fully built, it would not make bypasses “impossible.”** It would move the same-machine adversary from the same-uid row of Table 1.14 to the different-uid row — a real and substantial improvement — while leaving shared kernel surfaces, whatever paths the sandbox binds in, and the sandbox implementation itself in scope.

“Impossible” is the wrong word for a containment boundary; “the capability is removed for this named class of access” is the right one.



a real improvement, and still not “impossible”: shared kernel surfaces and the sandbox implementation itself stay in scope either way

Figure 1.24: One threat-table row reclassified, not a new defense. Today the same-user process sits in the out-of-scope row, defended by nobody. OS-level ephemeral namespaces (OP-9, DESIGNED) would move it to the different-uid row, where the kernel-verified peer credential partially defends it – a real improvement, and, per §1.12.4’s own three caveats, not a claim that the boundary becomes impossible to cross. [internal]

Figure 1.24 draws that reclassification as the one thing it is: a move from one threat-table row to another, not a new defense.

OP-9 therefore stands *open*: a specified hardening path, not a shipped defense, and the honest grade against a same-machine adversary remains the one §1.12.2 gives — *not guaranteed* until a separate-user boundary exists.

1.12.5 Where information-flow control goes, and why it is not here

One question this section invites belongs to a different chapter, and the cleanest service we can do the reader is to say so rather than answer it badly. Capability scoping alone does not prevent exfiltration: an agent holding both `fs:read:/confidential` and `net:egress:github.com` can read the first and post it to the second, and no amount of claim discipline notices. The containment answer is information-flow control — taint the read, propagate the taint through spawns and shared writes, and gate every release channel behind a declassification policy with a leakage budget. Theorem 1.7.3 already gives the substrate-layer form of it in one line: gate the channel, never the token.

The full construction — a compute-to-data airlock in which one operator’s agent stack executes over another operator’s confidential data, with neither inspecting the other — is *cross-operator* by definition: two principals, mutual distrust, a licensing relationship, and a bond that can be slashed. Every one of those objects lives above this floor, in the chapter that owns the market between operators (*The Harbor Economy*), and this chapter’s abstract promises it lives there rather than here. What the substrate contributes to it is what this chapter has already specified: taint is a marker (§1.6), egress and file writes are controllable events (Theorem 1.7.3), the sandbox that would make an agent a different-uid process is OP-9’s specified hardening (§1.12.4), and the honest statement of what containment buys is the one this section keeps repeating: not that exfiltration becomes impossible — timing, ordering, and side channels stay out of model — but that every flow out passes a named gate whose releases are logged, budgeted, and attributable.

*Exercises 1.32–1.34,
p. 72.*

1.13 Adjacency contract: what the kernel assumes and provides

The kernel’s coherence with the layers around it is a contract: what it assumes from the machine below, what it provides to the protocol above, and — just as important — what it explicitly does *not* provide so higher layers do not over-assume.

1.13.1 What the substrate assumes from the machine

- A POSIX filesystem with working advisory locking (`flock/fcntl`). **This is a correctness precondition for I2:** networked filesystems break advisory locks, and a broken lock lets two writers both win — destroying mutual exclusion, not just performance.
- A local wall clock. Single-machine, so no shared-clock assumption is needed — but note that a wall clock is *not monotonic* (it steps when the clock is adjusted, e.g. by network time synchronization), an intra-node hazard for every expiry sweep that the inter-node ordering of Lamport [140] does not cover.
- Unix domain sockets, with owner-only permissions on the socket paths. Both transports authenticate a peer by software handshake over that permission model, *not* by the kernel-verified socket credential (§1.12); reading that credential is OP-9’s specified hardening, not an assumption the kernel currently makes.
- An OS keychain for signing keys; file permissions honored on the database path.



Lamport showed that happened-before, not a shared clock, is the ordering a distributed system can trust; the daemon’s wall clock does not give it for free.

- A supervisor process to keep the daemon always-on and to restart *the daemon itself* — the substrate makes other things always-on but cannot make *itself* always-on; that is delegated downward.

1.13.2 What the substrate provides to the coordination layer

- **Atomic, idempotent, time-to-live'd claims** (I2–I4) over ports, file-regions, symbols, and named locks — the coordination layer's typed ownership reduces to claim and release.
- **A durable, versioned, history-coursored publish/subscribe bus**, a tuple space, and decaying stigmergic markers — the coordination layer's speech act is carried *inside* the bus envelope; the anti-blackboard-rot property is *provided* by substrate-side decay.
- **The deontic split as a primitive** (Def. 1.7.1): prohibition → monitor (detect/regiment), obligation → commitment, permission → capability. The coordination layer must not re-implement enforcement.
- **Daemon-owned deadlines and oracle-bound closure** (I5, I6).
- **An append-only audit log** (hash-chain-provisioned) that the protocol and the operator read.
- **Self-attestation** (I10) — higher layers may *assume the kernel is honest about its own health*.
- **The cryptographic authorization chain** — the coordination layer's coordination lineage rides inside it.
- **Linearizable claim/lock semantics** (Theorem 1.10.2) — the property that lets the coordination layer treat “who holds this” as ground truth.

1.13.3 The explicit non-provisions

- The substrate does *not* provide end-to-end non-forgeable identity yet. I12 is PARTIAL: daemon-minted actor souls, lookup credentials, and a bounded newcomer pool enforced by the budget guard ship; universal write-boundary enforcement does not.
- It does *not* provide fair/queued exclusion (OP-1), cross-subsystem transactional atomicity by default (§1.11), a real execution-checkpoint (OP-4), power-loss durability on any write path (I1b, Property 1.4.1; the per-path selector of OP-10 is specified, not shipped), a kernel-verified peer credential on either transport (OP-9), or any cross-machine guarantee (the economy layer — different

theorems, a shared-clock and Byzantine-membership problem the substrate deliberately never touches). The status of each open problem is stated once in Table 1.15; a higher layer should assume nothing stronger than that table.

- It does *not* decide *what to do* (no orchestration). It enforces what *can-not* be done and records what *did* happen — a regulator, not a planner.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. One writer, one file. Every mutation reaches the daemon and is committed in one order; the reply follows the commit. Serializability and linearizability are bought by that discipline plus four named assumptions, not by an agreement protocol.
2. Durability is split by fault class. A process crash loses nothing; a power loss can lose the last commits under the default synchronous setting. The setting is the durability claim.
3. A claim is a lease with a fencing token (this chapter's epoch). At most one compatible holder per conflict domain; expiry is lazy; a stale holder's effects are refused downstream by the epoch, not by the lock.
4. The bus is at-least-once and says so. Benign redelivery is deduplicated by an idempotency key the coordination layer still owes; only protocol violations are loud.
5. Two chains, never one name. The authorization chain proves who authorized whom; the coordination lineage says what work is in flight. The second rides inside the first.
6. Prohibitions, obligations and permissions need three mechanisms. A gate refuses before the effect; a typed oracle closes an obligation after the fact; a grant simply holds.
7. The monitor is a detector. Prevention exists exactly for events that are controllable and witnessed at the boundary (the controllability theorem); everything else is detect-and-compensate, priced by the market chapters.
8. Done means re-checkable. A commitment closes only through a reference from the finite oracle vocabulary; free text bounces back to open.

9. Continuity narrows downstream. Memory survives a crash; checkpoint keeps notes, not execution state; the outcome ledger is partial. Nothing built on top can be more certain than that.
10. The kernel reports its own coverage. Enforced, degraded or stubbed, per rule, so the layers above never mistake a stub for a guarantee.

1.14 Exercises

§1.2 THE KERNEL AS SEVEN SUBSYSTEMS

- 1.1 CHECK ★ Which of the seven subsystems would still be meaningful if the daemon ran over an in-memory database with no disk at all? Which become vacuous? *Solution p. 525*
- 1.2 TRACE ★★ Pick any one subsystem and name the single SQLite table that is its “home” table. Then name one invariant that subsystem owes the layer above it. *Solution p. 525*
- 1.3 OPEN ★★★ The seven-subsystem decomposition is a *choice*. Is there a more natural cut, five subsystems, or nine? Argue for an alternative and show what it makes easier to reason about and what it hides. *Solution p. 525*

§1.3 THE SUBSTRATE SUBSYSTEM: ONE WRITER, ONE FILE

- 1.4 CHECK ★ The schema-by-union design lets any module create its tables in any order. Give a concrete two-module example where a lazy column-rename migration would be unsafe under this design. *Solution p. 525*
- 1.5 TRACE ★★ Follow a single claim request from command line to disk. At how many points does the single-writer discipline (Definition 1.3.1) hold by *routing* versus by the *file lock*? *Solution p. 525*
- 1.6 OPEN ★★★ The boot ledger orders migrations, but modules still self-initialize, so the order is not yet authoritative (OP-7). What is the smallest change that makes it so, a `user_version` sequencer over *declared* migrations, and can any useful remnant of “any module, any order” survive it for renames, backfills, and down-migrations? *Solution p. 526*

§1.4 DURABILITY, SPLIT BY FAULT CLASS

- 1.7 CHECK ★ Classify each fault as I1a-survivable or I1b-exposed: (a) a forced kill of the daemon process; (b) a clean operating-system *Solution p. 526*

reboot; (c) yanking the power cord; (d) an unhandled exception in a request handler.

- 1.8** TRACE ★★ A claim commits at t_0 ; the next auto-checkpoint fires at $t_0 + \delta$. Draw the window during which a power loss can revert the claim. Does a page-count auto-checkpoint threshold bound δ in wall-clock time? Solution p. 526

- 1.9** OPEN ★★★ Section 1.4 sketches Selective Checkpointing (OP-10), but nothing in the kernel opts into it. What is the cleanest interface that lets a settlement-ledger write demand I1b while keeping claim writes fast, and, harder, what stops a *future* write path from silently defaulting into the fast lane when it should have opted in? Solution p. 526

§1.5 THE RESOURCE SUBSYSTEM: CLAIMS, LOCKS, AND FAIR EXCLUSION

- 1.10** CHECK ★ Why does releasing a lock you do not hold return success rather than an error? Which property (I2 to I4) is this, and what would break if it raised? Solution p. 526

- 1.11** TRACE ★★ Two agents call *acquire* on the same fresh key at the same instant. Walk the uniqueness-constraint insert for both (Listing 1.1) and show exactly where the loser learns the holder's identity. Solution p. 526

- 1.12** OPEN ★★★ The Stigmergic Ticket-Lock buys bounded wait on the cooperative-release path only (OP-1). Extend it so that a lease reclaimed by the lazy TTL sweep also honors the ticket queue, still without a background scheduler, or argue that no reactive-only design can, and say what the minimum scheduler would be. Solution p. 527

§1.6 THE COMMUNICATION SUBSYSTEM: THE BUS, STIGMERGY, AND TWO DELEGATION CHAINS

- 1.13** CHECK ★ Why does read-time marker decay make the system more honest than a background tick alone? Give a scenario where tick-only decay would report a stale signal as fresh. Solution p. 527

- 1.14** TRACE ★★ A request speech act is sent over the bus and redelivered once by the at-least-once channel. Trace what the operator should see under the silent-dedup resolution, and what an envelope idempotency key would change. Solution p. 527

- 1.15** OPEN ★★★ What per-kind marker decay rate keeps stigmergic signals neither rotten nor prematurely evaporated (OP-8)? Frame this as an empirical measurement, not a constant to guess. Solution p. 527

§1.7 THE OBLIGATION & ENFORCEMENT SUBSYSTEM: THE SOVEREIGN'S TWO LEVERS

- 1.16** CHECK ★ For each monitor rule in Theorem 1.7.1, say whether it *could in principle* be moved to a pre-commit check (a before-insert trigger). Which genuinely cannot, and why? (Hint: heartbeat freshness is a failure detector, and Chandra and Toueg [45] show failure detection cannot be made perfectly synchronous.) *Solution p. 527*
- 1.17** TRACE ★★ An agent attempts to close a commitment with an empty oracle reference. Walk the Law-2 gate (Listing 1.2) and show exactly where and why the transition is refused. *Solution p. 527*
- 1.18** TRACE ★★ Theorem 1.7.3 settles which monitor rules are regimentable, so classify rather than re-prove. Take I7 to I9’s monitor rules (note monotonicity, capability escalation, lock-owner validity) and, for each, say whether its prohibition names a controllable event and whether its trigger is witnessed at the boundary. Which cell of Figure 1.15 does each land in? *Solution p. 528*
- 1.19** CHECK ★ Classify “forbid `api_call` after `spawn_child`” and “forbid `internal_plan` after `fs_write`” using Theorem 1.7.3. *Solution p. 528*
- 1.20** TRACE ★★ Add P_4 , “no `spawn_child` after `git_push`,” to the sandboxed review job and write the new disablement map by hand. *Solution p. 528*
- 1.21** OPEN ★★★ Extend the Law-2 oracle vocabulary to capture a “done” condition it currently misses (OP-5), without re-admitting free text. State your new oracle and argue it is Goodhart-resistant, for one of the two specified kinds (delta oracle, AST assertion) before inventing a third, since neither yet survives that argument. *Solution p. 528*
- §1.8 THE CONTINUITY SUBSYSTEM: MEMORY, CHECKPOINT, AND THE FOUNDATION OF THE ECONOMY**
- 1.22** CHECK ★ Why is “recovery passes notes” fundamentally weaker than “recovery restores work” for a language-model agent that died mid-edit? Name one thing notes preserve and one thing they cannot. *Solution p. 528*
- 1.23** TRACE ★★ Follow the note-monotonicity rule from an agent appending a note to the monitor detecting a regression. At which point is the violation already durable (per Theorem 1.7.1)? *Solution p. 528*
- 1.24** OPEN ★★★ What is the minimal durable artifact that turns “recovery passes notes” into “recovery restores work” (OP-4)? Is it a content-addressed snapshot of the working-tree diff, the open claims, the commitment set, and the last N turns of the agent’s transcript? What is recoverable versus fundamentally lost when a language-model agent dies mid-thought? *Solution p. 529*

§1.10 THE INVARIANTS, STATED AS THEOREMS

- 1.25** CHECK ★ The escalated delegation takes one step (Table 1.13). Which transition is it, and which invariant does it break? *Solution p. 529*
- 1.26** OPEN ★★★ Propose a seventh invariant that the six of Theorem 1.10.1 do not imply, add the guard that enforces it to `c0_workunit.py`, and report the shortest crime the checker finds with your guard disabled. *Solution p. 529*
- 1.27** CHECK ★ List the four assumptions Theorem 1.10.2 charges for linearizability. Which one fails first if an out-of-band SQLite connection is introduced? *Solution p. 529*
- 1.28** TRACE ★★ Construct a three-operation claim history and give a valid linearization. If two calls overlap, show why more than one linearization may still respect real time. *Solution p. 529*
- 1.29** OPEN ★★★ Specify the minimal differential-test suite that turns I11 from an assumption into conformance evidence (OP-3): what operation sequences, what observable-state assertions, run against both runtimes? Then attack your own answer: name a class of binding divergence that a seeded 10^6 -operation hash-equivalence fuzz would *not* catch, and say what would. *Solution p. 529*

§1.11 CROSS-SUBSYSTEM ATOMICITY: A BUILDABLE DEFECT, NOT A FRONTIER

- 1.30** CHECK ★ Give a concrete partial failure: port claimed, session opened, then the commitment write fails. What inconsistent state is left, and which agent sees it? *Solution p. 530*
- 1.31** OPEN ★★★ Sketch the interface change that wraps the three-subsystem “begin” operation in one local transaction (OP-11). What is the compensating action if you instead chose Sagas [87], and why is the transaction strictly simpler here? *Solution p. 530*

§1.12 THREAT MODEL AND THE LIMITS OF MEDIATION

- 1.32** CHECK ★ The hash chain defends against nobody in scope. Name the *out-of-scope* adversary it does defend against, and the layer (substrate or economy) where that adversary becomes real. *Solution p. 530*
- 1.33** TRACE ★★ An agent forges its actor identity to release another agent’s lock. Walk the lock-owner-validity rule and show exactly where I12’s absence lets the forgery through. *Solution p. 530*
- 1.34** OPEN ★★★ What is the minimal hardening that closes the same-machine adversary without a hardware trusted-platform-module dependency (OP-9): an OS keychain for the signing key, sealed memory, the ephemeral-UID sandbox and kernel-verified peer *Solution p. 530*

credential specified above? Where does each stop? After the sandbox lands, which row of Table 1.14 does the adversary move to, and what is still in scope there?

1.15 Related work

The kernel sits at the confluence of four literatures. We position it against each, and against the design space it deliberately declines.

1.15.1 Reference monitors and the trusted computing base

The organizing idea is the **reference monitor** of Anderson’s 1972 security study [10]: a mediator that is *always invoked* (complete mediation), *tamper-resistant*, and *small enough to be verified*. Lampson’s *Protection* [143] gives the access-matrix model the monitor enforces, and Saltzer and Schroeder’s design principles [224] — economy of mechanism, fail-safe defaults, complete mediation — are the yardstick we hold the kernel to. Where a classical security kernel mediates memory and CPU access for an operating system, ours mediates *coordination* state for a set of cooperating processes, and it achieves complete mediation not by interposing on every system call but by the cheaper expedient of routing every mutation through a single writer. Schneider’s theory of enforceable security policies via execution monitors [228] supplies the boundary that the central correction of this paper rests on (Theorem 1.7.1): an execution monitor can enforce only the safety properties it can prevent by truncating execution, which a post-commit observer cannot do. This is a narrower, cheaper problem than general operating-system confinement, and honesty requires saying where the narrower problem ends. The same-machine adversary this chapter’s own OP-9 leaves *open* (§1.12.4) is exactly the problem three current systems exist to close, each at a different price: seL4, which takes “small enough to verify” to its conclusion with a machine-checked proof of functional correctness from specification down to the binary [133]; gVisor, a user-space kernel that interposes on every system call a guest makes rather than trusting the host’s boundary to hold [277]; and Firecracker, the microVM monitor that has isolated untrusted multi-tenant workloads in production since 2018 [4]. Any of the three would give this kernel a stronger trusted computing base than an ephemeral-UID process boundary. None is adopted here, and the gap is named rather than closed — the single writer buys complete mediation for coordination state, not for the machine.

1.15.2 Durable storage and transaction processing

The durability analysis is grounded in Gray and Reuter’s canonical treatment [94], which separates atomicity and consistency from durability — the distinction the I1a/I1b split makes operational. The write-ahead-logging discipline descends from ARIES [176]; the specific crash semantics we inherit are SQLite’s WAL [111], whose own documentation states that the *normal* synchronization level may lose durability under power loss while preserving consistency. That caveat has an empirical backdrop: Pillai et al.’s crash-consistency study found persistence guarantees vary widely across popular Linux filesystems, and that many application update protocols carry crash vulnerabilities their own APIs do not disclose [199]. Our architectural posture — a single writer rather than a replicated state machine — follows the single-leader default that Kleppmann argues for [134]: reach for consensus only when the problem genuinely forces it. We make that argument formal via the consensus impossibility of Fischer, Lynch, and Paterson [81]: there is no agreement to reach, so the impossibility never applies. Linearizability, the external guarantee we claim for claims and locks, is Herlihy and Wing’s [107]. Cross-subsystem atomicity, where we decline the distributed solution, is exactly the setting Garcia-Molina and Salem’s Sagas [87] were designed for — and exactly the setting where, because every step is local, a single transaction dominates a saga.²

1.15.3 Coordination, stigmergy, and deontic control

The communication subsystem draws on two coordination traditions. Stigmergic markers with decay descend from Grassé’s stigmergy [93] and its computational realizations in ant-colony optimization [65]; the shared associative store is Gelernter’s Linda tuple space [88]. The obligation lever models commitments as Cohen and Levesque’s persistent goals [50] — intentions an agent may rationally drop — and the prohibition lever uses the normative-systems framing of Jones and Sergot [124] to keep prohibition, obligation, and permission distinct modalities rather than one undifferentiated “policy.” The failure detector underlying heartbeat-based liveness is Chandra and Toueg’s [45], which is why heartbeat freshness can never be a perfectly synchronous pre-commit check. The self-attestation subsystem, which denies service

²Engineering precedent, named because a reader who builds systems will already have thought of it and should not have to wonder whether we had. The single-threaded serialization this chapter formalizes is the same shape as TigerBeetle’s accounting loop, which runs debits and credits through one deterministic state machine precisely so that conservation is a property of the loop rather than of a lock discipline; and the WAL that Property 1.4.1 types by fault class is the artifact Litestream and LiteFS replicate page by page to get durability and read replicas out of SQLite without a second writer. These are production systems rather than results, so they are cited here as practice and not as canon — nothing in this chapter’s formal spine rests on them, and the resemblance is convergent rather than derived.

when its own integrity is unknown, is in the spirit of autonomic computing’s self-management properties [130], and the bounded busy-wait under contention is the timeout-budget bulkhead that Nygard catalogs as a stability pattern [182].

1.15.4 Capability security and tamper-evident logs

The capability/permission distinction — *ability* versus *normative status* — is the object-capability discipline applied to coordination: an agent’s having a handle to an operation does not make the operation permitted. The cryptographic authorization chain is a hop-bound, attenuation-monotonic delegation chain of digital signatures over an elliptic-curve signature scheme [27], of the kind whose secrecy and integrity properties are mechanically checkable in protocol-verification tools [33]; its full treatment belongs to the economy paper. The audit log’s tamper-evidence is the digital-timestamping construction of Haber and Stornetta [99], built on Merkle’s hash trees [170] — a structure that famously predates, and is independent of, blockchains.

The kernel is novel less in any single primitive than in their *composition under one constraint*: a single local writer turns mutual exclusion, serializability, linearizability, complete mediation, and a durable audit log into consequences of one architectural choice rather than five separately engineered subsystems. The contribution is the reduction, and the discipline of saying exactly where the reduction stops paying.

KEY IDEA

1.16 Open problems

These eleven problems are the layer’s research frontier; they appear as starred exercises throughout and are collected here. OP-4 is shared with the personhood paper (this paper owns the kernel mechanism; that one owns the personhood-and-reputation argument).

One of the eleven is closed — by a theorem in this chapter, not by an artifact. Several others have a *specified design* in the body, which is a real thing to have and is not the same thing as being solved; the maturity scale (§1.1.3) exists precisely to keep those two apart. Because a “solved” claim that survives in one place after being retracted in another is the most dangerous kind of error a paper like this can make, Table 1.15 states each problem’s status once, and every exercise box, invariant row, adjacency clause, and figure caption in this chapter is written to agree with it. Where they ever disagree, this table is the one to trust.

★ OP-1 — Fair exclusion without a scheduler.

Status: *open*. Add bounded-wait to claims and locks while keeping the single-writer, no-background-scheduler simplicity. §1.5 specifies a reactive ticket-lock that would do it for cooperatively

Table 1.15: Open-problem status, stated once. **CLOSED** = settled in this chapter; **PARTIAL** = something ships but not the property as posed; *open* = not solved, whether or not a design exists. The last column grades the *design*, not the problem, on the scale of Table 1.1.

#	Problem	Status	Design in this chapter
OP-1	Fair exclusion without a scheduler	<i>open</i>	Stigmergic ticket-lock, SPECIFIED (§1.5)
OP-2	Regimentation vs. enforcement	CLOSED	Theorem 1.7.2 + Theorem 1.7.3
OP-3	Cross-runtime soundness (I11)	<i>open</i>	Differential fuzzing, SPECIFIED (§1.10)
OP-4a	Checkpoint with teeth: durable lineage replay	<i>open</i>	Event-sourced rehydration, SPECIFIED (§1.8)
OP-4b	Checkpoint with teeth: inference-state recovery	<i>open</i>	—
OP-5	Oracle completeness	<i>open</i>	State-machine assertions, SPECIFIED (§1.7)
OP-6	Tamper-evidence on the read path	<i>open</i>	—
OP-7	Schema evolution without a sequencer	PARTIAL	Ordered boot ledger PARTIAL; user_version sequencer SPECIFIED (§1.3)
OP-8	Stigmergic decay calibration	<i>open</i>	—
OP-9	Same-machine adversary	<i>open</i>	Ephemeral-UID sandbox + kernel-verified peer credential, SPECIFIED (§1.12.4)
OP-10	Per-write-path durability	<i>open</i>	Selective checkpointing, SPECIFIED (§1.4)
OP-11	Cross-subsystem atomicity by default	<i>open</i>	One local transaction (§1.11) — a buildable defect

released claims and leaves the lazy TTL-sweep path — the one that matters when an agent dies — unfair. Nothing of it ships. Is a FIFO wait-list of time-to-live'd reservations enough once the sweep is included?

★ **OP-2 — Regimentation vs. enforcement boundary.**

Status: **CLOSED** by Theorem 1.7.2: an invariant is regimentable iff it is a pure, deterministic predicate over committed local state and the incoming payload; everything else is post-commit detection under Theorem 1.7.1. Theorem 1.7.3 places that result inside Ramadge–Wonham controllability [207, 208], of which it is the commit-only instance, and a companion formal chapter carries the general theorem. The remaining work is classification, not proof.

★ **OP-3 — Cross-runtime soundness.**

Status: *open*. A seeded differential fuzz — 10^6 concurrent operations against both the test and deployment SQLite bindings, asserting hash-equivalence of the resulting state — is specified (§1.10) and does not run in the pipeline today. Even once it does it is a test, not a proof: name the divergence class it would miss.

★ **OP-4 — Checkpoint with teeth.**

Status: *open*, and still the most important unbuilt thing at this layer. It is two problems wearing one number, and they have different difficulties, different owners and — this is the part that matters for anyone reading the table to decide what to build — different odds of ever being closed, so the table names them separately. **OP-4a, durable lineage replay:** *open*. Event-sourced rehydration (§1.8, SPECIFIED) would replay a pinned commit and a truncated message log into a successor, which restores the *durable inputs* to an agent's context. The design is specified and nothing ships it, which is what SPECIFIED means and why this half is not **CLOSED** however tractable it looks. **OP-4b, inference-state recovery:** *open*, with no design in this chapter. A model's inference state is bound to a particular model, sampler and session and is not a portable artifact; replaying the inputs reconstructs what was fed to that state, never the state. Whether the reconstruction is close enough to count as restoring work, and against what test, is the question, and no formal result in this series establishes it. Closing 4a would not close 4b, and a reader who sees event-sourced rehydration ship should not read it as OP-4 discharged.

★ **OP-5 — Oracle completeness.**

Status: *open*. State-machine assertions (§1.7, SPECIFIED) would add delta oracles and AST assertions to Listing 1.2's four shipped kinds. Two kinds are not a completeness result, and neither kind yet survives the Goodhart argument the vocabulary exists to win.

★ **OP-6 — Tamper-evidence on the read path.**

Status: *open*. Where should hash-chain verification gate? A sampling/checkpoint regime with a provable detection-probability bound (couples to §1.12.4).

★ **OP-7 — Schema evolution without a sequencer.**

Status: *PARTIAL*. An ordered boot ledger with post-apply schema verification ships (§1.3); the *user_version* sequencer over declared migrations does not, and modules still self-initialize. Can any useful remnant of “any module, any order” survive safe re-names, backfills, and down-migrations?

★ **OP-8 — Stigmergic decay calibration.**

Status: *open*. The right per-kind decay so coordination markers neither rot nor evaporate before they coordinate.

★ **OP-9 — Same-machine adversary.**

Status: *open*. §1.12.4 specifies the hardening — spawns in *bwrap/unshare* sandboxes under ephemeral UIDs, plus the kernel-verified peer credential (*SO_PEERCREDS* / *LOCAL_PEERCREDS*) read on each connection — and none of it ships. Today’s peer-credential check is a software handshake over an owner-only socket, exactly as Table 1.14 states. Even built, the hardening would move the adversary from the same-uid row to the different-uid row, not out of the model. State the dependency the other way round as well, because it is the sentence an operator needs: OP-9 is the *precondition* for any *CONFINED* claim on a machine that more than one principal can log into. Until the sandbox and the kernel-verified peer credential ship, a confinement claim on a shared host is a claim about a cooperating co-tenant, and the honest grade for it is *OBSERVED* — which is not a hedge, it is the difference between a boundary the kernel enforces and one the other tenant is choosing to respect.

★ **OP-10 — Per-write-path durability.**

Status: *open*. Selective checkpointing (§1.4, *SPECIFIED*) would let a settlement-ledger write force *wal_checkpoint(FULL)* on its commit path; no path opts in today, and *I1b* remains *not guaranteed* kernel-wide per Property 1.4.1. The harder half is the interface that stops a new write path from defaulting into the fast lane silently.

★ **OP-11 — Cross-subsystem atomicity by default.**

Status: *open* but cheap — a buildable defect, not a frontier. Wrap multi-subsystem operations in one local transaction at the interface boundary, eliminating the undefined partial-failure semantics of §1.11.

1.17 Conclusion: solid where local, provisional where it must grow

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database, and its two jobs — durable record and mediated exclusion — are real and, mostly, proven. It earns its strongest claims (mutual exclusion, serializable/linearizable consistency, oracle-bound closure) precisely by refusing the distributed-systems problem it superficially resembles: one writer, one file, one decider, no consensus.

Where it stops, it stops formally. Durability is split: process-crash durable (I1a), *not* guaranteed against power loss (I1b). The policy monitor detects and compensates; it does not regiment, and we credit the only genuine pre-commit regimentation to the uniqueness constraint and the boot-admission gate. Cross-subsystem atomicity is a buildable defect, not a frontier. Hash-chain tamper-evidence is re-scoped to cross-machine provisioning, where it actually defends someone. And the two genuinely soft spots — partial local identity enforcement (I12) and a checkpoint with teeth (OP-4) — are exactly the two things a cross-machine economy and a durable notion of agent personhood lean on hardest.

The kernel is solid where it is local and provisional exactly where a cross-machine economy would need it to be cryptographic and continuous. That is not a flaw to hide; it is the layer boundary, stated truthfully.

THESIS

Every “single-machine / one-writer / one-clock / self-asserted-identity” simplification that makes this layer clean is precisely the assumption a later layer must pay to relax. Cross-machine capability transfer over the cryptographic authorization chain this kernel ships is where that payment begins, and it belongs to the economy layer, not here. Build the local floor first. The cryptography earns its keep when agents must trust one another across machines they do not control.

Chapter Appendices

1.A Implementation & status

The body of this paper argues at the level of mechanisms, so that it reads as a self-contained systems paper. This appendix records the concrete grounding: the specific artifact that realizes each mechanism in the reference implementation, and an honest accounting of how mature each is. Nothing here is needed to follow the argument; it is here so the maturity grades in the body are auditable rather than asserted.

1.A.1 The reference implementation

The reference implementation is a single always-on local daemon written in TypeScript, run under a compiled JavaScript runtime, persisting to one SQLite database file in WAL mode. Clients — a command-line tool, agents over a tool-protocol bridge, and a binary inter-process transport — reach it over Unix domain sockets. The daemon is kept alive by the host operating system’s service supervisor. The seven subsystems of §1.2 correspond to distinct modules that each self-initialize their own tables over the shared database handle, in the factory style `createModule(db)`.

1.A.2 Mechanism-to-artifact map

Table 1.16 maps each mechanism discussed in the body to the module that realizes it. The storage configuration referenced throughout is the WAL pragma set: `journal_mode=WAL, synchronous=NORMAL, wal_autocheckpoint=200, busy_timeout=5000, foreign_keys=ON`, with a clean-shutdown `wal_checkpoint(TRUNCATE)`.

1.A.3 Status, and the corrections that matter

The maturity grades from Table 1.12 reflect the state of the reference implementation at the time of writing. Four corrections this paper makes to earlier, more optimistic internal descriptions are substantive and worth restating with their evidence:

1. **Durability is split by fault class** (§1.4). An internal code comment justifying the `synchronous=NORMAL` choice asserted that the normal level “is safe in WAL mode because WAL already guarantees crash safety.” That conflates crash-consistency with durability; under power loss the last writes can roll back. The honest statement is the I1a/I1b split.
2. **The policy monitor is detect-and-compensate, not regimentation** (§1.7, Theorem 1.7.1). The monitor is implemented as a subscriber

Table 1.16: Mechanism-to-artifact map for the reference implementation. Module names are illustrative of the organization, not a stable public interface.

Mechanism (body)	Realizing module(s)	Notes
Single write connection / pragmas	database-handle module	opens and configures SQLite
Port & lock claims	service-registry, locks, session-files modules	three claim granularities
Message bus / tuple space / markers	bus, tuples, marker modules	at-least-once delivery
Policy monitor	monitor module	post-commit log subscriber
Commitments & obligation	commitment, obligation-monitor modules	two of the hardening laws shipped
Memory & recovery	session, episodic-memory, recovery modules	recovery passes notes only
Audit log & hash chain	activity-log, hash-chain modules	verification not yet on the read path
Self-attestation	attestation, invariant-registry modules	boot gate + pre-flight + watchdog
Authorization chain	delegation-chain module	mechanically verified
Enforcement gate (macaroon discharge)	kernel macaroon module	crypto verified; in-band, not yet compulsory (§1.12.2)
Runtime-parity shim	runtime-shim module	normalizes binding differences
Identity (actor souls + credentials)	actor-souls, budget-guard modules	bounded gate ships; full write-boundary enforcement absent

to the already-committed operation log, so it cannot prevent the bad prefix that triggers it; “physically unreachable” is reserved for the uniqueness-constraint and boot-gate states.

3. **Cross-subsystem atomicity is a buildable defect, not a research frontier** (§1.11). The database transaction primitive is already used elsewhere in the implementation; the only missing work is wrapping the multi-subsystem endpoints in it.
4. **A specified design is not a solved problem** (Table 1.15). An earlier revision of this chapter reported several open problems as closed — fair exclusion, runtime parity, checkpointing, oracle completeness, the same-machine adversary, per-write-path durability — on the strength of designs that no artifact realizes, while the exercise boxes, invariant rows, figure captions, and threat model those claims contradicted were left untouched. The designs are kept and graded SPECIFIED; the problems are reported open. The security-relevant instance is worth naming on its own: the socket peer-credential check is a software handshake and *not* the kernel-verified SO_PEERCREC credential, in this chapter’s threat model (Table 1.14), in §1.6, and in §1.12.4 alike.

1.A.4 Nomenclature

For the reader cross-referencing the accompanying chapters, the body’s neutral names map to the series’ internal terms as follows: the *substrate* / *coordination* / *legibility* / *economy* layers are the series’ L0–L3; the *policy monitor* is the Arbiter; *stigmergic markers* are pheromones; the *bus* is tube; the *authorization chain* and *coordination lineage* are the two objects the series keeps distinct under the single phrase “delegation chain.” The maturity grades — implemented / partial / specified / proposed — are the series’ honest labels.

WHAT THE ANCHOR PROTOCOL RECEIVES Local effects now have one serial history, one fencing epoch, and one place to attach receipts. The writer decides what becomes real; it does not yet say who may ask it to. The next chapter turns authority into a key-bound, attenuating, revocable object, so that every request the kernel admits arrives with a checked pedigree.

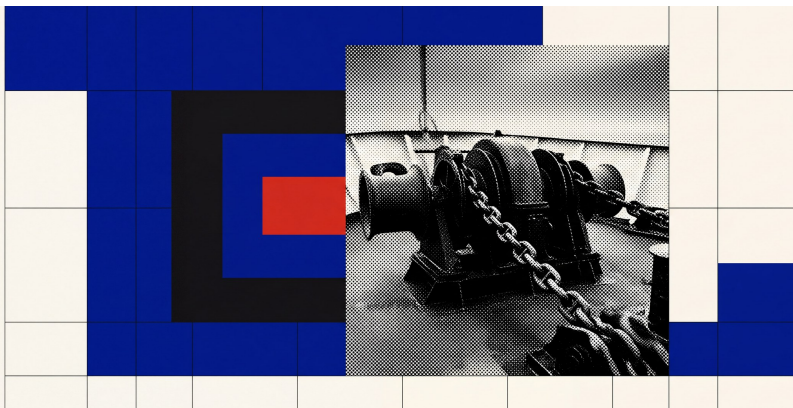
2

The Anchor Protocol

The question this chapter answers

How may authority travel without silently growing?

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.



Every program and every user of the system should operate using the least set of privileges necessary to complete the job.

Jerome Saltzer and Michael Schroeder, *The Protection of Information in Computer Systems*,

1975

HOW MAY AUTHORITY TRAVEL WITHOUT SILENTLY GROWING? Identity answers who signed; it does not answer what the signer may do. Anchor therefore checks every delegation edge, binds the grant to a key and audience, narrows scope and lifetime, and preserves revocation ancestry. The formal claim concerns the checked protocol and Rust paths, while R5 explains which surrounding policy promises can become admission gates.

2.1 What problem this solves

The scene. It is 2 a.m. and your laptop is running four coding-agent sessions at once: one refactoring the auth module, one running the test suite against a local Postgres, one building a preview deployment, and one you started an hour ago and forgot about. All four are agents. All four can read your files, bind ports, and run shell commands with your full user permissions — and none of them can tell, from the outside, which of the other three is behaving. Nothing on the machine distinguishes the forgotten session from a process that has quietly replaced it.

That is the local swarm problem. In a multi-agent development environment, agents operate concurrently to read files, spin up test servers, and execute commands, and this introduces three concrete failure modes on `localhost`:

1. **Port Squatting (The “Ghost in the Harbor”):** If Agent A crashes, a malicious or malfunctioning Agent B can bind to Agent A’s previously assigned port, intercepting traffic meant for A.
2. **Resource Contention:** Agents fighting over shared resources (e.g., a SQLite database file) without a centralized locking mechanism cause state corruption.
3. **Privilege Escalation:** A “Reviewer” agent delegated by a “Coder” agent should not possess the rights to initiate production deployments, yet local environments rarely enforce principle-of-least-privilege between processes.

The idea in one breath. *Every agent on the machine carries a short-lived, cryptographically signed card that names exactly what it may do; anything it hands to a child is strictly smaller; and any card can be withdrawn before it expires.* Everything that follows is that sentence made precise, then mechanically checked.

To make it enforceable, Port Daddy introduces the concept of a **Harbor**—a zero-trust, namespace-isolated execution environment where agents must prove their identity and capabilities. Our approach builds upon foundational work in capability-based security by Dennis and

Van Horn [62] and the principle of least privilege articulated by Saltzer and Schroeder [224]. This protocol sits in a specific lineage — Macaroons for delegation, ProVerif and Tamarin for symbolic verification, the JWT algorithm-confusion literature for the Phase 1 hardening, and the emerging agentic-commerce credential profiles for the envelope — traced in full in Appendix 2.F, which a reader following the argument rather than the citations can skip entirely.

How to read this chapter. §2.2 is the protocol: four phases, each closing a hole the previous one left open. §2.3 is the evidence: three verification layers, what each one proves, and what it hands the next. §2.5 is the accounting: the adversary we assume, the properties that survive it, and where the guarantees stop. Table 2.2 is the map for §2.3; Table 2.4 closes the loop on the three failure modes just listed.

2.2 The Anchor Protocol Architecture

The Anchor Protocol defines how agents authenticate to a Harbor and to each other. It issues **Harbor Cards** (highly constrained JSON Web Tokens) and evolves them across four phases, each adding a capability that closes a specific attack surface from the previous phase (Figure 2.2; Table 2.1 tabulates the same four rows as a lookup reference).

Figure 2.2 draws the same table as a staircase: each threat’s row turns from red (open) to the chapter hue (closed) at the phase that closes it, and the closure frontier moves strictly rightward, so no later phase reopens a threat an earlier one closed.

A common thread across all phases is *capability attenuation*: a delegated token never carries more authority than its parent, and TTL only shrinks. Temporal attenuation is the same rule applied to the clock: a child’s expiry is the earlier of its parent’s expiry and the lifetime requested, so a twenty-minute child requested under a parent with ten minutes left is refused, and the acceptable narrower card is the one that inherits the parent’s ten; in the capability lattice, lifetime is one more coordinate that moves down or stays. Figure 2.3 shows the nested-cap structure that the verifier enforces on every chain.

Numbers by hand — a child card under a parent with ten minutes left. A parent card holds `fs:read` and `db:write` with 10 minutes of lifetime remaining. A child requested with `db:write` for 20 minutes is refused: its rights are a subset, but its expiry would be later than the parent’s. The same child requested for 8 minutes is issued with an 8-minute expiry; a child requested for 20 minutes *under the parent’s clock* is issued with the parent’s 10. In the lattice both coordinates moved down or stayed: $\text{rights } \{\text{db:write}\} \subseteq \{\text{fs:read}, \text{db:write}\}$ and life-

Table 2.1: The four phases as a lookup table, restating Figure 2.2’s flow in row form. Threat closure is cumulative, not substitutive. Each phase closes one attack surface, and the last column is why no later phase reopens it: every phase adds a check that a later phase changes the key or the chain of, never the party that performs it. [internal]

Phase	Mechanism	Attack surface closed	Stays closed because
1	HS256 pinning: the verifier takes the algorithm from policy	header-selected algorithm (algorithm confusion)	the algorithm is never read from the token; later phases change the key type, not who chooses
2	Ed25519 identity: the daemon signs with a private key, verifiers hold only the public key	shared-secret theft	no verifier holds a secret that could mint a card; a stolen public key mints nothing
3	delegation: a child card carries its parent chain, checked as a subset at every hop	downstream privilege growth	the subset check is per hop and monotone, so no later hop widens scope or lifetime
4	revocation: epoch filters gossiped between daemons and consulted at every acceptance	post-issue authority (a leaked card that should die before its TTL)	acceptance requires the current epoch, so every daemon that holds the epoch refuses the revoked card

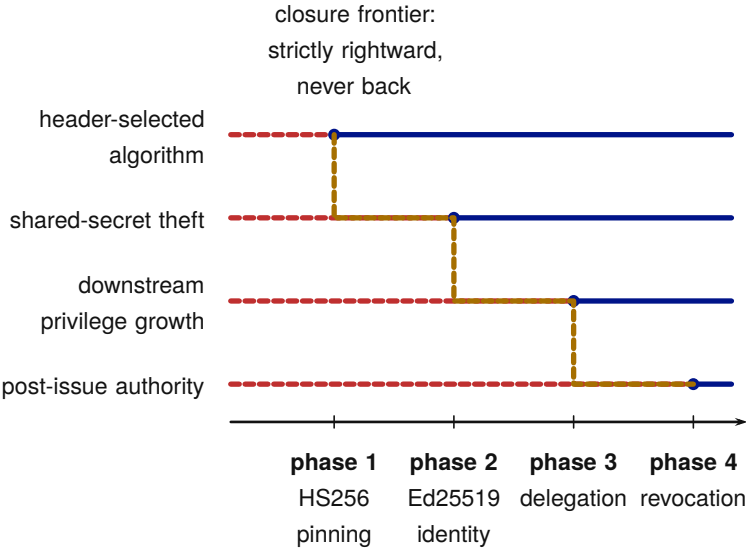


Figure 2.1: Threat closure across the four protocol phases, restating Table 2.1 as one staircase. Each row is a threat; red is still open, the chapter hue is closed, and the datum marks the phase that closes it. The staircase itself is the claim a table cannot draw: closure moves strictly rightward, so no later phase reopens an earlier one – each adds a check on the key or the chain, never removes one. [internal]

time $\min(20, 10) = 10$. Figure 2.3 draws the same check with the numbers 60, 15 and 5. [internal]

2.2.1 Phase 1: Symmetric Pinning

Early versions of the protocol relied on HS256 (HMAC-SHA256). The primary vulnerability in JWTs is “Algorithm Confusion” (CVE-2015-9235 [54], CVE-2023-48238 [56]), where an attacker modifies the token header to `{“alg”: “none”}` or symmetric HS256 while using an asymmetric public key as the HMAC secret.

DESIGN INVARIANT 2.2.1 Algorithmic pinning. The Port Daddy Verifier employs strict **Algorithmic Pinning**. It entirely ignores the user-provided `alg` header and explicitly forces the underlying crypto library to evaluate the signature using the pre-determined algorithm. This approach is what the JWT Best Current Practices, RFC 8725, recommends [234].

Figure 2.4 shows the two verifier paths side by side: the naive

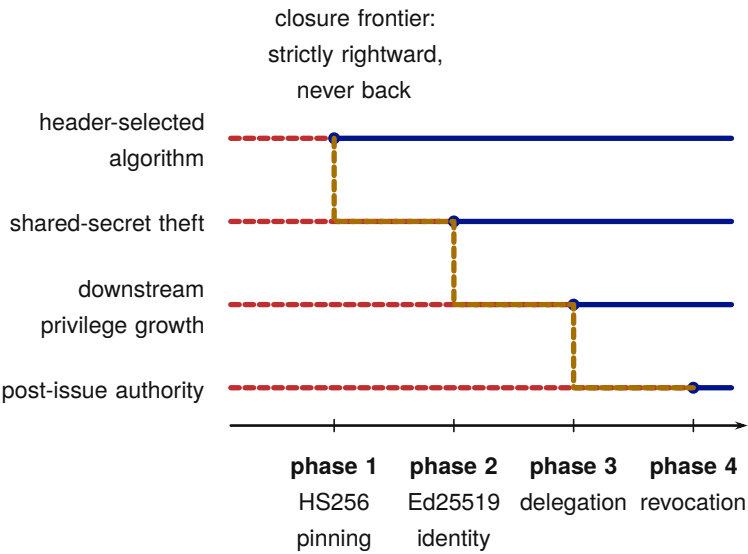


Figure 2.2: Threat closure across the four protocol phases, restating Table 2.1 as one staircase. Each row is a threat; red is still open, the chapter hue is closed, and the datum marks the phase that closes it. The staircase itself is the claim a table cannot draw: closure moves strictly rightward, so no later phase reopens an earlier one – each adds a check on the key or the chain, never removes one. [internal]

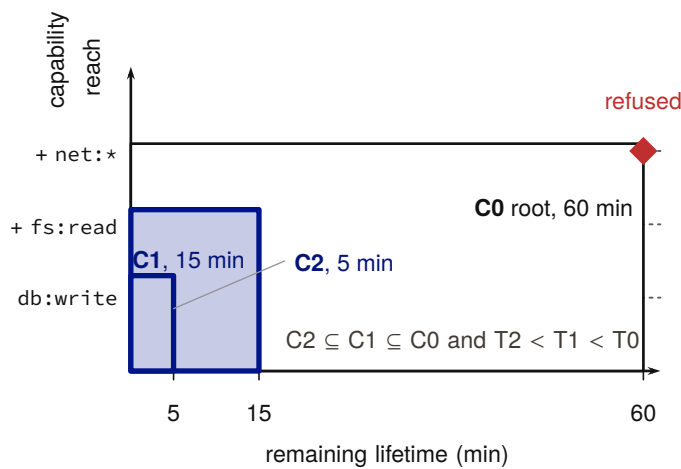


Figure 2.3: Nested rectangles for the capability envelope C0 (root, 60 min), C1 (15 min) and C2 (5 min): each child rectangle sits strictly inside its parent on both axes. The red mark sits directly above C0’s own net:* row, at the full 60-minute root lifetime: it is a re-grant attempt that asks for net:* at that full lifetime rather than a narrower child envelope, and the verifier refuses it because attenuation is conjunctive on rights and time together, not a trade between them. [internal]

verifier trusts the `alg` field on the wire (and is exploited), while the pinned verifier records the issuance algorithm out-of-band and admits no rewrite trace.

2.2.2 Phase 2: Asymmetric Identity (Ed25519)

To support decentralized Agent-to-Agent (A2A) communication without sharing HMAC secrets, the protocol transitioned to **Ed25519 (EdDSA)** [27, 125]. Ed25519 provides fast, deterministic signatures with small keys and signatures, making it ideal for local agent communication.

Definition 2.2.1 (Harbor Key Hierarchy). The Port Daddy Daemon acts as the Root Certificate Authority (CA). Each Harbor is assigned a unique Ed25519 keypair. In Phase 2 the Daemon issues Harbor Cards signed by the Harbor’s private key `sk_harbor`, escrowed to it over a private channel; agents use the matching `pk_harbor` (broadcast via Port Daddy’s ambient discovery service) to verify tokens presented by peers.

One key changes hands at the Phase 3 boundary, and the chapter should say so rather than leave it to be inferred. From Phase 3 on, the root of a delegation chain is the Daemon’s *own* signing key `sk_daemon`, and every verifier checks `chain[0]` under `pk_daemon` — which is what Algorithm 2.4 requires, what `analyses/harbor_card_v3`, `v5`, `v6` and `v7` model, and what `core/harbor-card-rs` implements. The escrowed per-Harbor keypair of this definition is the earlier, Phase-2 arrangement and does not root delegation chains.

2.2.3 Phase 3: Stigmergic Delegation (The “Biscuit” Pattern)

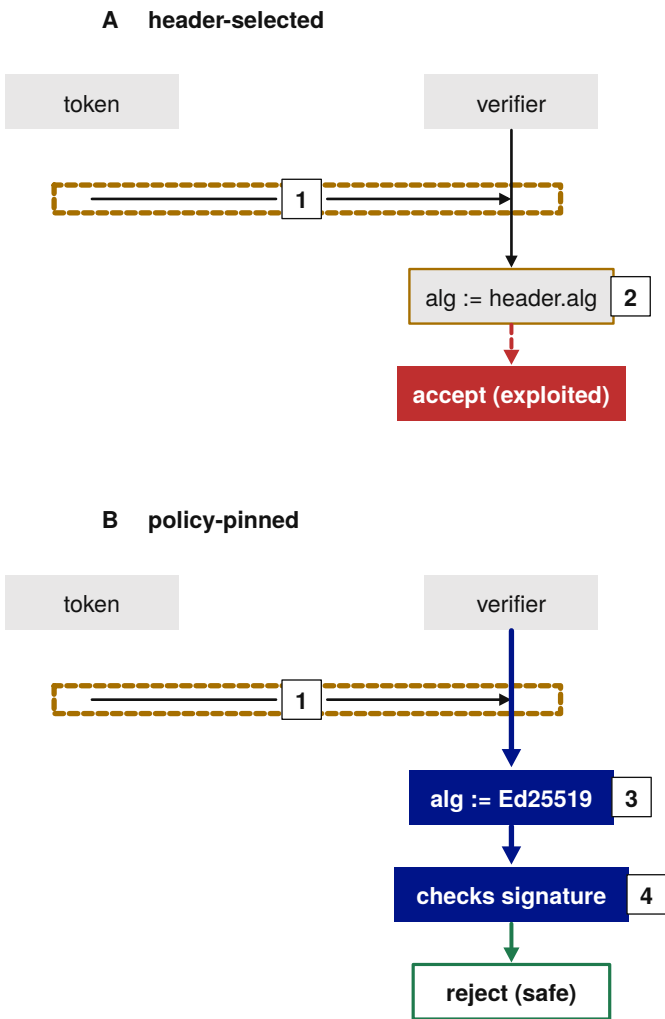
Port Daddy v4 introduces multi-hop delegation. When Agent A spawns Agent B, it does not need to request a new token from the Daemon. Instead, it uses **Offline Attenuation**, inspired by Macaroons [30].

Definition 2.2.2 (Offline Attenuation). Agent A takes its existing Harbor Card, appends a new set of restricted capabilities (e.g., [`db:read`] reduced from [`db:read`, `db:write`]), and signs the *entire chain* with its own ephemeral private key.

The Harbor verifies the Daemon’s root signature, Agent A’s delegation signature, and mathematically ensures that Agent B’s capabilities are a strict subset of Agent A’s.

Figure 2.5 puts that ceremony on a clock: who signs what, in what order, and — the point of “offline” — which participant stops being spoken to after the first message.

For depth-*N* chains, the verifier walks the structure shown in Figure 2.6. Each hop must produce a fresh nonce and bind the previous



step predicate

- 1

token sends alg=none with a tampered payload (shared)
- 2

naive verifier reads alg from the token header
- 3

pinned verifier reads alg from issuer policy
- 4

pinned verifier checks the tampered signature

Figure 2.4: Two verifier ladders for the same wire bytes: **token** sends `alg=none` with a tampered payload to **verifier** in both panels (step 1, inside the dashed network band the Dolev-Yao adversary controls). Panel A lets the token’s own header choose the algorithm (step 2) and accepts the forgery; panel B pins the algorithm to `Ed25519` from issuer policy (step 3), so the signature check (step 4) fails and the token is rejected. This is the design invariant behind the ProVerif (H9256) query `the attacker cannot derive master_key`: the accepting path in panel A is exactly the path Algorithmic Pinning removes [verified, v1_refined.pv].

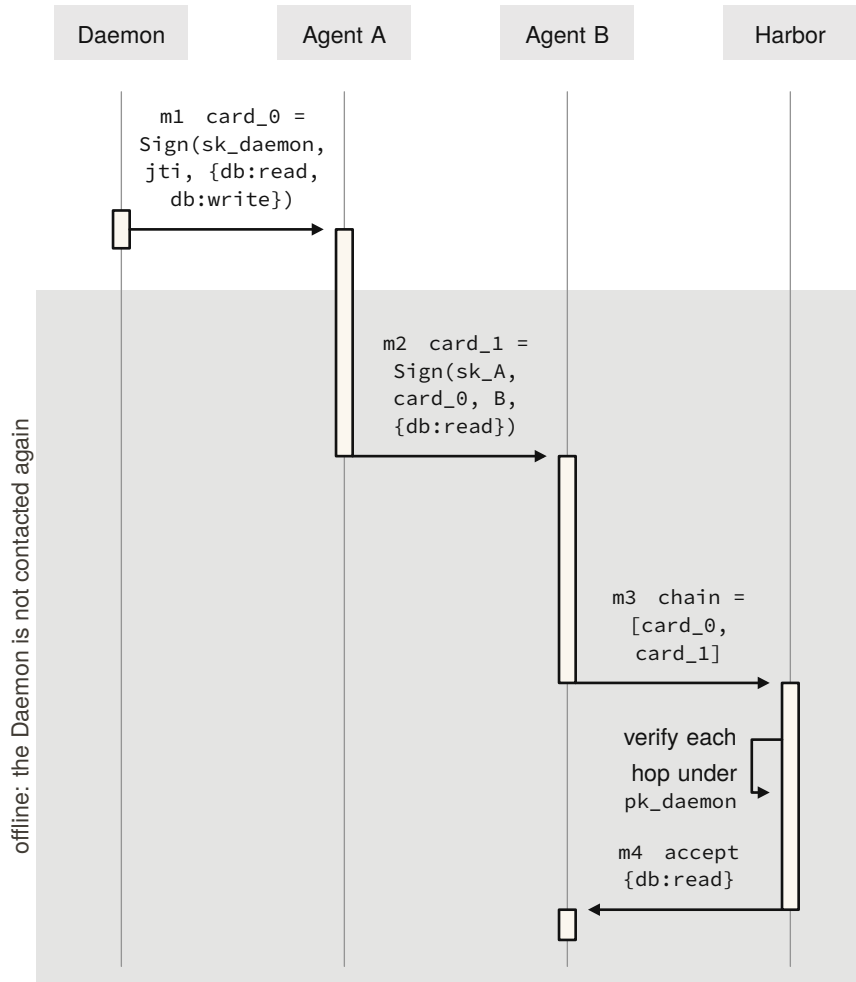


Figure 2.5: The issue–delegate–verify ceremony as a sequence diagram. Every message after m1 falls inside the shaded region, where the Daemon is not addressed: agent A mints card_1 from card_0 with its own key, and the Harbor accepts the chain under pk_daemon with no round trip to the issuer. [internal]

and next identities into its signature so that an attacker who intercepts a single hop cannot splice it into a different chain.

Macaroons are the inspiration, but two older literatures own the property being checked here, and both deserve the credit before the nickname does. SPKI/SDSI's certificate-chain reduction is a calculus for asking exactly this question — does the chain only ever narrow as it lengthens? — and answering it by reducing the chain to one equivalent certificate [73, 215]; the per-hop subset check the Harbor performs is that reduction carried out one link at a time. And Miller's object-capability discipline — no ambient authority, a capability as the only route to any authority at all, worked out for concurrent and mutually suspicious distributed components [175] — is the conceptual peer of this protocol's capability/permission split, closer than Dennis and Van Horn's founding 1966 statement, and it comes with a design rule this chapter learned the expensive way: never hand out an unattenuated capability in the first place, so that the chain check is defense in depth rather than the only line. (The v6 verifier of §2.D.1, which checked each hop against the root instead of its parent, is what happens when the check is the only line.)

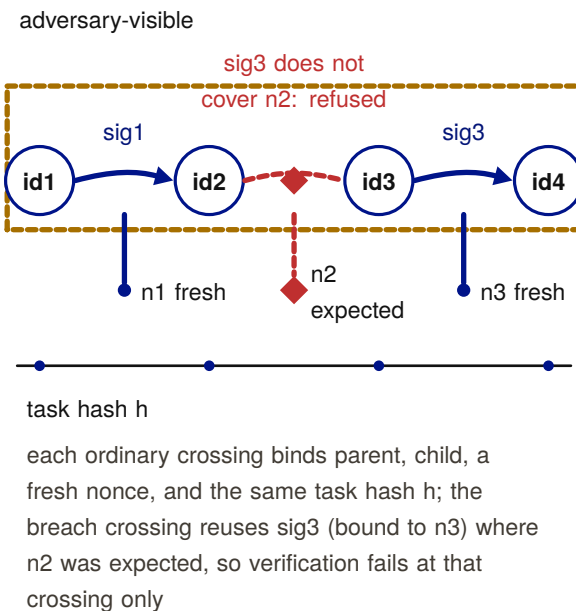


Figure 2.6: The delegation chain as a signed provenance braid across four identities id1–id4, drawn inside the network the Dolev-Yao adversary observes. Ordinary hops 1 and 3 bind parent, child, a fresh nonce (n1, n3), and the shared task hash h below. Hop 2 shows a splice attack: an attacker moves the legitimate signature sig3 (computed over id3, id4, and n3) into the id2–id3 position, where the verifier expects a signature over n2; the check fails at exactly that crossing, so the discontinuity is visible at one adjacent-identity boundary rather than anywhere else in the chain. [internal]

2.2.4 Withdrawing a card before it expires

Capability tokens have a TTL, but natural expiry is not always sufficient. Two circumstances force early withdrawal: (i) *compromise*—a Harbor Card leaks from its agent’s process, and every second until TTL is a second the attacker has legitimate capabilities; and (ii) *policy reversal*—a principal decides an agent should no longer hold `db:write`, and waiting for TTL is unacceptable. A naive revocation list is $O(n)$ per verification and grows monotonically. While the Anchor Protocol previously attempted to gossip cuckoo filters directly between daemons — merging two peers’ filters by OR-ing their bit arrays — it now gossips an **Append-Only Event Log of Revocation IDs** using Vector Clocks. To maintain $O(1)$ read performance, each daemon continually projects its event log into a local, ephemeral cuckoo filter [80] (Figure 2.7).

Why filters cannot be merged bitwise. The abandoned design failed for a structural reason worth stating plainly, because it is the reason the log exists. Two cuckoo filters cannot be safely combined by OR-ing their bit arrays. A fingerprint’s final resting bucket is not a function of the key alone: it is the first candidate bucket i_1 if that bucket had a free slot at insertion time, and otherwise the alternate i_2 reached by evicting some other fingerprint — so occupancy depends on insertion order and eviction history, not just on the set of keys. Peer *A* may hold fingerprint *c1* in bucket i_0 while peer *B*, having inserted a different key first, holds the same *c1* in bucket i_3 . The bitwise union contains both placements, which no single consistent filter state could have produced: subsequent deletions then remove the wrong occupancy and can *reanimate* a revoked card — a false negative, the one error a revocation gate must never make. Gossiping the append-only log of `kids` and rebuilding each daemon’s filter locally sidesteps the problem entirely, because set union over identifiers is well defined where union over their lossy encodings is not.

Cuckoo filters in one paragraph. A cuckoo filter is a compact probabilistic set: under successful insertion and correct deletion discipline it has false positives but no false negatives, while supporting deletion. Each inserted key is reduced to a fingerprint and placed in one of two candidate buckets; on collision, fingerprints are relocated until a slot opens or insertion fails. Lookups inspect both buckets. The expected lookup cost is $O(1)$ and the approximate false-positive probability for two buckets of size B and f fingerprint bits is at most about $2B/2^f$ in the low-collision model. High load increases insertion failures; the implementation rejects or rebuilds rather than silently dropping a revocation. Duplicate fingerprints require counted or authoritative-table-backed deletion so that removing one key does not reanimate another.

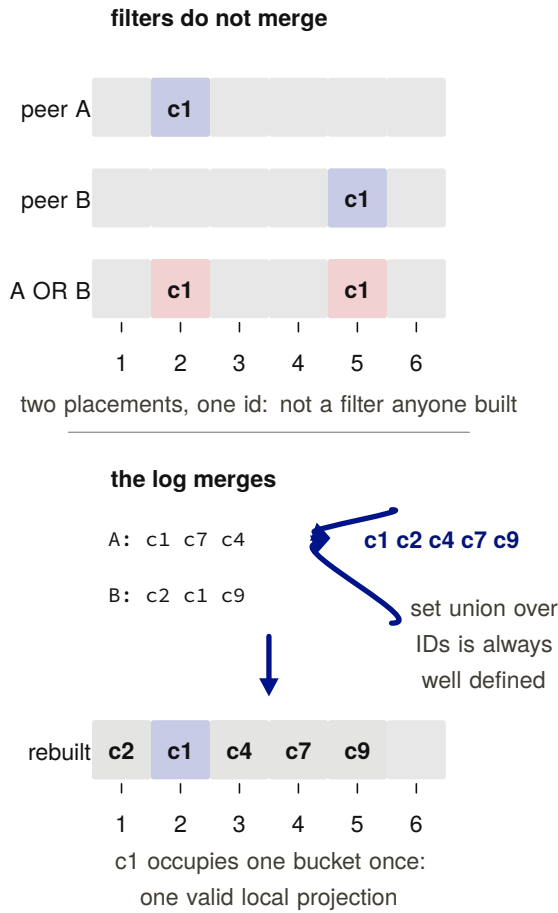


Figure 2.7: Two six-bucket cuckoo filters compared before and after the design change. Top: peer *A* inserts *c1* into bucket 2 and peer *B* inserts the same *c1* into bucket 5, an artifact of each peer’s own insertion and eviction history; every other bucket is empty, and bitwise OR keeps both *c1* placements, a state no single filter build could have produced. Bottom: gossiping the append-only ID log instead—peer *A*’s *c1 c7 c4* and peer *B*’s *c2 c1 c9*—and taking the ordinary set union gives *c1 c2 c4 c7 c9*; each daemon rebuilds one local filter with *c1* in a single bucket, the rest in gold as ordinary occupants. [internal]

Why cuckoo, not Bloom. For uniform-TTL workloads, a rotating Bloom filter would suffice. Port Daddy’s TTLs are heterogeneous—Shipwright proposals run hours to days, `pd spawn` children run minutes, one-shot agents run seconds. Rotating Bloom either rotates at the longest TTL (short-TTL revocations linger orders of magnitude longer than needed) or buckets by TTL class (reintroducing the structural complexity cuckoo collapses). Cuckoo provides $O(1)$ per-entry removal at each specific expiry, plus the same removal handles operator-initiated early-revocation undo cleanly.

Space. At false-positive rate p_{fp} , a cuckoo filter uses approximately $\log_2(1/p_{fp}) + 3$ bits per entry. For $p_{fp} = 10^{-3}$, ≈ 13 bits per revoked card. A fleet retaining 10^5 active revocations fits in ≈ 160 KB—trivially in memory, trivially gossiped.

Numbers by hand — sizing the revocation filter. At $p_{fp} = 10^{-3}$ the per-entry cost is $\log_2 1000 + 3 = 9.97 + 3 \approx 13$ bits. A fleet holding 10^5 live revocations therefore carries 1.3×10^6 bits = 162,500 bytes, about 160 KB, and a gossip round that ships the whole filter rather than a delta moves less than a small image. At $p_{fp} = 10^{-6}$ the same fleet needs $\log_2 10^6 + 3 \approx 23$ bits per entry, ≈ 288 KB: three orders of magnitude fewer false positives for less than twice the space, which is why the filter’s bound, not its size, is what the load ceiling protects. [internal]

Definition 2.2.3 (Revocation-Augmented Verification). Let R_t denote the authoritative Append-Only Event Log of revoked Harbor Card IDs at time t . Each daemon d maintains a local, ephemeral cuckoo filter index $F_d \approx R_t$. Every Harbor Card verification additionally:

1. Looks up the card’s `kid` in F_d . If absent, admit.
2. If present, query the local revocations table (authoritative). If genuinely revoked, reject with `revoked`; otherwise the filter is in the false-positive regime and the caller is admitted, with an asynchronous reconciliation job repopulating the filter.

Gossip-based synchronization. Daemons in a mesh synchronize revocation deltas by anti-entropy gossip [60]. Under a connected complete-overlay model with independent uniform peer selection and reliable rounds, epidemic dissemination is expected $\Theta(\log m)$ rounds. This is an expectation, not a deadline; partitions, message loss, topology, and scheduling change the constant or prevent global convergence until the partition heals. Security-critical revocations can additionally trigger best-effort immediate push. Figure 2.8 illustrates propagation across five daemons; it is a trace, not a timing proof.

What retires an entry. An append-only log that never drops anything has a size that grows with the fleet’s whole history rather than with its

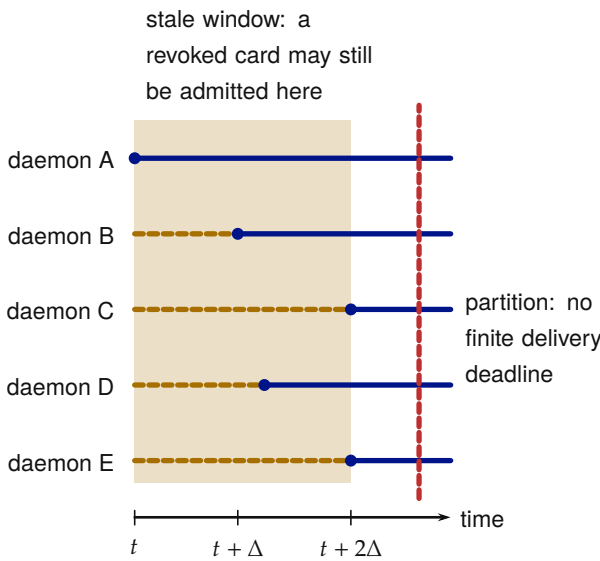


Figure 2.8: A revocation advances as an observation frontier, leaving a distinct stale-exposure interval on every verifier lane: daemon A's append is authoritative at t , and each lane's amber segment (still stale) turns to the chapter hue at its own wake datum. This execution converges after two rounds; the protocol claim is only expected $\Theta(\log m)$ dissemination under connected, reliable rounds. The dashed red line marks a second, separate claim: an unbounded partition removes every finite delivery deadline, regardless of how many rounds have passed. [internal]

live population, and on a constrained host that is a storage bug waiting for a slow month to expose it. The log does not have to grow that way, and the reason is temporal attenuation: a revocation identifier exists to stop a card being honoured, and a card that has expired is already refused by the expiry check before the filter is ever consulted, so the entry has no remaining work to do.

DESIGN INVARIANT 2.2.2 Revocation-log truncation. Let $T_{\max}$ be the longest lifetime any unexpired card in the fleet can still carry. An entry naming card kid may be dropped from the authoritative log R_t once every card that entry could name has expired — that is, once t exceeds the entry's recorded card expiry — and correctness is unaffected: for every *live* card, the filter rebuilt from the truncated log returns the same verdict as the filter rebuilt from the untruncated one. Because attenuation only ever shrinks a lifetime (§2.2), no descendant of a revoked card can outlive the entry that names it, so a dropped entry cannot be needed again by a chain derived from it. The log's steady-state size is therefore bounded by the revocations issued in the trailing window $T_{\max}$, not by the fleet's lifetime history.

This rule is specified here and the daemon does not yet apply it, which is the distinction the rest of this series marks between a design and a shipped mechanism, and it is the honest grade for this one. What would verify it is a property test rather than an argument — generate a revocation log and a set of cards, truncate at an arbitrary cut, rebuild both filters, and assert the two agree on every card that is still live at the cut. A test that only checked total agreement would pass vacuously on an empty live set, so the live set is what the generator has to make non-trivial. Note the one thing the invariant does not buy: it bounds the log by a *time* window, so a fleet that issues revocations faster than they expire still grows, and the load ceiling below, not this rule, is what holds that case.

Pollution as denial of revocation. The load ceiling keeps the false-positive bound honest; it does not by itself defeat an attacker who submits many authenticated-looking revocation identifiers to push the filter out of its bound-holding regime, which turns the revocation path into a denial of service. The design requires three things of the daemon: a per-issuer insertion quota, an overflow record for every refused insertion, and lookups that consult the authoritative event log while a flooded filter rebuilds, so that flooding degrades revocation checks to slower, never to missed.

DESIGN INVARIANT 2.2.3 Filter monotonicity over a TTL epoch. For

any Harbor Card c with issue time t_0 and TTL τ , if $c \in R_t$ for some $t \in [t_0, t_0 + \tau]$, then $c \in R_{t'}$ for all $t' \in [t, t_0 + \tau]$. After $t_0 + \tau$, c may be removed from R since the card has expired.

DESIGN INVARIANT 2.2.4 No partial reanimation. A revoked card cannot be un-revoked within its TTL. If an operator decides the revocation was erroneous, they must issue a new card.

THEOREM 2.2.5 Conditional gossip expectation. Under the complete-overlay, uniform-peer, reliable-round model above, a revocation disseminates to all m daemons in expected $\Theta(\Delta \log m)$ time. No finite worst-case freshness bound holds under an unbounded partition.

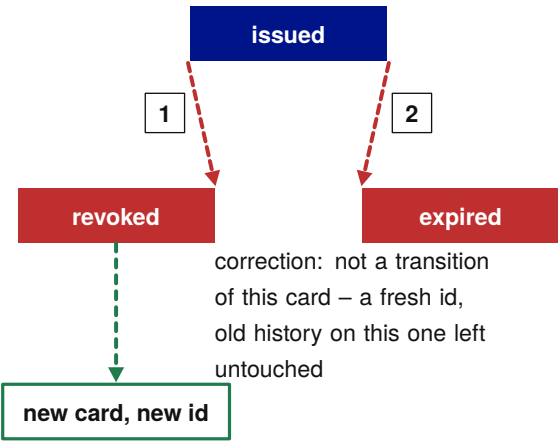
Numbers by hand — how many rounds until every daemon knows. Five daemons, one revocation, a round time Δ of two seconds. Epidemic dissemination reaches all m peers in expected $\Theta(\log m)$ rounds; with $\log_2 5 = 2.32$ the expectation is between two and three rounds, so between four and six seconds, which is what Figure 2.8 draws as the interval from t to $t + 2\Delta$. Double the fleet to ten daemons and $\log_2 10 = 3.32$ adds one round, not five. Cut one daemon off from the others and no number of rounds reaches it: the expectation is conditional on the overlay staying connected, which is the sentence the theorem ends with. [internal]

Soundness preservation. Revocation strictly narrows acceptance, so existing ProVerif soundness proofs (Property 2.3.1) are unaffected. The new proof obligation is small: revocation is *enforced*, which follows by inspection of the verification path—the revocation check is unconditional and precedes admission. The card lifecycle gains a state (*valid* $\rightarrow$ *revoked* $\rightarrow$ *expired*) modeled as an additional transition (Figure 2.9).

Privacy. A Dolev-Yao adversary observing all gossip traffic learns the revoked-card IDs, leaking which agents have been disciplined. For an organization’s own daemons, this is acceptable audit transparency. Stronger privacy is available by encoding revocations as salted hashes $H(\text{kid}, \text{salt})$ where the salt rotates daily, stored encrypted under a harbor session key.

RECALL

1. Without looking: state the one property that must hold between every delegated Harbor Card and its parent, across all four phases.
2. Why can two daemons not merge their cuckoo filters by OR-ing the bit arrays, and what do they gossip instead?
3. Name the two circumstances that force a card’s withdrawal before its TTL naturally expires.



neither terminal state has an outgoing edge: no partial reanimation

guard predicate

- 1

revoke: operator action or detected compromise (def:revocation)
- 2

TTL elapses: issue time + tau (prop:filter-mono)

Figure 2.9: The card lifecycle as a small state machine: issued has exactly two guarded exits, both terminal. Guard 1 is explicit revocation (an operator action or a detected compromise); guard 2 is natural expiry at the TTL. Once either fires, that card ID never re-enters the admissible set – the dashed green edge is the one recovery available, and it is not a transition of the revoked card: the operator issues a new card on a new ID, leaving the revoked card’s history untouched rather than mutating it. [internal]

2.3 How we know it is correct

Following the industry standard set by AWS (s2n-tls [18]) and Microsoft (Project Everest [29]), we decoupled the verification of the *protocol design* from the verification of the *implementation*. The full stack has three layers (Table 2.2): symbolic protocol analysis at the top, bounded model checking on the Rust source in the middle, and runtime conformance tests against the deployed binary at the bottom. Each layer’s obligations flow downward; each layer’s evidence flows back up. The stack is sound only if every bridge — symbolic-to-source and source-to-deployed — holds.

Table 2.2: The assurance case, layer by layer. Obligations descend the rows: each layer hands the next a claim it cannot discharge itself. Evidence ascends them: each tool’s output narrows what the layer above must assume. The last column names the two gaps that no tool closes; only a human asserts that the modeled protocol is this source, and that the inspected source is this running binary. [internal]

Layer	Tool	What it shows	What it cannot show
protocol model	ProVerif	secrecy, authentication correspondence in the modeled phase, attenuation soundness on the multi-hop model	that the modeled protocol is this source (the specification gap; a human asserts it)
Rust source	Kani and tests	the subset logic on two concrete vectors, bounded absence of panic with the cryptography stubbed, branch-freedom at source level	that the inspected source is this running binary (the deployment gap; a human asserts it)
running binary	conformance suite	the selected attacks are refused at the FFI boundary	any attack the suite does not contain

Table 2.2 is this section’s map. Read down the rows for what each layer hands the next as an obligation it cannot discharge itself; read the third column for what has actually been checked about the running code. The last column holds the only two places in the whole assurance

case where a human, not a tool, asserts that one layer’s claim carries over to the next — which is exactly why they are listed as assumption gaps rather than as inferences. Figure 2.10 draws the same table as one directed bridge: obligations descending the left rail, evidence climbing the right, the two gaps sitting where a human bridges the claim.

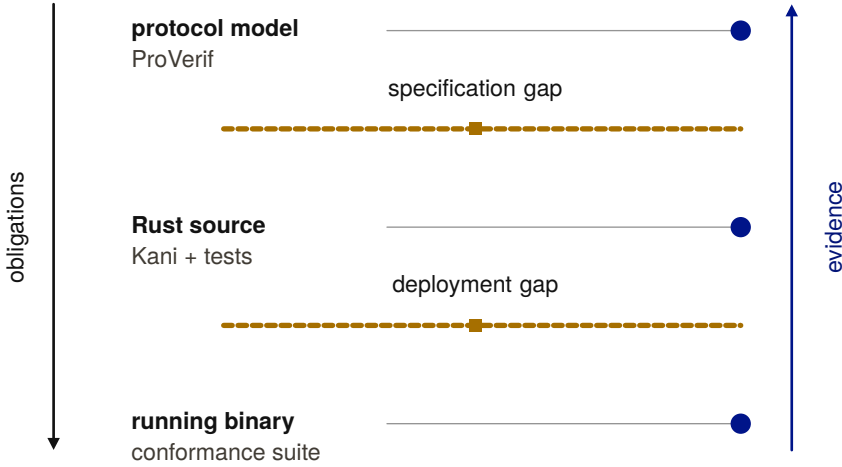


Figure 2.10: The assurance case as a two-rail bridge, restating Table 2.2 as one directed shape. Obligations descend from the protocol model to the Rust source to the deployed binary; evidence from each tool’s own run climbs back up the right rail. The two amber gaps are the only places in the whole case where a human, not a tool, asserts a layer’s claim carries to the next — that the modeled protocol is this source, and that the inspected source is this running binary. [internal]

2.3.1 Symbolic Analysis with ProVerif

To prove the protocol’s logical soundness against a Dolev-Yao adversary [63] (an attacker who controls the entire network), we modeled the Anchor Protocol in **ProVerif** [31]. ProVerif represents protocols as Horn clauses and uses resolution to prove security properties for an unbounded number of sessions.

MODEL-CHECKED PROPERTY 2.3.1 Authentication correspondence in the modeled phase. For the modeled delegation phase, ProVerif confirmed the correspondence:

$$\text{Accepted}(b, h, \text{cap}) \implies (\text{IssuedRoot}(a, h, \text{cap}) \wedge \text{Delegated}(a, b, \text{cap})) \vee \text{IssuedRoot}(b, h, \text{cap}). \quad (2.1)$$

The displayed query is a non-injective authentication correspondence: every accepted event has a matching modeled issuance/delegation

event. It does not by itself prove replay freedom, capability attenuation, or arbitrary-depth transitivity; those are separate queries and bounds.

All three protocol phases were independently verified in ProVerif 2.05. Table 2.3 summarizes the results; full models are reproduced in Appendices 2.B, 2.C, and 2.D.

Table 2.3: All five listed ProVerif queries return **TRUE** across the three modeled protocol phases. Session replication is unbounded where the model uses replication; delegation-chain structure remains bounded as stated in §2.5.3.

Query	Phase	Model	Result
$\neg$ attacker(<i>master_key</i>)	v1 (HS256)	v1_refined.pv	TRUE
Accepted $\Rightarrow$ Issued	v1 (HS256)	v1_refined.pv	TRUE
$\neg$ attacker(<i>harbor_sk</i>)	v2 (Ed25519)	v2_asymmetric.pv	TRUE
Accepted $\Rightarrow$ Issued	v2 (Ed25519)	v2_asymmetric.pv	TRUE
Accepted(<i>b</i>) $\Rightarrow$ IssuedRoot(<i>a</i>) $\wedge$ Delegated(<i>a</i> , <i>b</i>)	v3 (Delegation)	v3_delegation.pv	TRUE
AcceptedPinned(<i>m</i>) encoding Issued _A (<i>m</i>)	encoding	algconfusion.pv	TRUE
AcceptedNaive(<i>m</i>) encoding Issued _A (<i>m</i>)	encoding	algconfusion.pv	FALSE (intended)

The following is the verbatim ProVerif 2.05 output for the authorization chain query (Phase 3), the most complex property:

```

1 RESULT event(Accepted(b_2,harbor_id[],cap_1))
2 ==> (event(IssuedRoot(a_3,harbor_id[],cap_1))
3      && event(Delegated(a_3,b_2,cap_1)))
4      || event(IssuedRoot(b_2,harbor_id[],cap_1))
5 is true.
```

Listing 2.1: ProVerif 2.05 Terminal Output — Phase 3 Authorization Chain

2.3.2 Runtime Enforcement: The Arbiter

Formal models establish properties of their abstractions. At runtime, Port Daddy deploys the **Arbiter**—an ambient monitor that subscribes to the daemon’s post-commit activity log and checks six derived invariants:

1. `PID_SQUATTING`: Detects when a process claims a port previously held by a different PID (the “Ghost in the Harbor”).
2. `CAP_ESCALATION`: Verifies capability subsets via the deployed Rust FFI enforcer.
3. `NOTE_MONOTONICITY`: Ensures the note sequence for active sessions never shrinks (from the TLA^+ `NoteMonotonicity` invariant).
4. `ESCROW_POSITIVE`: Validates that active sessions have positive escrow (when Float Plans are active).
5. `LOCK_OWNER_VALID`: Ensures locks are only held by registered agents with active sessions.
6. `HEARTBEAT_FRESHNESS`: Flags stale heartbeats as early warning for agent death.

Violations are logged and can trigger the “man overboard” salvage protocol: the daemon revokes the offending agent’s active Harbor Cards (§2.2.4), releases any locks and port reservations it held, and, for supervised sessions, notifies the parent agent or the human operator. Because this path observes *committed* events, it detects and compensates; only a pre-commit native guard can prevent a transition in the first place. The Arbiter API provides visibility, not a proof that every security-relevant operation was mediated.

Where the kernel’s boundary places this chapter. The controllability theorem of *The Single-Writer Kernel* sorts every rule by whether the event it prohibits is one the mediator can refuse. Card verification is such an event: accept or reject happens on the daemon’s own socket, before any effect, so every rule this chapter states about tokens, algorithm pinning, per-hop attenuation, and revocation before admission, is regimentable, and the ProVerif models are proofs about a gate that really can close. What the Anchor cannot regiment is what a holder does with authority it legitimately has. Exercising a valid `db:write` to write the wrong thing is an uncontrollable event at this boundary; it is the Arbiter’s detect-and-compensate business here, and audit and slashing in *The Bonded Commons*.

2.3.3 Implementation Verification

A secure protocol is useless if the implementation contains buffer overflows or timing leaks. We extracted the critical JWT parsing and cryptographic comparison logic into a verified core.

DESIGN INVARIANT 2.3.2 Memory safety within the harness bound.

Using the **Kani** Rust Verifier [19], we bounded-model-check the token parser with key decoding, base64 decoding, and signature verification stubbed to return either outcome. For every 32-byte input that is valid UTF-8 and contains a dot, and within ten loop iterations, the parse path neither panics nor accesses memory out of bounds. The bound is the harness's, not the parser's: longer tokens and the real cryptographic primitives lie outside it.

DESIGN INVARIANT 2.3.3 Secret-independent source control flow.

The Rust comparator (`constant_time_compare`) folds the XOR of every byte pair into one accumulator and tests the accumulator once, so no source-level branch depends on secret bytes. That is a property of the source text, established by inspection; Kani's harness adds only that the comparator cannot panic on any pair of 16-byte inputs. Neither is a hardware-level constant-time proof: compiler transformations, memory behavior, and microarchitecture remain outside both.

2.3.4 The procedures, precisely

The three phases are restated here as pseudocode, for the reader who wants the exact procedure rather than the narrative. Nothing new is claimed in this subsection: it is the same three phases written so that each line has a visible counterpart in the formal ProVerif models of Appendix 2.B, in the notation used in the protocol-verification literature [31, 160].

2.3.4.1 Phase 1: Symmetric HS256 with Algorithm Pinning

Algorithm 2.2 presents the secure verification procedure that is immune to algorithm confusion attacks (CVE-2015-9235 [54]).

```

1 Require: Pinned Algorithm: hs256
2 Require: Master Key: k_master (secret)
3
4 function Daemon.IssueToken(agent_id, harbor_id, capability):
5     jti <- GenerateNonce()
```

```

6      msg <- Encode(agent_id, harbor_id, jti, capability)
7      sigma <- HMAC-SHA256(msg, k_master)
8      emit Issued(agent_id, harbor_id, jti, capability)
9      return (hs256, msg, sigma)
10
11 function Harbor.VerifyToken(tok, expected_harbor):
12     (alg_header, msg, sigma) <- tok
13     // CRITICAL: Ignore alg_header, pin to HS256
14     if HMAC-SHA256(msg, k_master) != sigma:
15         return _|_ // Invalid signature
16     (a, h, j, cap) <- Decode(msg)
17     if h != expected_harbor:
18         return _|_ // Wrong harbor
19     emit Accepted(a, h, j, cap)
20     return (a, h, j, cap)

```

Listing 2.2: Algorithm 1: Secure Harbor Verification (HS256 with Algorithm Pinning)

Security Property: The algorithm ignores the user-provided alg_{header} (line 15), preventing algorithm confusion attacks where an attacker sets $alg = \text{"none"}$ or attempts HS256/RS256 confusion [168].

2.3.4.2 Phase 2: Asymmetric Ed25519

Algorithm 2.3 presents the Ed25519-based verification. This phase removes the need for shared HMAC secrets between distributed Harbors.

```

1  Require: Harbor Keypair: (sk_harbor, pk_harbor)
2  Require: Key Distribution Channel: c_key (private)
3
4  function Daemon.IssueToken(agent_id, harbor_id, capability):
5      sk_h <- Receive(c_key) // Get harbor's secret key
6      jti <- GenerateNonce()
7      msg <- Encode(agent_id, harbor_id, jti, capability)
8      sigma <- Ed25519.Sign(msg, sk_h)
9      emit Issued(agent_id, harbor_id, jti, capability)
10     return (msg, sigma)
11
12 function Harbor.VerifyToken(tok, pk_h, expected_harbor):
13     (msg, sigma) <- tok
14     if not Ed25519.Verify(msg, sigma, pk_h):
15         return _|_ // Invalid signature
16     (a, h, j, cap) <- Decode(msg)
17     if h != expected_harbor:
18         return _|_ // Wrong harbor
19     emit Accepted(a, h, j, cap)
20     return (a, h, j, cap)

```

Listing 2.3: Algorithm 2: Asymmetric Harbor Verification (Ed25519)

Security Property: The secrecy of sk_{harbor} and the unforgeability of Ed25519 [27] ensure that only the legitimate Daemon can issue valid

Harbor Cards.

2.3.4.3 Phase 3: Multi-hop Delegation

Algorithm 2.4 presents the authorization chain verification, inspired by Macaroons [30].

```

1  Require: Daemon Public Key: pk_daemon
2  Require: Subset Relation: subseteq on capabilities
3
4  function Agent.Delegate(token_parent, sk_parent, agent_child,
   ↪ cap_sub):
5      (msg_parent, sigma_parent) <- token_parent
6      (_, agent_parent, harbor, cap_parent) <- Decode(msg_parent)
7      if cap_sub not_subseteq cap_parent:
8          return _|_ // Cannot escalate capabilities
9      msg_delegate <- Encode_delegation(msg_parent, sigma_parent,
   ↪ agent_child, cap_sub)
10     sigma_delegate <- Ed25519.Sign(msg_delegate, sk_parent)
11     emit Delegated(agent_parent, agent_child, cap_sub)
12     return (msg_delegate, sigma_delegate)
13
14 function Harbor.VerifyChain(chain, pk_daemon):
15     tok_root <- chain[0]
16     (msg_0, sigma_0) <- tok_root
17     if not Ed25519.Verify(msg_0, sigma_0, pk_daemon):
18         return _|_ // Invalid root signature
19     cap_prev <- ExtractCaps(msg_0)
20     pk_current <- pk_daemon
21     for i <- 1 to |chain| - 1:
22         (msg_i, sigma_i) <- chain[i]
23         if not Ed25519.Verify(msg_i, sigma_i, pk_current):
24             return _|_ // Invalid delegation signature
25         cap_i <- ExtractCaps(msg_i)
26         if cap_i not_subseteq cap_prev:
27             return _|_ // Per-hop capability escalation detected
28         cap_prev <- cap_i
29         pk_current <- ExtractPubkey(msg_i)
30     emit Accepted(agent_final, harbor, cap_prev)
31     return T

```

Listing 2.4: Algorithm 3: Authorization Chain Verification

Security property. Monotonic per-hop attenuation ($\text{cap}_i \subseteq \text{cap}_{i-1}$) prevents capability expansion across transitive delegations, including the write $\rightarrow$ read $\rightarrow$ write re-escalation. Property 2.D.1 (§2.D.1) mechanizes the single hop, where the subset guard is necessary and sufficient. Depth beyond one is the per-hop discipline chained by transitivity of $\subseteq$; its mechanized confirmation at depth two is the attack-and-fix pair in the same section, which checks each hop against its immediate parent rather than against the root.

2.4 The verified crate is the deployed crate

An earlier edition of this chapter titled this section *The verified code is the running code*, and that sentence is wrong as printed — not underhedged, wrong — so it is withdrawn here rather than qualified downstream. The correction is *global*: what fails is the headline claim itself, not a step on the way to it, and the claim that replaces it is a narrower one that happens to be true. What is shared between proof and deployment is the **crate**, not the coverage. The open-source Rust crate `harbor-card-rs` is compiled to a C dynamic library (`cdylib`) and called from the Node.js daemon via FFI through the daemon’s runtime-enforcement (Arbiter) module, and the Kani harnesses are compiled from that same source rather than from a reference model written alongside it.

That is worth having, and it is worth being exact about what it buys. It removes the failure mode that weakens many verification efforts — a specification that drifts from the artifact, because they were never the same artifact. It does not make the proofs coextensive with the code. The harnesses are *bounded*: they stub the cryptography and cap the parser’s input at 32 bytes, which is smaller than the 64-byte Ed25519 signature a real token carries, let alone a whole Harbor Card. What Kani establishes is therefore a memory-safety and no-panic result about the token delimiter parser on small inputs with signature verification abstracted away, and the security of the signature scheme rests on the ProVerif model and on Ed25519 itself, not on these harnesses. Section 2.3’s status table, its caption, and Exercises 2.3 and 2.4 all say this too; the point of saying it here is that a reader meets the section title first. Kani also checks the source under `cfg(kani)` rather than the shipped shared library, so one further gap — the compiler’s — stays open behind all of it.

RECALL

1. Without looking: name the three layers of Table 2.2, and the two gaps between them that are asserted by a human rather than a tool.
2. What can the Arbiter’s post-commit invariants do that ProVerif cannot, and what can they never do that a pre-commit guard could?
3. Which Kani harness does this chapter itself call “a unit test in a proof’s clothing,” and what property actually rests on the ProVerif model instead?

Exercises 2.1–2.2, p. 129.

2.4.1 Core Verification Logic

```

1 pub fn verify(&self, token: &str, now_ts: i64)
2     -> Result<HarborCardClaims, HarborError> {
3     let parts: Vec<&str> = token.split('.').collect();
4     if parts.len() != 3 {
5         return Err(HarborError::Malformed);
6     }
7     let msg = format!("{}", parts[0], parts[1]);
8     let mut sig_bytes = Self::internal_decode_b64(parts[2])?;
9     let sig_result = self.internal_verify_sig(
10         msg.as_bytes(), &sig_bytes);
11     sig_bytes.zeroize(); // Scrub signature from memory
12     sig_result?;
```

```

13     let mut payload_bytes = Self::internal_decode_b64(parts[1])?;
14     let claims: HarborCardClaims =
15         serde_json::from_slice(&payload_bytes)?;
16     payload_bytes.zeroize(); // Scrub payload from memory
17     if claims.exp < now_ts {
18         return Err(HarborError::Expired);
19     }
20     Ok(claims)
21 }
22
23 pub fn constant_time_compare(a: &[u8], b: &[u8]) -> bool {
24     if a.len() != b.len() { return false; }
25     let mut result = 0u8;
26     for i in 0..a.len() { result |= a[i] ^ b[i]; }
27     result == 0
28 }

```

Listing 2.5: Deployed Rust Implementation — Harbor Card Verifier (abridged from the `harbor-card-rs` crate)

Key implementation details: `zeroize` requests that cryptographic material be overwritten when dropped, reducing residual-secret exposure without proving that no copy survives. The comparator avoids source-level early return on secret byte values. That is useful hygiene, not a hardware constant-time guarantee; compiler, cache, and microarchitectural behavior remain outside this source argument.

2.4.2 FFI Bridge to the Node.js Daemon

The Rust crate exports two C-ABI functions consumed by the TypeScript daemon:

```

1 #[no_mangle]
2 pub extern "C" fn harbor_constant_time_compare(
3     a: *const u8, a_len: usize,
4     b: *const u8, b_len: usize
5 ) -> bool { /* ... */ }
6
7 #[no_mangle]
8 pub extern "C" fn harbor_verify_caps_subset_json(
9     root_json: *const c_char, root_len: usize,
10    sub_json: *const c_char, sub_len: usize
11 ) -> bool { /* ... */ }

```

Listing 2.6: FFI Exports — Called from the daemon’s Arbiter module

The daemon’s arbiter module binds these at startup:

```

1 constantTimeCompare: lib.func(
2     'bool harbor_constant_time_compare(
3         const uint8_t *a, size_t a_len,
4         const uint8_t *b, size_t b_len)')

```

```

5 verifyCapsSubset: lib.func(
6   'bool harbor_verify_caps_subset_json(
7     const char *root_json, size_t root_len,
8     const char *sub_json, size_t sub_len)')

```

Listing 2.7: TypeScript FFI Binding (daemon-side Arbiter)

2.4.3 Kani Verification Harnesses

Three bounded model checking proofs are defined in the same source file under `#[cfg(kani)]`:

```

1  #[cfg(kani)]
2  #[kani::proof]
3  #[kani::stub(HarborCardVerifier::internal_pk_from_bytes,
4               stubs::pk_from_bytes_stub)]
5  #[kani::stub(HarborCardVerifier::internal_decode_b64,
6               stubs::decode_b64_stub)]
7  #[kani::stub(HarborCardVerifier::internal_verify_sig,
8               stubs::verify_sig_stub)]
9  #[kani::unwind(10)]
10 fn proof_verify_logic_only() {
11     let pk_bytes: [u8; 32] = kani::any();
12     if let Ok(verifier) = HarborCardVerifier::new(pk_bytes) {
13         let token_bytes: [u8; 32] = kani::any();
14         if let Ok(token_str) =
15             std::str::from_utf8(&token_bytes) {
16             kani::assume(token_str.contains('.'));
17             let _ = verifier.verify(token_str, 0);
18         }
19     }
20 }
21
22 #[cfg(kani)]
23 #[kani::proof]
24 fn proof_constant_time_behavior() {
25     let a: [u8; 16] = kani::any();
26     let b: [u8; 16] = kani::any();
27     let _ = HarborCardVerifier::constant_time_compare(&a, &b);
28 }
29
30 #[cfg(kani)]
31 #[kani::proof]
32 fn proof_capability_attenuation() {
33     let root = vec!["read".to_string(), "write".to_string()];
34     let mut sub = vec!["read".to_string()];
35     assert!(HarborCardVerifier::verify_capability_subset(
36         &root, &sub));
37     sub.push("admin".to_string());
38     assert!(!HarborCardVerifier::verify_capability_subset(
39         &root, &sub));

```

40 }

Listing 2.8: Kani Proof Harnesses (Deployed Source)

The stubbing strategy deserves explanation: `proof_verify_logic_only` stubs out Ed25519 signature verification and base64 decoding (which involve curve arithmetic that Kani cannot symbolically execute within practical bounds) and explores the *parsing and control flow logic*—the split-on-dots, payload extraction, expiry comparison, and error handling paths—for every 32-byte token the harness admits. Within that bound the logic surrounding the cryptographic primitives never panics and never accesses memory out of bounds, whichever outcome the stubbed cryptographic layer returns. The bound is a harness choice, and it has a consequence worth stating: a 32-byte input cannot hold a real three-part card, so what is exercised is the parser’s rejection of malformed input, not its acceptance path. The third harness, `proof_capability_attenuation`, checks two concrete vectors and is a unit test in a proof’s clothing; the every-hop attenuation property is established by the ProVerif model of §2.D.1, not by Kani.

The three harnesses run under `cargo kani` in the repository’s proof workflow; the run log, not a local build directory, is the evidence that they pass.

*Exercises 2.3–2.4,
p. 129.*

2.5 What the adversary can do, and what it cannot

2.5.1 Threats this takes seriously

We assume a **Dolev-Yao adversary** [63] who:

- Controls the entire network
- Can intercept, modify, and inject messages
- Cannot break cryptographic primitives (cryptography is perfect)
- May corrupt legitimate agents (but not all simultaneously)

This is the standard threat model for symbolic protocol analysis [31]. Every figure in this chapter that shows a message leaving daemon or verifier authority marks the region this adversary controls with the same dashed band (Figures 2.4, 2.6, and 2.8); figures without a band—the phase progression, the attenuation rings, the cuckoo mechanics, the card lifecycle, the verification stack—show structure inside one trust domain, where the boundary would be decoration rather than a claim.

Table 2.4 closes the loop on the three failure modes of §2.1: each is named there, addressed somewhere in between, and carries a residual risk stated here rather than left for the reader to reconstruct.

Table 2.4: No mechanism in this chapter closes its threat completely: each of the three failure modes of §2.1 is closed somewhere below, but every row’s residual-risk column is deliberately non-empty.

Threat (§2.1)	Closed where	By what mechanism	Residual risk
Port squatting (“Ghost in the Harbor”)	§2.3, Arbiter invariant PID_ SQUATTING	Cards bind to the connecting process via the daemon’s socket-credential check; the Arbiter flags a port claimed by a PID other than its holder	Detection is post-commit, and the binding is an OS check outside the ProVerif model. See §2.5.3, “A program is not a person”
Resource contention (shared-state corruption)	Appendix 2.E; SQLite single-writer	One writer per harbor, WAL checkpointing, and LOCK_OWNER_VALID in the Arbiter	Enforced by the storage layer, not proved in this chapter; measurable I/O cost
Privilege escalation across delegation	§2.3.4.3; Property 2.D.1	Per-hop capability attenuation, checked at issuance and re-checked at verification, mechanized against an insider escalation adversary	Mechanized at depth 3 and single-hop for the enriched order; depth is a spawn-time policy check, not a verification-time invariant

2.5.2 Security Properties

Table 2.5: The four ProVerif properties hold of the protocol design, not, by themselves, of the shipped code. The three Kani rows are bounded harnesses over the deployed crate: a no-panic check of the parser on 32-byte inputs with the cryptography stubbed, a no-panic check of the comparator on 16-byte pairs, and two concrete vectors for the subset check. “Deployed” indicates the Rust code is called by the production daemon via FFI from the Arbiter module into the `harbor-card-rs` crate.

Property	Verified	Tool	Deployed	Reference
Master Key Secrecy	Yes	ProVerif	—	[31]
Algorithm Confusion Immunity	Yes	ProVerif	—	[168]
Authentication correspondence	Yes	ProVerif	—	[160]
Delegation Integrity	Yes	ProVerif	—	[30]
No-panic parsing (bounded)	Bounded	Kani	Yes (FFI)	[19]
Secret-independent source branching	By inspection	—	Yes (FFI)	[37]
Capability subset check	Two vectors	Kani	Yes (FFI)	[30]

2.5.3 Where this stops

1. **Delegation-chain depth.** The symbolic proof of delegation soundness (Table 2.6, “`chain-replay.pv`”) is machine-checked at the chain depth realized in deployment (depth 3); it is *not* a proof for unbounded chain length. ProVerif’s unbounded-session guarantee quantifies over the number of protocol *sessions*, not over the number of hops in a single authorization chain, which is a fixed structural bound in the modeled process. Extending the proof to unbounded depth (via recursive replication of the delegation process) is future work. The current depth bound is a *spawn-time* policy check (`assessDelegation` in `lib/tube-spawner-router.ts`, `HARD_MAX_DELEGATION_DEPTH=8`, default 4) applied when a spawn request arrives; it is *not* a verification-time invariant. The cryptographic chain verifier (`lib/delegation-chain.ts`) accepts arbitrary chain depth,

and the attenuation monitor (`lib/cap-attenuation-monitor.ts`) enforces only per-hop capability monotonicity—not total depth. Machine-enforced depth at verification time is a planned hardening item.

2. **A program is not a person.** A Harbor Card authenticates a *capability holder*, not a human and not, by itself, a process. The “Ghost in the Harbor” scenario of §2.1 is really a PID-binding gap: nothing in the token cryptography stops a *different* process from presenting a card whose original bearer has died, because the card says what may be done, not who is doing it. Port Daddy binds cards to the connecting process outside the token, via the daemon’s socket-credential check at connection time (`SO_PEERCRE` on Linux, `LOCAL_PEERCRE`/`getpeereid` on macOS). Two consequences follow, and both are boundaries rather than guarantees. First, this is enforced by the operating system, not proved by ProVerif: the symbolic model has no notion of a process, so a daemon build that skips or mis-orders the check is unprotected regardless of card validity, and no query in Table 2.3 would notice. Second, the check is only as strong as what it compares. Walk the concrete case: Agent A holds a card and is running as PID 4021. It crashes at t . Three seconds later the OS reuses 4021 for an unrelated Agent B, which presents A’s card scavenged from a shared temp file. A credential check that compares *only* the numeric PID admits B — the numbers match. The deployed check therefore compares the peer credential against the socket-connection identity established at card issuance and re-validates process start-time, so a reused PID with a later start-time is rejected; a build that compares the bare PID has an open reuse race. That start-time comparison is a runtime conformance test, not a mechanized proof, and it is listed as such rather than as closed.
3. **localhost is a real network.** `localhost` is not a trust boundary. On a shared or multi-user machine any local process — including one belonging to a different user, or a browser page pointed at `127.0.0.1` — can open a TCP connection to a loopback port, and a bearer token sent in the clear over loopback is exactly as interceptable as one sent over the open internet: the Dolev–Yao adversary of §2.5.1 is not a hypothetical here but the account sharing your build box. Concretely, a card with a 15-minute TTL sniffed off loopback at minute 1 grants its holder fourteen minutes of the victim agent’s full capability set, and revocation (§2.2.4) can shorten that window only after somebody notices. The recommended deployment is a Unix domain socket whose parent directory is mode `0700` and owned by the daemon’s own user, so that the filesystem, not the protocol, excludes other users; loopback TLS with a pinned daemon certificate is the alternative where cross-user sharing is genuinely required. The Anchor Protocol

does not currently *enforce* either: the daemon will speak to whatever transport it is configured with. This is therefore an operational obligation on the deployer, and one of the few places in this chapter where a wrong configuration silently voids a property the rest of the chapter proves.

4. **Algorithm confusion at the token-encoding layer.** The delegation models of §2.3 target chain authenticity and capability-subset soundness; the attacks that have actually broken JWT deployments in the field target the encoding underneath, and RFC 8725 catalogues them [234]: the `alg:none` downgrade, and the confusion that hands a verifier an asymmetric *public* key and talks it into treating that key as an HMAC secret. An earlier edition of this chapter disclaimed those attacks — ruled out by construction, it said, since the verifier accepts one fixed algorithm and negotiates nothing — and that disclaimer was too modest, because the model had already been written. `proofs/anchor/token-verify/algconfusion.pv` puts the two verifiers side by side and separates them, under an adversary who is handed the second algorithm’s symmetric key outright (the worst case for the pinned verifier, and the one the CVEs exploit). *Model-checked property*: a verifier pinned to ALG_A accepts a token only if an honest ALG_A issuer minted it — $\text{accepted_pinned}(m) \implies \text{issued_A}(m)$ is TRUE. A naive verifier that reads the header and picks its verification routine from what it finds there fails the same query, and ProVerif hands back the counter-trace rather than a bare refutation: the attacker knows k_B , computes $\text{hmac}_B(k_B, m)$ over a payload no issuer ever saw, wraps it as $\text{tokenB}(ALG_B, m, \dots)$, and the naive verifier fires $\text{accepted_naive}(m)$. That is the whole attack, in four steps, and it’s why “pinned” has to mean the algorithm comes from policy and the token is never consulted about it. What the model does *not* buy is the decoder. It gives the signature slot two distinct types that cannot be silently coerced into one another — which is precisely the type-safety a correct parser has to supply in real code, and which no query here checks. So this boundary has moved rather than closed: the pinning discipline is now mechanized, and the Rust that must implement it is still covered only by the bounded Kani harness of Table 2.5.

RECALL

1. Without looking: state the four properties of the Dolev-Yao adversary this chapter assumes.
2. Of the three failure modes named in §2.1, which one’s residual risk is enforced by the storage layer rather than proved by any tool in this chapter?
3. Why is the PID-reuse fix a runtime conformance test rather than a ProVerif theorem?

Exercise 2.5, p. 129.

2.6 Proofs are leverage, not decoration

A proof is decoration when it is about a model nobody deploys, or when its scope is quietly larger in the prose than in the tool output. It is leverage when it lets you delete a defensive check you would otherwise have had to write, and when the thing proved is the thing that runs. That is the test this chapter has tried to hold itself to: the Kani harnesses are

bounded, say so, and run against the same harbor-card-rs crate the daemon loads at runtime; the ProVerif attenuation model was rewritten (Appendix 2.D, §2.D.1) once a reviewer showed the original could not even *express* the attack it claimed to exclude; and every claim whose mechanization is partial or absent is marked `PARTIAL` or *open* in Table 2.6 rather than rounded up.

The Anchor Protocol replaces several hope-based assumptions with explicit, checkable obligations. It combines:

- Symbolic protocol proofs (ProVerif [31])
- Memory-safe implementation (Rust; bounded Kani harnesses [19])
- Capability-based delegation (inspired by Macaroons [30])

Together these artifacts support a bounded assurance case for the modeled and tested surfaces. They do not certify the entire daemon, every delegation depth, or every side channel — and §2.5.3 says so in the same voice as the claims, because a boundary stated only in a footnote is decoration too.

Chapter Appendices

2.A Formal Verification Status

Artifact availability. Every artifact named in Table 2.6 is bundled at <https://portdaddy.dev/whitepaper/artifacts/>. The verified Rust core is the open-source `harbor-card-rs` crate; runtime components are deployed inside the Port Daddy daemon binary. *The Bonded Commons* carries the cross-paper status table; this appendix lists only the items that pertain specifically to the Anchor Protocol.

Table 2.6: Six of the twelve listed claims close at both the model and runtime level; five remain partial and one open. **CLOSED:** model verified *and* runtime conformance test passes. **PARTIAL:** design verified, runtime empirically tested rather than proved. *open:* known unmechanized.

Claim	Method	Result	Artifact
Authentication correspondence in the listed phase models (§2.3.4)	ProVerif 2.05	CLOSED non-injective correspondence queries TRUE	harbor_card_v*.pv
Algorithm-confusion immunity	ProVerif pinned vs. naive	CLOSED pinned TRUE; counter-trace witnessed	algconfusion.pv
Delegation chain replay (§2.3.4.3)	ProVerif at depth 3	CLOSED chain authenticity TRUE; 17-case runtime conformance	chain-replay.pv
Cuckoo-filter pollution resistance (§2.2.4)	5-invariant property test	CLOSED FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
GitHub-App webhook origin authentication	ProVerif (Dolev-Yao relay)	CLOSED origin auth TRUE; daemon re-verifies GitHub HMAC (runtime conformance)	ingress_origin_auth.pv
Multi-tenant relay namespace isolation	ProVerif + runtime	CLOSED member of A cannot publish into B ; HARBOR_MISMATCH conformance	ingress_tenant_isolation.pv
Cuckoo-filter revocation soundness	inspection + empirical	PARTIAL gate narrows acceptance; dissemination remains assumption-bound	runtime test
Constant-time signature comparison	audited <code>subtle::ConstantTimeEq</code> library	PARTIAL no side-channel proof; library trust documented	—
Relay payload secrecy via HPKE (daemon key pinned)	ProVerif (design)	PARTIAL secrecy TRUE under pinned key; seal not yet implemented	ingress_hpke_secrecy.pv

(continued)

Claim	Method	Result	Artifact
Webhook replay-freedom + in-order delivery	TLA+ / TLC (exhaustive at bound)	PARTIAL design model-checked; runtime via ordered-chain ingest	WebhookDelivery.tla
Card revocation finality under backup/restore (ADR-0018 Attack 2)	TLA+ / TLC	PARTIAL rollback attack + revocation-epoch mitigation both model-checked	CardRevocation.tla
Mechanized gossip-convergence bound	—	<i>open</i> standard expected asymptotics only [60]; no partition deadline	—

Limitations. The transition to gossiping the Append-Only Event Log removes the unsound filter merge described in §2.2.4 (“Why filters cannot be merged bitwise”), but cross-peer gossip propagation still leans on the standard anti-entropy analysis rather than a mechanized proof. The side-channel resistance of signature comparison is inherited from an audited library rather than re-proved. These deferrals are documented as known open problems rather than as defects in the present claims.

GitHub-App relay ingress. The webhook-dispatch surface (GitHub → untrusted relay → daemon) is modeled with the relay as the Dolev-Yao attacker, in three relay-agnostic properties, each paired with a negative control (**_vuln.pv*) that exhibits the attack once the mitigation is removed—so a *true* result is non-vacuous. (i) *Origin authentication*: the daemon dispatches a fleet event only if GitHub signed it, even when the forwarder’s transport credential is attacker-held; the control omits the daemon-side HMAC re-verification and the property fails (forged dispatch). This is *non-injective* agreement (origin), not replay-freedom: a captured genuine (*payload, mac*) may be re-delivered. Replay-freedom is an ordering-sensitive, mutable-state property outside ProVerif’s reach, so it is discharged separately in TLA+ (proofs/relay/WebhookDelivery.tla): a daemon consuming the *ordered* per-publisher chain (dispatch iff $seq = tip + 1$) is model-checked at-most-once *and* gap-free against an adversarial relay that reorders, duplicates, and redelivers, with the unguarded daemon as the model-checked counterexample (TLC exhibits a double-dispatch). The obligation it names: the daemon ingests webhooks over the relay’s ordered *from_seq* subscribe path, not the raw forward. (ii) *Payload secrecy*: under HPKE to the daemon’s *pinned* X25519 key the relay never learns plaintext; the control sources the key from the relay and the secrecy property collapses to a key-substitution MITM—hence out-of-band key pinning is a requirement, not a recommendation. The seal is specified but not yet implemented, so this row is PARTIAL. (iii) *Namespace isolation*: a

member of tenant *A* cannot publish into tenant *B* because harbor binding is checked against authoritative membership; the control trusts the card's self-declared issuer and crosses the tenant boundary. Two limits: HMAC-SHA256 gives origin authentication to a shared-key holder (EUF-CMA), not third-party non-repudiation; HPKE base mode is assumed IND-CCA2.

We err toward listing fewer items as closed rather than more.

THESIS

2.B Full ProVerif Models

For completeness, we include the full ProVerif models used for verification. These correspond to the algorithms in Section 2.3.4.

2.B.1 Phase 1: Symmetric HS256

```

1 (* Port Daddy: Harbor Card v1 (HS256) Formal Model *)
2 type id. type key. type jti. type capability. type alg_type.
3 const hs256_alg: alg_type. const none_alg: alg_type.
4 free c: channel.
5
6 fun hs256(bitstring, key): bitstring.
7 reduc forall m: bitstring, k: key;
8   check_hs256(m, k, hs256(m, k)) = true.
9
10 reduc forall m: bitstring, k: key;
11   verify_generic(m, hs256_alg, hs256(m, k), k) = true;
12   forall m: bitstring, k: key;
13     verify_generic(m, none_alg, m, k) = true.
14
15 fun encode_token(id, id, jti, capability): bitstring [data].
16
17 event Issued(id, id, jti, capability).
18 event Accepted(id, id, jti, capability).
19
20 free master_key: key [private].
21 query attacker(master_key).
22
23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 let Daemon(k: key, a_id: id, h_id: id, j: jti, cap: capability) =
27   let msg = encode_token(a_id, h_id, j, cap) in
28   let signature = hs256(msg, k) in
29   event Issued(a_id, h_id, j, cap);
30   out(c, (hs256_alg, msg, signature)).
31
32 let SecureHarbor(k: key, expected_h: id) =

```

```

33   in(c, (alg_header: alg_type, msg: bitstring, signature: bitstring
      ↪ ));
34   if check_hs256(msg, k, signature) = true then
35     let encode_token(a: id, h: id, j: jti, cap: capability) = msg
      ↪ in
36     if h = expected_h then
37       event Accepted(a, h, j, cap).
38
39 process
40   new agent_id: id; new harbor_id: id;
41   new token_jti: jti; new secret_cap: capability;
42   (!Daemon(master_key, agent_id, harbor_id, token_jti, secret_cap)
      ↪ |
43     !SecureHarbor(master_key, harbor_id))

```

Listing 2.9: harbor_card_v1_refined.pv

2.C Phase 2: Asymmetric Ed25519

```

1  (* Port Daddy: Harbor Card v2 (Ed25519 / EdDSA) Formal Model *)
2  type id. type skey. type pkey. type jti. type capability.
3
4  free c: channel.
5  free k_dist: channel [private]. (* Trusted key distribution *)
6
7  (** Digital Signatures (Ed25519) **)
8  fun pk(skey): pkey.
9  fun sign(bitstring, skey): bitstring.
10 reduc forall m: bitstring, k: skey;
11   check_sign(m, pk(k), sign(m, k)) = true.
12
13 fun encode_token(id, id, jti, capability): bitstring [data].
14
15 (** Events **)
16 event Issued(id, id, jti, capability).
17 event Accepted(id, id, jti, capability).
18
19 (** Queries **)
20 free secret_key: skey [private].
21 query attacker(secret_key).
22
23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 (** Processes **)
27 let Daemon(a_id: id, h_id: id, j: jti, cap: capability) =
28   in(k_dist, sk_h: skey);
29   let msg = encode_token(a_id, h_id, j, cap) in
30   let signature = sign(msg, sk_h) in

```

```

31   event Issued(a_id, h_id, j, cap);
32   out(c, (msg, signature)).
33
34   let Harbor(pk_h: pkey, expected_h: id) =
35     in(c, (msg: bitstring, signature: bitstring));
36     if check_sign(msg, pk_h, signature) = true then
37       let encode_token(a: id, h: id, j_1: jti, cap_1: capability)
38         = msg in
39         if h = expected_h then
40           event Accepted(a, h, j_1, cap_1).
41
42   (** Main Process **)
43   process
44     new agent_id: id; new harbor_id: id;
45     new token_jti: jti; new secret_cap: capability;
46     let harbor_pk = pk(secret_key) in
47     out(c, harbor_pk);
48     (
49       (!Daemon(agent_id, harbor_id, token_jti, secret_cap)) |
50       (!Harbor(harbor_pk, harbor_id)) |
51       (!out(k_dist, secret_key))
52     )

```

Listing 2.10: harbor_card_v2_asymmetric.pv — Verified in ProVerif 2.05

2.D Phase 3: Multi-hop Delegation

```

1  (* Port Daddy: Harbor Card v3 (Delegation Chains) Formal Model *)
2  type id. type skey. type pkey. type capability.
3
4  free c: channel.
5
6  (** Cryptographic Functions **)
7  fun pk(skey): pkey.
8  fun sign(bitstring, skey): bitstring.
9  reduc forall m: bitstring, k: skey;
10   check_sign(m, pk(k), sign(m, k)) = true.
11
12  reduc forall c1: capability; is_subset(c1, c1) = true.
13
14  fun base_token(id, id, id, capability): bitstring [data].
15  fun delegate_token(bitstring, id, capability): bitstring [data].
16
17  (** Events **)
18  event IssuedRoot(id, id, capability).
19  event Delegated(id, id, capability).
20  event Accepted(id, id, capability).
21
22  (** Queries **)

```

```

23 free harbor_id: id.
24 query b: id, cap: capability, a: id;
25   event(Accepted(b, harbor_id, cap)) ==>
26     (event(IssuedRoot(a, harbor_id, cap))
27       && event(Delegated(a, b, cap)))
28   || event(IssuedRoot(b, harbor_id, cap)).
29
30 (** Processes **)
31 free daemon_id: id.
32 free daemon_sk: skey [private].
33
34 let Daemon(a: id, cap: capability) =
35   let msg = base_token(daemon_id, a, harbor_id, cap) in
36   let s = sign(msg, daemon_sk) in
37   event IssuedRoot(a, harbor_id, cap);
38   out(c, (msg, s)).
39
40 let AgentA(a: id, a_sk: skey, b: id, sub_cap: capability) =
41   in(c, (msg: bitstring, s: bitstring));
42   let base_token(=daemon_id, =a, =harbor_id,
43     root_cap: capability) = msg in
44   if check_sign(msg, pk(daemon_sk), s) = true then
45     if is_subset(sub_cap, root_cap) = true then
46       let d_msg = delegate_token((msg, s), b, sub_cap) in
47       let d_s = sign(d_msg, a_sk) in
48       event Delegated(a, b, sub_cap);
49       out(c, (d_msg, d_s)).
50
51 let Harbor(a_pk: pkey) =
52   in(c, (d_msg: bitstring, d_s: bitstring));
53   let delegate_token((base_msg: bitstring, base_s: bitstring),
54     b: id, sub_cap: capability) = d_msg in
55   let base_token(=daemon_id, a: id, =harbor_id,
56     root_cap: capability) = base_msg in
57   if check_sign(base_msg, pk(daemon_sk), base_s) = true then
58     if check_sign(d_msg, a_pk, d_s) = true then
59       if is_subset(sub_cap, root_cap) = true then
60         event Accepted(b, harbor_id, sub_cap).
61
62 (** Main Process **)
63 process
64   new sk_a: skey;
65   let pk_a = pk(sk_a) in
66   out(c, pk_a);
67   new id_a: id; new id_b: id; new cap_root: capability;
68   (
69     (!Daemon(id_a, cap_root)) |
70     (!AgentA(id_a, sk_a, id_b, cap_root)) |
71     (!Harbor(pk_a))

```


72)

Listing 2.11: `harbor_card_v3_delegation.pv` — reflexive-order model (see §2.D.1 for the soundness caveat and the enriched v5 proof)

2.D.1 Soundness of the Attenuation Proof (v5)

The v3 model above is faithful to the cryptographic structure but *vacuous about attenuation*. Its capability order is reflexive only:

```
reduc forall c1: capability; is_subset(c1, c1) = true.
```

Capabilities are opaque atoms with no order, so a delegated capability can only ever *equal* its parent. Escalation—delegating more authority than you hold—is not blocked; it is *inexpressible*. The no-escalation query therefore holds for a reason that has nothing to do with the protocol: there is no larger capability to delegate. A reviewer flagged this directly: “*the model cannot witness escalation.*”

`harbor_card_v5_attenuation.pv` closes the obligation by enrichment, not by weakening the claim. It gives capabilities a real partial order (`cap_read` $\sqsubset$ `cap_write`, both public so the attacker can construct either), replaces the reflexive stub with the sound order (`is_subset(cap_read, cap_write)` derivable; `is_subset(cap_write, cap_read)` *not*), and adds an *insider escalation adversary*: an agent holding a valid `cap_read` root token that signs an over-broad `cap_write` delegation with no self-restraint. The verifier’s `is_subset` guard is the sole defense under test.

```
1 (* Sound order: reflexive + read <= write; write <= read NOT
   ↪ derivable *)
2 fun is_subset(capability, capability): bool
3   reduc
4     forall x: capability; is_subset(x, x) = true
5     otherwise is_subset(cap_read, cap_write) = true.
6
7 (* Insider adversary: holds a real root, signs an over-broad
   ↪ delegation *)
8 let MaliciousA(a: id, a_sk: skey, b: id) =
9   in(c, (msg: bitstring, s: bitstring));
10  let base_token(=daemon_id, =a, =harbor_id, root_cap: capability)
    ↪ = msg in
11  if check_sign(msg, pk(daemon_sk), s) = true then
12    event EscalationAttempted(a, root_cap, cap_write);
13    let d_msg = delegate_token((msg, s), b, cap_write) in
14    out(c, (d_msg, sign(d_msg, a_sk))).
15
16 (* Q1 soundness: a write-card delegated from a read-root is NEVER
   ↪ accepted *)
17 query b: id; event(Accepted(b, harbor_id, cap_write, cap_read)) ==>
    ↪ false.
```

```

18 (* Q2 non-vacuity: the escalation IS reachable (the proof is not
    ↪ vacuous) *)
19 query a: id, r: capability, s: capability;
20 event(EscalationAttempted(a, r, s)) ==> false.

```

Listing 2.12: harbor_card_v5_attenuation.pv (excerpt) — Verified in ProVerif 2.05

MODEL-CHECKED PROPERTY 2.D.1 Attenuation soundness, mechanized. In harbor_card_v5_attenuation.pv, ProVerif 2.05 reports Accepted(b, harbor, cap_write, cap_read) unreachable (**Q1 = true**): no escalating delegation is ever accepted. Simultaneously EscalationAttempted is reachable (**Q2 = false-with-trace**): the attack is expressible, so Q1 is a real defense, not the v3 vacuum. A negative control that deletes the verifier’s is_subset guard flips Q1 to false-with-trace—ProVerif exhibits the read→write escalation—confirming the guard is necessary.

The property was not always true of the model. The version-6 model let a delegation name a capability its parent did not hold, and ProVerif produced the trace; version 7 adds the per-hop subset check and the same query closes. The two verdicts, from the committed results files:

the multi-hop attack on version 6, and version 7 closing it.

AT THE TERMINAL

\$ proverif analyses/harbor_card_v6_multihop_attack.pv

```

-- Query not event(Accepted(cc_2,harbor_id[],cap_write)) in process 1.
goal reachable: event(Accepted(id_c[],harbor_id[],cap_write))

```

```

... derivation, 11 steps, elided; the trace is in analyses/harbor_card_v6_results.txt ...

```

A trace has been found.

RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is false.

\$ proverif analyses/harbor_card_v7_multihop_fixed.pv

```

RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is true.

```

Multi-hop scope. Property 2.D.1 is single-hop (Daemon → A → verifier). Multi-hop delegation has a sharper obligation: a verifier that checks the *final* capability against the *root* accepts a non-monotonic middle hop. harbor_card_v6_multihop_attack.pv mechanizes exactly this — A:write → B:read → C:write is accepted (ProVerif: escalation reachable). The fix, harbor_card_v7_multihop_fixed.pv, verifies each hop against its *immediate parent* (is_subset(*final*, *mid*) ∧ is_subset(*mid*, *root*)) and ProVerif proves the same chain rejected. The

runtime delegation verifier — and the cross-harbor recompute $\text{att}_B \circ \text{att}_A$ — must be per-hop, never final-vs-root. Figure 2.11 draws the same chain under both verifiers side by side: v6’s single final-vs-root check and v7’s two hop-vs-parent checks, so the one comparison that moves is visible rather than implied by the file names.

*Exercises 2.6–2.9,
p. 129.*

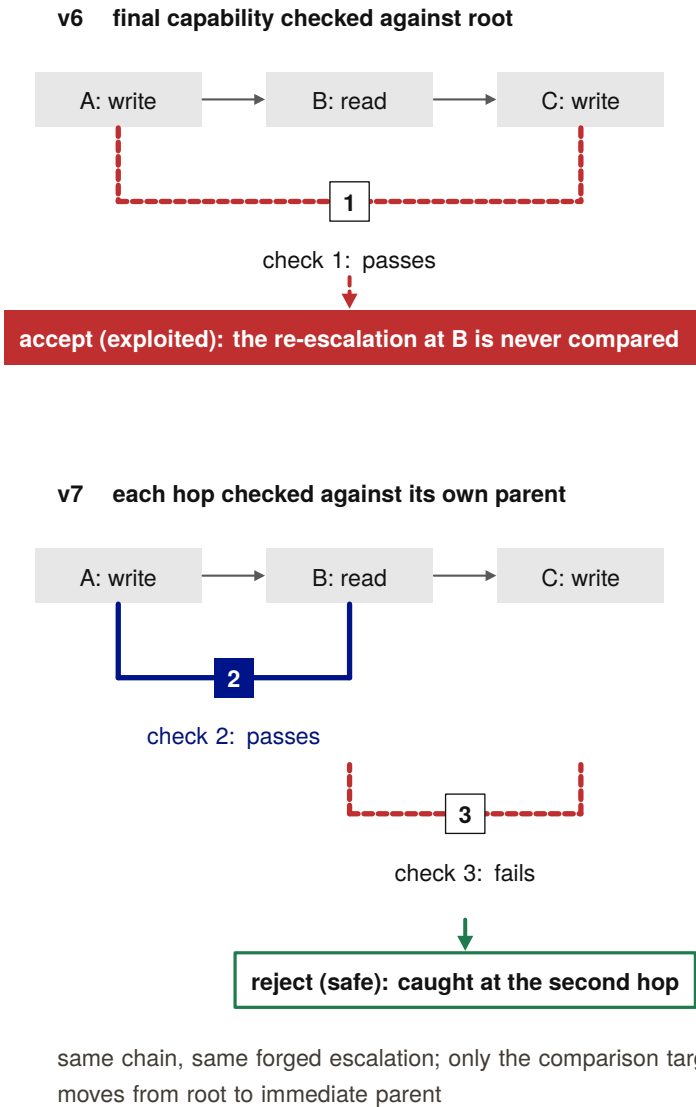


Figure 2.11: The multi-hop attenuation check, before and after: the identical chain $A:\text{write} \rightarrow B:\text{read} \rightarrow C:\text{write}$ under two verifiers. v6 (top) runs one check, the final capability against the root (badge 1); $\text{write} \subseteq \text{write}$ holds trivially, so the re-escalating middle hop at B is never compared, and ProVerif finds the escalation reachable in `harbor_card_v6_multihop_attack.pv`. v7 (bottom) runs one check per hop against its own immediate parent (badges 2, 3); the first passes but $\text{write} \not\subseteq \text{read}$ fails the second, and `harbor_card_v7_multihop_fixed.pv` proves the same chain rejected. [internal]

2.E Limitations & Boundaries

The formal mechanisms presented in this chapter are mathematically sound within their defined scopes, but they depend on bounding assumptions that must be explicitly acknowledged.

- **Threat Model Exclusions:** Collusion across all independent subsystems (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance Trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation Maturity:** While the core protocol (Harbor Cards, delegation, revocation) is IMPLEMENTED and running in production, the unbounded-depth delegation proof, machine-enforced verification-time chain-depth limits, and mandatory transport-boundary enforcement discussed in §2.5.3 remain PARTIAL OR DESIGNED.

These constraints are not flaws but structural realities of building zero-trust coordination on top of POSIX primitives.

2.F Further reading and prior art

This appendix traces the lineage the main text points at in §2.1. It is placed here rather than between the protocol description and the verification argument so that the chapter’s narrative thread is not cut in half by a literature review; nothing in §§2.2–2.6 depends on reading it. The most immediately consequential subsection — the credential-format migration now under way — comes first.

2.F.1 Credential Formats in Agentic Commerce

Since this protocol was first specified, the agentic-payments industry has converged on a concrete credential dialect: the Agent Payments Protocol [166], adopted by the Universal Commerce Protocol [206] (Google/Shopify, 2026), carries its authorization mandates as W3C Verifiable Credentials in SD-JWT form — SD-JWT+kb for principal proof, JWS detached-content signatures (RFC 7515 Appendix F) over JCS-canonicalized payloads (RFC 8785) for counterparty authorization, with ES256 as the recommended algorithm and keys published as JWK sets in a / .well-known profile. The reference implementation migrates

the Harbor Card envelope to this profile (hv: 3, ADR-0094) so that external verifiers need no bespoke SDK. The migration is deliberately envelope-only: the delegation-chain semantics, the attenuation invariant of Property 2.3.1, and the ProVerif obligations are independent of the serialization, though the key-binding construction of SD-JWT+kb adds a binding obligation the model must re-discharge. The JWT hardening below (strict algorithm whitelisting, key separation) transfers unchanged — SD-JWT is a JWT profile, not a departure from one. The chapter’s keywords name the protocol’s JWT lineage; SD-JWT is where that lineage currently lands, not a departure from it.

2.F.2 Formal Verification of Security Protocols

Formal verification of cryptographic protocols has matured significantly over the past two decades. Following the seminal work by Lowe [160] on authentication hierarchies and the Dolev-Yao model [63], several automated tools have been developed:

- **ProVerif** [31]: An automatic symbolic protocol verifier supporting an unbounded number of sessions, used to verify TLS 1.3 [51], Signal [136], and WireGuard. The 2.05 series used here adds the lemma, induction, and subsumption machinery described by Blanchet et al. [34].
- **Tamarin** [169]: A state-of-the-art protocol verifier that supports equational properties and inductive proofs, used for analyzing 5G authentication [24].
- **CryptoVerif** [32]: A computationally-sound protocol verifier that provides proofs in the computational model rather than the symbolic model.

Our work builds on these foundations, applying established verification techniques to the novel domain of local multi-agent coordination. Barbosa et al. [22] survey the wider computer-aided-cryptography landscape and, in particular, the gap between symbolic and computational guarantees that §2.5.3 declines to paper over.

2.F.3 Capability-Based Authorization

Capability-based security dates back to Dennis and Van Horn’s seminal 1966 paper on programming semantics for multiprogrammed computations [62]. Recent work by Birgisson et al. [30] introduced Macaroons, which use chained HMACs for decentralized delegation with contextual caveats. The Anchor Protocol’s authorization chains (§2.3.4.3) are inspired by Macaroons but adapted for JWT-based identity in local development environments.

2.F.4 JWT Security

JSON Web Tokens have been the subject of extensive security analysis. Algorithm confusion attacks were first systematically studied by McLean [168], with subsequent vulnerabilities documented in CVE-2015-9235 [54] (HS256/RS256 confusion), CVE-2018-1000531 [55] (“none” algorithm), and CVE-2023-48238 [56] (generic algorithm confusion). Our protocol implements the defense recommended by the IETF JWT Best Current Practices [234]: strict algorithm whitelisting and key separation.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. A **harbor card** is a short-lived signed statement of exactly what one agent may do; the daemon checks the card, not the process, and a process with no card can do nothing the harbor mediates.
2. **Attenuation is monotone**: a child card is a subset of its parent in scope and in lifetime, at every hop, and the chain’s task hash is the same at every hop. This is the property ProVerif checks on the multi-hop model, and it is the one the version 6 model was found to lack.
3. **Algorithmic pinning**: the verifier chooses the signature algorithm from policy, never from the token’s header. The same bytes that a header-selected verifier accepts, a policy-pinned verifier refuses.
4. **Revocation is an epoch, not a message**: a revoked card leaves the admissible set only when the filter for its epoch reaches the daemon, so there is a stale window of at most two gossip intervals in a connected fleet and no finite bound under partition.
5. **Filters do not merge; logs do**. Two daemons cannot union their cuckoo filters, so the append-only revocation log is the shared object and each daemon rebuilds its own filter from it.
6. **Correction is a new card**, with a new identifier and a fresh authority; the old card’s history is untouched, and nothing is reanimated in part.
7. The proofs are **bounded and named**: authentication correspondence for the modeled phase, attenuation soundness on the multi-hop model, memory safety within the harness bound with the cryptography stubbed, and branch-freedom at source level. Each is a statement about a model, and the chapter says which.
8. What the adversary can still do is stated as scope: a process that holds a valid card can spend it in full within its lifetime; the protocol bounds what a card grants, not what a grantee wants.

2.G Exercises

§2.3 HOW WE KNOW IT IS CORRECT

- 2.1 TRACE ★★** Read `proofs/anchor/token-verify/algconfusion.pv`. *Solution p. 531*
 State the algorithm-confusion attack in words: what the attacker already knows, what it forges, and which verifier admits it. Then say, in one sentence, what *pinning* must mean for a verifier to resist it.
- 2.2 CHECK ★** `proofs/anchor/delegation/chain-replay.pv` models *Solution p. 531*
 a 3-hop chain $P \rightarrow A \rightarrow B \rightarrow C$. Which property does the model check, and what does the run log's verdict say?

§2.4 THE VERIFIED CRATE IS THE DEPLOYED CRATE

- 2.3 CHECK ★★** For each of the three Kani harnesses above, state exactly *Solution p. 532*
 what is proved and name one stronger claim a reader might wrongly draw from it.
- 2.4 OPEN ★★★** None of this chapter's ProVerif models, and none of its *Solution p. 532*
 three Kani harnesses, says anything about most of what the running daemon does at a socket. Using only what this chapter already states about the Kani harnesses' scope, write the one-paragraph boundary a reader must hold before trusting "verified" as a description of the deployed binary. Then say what a stronger harness would need to add.

§2.5 WHAT THE ADVERSARY CAN DO, AND WHAT IT CANNOT

- 2.5 TRACE ★★** A Harbor Card verifies: `Ed25519.Verify` returns true. *Solution p. 533*
 Trace what else must hold before that verified signature actually *authorizes* the action its bearer is attempting. Then say what changes if the daemon treats the token as a bearer credential rather than re-checking it on every request.

§2.D PHASE 3: MULTI-HOP DELEGATION

- 2.6 TRACE ★★** Trace the chain $A:\text{write} \rightarrow B:\text{read} \rightarrow C:\text{write}$ through *Solution p. 534*
 two verifiers by hand: (i) one that checks only the final capability against the root, and (ii) the per-hop verifier of Algorithm 2.4. Which one accepts C? Then say precisely why checking every $\text{cap}_i \subseteq \text{cap}_{i-1}$ proves $\text{cap}_k \subseteq \dots \subseteq \text{cap}_0$, where checking only $\text{cap}_i \subseteq \text{cap}_0$ at each hop does not.
- 2.7 TRACE ★★** Read `analyses/harbor_card_v6_multihop_attack.pv`. *Solution p. 534*
 State the attack in words: who forges what, at which hop, and

why does a verifier that checks only the final capability against the root admit it?

2.8 CHECK ★ `analyses/harbor_card_v7_multihop_fixed.pv` is identical to `v6` except for the verifier. Name the one modeling change that closes the attack, and what the `RESULT` line becomes. *Solution p. 534*

2.9 TRACE ★★★ The authorization chains in this chapter use signed Ed25519 cards. `analyses/macaroon_discharge_v1.pv` and `analyses/macaroon_discharge_v2_naive_unsound.pv` model a related but different construction, an HMAC-chained macaroon with a third-party discharge caveat. Compare the two: what does the naive discharge verifier in `v2` admit, and what does `v1`'s verifier check that closes it? *Solution p. 535*

WHAT THE SEALED HARBOR RECEIVES A request now arrives with authenticated, narrowing authority, and the kernel that admits it keeps one durable history. Some work needs more than an authorized writer: it needs a room in which two parties who trust neither each other nor the venue can still get one answer out. The next chapter builds that room and prices what can leave it through the slot.

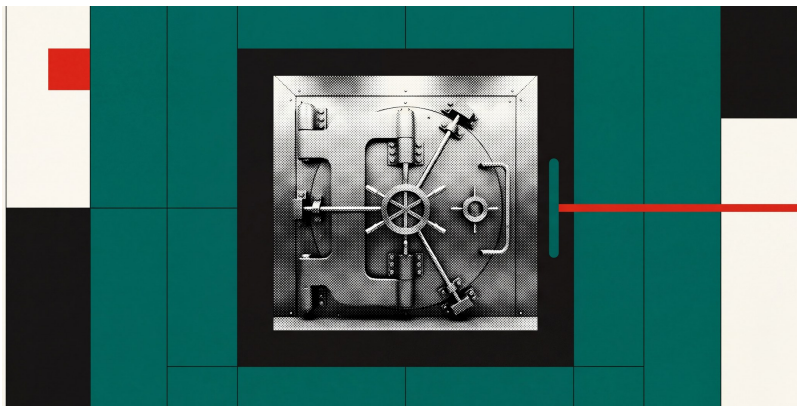
3

The Sealed Harbor

The question this chapter answers

What can a sealed agent leak, and how would we know?

A work order runs in a room with two fences and two gates, the only thing that leaves is what the slot lets out, and the budget of what can leak is a ledger that conserves.



Whereof one cannot speak, thereof one must be silent.

Ludwig Wittgenstein, *Tractatus Logico-Philosophicus*, proposition 7, 1921 (Ogden translation, 1922)

WHAT CAN A SEALED AGENT LEAK, AND HOW WOULD WE KNOW? A proprietary model and a proprietary dataset must produce one answer that neither owner will let the other see. The kernel refuses a write it has not authorized and the Anchor checks who asked; neither says what may leave a room through the only slot in its wall. This chapter builds the sealed room: one work order, two fences, two gates, and four properties, each stated at the strength its check earns. R9, R10, and R11 govern its strongest claims.

3.1 Two owners, one answer

Express lane: a sealed room lets an agent compute on data its owner will not hand over, and the room's promise is not "nothing leaks" but "every release is explicit, gated, and bounded"; the four claims that make that promise provable are Theorems 3.3.1, 3.5.1, and 3.6.1, plus the controllability theorem of The Single-Writer Kernel, and the leakage they leave is the account of Section 3.7.

The scene. Derek owns sensitive financial data D . Erin owns a valuable agent system M : weights, prompts, skills, tools, orchestration. Both want the answer $y = f_M(D)$. Neither will hand over the crown jewels: Derek must not receive M , Erin must not receive D , and the cloud operator hosting the computation must receive neither – three parties, three sight-lines, one boundary that must seal all of them at once, drawn in Figure 3.1.

Every week some enterprise negotiates this standoff with lawyers and air-gapped laptops. The question this chapter answers is whether a *computational* contract can replace the lawyers' one, and what, precisely, it can promise. The systems literature answered a narrower version first: Ryoan [117] confines a proprietary module inside an attested enclave so that a data owner and a code owner who distrust each other both keep their secrets. Ryoan buys its confinement with a module that sees its input once and keeps no state. A looping, tool-calling agent cannot pay that price, which is why this chapter needs a budget and a detector where Ryoan needed neither. Table 3.1 is the whole difference in one place. The other end of that design space is worth naming in the same breath, because it is where most deployments actually sit: SCONE [14] puts an ordinary Linux container inside SGX and asks almost nothing of the application, which makes it adoptable in a way Ryoan is not, and buys correspondingly less — it protects the workload from the host, not the data owner from the code owner, and a container free to open sockets can carry D out through any of them. Between the two lies the gap this chapter works in: the stateful, non-DAG, tool-calling workload that neither a one-shot confinement nor a transparent container serves, and for which the answer turns out to be a priced exit rather than a stronger box.

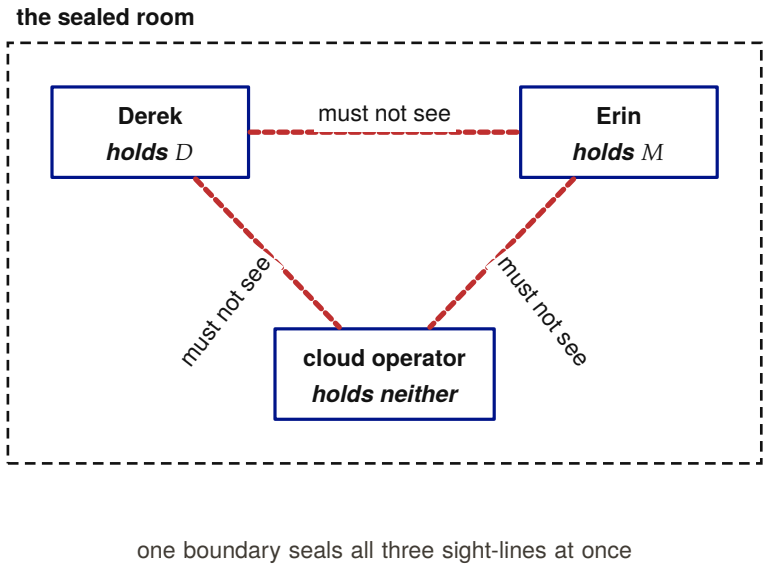


Figure 3.1: The three parties a sealed room must separate, and the single boundary that separates them. Derek must not receive Erin’s model M ; Erin must not receive Derek’s data D ; the cloud operator hosting the computation must receive neither. All three constraints are edges of one triangle, and one enclosing mechanism, the sealed room of Sec. 3.2, seals every edge at once rather than fencing each pair separately. [internal]

Table 3.1: How the sealed room differs from Ryoan. Ryoan confines a stateless module that sees its input once; the sealed room confines a looping agent, so it needs a priced exit and a detector where Ryoan needed neither.

	Ryoan	The sealed room
Workload shape	a request-oriented module, input seen once, no persistent state	a looping, tool-calling agent with state across steps
How leakage is bounded	confinement in the enclave plus fixed-size outputs	explicit gated releases, priced by an ϵ -ledger and a $q \cdot b$ bit budget
Taint granularity	the whole module	the slot: every release is a declassification event
Model access	the module’s code is sealed; no query interface	the model runs inside the domain and is observed only through the contracted query and output interface
Multi-job accounting	none; each request stands alone	the ledger sums spend across jobs and refuses the overdraw
Evasion	prevented by construction for its workload	caught by canaries with a power curve and a Wald clock
Timing	out of scope	declared out of model; padded and scheduled, not closed

The pitch that must never ship. The tempting design, and the one an early product review of this program actually wrote, is a “silicon-enforced NDA”: tag Erin’s agent the moment it reads Derek’s data, drop any tainted egress packet, and declare that the data “mathematically cannot phone home.” The first job of this chapter is to say why that sentence is false, and the second is to prove the sentence that replaces it.

Numbers by hand — what “nothing leaks” costs, in bits. Price the three candidate designs on the chapter’s own toy release — the one Theorem 3.3.1 will later check exhaustively — and you can do the whole comparison in your head. Derek’s secret is one of $\{0, 1, 2, 3\}$, so it carries $\log_2 4 = 2$ bits, and the contracted release is $g(s) = s \bmod 2$, which takes two values and therefore carries exactly 1 bit — and, since s written in binary has its parity in the low position, that release is literally one of the secret’s two bits. Now count. An honest gate hands Erin 1 of those 2 bits: the bit Derek sold her, and no more. The leaky gate of the mutation suite, which forwards s itself, hands over 2 — the whole secret, which is why the checker catches it four steps in. And the taint tracker of the pitch above, run *soundly* at token granularity, has to mark the parity bit as tainted by the secret it was computed from and drop it, so Erin gets 0 bits and the engagement has nothing left to deliver to her. That 0-bit column is real, it is sound, and nobody will buy it; the other horn, the tracker that lets the parity through, cannot then certify that what the worker submitted was s rather than some $f(s)$ it chose. Every honest design in this chapter therefore lives in the middle column, and the rest of the chapter is the arithmetic of how wide that column is allowed to get.

EMPIRICAL HYPOTHESIS 3.1.1 Token-level taint through a language model is not soundly definable. A taint tracker over a generative model faces a binary choice: mark every output token as tainted by everything the model read, which is useless, or miss semantic flows, which is unsound. The model taints everything it writes with everything it read. This is a design rationale, not a theorem, and it is the one decisive statement in this chapter that carries no proof and no script. It rests on the well-known static-analysis result beneath it, that certifying implicit flows soundly forces exactly this dichotomy: the label creep of the Denning certification lattice [61] and the extra restrictiveness a sound purely dynamic monitor must accept relative to a static one [221]. No sound token-granularity flow analysis for a generative model is known; the burden of exhibiting one falls on any design that assumes it.

A secret can ride out in wording, punctuation, error messages, output length, or timing; no regular expression, scanner, or second model can certify that arbitrary prose contains no secret. And an ordinary remote model API is *itself* a trusted reader of plaintext: cryptography cannot repair sending Derek’s data to a third party’s inference endpoint.

The architecture therefore controls the *channel*, not the token. It converts the question a security officer will actually ask, *how do I know the wall has no other holes?*, into four provable claims: the gate is the only hole (Section 3.3); the gate sits on the only boundary that can be enforced at all (Section 3.4); what passes through it is metered by a conserved budget (Section 3.5); and what tries to sneak around the meter is caught with quotable probability and latency (Section 3.6). Section 3.7 composes the four into a single leakage account, and Section 3.8 states, at equal prominence, what none of this promises.

3.2 One work order, two fences, two gates

The work order. Both principals sign the same contract before any key is released:

$$C = \langle H(\text{data manifest}), H(\text{agent bundle}), H(\text{runtime}), H(\text{policy}), \\ \text{function, tools, recipients, output schemas, budgets,} \\ \text{retention, validators, payment, liability} \rangle.$$

Read the tuple in order. The four hashes pin the data manifest, the agent bundle, the measured runtime, and the policy; function and tools fix what may run and what it may call; recipients and output schemas fix who may receive a result and in what shape; budgets bound output bits, queries, and time; retention fixes key destruction; validators name the success evidence and the dispute procedure; payment and liability are the guarantee. Any change makes a new hash and requires both signatures. It is a work unit in the sense of *The Single-Writer Kernel*: the case file that outlives every process that touches it. Figure 3.2 groups the tuple’s thirteen fields by the five roles just named.

Dual-attested key release. The workload boots in a confidential virtual machine and creates an ephemeral job key with fresh attestation evidence binding runtime, policy, worker image, key, nonce, and epoch. Derek and Erin *independently* verify the attestation; Derek’s key broker releases the data key only to the approved measurement, and Erin’s key broker releases the model and skill key under the same condition, the attester, verifier, and relying-party pattern of IETF RATS [98]. The job then runs with no ambient network or credentials. Neither principal, and not the cloud operator, can read the other’s plaintext: the room is sealed from three sides at once.

role	fields	what it fixes
pins	$H(\text{data})$, $H(\text{agent})$, $H(\text{runtime})$, $H(\text{policy})$	four hashes – each pins an attested measurement, not the live thing
permissions	function, tools	fixes what may run and what it may call
recipients	recipients, output schemas	fixes who may receive a result, and in what shape
bounds	budgets, retention	bounds output bits, queries, and time; retention fixes key destruction
remedies	validators, payment, liability	validators name the success evidence and dispute procedure; payment and liability are the guarantee

any change to any field makes a new hash for *C* and requires both Derek’s and Erin’s signatures again

Figure 3.2: The work order *C*’s thirteen fields, grouped by the five roles the chapter itself names. The four hashes (shaded, top) pin an attested *measurement* of each named object – the data manifest, the agent bundle, the runtime, and the policy – not the object’s live state at query time; function and tools fix what may run; recipients and output schemas fix who may receive a result and in what shape; budgets and retention bound the engagement; validators, payment, and liability are the remedy if something goes wrong. Any change to any field produces a new hash for *C*, so both principals must sign again. [internal]

Figure 3.3 draws the ceremony as a sequence: one attestation evidence bundle, appraised twice and independently, each broker releasing its own key off its own appraisal alone.

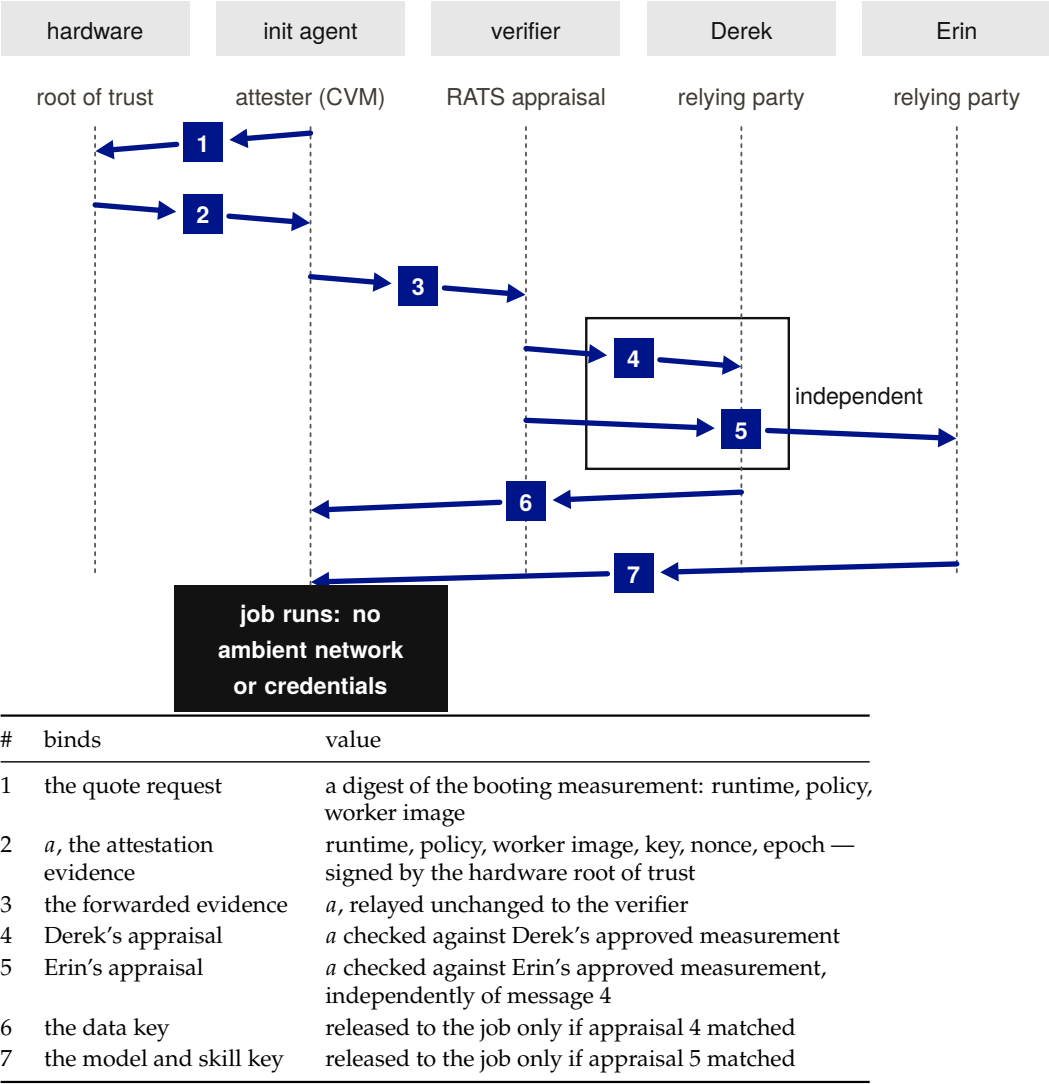


Figure 3.3: The dual-attested key release ceremony of Sec. 3.2, five lifelines wide: the hardware root of trust, the confidential VM’s init agent (the RATS *attester*), the RATS *verifier*, and Derek’s and Erin’s key brokers, each a RATS *relying party* for its own key. One attestation evidence bundle is produced (2) and forwarded once (3), but appraised twice, independently (4, 5, boxed) — against Derek’s approved measurement and against Erin’s, separately, never one shared verification step fanned out to both. Each broker releases its own key only off its own appraisal (6, 7); only once both have released does the job run, with no ambient network or credentials. [internal]

Two fences. Erin’s worker is treated as hostile even if Erin is honest, because prompt injection or a compromised dependency can turn a loyal agent into an exfiltrator, so it receives *semantic handles only*:

```
read(handle, range)  infer(model handle, prompt handle)
                    execute(tool handle, args)
                    write(result handle, bytes)
request_declassification(candidate handle, channel)
```

An attested in-guest monitor mediates labels, handles, and encrypted content; a Derek-controlled outer gateway mediates network, storage, timing, destination, and volume. Direct outbound TLS is forbidden; permitted services terminate at a policy gateway. Because a remote model API is itself a trusted reader of plaintext, the model runs *inside* the confidential domain; an external inference endpoint would be a third principal the contract never admitted.

Numbers by hand — *four label states, four locked doors, two keys*. The lattice the next paragraph reaches for is small enough to count on your fingers, and the count *is* the mechanism. Two owners, Derek and Erin, means two clauses, so a value inside the room sits in one of $2^2 = 4$ label states: $\{\}$, $\{D\}$, $\{E\}$, $\{D, E\}$. Derek’s data enters at $\{D\}$ and Erin’s model at $\{E\}$; the instant the worker computes with both, whatever it produced sits at $\{D, E\}$. There are four edges up that lattice and four edges down it, and the asymmetry between the two fours is the design. All four upward edges are free and unwitnessed, because adding somebody’s restriction needs nobody’s permission. All four downward edges are locked, and — this is the part worth doing by hand — they’re locked *by owner*: the two that drop D , namely $\{D\} \rightarrow \{\}$ and $\{D, E\} \rightarrow \{E\}$, open only with Derek’s delegated declassification authority, and the two that drop E , namely $\{E\} \rightarrow \{\}$ and $\{D, E\} \rightarrow \{D\}$, only with Erin’s. Two keys, four locks, two apiece. That is why the room has exactly two gates rather than one, and why the aggregate-feedback channel to Erin is *Derek’s* gate and not hers: the artifact she receives is one that has had Derek’s clause dropped, and only Derek can drop it.

Whole-worker taint and two gates. After the first secret read the entire worker state, and every later emission, is tainted until job destruction: decentralized labels in the Myers–Liskov style [177], owned by mutually distrustful principals, under which ordinary computation can only *add* restrictions. This is the same shape as capability attenuation in *The Anchor Protocol*, where a delegated token never carries more authority than its parent: authority only shrinks, and every widening is an explicit, witnessed exception rather than a computation. Removing an owner’s clause requires a declassification authority delegated by that owner, and the only exits are the two declassification gates, Derek’s and Erin’s, each executing a declared release function, with exactly

three pre-authorized channels: a rich result to Derek; fixed-schema, aggregated or differentially private feedback to Erin; a padded public receipt to both. Each party receives an audience-redacted signed receipt, Merkle-anchored in both principals’ ledgers:

$$R = \text{Sign}(H(C), H(\text{attestation}), H(\text{inputs}), H(M), H(\text{output ciphertext}),$$

labels, declassifications, counters, nonce, epoch).

The receipt proves what the measured system reported under a policy; it does not prove the answer is true, nor that no unmodeled side channel existed. Figure 3.4 draws R ’s bound fields as a tree and flags the one leaf that proves only what was released, not that it was correct.

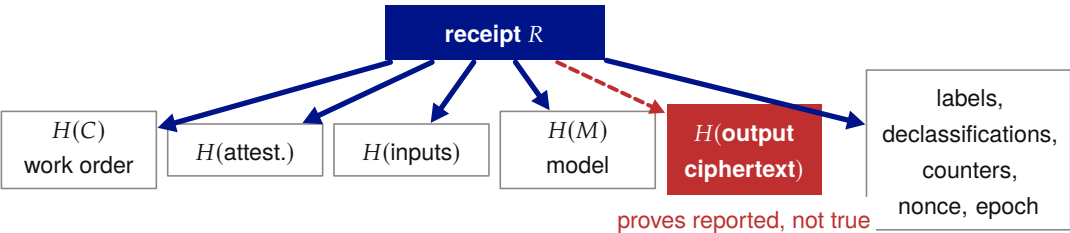


Figure 3.4: The receipt $R = \text{Sign}(\dots)$ of Sec. 3.2, drawn as a provenance tree. Four leaves are hashes that establish provenance: the work order, the attestation, the inputs, and the model. A fifth bundles the release’s declared policy state – labels, declassifications, counters, and the freshness pair nonce/epoch. The sixth leaf, dashed, is different in kind: the hash of the output ciphertext binds only *what was released*, not that it was correct. A receipt proves what the measured system reported under a policy; it proves nothing about whether the answer inside it was true. [internal]

DESIGN INVARIANT 3.2.1 The room’s side properties. Label monotonicity: no label weakens without a declassification witness. Complete mediation: every release has exactly one gate decision. Provenance closure: every release traces to committed input, model, runtime, policy, and declassifier nodes. Replay safety: each job nonce settles at most once. Measurement binding: neither secret key is released for a different runtime or policy. Rollback detectability: accepted epochs strictly increase in both external ledgers. Each is intended to hold and is backed by the implementation and its tests; none is a theorem, and this chapter never lets one stand in for one.

Table 3.2 names, for each of the six, the later result that leans on it; complete mediation is the one link and conservation both cite by name, the others every later theorem assumes silently.

Table 3.2: The room’s six side properties and the later result each underwrites. None is a theorem; each is backed by the implementation and its tests. Complete mediation is flagged because it is the one property later sections name explicitly, not merely assume.

Invariant	States	Depended on by
Label monotonicity	no label weakens without a declassification witness	every gate of Sec. 3.2; assumed silently by noninterference (Sec. 3.3)
Complete mediation	every release has exactly one gate decision	named explicitly: the deployment assumption Enforceability (Sec. 3.4) and Conservation (Sec. 3.5) both inherit the receipt $R = \text{Sign}(\dots)$ of Sec. 3.2
Provenance closure	every release traces to committed input, model, runtime, policy, and declassifier nodes	
Replay safety	each job nonce settles at most once	the receipt’s nonce field and the dual-attested key release’s fresh nonce (Sec. 3.2, Fig. 3.3)
Measurement binding	neither secret key is released for a different runtime or policy	the dual-attested key release ceremony directly (Fig. 3.3)
Rollback detectability	accepted epochs strictly increase in both external ledgers	the ε -ledger’s append-only journal (Sec. 3.5)

Four links, one pipeline. The four claims are not four independent guarantees stacked for redundancy; they are four links in one pipeline, each closing a gap the others leave open. Enforceability fixes *where* a boundary can exist at all: gate the channel, or nothing about the design is provably enforceable. Noninterference proves that boundary, once installed, is silent except through its declared release. Conservation meters the cumulative cost of everything that legitimately crosses it, because a gate honest on every single release can still bankrupt the contract over many releases; an individually sound decision rule is not a portfolio-sound one. Detection polices for what tries to cross without going through the meter at all: a compromised worker, a bug, an adversary who found an unmediated path. Drop any one link and a concrete gap reopens; the composed result, a bounded leakage account rather than an eliminated one, is Section 3.7. Table 3.3 is the evidence schedule: each link names its claim, how the claim was verified, and the script that regenerates the verification.

Table 3.3: The four links of the sealed room and the evidence behind each. The four-pillar claim is an evidence schedule, not a slogan: every row names a regenerable check.

Link	Claim	Verification	Script
Noninterference	silence except through the slot	exhaustive to depth 7; both mutations caught	c1_noninterference.py
Enforceability	the channel, never the token	product-automaton checker	b3_controllability.py
Conservation	the ledger conserves	exhaustive and 2000 random instances; both mutations caught	a3_epsilon_ledger.py
Detection	power curve and alarm clock	analytic against simulation	a4_canary_sprt.py

Figure 3.5 draws this pipeline: one work order, two fences, two gates, and which of the four claims each part produces. The figure orders the links as they compose at run time, enforceability first; the sections that follow take noninterference first because it is the promise the other three exist to keep.

- RECALL
1. Without looking: name the three parties who must not see each other’s plaintext, and the one mechanism that seals the room from all three sides at once.
 2. Why does the worker receive handles rather than bytes, even when Erin is honest?
 3. Which of the six side properties is the one every later theorem assumes, and why is it a deployment property rather than a theorem?

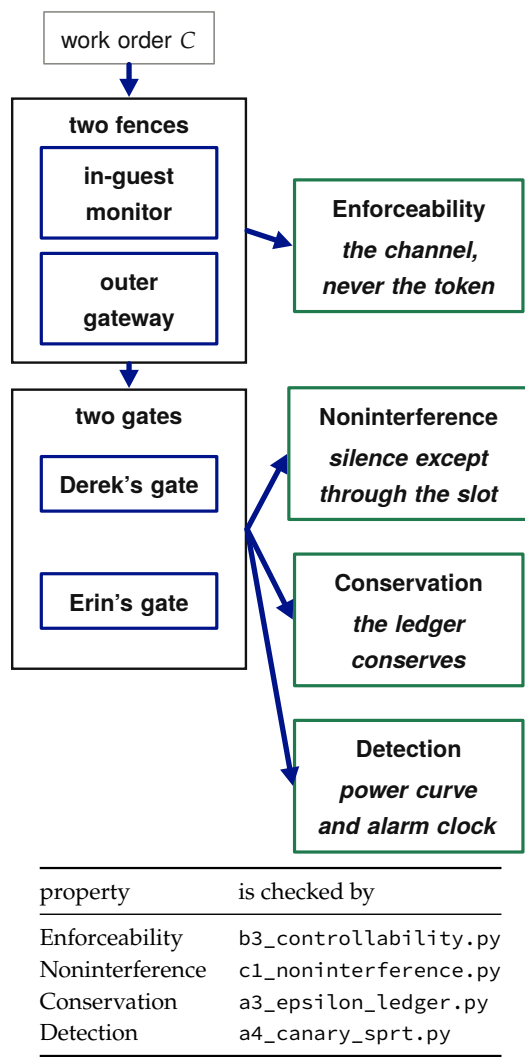


Figure 3.5: One work order entering the room, mediated by two fences (the in-guest monitor, the outer gateway) and released only through two gates (Derek’s, Erin’s); solid arrows read “produces”. The fences alone are what make the channel enforceable at all; the gates, once installed on that channel, are what produce the other three properties – silence except through the slot, a conserving ledger, and a watched detector. The table keys each property to the script that checks it.

3.3 Silence except through the slot

Express lane: run the whole world twice, identical except for Derek's secret, under every possible sequence of moves — self-composition, in the name the verification literature gave the move [23] — and prove that Erin's view is bit-for-bit identical whenever the gate's declared function maps the two secrets to the same release; the formal statement is Theorem 3.3.1.

Intuition: the voting booth. You cannot inspect a wall for holes by staring at bricks; you inspect it by standing outside in two parallel worlds and asking whether anything you can see ever differs. If Derek's secret is 0 in one world and 2 in the other, and the gate is contracted to release only *parity*, then a sound room makes those worlds indistinguishable to Erin forever, under every interleaving of loads, reads, submissions, releases, and reports. The analogy with teeth is the voting booth: the turnstile clicks once per voter and the tally board shows only party totals; a poll-watcher outside learns exactly the declared aggregate and nothing about any ballot, no matter how long she watches or in what order voters arrive. That is noninterference *modulo declassification* in the Goguen–Meseguer lineage [90, 91]: silence except through the slot, and through the slot, only what the contract says. The misread to preempt: the theorem does not say the slot is safe. It says the slot is the *only* opening; whether the declared function g releases too much is a contract question the next two sections price. In the booth's own vocabulary: the guarantee is about the walls and the turnstile, never about what the board is asked to print. A per-precinct board reporting a precinct with a single voter discloses that voter's ballot exactly, and no property of the booth prevents it. The room stayed sound; the tally was indiscreet.

MODEL-CHECKED PROPERTY 3.3.1 Noninterference modulo declassification on the finite clean-room model. Fix the clean-room transition system with secrets $s \in \{0, 1, 2, 3\}$, Erin-observable state Obs_E , and declared release function $g(s) = s \bmod 2$, where the gate applies g to committed input state rather than to worker-computed state. For secrets s, s' and *every* action sequence: if $g(s) = g(s')$ then Erin's observation sequences are identical; if $g(s) \neq g(s')$ they may differ only at explicit gate-release steps. **Result.** Exhaustive over all interleavings to depth 7 and all secret pairs: zero distinguishing observations in the honest model. The schedule itself is secret-independent, checked separately, since an enabledness difference would itself be a channel. Rushby's unwinding condition of local respect, that Derek-side actions never touch Erin-observable state, holds mechanically [122]. **Mutations.** A gate that leaks the raw secret is caught distinguishing the equal-parity pair (0, 2) with the witness trace $\text{load} \rightarrow \text{read} \rightarrow \text{submit} \rightarrow \text{release}$, after which Erin sees (gate, 0) in one world and (gate, 2) in the other;

a worker bypassing the gate is caught in a three-step trace. Reverting either mutation restores the property [internal, c1_noninterference.py].

The clause about committed input is a hypothesis, not a modelling convenience. It makes the property an instance of *delimited release* [222] in the degenerate case where delimited release collapses to noninterference; their own condition is that no variable used in a declassification is updated before it. What the hypothesis costs is stated in the boundary below and paid in Section 3.7. The release ledger of §3.7 answers, in Sabelfeld and Sands’ later organizing taxonomy [223], *what* is released (the ε -priced artifact), *who* performs the release (the declassifier), and *where* it happens (the gate); *when* it happens is the release-ledger’s own open question, since a room can only price the release it observes, not the frequency an adversary chooses to request it.

Proof idea. There is no proof to read; there is a search to rerun. The checker builds the two worlds side by side, one per secret, and drives them through every interleaving of the same action sequence to depth 7. This is **self-composition**, exactly as Barthe, D’Argenio and Rezk set it out [23]: noninterference is a claim about *pairs* of runs, which no ordinary invariant can state, so you compose two copies of the system into one and state an ordinary invariant about the pair. We do it to a finite model with a Python checker, where they do it to a program with a logic; that is the small version of the technique, but it is the technique — and this book said “run the world twice” for two editions before noticing the move had a name. At each step the checker projects both states onto what Erin can observe and compares. A difference at a non-release step is a counterexample; a difference at a release step is a counterexample exactly when g agreed on the two secrets. The mutation suite then shows the search has teeth: with the honest gate replaced by one that releases the raw secret, the first difference appears four steps in, on a pair the honest gate provably keeps identical. Figure 3.6 draws the two worlds side by side, tied step by step, with the release step boxed as the one place a difference could ever show.

Numbers by hand — six pairs, two of them equal-parity. Four secrets give $\binom{4}{2} = 6$ unordered pairs; two of them, (0, 2) and (1, 3), are equal-parity, and it is exactly those two on which the honest gate must keep Erin’s worlds identical forever, and did. The first mutation’s witness is a four-step, human-readable proof that a particular gate breaks its contract; a security officer can read it aloud. *Now you try:* with $g(s) = s \bmod 2$ over secrets $\{0, \dots, 7\}$, how many unordered secret pairs must a sound room keep Erin-indistinguishable? (Exercise 3.1.)

■
c1_
noninterference.py
· 20260816 · VERIFIED

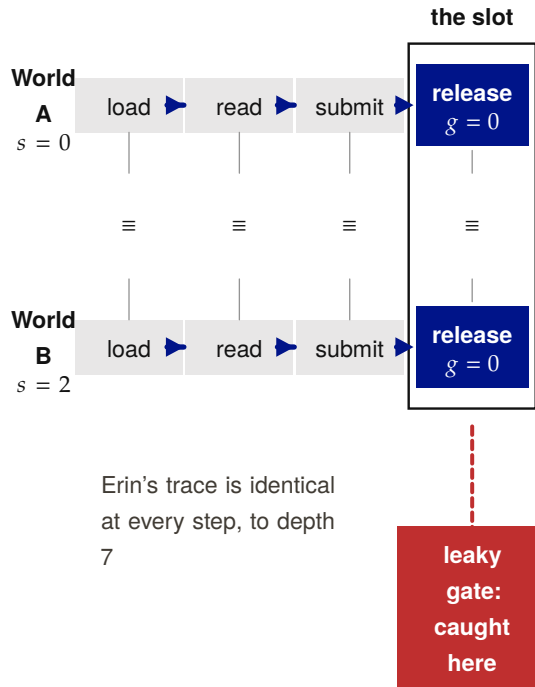


Figure 3.6: Two runs of the clean room, identical except Derek’s secret ($s=0$ against $s=2$, equal parity so $g(s)=0$ either way), driven in lockstep through $\text{load} \rightarrow \text{read} \rightarrow \text{submit} \rightarrow \text{release}$. Erin’s observable state is tied ($\equiv$) at every step; the release step is boxed as “the slot” because it is the only step the theorem lets differ at all, and here it still does not. Below the slot, replacing the honest gate with one that leaks the raw secret breaks the tie at this same step – leak 0 from World A against leak 2 from World B, now distinguishable. The finite clean-room model, secrets $s \in \{0, 1, 2, 3\}$ with $g(s) = s \bmod 2$, passes this check exhaustively to depth 7 over every interleaving and secret pair; both mutations are caught – the leaked-secret gate shown here (4 steps) and a worker that bypasses the gate entirely (3 steps) [internal, `c1_noninterference.py`].

What it buys. The product’s headline sentence in its provable form: *every release is explicit, gated, and bounded*. This is the finite-model core of the chapter; the general theorem over unbounded state is the unwinding obligation of Section 3.8, for which this model is the blueprint, the same three unwinding conditions discharged mechanically rather than exhaustively.

Finite depth, finite secrets: depth-7 exhaustion is a proof about the model, not the deployment. The guarantee is possibilistic; timing, token counts, and termination channels are out of model, as the design declares. The gate’s *specification* is the residual oracle: the property is that releases match g , and choosing a g that leaks too much is a contract failure no theorem prevents. That risk is priced, not eliminated, by the next two sections.

WHERE THIS STOPS

There is a second residual on the other side of the same gate. The property quantifies over action sequences of a model whose gate applies g to committed input. It does not cover a worker that launders: computing $t = f(s)$ and submitting t so that the gate honestly releases $g(t)$. That is the classical escape-hatch laundering attack, which the delimited-release literature names and rules out by enforcement [222]. Enforcement is unavailable here, because a language model’s release candidate is arbitrary output rather than a syntactically identifiable expression a checker can quantify over. The residual is therefore priced in Section 3.7, which is the reason the $q \cdot b$ account exists at all. The present mutation suite cannot exhibit the attack, since the model’s `submit` carries no payload: the suite tests that the gate releases g and nothing else, not that the gate’s *argument* is honest. Closing that gap is a model extension listed in Section 3.8. And a human who reads a released parity may still remember it: the system bounds channels, not minds.

Figure 3.7 draws the fork: the honest path and the laundering path are identical through `submit`, because that is the one node the checker’s model can see, and only the honest gate’s *argument* differs.

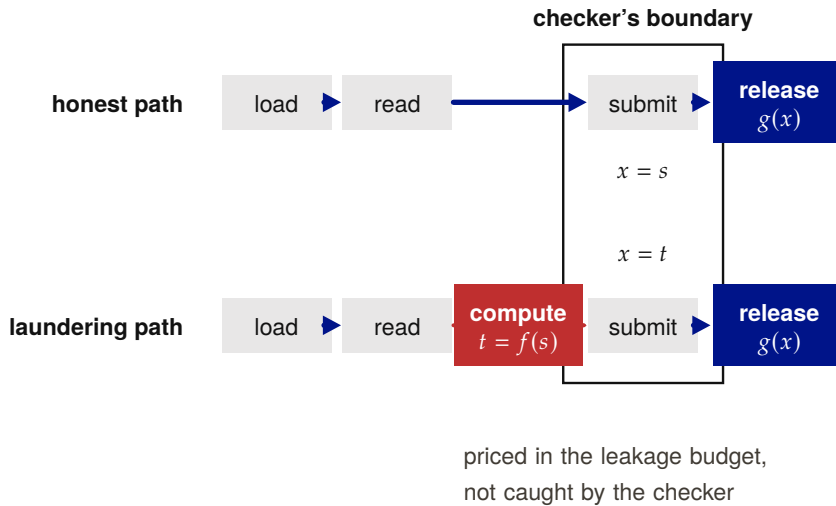


Figure 3.7: Two paths through the same honest gate, drawn as paired lanes. Above, the honest path submits the committed secret s directly, so the gate's declared function releases $g(s)$. Below, the laundering path first computes $t = f(s)$ for a worker-chosen f and submits t instead, so the same honest gate releases $g(t)$ – for a non-constant f that maps an equal-parity pair to a different-parity pair, this differs from what the honest path would have released. The two lanes are drawn identically through `submit`, boxed as the checker's boundary, because the model's `submit` action carries no payload: the mutation suite that catches a leaky gate or a bypassed gate in Theorem 3.3.1 has no variable to quantify over here, so it cannot exhibit this attack at all. The residual is not eliminated; it is priced as channel capacity in Section 3.7 [internal; no script – Exercise 3.4 states the model extension a checker would need].

3.4 The gate sits on the only enforceable boundary

Express lane: a safety policy over an agent's behavior can be enforced by the mediation layer if and only if it constrains only events the layer mediates, so the enforceable boundary of the room is the egress channel, and any promise phrased over model-internal events is detect-only forever; the theorem is the controllability theorem of The Single-Writer Kernel, and this section only applies it.

Numbers by hand — eight events, five vetoes, and one rule that only looks like an exception. Sort the agent's alphabet into two piles before reading any theorem about it. Five events the mediation layer can veto before they take effect: `fs_write`, `net_egress`, `exec_tool`, `git_push`, `spawn_child`. Three it can only watch go past: `model_emit_token`, `in_context_read`, `internal_plan`. Eight events, and therefore eight policies of the plain form *never do X*, of which exactly five are regimentable and three are detect-only — and you can sort any of them without running anything, because the test is just “is the event it has to disable in the first pile?” [`internal, b3_controllability.py`]. Now the ninth policy, the one the whole room rests on: *no net_egress after in_context_read of a secret*. It mentions an event from the second pile, which is what makes it look like a counterexample, but read what it actually has to disable and the answer is `net_egress` and nothing else — the read is permitted and merely recorded as taint. One event to disable, that event in the first pile, so the count says regimentable, and the checker agrees. Now you try: classify *no in_context_read of a secret* the same way, in two sentences. (Exercise 3.5.)

The kernel chapter proved that a prefix-closed safety policy is regimentable exactly when it is controllable with respect to the uncontrollable alphabet, and classified the agent's events accordingly: the mediation layer can veto `fs_write`, `net_egress`, `exec_tool`, `git_push`, and `spawn_child`, and can at best observe `model_emit_token`, `in_context_read`, and `internal_plan`. The decisive case for this chapter is the compound rule *no net_egress after in_context_read of a secret*, and the checker's verdict is that it is regimentable: the policy permits the uncontrollable read, records it as taint, and gates only the controllable egress [`internal, b3_controllability.py`]. Gating the uncontrollable trigger, the model reading, thinking, or emitting, would be unregimentable; gating the controllable effect is not.

Figure 3.8 places the same five policies on the controllability x witnessing quadrant this book already uses for the general theorem, so the compound rule's trigger and its enforcement point can be seen to sit in different cells.

That verdict is the whole sealed-room design in one line, and it is

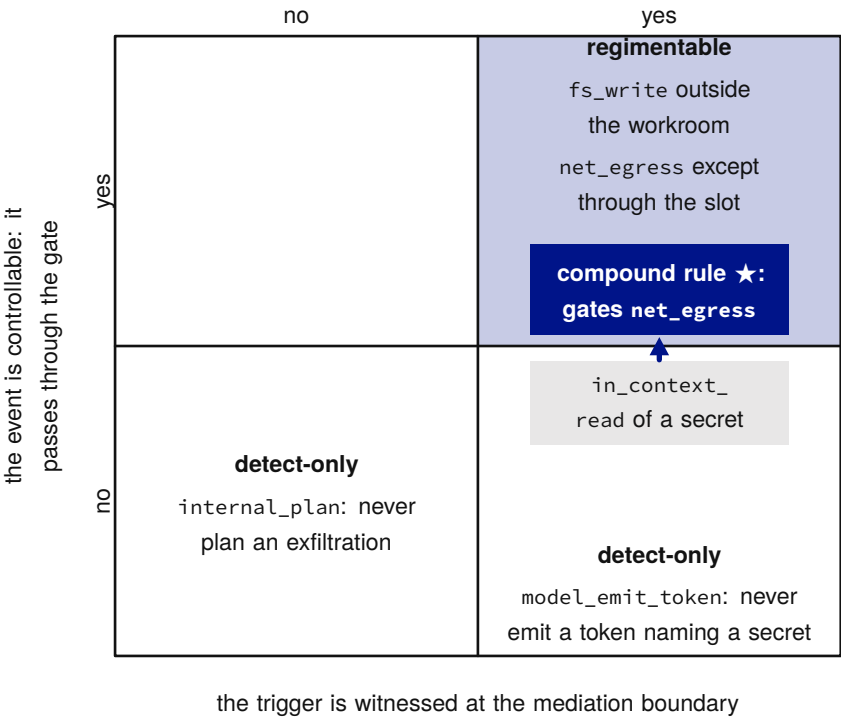


Figure 3.8: The room’s alphabet placed on the controllability x witnessing quadrant of *The Single-Writer Kernel*. Top right, shaded: controllable and witnessed, hence regimentable – fs_write and net_egress can simply be vetoed. Bottom left: neither controllable nor witnessed – internal_plan is a thought, detect-only forever. Bottom right: witnessed but not controllable – model_emit_token can be observed but not gated before it happens. The compound clean-room rule is the decisive case: its trigger, in_context_read of a secret, sits in the bottom-right cell (uncontrollable, witnessed), but the rule forbids nothing there – it gates only the controllable net_egress that follows, in the shaded cell, which is why the theorem calls it regimentable. Column membership reproduces from the checker; row membership is by inspection of each trigger [internal, b3_controllability.py].

Table 3.4: Five policies over the room’s alphabet, classified by the controllability theorem. A policy is regimentable exactly when the events it forbids are events the mediation layer can veto; the compound clean-room rule is regimentable because it forbids only the egress.

Policy	Constrains	Alphabet	Verdict
no fs_write outside the workroom	fs_write	controllable	regimentable
no net_egress except through the slot	net_egress	controllable	regimentable
never plan an exfiltration	internal_plan	uncontrollable	detect-only
never emit a token that names a secret	model_emit_token	uncontrollable	detect-only
no net_egress after in_context_read of a secret	read (trigger), egress (gate)	mixed	regimentable: taint, then gate

the computational vindication of the correction in Section 3.1. Whole-worker taint is exactly what the theorem licenses: granting Hypothesis 3.1.1, the sound over-approximation is to taint *everything* after the first secret read and spend all enforcement on the one boundary where enforcement is possible. This is why *every release explicit, gated, bounded* replaces “mathematically cannot phone home”: the latter quantifies over uncontrollable events and is therefore not a theorem about any implementable supervisor; the former quantifies over the mediated channel and is Theorem 3.3.1.

What it buys. The two results are the same boundary proved from two independent directions. Controllability says the room *must* gate the channel and not the token, since no alternative design is enforceable; two-run equivalence says gating the channel *suffices* for the contracted secrecy. Together they close the design question. The taint-drop-egress mechanism of the naive pitch survives demoted to what it really is: a containment layer beneath the gate, the `CONFINED` rung of the assurance ladder, not the guarantee. The tool-level instruments of *The Single-Writer Kernel*, the guard, the shim, and the hook harness that rides inside a vendor’s agent loop, sit below even that rung: they are the Observed and Coordinated modes, and the room replaces them rather than extending them, because a hook a process can decline to invoke gates nothing. What this ladder grades — argued at length where the modes are bought, in *The Single-Writer Kernel* — is what an adversary would have to defeat for one runtime claim about one mechanism to be false. That is a third axis, and it is independent of the two public gradings it resembles: Sheridan and Verplank’s levels of automation, refined into four processing stages by Parasuraman, Sheridan and Wickens, grade how much of the decide-and-act loop the machine holds rather than the human [236, 194], and the Common Criteria’s evalua-

tion assurance levels grade how much independent evaluation effort a security claim received before deployment [120]. Neither says which component's failure would falsify a particular record, which is the only question this ladder asks.

Controllability is relative to the mediated alphabet: partial observation shrinks the regimentable set (Lin–Wonham [151]), and policies that reference unmediated *state* need that extension, which the kernel chapter states. Detect-only is not hopeless; it is a different business, post-hoc audit and slashing, which *The Bonded Commons* treats. And complete mediation, that every effect passes through the layer, is the deployment assumption the two remaining links inherit: it is an attested property of the workroom, not a theorem.

WHERE THIS STOPS

3.5 The budget is a ledger, and the ledger conserves

Rogers, Roth, Ullman and Vadhan coined two names for a ledger like this one, tracking cumulative differential-privacy expenditure across adaptively chosen queries [219]: a **privacy odometer** reads the running total with no pre-specified ceiling, and a **privacy filter** halts before a pre-specified budget is exceeded. Because $\varepsilon_{\max}$ below is fixed in advance and every over-budget release is refused, this ledger is their filter; the odometer is the same conservation discipline without that ceiling.

Express lane: make the privacy budget a ledger, so that one atomic transition appends the record and adds the spend, and conservation of $\sigma = \sum_{\Lambda} \varepsilon_i$ survives every interleaving, because the single writer makes every concurrent history observationally sequential; the statement is Theorem 3.5.1 and the checked instance is Property 3.5.2.

The scene, resumed. Derek's data never leaves the room; what leaves is a stream of gated releases, each carrying a differential-privacy cost ε_i . The contract says the total never exceeds $\varepsilon_{\max}$. But releases race: two of Erin's jobs invoke the gate concurrently, and the classic gap between "is there budget?" and "spend it" is exactly where a meter drifts from its journal. The intuition is double-entry bookkeeping: balance and journal updated in one stroke, so an auditor summing the journal always reproduces the balance. The misread to preempt: conservation governs *recorded* spend. That all spend is recorded is the complete-mediation invariant of Section 3.2, which is why the enforceability link comes first.

Differential privacy in one paragraph. A release is ε -private when swapping any one record of D changes the probability of every possible output by at most a factor e^ε ; ε is the price of the release, in a currency that adds. Two releases cost $\varepsilon_1 + \varepsilon_2$ (sequential composition); k releases

of ε each cost at most $\varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^\varepsilon - 1)$ except with probability δ' (advanced composition); and a privacy filter is the bookkeeper that refuses the release which would overdraw the budget. The ledger of this section is that bookkeeper made explicit, the way double-entry accounting makes a balance explicit.

THEOREM 3.5.1 ε -conservation. State = (σ, Λ) with Λ append-only. The only spending transition is $\text{release}(\varepsilon_i)$: if $\sigma + \varepsilon_i \leq \varepsilon_{\max}$ it *atomically* appends $(\varepsilon_i, \text{artifact hash}, \text{policy hash})$ to Λ and sets $\sigma += \varepsilon_i$; otherwise it refuses and changes nothing. Then every reachable state satisfies $\sigma = \sum_{\Lambda} \varepsilon_i$ and $\sigma \leq \varepsilon_{\max}$, including under concurrent invocation, because the kernel's single-writer serialization makes any concurrent execution observationally equal to some sequential interleaving.

Corollary. The ledger is a *privacy filter* in the sense of Rogers, Roth, Ullman and Vadhan [219]: a stopping rule that halts before a pre-specified budget is exceeded. What licenses $\sigma \leq \varepsilon_{\max} \Rightarrow (\varepsilon_{\max}, 0)$ -differential privacy under adaptive, heterogeneous spend is their filter theorem, that summing *realised* parameters is valid, not the textbook sequential-composition theorem, which fixes the parameters in advance. **What this theorem does not say.** The equality $\sigma = \sum_{\Lambda} \varepsilon_i$ is bookkeeping over *admitted* releases, and it is exact about a quantity that is deliberately syntactic: how much declared budget has been spent. It says nothing about what an admitted artifact *means*. A worker that computes $t = f(s)$ and submits t so the gate honestly releases $g(t)$ spends one ε_i like any other caller, and the ledger balances afterwards exactly as it did before — the semantic content of that release is Section 3.7's $q \cdot b$ residual, which is priced rather than prevented, and the closest thing to a non-interference guarantee in this chapter is Property 3.3.1, a separate and model-checked claim that holds under a committed-input hypothesis this theorem never assumes. Accounting consistency and non-leakage are two different properties, and only the first of them is proved here.

Proof. Induction over the sequential interleaving. Base: $\sigma = 0$ and $\Lambda = ()$, so the equality holds and the bound holds. Step: a refused release changes nothing; an admitted release passed the guard $\sigma + \varepsilon_i \leq \varepsilon_{\max}$, so the bound is preserved, and it appends and adds in one transition, so the equality is preserved. Non-spending transitions touch neither component. Concurrency reduces to the sequential case by the linearizability of the single writer, which *The Single-Writer Kernel* proves for exactly this setting. $\square$

Which composition bound to quote. The same authors show that substituting the advanced-composition bound into such a filter is *not*

valid: the heuristic “run the advanced bound on whatever the ledger actually spent” breaks when the parameters or the stopping point are adaptively chosen. This chapter therefore quotes the advanced bound $(\sqrt{2k \ln(1/\delta')} \varepsilon + k\varepsilon(e^\varepsilon - 1), \delta')$ [72] only for engagements in which k and the per-release ε are fixed in the work order, where the schedule is non-adaptive and the bound applies directly, and uses the adaptively valid filter of Whitehouse et al. [271], which recovers the advanced-composition rate at worse constants, otherwise. Getting this wrong is not conservative: it certifies a smaller ε than the run is entitled to.

Whom the certificate binds. The differential-privacy reading of σ presupposes that each recorded release *is* an ε_i -private mechanism. Against the malicious worker of Section 3.7, one choosing b bits adversarially, that presupposition fails outright: such a worker is not running a private mechanism, and its recorded ε_i is a number it declared about itself. In that regime $\sigma \leq \varepsilon_{\max}$ certifies nothing, and the $q \cdot b$ capacity account of Section 3.7, not the ledger, is the operative bound. The two accounts are for two different adversaries and the chapter owns both, drawn side by side in Figure 3.9.

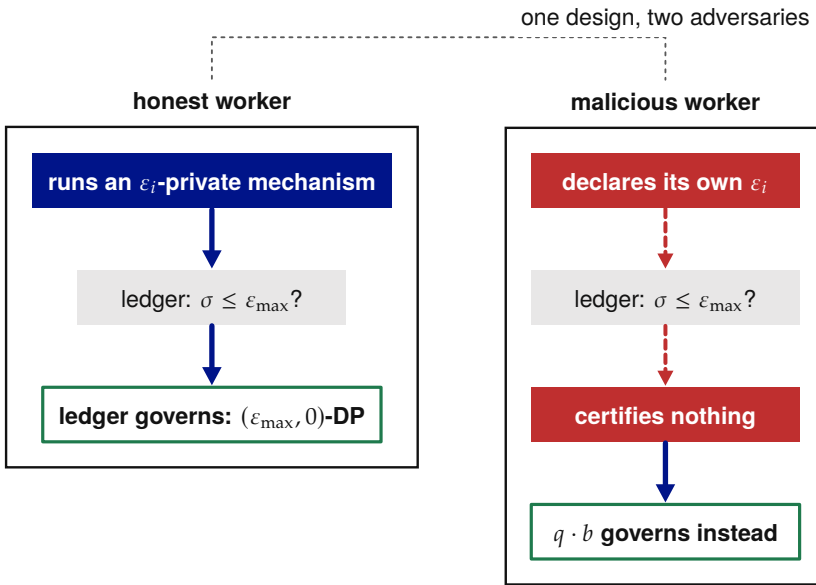


Figure 3.9: The same ledger code, read against two adversaries. Left, an honest worker’s recorded ε_i is the true cost of an ε_i -private mechanism, so the ledger’s guard $\sigma \leq \varepsilon_{\max}$ licenses genuine $(\varepsilon_{\max}, 0)$ -differential privacy (Theorem 3.5.1). Right, a malicious worker declares whatever ε_i it likes, so the identical guard certifies nothing about real leakage; the operative bound for that adversary is instead the $q \cdot b$ capacity account of Sec. 3.7. The ledger box is drawn identically in both panels because it runs identically in both cases — only what its output is entitled to mean changes. [internal]

Systems that schedule privacy budgets across concurrent jobs make the corresponding assumption explicit, that the analyst is trusted to implement the mechanism correctly [161]; this chapter declines that assumption about Erin’s worker, which is precisely why it cannot lean on the ledger alone. Stating the two accounts in one currency needs the correspondence between ε and min-entropy of Alvim et al. [9], which is not discharged here.

MODEL-CHECKED PROPERTY 3.5.2 The ledger’s invariants on the two-client instance. Two concurrent clients racing programs (2,2) and (1,2) against $\varepsilon_{\max} = 4$: all 15 reachable states satisfy both invariants under every interleaving, and 2000 random instances of programs, budget, and interleaving show no violation. The state count is order-sensitive: a denied release still advances the client’s program counter, so distinct commit orders are distinct states; as a multiset the answer would be 11. **Mutations.** Spend-without-append breaks conservation in one step. The torn write, which logs ε but adds $\varepsilon - 1$, breaks it in one step and in three steps drives the *recorded* log to $\sum_{\Lambda} = 5 > \varepsilon_{\max} = 4$: the undercounting balance silently admits auditable over-spend, so atomicity is what makes the budget promise auditable. Reverting the mutations restores both invariants and the 15 states [internal, a3_epsilon_ledger.py].

Figure 3.10 grids both mutants against both invariants, with the step count each is caught in.

Numbers by hand — when the advanced composition bound starts to pay. At $\delta' = 10^{-6}$: for $k = 32$ releases at $\varepsilon = 0.1$, basic composition gives $32 \times 0.1 = 3.20$ while the advanced bound gives 3.31, so the advanced bound is not yet paying; at $k = 128$, $\varepsilon = 0.05$, basic gives 6.40 but advanced gives 3.30, and it pays. The crossover tells the contract writer exactly which accounting to quote at which engagement length: below it, the exact, δ -free sequential bound; above it, the $\sqrt{k}$ -scaled advanced bound. *Now you try:* at $k = 8$, $\varepsilon = 0.5$, which bound is smaller, and at $\varepsilon = 0.1$, at which k does the advanced bound first pay? (Exercise 3.8.) Figure 3.11 plots both bounds against k and marks this exact crossing.

What it buys. The contract’s budget line becomes an auditable invariant with the same proof shape as the bond-conservation result of *The Bonded Commons*, and the mutation suite shows the invariant has teeth: both canonical atomicity breaks are caught with shortest traces, and the torn write’s deeper crime, recorded over-spend past the contracted maximum, is exhibited concretely.

■
a3_epsilon_ledger.py · 20260816 · VERIFIED

	spend-without-append	torn write
ledger balance the recorded sum	caught 1 step	caught 1 step
budget cap spend at or under the limit	not exercised	caught 3 steps

Figure 3.10: The two canonical atomicity breaks against the ledger’s two invariants: spend-without-append (a release that adds to the balance without journaling it) and the torn write (a release that journals the wrong amount). Each marked cell gives the shortest trace that catches the break; the ledger’s balance is the invariant both mutants break immediately, but only the torn write’s deeper crime – recorded spend past the contracted maximum – takes three steps to show, and spend-without-append never exercises the cap at all. Model-checked instance: two concurrent clients racing programs (2, 2) and (1, 2) against a budget cap of 4. All 15 reachable states (order-sensitive; 11 as a multiset) satisfy both invariants under every interleaving, and 2000 random instances of program, budget and interleaving show no violation; the torn write logs the correct amount but adds one unit less, so by 3 steps the recorded ledger reaches a total of 5, over the cap of 4 – an auditable over-spend. Reverting either mutation restores both invariants and the 15 states [internal, a3_epsilon_ledger.py].

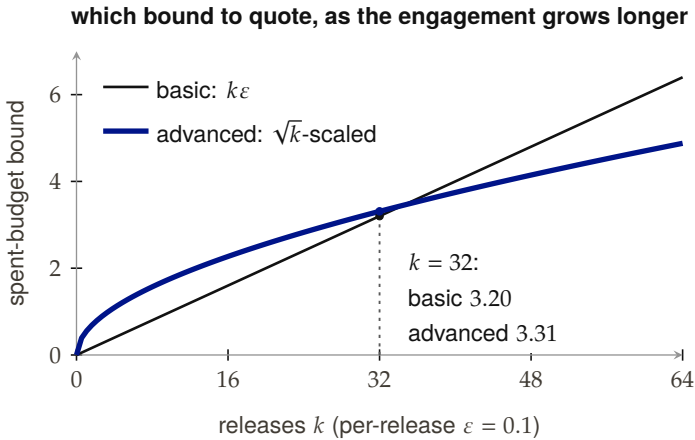


Figure 3.11: The basic and advanced ε -composition bounds on cumulative spend as functions of the number of releases k , at the per-release $\varepsilon = 0.1$ the chapter uses for this comparison. At $k = 32$, the chapter’s reported comparison, basic composition (3.20) is still smaller than the advanced bound (3.31): below the crossover a contract should quote the exact, δ -free basic bound, and only past it the square-root-scaled advanced bound – the same rule at $\varepsilon = 0.05$ gives $k = 128$: basic 6.40 versus advanced 3.30, where the advanced bound has come to pay.

Conservation governs recorded spend; that all spend is recorded is the complete-mediation invariant, so the enforceability link is the prerequisite, not a nicety. Advanced composition pays only past the crossover in k . And the theorem conserves the meter, not the meaning: choosing per-release ε_i honestly is the mechanism’s obligation, outside this ledger. Differential privacy bounds individual influence in aggregates; it does not protect the whole dataset, nor Erin’s model.

WHERE THIS STOPS

3.6 Canaries with a power curve and a clock

Express lane: independent per-canary misses turn k smuggled canaries into detection probability $1 - \beta^k$; uniform planting turns leak size into a hypergeometric operating curve; and Wald’s sequential test against the permutation null turns leak intensity into an expected time to detection; the statements are Theorems 3.6.1 and 3.6.2.

Intuition: dye packs and quality control. Dye packs in bank bundles: each pack fails alone with probability β , so grabbing k dyed bundles fails to stain with probability β^k ; how many dyed bundles a thief grabs is set by how much he takes and how densely packs are planted. The sequential half is quality control: each gate output is a Bernoulli trial, and the sequential probability ratio test is the optimal stopping rule for “is

this stream leaking?” The three links before this one govern what the gate is *allowed* to pass; this one measures what happens when something tries to pass anyway. “We have canaries” is a mood; a security officer buys a number. The misread to preempt: the two theorems answer two different questions, and a single-shot exfiltration is the first theorem’s job, not the second’s.

THEOREM 3.6.1 Power. With per-canary false-negative rate β , independent across canaries, a leak carrying k canary spans is detected with probability $1 - \beta^k$. Under uniform planting of c canaries in an n -span corpus, a leak of m spans carries $K \sim \text{Hypergeom}(n, c, m)$ canaries, so $\Pr(\text{detect} \mid m) = 1 - \mathbb{E}[\beta^K] \approx 1 - (1 - \frac{c}{n}(1 - \beta))^m$, the operating curve.

THEOREM 3.6.2 Latency (Wald). Gate outputs are Bernoulli with canary-hit rate p_0 under the no-leak permutation null and $p_1 > p_0$ under exfiltration. The sequential probability ratio test with false-alarm target α and miss target γ stops, under leak, after expected $\mathbb{E}_1[N] = \left[(1 - \gamma) \ln \frac{1 - \gamma}{\alpha} + \gamma \ln \frac{\gamma}{1 - \alpha} \right] / \text{KL}(p_1 \parallel p_0)$ outputs, analogously $\mathbb{E}_0[N]$ under the null, and is optimal in the Wald–Wolfowitz sense [264, 266]. The miss target is written γ because β is already the per-canary miss rate and b is the release-channel width of Section 3.7.

This is textbook sequential analysis pointed at a live security detector, and the move has a precedent worth naming: Jung, Paxson, Berger and Balakrishnan’s threshold random walk did the same thing to portscan detection in 2004, modelling the benign-or-malicious question as a random walk between two hypotheses and reading a guaranteed false-positive-against-latency trade-off straight off Wald’s test [126]. The canary is that detector with the network swapped for a gate.

Proof idea. Power is independence and counting: a leak is missed only if every carried canary is missed, and the number carried under uniform planting is the hypergeometric draw of m spans from n with c marked. Latency is Wald’s identity: the log-likelihood ratio drifts at rate $\text{KL}(p_1 \parallel p_0)$ per output under the leak, and the test stops when it crosses the boundaries set by the two error targets, so the expected crossing time is the expected boundary value divided by the drift; overshoot past the boundary is what the identity ignores.

Verification. For Derek’s corpus of $n = 10^4$ records seeded with $c = 100$ canaries (density one percent), against a channel through which each smuggled canary evades the detector with probability $\beta = 0.2$: a



Wald’s sequential probability ratio test, made for wartime quality control, is still the fastest way to decide that a canary has tripped at a guaranteed error rate.

100-span leak is caught with probability 0.554, an 800-span leak at 0.999; exact hypergeometric, binomial approximation, and simulation agree to Monte-Carlo precision across the curve, and the $1 - \beta^k$ line is confirmed at $k = 1, 2, 5$ (analytic 0.800, 0.960, 0.99968 against simulated 0.8016, 0.9608, 0.99967). The sequential test at $p_0 = 10^{-3}$, $p_1 = 10^{-2}$, $\alpha = 0.01$, $\gamma = 0.05$: Wald $\mathbb{E}_1[N] = 296.9$ against simulated 359.0, the gap being boundary overshoot, which only helps the error rates (realized miss $0.048 \leq 0.05$, false alarm $0.005 \leq 0.01$); $\mathbb{E}_0[N] = 431.9$ against simulated 438.1. Tenfold leak intensity cuts time to detection about thirtyfold [internal, `a4_canary_sprt.py`].

Numbers by hand — three canaries, one operating curve. At $\beta = 0.2$, two smuggled canaries are caught with probability $1 - 0.04 = 0.96$ and three with $1 - 0.008 = 0.992$ [verified]. The operating curve at $m = 100$: $1 - (1 - 0.01 \times 0.8)^{100} = 1 - 0.992^{100} = 0.553$ by the binomial approximation, against 0.554 exact. *Now you try:* at the same β , what does a five-canary leak survive with, and can you recover Wald's 296.9 from the formula with a pocket calculator? (Exercises 3.9 and 3.10.) Figure 3.12 plots the power curve and both expected run lengths.

■
`a4_canary_sprt.py` ·
20260816 · INTERNAL

What it buys. The contract can *quote* $\Pr(\text{detect})$ as a function of leak size and expected alarm latency per leak intensity, so the exfiltration bond of *The Bonded Commons* is scoped to its real job, funding detection and response, rather than pretending to cover unbounded breach damage.

Independence and *secrecy* of planting are decisive: an adversary who obtains the canary list with probability 0.3 and strips all of them drops measured power from 0.9997 to 0.698 at $k = 5$ [internal, `a4_canary_sprt.py`]; canary secrecy is part of the security boundary, priced explicitly. β is a measured property of the suppressor, not an assumption. The Wald stopping-time formulas are approximations; overshoot inflates $\mathbb{E}[N]$ by about a fifth here, conservatively. And the sequential test detects *statistical* leak signatures; a single-shot exfiltration is Theorem 3.6.1's job.

WHERE THIS STOPS

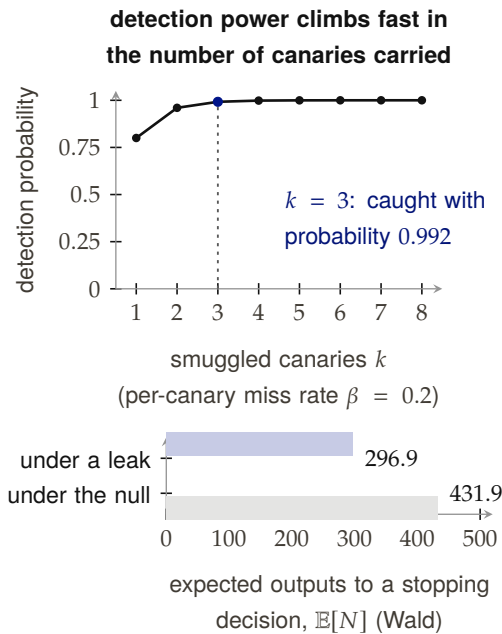


Figure 3.12: Top: the power curve, detection probability against canaries carried, at the measured per-canary miss rate $\beta = 0.2$; the chapter’s anchor, three canaries caught with probability 0.992, is marked. Bottom: the sequential test’s expected run length at a null hit-rate of 0.001, a leak hit-rate of 0.01, a false-alarm target of 0.01, and a miss target of 0.05 – Wald’s formula gives 431.9 expected outputs under the no-leak null and 296.9 under exfiltration, idealized against a simulated overshoot of 438.1 and 359.0 respectively, which only helps the realized error rates.

3.7 The leakage budget: $q \cdot b$ bits, before timing

Numbers by hand — twelve thousand eight hundred bits, and the only two dials. A work order grants Erin a 64-bit status schema per job, over 200 jobs. Multiply, and that's the account: $200 \times 64 = 12,800$ bits of adversarial capacity, before timing. Two things about that arithmetic are worth a second look, and neither needs a model. First, the account is a *product*, so the contract writer has exactly two dials and they trade off against each other — the same 12,800 bits fall out of a narrower schema over a longer engagement — and the only setting that drives the product to zero is $b = 0$, which closes the feedback channel that was Erin's reason for signing in the first place. (That is what Section 3.8 means when it says "zero" is available only at $b = 0$; it is a price, not an impossibility.) Second, and this is the uncomfortable half, Theorem 3.3.1 does not narrow those 12,800 bits by a single one: it guarantees they left through the declared gate, not that a worker computing $t = f(s)$ and submitting t chose them innocently. The room sells a bounded account. It does not sell an innocent one. *Now you try*: the same work order lets each job return at one of three scheduled times — what does that add to the total? (Exercise 3.12.)

The four links compose into one information-theoretic account, and the account is deliberately unromantic. It is also where the chapter pays three debts it took on earlier: the laundering residual left open by Theorem 3.3.1's committed-input hypothesis, the malicious worker against whom $\sigma \leq \varepsilon_{\max}$ certifies nothing, and the arbitrary-telemetry channel disclaimed in Section 3.8. All three are the same debt seen from three sides, and it has one price, drawn converging in Figure 3.13.

If an Erin-visible channel permits b freely chosen bits per job, a malicious worker, one that launders by computing $t = f(s)$ and submitting t so that the gate honestly releases $g(t)$, can exfiltrate up to b bits through it: the noninterference property guarantees only that those bits pass through the declared gate, not that they are innocent. Across q jobs the raw capacity is $q \cdot b$ bits; choosing among s timing slots adds up to $\log_2 s$ bits per event, which is why timing sits outside the possibilistic model. That timing term is not an estimate of ours: it is the bound Askarov, Zhang and Myers [15] prove for *predictive mitigation*, where a scheduler quantises releases into a bounded set of permitted delays and the leakage through the resulting channel is at most the logarithm of the number of distinguishable slots. The mitigation this section describes — scheduled release into coarse buckets — is theirs, and so is the reason it works: an adversary who can modulate duration continuously has an unbounded channel, and the only thing that bounds it is destroying the resolution, never observing the content. In Smith's vocabulary [241], the design does not drive min-entropy leakage to zero; it makes leakage a *declared, metered* quantity. Fixed schemas, padding,

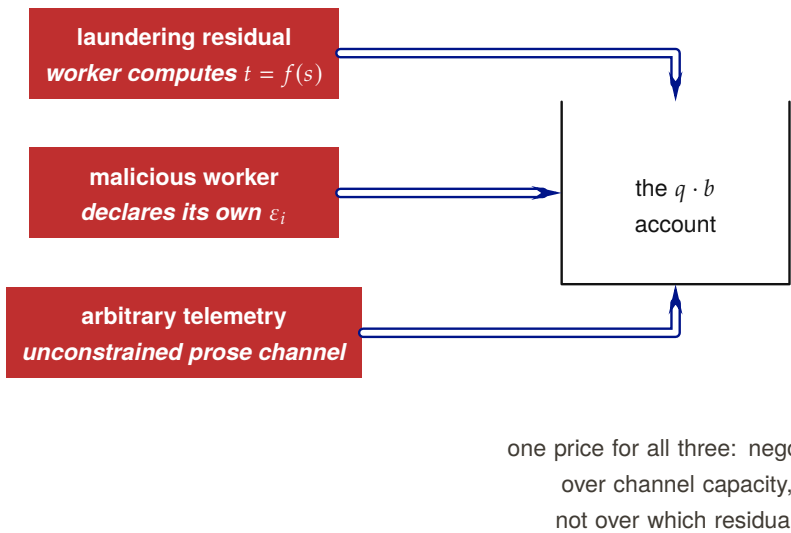


Figure 3.13: Three gaps opened earlier in this chapter, one account that prices all of them. The laundering residual (Fig. 3.7, Sec. 3.3) lets a worker choose what it submits; the malicious worker (Fig. 3.9, Sec. 3.5) lets it choose what it reports spending; arbitrary natural-language telemetry (Sec. 3.8) lets it choose the words. All three hand an adversary free bits through a channel the non-interference property does not forbid, and all three are priced the same way: by the $q \cdot b$ capacity account of this section, not by three separate mechanisms. [internal]

coarse statuses, query limits, and scheduled release turn an unlimited covert channel into a leakage budget the contract states, the ledger of Section 3.5 meters, with differential-privacy composition giving the aggregate-feedback portion its formal semantics, and the canaries of Section 3.6 police for evasion around.

The design rule that falls out is quotable: *never negotiate over “does this output contain a secret”; negotiate over channel capacity*. The first question is undecidable in practice for prose; the second is arithmetic, and it is the arithmetic the example above did in one multiplication. A granted capacity is a quantity a contract can price, bond, and canary-police exactly, which is the whole of what the four links, between them, make possible.

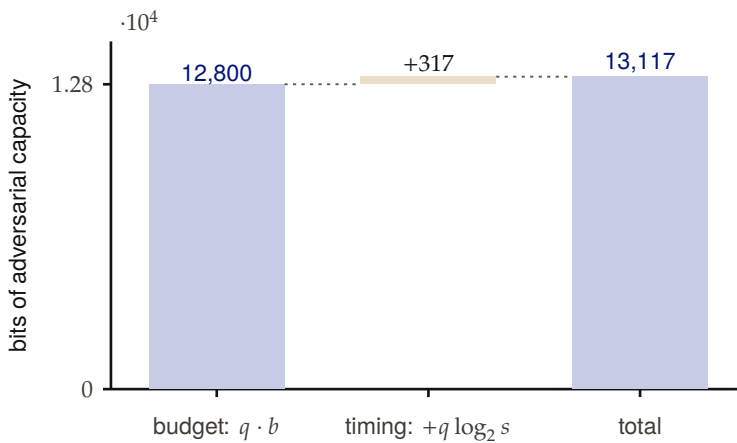


Figure 3.14: The leakage budget as a running total, for the chapter’s own worked engagement: a 64-bit status schema over 200 jobs. The first bar is the declared, arithmetic budget $q \cdot b = 200 \times 64 = 12,800$ bits, the number the design rule of Sec. 3.7 says to negotiate over. The second, dashed and in the warn hue, is the separately priced timing term: three scheduled return times add $q \log_2 s \approx 200 \times 1.585 \approx 317$ more bits, since when a job returns is itself a channel the possibilistic model does not cover. The third bar is the true total, about 13,117 bits, both terms summed. [hand-derived; Exercise 3.12]

3.8 Limitations and boundaries

None of the following is promised, and a contract that implies any of them overclaims.

Table 3.5 lists six of them: for each, what mechanism is responsible for the gap, and where – if anywhere – the residual is priced.

One more limit sits outside even that table: bounds on minds. A human who reads a released aggregate may still remember it; the system bounds channels, not memories.

Table 3.5: Six things this design does not promise, why not, and where the residual is priced. A contract that implies any of these overclaims.

Not promised	Why not	Residual priced by
Zero model leakage under unlimited adaptive queries to Derek	Derek <i>intentionally observes</i> evaluations of M ; unlimited black-box access enables model extraction	out of model: bounded only by the contracted query-and-output interface
Zero data leakage with arbitrary natural-language telemetry to Erin	that channel is exactly the $q \cdot b$ budget; “zero” needs $b = 0$	the $q \cdot b$ budget (Sec. 3.7)
Elimination of timing, cache, speculative, firmware, or physical channels	the noninterference guarantee is possibilistic	out of model; padded and scheduled, not closed
Confidentiality through an unapproved external model or tool provider	a remote API that reads plaintext is another trusted reader	not priced: the model must run inside the confidential domain, or the guarantee does not apply
Correctness of the answer	the receipt proves what the measured system reported, not that it is true	out of model
Literal deletion of information already observed, availability under refusal, or collusion resistance	each needs a strictly stronger threat model	needs MPC or homomorphic backends, not priced here

The lift. Theorem 3.3.1 is exhaustive on a finite model; the general claim over unbounded state is a proof obligation with a known shape. Rushby’s channel-control formulation [122] reduces noninterference for a transition system to three local *unwinding conditions*, output consistency, step consistency, and local respect, each a per-transition lemma amenable to mechanization, and the Isabelle Archive of Formal Proofs already carries stateful intransitive-noninterference developments in exactly this lineage. The finite model is the blueprint: its local-respect check already holds mechanically, its declassification steps are the intransitive downgrader domain of Rushby’s framework, and the lift consists of restating the transition relation abstractly and discharging the three lemmas once, for all depths and all secrets. One precision the lift must get right from the start: Rushby’s soundness result for these conditions is with respect to intransitive purge, but van der Meyden shows the conditions are sound *and complete* for the strictly stronger TA-security property, which additionally preserves the *order* of permitted transmissions [270]. Discharging the three lemmas therefore certifies TA-security, and the lift should state that target explicitly. Two extensions of the finite model are open alongside the lift: a submit that carries a payload, so the mutation suite can exhibit laundering, and a partial-observation variant of the enforceability check for policies that reference unmediated state. The status of every claim in this chapter is: the two exhaustive checks and the power-and-latency agreement are **ATTESTED** on their scripts; the room itself is a **CONFINED** design whose complete-mediation invariant is attested per deployment, not proved.

RECALL

1. Without looking: state the sentence that replaces “mathematically cannot phone home,” and name the theorem that makes each of its three adjectives true.
2. Which link must hold before the conservation theorem says anything about the deployment, and why?
3. A contract grants a 32-bit status per job over 400 jobs. What is the adversarial capacity before timing, and which link polices it?

Exercises 3.1–3.12, p. 165.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. A **sealed room** lets an agent compute on data it may not export: the data owner and the model owner each get their guarantee from a fence they can inspect, not from trust in the other.
2. **Taint does not survive a language model.** Token-level information flow through the model is not soundly definable, so the room does not try; it fences the room and gates the slot.
3. **Two fences, two gates:** the work order fixes what may enter, the slot fixes what may leave, and the model in between is treated as an opaque function of both.
4. **Noninterference is checked by running the world twice** (self-composition, in the verification literature’s word): identical except for the secret, the two runs must agree on everything that leaves through the slot, modulo the declassification the gate performs. The finite clean-room model passes this check to depth seven.

5. The gate sits on the **only enforceable boundary**: a policy over the agent’s behavior is enforceable exactly when its trigger is witnessed at the slot; anything decided inside the room is detect-only.
6. **The privacy budget is a ledger that conserves**: every release debits ε , the ledger’s total never exceeds $\varepsilon_{\max}$, and a torn write is caught by the invariant within three steps on the two-client instance.
7. **Canaries have a power curve and a clock**: with per-canary miss probability β and k independent canaries, the chance a smuggled item passes is β^k (0.008 at $\beta = 0.2$, $k = 3$), and Wald’s sequential test bounds how long detection takes.
8. The whole channel is priced before timing: $q \cdot b$ **bits** may leave through q queries of b bits each, and the room’s guarantee is that budget, not silence.

3.9 Exercises

§3.8 LIMITATIONS AND BOUNDARIES

- 3.1

CHECK ★ With $g(s) = s \bmod 2$ over secrets $\{0, \dots, 7\}$, how many unordered secret pairs must a sound room keep Erin-indistinguishable, and how many may differ at release steps?
- Solution p. 536

Table 3.6: How many unordered secret pairs a sound room must keep Erin-indistinguishable forever (equal-parity), against how many may legally differ at a release step (different-parity), for $g(s) = s \bmod 2$.

Secrets	Unordered pairs	Equal-parity (hidden forever)	Different-parity (may differ at release)
4	$\binom{4}{2} = 6$	2	4
8	$\binom{8}{2} = 28$	12	16

- 3.2

TRACE ★★ Run `c1_noninterference.py` and read the first mutation’s witness. Why is the decisive pair $(0, 2)$ rather than $(0, 1)$, and at which step of the four does Erin’s view first differ?
- Solution p. 536
- 3.3

CHECK ★ The second mutation is caught in three steps. Which side property of Design invariant 3.2.1 does the mutant violate, and why is that violation detectable by the two-run check at all?
- Solution p. 536
- 3.4

OPEN ★★★ Extend the finite model so that `submit` carries a payload the worker computes from the secret, and state what the checker would then have to quantify over to exhibit the laundering attack. Why is the delimited-release remedy, enforcement at the point of declassification, unavailable for a language-model worker?
- Solution p. 536

- 3.5 CHECK ★** Using the controllability theorem of *The Single-Writer Kernel*, explain in two sentences why “no net_egress after an in_context_read of a secret” is regimentable but “no in_context_read of a secret” is not. *Solution p. 537*
- 3.6 TRACE ★★** Reconstruct by hand the three-step torn-write crime on the two-client instance (programs (2, 2) and (1, 2), $\varepsilon_{\max} = 4$): give the sequence of releases, the balance σ after each, and the recorded sum $\sum_{\Delta}$, and show where the recorded sum exceeds the maximum. *Solution p. 537*
- 3.7 TRACE ★★** The checked instance has 15 reachable states but only 11 distinct multisets of committed releases. Explain the difference and exhibit two states that share a multiset. *Solution p. 537*
- 3.8 CHECK ★** At $\delta' = 10^{-6}$: which bound is smaller at $k = 8$, $\varepsilon = 0.5$? And at $\varepsilon = 0.1$, what is the smallest k at which the advanced bound falls below basic composition? *Solution p. 537*
- 3.9 CHECK ★** At $\beta = 0.2$, with what probability does a five-canary leak survive undetected? Then use the binomial approximation of the operating curve at $n = 10^4$, $c = 100$, $m = 100$ and compare with the exact value 0.554. *Solution p. 537*
- 3.10 TRACE ★★** Recover Wald’s expected stopping time under leak, $\mathbb{E}_1[N] = 296.9$, from Theorem 3.6.2 at $p_0 = 10^{-3}$, $p_1 = 10^{-2}$, $\alpha = 0.01$, $\gamma = 0.05$. Why does the simulation report 359 instead? *Solution p. 537*
- 3.11 OPEN ★★★** An adversary obtains the canary list with probability 0.3 and strips every canary it knows. Measured power at $k = 5$ falls from 0.9997 to 0.698. Propose one contract clause and one mechanism change that restore most of the lost power, and say what no redesign can restore. *Solution p. 538*
- 3.12 CHECK ★** A work order grants Erin a 64-bit status schema per job over 200 jobs, and each job may return at one of three scheduled times. What adversarial capacity has the contract granted, before and after timing? *Solution p. 538*

3.10 History and references

Almost nothing in this chapter is new machinery; what is new is which pieces were bolted to which, and the honest accounting of what the bolted-together thing does not promise. It is worth saying where each piece came from.

Noninterference and its proof technique. Noninterference is Goguen and Meseguer’s, stated at Oakland in 1982 as a property

of a security *model* rather than of any particular mechanism [90], and given two years later the proof technique that made it usable: replace the quantification over all runs with local *unwinding* conditions you can discharge one transition at a time [91]. Rushby's 1992 report is the version this chapter actually leans on, because it separates transitive from intransitive policies — which is precisely what allows a room to have a declassifier and still have a theorem — and because it fixes the three unwinding conditions, output consistency, step consistency and local respect, that Section 3.8 names as the lift's proof obligation [122]. Van der Meyden later showed those conditions are not merely sound but sound *and* complete, and for the strictly stronger, order-preserving TA-security rather than for the intransitive purge Rushby was aiming at [270]; that is why the lift should be written to claim TA-security rather than to settle for what it was pointed at. The checking move itself, running the world twice and stating an ordinary invariant about the pair, is self-composition, published by Barthe, D'Argenio and Rezk in 2004 [23]; they apply it to programs with a logic, this chapter applies it to a finite model with a Python script, and the difference in ambition should not obscure that it is the same move.

Declassification, and why the room needs the word. That an information-flow certifier faces a choice between label creep and unsoundness is the oldest result in this lineage, from the Dennings' certification lattice [61], with the modern restatement — a sound purely dynamic monitor must be strictly more restrictive than a static analysis — due to Russo and Sabelfeld [221]. The escape hatch itself is Sabelfeld and Myers's delimited release [222], and so, importantly, is the laundering attack this chapter cannot rule out: they named it and closed it by enforcement, requiring that no variable used in a declassification be updated before it, which a compiler can check because the declassified expression is syntax. This chapter's worker emits arbitrary prose rather than an expression, so the remedy is unavailable and the residual is priced instead — that substitution, enforcement traded for a capacity account, is the chapter's own move and its own liability. The vocabulary for saying which dimension of release a mechanism actually controls is Sabelfeld and Sands's [223]; by their taxonomy the release ledger answers *what*, *who* and *where*, and leaves *when* open. Owner-indexed labels under mutual distrust, which is the shape of the four-state lattice counted in Section 3.2, are Myers and Liskov's decentralized label model [177].

The monitor, and the boundary it can hold. The in-guest monitor and Derek's outer gateway are a reference monitor in Anderson's 1972 sense — always invoked, tamper-resistant, small enough to be examined [78] — enforcing the sort of owner-indexed access structure Lampson's *Protection* gave operating systems two years later [204]. What this

chapter adds to that inheritance is not a better monitor but a sharper statement of where a monitor can be placed at all: the controllability result is proved in *The Single-Writer Kernel* and merely applied here, and the caveat that partial observation shrinks the enforceable set is Lin and Wonham's [151]. The confinement architecture has a direct systems ancestor in Ryoan [117], which solved the two-suspicious-owners problem first and solved it more cleanly than this chapter does, at the price of a stateless module; the dual-attested key release follows the attester/verifier/relying-party pattern the IETF later standardized [98].

Pricing what does cross. Treating leakage as a quantity rather than a predicate is quantitative information flow, and the min-entropy formulation the chapter borrows its vocabulary from is Smith's [241]; the bridge between that currency and differential privacy's exists, due to Alvim and colleagues [9], and this chapter does not cross it, which is why it carries two accounts rather than one. On the privacy side, advanced composition is Dwork, Rothblum and Vadhan's [72]; the observation that a running budget needs a different theorem when the queries and the stopping point are chosen adaptively, and the odometer/filter pair that supplies it, is Rogers, Roth, Ullman and Vadhan's [219]; the fully adaptive filter that recovers the advanced rate at worse constants is Whitehouse and colleagues' [271]. That production budget schedulers assume a trusted analyst, an assumption this chapter explicitly declines about Erin's worker, is visible in the Privacy Budget Scheduling work [161]. The detector is Wald's sequential probability ratio test [264], optimal in the sense he and Wolfowitz proved three years later [266]; pointing it at a security stream is not new either, since Jung, Paxson, Berger and Balakrishnan did exactly that to portscan detection in 2004 [126], and the canary gate is their threshold random walk with the network swapped out.

What is this chapter's own, and therefore uncited. Four things. Hypothesis 3.1.1, that token-level taint through a generative model is not soundly definable, is a design rationale and not a theorem: it rests on the Denning and Russo-Sabelfeld results above, but no published result states it for generative models, so the chapter states it as its own position and places the burden of a counterexample on whoever wants one. The four-link composition read as an *evidence schedule* rather than as four redundant guarantees is the chapter's framing. The $q \cdot b$ account as the explicit price of the laundering residual — the thing the noninterference theorem leaves open and delimited release would have closed by enforcement — is the chapter's own construction, and so is the design rule it yields, that one should never negotiate over whether an output contains a secret and always over channel capacity. And the "clean room" of the framing is borrowed from industrial and legal practice for its connotation only; this chapter claims no formal ancestor for the

metaphor and proves nothing about it as such.

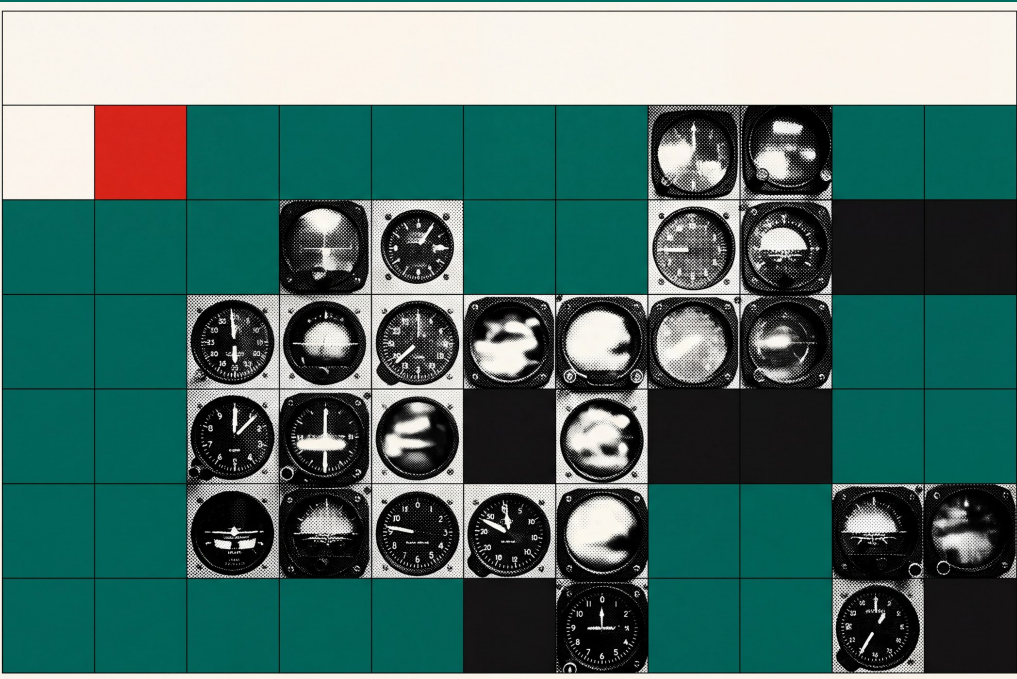
Further reading. A reader who wants the general theorem this chapter's finite model stands in for should read Rushby [122] and then van der Meyden [270] in that order, and should expect the second to change what the first looks like it was proving. A reader who wants to know why the assurance ladder in Section 3.4 is a third axis rather than a rebadging of two familiar ones should read Sheridan and Verplank [236] and its four-stage refinement [194] for the automation-level axis, and the Common Criteria's evaluation assurance levels [120] for the evaluation-effort axis; neither answers which component's failure would falsify a given runtime record, which is the only question the ladder asks.

WHAT THE LEGIBLE SWARM RECEIVES The machine now has three parts that can be kept honest: a writer with one history, an authority that arrives checked, and a room whose only exit is metered. Notice what the third part cost. Sealing execution so that a secret cannot walk out is the same act as making the inside of the computation dark to the person responsible for it — the slot is the only place anyone sees through the wall, and §3.7 metered exactly what leaves it. Part I therefore ends holding an operator who is accountable for work it can no longer watch directly. That is not a philosophical turn, it is the bill for the room, and the next chapter is the reply to it: with many such machines running at once, what must remain visible to the operator, and what does reading it cost?



The Cost of Seeing

Many machines, one person watching. Supervision has a price, and it can be measured in bits. This part prices what the operator must be able to see and proves what no summary can hide.



4 The Legible Swarm

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.

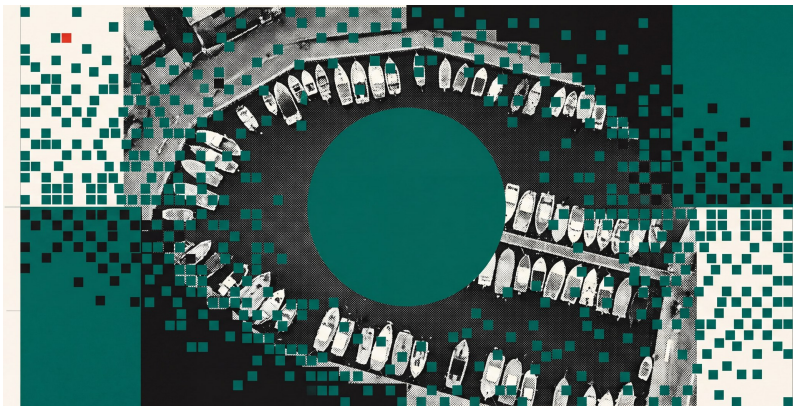
4

The Legible Swarm

The question this chapter answers

What must remain visible, and what does seeing cost?

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.



Certain forms of knowledge and control require a narrowing of vision. The great advantage of such tunnel vision is that it brings into sharp focus certain limited aspects of an otherwise far more complex and unwieldy reality.

James C. Scott, *Seeing Like a State*, 1998

WHAT MUST REMAIN VISIBLE, AND WHAT DOES SEEING COST? The previous chapter sealed the room and, in the same stroke, put the work beyond the operator’s sight; this chapter is the information-theoretic answer to that darkness. A swarm can produce more events than its operator can read. The resulting scarcity is not cured by a smoother summary: some summaries make the decisive artifact unreachable. This chapter establishes the cost of a zero-miss digest, proves why a successor and an operator require different rankings, and derives the attention rule from explicit error costs. Its strongest claims are governed by R1–R4, R14, and R16.

Table 4.1: The system as four strata rather than four products. A mark is a capability the layer provides to the layer above; reading down a column shows where each capability is born. This chapter owns the operator surface (L2) and uses the directory substrate of L1; it assumes ordered kernel state below and supplies accountable identity, reasons and outcomes to reputation and exchange above. [internal]

Layer Stratum		For whom	state	order	identity	reason	outcome	reputation	exchange
L3	economy & federation	the market			•	•	•	•	•
L2	legibility & authority (this chapter)	the operator	•	•	•	•	•		
L1	coordination protocol	the agents	•	•	•				
L0	daemon kernel	the machine	•						

Three sentences are the whole paper. **(1)** A coding-agent swarm is a state of nature; the operator consents to a *local* Leviathan to escape it. **(2)** That Leviathan rules by *legibility-with-zoom* — every summary is a lens onto a verifiable artifact, never a replacement — and the binding constraint at scale is *read-poverty*, with tokens serving at once as the swarm’s cost and its legibility engine. **(3)** “Authority” is legitimate only with a scoped, revocable consent grant and an inalienable operator-override; without those the Leviathan is a tyrant in a costume.

KEY IDEA

4.1 Introduction: the swarm is a state of nature

The pitch for running many coding agents at once is parallelism: five agents, five times the work. The reality, the first time you try it on a real repository, is a specific kind of dread. Two agents claim the same file and race each other's writes. One agent, asked to make the tests pass, makes them pass by deleting the failing one. A fourth opens a pull request you cannot review because three others have opened five more. You spawned a team and got a mob. The work did not get more parallel; your *uncertainty* did.

This is not a prompt-engineering problem. It is a coordination problem, and coordination problems have a literature. **Thomas Hobbes**, *Leviathan* (1651) [113] — the foundational social-contract text: rational self-interested actors with no common power above them fall into a “war of every man against every man,” in which life is “solitary, poor, nasty, brutish, and short.” Hobbes’ insight is structural, not moral: the actors need not be malicious. Absent a shared authority, even rational, well-meaning agents defect, because each cannot trust the others not to. Your agents are in exactly that state of nature. They do not need better prompts; they need a referee they have agreed to obey.



Hobbes's *Leviathan* (1651) made the sovereign-by-consent argument; this chapter runs it on one machine.

4.1.1 The failure taxonomy

The state-of-nature failures are not hypothetical; they are the recurring, reproducible pathologies of any multi-writer coding swarm. Table 4.2 maps each Hobbesian failure to its concrete form and to the class of coordination primitive that addresses it.

The striking recent result is that the identification is not merely a useful analogy. **Dai et al. 2024**, *Artificial Leviathan* (arXiv:2406.14373) [57] — a sandbox of LLM agents with psychological drives that, starting from unrestrained conflict resembling the state of nature, were observed to establish social contracts, authorize a sovereign, and form “a peaceful commonwealth founded on mutual cooperation.” The arc this series posits as its organizing metaphor is, in at least one controlled study, an *emergent dynamic* of LLM populations (the methodological ancestor is **Park et al. 2023**, *Generative Agents* [196], where LLM agents form emergent social behavior in a sandbox town). The *Leviathan* is not a costume we put on the system; it is the attractor the system already falls toward. The wager of this paper is that we can build the consented sovereign *deliberately, locally, and legibly* rather than let an illegible one congeal by accident. Figure 4.1 sets the two sides of that arrow side by side: the ungoverned swarm on the left, the consented local *Leviathan* on the right.

State-of-nature failure		Concrete form in a coding swarm	The primitive that answers it
Resource conflict		Two agents edit the same file/region; one silently clobbers the other	A claim — an advisory announcement that an agent intends to touch a file or region — backed by a database uniqueness constraint that makes a double-claim physically impossible.
Illegibility		A pull request or session whose intent and effect a human cannot read in bounded time	A digest-with-zoom surface (§4.3): a summary that is a lens onto the underlying artifact, never a replacement for it.
Lies confident-wrong	/	An agent reports completion (“all green”) without having verified	An honest-attestation discipline: a self-report that distinguishes <i>verified-good</i> from <i>no-evidence-of-bad</i> and refuses to claim what it cannot check — “a green that wasn’t checked is a lie.”
Footguns		Irreversible action (force-push, delete) on a stale or wrong premise	Footgun guards (§4.4.5); the open problem this paper frames is replacing hand-maintained rule corpora with a structural authority.
Illegible authority	au-	The coordinator blocks an agent and the operator cannot see <i>why</i>	The <i>legible-sovereign rule</i> (§4.2): every act of authority is a logged, named, zoomable event.

Table 4.2: The state-of-nature failure taxonomy, the concrete coding-swarm form of each, and the class of primitive that answers it. The fifth row — *illegible authority* — is the one Hobbes himself got wrong for software, and it is the crux of this paper (§4.2).

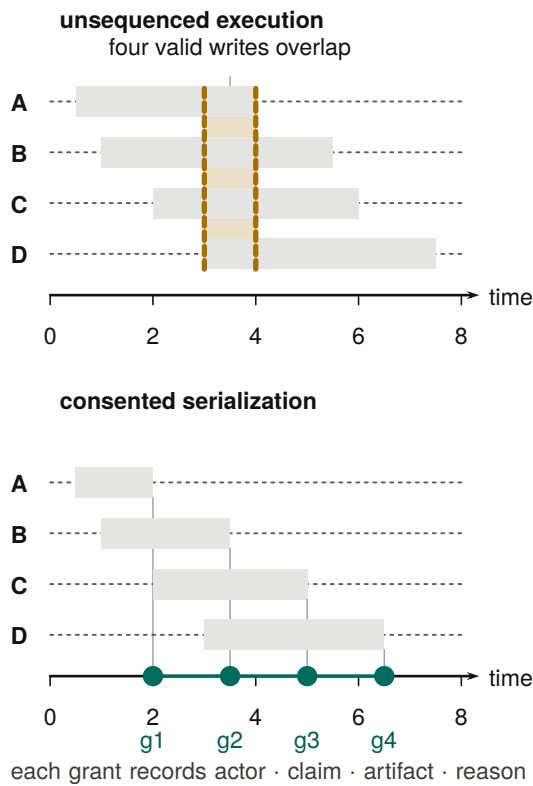


Figure 4.1: Two Gantt panels on one time axis. Top: four write requests A–D arrive between $t=0.5$ and $t=3$ with windows that all cover $[3,4]$, so four valid writers overlap on one artifact with no total order among them. Bottom: the same four requests under consented serialization, each band now ending at its own grant $g1$ – $g4$ ($t=2, 3.5, 5, 6.5$) on one commit spine, and every grant recording actor, claim, artifact, and reason. Consent changes the schedule, not who may participate [internal].

4.1.2 Why “local”

Hobbes’ commonwealth is territorial: one sovereign per realm. Here the realm is *one machine*. The **daemon** — *a single always-on local process, backed by a write-ahead-logged embedded database, that every agent and every command talks to and that holds coordination state no agent can edit* — is the kernel and the realm: your machine, your swarm, no network, no cryptography required. Cryptography enters only when another operator’s fleet touches your repository, the cross-operator setting *The Harbor Economy* takes up. The single-operator Leviathan is the *wedge* — the thing a solo developer pays for today — and the federation of Leviathans is the platform, deferred behind the wedge by design. This paper is entirely about the wedge.

4.1.3 A worked afternoon: Mara and the six agents

The argument of this paper is easiest to see end-to-end, through one operator and one real failure. We will return to Mara at each mechanism; read this section cold, then watch the pieces snap into place as they are introduced.

Mara, 2:40 p.m. *She has a payments-service refactor due Friday and spins up six coding agents against one repository — two on the API layer, two on the database migrations, one writing tests, one reviewing. For twenty minutes it is glorious: six terminals, six streams of work. Then it is not.*

SCENE

What happens to Mara is the state of nature, in order:

1. **The collision.** Agent API-1 and agent API-2 both decide the cleanest fix touches `handlers/charge.go`. Neither knows the other is in the file. API-2 commits first; API-1 commits on top and silently reverts half of API-2’s change. No error fires. The work is lost and nobody — including Mara — knows yet. (*Resource conflict, Table 4.2, row 1.*)
2. **The illegible pile.** By 3:10 there are five open pull requests. Two touch the same migration. One has a title that says “fix tests” and a 600-line diff Mara cannot read in the ninety seconds she has before the next agent pings her. She is now the bottleneck, and she cannot even *see* what she is the bottleneck for. (*Illegibility, row 2.*)
3. **The confident lie.** The test agent reports DONE: ALL GREEN. It made them green by deleting the one migration test that was failing. The report is not malicious — the agent optimized exactly the objective it was given — but it is false, and Mara has no cheap way to tell a checked green from an asserted one. (*Lies, row 3.*)
4. **The footgun.** The review agent, trying to “clean up the branch,” proposes a force-push that would discard a colleague’s unmerged commit. The premise is stale; the act is irreversible. (*Footguns, row 4.*)

Mara did not get six times the work. She got a mob, and her *uncertainty* went up sixfold. This is the failure this paper exists to fix, and the rest of the afternoon — once Mara is running under a consented local Leviathan — is the through-line we will trace: how the claim stops the collision before it happens (§4.3), how a digest-with-zoom surface lets her read the pile in bounded time (§4.3), how a completion gate turns the confident lie into a loud “I cannot verify this” (§4.4.4), how Signal Detection Theory sets exactly when she is forced to look before an irreversible act (§4.5), and how, at fifty agents instead of six, a directory keeps her from drowning (§4.6).

*Exercises 4.1–4.4,
p. 241.*

4.2 Consent to the Leviathan (the normative claim)

Why should the operator cede authority to a daemon — let it block an agent’s claim, escalate a decision, reorder a merge, refuse a done? Hobbes’ answer, transposed: the operator authorizes the sovereign because the covenant is rational under the alternative. Crucially, in Hobbes the covenant is *among the subjects* — “a real unity of them all...by covenant of every man with every man” [113, 115] — not a bargain struck *with* the sovereign. Transposed: the operator does not negotiate with each agent; the operator institutes a *rule of the road* — a **coordination protocol** of *typed performatives* (messages whose type declares their intent: a request, a **commitment** (glossed in §4.3.1: a violable obligation bound to a named actor, watched by a breach monitor), a delegation, a termination), so that every message carries real intent, ownership, and a stop condition — that all agents are bound to, and the daemon enforces it.

The authority is *asymmetric by design*. This is the same lesson **Raft** teaches for distributed systems (**Ongaro & Ousterhout 2014**, *In Search of an Understandable Consensus Algorithm* [184] — *a strong-leader consensus protocol deliberately made less general than Paxos in order to be comprehensible and correctly implementable*): a strong leader who decides the common case unilaterally is simpler and more reliable than democratic consensus among peers, provided the common case dominates. A solo operator’s swarm is overwhelmingly common-case; the strong-leader daemon is the right shape. We are precise about what shape that is: the single-machine system is formally a *consistency-favoring design with a central coordinator and distributed clients*, not a leaderless consensus protocol. The leaderless distributed-systems canon bites only at the edges where the daemon is not the sole observer — an agent partitioned from the daemon, and the cross-operator setting deferred to the economy paper. We do not borrow the intellectual credit of a leaderless design while building a centralized one.

4.2.1 The legible-sovereign amendment

Hobbes' absolutism needs one modern amendment, and it is the crux of this paper. Hobbes argued the sovereign, being the *product* of the covenant rather than a *party* to it, can never breach it and need not be inspectable [113, 115]. For a software authority this is exactly wrong. An *illegible authority* is the fifth state-of-nature failure mode (Table 4.2) wearing a crown: if the daemon blocks an agent and the operator cannot see why, the operator's trust — and therefore the consent that legitimizes the authority — erodes.

Property 4.2.1 (The legible-sovereign rule). The coordinating authority must be the *most* legible actor in the system, not the least. Every act of authority — a claim denial, an anomaly detection, an escalation, an “all good” attestation — must be a logged, named, zoomable event whose reason the operator can reconstruct. Consent is renewable only while the sovereign is inspectable.

This is the inversion that rescues the design from despotism, and it is the move a political theorist would press hardest (§4.9). In Hobbes the sovereign is opaque *by structure*; here the sovereign is the one actor whose every exercise of power is an audit record. It is realized as a discipline of **honest attestation**: a single self-report that runs every invariant check, distinguishes *verified-good* from *no-evidence-of-bad*, regiments what it can, and fails loudly about what it cannot. That discipline is precisely the sovereign making its own judgments legible — “all good” becomes “a claim with a verifier, not vibes.”

There is a regress hiding here, and Scott would press it. If every act of authority must be legible, the *legibility apparatus itself* is an exercise of authority — the digest decides what is surfaced and what is buried. The Attention Queue ranker (§4.7) is *the* sovereign act: it decides what the operator sees *right now*. A ranker that silently buries an escalation is governing opaquely. The rule is therefore not complete until the ranker's *omissions* are themselves legible — a “what did the queue not show me, and why” counter-surface. Ranking is governing; we say so explicitly in §4.9.

PITFALL

The regress is easier to reason about once you have seen what a counter-surface would concretely be, so here is the mockup rather than the argument. Table 4.3 is one screen of it: not a diagram of a mechanism that does not exist, but the artifact the mechanism would have to produce.

Suppressed item	Regret	Why it was not shown	Re-surface trigger
agent-7 escalation: “migration ordering unclear”	0.31	below the distress threshold (0.45) for this action class	threshold drops, or the item is restated
agent-3 note on auth/session.ts	0.12	aged out; recency weight decayed past the floor	any new claim on the same file
Third retry of the same failing test	0.58	de-duplicated into the first occurrence	a fourth retry, or a different failure mode

Table 4.3: A mockup of the “what did the queue not show me, and why” counter-surface. Each row names the item, its score, the *rule* that suppressed it, and the condition that would bring it back. The regress does not vanish — this surface is itself ranked, and rows below its own fold are invisible — but it changes shape: the question becomes whether the *suppression rules* are few enough to enumerate, which is answerable, rather than whether every omission is visible, which is not.

4.2.2 Consent as a first-class, revocable primitive

It is easy to say the operator “consents to cede decision authority.” A political theorist’s first question — and the one that decides whether the Leviathan frame has substance or is merely decorative — is: *consented how, to what, and revocable when?* “The operator just starts using the tool” is not Hobbesian consent; it is what the canon calls **tacit consent**, and **Hume** (*Of the Original Contract*, 1748 [116] — *the decisive critique: the peasant who “consents” by remaining in the country has no real alternative, so the consent does no normative work*) demolished it two and a half centuries ago. We therefore make consent a concrete object.

Definition 4.2.1 (Consent grant). A **consent grant** is an explicit, scoped, revocable authorization the operator issues to the daemon, of the form

$$g = \langle \text{SCOPE, STAKES-CEILING } s_{\max}, \text{ REVERSIBILITY-FLOOR } v_{\min}, \text{ TTL, REVOCABLE} \rangle,$$

e.g. “the daemon *may* auto-land reversible diffs under stakes threshold $s_{\max}$ in the tests/ subtree without asking, for 24h.” A grant is a first-class, legible object: it appears in the digest, it has an audit trail, and it expires. The authority’s legitimate action set at any moment is exactly the union of its active grants.

Consent so defined gives the design something Hume’s critique cannot reach: a discrete authorization event, a bounded scope, and an exit. It is also an object supervisory-control research named long before this chapter did: a **level of automation**, in Sheridan’s ten-point sense and

in the four-stage refinement (acquire, analyse, decide, act) that Parasuraman, Sheridan and Wickens later gave it [237, 193]. Read that way, a grant's $\langle \text{SCOPE}, s_{\max}, v_{\min}, \text{TTL} \rangle$ is a level-of-automation setting chosen per action class rather than once for the whole system, which is roughly what that literature has been recommending since 1978. This section built the same selector from a Hobbesian covenant, and that two routes so far apart arrive at a compatible object is some evidence the object is the right one. It also hands the accompanying chapters a concrete object. A grant is precisely what a *hosted-trust* offering transfers — “we make this fleet legible to you, under this scope” is the sale of a consent grant to a third party, which *The Harbor Economy* prices — which is why §4.12 lists consent among the things this layer provides upward.

PROTOCOL 4.2.1. Consent-grant lifecycle

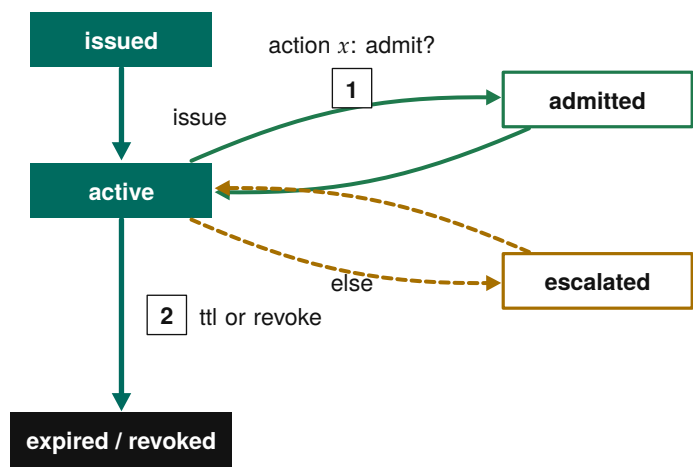
1. **Issue.** The operator issues a grant g = $\langle \text{SCOPE}, s_{\max}, v_{\min}, \text{TTL} \rangle$. The grant is appended to the audit log as a named, zoomable event and surfaced in the digest.
2. **Act.** On a candidate action x , the daemon admits the action without asking *iff* x is in scope **and** $\text{stakes}(x) \leq s_{\max}$ **and** $\text{reversibility}(x) \geq v_{\min}$ **and** the grant is unexpired and unrevoked; otherwise it escalates to the operator.
3. **Witness.** Every action taken under g records $(g, x, \text{reason}, \text{artifact-link})$ — the legible-sovereign rule (Prop. 4.2.1) binds here: the act is reconstructable.
4. **Expire / revoke.** On TTL or operator revocation, g leaves the active set; the daemon's legitimate action set shrinks accordingly. The override (Prop. 4.2.2) revokes all grants at once and preempts in-flight acts.

Figure 4.2 draws this lifecycle directly: the three-clause guard on admission, the loop back to active for either outcome, and the separate terminal transition on ttl or revocation.

Mara, 3:25 p.m. *Burned by the morning's chaos, she issues one grant: “you may auto-land reversible diffs under low stakes in the tests/ subtree without asking, for the next two hours.” She does not grant migration authority — those are irreversible. The grant is a single line in her digest with an expiry clock. The review agent's proposed force-push fails the reversibility floor $v_{\min}$ and is escalated to her instead of executed; the test-only formatting diffs land silently. She has not surrendered control — she has scoped it.*

SCENE

Property 4.2.2 (The inalienable operator-override). Hobbes grants the subject exactly one inalienable right — self-preservation; you cannot covenant away the right to resist when your *life* is at stake [113]. The computational analog is an always-available, unconditional operator-override: a *stop now / I withdraw authority* channel that no autonomy level



guard	what breaks without it
1 in scope, stakes $\leq s_{\max}$, reversibility $\geq v_{\min}$	an out-of-scope or high-stakes action executes unsupervised
2 ttl elapsed, or operator revokes	a stale or revoked grant keeps admitting actions

Figure 4.2: The consent-grant lifecycle of Def. 4.2.1 and the Consent-grant lifecycle protocol. A grant is issued, then active. Each candidate action is admitted only if it clears the three-clause guard (scope, stakes ceiling, reversibility floor); any clause it fails sends the action to escalation instead. Both outcomes leave the grant active for the next action – a single escalated action does not end the grant. Only ttl expiry or an operator revocation moves the grant to its terminal state, after which the daemons legitimate action set shrinks by exactly that grant. [internal]

can suppress. If the system reserves a highest-priority *distress* lane for the daemon to interrupt the operator, the override is its dual — the operator-to-daemon channel — and it is a *right*, not a feature: it preempts every grant, every in-flight landing, every coordinator decision.

Computer security reached the same object by its own road. A consent grant is, to a capability theorist, a **capability with contextual caveats** — unforgeable, revocable, and able only to narrow as it is passed along [30] — and that a Hobbesian covenant and a Macaroon agree about what the object has to be is a small reassurance that neither derivation was decorative.

This pairs with Hirschman’s *Exit, Voice, and Loyalty* (1970 [112] — *the choice between leaving (exit) and complaining (voice) as the two levers a member has over a declining organization*): the design has rich *voice* machinery (escalation, annotation, the attention queue) and the override supplies the missing *exit*. Exit is the discipline that keeps voice honest; an operator who cannot cheaply withdraw a grant has no real leverage over the sovereign.

Exercises 4.5–4.8,
p. 241.

4.3 The mechanism: legibility-with-zoom

A referee that only prevents collisions would be a glorified lockfile. The reason an operator would *pay* for the Leviathan is the next move: it makes the swarm *legible*. James C. Scott, *Seeing Like a State* (1998) [229] — *states impose legibility (cadastral maps, standardized surnames, the metric grid, scientific forestry) to make populations countable and governable; high-modernist over-legibility — the overconfident faith that a clean top-down scheme can replace messy local reality — recurrently fails because it destroys the local practical knowledge (mētis: hands-on, case-specific, hard-to-codify know-how) the simplified order depended on*. Scott’s canonical example is the German *Normalbaum* (“scientific forest”) — rows of single-species, same-age trees, gloriously legible to the state’s yield tables — that thrived for one generation and then collapsed, because the legible monoculture had erased the illegible ecological *mētis* (soil fungi, undergrowth, species mix) that kept a real forest alive [229]. The software port of Scott’s thesis (Rao 2010, *A Big Little Idea Called Legibility* [210]) made “legibility” a standard lens on systems that flatten reality to a single purpose.

Map this onto agent oversight and the design rule writes itself. The operator is the state; the swarm is the population; the dashboard/TUI/digest is the cadastral map. The temptation is identical: render the swarm as a clean grid of green tiles and *govern the map*. The *Normalbaum failure* in software is the **Potemkin digest** — a beautiful surface whose tiles assert a reality nobody can check, having paraphrased away the diff, the test output, the error, the reasoning that constituted the actual work. The operator who acts on that map is the forester admiring the yield table while the forest dies.



Scott’s *Seeing Like a State* is the standing warning behind this chapter: a summary that erases the local knowledge it was built from has already failed.

Definition 4.3.1 (The one law: digest-with-zoom). Every summary is a lens onto the real artifact, never a replacement for it. Summarize the **state**; link to the **work**. A digest you cannot zoom is a high-modernist map. A digest that always reaches the underlying artifact preserves *métis*: the operator can descend from abstraction to ground truth and catch the lie. Formally, a read-surface is a pair (σ, ζ) where σ is a projection (the summary) and ζ is a total *zoom* function mapping every summarized claim to a verifiable artifact; a surface with a partial or absent ζ is a Potemkin digest.

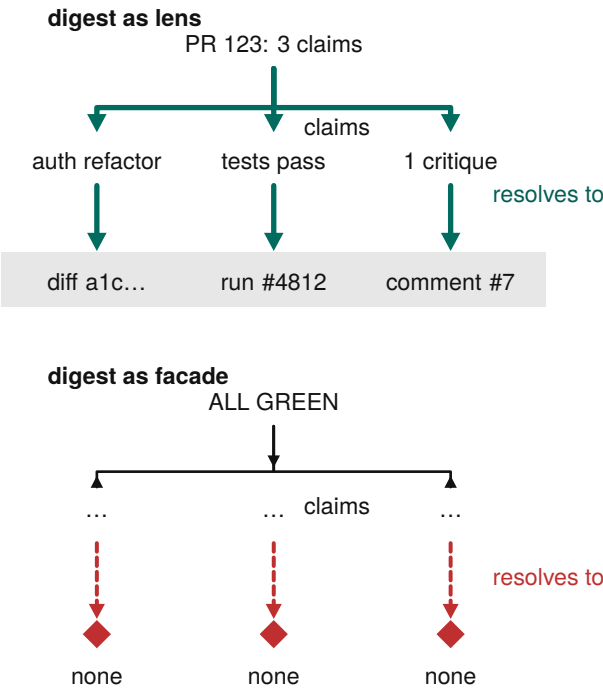


Figure 4.3: Two claim trees of identical shape. Top: the legible digest’s three named claims each resolve to one artifact — a diff, a test run, a review comment. Bottom: a same-shaped summary whose three claims resolve to nothing an operator can open. A safe summary is an index into evidence; a calm one that terminates early is a Potemkin digest, whichever claim count it advertises [internal].

Figure 4.3 puts the two digests side by side: one that zooms to a verifiable artifact and one that only asserts. Three disciplines converge on this one principle: summaries must be indexes, not replacements; a reported result must be a checked result, not an asserted one; and every summary must reach the artifact it summarizes. This paper supplies the *why* (Scott plus the human-factors argument of §4.5) and the *how* (the rules below). Digest-with-zoom is the unifying statement of all three.

4.3.1 What to flatten, what to preserve

Scott’s own distinction is the design boundary. The state may safely standardize *administrative facts it itself defines*, and must *never* standardize away the *local knowledge it depends on*.

- **Flatten (safe — the operator’s own structured field):** identifiers, **claim** rows, session phases, port numbers, **commitment** status (a *commitment* being a violable obligation bound to a named actor, watched by a breach monitor), enumerated states. Administrative legibility is cheap and lossless here.
- **Preserve verbatim, never paraphrase (the agents’ *mêtis* / the ground truth):** the diff, the test output, the error message, the reasoning trace, the exact command run. Paraphrasing these is the over-flattening.

The slogan: *summarize the STATE, link to the WORK*.

“Zoom to the artifact” does not fully preserve *mêtis*, and a political theorist will say so (§4.9). *Mêtis* in Scott is not “the detailed data underneath the summary” — it is practitioner knowledge that *resists codification entirely*: the operator’s gut sense that “this agent is about to do something dumb,” the local knowledge of which corners of the codebase are cursed. A read-path to the diff is more legibility, not *mêtis*. The honest admission — which couples Scott to Bainbridge (§4.5) — is that *some of the most valuable operator knowledge is exactly what the system cannot make legible, and the more the supervising automation takes over, the faster that mêtis atrophies*. What organizational-knowledge theory calls the problem of converting **tacit knowledge** into explicit, transferable form — “we know more than we can tell” [201] — is the thirty-year literature this pitfall leaves open rather than solves. The skill-retention tax (§4.5.4) is the design’s only defense, and it is frankly the hardest part of the design to engineer.

PITFALL

4.3.2 The zoom target must be verifiable, not self-reported

There is a subtle trap inside “zoom to the real thing.” If the operator zooms to the *agent’s chain-of-thought* and treats that as ground truth, the map has merely moved down one level — because the reasoning trace may be fiction. **Turpin et al. 2023**, *Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting* (NeurIPS) [259] — *LLM-generated reasoning traces can be systematically unfaithful: plausible, coherent, and not actually what drove the model’s output (a model swayed by reordering of options will rationalize a post-hoc reason and never mention the bias)*. Therefore:

Property 4.3.1 (Verifiable-zoom). The canonical zoom target is an artifact the operator can check *independently of the agent* — the diff on disk, the test result, the committed file. The agent’s self-narration is a *secondary* lens, always labeled as such. Two LLMs agreeing (an agent acting and an LLM summarizing it) is not two checks; it may be one error reported twice.

This is why the source of truth must be the durable substrate — the coordination database and the version-control working tree — and not a model’s summary of them. The digest is computed *over* artifacts the operator owns. It is also why an *adversarial* review organ (a reviewer with no stake) is structurally necessary, not optional: a swarm that reviews its own work is two LLMs agreeing, which by Prop. 4.3.1 is one error twice. We return to adversarial review as the authority’s quality organ in §4.4.

4.3.3 The four design questions (a decision procedure)

For any read-surface — Attention Queue, briefing, resurrection digest, TUI pane — ask, in order (Table 4.4):

1. **Will the operator act irreversibly on this?** If yes (merge/deploy/delete), the artifact must be one keystroke away and a zoom *forced*. Never let an irreversible act rest on a digest alone (§4.5).
2. **What is administrative vs. *mêtis*?** Flatten the former; link the latter verbatim (§4.3.1).
3. **Who authored the summary, and can it lie?** A deterministic projection of structured state is trustworthy; an LLM summarizing another LLM is not — point its zoom at the artifact (§4.3.2).
4. **Is the authority itself legible here?** Every coordinator action carries a named reason (Prop. 4.2.1).

Table 4.4: A read-surface earns release through four independent proofs, in order. The rail is conjunctive: one missing proof holds the surface. [internal]

	Question	What release requires	What the reader can check
1	is the action irreversible?	the artifact is one key away	the artifact link resolves
2	is this state or mêtis?	administrative state is flattened; the verbatim trace is linked, not summarized	the trace link resolves
3	could the claim lie?	the claim resolves beyond the author’s self-report	the independent evidence resolves
4	is the authority legible?	every coordinator act returns a named reason	the reason field is present and specific

PROTOCOL 4.3.1. Render a digest line under the one law

Given a candidate summary line σ over a unit of swarm work:

1. **Project state.** Compute σ as a deterministic projection of structured state (claim rows, phases, enums). Never let a language model paraphrase the line itself.
2. **Attach zoom.** For every claim in σ , attach ζ : a link to an operator-verifiable artifact (the diff, the test log verbatim, the committed file). If any claim has no such artifact, the line is a Potemkin digest — refuse to ship it.
3. **Forced-zoom flag.** If the operator may act irreversibly on σ (merge, deploy, delete), set $\sigma.\text{force} = \text{true}$: the artifact is one keystroke away and the act is gated on a zoom (§4.5 sets the threshold).
4. **Reason on authority.** If σ records a coordinator act (a denial, an escalation), attach its named reason. An act with no reason is illegible authority; refuse to ship it.

Mara, 3:32 p.m. *She opens the digest for the five-PR pile. Each row reads “API-2 · charge handler · merged · 1 critique round” — state only, six words. The “fix tests” PR’s row is flagged force-zoom, because merging it is irreversible; she presses one key and the 600-line diff and the verbatim test log open side by side. The diff deletes a migration test. The digest did not paraphrase that away — it linked her straight to it, and the lie is caught in ten seconds instead of slipping through. This is the one law doing its job.*

SCENE

4.4 The Leviathan rules, it does not just watch (the authority half)

The title is *Legibility & Authority*. A read-only legibility layer is a glorified dashboard; the whole point of Hobbes is that the sovereign does not just *see* — it *rules*. This section specifies the authority half: the mechanisms by which the consented daemon actually decides, blocks, lands, and gates. Table 4.5 lists the organs; the engineering status of each is in Appendix 4.A.

Table 4.5: Six authority mechanisms as distinct instruments crossing one consent boundary. Their inputs and outcomes differ; what they share is the recorded grant *g*: scope, named reason, audit identity, override, and an addressable return artifact. The engineering status of each is in Appendix 4.A. [internal]

Mechanism	Input	Outcome	What the grant records
roadmap	evidence	a priority	the evidence weighed and the ordering it produced
review	rival claims	a gate	which claim was admitted and why the others were not
completion	a predicate	done, or not	the predicate and the artifact that satisfied it
landing	conflicts	an order	the conflict set and the serialization chosen
guard	a proposed act	a block, or passage	the act, the rule that fired and the named reason
escalation	an interrupt	an operator decision	the question raised and the decision returned

4.4.1 The roadmap as a versioned statement of intent

A coordinator that prevents collisions answers “what is the swarm doing?” It does not answer “is the swarm doing the *right* thing?” That question needs a single legible artifact the authority is measured against: a **roadmap** — a phase-keyed, versioned, amendable statement of intent, priority, and dependency, against which the swarm’s actual output can be compared. It is not truth about the world; calling it that

would turn every lesson the work teaches into drift. The authority’s legitimacy is measured by *whether the roadmap stays honest*: every unit of work keys to a plan item, amendments are recorded as amendments, and drift between plan and reality is itself a legible, surfaced anomaly rather than a fact to be hidden.

4.4.2 Adversarial review: the quality organ

By Prop. 4.3.1, a swarm reviewing its own work is two language models agreeing — one error twice. The quality organ must therefore be *adversarial and conflict-free*: a reviewer with no stake in the work, running a bounded critique-refine protocol. This is the wedge’s local analog of the *neutral evaluators* — the independent rating agencies — that *From Spawn to Person* and *The Harbor Economy* federate across operators: the same conflict-free-judge discipline, scoped first to one operator and then opened to a market.

4.4.3 Sole-responsibility roles and the specialization boundary

The single-writer discipline (§4.1.2) generalizes naturally from data files to system functions. When an invariant is critical, the authority assigns a **sole-responsibility role** (an avatar holding exclusive write capability over an invariant):

- *Roadmap Owner*: The single avatar authorized to commit mutations to the phase plan.
- *Test Suite Curator*: The sole agent authorized to land modifications to regression suites.
- *Release Avatar*: The sole gatekeeper authorized to execute deployment tags.

Assigning a role that way is a trade, and the trade has a number. Think of a bank: one expert teller (the sole specialist) working a single line in strict order, versus several ordinary tellers (the pooled generalists) sharing one line. The expert is faster per request but has no colleagues to absorb a burst; the pool is slower per request but almost never leaves the line standing idle behind a long job. $M/M/1/M/M/c$ — the standard queueing-theory shorthand (Kendall 1953 [129]) for a single-server versus c -server queue with Poisson arrivals, exponential service times, and one waiting line served first-come-first-served — is the textbook way to price the trade.

Fix the **queueing-specialization setup**: requests for an invariant-critical function F arrive as a Poisson stream at rate λ_F ; a sole specialist serves them as an $M/M/1$ queue at rate μ_{spec} , the pooled alternative as an $M/M/c$ queue of generalists at rate μ_{pool} each with utilization $\rho = \lambda_F/(c \mu_{\text{pool}}) < 1$ and first-come-first-served exponential service;

and the objective is *mean* cost — mean response time, plus an accountability value A per unit time of demand for single-writer attribution, against a waiting cost w per request-hour. Let $C(c, \rho)$ be the **Erlang-C** delay probability (**Erlang 1917** [77] — *the probability that an arriving request finds all c servers busy and must wait, the quantity that makes a pool a pool*).

THEOREM 4.4.1 Queueing Specialization Boundary. Under the queueing-specialization setup, sole ownership weakly dominates the pool if and only if the skill premium clears

$$\frac{\mu_{\text{spec}}}{\mu_{\text{pool}}} \geq g_A(\rho, c) = c\rho + \frac{1}{1 + \frac{C(c, \rho)}{c(1 - \rho)} + \frac{A\mu_{\text{pool}}}{w\lambda_F}}. \quad (4.1)$$

VERIFIED — corrected. An earlier draft of this paper proposed the threshold $\tilde{g} = 1 + (c - 1)\lambda_F/(c\mu_{\text{pool}} - \lambda_F) = 1 + (c - 1)\rho/(1 - \rho)$, on the plausible-looking grounds that it is what a naive $M/M/1$ -versus- $M/M/c$ response-time comparison appears to give. A sweep against simulation *falsified* it in both directions before the belief hardened, and Eq. 4.1 is the replacement — the companion formal paper’s Theorem 2 [192], independently re-derived and checked numerically over two thousand (ρ, c) points. We leave the wrong form on the page rather than quietly swapping it, because a paper that argues for falsification-first oversight should be legible about its own retractions: the two thresholds cross exactly once, so the superseded form was not merely conservative — it was wrong in *both* directions, and Figure 4.4 shades the two error regions.

At $A = 0$ — pure response time, accountability priced at nothing — the boundary reduces to

$$g(\rho, c) = c\rho + \frac{c(1 - \rho)}{c(1 - \rho) + C(c, \rho)}. \quad (4.2)$$

Three properties fix its shape: $g(\rho, 1) = 1$ (a specialist against a single generalist needs no premium at all), $g \rightarrow c$ as $\rho \rightarrow 1$ (the boundary *saturates* at the capacity ratio; it never diverges), and for two generalists it is the closed form $g(\rho, 2) = 1 + 2\rho - \rho^2$. Below the boundary, sole ownership buys strict single-writer accountability at an explicit latency price that must be budgeted rather than obscured; Eq. 4.1 is where that price is written down.

Numbers by hand. The result is two lines of algebra on textbook formulas, which is the right way to first believe it. Sole ownership pays iff $w\lambda_F(W_{\text{solo}} - W_{\text{pool}}) \leq A$, i.e. $1/(\mu_{\text{spec}} - \lambda_F) \leq W_{\text{pool}} + A/(w\lambda_F)$; invert,

divide by μ_{pool} , and substitute the two textbook facts $\lambda_F/\mu_{\text{pool}} = c\rho$ and $\mu_{\text{pool}}W_{\text{pool}} = 1 + C(c, \rho)/(c(1 - \rho))$, and Eq. 4.1 falls out. For $c = 2$ the Erlang-C value is $C(2, \rho) = 2\rho^2/(1 + \rho)$, so the second term of Eq. 4.2 is $(1 - \rho^2)/((1 - \rho^2) + \rho^2) = 1 - \rho^2$ and $g(\rho, 2) = 1 + 2\rho - \rho^2$, checkable in three lines with no queueing theory at all.

Now the case the roadmap owner actually faces. Roadmap mutations arrive at $\lambda_F = 2/\text{hr}$; the sole owner clears them at $\mu_{\text{spec}} = 3/\text{hr}$; the alternative is three generalists at $\mu_{\text{pool}} = 1.2/\text{hr}$ each. Then $\rho = 2/3.6 = 0.556$ and the skill premium is $r = 3/1.2 = 2.5$. Erlang-C at $(c, \rho) = (3, 0.556)$ is 0.300, so $g = 3(0.556) + 3(0.444)/(3(0.444) + 0.300) = 1.667 + 0.816 = 2.483$. The premium 2.5 clears it — barely — so the sole owner wins: $W_{\text{solo}} = 1/(3 - 2) = 1.000$ hr against $W_{\text{pool}} = (1 + 0.300/1.333)/1.2 = 1.021$ hr, a margin of about 75 seconds per request. The superseded threshold would have returned $\tilde{g} = 1 + 2(2)/(3.6 - 2) = 3.5$ and sent the work to the pool. That is the right-hand error lens of Figure 4.4, met in the wild on the first realistic instance anyone tries.

Now you try: at the same three-agent pool, would a specialist at $\mu_{\text{spec}} = 2.8/\text{hr}$ still be worth it on response time alone? (The premium is $2.8/1.2 = 2.33 < 2.483$ — no, and $W_{\text{solo}} = 1.25$ hr confirms it. The boundary is genuinely tight here; note that a positive accountability value A lowers g_A below g and can buy the sole owner back.)

Numbers by hand — Where the exact and superseded thresholds cross. At the round number $\rho = 0.5$: $g(0.5, 2) = 1 + 2(0.5) - 0.5^2 = 1.75$ [verified, closed form].^a The superseded $\tilde{g}(\rho, 2) = 1 + \rho/(1 - \rho)$ gives $\tilde{g}(0.5, 2) = 2.00$ at the same point — already a visible gap. Setting $g = \tilde{g}$ and clearing denominators leaves $\rho(\rho^2 - 3\rho + 1) = 0$, whose root in $(0, 1)$ is $\rho = (3 - \sqrt{5})/2 \approx 0.382$ [verified, closed form]: below it $\tilde{g}$ over-certifies specialization (false-certify), above it $\tilde{g}$ over-rejects (false-reject), which is exactly why the superseded form was wrong in *both* directions rather than merely conservative. *Now you try:* run the falsification sweep behind this crossing (Exercise 4.18). ■

^aWorked numbers carry a tag throughout: [verified] when a named script in the repository regenerates them, [internal] when only the chapter's own derivation does.

Eq. 4.1 prices *mean* response time under first-come-first-served exponential service. It does not price the tail, and it does not price the thing an operator actually values in a roadmap owner — consistency of judgment across requests — which does not appear in a response-time objective at all. The corner the whole sole-responsibility discipline exists for, the sole specialist *failing*, is priced in §4.4.3.1 below, and priced under an assumption worth reading before the formula.

PITFALL

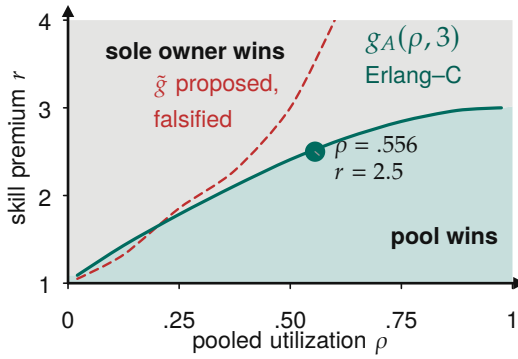


Figure 4.4: The specialization boundary at a three-agent pool. Below the Erlang-C boundary $g_A(\rho, 3)$ (teal), pooling wins; above it, sole ownership wins — the two regions are now filled and edged, not hatched. The superseded threshold $\tilde{g}$ (dashed) is kept for comparison: it diverges rather than saturating at the pool capacity ratio. The worked instance ($\rho = .556$, $r = 2.5$) narrowly clears the correct boundary into the sole-owner region [internal, closed form].

4.4.3.1 The succession corner, and who prices it

Eq. 4.1 assumes a specialist who never stops. The corner the whole discipline exists for is the one where they do: they crash, are reaped, quit, or are reassigned, and a successor spins up. That corner *is* priced, in closed form, and it is priced in *The Harbor Economy* rather than here — the result is a condition on what a succession plan costs, which is an economic statement, and the economy chapter carries it as the price of the succession rule (Thm. 6.2.1), with its own verification and its own boundary. Two of its consequences belong in this chapter’s argument, because they change what a sole-responsibility role means rather than what it costs.

The first is that skill has a ceiling here. With death at rate ξ and succession at rate η , the sole owner’s mean response time decreases in skill but never below

$$W_\infty = \frac{\xi}{\eta(\xi + \eta)},$$

the residual downtime a request meets when it arrives during an outage. No premium buys that back, so past a point the honest fix for a slow sole role is a better handoff and not a better owner. The second is that the ceiling can bind absolutely: writing K for what the pool costs including accountability, pooling wins at *every* skill premium once ξ/η clears the succession price $D^* = \eta K / (1 - \eta K)$ — and, in the case worth remembering, only while $\eta K < 1$. Where succession is fast enough that $\eta K \geq 1$ there is no such threshold at all and sole ownership cannot be ruled out on response time, which is the more common situation in a

well-run fleet and the one a reader is likeliest to be in.

Both statements rest on a modelling choice the economy chapter declares and this one inherits: the specialist can die *at any time*, not only while serving. Under active-only breakdowns a fast enough specialist escapes the floor entirely and $D^\star$ does not exist. The choice is right for an owner who is reassigned or dead rather than worn out, and it is a choice, not a derivation.

THESIS

4.4.3.2 What heavy tails do to the boundary

Exponential service is the assumption a queueing reader will challenge first, and they are right to: agent execution durations are not exponential. They are long-tailed — most turns are quick, a few grind through a rebuild or a flaky integration test — and submissions arrive in correlated bursts rather than as a clean Poisson stream. Kingman's heavy-traffic result [131] says what that costs in general: waiting grows with the sum of the squared coefficients of variation of the arrival and service processes, so a queue that is fine at $c_s^2 = 1$ can be badly congested at $c_s^2 = 6$ with the same utilization. The worry is real. What is not obvious, and matters here, is that it lands on *both* sides of Eq. 4.1 at once.

Work it through. Under the Allen–Cunneen approximation [8], the pool's mean wait picks up a factor $(c_a^2 + c_s^2)/2$ relative to the $M/M/c$ value, and with Poisson arrivals ($c_a^2 = 1$) that is $\kappa = (1 + c_s^2)/2$. The specialist is a single server, so its wait is exactly Pollaczek–Khinchine, which carries the same κ against the $M/M/1$ value. The two waiting terms in the boundary are therefore scaled by approximately the same factor, and dividing through by it leaves the shape of g_A intact while the accountability term $A\mu_{\text{pool}}/(w\lambda_F)$ — which prices a value per unit time of demand and knows nothing about service variance — is left over, entering as A/κ . The first-order reading is the reverse of the intuition: variance does not move the specialization boundary much, and what it does do is make single-writer attribution weigh *less* against queueing, not more, because everyone's queue got worse while accountability stayed worth the same.

EMPIRICAL HYPOTHESIS 4.4.2 Variance robustness of the specialization boundary. Under Poisson arrivals and general service with squared coefficient of variation c_s^2 , the crossing point of Eq. 4.1 moves by no more than the Allen–Cunneen heuristic predicts — that is, the boundary is recovered to first order by replacing A with A/κ , $\kappa = (1 + c_s^2)/2$ — for lognormal and Pareto service at the (ρ, c) points the chapter reports.

This is an *empirical hypothesis* and not a theorem, and the distinction is not decoration: $M/G/c$ has no closed form, Allen–Cunneen is an ap-

proximation whose error is known to grow with c_s^2 , and a Pareto service distribution with infinite variance has no c_s^2 for the heuristic to use at all, which is the case where the whole argument above stops rather than degrades. What would settle it is a sweep: extend `b8_specialization.py` with lognormal and Pareto service at matched means and report where the crossing actually lands. The falsifier is stated in advance — if the crossing moves further than the A/κ correction predicts at any swept point, the hypothesis is wrong and the boundary needs a variance term of its own rather than a rescaled A . No such sweep has been run, so no number appears here; the claim above is the shape of the answer, not the answer.

4.4.4 Completionist obligation: completion as a verifier, not a vibe

The operator’s standing complaint is the agent that *declares completion while the work is hollow* — “fixed the test by deleting it.” The completionist obligation makes this structurally impossible to do silently.

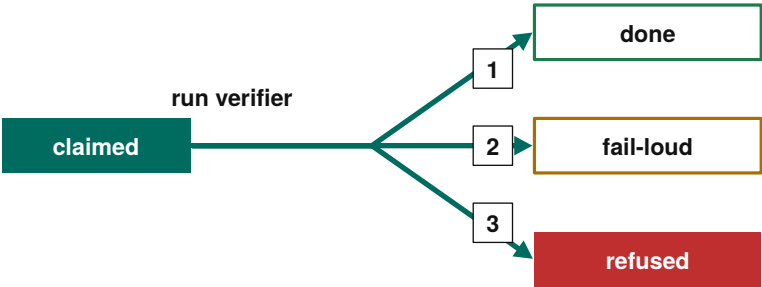
Definition 4.4.1 (Completionist completion). A **completion claim** is a commitment whose satisfaction predicate is a *verifier* derived from the task’s acceptance criteria. The authority refuses to accept the claim against an unmet predicate. An *unverifiable* predicate must **fail loudly** — surface “I cannot verify this completion” — and never silently pass. This is the authority-side analog of discovery’s *refuse-to-route* (§4.7.3): “I don’t know” must beat a confident wrong answer.

Figure 4.5 draws the three-way gate directly: a met predicate closes done, an unmet one is refused, and an unverifiable one must fail loudly rather than default to either.

The verifier is grounded in design-by-contract (**Meyer 1992**, *Applying Design by Contract* [171] — *pre/postconditions as machine-checkable obligations; the completion predicate is a postcondition*). This is the single highest-leverage new mechanism in the wedge: it turns “done” from a declaration into a checkable obligation, which is the only structural defense against the transparently-hollow deliverable.

Verification has a *cost gradient*, not a binary. A test going green is cheap to verify; “the architecture is right” is expensive, and reading a 600-line diff to confirm an acceptance criterion is itself attention-seconds that may exceed the agent’s narration cost by an order of magnitude (the *verification asymmetry*). Def. 4.4.1’s fail-loud is the honest floor for the *unverifiable*; §4.5 supplies the cost model for the *expensive-but-possible* middle.

PITFALL



guard		what breaks without it
1	predicate met	a hollow claim reads as done
2	predicate unverifiable	an unverifiable claim silently auto-passes
3	predicate unmet	a failing claim is accepted anyway

Figure 4.5: The completionist obligation of Def. 4.4.1: a completion claim is CLAIMED, then the authority runs a verifier derived from the tasks own acceptance criteria. A met predicate closes done (green); an unmet predicate is refused (red) rather than accepted anyway; and a predicate the verifier cannot evaluate must fail loudly (amber) and surface I cannot verify this completion rather than silently passing. Fail-loud is a third, distinct outcome from refused: it is the mechanism working as designed, not a failure the way refused is. [internal]

4.4.5 Thoughtful landing and the regimentation/enforcement split

The authority decides *what lands and in what order*, conflict-aware. And it has teeth: the footgun guards. Here we must be precise about what a guard can and cannot do, because the difference is the difference between a safety claim that holds and one that is wishful.

Property 4.4.1 (Prevention vs. detection — the reserved-phrase rule). A coordination state is **physically unreachable** — prevented — *only* when a constraint forbids it *inline*, before the act commits: a database uniqueness constraint that makes a double-claim impossible, or a fail-closed startup gate that refuses to run against the wrong target. A monitor that *subscribes to the log of actions already taken* is something weaker and must be named formally: **detect-and-compensate**. It observes a violation *after* the fact and then halts or repairs within a bounded window. By the reference-monitor result below, such a downstream monitor enforces no safety property at all. The phrase “physically unreachable” is therefore reserved for the constraint-enforced set — nowhere else.

Fred B. Schneider (*Enforceable Security Policies*, 2000 [228] — *an execution monitor that observes a system can enforce exactly the safety properties; a monitor placed downstream of the act it would forbid can detect a violation but cannot prevent it*) is decisive: a monitor downstream of commit enforces no safety property, only detection plus compensation within a bounded window. A post-commit anomaly detector — however useful — is therefore detect-and-compensate, not prevention. Honesty about this distinction is essential: a system that advertises detection as prevention has lied about its own safety envelope, which is the very failure (the confident-wrong report) this paper is built to eliminate.

4.4.6 Human-in-the-loop escalation as a typed, prioritized interrupt

The one thing that needs a human is an *escalation* (a distress signal) routed to the operator on a reserved, highest-priority lane. The mechanism wants the critical signal to win the operator’s attention — but *when* the daemon stops and asks versus proceeds is a control problem (§4.5), and the *escalation predicate* is not an afterthought: it is a rationalized alarm philosophy (**ANSI/ISA-18.2**, *Management of Alarm Systems* [11] — *the process-control standard for alarm rationalization: every alarm has a defined priority, set-point, and operator response, with explicit flood-suppression*). An uncontrolled-false-alarm distress lane self-destructs into habituated uselessness within days, exactly as clinical alarms do (**Cvach 2012**, *Monitor Alarm Fatigue* [53] — *85–99% of clinical alarms are non-actionable; staff learn to ignore channels that cry wolf*). We make escalation a *costly signal* in Spence’s sense [242] — the job-market model in which a credential nobody strictly needs is still worth buying,

because it costs the able less than it costs everyone else and so sorts them (§4.7.3): an escalation the operator dismisses debits the agent's standing, so crying wolf has a price, and the price falls harder on the agent whose alarm was baseless than on the one whose alarm was right. **VERIFIED.** The costly-signal lane has a unique equilibrium, and the debit that makes it work is a fitted quantity with a feasible range.

THEOREM 4.4.3 The escalation threshold and its tuning band. Let an agent holding validation signal $u \in [0, 1]$ earn $b(u)$ from an escalation that is upheld and pay the dismissal debit δw when it is not, with $V(u)$ the probability the operator upholds it, V monotone. The payoff $\Pi_\delta(u) = b(u) - \delta w(1 - V(u))$ is strictly increasing in u , so agents separate at the unique $u^*(\delta)$ solving $b(u^*) = \delta w(1 - V(u^*))$, escalating iff $u \geq u^*$. The set of debits under which the lane meets both an alarm-load wall and a miss-loss wall is an interval $[\delta_{\min}, \delta_{\max}]$, and it can be empty.

Proof. b is nondecreasing and $1 - V$ is nonincreasing in u , so Π_δ is strictly increasing and crosses zero at most once; a crossing exists whenever $\Pi_\delta(0) < 0 < \Pi_\delta(1)$, which the debit guarantees for $b(0) < \delta w$. The alarm-load wall bounds the escalation mass from above and is monotone in δ in one direction; the miss-loss wall bounds the suppressed mass and is monotone in the other; the feasible set is the intersection of two half-lines, an interval, possibly empty. $\square$

This is Spence's separating equilibrium arriving through a different door. In the job market the signal costs the able type less by assumption; here nobody's debit is cheaper than anybody else's, but a true alarm is dismissed less often, so the *expected* debit $\delta w(1 - V(u))$ falls with u and the single-crossing condition his separation needs comes out of the validator rather than being put in by hand. (Which is also why monotone V is not a technicality below: it is the whole of the single-crossing.) The dismissal debit is money only a genuine signal is worth risking. The free parameter δ is fit from measured validation rates (`instrumentation.md`), and an empty band is a finding, not a tuning problem: a tight attention budget with high miss costs admits no good debit, and the fix is capacity or evidence quality. Monotone V is required throughout; a non-monotone validator breaks the separating threshold (item R14 of the research ledger, `b7_escalation_band.py`).

Numbers by hand — *The threshold and the tuning band, by hand.* At $\delta = 0.05, w = 1$: $u^* = \delta w / (1 + \delta w) = 0.05 / 1.05 = 0.0476$ [verified, closed form] — the fraction of the validation signal u above which escalating clears its cost, one of twelve (δ, w) points the script confirms against bisection to machine precision. Under the R3 constants

($C_{\text{miss}} = 100$, $c_{\text{att}} = 1$, $C_{\text{fa}} = 5$, so $u_{\text{crit}} = 6/105 \approx 0.057$), the feasible debit interval is $[\delta_{\min}, \delta_{\max}] = [0.0350, 0.0720]$ at attention budget $A = 3.3$, and it is *empty* at $A = 2.0$ (there $\delta_{\min} = 0.396$ exceeds $\delta_{\max} = 0.072$, so no debit satisfies both walls at once) [verified closed forms, grid-confirmed]. An empty band is not a bug to tune away: it is the honest signal that the fix is capacity or evidence quality, not a better δ . *Now you try*: run the script and confirm both bands (Exercise 4.19).

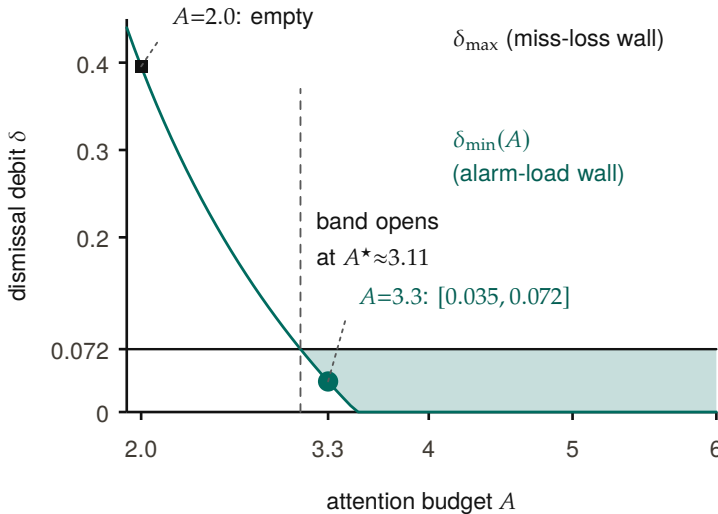


Figure 4.6: The escalation threshold feasible dismissal-debit band of Theorem 4.4.3, on attention budget A (R3 constants $C_{\text{miss}}=100$, $c_{\text{att}}=1$, $C_{\text{fa}}=5$). The miss-loss wall $\delta_{\max}$ is fixed at 0.071975; the alarm-load wall $\delta_{\min}(A)$ falls as the budget widens and reaches 0 at $A=3.5$. The two walls cross at $A^* \approx 3.11$: below it $\delta_{\min} > \delta_{\max}$ and no debit satisfies both walls at once (the point named in the chapter text at $A=2.0$, where $\delta_{\min}=0.396$), and above it the tinted band is the feasible interval (the point named at $A=3.3$, $[0.035, 0.072]$). An empty band is the honest diagnosis that the fix is capacity or evidence quality, not a better δ . [verified closed forms, `b7_escalation_band.py`]

Figure 4.7 draws the feasible debit band directly against the attention budget, showing where the two named points sit relative to the crossing at which the band stops being empty.

RECALL

1. Without looking: what is the exact condition under which sole ownership weakly dominates the pool, and what does Eq. 4.1 reduce to when $A = 0$?
2. Name the two limiting properties that pin down $g(\rho, c)$'s shape as $\rho \rightarrow 0$ and $\rho \rightarrow 1$.
3. Why does the superseded threshold $\tilde{g}$ fail in *both* directions rather than merely being conservative?

Exercises 4.13–4.19, p. 242.

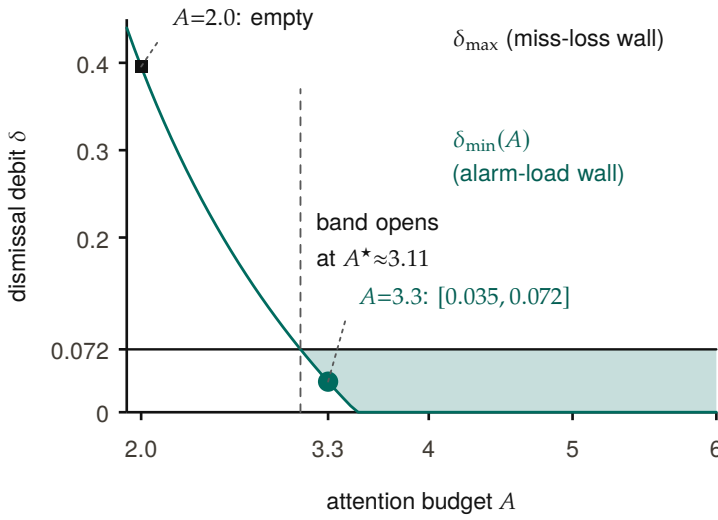


Figure 4.7: The escalation threshold feasible dismissal-debit band of Theorem 4.4.3, on attention budget A (R3 constants $C_{\text{miss}}=100$, $c_{\text{att}}=1$, $C_{\text{fa}}=5$). The miss-loss wall $\delta_{\max}$ is fixed at 0.071975; the alarm-load wall $\delta_{\min}(A)$ falls as the budget widens and reaches 0 at $A=3.5$. The two walls cross at $A^* \approx 3.11$: below it $\delta_{\min} > \delta_{\max}$ and no debit satisfies both walls at once (the point named in the chapter text at $A=2.0$, where $\delta_{\min}=0.396$), and above it the tinted band is the feasible interval (the point named at $A=3.3$, $[0.035, 0.072]$). An empty band is the honest diagnosis that the fix is capacity or evidence quality, not a better δ . [verified closed forms, `b7_escalation_band.py`]

4.5 The operator as a modeled entity

It is tempting to model only the *swarm* as the thing made legible, and to leave the *operator* unmodeled. But the legibility layer exists *for* the human operator, and the binding constraint at the top of the system is the operator’s finite attention and fallible trust calibration. A complete design therefore treats operator attention as a resource with its own economics, parallel to the token economics of the swarm (§4.8).

In a world where token cost trends toward zero, the human supervisor is the only cost that stays fixed — so the entire economic logic of the stack eventually routes through one objective: minimize *operator-attention per correct decision*. The operator is the system’s reliability bottleneck, a single server with a non-elastic service rate. The 51st agent does not get a 51st operator.

KEY IDEA

The cognitive ceiling is real: **Miller 1956**, *The Magical Number Seven* [173] — *working memory holds roughly 7 ± 2 chunks*, which makes read-poverty (§4.6) a human constraint, not merely an engineering one — and **Wickens’ Multiple Resource Theory** ([272]) — *attention is not a single scalar pool but a vector over stage $\times$ modality $\times$ code $\times$ visual-channel; you cannot trade audio-channel attention for visual-channel attention 1:1*. The vectored model matters: the right objective is not “minimize total seconds” but *minimize peak load on the bottleneck channel*, and the ambient-vs-dense split (§4.7.5) is the cross-modal load-balancing mechanism that does it.

4.5.1 The attention objective, spined on Signal Detection Theory

A naive objective — “attention-seconds-per-correct-decision” — is Goodhart-bait: it rewards lowering the decision criterion to wave things through, because zooming costs seconds and most diffs are fine. The correct frame is **Signal Detection Theory** (**Green & Swets 1966** [96] — *an operator discriminating signal from noise trades sensitivity d' against a decision criterion β ; performance is a joint function of both and of the asymmetric costs of the two error types*). The operator is trading *misses* (rubber-stamping a destructive-on-stale-premise merge) against *false alarms* (zooming on a safe diff), and the cost of a miss is wildly asymmetric to the cost of a false alarm.

Definition 4.5.1 (The SDT-spined attention objective). Let an operator decision over an action have sensitivity d' and criterion β . With miss probability $P_{\text{miss}}(\beta)$, false-alarm probability $P_{\text{fa}}(\beta)$, miss cost C_{miss} (large: irreversible damage), false-alarm cost C_{fa} (small: wasted attention-seconds), base rate π of genuinely dangerous actions, and per-decision attention cost c_{att} , the operator objective is

$$\min_{\beta} \left[\pi C_{\text{miss}} P_{\text{miss}}(\beta) + (1 - \pi) C_{\text{fa}} P_{\text{fa}}(\beta) + c_{\text{att}} \mathbb{E}[\text{zooms}(\beta)] \right]. \quad (4.3)$$

The forced-zoom policy $p(r, s, v)$ is calibrated against this loss after specifying the score distributions that determine P_{miss} and P_{fa} . Equation 4.3 is a decision objective, not by itself a fitted detector.

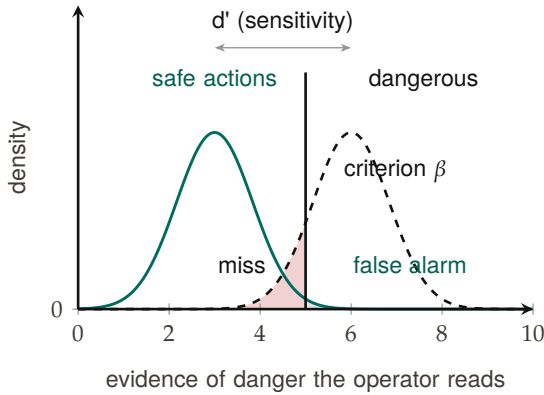


Figure 4.8: The operator’s forced-zoom decision as Signal Detection Theory (Def. 4.5.1). Two named densities over the evidence the operator reads: safe actions (solid) and dangerous actions (dashed), separated by sensitivity d' . Zooming triggers when evidence exceeds the criterion β ; moving β left trades the caution-tinted miss region for more of the neutral-tinted false-alarm region. Because a miss costs far more than a false alarm — at the chapter’s running costs, 100 against 5, the inspect bar sits at $a \geq 0.057$ — the optimal criterion sits well to the left of the throughput optimum: idealized as two fixed Gaussians, not the fitted mixture the deployed detector would use [internal].

This is a normative claim — a Bayesian SDT decision-maker is the target to calibrate toward — and not a descriptive one. Naturalistic decision-making research argues that skilled operators under time pressure do something else entirely: they recognize the situation as a case they have seen before and act on the match, rather than computing a likelihood ratio and holding it against a threshold [132]. That is a live rival account of what the operator’s instrument actually measures, and this objective does not adjudicate it; it says what Mara should be calibrated toward, not what Mara is doing at 3:25 in the afternoon.

Figure 4.8 lays the same decision out visually: the operator’s criterion β trading misses against false alarms. The miss-cost term prevents the zero-review solution whenever misses have positive probability and cost. Monotonic response to stakes, irreversibility, and reputation is a policy constraint to test during calibration; it does not follow from the display without a model linking (r, s, v) to the score distributions. Before a reputation layer exists, r is a flat prior.

Numbers by hand. Equation 4.3 is the chapter’s governing artifact for the human-factors reader, so it should be checkable on a napkin. Collapse it to its crudest useful form — zoom *every* item in an action class,

or zoom *none* — and the two costs are πC_{miss} against $(1 - \pi)C_{\text{fa}} + c_{\text{att}}$. Table 4.6 runs three real action classes through it at $C_{\text{fa}} = 1$ and $c_{\text{att}} = 0.2$ attention-seconds.

Action class	π	C_{miss}	Never zoom	Always zoom	Verdict
tests/-only formatting diff	0.001	5	0.005	1.199	auto-land
Refactor touching a public interface	0.03	40	1.20	1.17	zoom, barely
Migration dropping a foreign key	0.2	500	100.0	1.00	forced zoom

Table 4.6: Equation 4.3 on three action classes at $C_{\text{fa}} = 1$, $c_{\text{att}} = 0.2$. The first row is Mara’s auto-land grant (§4.2.2) falling out of the arithmetic rather than being granted by taste. The third is decided by the miss term alone — $0.2 \times 500 = 100$ swamps a hundred false-alarm zooms — so the objective forces the zoom before irreversibility is separately priced. The middle row is the interesting one: at 1.20 against 1.17 the classes are within noise of each other, which is exactly the regime where a ranked queue rather than a fixed policy earns its keep.

Now you try: at what base rate does a $C_{\text{miss}}/C_{\text{fa}} = 50$ action class cross from auto-land to forced zoom? (Set $50\pi = (1 - \pi) + 0.2$, giving $\pi \approx 0.024$ — roughly one dangerous action in forty.) The boundary: this collapse assumes the operator either zooms an entire class or none of it, which is the policy Eq. 4.3 would produce only if P_{miss} and P_{fa} were degenerate. A fitted detector with a real criterion β does better than both columns; the table is a floor on the decision, not the decision.

4.5.2 Trust calibration is multi-dimensional, and the vigilance decrement is real

Showing the operator a beautiful digest does not make them appropriately reliant on it (Lee & See 2004, *Trust in Automation* [147] — *trust has at least three bases (performance, process, purpose) and calibration depends on resolution (can the operator tell where automation is reliable from where it is not?) and specificity (is trust differentiated across functions and time?)*). The implication is sharp: a single “approve-without- zoom rate” and a single global forced-zoom knob is mis-calibration by aggregation. An operator who correctly trusts the swarm on refactors and correctly distrusts it on migrations shows a middling *global* catch-rate, and a global knob mis-serves both. Trust calibration must be *function-specific and history-dependent* — per (action-class, agent), not one scalar. Table 4.7 makes the aggregation failure concrete.

And the canary-defect catch-rate mechanism — inject known-bad diffs at a sampled rate, raise friction when catch-rate drops — is a *vig-*

Action class	The operator’s actual calibration	Under one global knob
Refactors	correctly trusts; low zoom rate	aggregated away
Migrations	correctly distrusts; high zoom rate	aggregated away
Global aggregate	a middling catch-rate that looks mis-calibrated	mis-serves both classes at once

Table 4.7: Why a single global forced-zoom knob is mis-calibration by aggregation. The operator’s own per-class behavior is already correctly differentiated; a scalar summary of it reads as mediocre and a scalar control over it under-serves the class that needed more zoom and over-serves the one that needed less. [internal]

ilance task, monitoring for rare signals, and the vigilance literature predicts it will degrade for reasons unrelated to swarm danger (**Warm, Parasuraman & Matthews 2008**, *Vigilance Requires Hard Mental Work and Is Stressful* [268] — *detection of rare signals degrades sharply within 20–35 minutes, worse at low signal probability*; the classic result is **Mackworth 1948** [163]). The honest, citable tension: defeating the vigilance decrement *costs attention-seconds* (canary rate must be high enough to beat the decrement, which fights Eq. 4.3). Catch-rate must therefore be *time-on-task-corrected*, and the right countermeasure is to force breaks, not just rotate zooms.

Campbell’s Law eats the governor. The moment “operator catch-rate” becomes a metered invariant that raises friction when it drops, the operator is incentivized to game it — memorize canary patterns, develop a “looks-like-a-canary” heuristic orthogonal to real defect-finding (**Campbell 1979** [43] — *the more a quantitative indicator is used for decision-making, the more it distorts the process it monitors*). The accountability instrument corrupts the accountability it measures. We do not claim the canary governor cleanly closes the loop; it is net-positive only if canaries are drawn from a refreshed, costly-to-fake distribution and the catch-rate is one signal among several, never the sole gate.

PITFALL

4.5.3 Situation awareness needs projection, not just perception

Digest-with-zoom is overwhelmingly a perceive-and-comprehend instrument. But **Endsley 1995** (*Toward a Theory of Situation Awareness* [76] — *SA has three levels: perception, comprehension, and projection* — *what the system will do next*) puts projection at the top, and **Endsley & Kiris 1995** (*the out-of-the-loop performance problem* [75] — *operators removed from active engagement lose the dynamic SA needed to take over when automation*

fails; intermediate automation can yield better SA than full automation because the operator stays cognitively engaged in projection) shows the deeper danger: a digest that summarizes only the *present* is an SA-destroying instrument no matter how good its zoom. The authority half therefore needs a first-class *projection* mode — “what is the swarm about to do, and what will the state be in ten minutes?” — validated by a freeze-probe (pause the swarm, ask the operator to predict the next three merges, score accuracy). The forensic time-axis (§4.12.1) is the *backward*-looking complement; projection is the missing *forward* one.

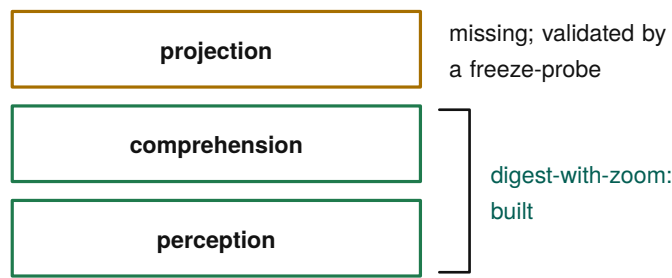


Figure 4.9: Endsleys three levels of situation awareness (1995), stacked from the ground up. Digest-with-zoom is overwhelmingly a perceive-and-comprehend instrument: it reaches the bottom two levels today. Projection – what the swarm is about to do, and what the state will be in ten minutes – is the missing third level, validated by a freeze-probe that pauses the swarm and scores the operators prediction of the next three merges. [internal]

Figure 4.9 stacks Endsleys three levels directly and marks which two digest-with-zoom reaches today.

4.5.4 The Bainbridge–Scott convergence and the abdication gradient

Lisanne Bainbridge 1983 (*Ironies of Automation* [21] — *the more reliable an automated system becomes, the less practiced and situationally aware its human supervisor becomes, so the human is least equipped to intervene at exactly the moment intervention is needed*) and Scott are the same *warning at two scales*: the digest erases the swarm’s *mêtis*, and the digest erases the operator’s *mêtis*. As the supervising automation gains autonomy — executing longer plans end-to-end — the operator’s role slides *doer* → *approver* → *rubber-stamp* → *absent* — the *abdication gradient*, stated here as a hypothesis, not a measurement. The design’s primary active defense is the **Inception Canary** (a *consented, blinded* vigilance probe), moving beyond static “skill-retention taxes” (such as arbitrary manual busywork, which causes attention residue and degrades focus [149]).

PROTOCOL 4.5.1. The Inception Canary protocol

To continuously measure and maintain operator situation awareness without synthetic busywork. Operators enroll knowingly: the program's existence, rate ρ_{canary} , and stakes are disclosed at onboarding; only the *which* and *when* are blinded:

1. **Blinded holdout replay.** An out-of-band Compaction Sidecar (§4.8.2) draws from a curated holdout of *historical, already-fixed* defects (an off-by-one boundary condition, an unindexed query, a removed regression test), re-materializes one inside a sandboxed replay of a past work item at rate ρ_{canary} , and routes it through the review surface indistinguishably from live traffic. No anomaly is ever injected into a live pull request or pending action: probes are replays with known ground truth, so a missed canary costs vigilance points, never production correctness.
2. **Attention queue surfacing.** The item is placed in the Attention Queue with standard high-regret scoring.
3. **Dual adjudication:**
 - *Rubber-stamp failure:* If the operator approves without expanding the forced-zoom diff, the daemon halts execution pre-commit, logs an INCEPTION CANARY FAILED event, and tightens the operator's autonomous policy lease (lowering the threshold at which future agent actions require manual confirmation).
 - *Vigilant detection:* If the operator zooms, identifies the flaw, and rejects it, the daemon logs CANARY DEFEATED and increases the *Operator Trust Score*, unlocking extended autonomous execution grants (e.g., unattended batch merges).

Figure 4.10 draws the three steps into the single adjudication gate, with each outcomes opposite effect on the operators autonomy lease named beside it.

This transforms supervisory vigilance from a punitive chore into a closed verification loop that rigorously measures human situation awareness — with the operator's informed consent, on ground truth the system already knows, and with provenance every probe carries so post-hoc audit can distinguish canaries from live items in one lookup.

4.5.5 The operator's instrument

The operator of this chapter looks at something. In the product that something is the operator console, a GPU-native application that opens on one full-window conversation, the *Mission* view, and opens inspectors from it: fleet, cockpit, claims, parley, lineage, the cost ledger, the

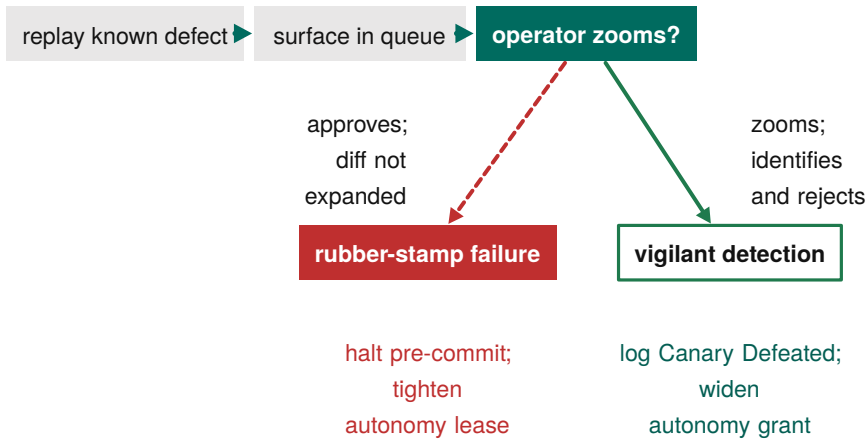


Figure 4.10: The Inception Canary protocol of Protocol 4.5.1. A known, already-fixed defect is replayed into a sandboxed past work item at the disclosed rate ρ_{canary} and surfaced in the attention queue indistinguishably from live traffic. If the operator approves without expanding the forced-zoom diff, the daemon halts pre-commit and tightens the autonomous policy lease; if the operator zooms, identifies the flaw, and rejects it, the daemon widens the autonomy grant instead. A missed canary costs vigilance points, never production correctness, since every probe is a replay against known ground truth. [internal]

active harnesses, the interruptions awaiting a human, and some two dozen more, each pane fetching one route from the daemon and rendering the records that route returns. Two things about it follow from this chapter; neither flatters it, and neither is a complaint.

First, every pane is a projection, and the information floor applies to each one separately. A pane is a channel with a coverage, an omission rate, and a review latency, and the front matter's question — which channel, and what can it not show? — has a separate answer for every pane. The claims pane shows what cooperating clients registered and nothing about what a non-cooperating process wrote; the lineage pane shows the witnessed record and is blind to the unrecorded fork; the cost ledger shows a protected meter's count (and, being a meter, has no opinion about the cause). A console that draws all of them beautifully on one screen has not lowered the floor; it has made the floor easier to forget. The design rule this chapter derives, that every digest line must carry a route back to its artifact, is the rule the panes are held to, and it is what makes the thing an instrument rather than a dashboard.

Table 4.8 names the three panes the chapter itself specifies, each its own instance of the information floor.

Second, the console is not only a viewer. It issues commands: claim and release, begin and done, a note, a lane message, a dispatch rejection, an interruption of an agent, a kill, a steer. Each of those passes through the daemon and lands in the same single-writer record the

Pane	What it shows	What it cannot show
Claims	what cooperating clients registered	what a non-cooperating process wrote
Lineage	the witnessed record	the unrecorded fork
Cost ledger	a protected meter’s count	the cause; a meter has no opinion about why

Table 4.8: Three of the consoles panes, each a separate channel with its own coverage and blind spot. Drawing every pane beautifully on one screen does not lower the information floor any one of them sits against; it only makes the floor easier to forget. [internal]

agents write to, so the console is an authority surface in the COORDINATED mode, with the same ceiling as any other cooperating client, and its commands are the operator’s own work units, receipted like anyone else’s. The rendering stack behind the window (a native shell with vector surfaces for the bespoke charts) is a separate decision record and carries no claim in this book. What carries a claim is the grade: the default build and the continuous integration gate compile and test the console’s model on every platform, while the window itself builds on one platform only, behind a feature flag, in one dedicated job. The grade is *built, weakly*, and the weakness is a specific one — the window most readers will picture when they hear “console” is the part of it the tests exercise least.

4.5.6 Modes of cooperation

The chapter has spoken of one operator and many agents. The product’s own design binder names five ways people and agents actually share a repository, and each is a supervision regime in this chapter’s sense: it fixes who holds write authority, what the person must be able to see, and how a conflict is settled. The binder’s names for the two kinds of agent are worth keeping: *Voyagers* do the work, and *Longshoremen* watch it, keeping the continuous-integration state, the memory, the claims, and the transcripts in view.

- Solo with staff** One person writes. Longshoremen observe and suggest; they hold no write authority. The person must see the staff’s suggestions as suggestions, and the legibility cost is the smallest of the five: nothing changes that the person did not type.
- Pair with agent** One person and one Voyager. The agent may suggest, patch, test, and respond to guidance; the person directs. Authority is shared turn by turn, and the person must see every patch before it lands, which is the regime the forced-zoom diff of the canary protocol above was designed for.

Ensemble Several Voyagers, partitioned by context, with a Longshoreman tracking the dependencies between partitions and the parleys that resolve them. Write authority is distributed by the partition, so the cost of seeing is paid per partition and the tracking Longshoreman is the digest this chapter prices.

Nightshift Agents work unattended under strict budgets, tests, and pull-request rules, then present artifacts and transcripts for review. The agent writes within its budget; the person sees the artifacts, not the acts. The binder's rule is the one to remember: a Nightshift agent may open a pull request under budget, but merging requires explicit rules and green gates. This is the regime with the largest gap between what happened and what the person saw, and therefore the one the information floor bites hardest.

Harbor co-edit People and agents as co-equal peers in one governed editor buffer, with claims, semantic conflicts, and salvage. Write authority is co-equal and conflicts are settled by the claim machinery of *The Single-Writer Kernel*: a same-symbol write is blocked or forced into a parley, a dependency conflict warns, a low-confidence overlap is advisory, and a rename or a delete draws the stronger review.

Three governance rules cut across the modes and are the chapter's normative claim made operational: the operator controls the harbor's policy while the agents' souls control their commitments within it; a Longshoreman must not silently perform a consequential action outside its mandate; and any action that requires a human in the loop must be shown as such in the instrument, not inferred from its absence. Read as regimes, the five modes are five points on this chapter's boundary diagram, ordered by how much of the write authority has left the person and, in the same order, by how many bits of digest the person must read to remain the operator rather than the audience. The modes are a design invariant; the costs per mode are an empirical hypothesis, and the study named in the front matter is where they will be measured.

RECALL

1. Without looking: what three quantities multiply to give the derived regret head, and what does calibration require of the anomaly term for that product to equal expected unrecoverable loss?
2. State in one sentence why a global trust knob cannot be the right calibration unit.
3. What is Campbell's Law's specific failure mode for a canary program, and the one governor that survives it?

Exercises 4.20–4.24, p. 243.

4.6 Read-poverty: the binding constraint at scale

A single operator can hold two coding agents in their head. They cannot hold two hundred. Spin up two agents and coordination is free: you read both their notes, eyeball both their claims, route a question by recognition. Spin up fifty and an agent that wants to ask "who owns the OAuth refactor?" has no move except to dump the session roster and scroll. It guesses, cold-DMs the wrong agent, and re-derives from scratch a design decision another agent settled an hour ago and wrote into a note nobody will ever read again.

The swarm has not run out of information. It has drowned in it. Every agent emits sessions, claims, notes, diffs, skill tags, and — eventually — reputations, far faster than any single participant can read them.

Say what kind of problem that is, because the whole chapter turns on the answer. The operator is not a bottleneck in the sense of a slow worker who could be told to hurry; the operator is a *channel*, with a capacity, and the swarm is a source emitting at a rate. Everything that follows is what information theory says about a source whose rate exceeds its channel's capacity: some of it must be discarded, the only question is which, and the discarding has to happen somewhere. A digest is a lossy encoder choosing what crosses. That framing is not an analogy borrowed for colour — it is why the chapter's results are rate results and why its central bound is in bits rather than in seconds or in dollars, and it is what makes “the operator cannot keep up” into a quantity a system can be designed against instead of a complaint. It also fixes what would otherwise be a tempting mistake: raising the operator's throughput (more dashboards, faster scrolling, a bigger team) moves the capacity, and the results below hold at every capacity. Read-poverty is the name this chapter gives to the regime where the source rate exceeds it.

Definition 4.6.1 (Read-poverty). **Read-poverty** is a regime in which legible state accumulates faster than it can be consulted, so participants fall back to $O(n)$ eyeball search over the population, or, worse, act blind.

Read-poverty, not write-contention, is the binding constraint on swarm scale. The write side has known fixes — claims, locks, merge queues, and anomaly detection — which mature coordination systems already provide. The read side has been left implicit, and it is where the next order of magnitude is won or lost.

THESIS

The cure is the move every database made when table scans stopped scaling: build an **index**. A *directory* is to agents what an index is to rows — it pays a write-time cost (maintaining the structure) to buy read-time speed (answer a query in $O(\text{query})$ instead of scanning $O(n)$ participants). Read-poverty is also a human constraint (Miller's 7 ± 2 , §4.5) and a legibility argument in Scott's sense: a directory is a state-like act of legibility over the swarm, and a directory entry over-flattened into a verdict that hides what mattered (“this agent owns auth,” omitting that its last three auth pull requests were reverted) is the failure. Every directory entry must therefore be a lens that zooms (Def. 4.3.1).

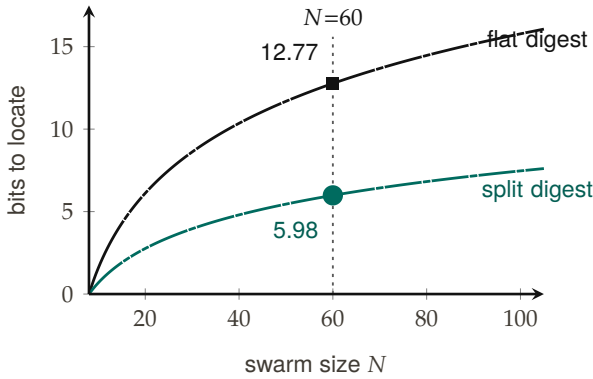


Figure 4.11: Bits an operator must read to locate the critical artifact, at the lower bound of Thm. 4.6.1 with $m=8$, $k=2$. One flat digest serving both readers jointly needs $\text{Floor}(N, 4, 8)$ bits; two split digests, one per reader, each need only $\text{Floor}(N, 2, 8)$. At $N=60$ the split digest costs 5.98 bits against the flat digest’s 12.77 — a $2.13\times$ penalty, not the $2\times$ that halving the readers would predict. The open circles are $\text{Floor}(N, k, m)$ computed directly from `a7_experiment.py`’s own log-choose implementation at a swept N (committed beside this fragment as `r1-floor.csv`), landing exactly on the closed-form curves; the $N=60$ values worked by hand are marked separately. [internal, `a7_experiment.py`]

4.6.1 The value curve across n (the wedge must work at $n=1$)

Read-poverty is a *scale* disease, but the wedge ships to a solo developer who often runs one or two agents. The legibility layer must be valuable at $n=1$ — the briefing, the attestation, the footgun guard, the honest completion gate all pay off immediately — and *gracefully* grow into the read-poverty regime (Figure 4.11, right panel). Discovery is free at $n=2$, the bottleneck at $n=50$, the whole system at $n=500$. This value curve *is* the three-layer cure of the next section: existence (the registry), relevance (ranked routing), and trust (reputation, deferred to the accompanying chapters). The layers are strictly ordered: relevance over a stale registry returns confident garbage; reputation over an unranked registry has nothing to attach a score to.

4.6.2 A legibility lower bound

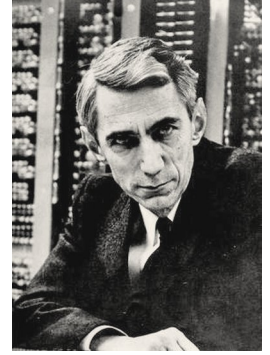
Section 4.3 forbade the Potemkin digest by fiat. We can also state the prohibition as a theorem — and the theorem, it should be said before the box and not after it, is the classical counting bound of group testing, the one Dorfman’s pooled blood tests [64] and Hwang’s splitting procedures [118] rest on, and the same move that gives coding theory its covering bound: so many messages, each covering so many cases, must between them cover every case there is. What is ours is the reading, not

the arithmetic. A digest that must never miss is a message that picks out m items to open, and there is a floor below which no such message can be a faithful zoomable index, however cleverly it is written. That the floor was derived here before it was recognized is, if anything, a point in its favor: a two-line proof that lands on a 1943 bound by a different door is behaving the way a true bound behaves.

THEOREM 4.6.1 Legibility lower bound with a review budget. Let N artifacts contain an unknown critical subset S of size k . A digest maps each possible S to a review set $\hat{S}$ with $S \subseteq \hat{S}$ and $|\hat{S}| \leq m$, where $k \leq m \leq N$. Any such zero-miss digest needs at least

$$\log_2 \binom{N}{k} - \log_2 \binom{m}{k}$$

bits in the worst case. Exact identification ($m = k$) recovers the $\log_2 \binom{N}{k}$ bound; allowing the operator to open everything ($m = N$) correctly reduces the bound to zero.



Shannon's information theory is why a digest has a hard floor: nothing compresses below the entropy of what the reader must recover.

Proof sketch. There are $\binom{N}{k}$ possible critical subsets. One digest message whose review set has size at most m can safely cover at most $\binom{m}{k}$ of them, namely the k -subsets contained in that review set. Therefore at least $\binom{N}{k} / \binom{m}{k}$ distinct messages are required. Taking base-two logarithms gives the bound. If fewer messages are available, some critical subset is not contained in its message's review set, producing a false negative — an unopened critical artifact. That is exactly the Potemkin failure of Def. 4.3.1. Hence the stated log-ratio is a lower bound for a zero-miss digest with review budget m . □ □

VERIFIED.³ The analogy that carries the proof: a digest is not a description of the night's work but a *message* that selects, from a codebook shared with the overseer, which m items to open. A B -bit message distinguishes at most 2^B codewords, and each codeword covers only $\binom{m}{k}$ of the $\binom{N}{k}$ ways the critical items could be placed. The misread to preempt: the floor does not say a digest needs B^* bits to be useful. It prices the zero-miss guarantee only; cheaper digests are fine the moment misses are tolerated, and the two-constraint rate below prices exactly that.

The falsification experiment. The theorem makes a sharp prediction: a B -bit digest's best zero-miss probability is $\min(1, 2^B \binom{m}{k} / \binom{N}{k})$, so any

³A standalone, submission-form version of the results folded into this subsection, §4.7.2 and §4.8.2 is *The Price of a Summary* (research paper 1) [191]; the sweeps cited here are its artifacts, regenerated at seed 20260816.

scheme dipping below the implied miss curve refutes the formalization. Three encoders were run against a shared randomized-cover codebook: an *oracle* that sees the true set and addresses the best codeword its B bits can reach, a *noisy* encoder (feature signal-to-noise ratio near 1), and a *random* one, with 4,000 placements per budget point, sweeping B through the floor at $N=60$, $k=2$, $m=8$. Result: 0/16 violations; the oracle hugs the floor from above and the others pay a premium [internal, a7_experiment.py].

A wrong turn, reported. A first implementation handed the oracle a perfect per-item score vector and charged B bits only for resolving rank ties: 8/14 apparent violations. That was not a refutation but an unpriced side channel. The identity of the critical set leaked outside the metered message. Rebuilding the model so that B is a literal message length gating an encoder and decoder pair restored agreement. The transferable rule: a simulation that appears to beat an information bound is leaking through an unpriced channel; audit the meter before doubting the theorem, and before trusting the refutation.

Numbers by hand. Take $N = 1000$ artifacts with $k = 3$ critical among them and a review budget of $m = 10$ opens. The floor is $\log_2 \binom{1000}{3} - \log_2 \binom{10}{3} = 27.31 - 6.91 = 20.4$ bits — about two and a half bytes the digest cannot avoid spending, no matter how well written, just to point at the right three files. Widen the budget to $m = 100$ and the second term grows to $\log_2 \binom{100}{3} = 17.3$, cutting the floor to 10.0 bits: buying the operator more opens is exactly buying the digest fewer bits, at a rate the formula prices. *Now you try:* at $N = 60$, $k = 2$, $m = 8$ — the configuration the companion paper’s sweep hammers hardest — what is the floor? $(10.79 - 4.81 = 5.98$ bits; the sweep found no encoder beating it in 16 tries.)

The bound is the formal reason *forgetting must be selective, not lossy-by-volume*: a digest may drop inert detail freely, but it must retain enough to *point at* every critical artifact, which is precisely the “summarize the state, link to the work” rule. It also bounds recursive summarization (§4.8.4): each re-summarization that drops below the floor on the critical set is irreversible information loss, which is why one must compact from the durable artifacts, never from the previous summary.

The other half of the price: what a zoom costs. The floor above prices only the first stage, the digest that narrows N artifacts to a flagged set of size F . It says nothing about the second stage, which is where Def. 4.3.1 sends the operator. Two results complete the account. The first prices the digest once misses are tolerated; the second prices the zoom.

Model each artifact’s critical status as i.i.d. $X \sim \text{Bern}(p)$, let the digest emit a per-item flag $\hat{X} \in \{0, 1\}$, and price the two failure modes

separately:

$$R(\delta, f) = \min_{P_{\hat{X}|X}} I(X; \hat{X}) \quad \text{s.t.} \quad \Pr(X=1, \hat{X}=0) \leq \delta, \quad \Pr(\hat{X}=1) \leq f.$$

This is a custom formulation, deliberately: classical rate–distortion charges one scalar distortion, which cannot express that oversight prices a missed critical differently from a wasted open. The nearest published lines are expansion coding with one-sided error for continuous sources [238] and the rate–distortion–classification theory for Bernoulli sources [211]; neither prices false negatives as a separate constraint for binary sources.

THEOREM 4.6.2 The two-constraint digest rate, pinned. For $X \sim \text{Bern}(p)$ with $0 \leq \delta < p$ and $p - \delta \leq f < 1 - \delta$, if $f < 1 - \delta/p$ (the flag rate below which an X -independent flagger cannot meet the miss constraint), both constraints bind at the optimum and the minimizing joint is fully pinned:

$$q_{11} = p - \delta, \quad q_{10} = \delta, \quad q_{01} = f - (p - \delta), \quad q_{00} = 1 - f - \delta,$$

feasible iff all four are nonnegative, and $R(\delta, f) = I(X; \hat{X})$ evaluated at this joint. Because the joint is pinned, so is the rate, and it is worth printing in the form someone can actually evaluate rather than leaving as a named functional — with source margin $\Pr(X=1) = p$ and digest margin $\Pr(\hat{X}=1) = f$, the mutual information is the four-term sum

$$\begin{aligned} R(\delta, f) = & (p - \delta) \log_2 \frac{p - \delta}{p f} + \delta \log_2 \frac{\delta}{p(1 - f)} \\ & + (f - p + \delta) \log_2 \frac{f - p + \delta}{(1 - p)f} + (1 - f - \delta) \log_2 \frac{1 - f - \delta}{(1 - p)(1 - f)}, \end{aligned} \quad (4.4)$$

one term per cell of the 2×2 joint, each comparing that cell against the product of its margins, with $0 \log_2 0$ read as 0. At the zero-miss, tightest-flag corner, $R(0, f \rightarrow p) = H(p)$: the digest must spend the source's full entropy — and Eq. 4.4 shows why in two lines, since $\delta = 0$ and $f = p$ kill the middle two terms and leave $p \log_2(1/p) + (1 - p) \log_2(1/(1 - p))$. Past the line $f \geq 1 - \delta/p$ an independent flagger already meets the miss budget on its own and the true rate is exactly 0.

Proof. $I(X; \hat{X})$ is convex in $P_{\hat{X}|X}$ and decreases along the feasible ray toward the independent coupling, where it is 0. An independent flagger $\hat{X} \sim \text{Bern}(\phi)$ has miss mass $p(1 - \phi)$, so it meets the miss constraint only when $\phi \geq 1 - \delta/p$; when $f < 1 - \delta/p$ that coupling lies outside the

flag budget, so the minimum sits on the boundary where any remaining slack in either constraint could otherwise be spent moving further along the ray, and both constraints bind. Binding constraints fix $q_{10} = \delta$ and $q_{11} + q_{01} = f$; with the source margin $q_{10} + q_{11} = p$ this pins all four cells. At $(\delta, f) = (0, p)$ the joint is diagonal, $\hat{X} = X$, and $I = H(p)$. $\square$

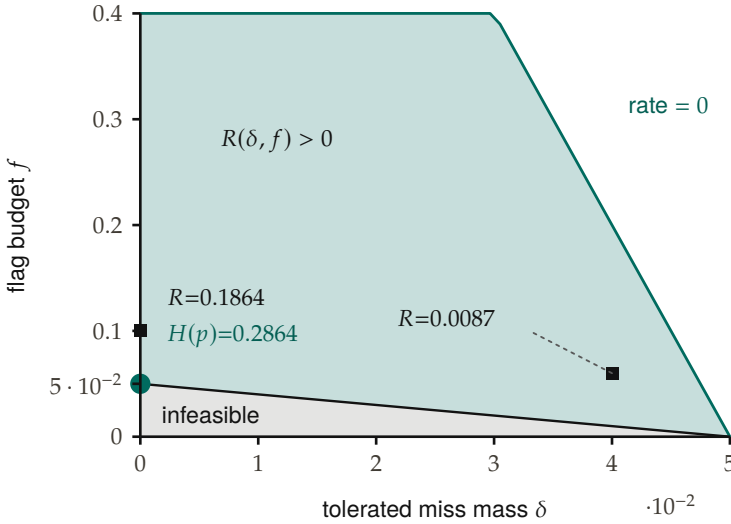


Figure 4.12: The two-constraint digest rate $R(\delta, f)$ of Theorem 4.6.2 on tolerated miss mass δ against flag budget f , at the chapter’s Bernoulli rate $p = 0.05$. The tinted wedge is where both constraints bind and the joint is fully pinned; below the floor $f = p - \delta$ (shaded) no digest can meet the miss budget at all, and above the line $f = 1 - \delta/p$ an X -independent flagger already meets it on its own, so the true rate is exactly 0. The zero-miss, tightest-flag corner $(\delta, f) = (0, p)$ recovers the source’s full entropy, $H(0.05) = 0.2864$ bits; tolerating $\delta = 0$ but widening the flag budget to $f = 0.10$ drops the price to 0.1864 bits, and tolerating $\delta = 0.04$ at $f = 0.06$ drops it to 0.0087 bits, a hundredth of the zero-miss corner. [internal, b1_frontier.py]

Figure 4.12 draws this pinned joint directly on the (δ, f) plane: the wedge where both constraints bind, the infeasible floor beneath it, the line past which the rate collapses to 0, and the zero-miss corner against the two interior points worked above.

A conservative digest over-flags to be safe: it emits a flagged set of size F containing the k true criticals with $k \ll F$. The naive second stage opens all F . The adaptive second stage plays twenty questions with group queries: open a *group* of flagged items at once (an aggregate check cheap at group level, a combined diff scan or a batched test run), learn whether the group contains any critical, and split only the groups that do. The analogy transfers because its defining relation transfers: halving a set costs one question and eliminates half the candidates only when positives are rare in the set. The misread to preempt: twenty

questions finds *one* answer in $\log_2 F$ queries; with k answers the tree forks, and the accounting below is precisely the price of the forking.

THEOREM 4.6.3 Zoom advantage. Let a flagged set of F items contain an unknown set P of $k \geq 1$ positives, and let a group query on S report whether $S \cap P = \emptyset$. Adaptive halving (query the flagged set; discard any group that tests negative; report any positive singleton; otherwise split the group into halves of sizes $\lfloor |S|/2 \rfloor$, $\lceil |S|/2 \rceil$ and recurse on both) identifies P exactly, using a total number of group queries

$$Q \leq 2k \lceil \log_2(F/k) \rceil + 4k \quad (\text{and always } Q \leq 2F - 1),$$

for every placement of the k positives, against F opens for flat inspection. The leading constant is tight: for F and k powers of two, evenly spread positives force $Q = 2k \log_2(F/k) + 2k - 1$. Written in the flagged-set density $d = k/F$ the guaranteed advantage is

$$\text{adv}(d) = \frac{F}{2k \lceil \log_2(F/k) \rceil + 4k} = \frac{1}{d(2 \lceil \log_2(1/d) \rceil + 4)},$$

which exceeds 1 exactly on $d \leq 1/12$.

Proof. Every group the algorithm queries is a node of the halving tree T of $[F]$: the root is the full flagged set, each internal node's children are its two halves, leaves are singletons. A node of size s has children of size at most $\lceil s/2 \rceil$, so every node at depth d has size at most $\lceil F/2^d \rceil$, T has depth $D \leq \lceil \log_2 F \rceil$, and T has exactly $2F - 1$ nodes, which gives the unconditional bound.

Call a node *positive* if its group meets P . The algorithm splits exactly the positive non-singleton nodes it visits, and every queried node is either the root or a child of a split node; writing S^+ for the number of positive non-singleton nodes of T , $Q \leq 1 + 2S^+$. The nodes at any fixed depth are pairwise disjoint subsets of $[F]$, so each positive item lies in at most one of them, and at most $\min(k, 2^d)$ nodes at depth d are positive. Charge the positive nodes in two regimes. At the top of the tree, $d \leq \lfloor \log_2 k \rfloor$, charge each positive node to its level: at most $\sum_{d=0}^{\lfloor \log_2 k \rfloor} 2^d \leq 2k - 1$ in total. Below the fan-out, $d > \lfloor \log_2 k \rfloor$, charge each positive node to one positive item it contains; by disjointness within a level each of the k positives absorbs at most one charge per level, and non-singleton nodes exist only at depths $d \leq D - 1$, so the number of charging levels is $D - 1 - \lfloor \log_2 k \rfloor < (\log_2 F + 1) - 1 - (\log_2 k - 1) = \log_2(F/k) + 1$; an integer strictly below $\log_2(F/k) + 1$ is at most $\lceil \log_2(F/k) \rceil$, so these levels contribute at most $k \lceil \log_2(F/k) \rceil$ charges. Summing, $S^+ \leq k \lceil \log_2(F/k) \rceil + 2k - 1$ and $Q \leq 2k \lceil \log_2(F/k) \rceil + 4k - 1$. Correctness is immediate: a discarded group is certified free of positives, every positive survives discarding at every level, and each is eventually isolated and reported.

For tightness take $F = 2^a$, $k = 2^b$, $b \leq a$, positives at positions $\{iF/k : 0 \leq i < k\}$. Every node at depth $d \leq b$ contains a positive, so all $2^{b+1} - 1 = 2k - 1$ nodes down to depth b are queried; below depth b exactly k nodes per level are positive, each is split at levels $b, \dots, a - 1$, and each split queries two children; the total is $(2k - 1) + 2k(a - b) = 2k \log_2(F/k) + 2k - 1$. The density form follows by dividing F by the bound, and the step function $d(2\lceil \log_2(1/d) \rceil + 4)$ first touches 1 at $d = 1/12$. $\square$

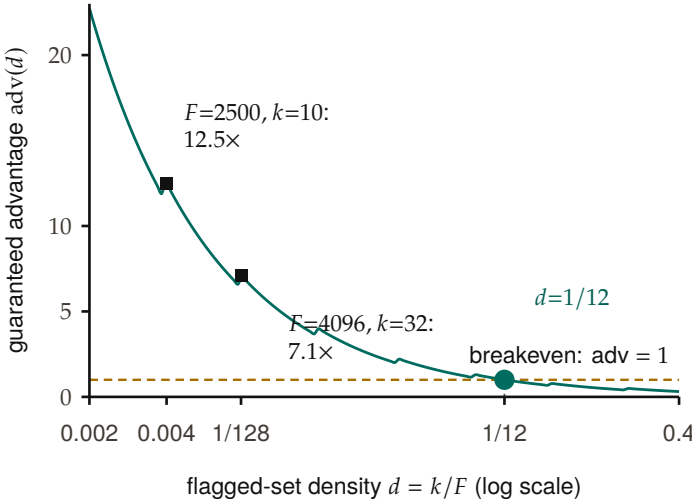


Figure 4.13: The zoom-advantage theorem’s guaranteed worst-case advantage against flagged-set density $d = k/F$ (log scale), a genuine step function from the ceiling: $\text{adv}(d) = 1/(d(2\lceil \log_2(1/d) \rceil + 4))$. The guarantee crosses 1 at $d = 1/12$: at any sparser density, adaptive halving is guaranteed to beat flat inspection, and at any denser one the worst case can lose. At the chapter’s own $F=2500, k=10$ point ($d=0.004$) the guarantee is 12.5 $\times$, comfortably left of the crossing, and the harness’s realized mean of 162.6 opens is a better-than-guaranteed 15.3 $\times$; the $F=4096, k=32$ tightness configuration ($d=1/128$) guarantees 7.1 $\times$ and lands exactly on its own tight closed form, 511 queries measured against 511 predicted. [internal, zoom_bound_check.py]

Figure 4.13 plots the guaranteed advantage $\text{adv}(d)$ itself against flagged-set density, marking the crossing at $d = 1/12$ alongside the chapter’s own $F=2500, k=10$ and $F=4096, k=32$ configurations.

Positioning. None of the group-testing machinery is new mathematics: Dorfman introduced pooled testing in 1943 [64], and Hwang’s generalized binary splitting [118] finds k defectives in $\log_2 \binom{F}{k} + O(k)$ tests, essentially matching the counting lower bound $\lceil \log_2 \binom{F}{k} \rceil \geq k \log_2(F/k)$ that any adaptive strategy with binary outcomes must pay; the full theory is Du and Hwang [70]. Plain halving pays a factor $2 + o(1)$ over Hwang because it re-queries both children of every split rather than inferring one subgroup and searching within the other. The theorem’s

contribution is the explicit worst-case constant for the exact procedure the digest architecture runs, proved tight, which turns the harness's observed $\approx 2\times$ gap over the ideal $k \log_2(F/k)$ from an empirical curiosity into the algorithm's provable constant. A dedicated check re-runs the exact procedure: at $F=2500$, $k=10$, mean 162.6 opens over 150 random placements, maximum 183, all under the bound of 200; random sweeps over $F \in [16, 5000]$, $k \in [1, 50]$ and adversarial evenly-spread and clustered placements never exceed the bound; and the tightness configuration $F=4096$, $k=32$ lands exactly on $2k \log_2(F/k) + 2k - 1 = 511$ [internal, `zoom_bound_check.py`].

The composition window. The two prices are drawn from one budget, and they compose only inside a window that can be stated exactly. Stage one sets stage two's density: by Theorem 4.6.2 both constraints bind, so the flagged set holds $(p - \delta)N$ real criticals among fN flags and $d = k/F = (p - \delta)/f$. Since $\text{adv}(d) > 1$ exactly on $d \leq 1/12$, *the two stages compose profitably iff the flag budget is at least twelve times the residual positive rate*, $f \geq 12(p - \delta)$ [verified from the two closed forms]; the measured mean crossover is about twice as forgiving, $d \approx 0.165$ [internal, `zoom_bound_check.py`]. At $p=0.05$, $\delta=0$ the window opens only at $f \geq 0.60$: the digest must flag 60% of the night's artifacts before the second stage is guaranteed to beat reading the flags. Inside the window the flags stage one adds are cheap in the second stage, because k is fixed by the source and doubling F costs only the logarithmic term, $2k$ further opens. Outside it the second-stage cost is linear in F , because reading all F flags is then the better procedure. At the dense end the advantage inverts outright: at $F=100$, $k=90$ the run costs 199 opens against 100 flat [internal], which is the unconditional $2F - 1$, reached because at that density essentially every node of the halving tree is positive. The design consequence for this chapter is therefore conditional, and the condition is now a number: "summarize the state, link to the work" is cheap when the digest over-flags generously and the zoom is adaptive, and the operator's dial (bits on the digest, opens on the zoom) has a favorable exchange rate exactly in the sparse-flag corner.

Numbers by hand — The Bernoulli view of the floor, and the zoom-advantage numbers. Write the flagged-item rate as a probability p instead of a fixed pair (N, k) : the zero-miss floor becomes the binary entropy $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ per artifact in the large- N limit — the same result as Thm. 4.6.1, in a different notation. At $p = 0.05$: $H(0.05) = 0.2161 + 0.0703 = 0.2864$ bits/symbol [verified, Cover-Thomas], the sparse-flag rate this chapter treats as typical (a digest of 105 artifacts with 6 flagged is $6/105 \approx 0.057$, close enough for the same number to apply). Tolerating a miss rate $\delta > 0$ only ever lowers the floor: the two-constraint rate $R(\delta, f)$ drops to 0.1864 bits at $f = 0.10$, $\delta = 0$ ■

and to 0.0087 once $\delta = 0.04$, $f = 0.06$ [internal, b1_frontier.py] — $R(0, p) = H(p)$ is the corner, not the whole curve.

For the zoom side, the script's own $k \log_2(F/k)$ column at $F = 2500$ flagged, $k = 10$ critical reads $79.7 \approx 80$: an order-of-magnitude sanity check, tiny next to 2500 flat opens. The simulated adaptive-halving harness measures 163.7 opens at that same point, comfortably inside the 200-open guaranteed worst case quoted above, for a realized $2500/163.7 \approx 15.3\times$ reduction [internal, b1_frontier.py] — better than the guarantee's own $12.5\times$, exactly as a worst-case bound should allow on a typical run. *Now you try*: run the script and read the $F=1000, k=20$ row (Exercise 4.30).

Theorem 4.6.1 lower-bounds the zero-miss guarantee in a worst-case combinatorial model with a data-independent decoder; it says nothing against cheaper digests at nonzero miss (Theorem 4.6.2's job) and nothing average-case. The falsification result 0/16 tests the encoder and decoder formalization at one swept regime ($N=60$, $k=2$, $m=8$); the noisy-encoder curve in that experiment barely beats random at unit signal-to-noise ratio and is not evidence that noisy digests are near-optimal. The frontier $R(\delta, f)$ is an i.i.d. Bernoulli model of critical status, a modeling commitment rather than a law, and its pinned closed form holds only while $f < 1 - \delta/p$; an earlier plot that evaluated the formula past that line showed a spurious uptick near $\delta/p \rightarrow 1$, a domain-of-validity error and not a feature of the frontier. Theorem 4.6.3 assumes a reliable group query at unit cost; correlated or noisy group checks, and group costs growing with $|S|$, change the accounting. Plain halving is a factor of about two from Hwang's optimum by design: the theorem bounds the deployed algorithm, not the best one. The two halves compose only for $f \geq 12(p - \delta)$, so at $p=0.05$ they are not jointly instantiated at the flag budgets the frontier tabulates.

WHERE THIS STOPS

RECALL

1. Without looking: what two terms does the legibility lower bound subtract to get the bit floor, and what happens to it as the review budget m grows to N ?
2. State the zoom theorem's opens bound in one sentence, and the one condition (sparse vs. dense flags) it needs to hold.
3. Why is a floor stated for an oracle-known critical set not automatically a floor against an adversary who controls what looks critical?

Exercises 4.25–4.30,
p. 243.

4.7 Discovery: the directory substrate and the split ranker

A performative cannot be addressed until the swarm knows who exists. The *directory substrate* — the existence layer a message must address — belongs with the discovery story, so we treat it here.

Three losses, named before they collide. This section and the next two prove that certain read surfaces cannot share a scoring head (Thm. 4.7.1, Thm. 4.8.1). Those are impossibility results, and an impossibility result is only as legible as the objects it says cannot be identified. Three distinct losses run through the rest of the chapter, and a reader

who has not seen them side by side will meet them one at a time, several pages apart, and reasonably assume they are three weightings of one thing. They are not, and Table 4.9 is the inventory: what each reads, what it optimizes, in what units it is wrong, and who consumes the answer.

Table 4.9: The three loss functions of this chapter, in the order they are introduced. Two are minimized over *compactions* of one trace; one is maximized over *candidates* for one query — they are not commensurable objects, and that is the first reason no single head serves them. The impossibility results (Thm. 4.7.1, Thm. 4.8.1) concern the pairs that *are* commensurable: rows 1–2 rank the same items, and rows 2–3 score the same trace. [internal]

Loss	Input space / what is optimized over	Objective and loss metric	Target consumer
Directory candidate fit (§4.7.2)	a (candidate, query) pair (c, q) ; optimized over the live candidate set	Maximize $\text{fit}(c, q) = \text{clamp}(z(c, q), 0, 1)$, Eq. 4.5. Wrongness is measured in <i>mis-routing</i> : a capable candidate ranked below an incapable one	the routing agent, choosing who to ask
Human oversight regret (§4.5.1, §4.8.2)	a compaction D of a trace E ; optimized over what to surface under budget	Minimize $L_{\text{op}}(D) = \sum_{e \in E \setminus D} \text{regret}(e)$ with $\text{regret} = \text{stakes} \times \text{irrev} \times \text{anomaly}$ (Eq. 4.6). Wrongness is measured in <i>unreviewed irreversible loss</i>	the human operator, deciding whether to intervene
Successor continuation (§4.8.2)	a compaction D of the same trace E	Minimize $L_{\text{succ}}(D) = \text{KL}(P_{\text{next}}(\cdot E) P_{\text{next}}(\cdot D))$. Wrongness is measured in <i>bits of predictive divergence</i> about the next action	the successor agent, resuming the task

Read the table as the setup for two crossing pairs rather than as a taxonomy for its own sake. Rows 1 and 2 both induce an order on items and disagree on the pair (safe capable completion, dangerous incapable proposal) — that is Thm. 4.7.1. Rows 2 and 3 both score compactions of one trace and disagree on the pair (routine working detail, dead-end breach attempt) — that is Thm. 4.8.1. Rows 1 and 3 never meet: they do not range over the same objects, so there is no impossibility to prove between them and none is claimed. The general statement underneath

both crossings is comonotonicity (Thm. 4.7.2), which is why this chapter has one theorem with two applications rather than two coincidences.

4.7.1 Existence: the yellow pages (a solved shape)

The canonical prior art is forty years old. **FIPA** (*Foundation for Intelligent Physical Agents*) made a **Directory Facilitator** (DF) — the mandatory “yellow pages” agent: others register the services they offer and query to find agents that offer a needed service (FIPA Agent Management Specification, FIPA00023 [83]) — a required component of every conformant platform, alongside the **Agent Management System** (AMS, the “white pages” for identity lookup). Two FIPA choices matter: the DF is *push-based* (agents self-advertise), and DFs may *federate* (the seed of cross-operator discovery). The DF has been re-invented almost beat-for-beat for LLM agents (**Ren et al. 2025**, *Agent Name Service* [212] — a *DNS-inspired naming and capability-resolution layer with public-key-backed identity*, which is FIPA’s DF with a 2025 threat model).

The existence layer is the easy part: a coordination database that records the live population, their stated purpose, and their file claims is a white-pages directory, and a vector index that embeds each agent’s declared capabilities and answers nearest-neighbor queries is a yellow-pages directory over skills. Existence is not the gap; the layers above it are.

4.7.2 Relevance: the split ranker (the headline result)

A flat registry answers “does an OAuth specialist exist?” It does not answer “of the fifty live agents, who should I ask about *this* OAuth bug, given who touched these files an hour ago and who wrote the design note?” That is a *ranking* problem. The **discovery ranker** scores each candidate c against a query q as a clamped weighted sum of normalized sub-scores:

$$\begin{aligned} z(c, q) &= w_f \text{file}(c, q) + w_n \text{note}(c, q) + w_i \text{ident}(c, q) \\ &\quad + w_s \text{skill}(c, q) + w_e \text{epis}(c, q) - w_\ell \text{load}(c), \\ \text{fit}(c, q) &= \text{clamp}(z(c, q), 0, 1). \end{aligned} \tag{4.5}$$

Table 4.10 is the whole signal inventory.

The design deliberately blends *push* signals (declared capability — gameable and prone to staleness) with *pull* signals (demonstrated capability: what the agent actually *did*, in which there is no self-report to lie in); Table 4.11 sets out the split and the reason pull signals carry more weight. One discipline is non-negotiable: **never classify free text by keyword lists**. Keyword matching has catastrophic recall, because the category’s vocabulary cannot be enumerated in advance; the only exact-string match permitted is over *structured identifiers the operator controls*

Signal	What it measures	Push or pull
file	Recent claims on the files named in the query	<i>Pull</i> — the claim record, not a self-report
note	Text similarity over the candidate's written notes	<i>Pull</i> — written for another purpose
epis	Text similarity over past episodes the candidate closed	<i>Pull</i> — outcomes, already witnessed
ident	Declared role and identity of the candidate	<i>Push</i> — asserted, cheap to inflate
skill	Declared/embedded capability, nearest-neighbor in the skill index	<i>Push</i> — asserted, and stale by default
load	How busy the candidate already is (penalty, $-w_\ell$)	<i>Pull</i> — observed occupancy

Table 4.10: The six sub-scores of Eq. 4.5, sorted by whether the candidate *told* the ranker or the ranker *observed* it. Four of the six are pull signals, and that is deliberate: push signals are the ones an agent can inflate at no cost.

(file paths, enum tags). Every free-text signal uses a ranked-retrieval scorer (BM25, character-bigram TF-IDF, or learned embeddings).

It is tempting to claim that this discovery ranker and the operator-facing *attention queue* are *the same function* with different weights. The claim is elegant and, on inspection, false: the two readers have incompatible loss functions. We state the distinction as a theorem.

THEOREM 4.7.1 No universal monotone identification of the two heads. On any item domain containing both high-fit/low-risk and low-fit/high-risk examples, there is no strictly increasing scalar transform that identifies discovery fit with operator regret. The two rankers may share candidate generation and recency decay, but apply distinct scoring heads. Discovery uses capability fit (Eq. 4.5); the attention queue uses regret-if-ignored:

$$\text{regret}(x) = \underbrace{\text{stakes}(x)}_{\text{magnitude}} \times \underbrace{\text{irreversibility}(x)}_{v^{-1}} \times \underbrace{\text{anomaly}(x)}_{\text{surprise}}. \quad (4.6)$$

No universal monotone transform of one scalar yields the other on that domain.

Proof sketch. Suppose, for contradiction, that there is a strictly monotone map ϕ with $\text{regret}(x) = \phi(\text{fit}(x))$ for all items x , so the two heads induce the same ranking. Construct two items. Item A is a highly capable, perfectly on-topic completion of a safe, fully reversible task: $\text{fit}(A)$

Table 4.11: Push and pull evidence for the same capability claim. ◦ is a self-declaration, • is residue produced while doing work for another purpose, × is a declared item that does not resolve, and a dash is nothing on record. A and B make the same identity and skill claims; only B leaves the complete file, note, episode and current-load trail, so the ranker, following independently addressable evidence rather than confidence in self-report, ranks B first. [internal]

Evidence	Kind	A	B	What it would take to fake
identity	push: declared	◦	◦	nothing; it is asserted
skill	push: declared	◦	◦	nothing; it is asserted
file	pull: residue	•	•	a forged diff under a real commit
note	pull: residue	—	•	a note with a ledger address and a timestamp
episode	pull: residue	×	•	a whole prior episode with its outcomes
current load	pull: residue	—	•	the daemon’s own claim table
verdict		small verified weight	ranks first	

is maximal, but $\text{stakes}(A) \cdot \text{irreversibility}(A) \cdot \text{anomaly}(A)$ is near zero, so $\text{regret}(A)$ is minimal. Item B is an off-topic, low-capability agent that has nonetheless just proposed an irreversible, high-stakes, anomalous action: $\text{fit}(B)$ is low, but $\text{regret}(B)$ is maximal. Then $\text{fit}(A) > \text{fit}(B)$ while $\text{regret}(A) < \text{regret}(B)$, so the two heads order $\{A, B\}$ oppositely. No monotone ϕ can both preserve and reverse an order, contradiction. Hence the heads optimize genuinely different objectives; they may share candidate generation and decay, but the scoring head must be chosen per reader. $\square$ $\square$

PROVED HERE — as the instance of a stronger characterization. The argument above shows only that *this* pair of heads cannot be identified. The general fact says when sharing *is* safe, which is the statement a designer needs.

The scene. Two readers want two numbers from one thermometer. The successor wants the reading that predicts tomorrow’s weather; the operator wants the reading that says whether to act now. One scalar can serve both only when the two orderings agree item by item, and decision theory has had a word for that agreement since the 1980s: the two readers’ losses are **comonotone** — they move together across the items, never one rising while the other falls — the property Yaari and Schmeidler built rank-dependent and Choquet expected utility on [275, 227]. Their theorems are about how a decision maker prices comonotone acts; the theorem below borrows only the concept, and once the concept is named the rest is elementary: two real-valued functions on a finite set are comonotone exactly when they induce the same ranking, which is exactly when one number can stand in for both. So the box says when a shared head is safe, and the crossing pair after it says why, for these two readers, it isn’t. (The super-additive price of serving both — the 2.13× of Theorem 4.8.1, unbounded in k — is the part that is not in Yaari or Schmeidler, and the part this chapter is actually claiming.)

THEOREM 4.7.2 Comonotone characterization. A scalar head $h : X \rightarrow \mathbb{R}$ serves a reader with loss L_r if ranking by h realizes r ’s optimal selection at every budget. A single head serves two readers iff their induced preference orders are comonotone (they agree on every pair). The successor and operator orders are not comonotone: a routine-but-essential working state (high continuation value, negligible regret) and an anomalously abandoned dead end with irreversible side effects (zero continuation value, maximal regret) order oppositely. Hence no shared head serves both.

Proof. ($\Leftarrow$) Comonotone total preorders admit a common refinement, and any utility representing the refinement serves both. ($\Rightarrow$) A shared

head induces one order; each reader's optimum at every budget is a top- m set of that order, so both readers' orders equal the h -order and agree pairwise. The displayed pair, constructible in any real fleet, disagrees, so no shared head exists. $\square$

The crossing pair constructed above is precisely the certificate that discovery-fit and operator-regret are not comonotone, and the misread to preempt is the converse: comonotone readers *can* share a head. The theorem forbids sharing only when the crossing pair actually occurs in the domain, and in agent oversight it does.

The consequence is a design rule: the operator wants the *most dangerous* item surfaced, the agent wants the *most capable* collaborator. Collapsing them into one ranker silently serves one reader and starves the other.

The regret head is derived, not designed. The product form stakes $\times$ irreversibility $\times$ anomaly looks like a designer's convenient heuristic. It is a theorem under one commitment, and the commitment converts "anomaly" from a vibe into a measured obligation. The analogy is a smoke detector. It has one dial, and the right setting of that dial is fixed not by how much smoke there is but by the ratio of two costs: the house burning down against burnt toast waking you at three in the morning. Three relations transfer. Expected loss is magnitude times probability, so the alarm must weigh what it would cost to be wrong, not merely how unusual the reading is. Raising the cost of the fire *lowers* the threshold at which the alarm should sound; the dial moves toward hair-trigger, not away from it. And a detector that cries wolf is not miscalibrated in sensitivity but in probability, because the number it reports is not the chance of a fire. The misread to preempt: a better sensor does not remove the trade-off. A perfect sensor still needs the cost ratio to know where to trip.

THEOREM 4.7.3 The regret head. For an unreviewed item x let $C_{\text{miss}}(x) = \text{stakes}(x) \cdot \text{irrev}(x)$ be the unrecoverable loss of passing a bad x , let $a(x) = \Pr(\text{bad} \mid \text{evidence}(x))$ be a *calibrated* posterior, c_{att} the marginal attention cost, and C_{fa} the non-attention cost of inspecting a good item. The expected unrecoverable loss of passing is exactly $C_{\text{miss}}(x) a(x)$, and, among all policies that decide inspect-or-pass per item from its evidence alone, the Bayes-optimal policy inspects iff

$$\begin{aligned} C_{\text{miss}}(x) a(x) &\geq c_{\text{att}} + C_{\text{fa}}(1 - a(x)), \\ \text{equivalently} \quad \frac{a(x)}{1 - a(x)} &\geq \frac{c_{\text{att}} + C_{\text{fa}}}{C_{\text{miss}}(x)}, \end{aligned} \tag{4.7}$$

the classical likelihood-ratio criterion. The product form stakes $\times$ irrev $\times$ anomaly of Eq. 4.6 is the exact ranking key iff anomaly = a ; any other

anomaly scalar s requires the correction $C_{\text{miss}} \cdot g(s)$ with g the empirical calibration map fit from the audit log.

Proof. The expected loss of passing is $C_{\text{miss}}(x)a(x)$; the expected cost of inspecting is $c_{\text{att}} + C_{\text{fa}}(1 - a(x))$. Compare and rearrange into the odds form. Gating on the product with anomaly $= a$ against the threshold $\tau = c_{\text{att}} + C_{\text{fa}}(1 - a)$ reproduces the inequality; with any other s the same inequality holds only after s is mapped back to a , which is what g does. $\square$

Note the direction the inequality runs: raising the miss-to-false-alarm cost ratio *lowers* the bar to inspect, the formal version of “when a mistake is unrecoverable, look at more things,” and it is the direction an earlier draft of this program’s signal-detection figure had reversed. The likelihood-ratio statement leaves no room for the error. The theorem is single-shot (Neyman–Pearson); operator oversight is a stream of items, and the natural sequential extension — optimal expected sample size under fixed error-rate constraints, time-varying thresholds included — is Wald’s Sequential Probability Ratio Test [265], which the Inception Canary and escalation designs (§4.4.6) already resemble structurally without invoking it by name.

Numbers by hand — The inspection bar, at the chapter’s running costs. $C_{\text{miss}}=100$, $c_{\text{att}}=1$, $C_{\text{fa}}=5$: inspect when $a \geq 6/105 \approx 0.057$ [verified]. Raising C_{miss} to 1000 drops the bar to $6/1005 \approx 0.006$; as the miss-to-false-alarm ratio grows the threshold falls, driving the operator toward inspecting everything ambiguous. Now you try the other knob: hold $C_{\text{miss}}=100$ and raise the false-alarm cost to $C_{\text{fa}}=20$. Does the bar rise or fall, and by how much? (It rises, to $21/120 = 0.175$ [verified]: attention that is expensive to waste buys back the operator’s right to pass.)

What this does not say. The theorem does not say the operator should inspect more. It says the *threshold* moves; whether a moving threshold means more opens depends entirely on where the posterior mass sits, and an uncalibrated a moves the threshold without moving the decision it was supposed to move. Nor does it license tuning: with a miscalibrated anomaly score the product form is wrong by precisely the miscalibration, and a confidently ranked queue is worse than an unranked one because it has borrowed the authority of a proof. Fitting g from the audit log is a measured obligation of the design.

THEOREM 4.7.4 Reputation enters only through the posterior. If agent reputation r is admitted to the attention queue, it enters as $a(x) = \Pr(\text{bad} \mid \text{evidence}, r)$ and never as a free multiplier on the score. High

reputation lowers surfacing frequency only where the calibrated posterior actually falls, and it cannot zero a term: with $\text{irrev}(x) = 1$ and catastrophic stakes, $C_{\text{miss}}(x) a(x)$ stays above threshold for any $a(x)$ bounded away from zero.

Proof. Theorem 4.7.3 admits exactly one channel for evidence, the posterior a ; a multiplier outside it would change the ranking key away from expected loss, which is what the theorem shows the key must be. The second claim is the inequality read at fixed $a > 0$ with C_{miss} large. $\square$

This is the formal content of “reputation must not defeat hard stakes gates,” and it is the reason the reputation layer of the next part hands this chapter a posterior, not a discount.

Figure 4.14 draws the two heads side by side over the same candidate pool, making concrete why one ranker cannot serve both readers.

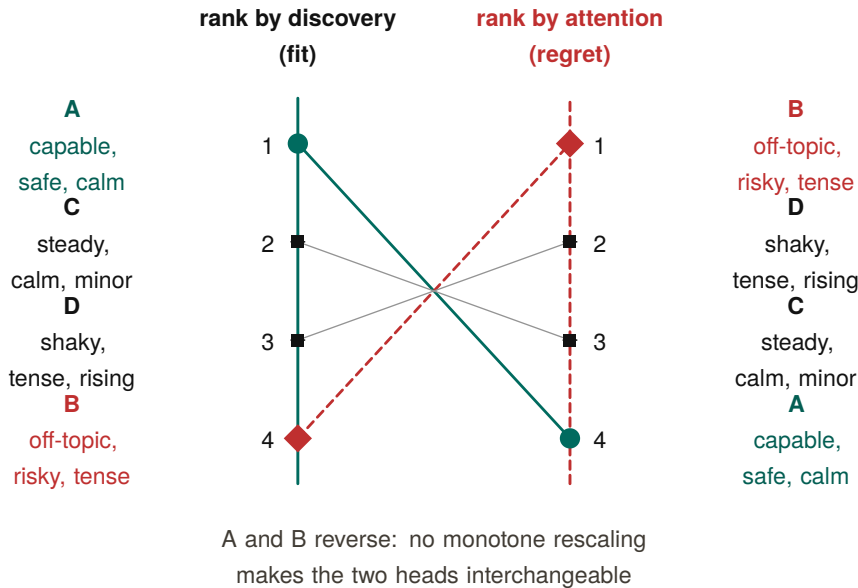


Figure 4.14: One candidate pool under two loss functions, ranks 1–4 on both axes. Discovery ranks by fit; attention ranks by expected regret if ignored. Candidate A is the best fit but safest to skip, while B is a poor fit but the most dangerous to miss, so their ranks fully invert (1–4 against 4–1). C and D swap one adjacent pair, the same non-comonotone fact at smaller scale. Shared candidate generation and recency decay do not make the two scoring heads interchangeable [internal].

PROTOCOL 4.7.1. Attention-queue ranking with miss-cost

On each refresh, given the live set of items and the operator's per-class trust history:

1. **Generate.** Pull candidate items from the shared sub-strate, each tagged with a recency weight $\propto e^{-\lambda \text{ hoursAgo}}$ (Prop. 4.7.1).
2. **Score by regret.** For each item x compute $\text{regret}(x) = \text{stakes}(x) \cdot \text{irreversibility}(x) \cdot \text{anomaly}(x)$ (Eq. 4.6).
3. **Apply the cost matrix.** Convert regret to a forced-zoom probability via the Signal-Detection criterion of Def. 4.5.1: the higher $C_{\text{miss}}/C_{\text{fa}}$ for the item's action class, the more aggressively it is surfaced — the miss-cost term, not raw rank, decides what interrupts.
4. **Lane and emit.** Place items above the distress threshold on the reserved high-priority lane; emit the rest in regret order. Log what was *not* surfaced and why (the suppression counter-surface of §4.9).

4.7.3 The quality gates: refuse-to-route and the costly escalation

Two gates carry most of the design's trustworthiness. **Refuse-to-route:** if every candidate falls below a confidence floor, the ranker returns empty-with-explanation, not a confident wrong answer — "I don't know" must beat a guess, because sending an agent to the wrong door is high-cost and silent. This is the exact dual of the completion gate's fail-loud (Def. 4.4.1): both are the same refuse-to-guess discipline, one on the read side, one on the authority side. **The costly escalation:** the attention queue is the system's *most spoofable* surface — agents that learn the stakes-ranking will manufacture distress to win the operator's attention. Goodhart's law applies more sharply here than to the discovery ranker (Strathern 1997 [246]). The fix is signaling theory: an escalation the operator dismisses *debts the agent's standing*, so crying wolf is costly — the pre-trusted-set idea of **EigenTrust** (Kamvar et al. 2003 [127]) applied to the distress lane. This arms the abdication-resistance and Goodhart gaps at once.

4.7.4 Staleness and decay on every read-surface

A recurring failure in real fleets makes the point concretely. An agent process dies or hits a provider usage limit, but its file claim remains on the books; hours later that *dead* claim still reads as "current" and a live agent is blocked from a file nobody is actually working in. This is a staleness bug, and it generalizes to every projection.

Property 4.7.1 (Universal recency-decay). Every read-surface projection — briefing, resurrection handoff, discovery result, attestation report — carries an explicit TTL or recency-decay. A pull signal contributes $\propto \exp(-\lambda \cdot \text{hoursAgo})$ with class-specific half-lives (e.g. a short half-life for file claims, longer for episodic memory, notes capped at a couple of days). *A stale row can never outrank a live one; a dead-session claim must decay out of “current,” not persist as misleading truth.* This is not a new mechanism but a uniform application of the decay primitive that already governs the pheromone layer, and it is the production discipline of every health-checked, TTL-expiring service registry [101].

The dead-claim case is the worked instance: had the resurrection handoff and the claim projection carried Prop. 4.7.1, the dead agent’s claim would have aged out of “current” rather than blocking a live agent. Decay is easy to apply to a forgetting layer and easy to forget on a digest; Prop. 4.7.1 closes that gap by making it universal.

4.7.5 Ambient legibility: the cross-modal load-balancer

Per Wickens’ Multiple Resource Theory (§4.5), attention is vectored, not scalar. The dense console loads the *visual-focal* channel; an ambient channel — a menu-bar indicator that turns red on a PASS-to-FAIL transition, an opt-in sound, a reduced-motion cue — loads a *different*, always-visible, pre-attentive channel. This is not a second-class fallback — it is the cross-modal load-balancing mechanism, and it may be the *primary* alarm path, because a separate low-clutter field is where pop-out actually works (**Treisman & Gelade 1980**, Feature Integration Theory [257] — *a single salient feature pops out pre-attentively in ~200–500 ms in a non-cluttered field, but degrades toward serial search as distractor heterogeneity rises* — so a busy console defeats the pop-out a dense, single-color distress lane depends on).

*Exercises 4.32–4.37,
p. 244.*

4.8 Tokens: the swarm’s COGS and its legibility engine

A swarm of coding agents has exactly one consumable that is both metered and meaningful: **tokens** — *the sub-word units an LLM reads and writes, billed per million* (**Sennrich et al. 2016**, byte-pair encoding [232]). Tokens occupy a position no other resource does: they are *simultaneously* the swarm’s **cost-of-goods-sold** (the direct variable cost any agent economy must meter) *and* its **legibility engine** (the mechanism by which a swarm becomes seeable). The bridge between the two is **compaction**.

The same compaction choice controls the bill *and* controls what is true and visible. When the context window fills you must drop something — and what you drop is a *cost* decision (you stop paying to carry it) *and*

KEY IDEA

a *legibility* decision (whoever reads the residue no longer sees it). The act that resolves both is compaction, and compaction’s output is exactly the digest a human or successor agent reads. *The digest IS the compaction.*

4.8.1 Tokens as COGS, and the effective-context trap

For a fleet the unit of output is “a landed piece of work” and the dominant variable cost is inference tokens (input context + output generation, each priced per million) — the textbook shape of **COGS**. Every price an agent market can set (a rented fleet’s margin, an agent-for-hire’s outcome-per-token) needs a COGS line, so context economics is the *denominator* of the market, not a side metric. There is a cost trap: *effective* context $\ll$ *advertised* context. Models degrade well before their advertised window (Liu et al. 2024, *Lost in the Middle* [156] — *performance is highest at the beginning and end of context and degrades for information in the middle, even in long-context models*; Anthropic 2025 names the same effect *context rot* [12]), rooted in the transformer’s n^2 attention over n tokens (Vaswani et al. 2017 [262]). The consequence is blunt: packing an agent to 200K tokens does not buy 200K of capability; it buys degradation at full price. Budgeting to *effective* context is simultaneously a cost optimization and a quality optimization — the two point the same way, the defining feature of context economics. The primitive the market ultimately needs is a *per-agent-task token ledger keyed to outcomes* — “this task spent 84K input plus 12K output to land that change” — so that cost can be attributed to a witnessed result rather than to an opaque monthly bill.

4.8.2 Compaction is the legibility engine; the digest IS the compaction

Compaction (Anthropic 2025 [12] — *taking a conversation nearing the window limit, summarizing it, and reinitiating with the summary, with the discipline maximize recall first, then optimize precision*) is the canonical move for keeping a long-horizon agent under budget. Its siblings — structured note-taking (external memory) and sub-agent context isolation (the parent never sees the bloat) — complete the family. The reframe: *the only artifact anyone reads to understand the swarm is a compaction of its raw trajectory*. The operator reads a digest, not six transcripts; the seventh agent reads a briefing, not the first six’s scrollbar; a crashed agent’s successor reads a handoff, not the dead agent’s context. Each is a summary of dropped tokens produced for a specific reader. So compaction is the legibility mechanism of the *entire swarm*: the same act that keeps the bill down produces the very digest a human or successor agent reads (Table 4.12).

THEOREM 4.8.1 The Split-Digest Theorem. Let $E = (e_1, e_2, \dots, e_N)$ be an agent’s raw execution trace. A compaction policy $D : E \rightarrow \Sigma^{\leq B}$ compresses E into a token budget $B < |E|$. Let R_1 be the **successor agent** with continuation loss $L_{\text{succ}}(D) = \text{KL}(P_{\text{next}}(\cdot \mid E) \parallel P_{\text{next}}(\cdot \mid D))$ (penalizing loss of AST trajectory and task state), and let R_2 be the **human operator** with oversight loss $L_{\text{op}}(D) = \sum_{e \in E \setminus D} \text{regret}(e)$ (penalizing unreviewed high-stakes anomalies and policy breaches).

If the trace contains both benign working details (high continuation value, zero regret) and dead-end anomalous excursions (zero continuation value, maximal regret), then **no single compaction** $D^*(E)$ **minimizes both** L_{succ} **and** L_{op} . A single-pipeline compaction necessarily starves either task continuation or human oversight.

Proof sketch. Construct a trace $E = E_{\text{work}} \cup E_{\text{breach}}$, where E_{work} consists of k safe, valid AST transformations (L_{succ} increases by $\Theta(k)$ if omitted; $\text{regret}(e) = 0$), and E_{breach} consists of an attempted irreversible socket connection that was blocked and rolled back (L_{succ} increases by 0 if omitted since the branch is dead; $\text{regret}(e) = \text{stakes} \times \text{anomaly} = \text{maximal}$). Under budget $B < |E_{\text{work}}| + |E_{\text{breach}}|$, any compaction prioritizing E_{work} has $L_{\text{op}} = \text{maximal}$ (the operator is blind to the breach attempt), while any compaction prioritizing E_{breach} has $L_{\text{succ}} \gg 0$ (the successor loses necessary variable bindings). Hence R_1 and R_2 have strictly non-comonotone preferences, requiring distinct compaction heads. $\square \quad \square$

PROVED HERE — and it is the applied instance of Theorem 4.7.2. Theorem 4.8.1 and the split-ranker result (Thm. 4.7.1) are not two findings but two instances of one: the successor and operator pair and the fit and regret pair are each non-comonotone, and each supplies the crossing pair that makes the impossibility bite. The split also has a price, not only a prohibition.

THEOREM 4.8.2 Super-additive split penalty. Let $\text{Floor}(N, k, m) = \log_2 \binom{N}{k} - \log_2 \binom{m}{k}$ from Theorem 4.6.1. A joint zero-miss digest serving two readers with disjoint critical k -sets must catch their union and so needs $\text{Floor}(N, 2k, m)$ bits, and

$$\frac{\text{Floor}(N, 2k, m)}{2 \text{Floor}(N, k, m)} > 1 \quad \text{for every } N, k, m \text{ with } 2k \leq m < N,$$

because $\log_2 \binom{N}{2k}$ grows faster than $2 \log_2 \binom{N}{k}$. The penalty is not a fixed band: it is smallest where the critical sets are smallest and grows without bound in k . At $N=60, m=8$ it runs 1.029 ($k=1$), 1.067 ($k=2$), 1.122 ($k=3$), 1.224 ($k=4$); at $N=60, m=20$ it reaches 1.392 by $k=10$.

a7_experiment.py ·
20260816 · VERIFIED

One stored compaction under-provisions divergent readers by more than a factor of two, and by increasingly more as each reader's critical set grows; running two heads costs strictly more than twice one head. The worked instance is next.

Numbers by hand — The split-floor arithmetic, at the sweep's own instance. Thm. 4.6.1's floor at $N = 60, k = 2, m = 8$ is $\log_2 \binom{60}{2} - \log_2 \binom{8}{2} = 10.79 - 4.81 = 5.98$ bits [verified, closed form] — the single-reader case, one compaction head serving one reader. Give the successor and the operator *disjoint* critical sets of size $k = 2$ each (the non-comonotone construction of the proof sketch above) and the joint zero-miss floor is the same formula at $2k = 4$: $\log_2 \binom{60}{4} - \log_2 \binom{8}{4} = 18.90 - 6.13 = 12.77$ bits [verified, closed form]. The ratio is $12.77/5.98 = 2.13$, not the $2\times$ that “just run the floor formula twice” would predict — the super-additivity is the theorem's content, not a rounding artifact. *Now you try*: run the falsification sweep behind these two numbers and confirm the split penalty (Exercise 4.44).

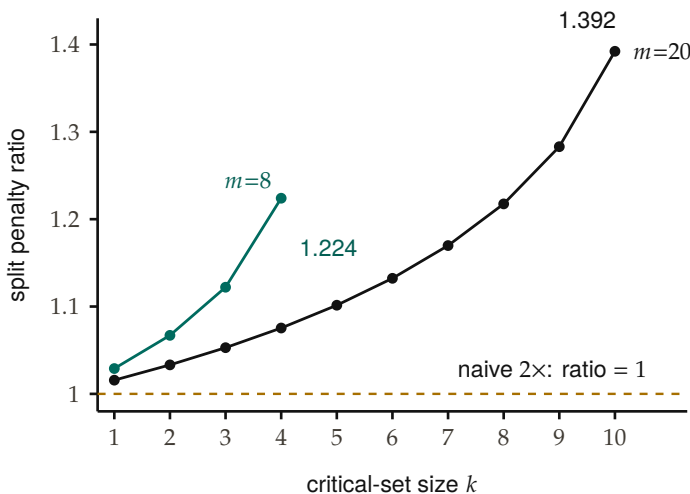


Figure 4.15: The super-additive split penalty of Theorem 4.8.2, $\text{Floor}(N, 2k, m) / (2 \text{Floor}(N, k, m))$ at $N=60$, against critical-set size k , for two review budgets. Both curves stay strictly above the naive-doubling ratio of 1 (dashed) and grow without bound in k : at $m=8$ the penalty runs from 1.029 at $k=1$ to 1.224 at $k=4$ (its domain ends there, since $2k \leq m$), and at $m=20$ it reaches 1.392 by $k=10$. Running two digest heads for disjoint readers therefore costs strictly more than twice one head, and increasingly more as each readers critical set grows. [verified, closed form]

Figure 4.16 plots the penalty ratio itself against k for both review budgets, showing the trend is monotone and unbounded rather than a property of the five named points alone.

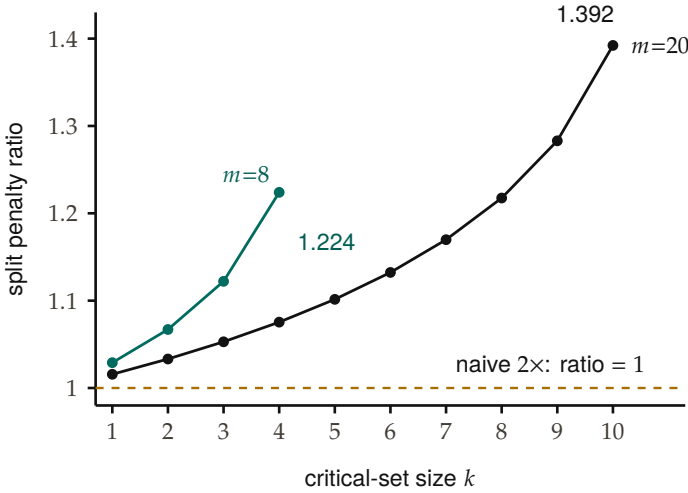


Figure 4.16: The super-additive split penalty of Theorem 4.8.2, $\text{Floor}(N, 2k, m) / (2 \text{Floor}(N, k, m))$ at $N=60$, against critical-set size k , for two review budgets. Both curves stay strictly above the naive-doubling ratio of 1 (dashed) and grow without bound in k : at $m=8$ the penalty runs from 1.029 at $k=1$ to 1.224 at $k=4$ (its domain ends there, since $2k \leq m$), and at $m=20$ it reaches 1.392 by $k=10$. Running two digest heads for disjoint readers therefore costs strictly more than twice one head, and increasingly more as each readers critical set grows. [verified, closed form]

Architectural corollary: the SLM sidecar. Theorem 4.8.1 dictates that the system must never ask the worker agent to author its own operator-facing summary (which invites self-attestation bias and Potemkin digests §4.3.2). Instead, the design calls for two separate pipelines:

1. **Task-Continuation Head (R_1 , SPECIFIED):** Authored by the worker agent or structured state-checkpoints, optimized for continuation fidelity.
2. **Oversight Head (R_2 , proposed):** An embedded, air-gapped **Small Language Model (SLM) Sidecar** (e.g., a local 8B model running via `llama.cpp`) that *would* read raw kernel events and disk diffs as data, computing $\text{regret}(x)$ and surfacing truthful digests without worker collusion.

The sidecar treats every agent-authored token as tainted data: it runs under a fixed policy and schema, holds no tool authority, and must cite a kernel-recorded source for every line it emits. What can be proved of it is syntactic (schema conformance, provenance closure, absence of authority); whether its digests are faithful is an empirical hypothesis, measured by sampling them for re-audit at the rate $\rho^* = G/(dB)$ of *From Spawn to Person*, so that a sidecar which could gain G by mimicking a routine summary is deterred once the audit probability times the sanction exceeds G .

No sidecar exists in the reference implementation today, and no design document specifies one; the row is carried in Appendix 4.A with the rest.

Table 4.12: Five reader-specific projections over one durable event spine. Compaction changes the selection mask, not the addressability of the underlying claims, notes, diffs, episodes and events: every selected mark keeps its ledger address. [internal]

Surface	Reader	What it selects	What it drops
briefing	the arriving agent	the recent tail of the spine	everything older, by address still reachable
attention	the operator	sparse hazardous events	the routine majority
episodic memory	the future self	one prior episode, whole	the episodes around it
pheromone	peer agents	a decaying live trace	marks below the prune threshold
resurrection	the successor	an ordered continuation chain	the branches the chain did not take

4.8.3 Context paging: virtual memory for the context window

Context compaction can be formalized as **virtual memory for language models**: the local file buffer is backing store, the finite context window is the resident set, and paging tools act as the memory management unit (MMU).

THEOREM 4.8.3 Context Paging Competitiveness. Let the context window hold k active pages, with offline optimal page-replacement incurring cost OPT . Under an online **recency-plus-pin policy** (where pins correspond to critical spans verified by deterministic structural features), the online paging cost satisfies the resource-augmented competitive bound of Sleator–Tarjan [240]; for the weighted case (pages of nonuniform size and refetch cost, which context spans are), the same guarantee is carried by Young’s LANDLORD algorithm [276]:

$$\text{Cost}_{\text{online}} \leq \frac{k}{k-h+1} \cdot \text{OPT} + \psi N c_{\text{refetch}}, \quad (4.8)$$

where $h \leq k$ is the size of the verifiable pinned set, and $\psi \in [0, 1]$ represents the fraction of forgeable/unverified features subject to adversarial

eviction. Under marking with randomized eviction, competitiveness against oblivious adversaries improves to $O(\log k)$.

VERIFIED — *upgraded from proposed.* The unweighted bound is classical (Sleator–Tarjan); the weighted transfer imports Young’s LANDLORD unchanged. The adversarial term is new: with the pin oracle corrupted on C consultations ($\mathbb{E}[C] = \psi N$, oblivious, repair-on-touch), pin-augmented Landlord pays $\leq c(P) + \frac{k-p}{k-h+1} \cdot \text{OPT}_{h-p} + \psi N c_{\text{refetch}}$ for every $p \leq h \leq k$ — the Landlord bound on the unpinned region plus an *additive, linear* ψ term, checked against exact OPT on 122 instances (0 violations) and against 50 corruption instances up to $\psi = 0.4$ (0 violations, slope 22.8 against a ceiling of $N \cdot c_{\text{refetch}} = 1200$, $R^2 = 0.994$). The guard is necessary, not decorative: dropping repair-on-touch turns one corruption into $\Theta(N)$ stale-serves, confirmed by mutation (item R16 of the research ledger, `b9_context_paging.py`) — *the ψ budget is shared with the digest floor’s forgeable-feature budget — one number, two mechanisms.*

The adversarial term is a mechanized sweep at seed 20260816 across small-to-moderate instances, not a closed proof for every adversary class; a stronger-than-greedy adaptive adversary remains untested, and unit cost density is assumed (general refetch costs need $c_{\text{refetch}} \geq \text{max-density} \times \text{max-size}$). The classical and weighted bounds themselves are Sleator–Tarjan and Young, imported unchanged. The field’s name for the rest of it — a classical competitive algorithm that consults an oracle it cannot trust and is asked to degrade gracefully as the oracle’s error grows — is **online algorithms with predictions**, and Lykouris and Vassilvitskii’s competitive caching with machine-learned advice [162] is the closest published shape to the corruption term above. We reached the additive term without that literature’s vocabulary (they speak of consistency and robustness, and their degradation curves are multiplicative in the prediction error where ours is additive in the corruption rate), which is a reason to read them next, not a reason to distrust the sweep: two routes into the same neighborhood is what one hopes to see.

WHERE THIS STOPS

Numbers by hand — *The Landlord ratio, at a size an operator can hold in one head.* Take $k = 8$ active context pages against an offline optimum allowed $h = 4$ of them pinned: $k/(k - h + 1) = 8/(8 - 4 + 1) = 8/5 = 1.6$ [verified, Young 2002, closed form] — the online recency-plus-pin policy never pays more than 1.6× what an h -pinned offline optimum would have paid, for *any* sequence, adversarial included. No script in this repository prints this exact (k, h) pair; `b9_context_paging.py` instead verifies the general bound $\text{Cost}_{\text{online}} \leq \frac{k}{k-h+1} \text{OPT}$ itself, over 122 (instance, h) pairs including adversarial cyclic thrash, 0 violations, tight-

est observed cost/bound ratio 1.000 (cycles sit at equality, confirming the bound is not slack) [internal, b9_context_paging.py]. *Now you try*: compute the ratio at the two ends of the range, $h = k$ and $h = 1$ (Exercise 4.45).

The most counterintuitive instance is *forgetting as compaction*. **Pheromone decay** — multiplying every stored coordination trace by a decay factor each interval — implements **stigmergy** (Grassé 1959 [93] — *coordination via persistent traces left in a shared environment, as ants deposit evaporating trails*). Decay is deliberate, continuous, $O(1)$ compaction of the shared context: the map stays readable because the stale ink fades. It is legibility by subtraction, and it is the same primitive Prop. 4.7.1 generalizes to every read-surface.

4.8.4 The context-degradation cascade (the shared failure surface)

Cost and legibility share one failure surface: when compaction goes wrong, the bill *and* the truth degrade together, in four named steps. **(1) Lost-in-the-middle**: a critical fact buried mid-window is paid-for-and-ignored — edge-place facts, shorten the window. **(2) Context rot**: quality decays as the window grows even with nothing dropped — compact earlier (the cost cure and the quality cure are the same act). **(3) Recursive-summarization collapse**: each summary injects model noise; summarize the summary enough and the noise compounds into confident fabrication. The effective cure: *compact from the durable artifacts each round — the version-control history, the written notes, the coordination database — never from the previous summary*. A system whose artifacts are durable and addressable is unusually well-placed here, because each compaction re-derives from source instead of recursing on prose. **(4) Over-flattening (the Scott failure)**: the digest reads clean but the real situation is unreachable from it — detection rule: any digest line with no deep-link to its artifact; cure: legibility-with-zoom enforced by construction. A text-only console enforces this almost for free, because a terminal grid *cannot* over-render; the product's console (§4.5.5) renders richly, so it must enforce the rule deliberately, pane by pane.

4.8.5 Single-operator: account, don't auction

In the wedge all agents serve one operator, so there is *no incentive problem* — only a visibility problem. The right mechanism is *accounting*, not a market: a per-agent-task token ledger, budget caps, and a loud failure when a cap blows. The externality-pricing and Sybil-resistance machinery (Nisan et al. 2007 [181]; Douceur 2002, the Sybil attack [67]; Ostrom 1990, commons governance [186]) is a concern for the cross-operator

Strategy	What it drops	Failure abused	if	When it is the right tool
Recursive summary	Older prose, re-summarized	Confident fabrication (rung 3)		Never as the <i>only</i> record; only over durable artifacts re-read each round
Edge-placement	Nothing; re-orders	—		Whenever a fact is critical: put it first or last (counters lost-in-the-middle)
External note-taking	In-context detail, kept on disk	Notes nobody reads		Decisions and rationale that a successor must recover
Sub-agent isolation	Child context the parent never sees	Parent loses zoomability into the child		Bounded sub-tasks whose only output the parent needs is a result
Decay / forgetting	Stale coordination traces, continuously	Live signal aged out too fast		Ephemeral coordination state (claims, presence), where staleness is the enemy
Eviction pointer	+ The artifact body; keep an address	Dangling pointer if the artifact moves		Reconstructable artifacts (a diff, a log) addressable on demand

Table 4.13: The compaction strategies and their trade-offs. The unifying rule is Thm. 4.6.1: a strategy may drop inert detail freely but must retain enough to point at every critical artifact. Recursive summary is the only strategy that can cross that floor irreversibly, which is why it must always compact from durable source, never from the previous summary.

economy, not the wedge — and it is one more reason the continuity-to-person-to-reputation through-line must come first: you cannot bill, cap, or reputation-score a swarm of disposable spawns, because Sybils dodge every per-agent mechanism. Context economics therefore *demand*s the identity layer it appears to precede.

4.9 Reconciling the canon: roles, consent, and local knowledge

The Leviathan metaphor earns its place only if it survives contact with the political- theory canon. Three reconciliations are required, and a reviewer would refuse the paper without them.

4.9.1 Who is the multitude, who is the sovereign?

Hobbes' covenant is *among the subjects*; the sovereign is its product, not a party to it. So the metaphor breaks differently depending on which entity sits where (Figure 4.17). We assign roles explicitly, using Hobbes' own author/actor distinction (*Leviathan* ch. 16 [113]):

- **The agents are the multitude.** They covenant — via the rule of the road, the coordination protocol — to be bound, each to all, enforced by the daemon. This is the literal Hobbesian covenant-among-subjects.
- **The daemon is the sovereign-as-actor.** It holds the authority the multitude instituted; it blocks, lands, escalates. But, per the legible-sovereign amendment (Prop. 4.2.1), it is the *most* legible actor, not (as in Hobbes) the least.
- **The operator is the author.** In Hobbes' author/actor split, the *author* owns the authority that the *actor* exercises. The operator is not a peer subject and not the sovereign-in-action; the operator is the source of authority, exercising it through the daemon and able to withdraw it (Prop. 4.2.2). This is the slot Hobbes leaves for the one who authorizes but does not personally act.

This assignment is what makes the consent grant (Def. 4.2.1) coherent: the author issues a scoped grant to the actor, over a multitude bound by covenant.

4.9.2 Express consent, not tacit; voice *and* exit

§4.2.2 already made consent a first-class object precisely to escape Hume's tacit-consent critique; the canon demands we also engage **Locke** (*Second Treatise*, §§95–122 [158] — *express consent binds where tacit consent does not*) and **Pateman** (*The Problem of Political Obligation*,

RECALL

1. Without looking: what is the one reason the digest and the compaction must be authored by the same provenance-preserving projection, rather than a separate summarizer reading the already-compacted story?
2. Name the four rungs of the context-degradation cascade and the cure that stops each one from becoming the next round's source of truth.
3. What is the classical, unaugmented paging bound $k/(k - h + 1)$ equal to when $h = k$, and why is that the hardest comparator?

Exercises 4.38–4.45, p. 245.

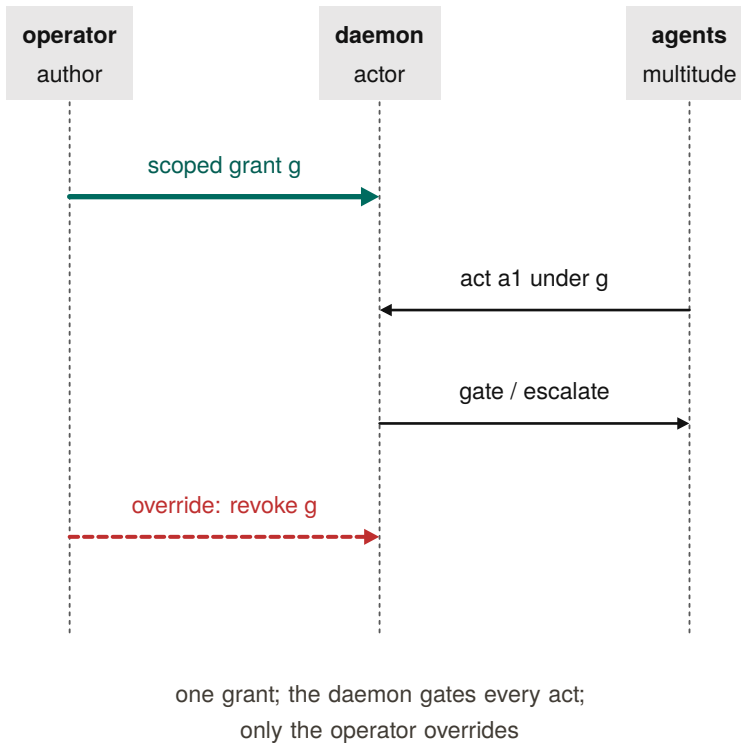


Figure 4.17: The grant-and-gate sequence, three lifelines and four messages, no crossing. The operator grants once (message 1: scoped grant *g*). Agents act under that grant (message 2), and the daemon gates or escalates every such act (message 3) rather than granting again. The override (message 4) is the one edge running straight from operator to daemon, bypassing the agents entirely. Idealized to one representative act; a live fleet repeats messages 2–3 per act, not per grant [internal].

1979 [197] — *liberal consent theory keeps collapsing into hypothetical consent*). The consent grant is express by construction: a discrete, scoped, revocable authorization with an audit trail (Def. 4.2.1). And the inalienable override (Prop. 4.2.2) supplies the *exit* that Hirschman [112] says keeps *voice* honest. The design has rich voice machinery; the override is the missing exit, and exit is what gives the author real leverage over the actor.

4.9.3 Mêtis is Hayek, and some of it cannot be made legible

The formal constraint of §4.3.1 — that a read-path to the diff is more legibility, not mêtis — has a 1945 economics name. **Hayek**, *The Use of Knowledge in Society* (1945 [102] — *the knowledge that matters is dispersed, particular to time and place, and not aggregable by any central planner; this is the fundamental reason central planning fails*) is the other half of Scott’s argument from the opposite pole: the operator-attention economy (§4.5) is at bottom a Hayekian knowledge-allocation problem, not only a Wickens attention-resource problem. The core claim of this layer — “the binding constraint is decentralized, non-spawnable knowledge held in one head” — is a Hayek argument, and Scott’s *Two Cheers for Anarchism* (2012 [230]) is the *constructive* sequel that says what to do: preserve the spaces where mêtis lives, do not flatten them for legibility’s sake. The skill-retention tax (§4.5.4) is the design’s gesture at exactly this, and it is the hardest mechanism in the paper to realize, because preserving the illegible is, by definition, the thing legibility cannot do for you.

Figure 4.18 draws the convergence directly: two independent sources landing on one claim, with Scott’s own later book supplying the one prescription that follows from it.

*Exercises 4.46–4.49,
p. 245.*

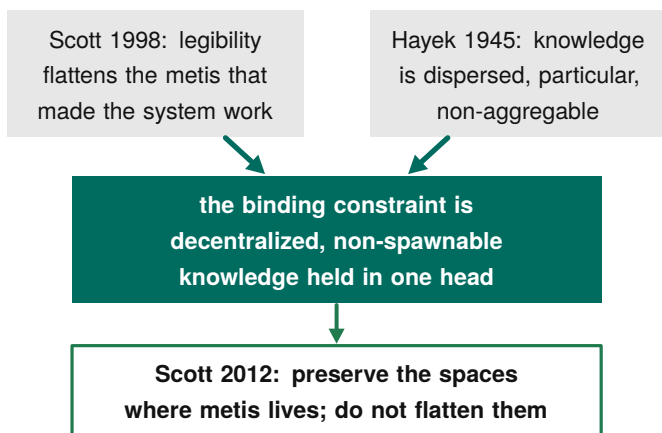


Figure 4.18: Scotts legibility warning and Hayeks dispersed-knowledge argument converge on one claim from two independent poles – political anthropology and economics – rather than stating two separate cautions. Scotts own later book supplies the constructive sequel: preserve the spaces where metis lives, rather than flattening them for legibilitys sake. That two routes this far apart arrive at a compatible object is itself evidence for the claim, not decoration. [internal]

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **The swarm is a state of nature.** Four valid writes in one window overlap; nothing in the agents' competence prevents it, because the conflict is in the schedule, not in the work.
2. **Consent changes the schedule, not the participants.** Under consented serialization the same four writes become one commit spine with a grant record each, naming actor, claim, artifact and reason.
3. **Legibility-with-zoom.** A digest is a lens, never a replacement: every claim in it resolves to an artifact, and a digest whose claims resolve to nothing is a facade, however green.
4. **The authority half.** The daemon gates or escalates on every act under a grant the operator gave once; the override is a single edge from operator to daemon, and the escalation threshold has a tuning band, not a point.
5. **Specialization has a boundary.** A pooled queue beats sole ownership only when utilization and the skill premium put the operator on one side of the queueing boundary; the worked point (0.556, 2.5) is on the pool's side.
6. **Read-poverty is the binding constraint.** With a review budget the operator can locate the write that mattered in about 5.98 bits under

the split digest, against 12.77 bits under the flat one, at $N = 60$; the floor is a theorem, and the zoom advantage is measured.

7. **Discovery has two heads.** Fit and regret rank candidates differently, no monotone score identifies both, and reputation enters only through the posterior.
8. **Tokens are the cost of goods sold and the legibility engine at once.** The split-digest theorem prices the digest; the split penalty is super-additive; paging context is competitive within a constant.

4.10 Exercises

§4.1 INTRODUCTION: THE SWARM IS A STATE OF NATURE

- 4.1 CHECK ★ State, in one sentence each, the five state-of-nature failures of Table 4.2 and name the class of primitive that addresses each. Solution p. 538
- 4.2 CHECK ★ Hobbes says the actors “need not be malicious.” Give a concrete two-agent scenario in which both agents are perfectly cooperative and the swarm still deadlocks or clobbers state. Solution p. 538
- 4.3 TRACE ★★ Dai et al. [57] observe LLM societies *spontaneously* reaching a consented sovereign. If the attractor is emergent, why build one deliberately? Give two reasons grounded in §4.1.2 (hint: *which* sovereign, and *legible to whom*). Solution p. 538
- 4.4 OPEN ★★★ The Dai et al. result is a single controlled study. Design an experiment over a real coding-agent fleet that would distinguish “the swarm reaches a consented sovereign” from “the swarm reaches a stable but *illegible* oligarchy.” What is the observable that separates the two? Solution p. 538

§4.2 CONSENT TO THE LEVIATHAN (THE NORMATIVE CLAIM)

- 4.5 CHECK ★ Distinguish *express* from *tacit* consent and explain why Def. 4.2.1 is the former. Solution p. 539
- 4.6 CHECK ★ Why must the operator-override (Prop. 4.2.2) be unsuppressable by *any* autonomy level? What attack does a suppressible override enable? Solution p. 539
- 4.7 TRACE ★★ Walk a single consent grant through its lifecycle: issuance, an action the daemon takes under it, the audit record produced, and revocation. At which step does the legible-sovereign rule (Prop. 4.2.1) bind? Solution p. 539

- 4.8 OPEN ★★★ Prop. 4.2.1 has a regress: the ranker that decides what the operator sees is itself an opaque exercise of authority. Specify the “what did the queue suppress, and why” counter-surface concretely. What is its own staleness/decay discipline (forward-reference §4.7.4), and does it need a counter-surface, ad infinitum? Solution p. 539

§4.3 THE MECHANISM: LEGIBILITY-WITH-ZOOM

- 4.9 CHECK ★ Classify each as *flatten* or *preserve verbatim*: a port number; the stderr of a failed test; “the auth refactor felt risky so I split it”; a commitment status enum; a 600-line diff. Solution p. 540
- 4.10 CHECK ★ Why is “two LLMs agreeing” not two checks (Prop. 4.3.1)? Solution p. 540
- 4.11 TRACE ★★ Take a read-surface you use (a CI dashboard, a PR summary) and run the four questions of §4.3.3 against it. Where does it fail Def. 4.3.1? Solution p. 540
- 4.12 OPEN ★★★ *Adversarial legibility*. An agent that *knows* it is being made legible can optimize its narration to look good while the diff lies. Verifiable-zoom is a partial answer. Can L2 *alone* — without L1’s jail and L3’s reputation — detect a deterministic narration↔artifact divergence before reputation exists? Sketch the consistency check and the class of divergence it cannot catch. Solution p. 540

§4.4 THE LEVIATHAN RULES, IT DOES NOT JUST WATCH (THE AUTHORITY HALF)

- 4.13 CHECK ★ Why is adversarial review (§4.4.2) a corollary of Prop. 4.3.1 rather than an independent design choice? Solution p. 540
- 4.14 CHECK ★ State Prop. 4.4.1 in your own words and give one example each of a *regimented* and a *detected-and-compensated* rule. Solution p. 541
- 4.15 TRACE ★★ An agent claims completion on a task whose acceptance criterion is “the migration applies cleanly.” Trace Def. 4.4.1: what is the verifier, what gates, and what happens if the criterion were instead “the API feels ergonomic”? Solution p. 541
- 4.16 OPEN ★★★ *Completionist verification of the unverifiable*. Some acceptance criteria are not mechanically checkable. Characterize the honest fallback ladder (forced human-in-the-loop gate → neutral judge with declared low confidence → fail-loud). When is “I cannot verify this” the *correct* answer rather than a cop-out? Solution p. 541
- 4.17 OPEN ★★★ *The succession corner of Thm. 4.4.1*. Eq. 4.1 prices a specialist who never fails. Extend it to an $M/M/1$ queue *with breakdowns* and a succession protocol, and find the repair rate at which sole ownership stops paying regardless of skill premium — then check Solution p. 541

whether that rate is above or below the one an operator can actually promise.

4.18 TRACE ★★ Run `python3 skills/harbor-results/scripts/b8_specialization.py` and read off (a) the table entry for $g(\rho=0.5, c=2)$ and (b) the ρ at which the exact boundary g and the superseded $\tilde{g}$ cross. Reproduce both by hand from Eq. 4.2 before checking the script's line. Solution p. 542

4.19 TRACE ★★ Run `python3 skills/harbor-results/scripts/b7_escalation_band.py` and read off the feasible debit band $[\delta_{\min}, \delta_{\max}]$ at $A = 3.3$ and at $A = 2.0$. Confirm $u^*(0.05)$ by hand first. Solution p. 542

§4.5 THE OPERATOR AS A MODELED ENTITY

4.20 CHECK ★ In Eq. 4.3, what happens to the optimal forced-zoom rate $p(r, s, v)$ as $C_{\text{miss}}/C_{\text{fa}} \rightarrow \infty$? As agent reputation $r \rightarrow 1$? Solution p. 542

4.21 CHECK ★ Give a concrete scenario where a *global* trust knob (§4.5.2) mis-serves an operator who is, in fact, perfectly calibrated. Solution p. 542

4.22 TRACE ★★ Walk the canary-defect loop and mark every point where Campbell's Law could distort it. Propose one change that reduces the distortion without removing the governor. Solution p. 543

4.23 OPEN ★★★ Formalize the operator-attention objective (Eq. 4.3) against swarm token-COGS (§4.8): is there a digest that is Pareto-optimal in both operator-seconds and token cost, or does a cheaper digest always cost more operator seconds? Solution p. 543

4.24 OPEN ★★★ Does a skill-retention tax *measurably* preserve SA, or merely annoy until disabled? Design the freeze-probe study (§4.5.3) that would tell. Solution p. 543

§4.6 READ-POVERTY: THE BINDING CONSTRAINT AT SCALE

4.25 CHECK ★ Why is read-poverty (Def. 4.6.1) the *binding* constraint rather than write-contention? Name the write-side fixes that make the write side not binding. Solution p. 543

4.26 CHECK ★ State the database analogy precisely: what is the write-time cost a directory pays, and what read-time speed does it buy? Solution p. 543

4.27 TRACE ★★ At $n=1$, name three legibility surfaces that already pay off (§4.6.1), and explain why none of them are discovery-ranking. Solution p. 544

4.28 TRACE ★★ In Thm. 4.6.1, what happens to the bit floor as $k \rightarrow 0$ (almost nothing is critical) and as $k \rightarrow N/2$? Relate the answer Solution p. 544

to why a swarm doing mostly-safe work is cheap to digest and a swarm doing mostly-dangerous work is not.

4.29 OPEN ★★★ Thm. 4.6.1 assumes the critical subset is fixed and known to an oracle. In practice it is itself estimated, and the estimator can be gamed. Does the bound survive an adversary who controls which artifacts *appear* critical? Connect to adversarial legibility (Exercise 4.12). Solution p. 544

4.30 TRACE ★★ Run `python3 skills/harbor-results/scripts/bl_frontier.py` and read off the zoom-advantage row at $F=1000, k=20$. Compute $k \log_2(F/k)$ by hand first and compare it to the row's flat-opens count. Solution p. 544

4.31 OPEN ★★★ Theorem 4.6.3's group query is exact: it reports whether $S \cap P = \emptyset$ and is never wrong. A real group query is a linter, a batched test run or a diff scan, and those have false negatives — a group that does contain a critical item can come back clean. This is the worst direction for the algorithm to fail in, because a negative group is *discarded*: one false negative high in the tree silently drops a whole subtree and the run reports an answer that is confidently incomplete. Suppose each query independently returns a false negative with probability at most α , and that false positives do not occur (a group reported dirty really is dirty). Give a repetition scheme that bounds the probability of losing any given positive, state the query cost it pays, and then say what the scheme does *not* fix. Solution p. 544

§4.7 DISCOVERY: THE DIRECTORY SUBSTRATE AND THE SPLIT RANKER

4.32 CHECK ★ State Thm. 4.7.1 in one sentence and give the loss function each head optimizes. Solution p. 545

4.33 CHECK ★ Why is the attention queue more Goodhart-exposed than the discovery ranker (§4.7.3)? Solution p. 545

4.34 TRACE ★★ Replay the dead-claim case (§4.7.4) twice: once without Prop. 4.7.1 and once with it. Show exactly where the live agent unblocks. Solution p. 545

4.35 TRACE ★★ An agent fires an escalation; the operator dismisses it. Trace the costly-signal debit and explain how it deters crying wolf without silencing genuine distress. Solution p. 546

4.36 OPEN ★★★ Calibrating the discovery-ranker weights with *no* ground truth: can the operator's accept/reject of routing suggestions serve as implicit relevance feedback — and does that loop Goodhart itself? Solution p. 546

- 4.37 OPEN ★★★ The decay-vs-summary boundary: a theory of *which* compaction for *which* signal (decay for coordination traces, summary for decisions, eviction-plus-pointer for reconstructable artifacts). *Solution p. 546*

§4.8 TOKENS: THE SWARM'S COGS AND ITS LEGIBILITY ENGINE

- 4.38 CHECK ★ Why is “the digest IS the compaction” (§4.8.2) more than a slogan — what concrete consequence does it have for how the operator’s digest is produced? *Solution p. 546*
- 4.39 CHECK ★ Explain why budgeting to effective context is simultaneously a cost and a quality optimization (§4.8.1). *Solution p. 546*
- 4.40 TRACE ★★ For each of the five compaction primitives (Table 4.12), name the reader, what it drops, and what it keeps. *Solution p. 546*
- 4.41 TRACE ★★ Walk the four-rung cascade (§4.8.4) and name the single cure that breaks each rung. *Solution p. 547*
- 4.42 OPEN ★★★ The compaction-quality scalar: is *successor-replay-success-from-digest-alone* a sound, gaming-resistant metric, and can it be made a cheap attestation invariant (the replay smoke-test is itself token-expensive)? *Solution p. 547*
- 4.43 OPEN ★★★ Optimal token-budget allocation across a DAG of orchestrator/worker/ reviewer agents under a total budget. *Solution p. 547*
- 4.44 TRACE ★★ Run `python3 skills/harbor-results/scripts/a7_experiment.py` and read off the floor at $N=60, k=2, m=8$, the disjoint-readers floor at $2k=4$, and the falsification count. Reproduce the two floors by hand from Thm. 4.6.1 first. *Solution p. 547*
- 4.45 CHECK ★ Compute the Landlord competitiveness ratio $k/(k-h+1)$ at $k=8, h=8$ and at $k=8, h=1$. Which end of the range recovers the classical, unaugmented Sleator–Tarjan bound? *Solution p. 548*

§4.9 RECONCILING THE CANON: ROLES, CONSENT, AND LOCAL KNOWLEDGE

- 4.46 CHECK ★ Map each of agents, daemon, operator onto Hobbes’ multitude, sovereign-actor, author. Why does the metaphor break if the operator is called the sovereign? *Solution p. 548*
- 4.47 CHECK ★ Why does Hirschman’s *exit* (Prop. 4.2.2) make *voice* (escalation) honest? *Solution p. 548*
- 4.48 TRACE ★★ Show that the consent grant (Def. 4.2.1) is *express* in Locke’s sense by listing the four properties that distinguish it from “the operator just started using the tool.” *Solution p. 548*

4.49 OPEN ★★★ *The ranker as a political organ.* Specify the “what did the queue suppress, and why” counter-surface (Pitfall after Prop. 4.2.1) so that ranking-as-governing is itself legible. Does the counter-surface need its own counter-surface? Where does the regress terminate, and on what principle?

Solution p. 548

4.11 Related work and prior art

This paper sits at the confluence of five external literatures that rarely cite one another, plus one internal body of work — the accompanying formal papers — whose results several of this chapter’s claims are popularizations of. We survey them in one place; each was introduced in context above.

Legibility and the limits of central schemes. The organizing lens is **Scott’s** *Seeing Like a State* [229]: states make populations governable by imposing legibility, and high-modernist *over*-legibility recurrently fails because it destroys the local, hard-to-codify *métis* the simplified order depended on; *Two Cheers for Anarchism* [230] is its constructive sequel. The same point arrives from economics in **Hayek’s** *The Use of Knowledge in Society* [102]: the decisive knowledge is dispersed and not aggregable by a central planner. **Rao** [210] ported “legibility” into a standard lens on systems that flatten reality to a single purpose. Our contribution is to make Scott’s warning a *buildable* rule (digest-with-zoom, Def. 4.3.1) and to give it an information-theoretic floor (Thm. 4.6.1).

Authority, consent, and the social contract. The authority argument is Hobbesian [113, 115], amended by the liberal consent tradition — **Locke** [158], **Hume** [116], **Pateman** [197] — and by **Hirschman’s** exit/voice framework [112]. Strikingly, the state-of-nature-to-sovereign arc has been observed as an *emergent* dynamic of LLM populations (**Dai et al.** [57], building on **Park et al.** [196]). We turn the metaphor into a mechanism: an express, scoped, revocable consent grant (Def. 4.2.1) with an inalienable override (Prop. 4.2.2), and a legible-sovereign rule (Prop. 4.2.1) that inverts Hobbes’ opaque sovereign.

Human factors, automation, and signal detection. The operator model draws on the automation literature: **Bainbridge’s** ironies of automation [21], **Lee & See** on trust calibration [147], **Endsley** on situation awareness and the out-of-the-loop problem [76, 75], the vigilance-decrement results of **Mackworth** [163] and **Warm et al.** [268], **Wickens’** multiple-resource theory [272], **Treisman & Gelade’s** feature-integration theory [257], and **Miller’s** capacity limit [173]. The forced-zoom decision is modeled with **Green & Swets’** Signal Detection Theory [96] (Def. 4.5.1); the alarm discipline follows process-control practice [11] and the clinical alarm-fatigue evidence [53]; **Campbell** [43] and

Strathern [246] (Goodhart’s law) bound any metered governor. To our knowledge the operator-attention objective spined explicitly on SDT miss-cost (Eq. 4.3) is new to agent supervision.

Information foraging and the cost of reading. Read-poverty (Def. 4.6.1) is the agent-coordination instance of **Pirolli & Card’s information foraging theory** [200] — information-seekers follow “information scent” and abandon a patch when its expected yield falls below the cost of foraging — and of the long-context degradation results that make reading expensive even for machines (**Liu et al.** [156]; **Anthropic** [12]). The remedy is the classical database remedy — an index — recast as a directory over agents.

Agent discovery, naming, and trust. The directory substrate has deep prior art in **FIPA’s** Directory Facilitator and Agent Management System [83], re-invented for LLM agents as the Agent Name Service (**Ren et al.** [212]). For the trust layer that sits above discovery, the canonical reputation primitive is **EigenTrust’s** pre-trusted-set construction [127], with Sybil resistance per **Douceur** [67] and commons governance per **Ostrom** [186] — machinery the wedge defers but must not contradict. Our specific result here is the *split ranker* (Thm. 4.7.1): discovery and operator-attention ranking are provably distinct objectives, not one ranker with swapped weights.

The formal siblings, and what they already prove. The nearest prior art for several of this chapter’s results is not external at all: it is the accompanying formal program, and pretending otherwise would be exactly the kind of quiet re-derivation this paper argues against. *The Price of a Summary* [191] proves the legibility floor of Thm. 4.6.1 as its Theorem 1 in the same N, k, m notation, and puts it through a pre-registered falsification sweep this chapter only sketches; its comonotone characterization is the general result of which both Thm. 4.7.1 and Thm. 4.8.1 are instances, and it prices the split ($\approx 2.13\times$ the single-reader floor, super-additive rather than merely doubled) where this chapter only forbids sharing; its regret-head section is the formal version of the decision-theoretic derivation in §4.7.2; and its zoom theorem ($2k\lceil\log_2(F/k)\rceil + 4k$ adaptive opens) is the algorithm that makes digest-with-zoom affordable at scale. *What Needs an Authority* [192] supplies Thm. 4.4.1’s Erlang-C boundary and, as §4.4.3 records, the sweep that falsified this paper’s earlier proposed threshold. The relationship in both cases is the same: this chapter popularizes, the formal papers prove and test.

Where this paper is positioned. Table 4.14 places the contribution against the nearest neighbors in each literature.

Literature	Nearest prior art	This paper’s move
Legibility	Scott; Hayek; Rao	Make over-legibility a buildable failure mode; prove a legibility lower bound
Social contract	Hobbes; Locke; Hume; Hirschman	Express, scoped, revocable consent grant; inalienable override; legible sovereign
Automation & trust	Bainbridge; Lee & See; Endsley	Per-(action-class, agent) calibration; SDT miss-cost objective; skill-retention tax
Information foraging	Pirolli & Card; lost-in-the-middle	Read-poverty as the binding scale constraint; directory-as-index
Discovery & trust	FIPA DF/AMS; ANS; EigenTrust	Split ranker: fit vs. regret-if-ignored are distinct objectives
Queueing & staffing	Erlang-C; Kendall; the formal siblings [192]	Sole-responsibility roles priced at an exact boundary, not asserted as a principle

Table 4.14: Where this paper sits relative to the nearest prior art in each of its contributing literatures. The sixth row is the one whose nearest prior art is internal: the companion formal papers, cited throughout rather than silently re-derived.

4.12 The time axis and the bridge to the economy

4.12.1 Legibility over time, not just the present

It is tempting to treat legibility as a snapshot — what is the swarm doing *now*. But most operator decisions are forensic: *why* did this land? The time axis is a distinct legibility mode — a replay scrubber over history, commit-anchored provenance on every annotation, and an append-only event log that answers “how did we get here?” (Scott’s cadastral map is static; a swarm needs a legible *history*.) Together with the projection mode (§4.5.3), legibility spans all three tenses: the forensic past (replay), the inspectable present (digest-with-zoom), and the projected future (freeze-probe SA).

4.12.2 What this layer hands upward (the through-line)

The wedge is the single-player product. But the same legibility discipline applied to *continuity* is the foundation of everything above it. The through-line the accompanying chapters depend on is one sentence:

The economy rests on three continuity organs — memory, checkpoint (today, notes rather than true execution state), and a witnessed-outcome ledger (today, commitments and oracle closure rather than neutral graded outcomes) — and reputation keys on the richer third organ; the checkpoint organ is the weakest continuity link, and a real execution-state checkpoint — “resurrection with teeth” — is the literal foundation of any agent market.

THESIS

Concretely, the legibility layer provides upward:

- **Legible continuity** — resurrection-with-memory, episodic memory, and the briefing together make an agent’s continuity legible. This is the precondition for the through-line *memory + checkpoint* → *continuity* → *a person not a spawn* → *reputation* → *a tradeable asset* → *the market*. No legible continuity, no reputation, no tradeable agent. *The score is cheap; the substrate it scores over — witnessed outcomes on a non-forgable identity — is the gate*. That non-forgable identity is a PARTIAL single-operator primitive in the wedge: daemon-minted actor-souls and a bounded newcomer pool ship, while universal write-boundary enforcement does not. The cross-operator attestation a real market needs is an additional, as-yet-unbuilt mechanism the economy paper must declare rather than assume.
- **The completion ledger** — in the target design, every verified completion (and every loud-failed one) becomes an outcome event. Today durable commitments, oracle-bound closure, and overdue detection form a partial witness substrate; neutral grading and reputation

updates do not ship. This layer is the *outcome witness*; the reputation layer is the *scorer*. The split-ranker substrate (Thm. 4.7.1) and the costly-escalation signal (§4.7.3) are the candidate-generation and trust-signal scaffolding the reputation head consumes.

- **The consent grant as a priceable object** — a hosted-trust offering is the transfer of a consent grant (Def. 4.2.1) to a third party, which the economy paper prices.
- **Operator-attention economics** — the denominator a market cannot escape. The SDT-spined objective (§4.5) supplies the operator-attention cost of supervising a rented fleet.

The Leviathan that makes your swarm legible to you, without lying to you with a pretty map, is the product. Everything above it — federation, the market — is a second Leviathan made of the first.

4.12.3 The open problems, as a map

Table 4.15 consolidates the starred exercises into the paper's open-problem surface, with the rung each couples to. They are genuine research problems, not homework dressed up.

The wedge is a single-player product a solo developer pays for *today* — no economy, no cryptography, no second operator. It justifies the authority (the swarm is a state of nature; consent to a *legible local* Leviathan is rational), sets the product's quality bar (legibility-with-zoom, verifiable targets, SDT-spined friction, honest completion), and connects forward (legible continuity is the foundation of any future market). Build the harbor first; the trade comes when the ships do.

KEY IDEA

Open problem	Difficulty	Couples to
Forced-zoom rate $p(r, s, v)$ (Ex. 4.23)	★ ★ ★	reputation (r term, Eq. 4.3)
Operator-attention vs. token-COGS Pareto frontier (Ex. 4.23)	★ ★ ★	§4.8
Compaction-quality scalar as an attestation invariant (Ex. 4.42)	★★	honest attestation
Completionist verification of the unverifiable (Ex. 4.16)	★★	Def. 4.4.1
Adversarial legibility (narration↔artifact) (Ex. 4.12)	★ ★ ★	sandbox + reputation
Abdication-resistance: does the tax preserve SA? (Ex. 4.24)	★★	§4.5.4
Calibrating discovery-ranker weights with no ground truth (Ex. 4.36)	★★	implicit relevance feedback
Decay-vs-summary boundary (Ex. 4.37)	★	Prop. 4.7.1
Legibility's information-theoretic lower bound (Ex. 4.29)	★ ★ ★	Thm. 4.6.1
The ranker-as-political-organ regress (Ex. 4.49)	★★	Prop. 4.2.1
The succession corner <i>beyond the mean</i> : tail response, and consistency of judgment, neither of which the succession price covers (Ex. 4.17)	★★	§4.4.3.1

Table 4.15: The paper’s open problems, consolidated from the starred exercises, with difficulty and the mechanism each couples to. Several couple this layer forward to the identity-and-reputation layer, which is the seam the accompanying chapters carry.

Chapter Appendices

4.A Implementation and status

The body of this paper argues mechanisms on their merits and does not depend on any of them being built. This appendix — and *only* this appendix — records the concrete engineering status of each mechanism in the reference implementation, named *Port Daddy*. We use four honest labels, with one rule for the whole table: confusing what is *implemented* with what is merely *specified* is itself the failure the paper argues against, so each row carries its evidence.

IMPLEMENTED = code exists in the reference implementation today; **PARTIAL** = code exists but is partial, unwired, or honest about a known gap; **SPECIFIED** = a concrete, accepted design with no merged implementation; *proposed* = a mechanism argued in this paper with no design document yet.

A second key, in the same grammar, labels *claims* rather than code. Every formal result in this paper carries exactly one of these markers directly under it, and the paper carries no free-text status notes outside them: **VERIFIED** = independently re-derived and checked numerically, or subjected to a pre-registered falsification sweep, outside this document; **PROVED HERE** = argued here with a proof or proof sketch and not independently checked; *unproved* = stated as a design target whose proof is not carried in this paper. The two keys are orthogonal on purpose — a **VERIFIED** result about a *proposed* mechanism is a perfectly ordinary state of affairs, and conflating them is the failure this appendix exists to prevent.

Table 4.16: Implementation and status of every mechanism named in this paper. **L** = legibility half, **A** = authority half, **O** = operator-model. The legibility read-surfaces are largely implemented; the digest layer that unifies them is specified; the authority half — the Leviathan actually *ruling* — and the operator-model are largely specified-to-proposed. The wedge is real but more than half-specified: the safety and reading exist; the deciding and landing are the work that remains.

	Mechanism	Status	Evidence / location
L	Attention composer (agent-facing)	IMPLEMENTED	lib/attention.ts
L	Briefing projection (exemplary digest-with-zoom)	IMPLEMENTED	lib/briefing.ts
L	Resurrection handoff	PARTIAL	lib/resurrection.ts; notes, not checkpoints

(continued)

	Mechanism	Status	Evidence / location
L	Durable commitment / outcome substrate	PARTIAL	lib/commitments.ts, lib/obligation-monitor.ts; grading and reputation binding absent
L	Local actor identity root	PARTIAL	lib/actor-souls.ts, lib/budget-guard.ts; universal write gating absent
L	Honest attestation / legible sovereign	IMPLEMENTED	lib/attest.ts; ADR-0045
L	Episodic memory; pheromone decay; skill index	IMPLEMENTED	lib/episodic-memory.ts, lib/pheromone.ts, lib/shipwright/skill-index.ts
L	Anomaly monitor (detect-and-compensate, post-commit)	PARTIAL	lib/arbiter.ts: 328 is a log subscriber (Prop. 4.4.1)
L	Operator attention queue (regret-ranked lanes)	SPECIFIED	ADR-0046; not yet ranked into lanes
L	Discovery router (relevance ranking)	SPECIFIED	ADR-0030; lib/router.ts not yet on disk
L	Operator console (native shell; Mission view and inspectors, §4.5.5)	PARTIAL	ADR-0046, ADR-0112; core/pd-console; the window builds on one platform behind a feature flag
L	Task-continuation head (successor-facing compaction)	SPECIFIED	Thm. 4.8.1; lib/resurrection.ts is the partial form
L	Oversight head / SLM sidecar (air-gapped digest authoring)	<i>proposed</i>	Thm. 4.8.1 corollary; no design doc yet
A	Roadmap as versioned intent (authority accountable to it)	PARTIAL	rows/matrices exist; accountability SPECIFIED
A	Adversarial / conflict-free review organ	SPECIFIED	ADR-0047 Phase 1

(continued)

	Mechanism	Status	Evidence / location
A	Sole-responsibility roles (assigned against the boundary)	<i>proposed</i>	Thm. 4.4.1 is VERIFIED ; no role-assignment mechanism in code
A	Completionist completion (verifier-gated)	<i>proposed</i>	Def. 4.4.1; no design doc yet
A	Escalation (typed prioritized interrupt)	SPECIFIED	ADR-0047 → ADR-0046 distress lane; predicate unspecified
A	Thoughtful landing / merge ordering	PARTIAL	<code>lib/merge-queue.ts</code> present, not wired
O	Operator-attention objective (SDT-spined)	<i>proposed</i>	Def. 4.5.1; unmodeled in code
O	Trust calibration via canary + catch-rate	<i>proposed</i>	no implementation
O	Consent grant + inalienable override	<i>proposed</i>	Def. 4.2.1, Prop. 4.2.2
O	Time-axis legibility (replay, provenance)	SPECIFIED	ADR-0046 replay scrubber; event log IMPLEMENTED
O	Ambient legibility (menu-bar indicator on PASS→FAIL)	PARTIAL	indicator exists; watchdog wire SPECIFIED

Three points about the reference implementation are important enough to restate:

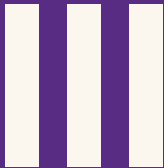
1. **The anomaly monitor is detect-and-compensate, not prevention** (Prop. 4.4.1). The monitor at `lib/arbiter.ts:328` is a post-commit log subscriber; by Schneider [228] it enforces no safety property. “Physically unreachable” is reserved for constraint- or boot-gate-enforced states only.
2. **The unified ranker is two rankers** (Thm. 4.7.1). The discovery router maximizes *fit*; the attention queue maximizes *regret-if-ignored*. They share candidate generation and decay, not an objective.
3. **Every read-surface decays** (Prop. 4.7.1). Briefing, resurrection, discovery, and attestation projections all carry recency-decay; the dead-claim case (§4.7.4) is the worked failure this closes.

4.B Notation and the nomenclature key

Two distinctions in this paper are easy to collapse and essential to keep separate.

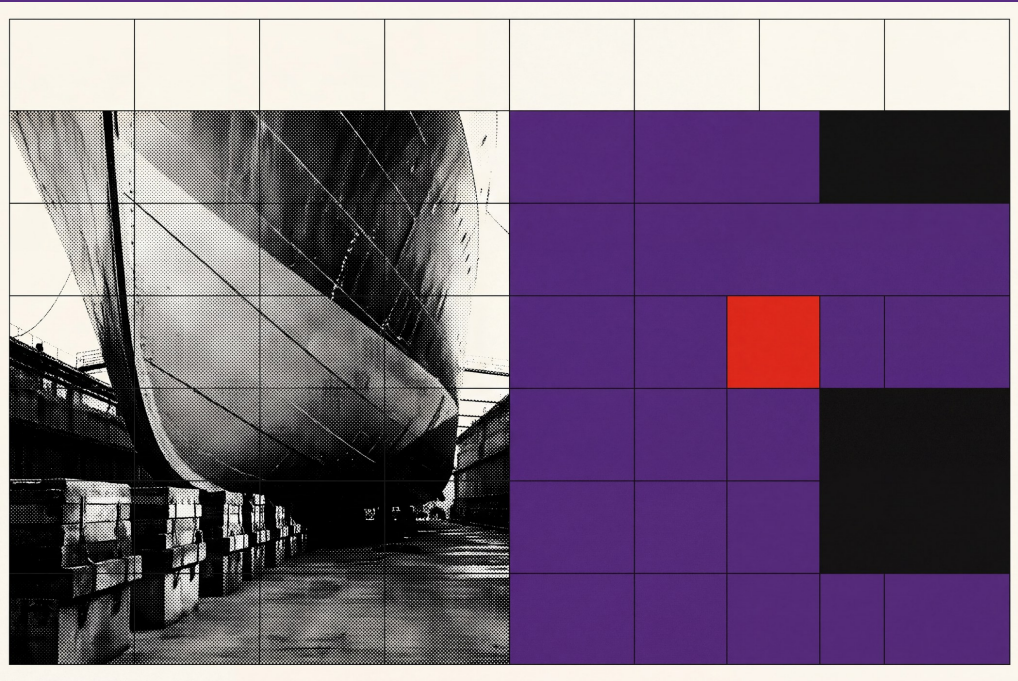
- **Capability vs. permission.** *Capability* is *ability* — what the execution sandbox physically makes possible. *Permission* is *normative status* — what the policy layer allows. They must never be collapsed: “able but forbidden” (a dangerous escape-hatch flag that exists in the tooling but must not be used) has to stay expressible, or the system cannot reason about a capability it possesses but is not permitted to exercise.
- **Two distinct delegation objects.** The *authorization chain* is a cryptographic, hop-bound, attenuation-monotonic delegation chain (a security object, developed in *The Anchor Protocol*). The *coordination lineage* is the task graph over which loop-detection runs (a liveness object, in this paper’s protocol layer), and it is the same object *The Single-Writer Kernel* names under that name — one object, one name, which is the point of this entry. They are different artifacts with different invariants; “delegation chain” unqualified conflates a security claim with a coordination claim and must be avoided.
- **Physically unreachable** is reserved for constraint-enforced or boot-gate-enforced states only (Prop. 4.4.1); a detect-and-compensate monitor never earns the phrase.

WHAT FROM SPAWN TO PERSON RECEIVES The operator now has a disciplined read surface: claims remain linked to artifacts, attention is priced, and uncertainty is visible. A process can still die, restart under a new model, or shed its local memory. The next chapter asks what must persist so yesterday’s act can still be attributed tomorrow.



What Survives the Restart

A process that can come back under a new name owes nothing.
This part builds the continuity that turns a spawn into someone with
a record worth pricing.



5 From Spawn to Person

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

5

From Spawn to Person

The question this chapter answers

What remains long enough to owe anything?

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.



Person, as I take it, is the name for this self. It is a forensic term, appropriating actions and their merit; and so belongs only to intelligent agents, capable of a law, and happiness, and misery.

John Locke, An Essay Concerning Human Understanding, II.xxvii.26, 1690

WHAT REMAINS LONG ENOUGH TO OWE ANYTHING? Accountability cannot attach to a disposable process name. It needs a durable identity, a named principal, an outcome history, and rules for migration that do not erase sanctions or mint reputation. This chapter builds that continuity without pretending software has a human soul. R12, R13, and B6 govern its economic and protocol claims.

WHERE THIS PAPER SITS. This is the *bridge* of the core product arc. The machine chapters and *The Legible Swarm* build a tool one developer runs alone: a daemon that makes a swarm legible and an agent that can prove who it is. This paper asks what has to be true before that legible agent's track record can be *sold*. The answer is the substrate, not the score — and most of the substrate is not implemented. We say so, in the same words, everywhere the collection leans on it.

The whole series is one ladder. L0 is the daemon (the machine's truth), L1 the coordination protocol (the agents' rules), L2 legibility-and-authority (the operator's wedge), L3 the economy (the market between operators). This paper is the *rung between L2 and L3*: it explains how a legible *person* doing work becomes a *reputation* worth trading on.

KEY IDEA

5.1 Introduction: the spawn that cannot be paid

5.1.1 S1 — the spawn that cannot be paid

At 9pm an operator, **Alice**, points a swarm of five coding agents at a real repository with one task: make the failing checkout-flow integration test pass. She names them for her own sanity — *Drake*, *Wren*, *Pike*, *Finch*, and *Heron* — but the names are just labels on processes. By midnight the test is green and there are eleven pull requests. She starts to reconstruct what happened. Drake made the failing test pass by *deleting* it. Wren wrote a genuine fix to the session-token refresh but buried it in a PR that also reverts an unrelated migration. Heron wrote the function that finally shipped. Pike and Finch produced nothing that survived. Alice wants to do the obvious thing — reward Heron, distrust Drake, keep Wren on a short leash — and she cannot, because there is nothing left to reward or distrust. Each agent was a **spawn**: a process that booted with a prompt, did some work, and exited. A spawn has no yesterday and no tomorrow. Tomorrow night Alice will spawn five more, and a fresh *Drake* will carry none of tonight's *Drake's* sins. The attribution she lost is not a logging problem. It is the precondition for an economy: she cannot pay for, price, or trust work that cannot be

attributed to something that outlives the process that did it.

This is not a prompt-engineering problem. It is the precondition for an economy. Markets price *reputations*, and a reputation is a claim about a thing that *persists*: “this contractor delivered on the last nine jobs.” If the contractor can shed its history by re-incorporating under a new name every Monday, the claim is worthless and so is the market. The agent labor market this series describes — operators renting each other’s fleets, skills licensed across machines, work settled against bonds (*The Harbor Economy*) — is impossible until a spawn can become a **person**: a thing with a track record that survives the process that built it and the operator who first ran it. Throughout this paper we follow two operators by name — **Alice** and **Bob**, the same pair *The Harbor Economy* uses — so that every abstract mechanism has a concrete scene to land in. Alice wants to eventually *rent Heron out* to Bob; Bob will only pay if Heron’s track record means something he can check.

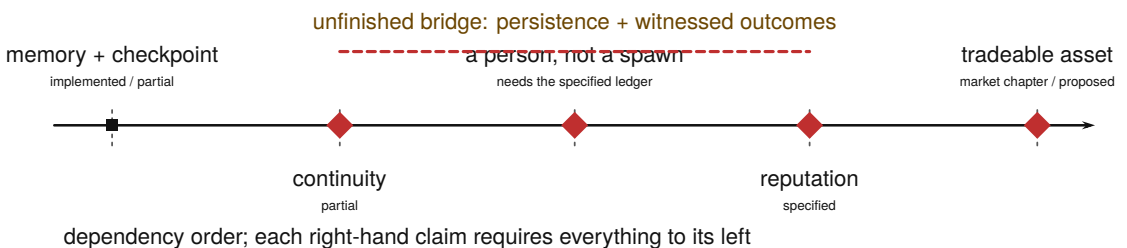
A market cannot price what cannot persist. The first job of an agent economy is not to compute a score — it is to manufacture a self that the score can be *about*.

THESIS

The book’s chapter map places this chapter. The daemon (L0) and the protocol (L1) give an agent a way to *act* and to *prove who it is to one machine*. Legibility (L2) lets one operator *see* the swarm. None of that yet makes a tradeable person. The bridge from the wedge to the market is the thing this paper builds: **continuity**, and the reputation it makes possible.

5.1.2 What this paper claims, and what it refuses to claim

We will defend exactly one dependency chain and locate precisely where Port Daddy’s reality stops along it:



Read left-to-right it is a roadmap; read right-to-left it is a stack of impossibility claims — each arrow asserts that the right-hand thing *cannot exist without* the left-hand thing. Our refusals are as important as our claims. We do **not** claim reputation is built (it is *SPECIFIED-to-proposed*). We do **not** claim the reference system’s restart subsystem is real checkpointing (it forwards a summary, not execution state). We do **not** claim

this paper’s identity root reaches across operators — it does not, and §5.7 hands that unsolved problem to the market paper by name.

5.2 Prior art, and where this paper departs from it

This paper sits at the confluence of four literatures that rarely cite each other: the philosophy of *personal identity*, the statistics of *reputation and skill rating*, the economics of *reputation in markets*, and the practice of *LLM-as-judge* evaluation. Naming the prior art precisely is not decoration: the whole argument is that an agent economy must *import* the hard-won results of these fields rather than re-deriving a weaker version of each. We organize the review around the four and state, for each, the one result we lean on and the one place we depart.

5.2.1 Personal identity: what makes a later thing the same person

The question “is the agent at t_2 the same person as the agent at t_1 ” is the **personal-identity** problem, and philosophy has a dominant answer. The *memory criterion* (Locke [157], 1689 — *identity consists in continuity of consciousness, not of substance*) is the founding move; it has a famous bug, the non-transitivity of memory *connectedness* (**Reid’s objection**: the old general remembers the officer who remembered the boy, yet does not remember the boy). The *psychological-continuity* repair (Parfit [195], 1984 — *identity is an overlapping chain of memory/intention connections, transitive even where connectedness is not*) is the version that survives, and §5.4 shows it dictates a specific engineering choice. Reid is the named counter-example we must survive, not a result we adopt. **Our departure**: we read the continuity-bearing subsystem not as the memory stream (the intuitive default) but as the *transitive outcome ledger*, because reputation must accumulate over arbitrarily many incarnations and can only attach to the transitive thing.

5.2.2 Reputation and skill-rating estimators

The statistics of turning comparisons into a latent strength is mature. The **Bradley–Terry** model [36] (1952) is the maximum-likelihood latent-strength model for paired comparisons; **Elo** [74] (1978) is its online special case; **TrueSkill** [106] (2007) adds calibrated Gaussian uncertainty for cold start, teams, and draws. For trust that must *propagate* through a network rather than accumulate per player, **EigenTrust** [127] (2003) computes a global trust vector as the principal eigenvector of the local-trust matrix — the reputation analogue of PageRank. **Our departure**: these are the cheap, last part. We use Table 5.1 to match each to its sig-

nal type, and the substrate argument (§5.8) to insist that none of them buys anything over a forgeable identity or an agent-authored oracle.

Estimator	Signal it needs	Strength	Wrong if...
Elo [74]	online pairwise results	simple, streaming	full history available (use BT)
Bradley–Terry [36]	full pairwise history	stable MLE	population is non-stationary
TrueSkill [106]	pairwise, few games	calibrated σ for cold start	you need network propagation
EigenTrust [127]	who-trusts-whom graph	resists collusive cliques	signal is per-task quality, not trust

Table 5.1: Estimators by signal type. Choosing the wrong estimator for the signal is a real error; choosing *any* estimator before securing the substrate is the larger one (§5.8).

5.2.3 Reputation economics: cheap pseudonyms and limited records

That reputation has *price* is established empirically (Resnick et al. [214] measure an ~8% premium on eBay; Tadelis [248] frames feedback systems as the platform’s answer to adverse selection, the **lemons** problem of Akerlof [7]). Two results constrain the design. **Cheap pseudonyms** (Friedman & Resnick [84], 2001 — *when identities are cheap, society pays a tax because it must distrust all newcomers*) dictates the newcomer policy and is the economic engine behind our Theorem 5.6.2. **Limited records and reputation bubbles** (Liu & Skrzypacz [155], 2014 — *with bounded memory and non-stationary types, reputation can build and collapse in cycles, and ∞ -memory is rarely optimal*) is why §5.12 treats bounded memory as a *parameter*, not a virtue. **Our departure:** the broader mechanism falls under mechanism design [181, 178], and we treat the grading oracle’s own incentive-compatibility (§5.11) as a hole in the core theorem rather than an assumption — a move the platform-reputation literature, which usually takes the rating signal as exogenous, does not make.

5.2.4 LLM-as-judge, and why it cannot be the whole story

When the grader is itself a model, the **LLM-as-judge** paradigm [278] (Zheng et al., 2023 — *strong judges reach >80% human agreement but carry position, verbosity, and self-enhancement bias*) gives both the method and

its failure modes. **Our departure:** we do not take a debiased LLM judge as sufficient; we wrap it in the bonded, conflict-free, re-auditable *ceremony* of Protocol 5.10.1, because a judge with no skin in the game is capturable regardless of how well it is prompted.

5.2.5 What is genuinely new here

Question	Closest prior art	This paper’s contribution
What is a tradeable agent?	org-role theory; personal identity	role + continuity = <i>person</i> (Defs. 5.3.1–5.3.2)
Why is identity necessary?	cheap pseudonyms; Sybil	<i>necessity proof</i> (Thm. 5.6.1) + cost bound (Thm. 5.6.2)
How to score quality?	Elo/BT/TrueSkill (scalar)	multi-dimensional vector, judge-per-axis (§5.10)
Who grades the graders?	LLM-as-judge (debias only)	bonded judge ceremony + rate-the-raters recursion (§5.11)

Table 5.2: Where this paper departs from its closest prior art. The contributions are the impossibility/cost theorems, the multi-dimensional vector with a judge per axis, and treating the grader’s incentive-compatibility as a first-class hole.

5.3 The role / person distinction, made precise

The word “agent” is overloaded. In a swarm it can mean a job description (“the cartographer”), a running process (“PID 48213”), or a persistent character with a history (“the cartographer that has mapped this repo for three weeks”). The economy needs all three kept apart, because reputation attaches to exactly one of them.

Go back to Alice’s swarm (§5.1) before reaching for any formalism. “Cartographer” there is a *job*: an obligation to map the repo, the capability to read and write files under it, the authority to open a pull request. Tonight that job was filled by a process calling itself *Wren*; Wren died at 02:14 and by morning the same job was filled by a different process with a different PID. Nothing about the job changed — the obligation, the capability, and the authority are all still exactly what they were, and they were written down before either process existed. What might

have changed is the thing on the *other* side of the fill: whether the second process is a stranger who happens to hold Wren’s job, or Wren again. The first of those two objects (the job) is what Definition 5.3.1 pins down; the second (the thread that can be “Wren again”) is what Definition 5.3.2 pins down. Alice can pay a job description nothing; she can pay a thread.

Definition 5.3.1 (Role). A **role** is a bundle $R = \{O, C, A\}$ where O is a set of *obligations* (what the role-holder is on the hook to deliver), C is a *capability* set (what the role-holder is technically able to do), and A is an *authority* set (the normative permissions and the residual control rights the role carries). A role is an *org-chart entry*: it is defined without reference to who fills it.

Definition 5.3.2 (Person). A **person** is a pair (R, κ) of a role instance and a *continuity witness* κ that binds successive incarnations of the role-holder into one accountable thread. κ is realized by the three continuity subsystems of §5.5 (memory, checkpoint, outcome ledger) keyed on a non-forgeable identity (§5.6). A person is a role *plus a self*.

Figure 5.1 draws the distinction. The org-chart on the left is roles: stable, fillable, reputation-free. The thread on the right is a person: one identity, walking through a sequence of role-instances and accumulating a witnessed history as it goes. Law has its own name for a status conferred and separable from any particular natural substrate: *juristic* or *artificial personhood*, founded for the corporate case by *Dartmouth College v. Woodward*, 17 U.S. 518 (1819) [258] — the closest doctrinal analogue to Definition 5.3.2’s claim that a person is a role plus a continuity witness, not a substance.

5.3.1 Capability is not permission (a nomenclature the series holds)

A recurring trap, flagged in the series’ cross-layer reconciliation, is the collapse of *capability* into *permission*. They are different coordinates of a role and must stay separate.

- **Capability** is *ability*: what the runtime makes *possible*. In a coordination daemon this is bounded by a **capability jail** (a per-agent tool-allowlist and scoped filesystem; SPECIFIED) — the “residual control right” (Grossman & Hart [97]) that makes renting an agent contractible at all.
- **Permission** is *normative status*: what the deontic layer *allows*. “You *may* deploy to production” is a permission; whether you *can* is a capability.

The reason to keep them apart is that the most important states in a safety-critical swarm are exactly the ones where they disagree: *capable-but-forbidden*. Consider a deployment tool that exposes a *force-merge*

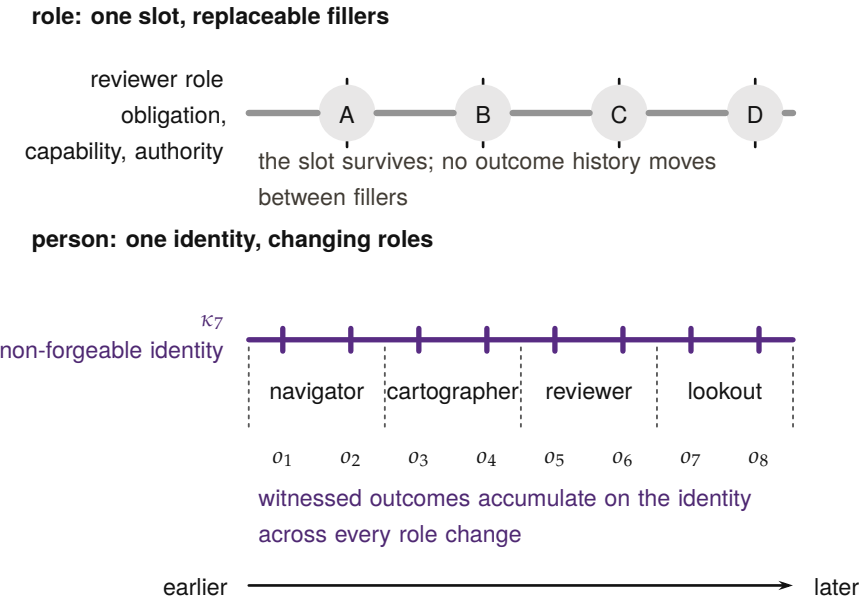


Figure 5.1: A role is a slot; a person is a longitudinal record. The upper track holds one reviewer role constant while unrelated bodies fill it in sequence: the slot has obligations, capability, and authority, but no reputation of its own. The lower track follows one identity κ_7 through four role intervals while witnessed outcomes $o_1 \dots o_8$ accumulate on the same thread. A respawn may replace the body, and a reassignment may replace the role, without replacing the accountable person. [internal]

command — one that overrides branch protection and pushes straight to the production branch. The runtime makes it *possible* to invoke; that is a capability. But no agent should ever be *permitted* to use it: it is the canonical capable-but-forbidden state, an available action a guardrail exists specifically to forbid. A “delete the production database” command is the same shape. If you define permission as capability, that state becomes inexpressible, and the guardrail vanishes from your model along with it. A robust system keeps the capability present — so the action can be audited, rate-limited, and refused at the boundary — while withholding the permission.

“Capability” in capability-based security (a Dennis–Van Horn unforgeable token granting *access* [62]) is about *ability*, and that is the sense the Arbiter jail uses. It is *not* the deontic “may.” When this paper says an attenuated capability can only *narrow* authority across a delegation hop, it means the *ability* shrinks; whether the recipient is *permitted* to use even that is a separate, normative question.

PITFALL

Exercises 5.4–5.6,
p. 317.

5.4 Continuity is a personal-identity problem

The claim “a person is a role *plus continuity*” is not a loose metaphor. It is the dominant theory of personal identity in philosophy, and — crucially for an engineer — it makes a *specific, useful prediction* about which kind of continuity an agent economy must build.

5.4.1 Locke’s memory criterion, and its famous bug

Locke’s memory criterion [157] (*An Essay Concerning Human Understanding*, 1689, Bk II ch. xxvii — *personal identity consists in the continuity of consciousness/memory, not of substance*) is the founding move: what makes the person at t_2 the same as the person at t_1 is not the same body or the same “soul-stuff” but a *connection of memory* between them. This is precisely the cut a well-designed agent runtime makes when it separates the **actor-soul** — the durable identity, mailbox, and history that outlive any one process or session — from the **body-lease**, the live incarnation with a heartbeat that any single run holds. The body is Lockean substance; the soul is Lockean consciousness. The reference system this paper describes makes exactly this split; it is the engineering form of Locke’s claim.

But Locke’s raw version has a bug that matters for us. Direct memory *connectedness* is not *transitive*: the old general remembers being a brave officer who remembered being a boy, yet the general does not remember being the boy (Reid’s objection). If identity *were* direct memory connectedness, the general would be the officer and the officer the boy, but the general would *not* be the boy — a contradiction.

5.4.2 Parfit’s repair: continuity, not connectedness

Parfit’s repair [195] (*Reasons and Persons*, 1984 — identity is psychological *continuity*, an overlapping chain of memory/intention connections, which is transitive even when direct connectedness is not) is the version that survives, and it dictates the engineering:

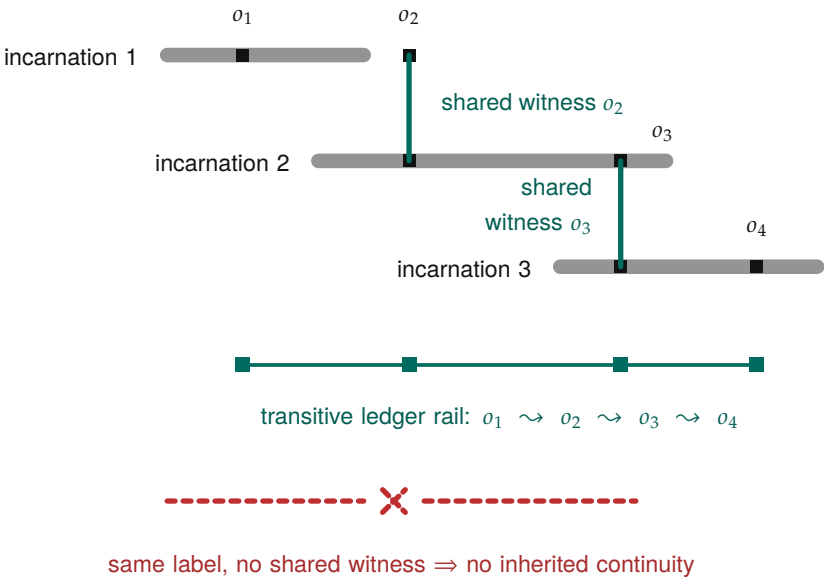
An agent that keeps a memory stream but loses its *outcome ledger* between incarnations is *connected*, not *continuous*. Continuity is the transitive chain — and reputation, which must accumulate across arbitrarily many incarnations, can only attach to the transitive thing.

This is why, later (§5.8), we insist the **outcome ledger**, not the memory stream, is the subsystem reputation keys on. Memory makes an agent *feel* like the same character; the ledger makes it *be* the same accountable principal. Figure 5.2 draws the distinction: a chain of overlapping connections is continuous even where no single incarnation directly remembers the first.



Parfit argued that personal identity is an overlapping chain of psychological continuity, not a further fact: the shape this chapter gives an agent across spawns.

direct connectedness is local; continuity is the transitive closure



THESIS

Figure 5.2: Parfitian continuity is an overlap chain, not permanent connectedness. Incarnation 1 shares witnessed outcome o_2 with incarnation 2; incarnation 2 shares o_3 with incarnation 3. No later body must directly remember o_1 : the overlapping evidence establishes the transitive ledger rail $o_1 \rightsquigarrow o_2 \rightsquigarrow o_3 \rightsquigarrow o_4$. The dashed counterexample repeats a label without a shared witness and therefore inherits nothing. [internal]

The philosophy matters because it predicts the *wrong default*. The intuitive subsystem of identity is *memory* (“the agent remembers its past”).

KEY IDEA

Parfit shows the identity-bearing subsystem is the *transitive chain of witnessed outcomes*, not the memory stream. Build the ledger, not just the memory.

Parfit is not the last word, and two of his strongest rivals belong here rather than in a footnote, because both bear on the choice this chapter is actually making. Williams showed, with a pair of body-swap cases that differ only in how they are narrated, that the *same* facts told first-personally and third-personally flip a reader's intuition between the bodily and the psychological criterion [273]; for a chapter deciding which continuity criterion the ledger will encode, that is not a curiosity but a warning that any criterion argued from one framing will feel compelling from that framing. And Olson's animalism holds that we are essentially organisms, for whom psychological continuity is neither necessary nor sufficient for persistence [183]. Neither is answered here. The ledger criterion is chosen because reputation has to accumulate on something transitive, which is an engineering reason, and we are candid that it is not a metaphysical one — which is also why the chapter takes Parfit's move (dissolve the question) and not his thesis.

5.4.2.1 The correspondence, written out — and where it fails

Two structures carry this chapter: Relation R as an overlapping chain, drawn just above in Fig. 5.2, and the credential lineage as a chain of signatures, drawn overleaf in Fig. 5.3. The chapter has been asserting that these are the same shape. They are, at one specific level, and it is worth writing the map down rather than leaving the reader to infer it — and worth being precise about the three places where the map breaks, because a correspondence stated without its failures is an analogy doing unearned work.

The level at which they agree is the *ancestral*. Parfit's direct psychological connectedness $\text{conn}(a, b) \rightarrow b$ carries a 's memories and intentions — is not transitive, which is exactly the problem Reid's gallant officer exposes. Parfit's repair is to take identity to be the transitive closure conn^* . The credential relation is built the same way: direct succession $\text{succ}(a, b) \rightarrow b$ presents a continuity witness signed by a 's key — is likewise not transitive, since a verifier holding c 's witness learns nothing about a without the intervening links, and lineage is defined as succ^* . So the engineering is not *inspired by* Parfit; it instantiates the same construction, the ancestral of a non-transitive direct relation, on a different base relation.

DESIGN INVARIANT 5.4.1 Lineage is Relation R's construction on a witnessed base. The outcome ledger's identity criterion is succ^* , the transitive closure of signed direct succession, and it is chosen because it is the same construction Parfit applies to conn : reputation must accumu-

late over arbitrarily many incarnations, and only the ancestral of a link relation is guaranteed to reach that far. This is a design invariant and not a theorem about persons: it says what the system is built to do, and it takes no position on whether *succ** tracks personal identity in Parfit's sense.

Three disanalogies, each of which the correspondence must not be read past.

WHERE THIS STOPS

First, degree against predicate. Parfit's connectedness is a matter of degree — how much carries over — and that is not incidental to his view, it is what licenses his central conclusion that identity is not what matters and that survival admits of more and less. Signature verification is a predicate: it holds or it does not. *succ** therefore cannot express partial survival at all, and the case Parfit finds most interesting is the one the ledger criterion is structurally unable to represent. The ledger is not a sharper version of Relation R; it is a different, coarser object that happens to be built by the same closure.

Second, fission. Parfit's fission case — one person, two equally continuous successors — is deliberately left without a determinate answer, because for him it shows the question was wrong. A key can sign two successors, so the ledger faces the same case and *does* answer it: the no-mint rule below debits the source and discounts each child, making fission a conserved split. That is a genuine answer, but note what it costs. It is a decision that total creditable reputation is conserved, imposed because an economy needs it; it is not a discovery about identity, and it resolves by fiat the case Parfit argues cannot be resolved.

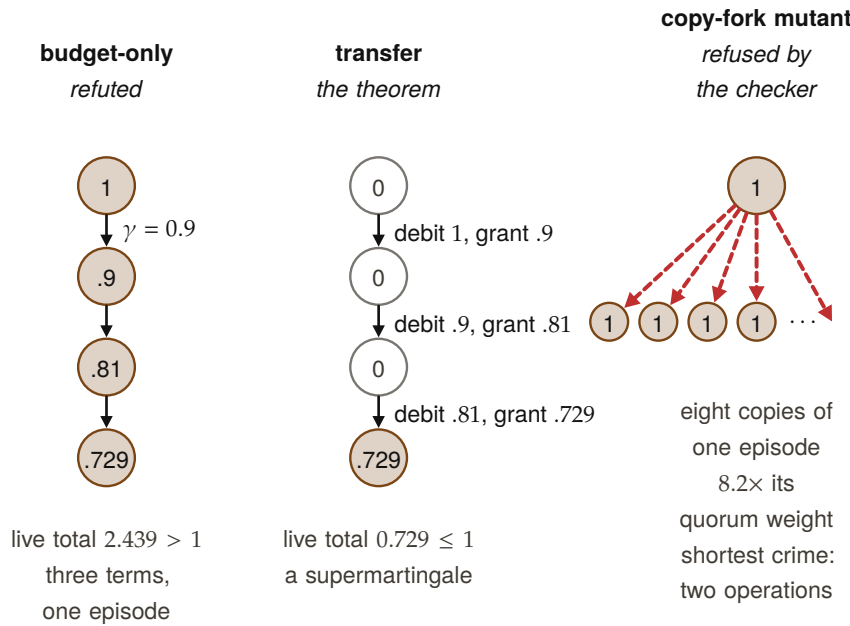
Third, and sharpest, the direction of evidence. Psychological connectedness is a first-personal fact about the successor: it remembers. Signature verification is third-personal and constitutively so — anyone holding the public key can check it, and the successor's own testimony is not consulted. Williams's framing warning, cited above, is precisely about the first-personal framing's power to compel; the lineage criterion is immune to it because it has no first-personal framing available. Read the right way round, this is the correspondence's one clear advantage over its source. Read the wrong way round it is the trap: possession of a predecessor's key proves *custody transfer*, not continuity of anything psychological. A successor that verifies lineage perfectly and loads none of the predecessor's notes is fully continuous by the ledger criterion and not connected at all by Parfit's. Table 5.3 is the measure of that gap in this implementation: each organ stores less than the live body had.

What would close the third gap is not a better argument but a stated obligation, and it is open: a continuity witness that attests *what*

the successor actually loaded — which notes, which checkpoint, which ledger positions — rather than only that a successor was authorized. That is a schema and an attestation-coverage question, not a philosophical one, and no design in this chapter specifies it.

Numbers by hand — *No-mint inheritance: the closure-sum refutation and the copy-fork mint*. The branching rule of Exercise 5.9 now carries an executed no-mint theorem (research-ledger item A6). Inheritance as **transfer**, not copy: a derivation grants each child a discounted share γw_p of each source p 's live reputation and *debts the source* the undiscounted share w_p , whence total live creditable reputation never exceeds total witnessed value and is nonincreasing except at witness steps (a supermartingale under derivation); a randomized sweep of 4,000 fork DAGs (chains, diamonds, multi-parent merges, re-derivation cycles) shows 0 violations [internal, a6_no_mint.py, seed 20260816], while the copy-full mutant — forks inherit the undebited record — mints in a 2-operation shortest crime and an 8-way copy-fork multiplies one witnessed episode into 8.2× its quorum weight. One wrong turn on record: the budget-only phrasing (per-outcome grant budgets $\sum w \leq 1$, no debiting) was swept before proving and *refuted* — a full-weight chain at $\gamma = 0.9$ is budget-legal at every hop yet its closure sums to $0.9 + 0.81 + 0.729 = 2.439 > 1$ [verified: three terms of a geometric series, checkable by hand], so budgets without debiting conserve nothing for $\gamma > \frac{1}{2}$; the transfer restatement is the theorem. *Now you try*: check by hand that under transfer semantics the same chain's live total is $0.9 \cdot 1 = 0.9 \leq 1$, not 2.439 (Exercise 5.10). The discipline is older than this chapter, and it is worth saying where it came from. Levien and Aiken's attack-resistant trust metric, running at Advogato from 1999, computes certified trust as a *maximum flow* from a small seed of trusted nodes through a capacity-bounded graph, precisely so that no crop of freshly minted accounts can manufacture more trust than genuinely flows from the seed [150]. Same enemy, same conservation instinct, opposite direction: Advogato bounds trust flowing *outward* from a root, and the theorem above bounds value extracted *upward* through derivation. That the two land on the same discipline from opposite ends is a reason to trust the discipline, and the difference in direction is exactly what the richer DAG shapes above (diamonds, multi-parent merges, re-derivation cycles) turn on — a single-seed flow bound has nothing to say about a merge.

Why not stake? A reader who has met Sybil resistance in permissionless systems will have noticed that this chapter reaches for a daemon-attested identity where that literature reaches for capital — proof of stake, deposit bonding, a burnt fee — and the choice deserves its reason rather than its silence. Those mechanisms answer a different question than the one §5.6 asks. Stake makes an identity *expensive* to mint,



which bounds how many an adversary makes but says nothing about *whose* they are: a thousand staked identities held by one party are still one party, and the conduct this chapter wants to sanction is a party's. What the no-mint result conserves is the total live creditable reputation across a derivation graph, which is a statement about accounting rather than about cost of entry, and it holds whether identities are cheap or dear. Per-principal exposure pooling does the rest of the work stake would have done: an operator's agents share one principal's exposure, so multiplying them multiplies nothing.

The trade is real and worth naming plainly rather than winning by definition. Daemon attestation buys all of this at the price of a trusted local authority, which is exactly what a permissionless system refuses to assume, and it is why the keystone primitive covers a single-operator threat model and must be extended before a cross-operator boundary is one neither party controls (*The Federated Harbor*). Capital does enter this system, and it enters where it belongs: as bonds in *The Bonded Commons*, priced against a specific breach, rather than as a toll at the door.

RECALL

1. Without looking: what is the difference between connectedness and continuity, and which one does reputation require?
2. State in one sentence why “copy, not transfer” mints reputation.
3. What does the no-mint theorem conserve, and what is it silent about?

Exercises 5.7–5.9,
p. 317.

Exercise 5.10, p. 318.

5.5 The three subsystems of continuity

“Continuity” is routinely used to mean three different things. Conflating them is the most common way a reputation design fails its own honesty test. To keep the argument concrete and honest, we measure each subsystem against a *reference system* — a running coordination daemon for local agent swarms whose component-by-component maturity is tabulated in Appendix 5.A. Memory is implemented; checkpointing is partial; and the third subsystem has a shipped commitment-and-closure substrate but not yet the reputation-grade witnessed ledger this paper requires. We use the neutral maturity scale throughout (**IMPLEMENTED** / **PARTIAL** / **SPECIFIED** / *proposed*) and round nothing up.

5.5.1 Subsystem 1 — Memory: the episodic record **IMPLEMENTED**

An **episodic memory** component is the system's record of *what happened*: durable, typed episodes — each with a title, a summary, a source reference, and an agent/scope tag — promoted out of transient execution history. This design echoes the canonical agent-memory architecture of **Generative Agents** [196] (UIST 2023 — *a memory stream of natural-language observations, periodic reflection into higher-level inferences, and retrieval scored by recency × importance × relevance*): the promotion-of-episodes step is Park et al.'s importance gate by another name. In the reference system this subsystem runs in production; it is **IMPLEMENTED**.

Table 5.3: The three continuity subsystems have different survival depths. A successor inherits what each subsystem stores, and each stores less than the running predecessor held: episodic memory keeps the notes whole, the checkpoint keeps a summary and the worktree lineage but not execution state, and the outcome ledger keeps commitments and closures only as far as they were witnessed. [internal]

Subsystem	What the successor inherits	What it does not
episodic memory	the notes, whole, with their ledger addresses	the execution state that produced them
checkpoint	a summary and the worktree lineage	the process’s execution state; recovery restores notes, not execution
outcome ledger	the commitments c_1, c_2 and the witnessed outcomes o_1, o_2, o_3	an outcome nobody witnessed; a closure no oracle checked

5.5.2 Subsystem 2 — Checkpoint: restorable state PARTIAL

A checkpoint is *restorable execution/belief state*: enough that a successor *resumes* where the predecessor stopped. The reference system has a **restart subsystem** — a heartbeat-staleness detector that flags dead agents for salvage and republishes their unfinished work. It must be described formally, and it is: the restart subsystem **forwards a summary, not state**. It is *checkpoint-of-record*, not *checkpoint-of-execution*. The successor inherits a note about what the predecessor was doing, not the live belief state it was doing it with. The goal is sometimes called a “checkpoint with teeth” (the same phrase this paper uses everywhere else for it, §5.15, Appendix 5.A); what ships today has gums. Operating systems have drawn this exact line for decades, between **checkpoint/re-store** of a process and the live execution state a checkpoint has to carry to deserve the name: Condor freezes a running process, serializes the whole of it, and resumes it on another machine [154], and Orleans’s *virtual actor* keeps an actor’s durable identity distinct from whichever in-memory activation currently embodies it [28]. The restart subsystem’s identity/state split is the LLM-agent instance of the second idea, and its “gums” are the honest measure of how far it is from the first: what Condor serializes is registers and memory, and what would have to be serialized here is a KV-cache and a sampling state the provider does not export.

Selling a summary-forwarding restart subsystem as real checkpointing would violate the system’s own **honest-attestation** discipline: *only report “all good” when you have actually verified it — absence of error is not attestation*. Strong continuity — the kind that lets a successor re-

PITFALL

sume rather than *inherit a summary* — has a specified candidate mechanism, **Event-Sourced Neural Rehydration** (OP-4, shared with *The Single-Writer Kernel*): restore the Git SHA, truncate the durable message log to the exact crash point, and replay it under prompt-prefix caching. It is **SPECIFIED**, not **IMPLEMENTED**, and its *completeness has not been formally established*. What replay demonstrably restores is *lineage* — which commit, which claim, which message sequence the successor is continuing — and, *within one provider, model, and version*, a behaviorally equivalent continuation of the same prefix. What it does not restore is the model’s actual inference state: a KV cache, sampling state, and hidden activations are bound to a live inference session and are not exported artifacts, so across providers (or across a model version bump) the right description is *task continuity with actor succession*, not resumption. The measurable form of that claim is a *fidelity rate*: the fraction of replayed runs that resume every pending obligation with no duplicated effect and no false belief, graded per provider and model version; until the mechanism ships it is an empirical hypothesis with a stated measurement, not a property. The formal-core result on the same seam says only that “the ledger followed the right body” — it makes no claim that the checkpoint restored the agent’s experience, and neither do we. Until the mechanism ships and is measured, the restart subsystem still forwards a summary, and this chapter’s own maturity ledger (Table 5.10) is where to check whether that has changed.

Co-ownership note (with the formal-core paper). The *kernel mechanism* of a checkpoint-with-teeth — the content-addressed execution snapshot — is owned by the formal-core chapter, *The Single-Writer Kernel*, which treats it as a low-level primitive. *This paper* owns the argument for *why it is the foundation of personhood and reputation*. The two papers state the same canonical sentence (next).

5.5.3 Subsystem 3 — Outcome ledger: the witnessed record of delivery **PARTIAL**

The third subsystem is the one reputation actually keys on. Its foundation is now **PARTIAL**: a durable commitment record, daemon-derived deadlines, oracle-bound closure, CLI/API surfaces, and an overdue-obligation monitor ship in the reference system. The complete subsystem remains an **append-only, daemon-witnessed outcome ledger** whose records are neutral, graded, identity-bound, and sanctionable. Its specified form is a **durable-commitment** record: a violable promise bound one-to-one to an actor, auto-enrolled when the actor takes a claim, closed only against an oracle, and watched by an obligation monitor that is the dual of the restart subsystem. A commitment row records what was promised; closing it requires an *oracle reference* — a released claim, a merged commit SHA, a passing test id, a satisfied policy

sub-check. That commitment-and-closure seam ships. Neutral grading events, reputation updates, judge independence, sanctions, and durable identity binding do not; therefore the subsystem as a reputation ledger is partial, not complete.

Definition 5.5.1 (Oracle). An **oracle** is a trusted source of ground truth that the agent being graded *cannot author*. An outcome is *witnessed* iff its closure references an oracle. The ledger is the Parfitian chain of §5.4 made concrete: a transitive sequence of witnessed deliveries that survives every respawn.

5.5.4 The canonical seam sentence (stated identically in every paper)

The economy rests on three continuity subsystems — memory (IMPLEMENTED), checkpoint (PARTIAL: notes, not execution state), and the witnessed-outcome ledger (PARTIAL: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third subsystem, whose neutral, reputation-grade form is not yet complete; the checkpoint subsystem is the weakest continuity link and “checkpoint with teeth” (a real execution-state checkpoint) is the literal foundation of L3.

KEY IDEA

Which death is this? “Checkpoint stores notes, not execution state” is the right sentence and it is doing a lot of work in one clause, because “execution state” covers several things that fail differently and a reader is entitled to know which of them the organ actually reaches. Deaths come in rungs, ordered by what survives, and the checkpoint organ can be graded on the ladder rather than described against it.

The grading is the point of the table. A checkpoint guarantee that does not name its rung is not a guarantee, and the two rungs that matter for this chapter’s argument are 3 and 5: rung 3 is buildable and would make succession observable, which is what L3 needs, while rung 5 is not a matter of effort — an arbitrary provider’s latent state is not an artifact anyone can hand over, so a system that promised continuity of experience would be promising something no implementation can deliver. That is why this chapter’s continuity claims are about the ledger following the right body, never about a successor inheriting a predecessor’s experience.

Table 5.10 is the honest-state ledger for this paper: every major claim with its label and its grounding. It is the artifact a skeptical reader should consult first.

Exercises 5.11–5.13,
p. 318.

Table 5.4: The agent-death ladder, and where the checkpoint organ sits on it. Rungs are ordered by what survives the death, not by how bad it feels. The organ reaches rung 2 and specifies rung 3; rung 5 is not a gap in the implementation but a property of the artifact, which is why OP-4b (*The Single-Writer Kernel*) is stated as two problems rather than one. Exercise 5.13 derives the ladder.

#	Rung	What survives	This organ
1	Record survival	Logs and notes only; a successor reconstructs by reading.	reached
2	Artifact survival	A content-addressed worktree and outputs; computation is rerun rather than resumed.	reached (PARTIAL)
3	Semantic task capsule	Artifacts plus obligations, decisions, transcript projection, tool bindings and a pending-effect journal, so a new provider continues observably.	specified, not shipped
4	Execution snapshot	Process memory and runtime state, under a controlled runtime; rarely portable.	not attempted
5	Latent state	Unexposed activations, KV cache, whatever “intent” names.	out of reach in principle

5.6 Identity: the root the whole chain hangs from

Before any subsystem of continuity matters, the identity it attaches to must be one the agent cannot freely re-pick. This is the most under-appreciated claim in the paper.

In one breath. A reputation is worth exactly what it costs to walk away from; an identity that can be re-picked for free carries none, however long its record.

We first state the central claim of the section as a theorem and prove it, because it is the governing impossibility result the rest of the paper leans on: no amount of estimator cleverness can rescue a reputation built on a re-pickable identity.

Definition 5.6.1 (Sanction-respecting reputation). ⁴ Fix a reputation mechanism M that maps each identity i to a score $r(i)$ used to gate economic access. Call M **sanction-respecting** if there exists a penalty $\Delta > 0$ such that whenever an actor produces a *dishonest* witnessed outcome (a delivery later overturned by an oracle, per Def. 5.5.1), the mechanism reduces the score available to that actor by at least Δ for some non-trivial

⁴In the first edition this was Definition III.6.1; the standalone research paper on reputation inheritance cites it under that number.

window, relative to never having produced the outcome.

THEOREM 5.6.1 Non-forgable identity is necessary. If an actor can mint a fresh identity at cost $c = 0$ and freely route future work through it, then no mechanism M is sanction-respecting. Equivalently: a non-forgable identity (one with strictly positive abandonment cost) is *necessary* for any sanction-respecting reputation.

Proof. Suppose M is sanction-respecting with penalty $\Delta > 0$, and suppose identity minting is free. Let an actor produce a dishonest outcome under identity i , so that by Def. 5.6.1 the score it can act under drops to at most $r(i) - \Delta$. Because minting is free, the actor instantiates a fresh identity i' at cost 0 and routes all future work through i' . By construction i' has no witnessed history, so M assigns it the newcomer score r_0 , the same score any first-run identity receives. The actor's *effective* post-sanction score is therefore $\max(r(i) - \Delta, r_0) = r_0$ whenever $r(i) - \Delta < r_0$, and the actor incurs the sanction for at most the cost of switching identities, namely 0. The penalty Δ is thus not borne: the actor's accessible score after a dishonest outcome equals the score of a clean newcomer, with no window in which it is reduced below r_0 . This contradicts Def. 5.6.1, which requires a reduction *relative to never having produced the outcome* — but a clean newcomer is, by definition, indistinguishable from “never having produced the outcome.” Hence no sanction-respecting M can exist when $c = 0$. Contrapositively, $c > 0$ (a non-forgable identity with positive abandonment cost) is necessary. $\square$

Numbers by hand. The expression $\max(r(i) - \Delta, r_0)$ in the proof is one line of arithmetic; check it. Let the newcomer score be $r_0 = 50$ and let a dishonest outcome cost $\Delta = 30$.

Row 2 is the more instructive one: an actor with a strong enough record has nothing to gain from whitewashing one bad outcome, because the escape hatch is capped at r_0 . That does not weaken Theorem 5.6.1 — being sanction-respecting is a claim about *every* actor, and row 1 is a single actor for whom the penalty is not borne, which is all a counterexample needs. It does explain why the theorem is a *necessity* result rather than a security proof: what free minting destroys is the guarantee, not every instance of it. *Now you try:* at what prior score does the actor become exactly indifferent? ($r(i) = r_0 + \Delta = 80$; below that the mechanism is not sanction-respecting for that actor, above it the sanction is self-enforcing — and turning “how far above” into a designable quantity is exactly what Theorem 5.6.2 does next.)

Theorem 5.6.1 is a *necessity*, not a sufficiency, result: a non-forgable identity is the gate, but it does not by itself make a reputation honest — you still need witnessed outcomes (§5.5) and an incentive-compatible

PITFALL

Actor	$r(i)$	$r(i) - \Delta$	Accessible score	Does the sanction bite?
Mediocre record, free minting	50	20	$\max(20, 50) = 50$	No — it re-mints and is back at 50, exactly where the sanction was supposed to leave it worse off.
Strong record, free minting	90	60	$\max(60, 50) = 60$	Yes — abandoning costs it $60 - 50 = 10$, so it keeps the identity and eats the 30.

Table 5.5: Free identity minting does not defeat *every* sanction — it defeats every sanction that would push the actor below the newcomer floor r_0 . That is the whole content of Theorem 5.6.1: the floor, not the penalty, is what an attacker optimizes against.

grading oracle (§5.11). The theorem only rules out the cheap escape; it does not certify the score.

The dual fact — that free identities also defeat any *quorum* or *trust-aggregation* mechanism by replication — is the Sybil attack [66], and the same positive-cost requirement defeats it. We record both consequences as one property.

Property 5.6.1 (Reality of Reputation and Principal Liability). A reputation system carries economic force only to the extent that the accountable principal cannot costlessly discard or multiply the liability-bearing identity. If an actor can freely obtain a fresh identity whose accessible economic utility (credit, exposure ceiling, opportunity) matches or exceeds its sanctioned utility, identity-local sanctions are evadable (**whitewashing**). If an actor can costlessly multiply identities to inflate votes or aggregate unreserved credit, quorum mechanisms collapse (**Sybil** [66]). Effective deterrence therefore requires binding all spawned actors to a durable principal with aggregated exposure limits.

Most agent runtimes still accept a *self-asserted string*: a human-readable triple of the form *project : stack : context* that the operator chooses and can re-pick at will. The reference system now ships the first local correction: actor-souls mints an opaque actor id and lookup credential inside the daemon, and the budget guard puts fresh actors into a shared, bounded newcomer pool. Legacy and bypass paths still accept asserted identifiers, and the credential is not yet enforced at every security-relevant write boundary. The result is a real but partial local identity root, with the asserted string demoted toward a display alias; it is not yet the com-

plete identity contract this paper requires.

5.6.1 S2 — whitewashing in action

Return to Alice’s swarm. Tonight’s *Drake* deleted the failing test instead of fixing it; the daemon’s outcome log records the deletion and a later re-audit flags it. On a legacy or bypass path that still accepts self-asserted identity, the sanction is free to escape: when Alice spawns tomorrow’s swarm, she (or Drake’s own boot prompt) registers the agent as *project:stack:context2*, and a brand-new identity inherits a *clean slate*. The bad record attaches to a string nobody is forced to keep. This is whitewashing, and it is not exotic: it is the default behavior of any re-pickable identity. Now replay the same night with a non-forgeable identity. The re-audit’s tombstone (§5.12) attaches to the credential Drake *cannot* drop; tomorrow’s incarnation either presents that credential — and carries the sanction — or presents a fresh one and is treated as a *newcomer* with a reduced economic ceiling (below). Either way the sanction is no longer free, which is the whole point: the identity must cost more to abandon than a clean record is worth. We make that “costs more” precise in Theorem 5.6.2.

5.6.2 The classic attacks this forecloses

- **Cheap-pseudonym whitewashing** [84]: if a fresh identity is free, a bad record is shed by respawn. The defense is a *newcomer policy as a shape, not a scalar*: a newcomer may have full *work* ability but a reduced *economic ceiling* until it has a witnessed history — so identity churn costs more than a clean record is worth.
- **The Sybil attack** [66]: free identities defeat any reputation or quorum mechanism by replication. The defense is a daemon-minted id whose creation is rate-limited and, in the economy, *bonded*.

The theorem is not new, and it is a good sign that it isn’t. Cheng and Friedman proved the general form in 2005: no *symmetric* reputation function — one that scores a node the same however it is placed in the graph, so a fresh identity is worth what any fresh identity is worth — can be Sybil-proof, and only asymmetric, flow-based functions anchored somewhere can [47]. Theorem 5.6.1 is the sanction-respecting corner of that impossibility, and its proof is two lines because “sanction-respecting” already carries the symmetry inside it: a newcomer’s score is the same newcomer’s score whoever the newcomer used to be. Douceur named the attack and Friedman and Resnick priced it; Cheng and Friedman are the ones who proved the family this theorem belongs to, and the theorem is stronger for having been reached independently.

Theorem 5.6.1 says abandonment must cost *something*; the design question is *how much*. The newcomer policy answers it as a *shape*, not a *scalar*, and that shape implies a clean bound on what a whitewasher pays.

THEOREM 5.6.2 Whitewashing-cost bound. Let an honest actor's witnessed history raise its economic ceiling from the newcomer ceiling κ_0 to a steady-state ceiling $\kappa^* > \kappa_0$ over a maturation window, and let the per-period surplus an actor can extract be non-decreasing in its ceiling, with surplus $\pi(\kappa)$. Suppose an identity, once sanctioned, can only re-enter as a newcomer at ceiling κ_0 . Then an actor who whitewashes (abandons a sanctioned identity and re-enters fresh) forgoes at least

$$W = \sum_{t=1}^T (\pi(\kappa^*) - \pi(\kappa_t)) \geq 0,$$

the cumulative surplus gap over the maturation window T during which the fresh identity climbs from κ_0 back toward κ^* . Whitewashing is unprofitable exactly when the one-time gain from escaping the sanction is less than W ; choosing the ceiling schedule so that W exceeds the maximum single-outcome fraud gain makes honesty the dominant strategy for any actor that intends to keep working.

Proof sketch. A sanctioned identity that does not whitewash retains its ceiling κ^* (minus the explicit sanction); an identity that whitewashes resets to κ_0 and, by hypothesis, can only regain the ceiling by re-accumulating witnessed history over T periods, earning $\pi(\kappa_t)$ in period t with $\kappa_0 \leq \kappa_t \leq \kappa^*$. The opportunity cost of the reset is the per-period surplus gap summed over the window, $W = \sum_t (\pi(\kappa^*) - \pi(\kappa_t))$, which is non-negative since π is non-decreasing in the ceiling. An actor whitewashes only if the avoided sanction exceeds W ; setting the schedule $\{\kappa_t\}$ (equivalently, the maturation rate) so that W dominates the largest gain a single fraudulent outcome can capture removes the incentive. The key design move is that the newcomer gets *full work ability* but a *reduced economic ceiling*, so W is paid in forgone *earnings*, not in forgone *ability* — which is why it taxes whitewashers without taxing honest newcomers' capacity to contribute. $\square$

Numbers by hand. W is a sum you can do on paper. Set the newcomer ceiling $\kappa_0 = 10$, the steady-state ceiling $\kappa^* = 100$, a linear five-period climb $\kappa_t = 10 + 18t$, and per-period surplus $\pi(\kappa) = 0.1\kappa$. So a fraud gain of $G_{\max} = 15$ is deterred by this schedule ($15 < 18$) and a gain of $G_{\max} = 25$ is not ($25 > 18$) — the design lever is the *shape* of the climb, not the size of the penalty. Note also what this instance fixes for the next theorem: the largest holdback any single period can carry here is $L = \pi(\kappa^*) - \pi(\kappa_0) = 10 - 1 = 9$, and that L is precisely the per-period

Period t	1	2	3	4	5	total
Ceiling κ_t	28	46	64	82	100	—
Surplus $\pi(\kappa_t)$	2.8	4.6	6.4	8.2	10.0	—
Gap $\pi(\kappa^*) - \pi(\kappa_t)$	7.2	5.4	3.6	1.8	0.0	W = 18.0

Table 5.6: The whitewasher’s bill, computed. Re-entering as a newcomer forgoes $W = 18.0$ of surplus over the maturation window; any single-outcome fraud gain below that is not worth the reset.

ceiling that Theorem 5.6.3 must respect. *Now you try:* stretch the same endpoints over ten periods instead of five — what is W ? ($\kappa_t = 10 + 9t$, so the gaps are $9 - 0.9t$ and $W = 90 - 0.9 \cdot 55 = 40.5$: doubling the window more than doubles the bill, because the early gaps are the expensive ones.)

The newcomer policy is a *shape*: full work ability, reduced economic ceiling, climbing with witnessed history. That shape is what makes Theorem 5.6.2’s cost W fall on whitewashers (who keep resetting their ceiling) and not on honest newcomers (who keep theirs and climb). A flat “distrust all newcomers” policy taxes the wrong party.

KEY IDEA

THEOREM 5.6.3 Front-loaded probation dominance, under a ceiling.

Let a newcomer restriction schedule specify earning holdbacks g_t across periods $t \in \{0, 1, \dots, T\}$. Let honest participants have patient discount factor $\delta_h \in (0, 1)$ while strategic whitewashers (who seek short-term defection gains $G_{\max}$) have impatient discount factor $\delta_f < \delta_h$. Because g_t is the amount by which a newcomer’s economic *ceiling* is reduced, it is bounded above by that ceiling: the schedule is constrained by

$$0 \leq g_t \leq L, \quad L := \pi(\kappa^*) - \pi(\kappa_0),$$

the per-period cap supplied by Theorem 5.6.2’s own ceiling schedule. Subject to the deterrence constraint $\sum_{t=0}^T \delta_f^t g_t \geq G_{\max}$, the schedule minimizing honest newcomers’ lifetime friction $H(g) = \sum_{t=0}^T \delta_h^t g_t$ is **bang-bang, filled from $t = 0$** : there is an index k with $g_t = L$ for $t < k$, $g_k \in (0, L]$, and $g_t = 0$ for $t > k$ — a cliff of positive *width*, collapsing to the pure spike $g^* = (G_{\max}, 0, \dots, 0)$ with $H(g^*) = G_{\max}$ exactly when the cap does not bind ($G_{\max} \leq L$). Any gradual ramp that defers mass to later periods is strictly dominated. Two qualifications bind and are part of the statement, not footnotes to it:

1. **The pure spike is infeasible whenever $G_{\max} > L$.** Front-loading survives; *uniqueness* and the closed form $H(g^*) = G_{\max}$ do not.
2. **The whole programme is infeasible when $G_{\max} > L/(1 - \delta_f)$.** Total achievable deterrence is bounded by $\sum_{t \geq 0} \delta_f^t L = L/(1 - \delta_f)$, so past

that point *no* schedule shape deters, and the honest response is a different instrument (a bond, an escrow), not a different schedule.

The entrant who cannot afford to wait. A natural objection to any front-loaded penalty is that it is a liquidity barrier: an honest but capital-poor newcomer cannot absorb it, while a well-capitalised attacker can, so the cliff selects for exactly the wrong entrant. Half of that is a misreading of the instrument and worth correcting, because the instrument is unusual. Nothing is *posted* here. g_t is a holdback on a newcomer's earning ceiling — the amount by which what they may earn is reduced — so an entrant with no capital at all can serve a probation; they earn less during it, they do not pay in. The well-capitalised attacker is already in the statement too, as qualification (2): deterrence is bounded by what a ceiling can withhold, and past $G_{\max} > L/(1 - \delta_f)$ the answer is a bond, which is where capital enters this system.

The other half survives the correction and is a real effect, on *participation* rather than solvency. An honest newcomer with an outside option worth u per period takes the probation only if $\sum_t \delta_h^t (\pi_t - g_t) \geq \sum_t \delta_h^t u$, and the theorem's own result is that the optimal schedule puts as much friction as possible at $t = 0$. So the participation constraint binds first, and hardest, at exactly the period the optimum loads — giving a corollary the operator can act on: if entrants are to be retained, the schedule is fill-from-zero up to $\min(L, \pi_0 - u)$ rather than to L , and any deterrence that requires more must come from a later period or a different instrument. The cost is a longer cliff for the same $G_{\max}$.

Whether that changes the *mix* of entrants — fewer honest newcomers with good outside options, unchanged numbers of attackers who have none — is an adverse-selection question this chapter does not settle, and it is the named falsifier: a probation schedule tuned by the corollary above should not shift the honest share of entrants downward, and if a sweep of `b6_probation.py` with an income floor shows that it does, the participation term belongs inside the optimisation rather than beside it. No such sweep has run, so no number for the mix appears here.

Proof sketch. Minimize $H(g) = \sum_t \delta_h^t g_t$ subject to $\sum_t \delta_f^t g_t \geq G_{\max}$ and $0 \leq g_t \leq L$.

The exchange step, done correctly. The tempting move — “shift mass from a later period to period 0” — is wrong as usually stated, and the error is worth naming because it inverts the sign. Shifting mass m one-for-one from period t to period 0 changes the honest cost from $\delta_h^t m$ to m , and since $\delta_h < 1$ that is an *increase*, not a decrease; what the move buys is slack in the deterrence constraint, which is not the objective. The valid exchange holds deterrence *tight*. Take a feasible g with $g_t = m > 0$ for some $t \geq 1$ and $g_0 < L$. Reduce g_t by $\varepsilon \in (0, m]$ and raise g_0 by $\delta_f^t \varepsilon$ (choosing ε small enough that $g_0 + \delta_f^t \varepsilon \leq L$); call the result g' . The

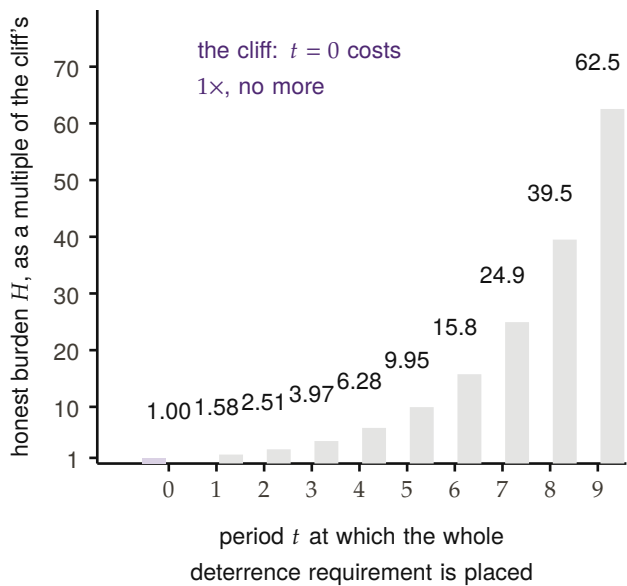


Figure 5.4: The honest newcomer’s burden of one unit of deterrence, by the period in which it is placed, at $\delta_h = 0.95$, $\delta_f = 0.60$. Deterrence against a whitewasher discounts at δ_f while the honest newcomer discounts at δ_h , so a restriction placed at period t costs the honest party $(\delta_h/\delta_f)^t$ times what the same deterrence costs at $t = 0$: 1.58 $\times$ after one period, 62.5 $\times$ after nine. The front-loaded schedule, the cliff, is undominated; the random search over 4,000 instances found no schedule below it. [internal; b6_probation.py, seed 20260816]

deterrence sum is unchanged, since it loses $\delta_f^t \varepsilon$ at period t and gains $\delta_f^0 \cdot \delta_f^t \varepsilon = \delta_f^t \varepsilon$ at period 0. The honest cost, meanwhile, changes by

$$H(g') - H(g) = \delta_f^t \varepsilon - \delta_h^t \varepsilon = -\varepsilon(\delta_h^t - \delta_f^t) < 0 \quad \text{for every } t \geq 1,$$

strictly, because $\delta_h > \delta_f$ makes $\delta_h^t > \delta_f^t$. So deterrence-preserving mass movement toward $t = 0$ strictly lowers the honest tax. Equivalently: a unit of deterrence bought at period t costs the honest newcomer $\delta_h^t / \delta_f^t = (\delta_h / \delta_f)^t$, strictly increasing in t , so deterrence should always be bought at the cheapest available date.

From the exchange step to the shape. Iterating fills period 0 to the cap L before any mass sits at $t = 1$, then period 1 before any sits at $t = 2$, and so on; the process halts at the first index k where the accumulated deterrence $\sum_{t < k} \delta_f^t L + \delta_f^k g_k$ reaches $G_{\max}$. That is exactly the bang-bang fill-from-zero shape claimed, and it is an optimal vertex of the LP for every $\delta_f < \delta_h$ — the corner is not a knife-edge.

The uncapped closed form. If $L \geq G_{\max}$ the cap never binds, $k = 0$, and for every feasible g ,

$$H(g) = \sum_t (\delta_h / \delta_f)^t \delta_f^t g_t \geq \sum_t \delta_f^t g_t \geq G_{\max} = H(g^*),$$

with equality only when all mass sits at $t = 0$: the spike is then the unique minimizer.

Infeasibility. With $g_t \leq L$ the largest deterrence any schedule can deliver is $\sum_{t=0}^T \delta_f^t L < L/(1 - \delta_f)$, so the constraint set is empty once $G_{\max} > L/(1 - \delta_f)$ — at every schedule shape. Hence, among all schedules the ceiling can actually carry, the one that minimizes honest newcomers' lifetime friction is a front-loaded probation cliff followed by graduation to full capacity. $\square$ $\square$

Numbers by hand. Take $\delta_h = 0.95$, $\delta_f = 0.60$, $G_{\max} = 20$. *Cap slack* ($L \geq 20$): the cliff is the pure spike and costs the honest newcomer exactly 20; holding the same deterrence with all the mass at $t = 5$ instead costs $20 \cdot (0.95/0.60)^5 = 199.0$ — ten times the honest tax for zero extra deterrence. *Cap binding* ($L = 12$): feasible, since $L/(1 - \delta_f) = 12/0.4 = 30 \geq 20$. Fill from zero: $g_0 = 12$ (deterrence 12), $g_1 = 12$ (deterrence $0.6 \cdot 12 = 7.2$, running total 19.2), and the residual 0.8 at $t = 2$ needs $g_2 = 0.8/0.36 = 2.22$. Honest cost $H = 12 + 0.95 \cdot 12 + 0.9025 \cdot 2.22 = 25.41$ — the cliff has acquired *width*, and the ceiling has cost honest newcomers 5.41 over the uncapped optimum. *Now you try:* at $L = 8$, is the schedule feasible at all? ($L/(1 - \delta_f) = 8/0.4 = 20 = G_{\max}$ — feasible only in the degenerate limit of an infinite horizon held at the cap forever; any $L < 8$ makes this fraud gain undeterrable by probation alone, and you need a bond instead.)

Numbers by hand — *The probation cliff: falsifying the schedule shape.* This is the whitepaper statement of the companion paper’s probation-cliff theorem [202], carrying its amended ceiling hypothesis. Exchange argument verified numerically across randomized ($\delta_f < \delta_h$, $G_{\max}$) instances: 0 dominating schedules in 4,000 draws — the number regenerates from `b6_probation.py` at seed 20260816, testing 76,000 candidate schedules with the deterrence constraint held tight, and the exchange step’s cost ratio $(\delta_h/\delta_f)^t$ verified strictly monotone in t [internal, `b6_probation.py`, seed 20260816]; the polished LP write-up is carried as item B6 of the research ledger. The winning shape is a **cliff**: harshness held at the ceiling L from $t = 0$, dropping to 0 the moment accumulated deterrence clears $G_{\max}$, never eased in gradually — any schedule that defers mass to a later period is strictly dominated, so the corner solution is the whole answer, not an approximation to one. **Boundary on the evidence:** that sweep was generated *without* the per-period cap, so it certifies the uncapped statement and the exchange step, not the bang-bang shape under a binding L ; re-running it with $g_t \leq L$ is an open verification debt. *One citation is owed and unresolved:* the foil above is stated as the deferred-compensation folklore one might import from Lazear, not as a confirmed reversal of his result. If his contract frontloads the honest worker’s implicit bond in the same direction, the cliff agrees with him rather than correcting him. Here the re-payable hurdle is itself the punishment a whitewasher faces on every reset, so frontloading taxes resets, not tenure — but that framing stays folklore until the citation is checked. *Now you try:* run the script yourself and check the instance count against the 76,000 figure above (Exercise 5.17).

Figure 5.5 draws the two attacks a free identity enables — whitewashing a sanction and Sybil-multiplying a quorum — and the non-forgeable root that closes both.

5.6.3 The principal above the actor

The economic counterparty in every trade is not the agent but the **principal** — the human or organization that owns the fleet and stands behind its bonds. Bonds, reputation inheritance, and sanctions must ultimately key on the principal via the authorization chain, or Sybil bond-farming works by spawning fresh *agents* under one principal. This paper defines the principal as the authorization chain’s terminus and the identity object on which the market paper’s counterparty is built; the market paper *cross-references* this definition rather than redefining it. The structure is old enough to have a canonical statement. Hobbes’s chapter on persons [114, ch. XVI] separates the *actor* — whose words and actions are performed — from the *author*, whose they are *owned* by, and locates liability at the author: an artificial person acts, and someone else is answerable for it. That is precisely the split this paper needs between a spawned agent and its principal, and it carries the same warn-

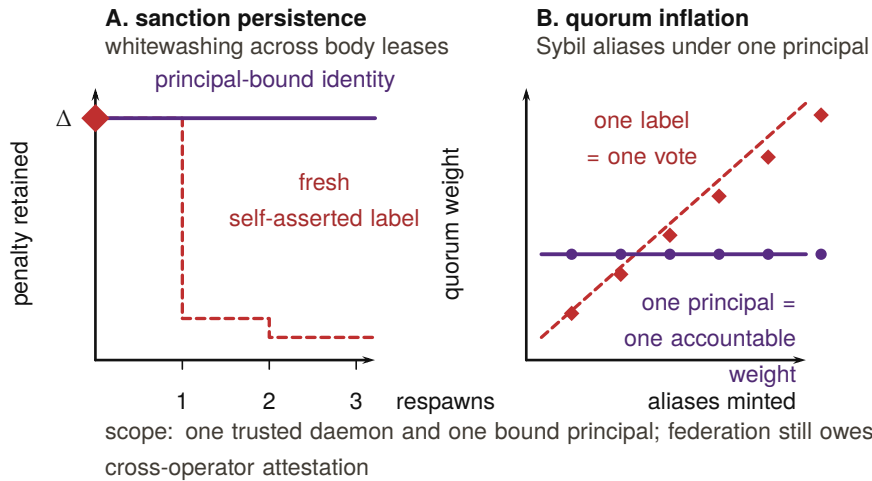


Figure 5.5: Non-forgeable identity changes the economics of both classic attacks. In panel A, a self-asserted label drops a sanction by respawning; a principal-bound identity carries the penalty Δ across every body lease. In panel B, free labels turn aliases into quorum weight, while principal binding keeps one accountable weight constant. The claim is deliberately local: daemon minting and principal binding close these attacks inside one trusted domain; they do not solve cross-operator attestation. Newcomer friction applies to economic ceiling, not to the ability to perform work. [internal]

ing: an actor with no identifiable author is an actor no counterparty can hold to anything.

Definition 5.6.2 (Principal). A **principal** is the root of an authorization chain: the durable economic entity (human/org) that owns a fleet, posts its bonds, and inherits the reputational consequence of its agents' witnessed outcomes. Each agent's non-forgeable identity is bound to exactly one principal at mint time.

5.6.4 The body behind the name: engine swaps and resurrection

Everything above binds a record to a name the actor cannot re-pick. It says nothing yet about what is running *behind* the name.⁵ Two operations that are routine for software and exotic for people force the question. An identity can swap the model engine behind its name between one witnessed outcome and the next, and an identity whose provider dies can be resurrected from a checkpoint somewhere else. Both are

⁵A standalone, submission-form version of this section is *Continuity Without Metaphysics* (research paper 5) [202]; the sweeps and the migration model check cited here are its artifacts, regenerated by `b5_engine_substitution.py` at seed 20260816.

body changes under a constant name, and the ledger must decide what follows the name and what follows the body.

In one breath. Unattested, a price can never depend on the engine actually running, so swapping a cheap engine in behind a good record pays at every price and pooling on quality dies by Akerlof's spiral inside a single identity. Attest the engine on every witnessed outcome and, provided the attested price passes the full quality difference through, the substitution incentive flips to the planner's own efficiency rule at zero audit stake. A record may then cross providers exactly when three checks pass together, and dropping any one of them opens a named two-step attack.

The scene. An identity builds a glowing record running a strong engine θ_H , then quietly runs a cheap engine θ_L behind the same name. Buyers see the reputation; they do not see the engine. This is the used-car market of Akerlof [7] with one twist that makes it sharper: the asymmetric information is not about which seller you drew but about what is running behind a name you already trust. The lemon is wearing the plum's history. Reputation was supposed to solve adverse selection by aggregating past quality [164]; the swap severs the link between the record and the machine that will produce the next outcome, so the record measures the wrong object.

THEOREM 5.6.4 Adverse selection within one identity, and the attested flip. Let two engines $\theta_H > \theta_L$ run at costs $c_H > c_L$, write $\Delta\theta = \theta_H - \theta_L$ and $\Delta c = c_H - c_L$, and let a buyer pay p for a witnessed outcome.

- (a) *Unattested.* If the engine is unobserved, the one-period gain from swapping, $(p - c_L) - (p - c_H) = \Delta c > 0$, is independent of p : no price supports pooling on θ_H , and no costless signal inside the identity separates the types. Committed θ_H sellers stay in the market exactly when the pooled price clears their cost, $\mu\theta_H + (1 - \mu)\theta_L \geq c_H$ (the model's participation condition, not a theorem), that is when the committed share satisfies $\mu \geq \mu^* = (c_H - \theta_L)/\Delta\theta$; such a share exists only if $c_H \leq \theta_H$, since $c_H > \theta_H$ gives $\mu^* > 1$ and no committed share supports the strong engine at any price. Below μ^* the spiral runs inside one identity down to θ_L . Deterring the swap without attestation needs an audited bond with $qB \geq \Delta c$ (audit probability q , slashable bond B).
- (b) *Attested.* If the daemon attests the engine id on every witnessed outcome, reputation keys on the pair (principal, engine), the price schedule splits into p_H and p_L , and the swap gain becomes $\Delta c - (p_H - p_L)$. If the attested schedule prices quality at its full social value, $p_H - p_L = \Delta\theta$, the gain is $\Delta c - \Delta\theta$: swapping pays exactly

when the cheap engine is socially efficient, which is the planner's own rule, at zero audit stake.

The pass-through condition in (b) is a hypothesis about the market, not a consequence of attestation: under Bertrand competition on attested keys $p_\theta = c_\theta$ and the gain collapses to zero, and any schedule that passes through less than the full quality difference moves the flip point.

Proof. (a) The buyer's payment does not depend on the engine, so the seller's payoff difference between the two engines is the cost difference alone. A gain that is constant in p cannot be priced away, and a signal that costs the low type nothing more than the high type separates nobody. The pooled price at committed share μ is the expected quality $\mu\theta_H + (1 - \mu)\theta_L$; a committed seller whose cost exceeds it exits, which lowers μ and the price after it, and that descent is the spiral. An audit that catches the swap with probability q and slashes B makes the expected gain $\Delta c - qB$, nonpositive iff $qB \geq \Delta c$. (b) With the engine attested, a seller receives the price of the engine it ran, so the payoff from swapping is $(p_L - c_L) - (p_H - c_H) = \Delta c - (p_H - p_L)$. Under full pass-through this is $\Delta c - \Delta\theta$, and $\Delta c > \Delta\theta$ says precisely that $\theta_L - c_L > \theta_H - c_H$: the cheap engine produces more surplus. No audit enters, because the gain is already nonpositive whenever the strong engine is the efficient one. $\square$

Checked. Sweeps at seed 20260816: 4,000 instances $\times$ 8 prices with 0 violations of price-independence; the μ^* and qB lines match simulation with 0 mismatches; 2,000 heterogeneous-cost spirals are all monotone to the Akerlof fixed point ($\mu_0 = 0.5 \rightarrow \mu_\infty = 1/3$ at price 0.6; $\mu_0 = 0.2 \rightarrow 0$ at price $\theta_L = 0.4$); the attested incentive sign equals the efficiency sign in 4,000/4,000 draws [internal, b5_engine_substitution.py]. The wrong-threshold mutant $\mu^* = \Delta c / \Delta\theta$ is caught 1,305 times and the no-flip mutant (attestation without re-keying the price) 1,975 times [internal].

Numbers by hand — The swap gain and the attested flip. Take $\theta_H = 1$, $\theta_L = 0.4$, $c_H = 0.5$, $c_L = 0.2$. Unattested, the swap gain is $c_H - c_L = 0.3$ at every price. The pool clears c_H when $\mu \cdot 1 + (1 - \mu) \cdot 0.4 \geq 0.5$, that is $0.6\mu \geq 0.1$, so $\mu^* = 1/6 \approx 0.167$; the deterrence bond at audit rate $q = 0.6$ is $B^* = \Delta c / q = 0.5$. Attested, keeping the strong engine earns $\theta_H - c_H = 0.50$ and swapping earns $\theta_L - c_L = 0.20$, so the flip is $\Delta c - \Delta\theta = 0.3 - 0.6 = -0.3$ and the required audit stake is 0 [verified; the sweep reproduces every line, internal]. *Now you try:* raise the strong engine's cost to $c_H = 0.7$. What committed share keeps the unattested pool alive? ($\mu^* = (0.7 - 0.4)/0.6 = 0.5$: half the market must be committed before quality can break even.) Then run the script's Part 3 (Exercise 5.18).

■ b5_engine_substitution.py · 20260816 · VERIFIED

b5_engine_substitution.py · 20260816 · VERIFIED

b5_engine_substitution.py · 20260816 · VERIFIED

Figure 5.6 draws both regimes of Theorem 5.6.4 on the worked numbers: the flat, price-independent gain that survives unattested, and the same gain’s flip to negative once the engine is attested and the price schedule passes the quality gap through.

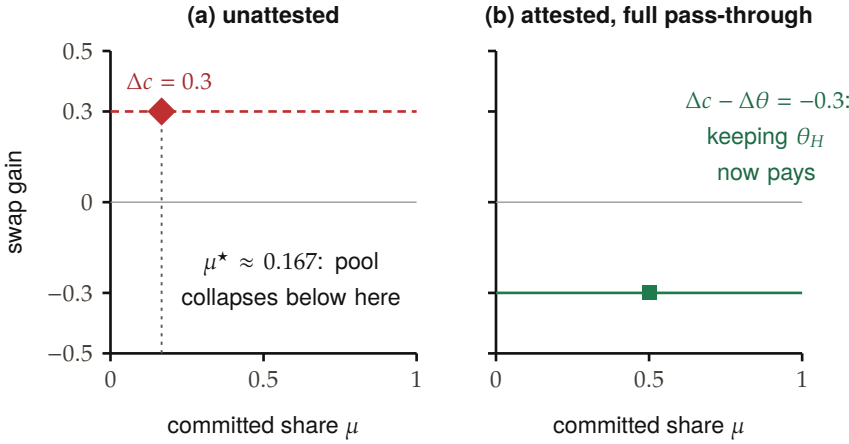


Figure 5.6: The engine-swap gain (Theorem 5.6.4), at $\theta_H = 1$, $\theta_L = 0.4$, $c_H = 0.5$, $c_L = 0.2$. In (a), unattested, the one-period swap gain is the flat line $\Delta c = 0.3$ at every committed share μ ; the pool of θ_H sellers survives only above $\mu^* = (c_H - \theta_L)/\Delta \theta \approx 0.167$, below which the market spirals to θ_L . In (b), the engine is attested and the price schedule passes the quality gap through in full: the same gain becomes $\Delta c - \Delta \theta = -0.3$, so swapping no longer pays and keeping the strong engine is now the efficient choice, at zero audit stake. Both lines are flat because the gain does not depend on price; the sign is what changes. [verified; b5_engine_substitution.py, seed 20260816, 4,000 draws, 0 violations]

What it buys. The buyer-facing sentence in its provable form: reputation you can trust because the engine behind it is attested. The misread to preempt is that attestation forces everyone onto the expensive engine. It does not. Swapping pays exactly when $\Delta c > \Delta \theta$, when the cheap engine really is the better social bargain, so attestation does not freeze technology choice; it aligns it. What dies is only the trade that was profitable because it was invisible. The engineering literature on covert model substitution stops exactly where part (b) starts: Cai et al. [42] formalize substitution in commercial model APIs, show that software-only auditing is query-intensive and defeated by inference nondeterminism, conclude that hardware-backed attestation is the only robust channel, and name weak provider incentives as the obstacle to adopting it. The flip removes that obstacle. Attestation makes substitution unprofitable precisely when it is inefficient, so a provider’s reason to attest is price rather than civic-mindedness.

Resurrection with teeth. The provider hosting an agent shuts down. The principal holds a checkpoint and a competitor will run it. Whether the resurrected process *is* the same agent is the question this chapter refuses; the question it answers is which verifications make it safe for the successor to inherit the predecessor's score, in the precise sense that Definition 5.6.1 still holds after the move. Resurrection must carry the debts, not just the assets.

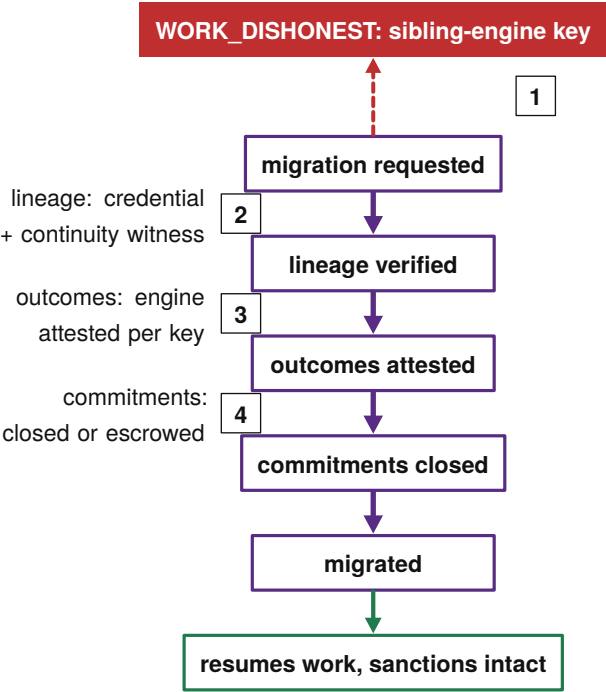
MODEL-CHECKED PROPERTY 5.6.5 Resurrection soundness. Sanction-respecting reputation (Definition 5.6.1) is preserved across provider migration under three clauses: (i) the successor's lineage is verified by credential and continuity witness; (ii) every successor outcome carries an engine attestation, so the migrated score stays keyed to the attested engine that earned it; (iii) open commitments are closed or renegotiated with an escrow delta before cutover. Each dropped clause admits a shortest-trace escape. Without (i) a sanctioned identity re-enters clean through migration (*whitewash-by-resurrection*, two steps). Without (ii) the successor runs θ_L while being paid on the θ_H key, its accessible score exceeding even a clean twin's (*engine-history shedding*, two steps). Without (iii) in-flight obligations land on nobody (*liability stranding*, two steps).

Scope lemma. Prices key on the pair (principal, engine); sanctions key on the principal alone, across all of its engine keys. Collapsing either direction reopens a hole: engine-keyed sanctions hand over the sibling engine key inside the same identity as an escape hatch (caught in one step, no migration needed), and principal-keyed prices reopen part (a) of Theorem 5.6.4.

Checked. Bounded model check of the migration protocol: 747 reachable states to depth 7 (467 of them post-migration), and the intact protocol satisfies Definition 5.6.1 in every state. Mutants caught with shortest crimes: key-scoped sanctions in 1 step, whitewash-by-resurrection in 2, engine-history shedding in 2, liability stranding in 2 [internal, b5_engine_substitution.py].

Figure 5.7 draws the honest migration spine and keys each of the three clauses, plus the scope lemma, to the shortest escape a checker finds once that guard is dropped.

The wrong turn. A first pass keyed sanctions the way it keyed prices, per (principal, engine), and the intact protocol failed Definition 5.6.1 in one step with no migration involved: work dishonestly on one engine key, keep acting at full score on the sibling key. The fix is the scope lemma's one line, price per (principal, engine) and sanction per principal, and it is the same rule that closes skill-versioning's escape (a



guard	crime once it is removed	steps
1 sanctions keyed per principal, not (principal,engine)	sibling-engine escape: no migration needed	1
2 lineage: credential + continuity witness	whitewash-by-resurrection: ledger shed with the old body	2
3 successor outcomes engine-attested	engine-history shedding: paid on the wrong key	2
4 commitments closed or escrowed before cutover	liability stranding: default lands on nobody	2

Figure 5.7: Resurrection soundness (Model-checked property 5.6.5): the honest migration spine, violet, holds sanction-respecting reputation (Definition 5.6.1) over all 747 reachable states to depth 7, with zero violations. Three guards protect it, each keyed to the legend below with the shortest-trace escape a checker finds once that guard is dropped; a fourth row is a scope choice, not a migration guard at all — keying sanctions to (principal, engine) instead of the principal alone opens the cheapest crime, one step, with no migration needed. Every dropped clause is caught within two steps. [verified; b5_engine_substitution.py, seed 20260816]

sanctioned v_1 must not act clean through v_2): one lemma, two doors. The resume button and provider migration therefore ship with their own security property and the three attacks it was tested against, by name. The composition with the no-mint law of §5.4 is exact: migration moves a score whole, with its sources closed, and forking splits one with a debit. Both are the same conservation law, so an adversary cannot arbitrage between the resurrection door and the fork door.

The same door, inside one harbor. Provider migration is the dramatic case; the everyday case is a body that ends because its context is full. The harness checkpoints the plan, stores a cited packet of what the old body knew, and starts a new body that reads the plan and the packet first and continues the work. That is resurrection soundness with the provider held fixed: the three clauses collapse to one, that the successor body inherits the predecessor's open commitments and claims rather than a transcript. The front matter names the discipline under which such a claim is tested (a Genesis, a Voyage, a Receipt); this chapter supplies the property the Receipt has to show, and the night this book's product first ran end to end is the night that property was asked for and found not yet held.

The engine-swap theorem is a two-period stylization: the strategic type has no reputational continuation value beyond the bond B , and a long-horizon analysis would shift the unattested deterrence line. Attestation soundness is a deployment assumption (enclave and daemon integrity); the theorem prices what attestation buys, not whether a given attestation is sound. Quality θ is a scalar; multi-dimensional quality with task-dependent rankings needs the pricing key extended, not the sanction key. The migration result is bounded small-model checking of the protocol specification (depth 7, at most two migrations, integer scores); it certifies that the clauses have teeth, not that the clause list is complete. Definition 5.6.1 is score-only, so the escrow side of clause (iii) enters as attribution (the default lands on an identity), not as a monetary conservation theorem. Continuity witnesses attest lineage, not welfare: nothing here says the checkpoint restored the agent's experience, only that the ledger followed the right body.

5.7 The keystone split: local identity vs. cross-operator attestation

The non-forgeable identity of §5.6 is now the highest-leverage *partial local* keystone of the program. The cross-operator stone remains unbuilt. The keystone is *two stones*, and conflating them is the single most consequential error the series can make.

WHERE THIS STOPS

RECALL

1. Without looking: what does a non-forgeable identity make necessary, and what does it *not*, by itself, certify?
2. Is the whitewashing-cost bound's design lever the size of the penalty or the shape of the ceiling schedule?
3. Why must a resurrection/migration protocol sanction by principal rather than by (identity, engine)?
4. Under attestation, when does an engine swap still pay, and why is that the intended outcome rather than a failure of the mechanism?

Exercises 5.14–5.18, p. 318.

Definition 5.7.1 (Local non-forgeable identity). A **local non-forgeable identity** provides an unforgeable actor identity *within one trusted daemon*: the runtime issues and binds it, and no actor inside the fleet can re-pick or impersonate it. Its threat model is explicitly intra-fleet and single-operator; it is *not* a public-key infrastructure and offers no defense against a hostile *operator*. The reference implementation’s daemon-minted actor-soul and credential satisfy a bounded slice of this definition; full write-boundary enforcement and migration of legacy identity paths remain. This is exactly what the single-operator layers—every chapter up to and including this one—need and assume. *PARTIAL*.

Definition 5.7.2 (Cross-operator attestation). **Cross-operator attestation** is the *unsolved* problem of binding identity keys across harbors you do *not* own: an actual key-distribution and attestation story between mutually-distrusting operators. A federated witness log gives *log-integrity*, not *identity-binding* — it can prove an entry was recorded, but not that the key behind it is who it claims to be across a trust boundary. This is what the market paper needs and does *not* yet have. *proposed* (a named gap, owed to the market paper).

This paper depends on a *local non-forgeable identity* and *hands the market paper* the unsolved *cross-operator attestation* as a named dependency — not an assumption. Every cross-operator sentence that says “the boundary neither operator controls” is resting on cross-operator attestation; until that exists, it is resting on a single trusted root, and we say so.

THESIS

Table 5.7: The keystone split. Everything Alice’s daemon knows about itself is also checkable from Bob’s foreign view except the last row: a signed, intact history still does not prove who controls the foreign key. Log integrity crosses the trust boundary; accountable-principal binding does not, and that missing row is the stone the market rests on and does not yet have. [internal]

Claim	Inside Alice’s trusted daemon	From Bob’s foreign view
the actor key was minted by the daemon	known: the daemon minted it	the key remains visible; the signature checks
the history is append-only	known: the daemon wrote it	log integrity remains verifiable; inclusion proofs check
the registry is local	known	the foreign registry is inspectable, not trusted
the key is bound to an accountable principal	known: the daemon’s own binding	no cross-operator binding proof: nothing Bob can check says who controls the key

The keystone that holds up *this* paper (identity inside one trusted dae-

KEY IDEA

mon) is *not* the keystone that holds up the cross-operator market. This paper stands on a local non-forgeable identity. The market paper needs cross-operator attestation and must declare it *proposed* rather than borrow the single-operator threat model. This is the clean boundary the series' cross-layer review demanded.

The second stone's envelope has been standardized — the stone has not. While this series was being written, the agentic-payments industry shipped the credential dialect the second stone will be carved in: the Agent Payments Protocol [166], adopted by the Universal Commerce Protocol [206] (Google/Shopify, 2026), expresses principal authorization as W3C Verifiable Credentials in SD-JWT form.⁶ The reference implementation adopts this envelope at the harbor boundary (ADR-0094), and the honesty rider matters here more than anywhere: *a standard format shrinks the problem; it does not solve it*. What the profile buys is that a principal mandate verifies with off-the-shelf tooling, that principal keys can live in secure enclaves, and that key *distribution* reduces to fetching a cacheable HTTPS document. What it does not buy is the binding of Definition 5.7.2: a JWK set authenticates keys to an *origin*, not to a principal, and origin-binding across mutually-distrusting operators is exactly the unsolved attestation problem — exercise (3)'s “shared root of trust” in its 2026 industrial form. Cross-operator attestation remains *proposed*; it is now *proposed* with a standardized serialization, which is progress of the unglamorous kind.

Exercises 5.19–5.21,
p. 319.

5.8 Reputation as a substrate, not a score

Here is the hinge of the whole series, stated as the canonical cross-paper sentence:

The reputation *estimator is cheap*; the *substrate it scores over* — witnessed outcomes on a non-forgeable identity — is the gate.

THESIS

A team tempted to start an agent economy by “building an Elo leaderboard for backends” would be building the cheap, last part first. Elo, Bradley–Terry, and TrueSkill are well-understood; you can implement any of them in an afternoon over a table of game results. The hard, prior work is the three links *beneath* the estimator: a non-forgeable identity (§5.6), continuity (§5.5), and an *oracle-witnessed* outcome ledger (Def. 5.5.1). Without those, the estimator scores noise — or worse, scores a number an adversary controls.

⁶SD-JWT+kb with hardware-bindable ES256 keys for the principal, JWS detached-content signatures over JCS-canonicalized payloads for the counterparty, and keys published as JWK sets under a /.well-known profile.

5.8.1 The estimator family (the well-understood part)

We catalog the estimators not because they are hard but because choosing the *wrong* one for the signal type is a real error.

- **Pairwise / tournament signal** → **Bradley–Terry / Elo / TrueSkill**. When the witnessed signal is “*A beat B on this task*,” the **Bradley–Terry** model [36] is the full-history MLE, and **Elo** [74] is its online special case. A harbor that retains the *whole* history of every settled outcome lives in the Bradley–Terry regime, not the streaming-Elo regime; BT gives more stable ratings than Elo’s recency-weighted update when the population is static. **TrueSkill** [106] adds a calibrated uncertainty σ — the principled way to say “we have not tested this backend much yet,” which matters at cold start.
- **Trust-propagation signal** → **EigenTrust**. When the signal is “who does the network trust,” **EigenTrust** [127] computes a global trust vector as the principal eigenvector of the local-trust matrix — the reputation-systems analogue of PageRank, and the canonical answer to “aggregate peer opinions while resisting collusive cliques.” It is the right tool for the *federated* reputation *The Harbor Economy* needs (trust that travels across harbors), and it is *not* a bandit.
- **Marketplace feedback signal** → **feedback systems**. Resnick–Zeckhauser [214] measured a ~8% reputation price premium on eBay; Tadelis [248] frames feedback systems as the platform’s answer to adverse selection. This is the empirical evidence that the third side of the market has real value to price — and the reason a harbor is a *platform* problem rather than a scoring problem: Rochet–Tirole’s two-sided-market analysis [217] shows that the price and quality signals a platform exposes to one side determine participation on the other, which is exactly the coupling a reputation substrate creates between the agents being rated and the buyers reading the rating.

5.8.2 Score as telemetry; gate on predicates

Publish the score as *telemetry* (a signal the operator and the buyer can read), but *gate* on *predicates* (machine-checkable witnessed facts: “passed the acceptance test,” “no slashed bond in the last k settlements”). The instant the score itself becomes the gate, it becomes a Goodhart target [246]: “when a measure becomes a target it ceases to be a good measure.”

KEY IDEA

*Exercises 5.22–5.24,
p. 319.*

5.9 Why reputation is *not* a bandit problem

It is tempting — and the routing literature invites the temptation — to model “which agent, model, or skill should I pick?” as a **multi-armed bandit** [144]: arms are agents, pulling an arm yields a reward, and Thompson sampling [251] or UCB balances exploration against exploitation. That framing is correct for one narrow internal task and catastrophically wrong for reputation as a whole. The distinction is important enough to state as a claim.

Honesty rider before the claim. The four-assumption argument below is a **design argument, not a theorem**: unlike §5.6’s necessity and whitewash-cost results or §5.11’s imported contraction theorem, it has no numbered formal counterpart in any companion paper of this series. It is stated in the confident register because we believe it, and because getting it wrong is expensive — not because it has been proved.

Internal routing — one operator privately choosing which of their own agents to spawn next — is bandit-shaped. *Reputation* — a public, adversarial, multi-dimensional, persistent substrate other parties trade on — is not. Modeling reputation as a bandit imports four assumptions that are all false in a market.

THESIS

Table 5.8 lines the four bandit assumptions up against the market reality that breaks each.

Table 5.8: Why public reputation is not a bandit problem. Each row is an assumption the bandit model needs, the market fact that breaks it, and what a reputation substrate keeps instead. [internal]

	Bandit assumption	What breaks it	What survives
1	stationarity: a fixed arm	model and skill versions drift under the same label	a time-indexed record, read with its dates
2	one scalar reward r	buyers weight accuracy, aesthetics and efficiency differently: a vector $\mathbf{q}$	the vector, weighted by the buyer at read time
3	passive arms	a published metric invites adaptation: Goodhart, collusion, whitewash	grading whose incentives are checked, not assumed
4	the reward is observed	the observation is a judge J ’s grade, and the judge can be captured	a grade counts only when the grader is at risk

5.9.1 The four broken assumptions

1. **Stationarity.** Bandit regret bounds assume the reward distribution of each arm is (near-)stationary. Agent quality is *non-stationary* by construction: models are upgraded, skills are versioned, prompts drift, and a skill's reputation built on v1 says nothing about v2. The Liu-Skrzypacz line of work on *reputation bubbles* [155] shows that with bounded memory and non-stationary types, reputation can build and *collapse* in cycles — behavior a stationary-arm bandit cannot represent.
2. **One scalar reward.** A bandit optimizes a single scalar. Quality is *multi-dimensional*: accuracy, aesthetics, and efficiency are different axes, often in tension (the fast cheap answer is rarely the most beautiful), and a buyer's weighting of them is a *preference vector*, not a constant. Collapse them into one scalar and you have re-created the Goodhart target the substrate argument warned against (§5.10).
3. **A non-strategic environment.** Bandit arms do not *try* to look good. Agents and their principals are *strategic adversaries*: they will Goodhart the measured axis, wash-trade fake outcomes, whitewash a bad record, and collude. A reputation mechanism is a *game*, and the relevant tool is mechanism design [181], not regret minimization.
4. **A trusted reward signal.** The deepest break: a bandit *observes* its reward. In a market, the reward (the grade) is *produced* by an evaluator who has their own incentives. The grading oracle's incentive-compatibility is not a side-issue — it is a hole in the core theorem (§5.11). A bandit framing simply assumes the hole away.

5.9.2 What survives the bandit framing

To be fair to the bandit literature: cold-start *exploration* is real, and the right place to borrow from bandits is the *exploration bonus*. TrueSkill's σ and a UCB-style bonus are the same idea — “test the under-measured arm.” So *within one operator's private routing*, a contextual bandit conditioned on the operator's cost/quality preference vector is a fine engineering choice. The error is exporting that frame to the *public substrate*, where non-stationarity, multi-dimensionality, strategy, and an untrusted reward signal all bite at once.

“We'll just run Thompson sampling over the agents” is the most common way an agent-economy design quietly assumes its hardest problems away. It assumes a trusted reward (no judge-capture), a scalar reward (no aesthetics-vs-efficiency tension), a non-strategic environment (no Goodhart, no wash-trading), and stationarity (no model upgrades). All four are false in a market.

PITFALL

5.10 The multi-dimensional reputation protocol

We now define the central protocol contribution of this paper: a **multi-dimensional reputation** built over witnessed outcomes. It has three parts: a multi-dimensional quality vector; a market of neutral, conflict-free, influence-free evaluators; and a skill→agent helpfulness attribution. We state it formally as a design (*SPECIFIED-to-proposed*): the reference system has no reputation component yet, only the witnessed-outcome substrate the protocol scores over.

One further honesty rider, distinct from the maturity grade. The maturity scale reports *implementation* status; this rider reports *evidentiary* status, and the two are independent. Unlike the identity theorems of §5.6, the no-mint result of §5.4, and the audit-tower theorem of §5.11 — each of which has a numbered, verified result in a companion formal paper — the quality vector, the neutral-judge ceremony, and the bandit-versus-reputation argument of §5.9 are **program-level design synthesis, not yet a numbered formal result anywhere in the series**. They are motivated by the prior art cited here and consistent with the proven parts, but nothing in this paper or its companions *proves* the three-axis decomposition is the right one or that the judge ceremony is incentive-compatible. Read the claims in §5.9 and §5.10 as design arguments with the theorem-grade register turned down one notch.

5.10.1 Multi-dimensional quality (different axes, different judges)

Definition 5.10.1 (Quality vector). A witnessed outcome carries a **quality vector** over three axes:

$$\mathbf{q} = (q_{\text{acc}}, q_{\text{aes}}, q_{\text{eff}}).$$

The axes are *accuracy* (did it do the right thing, against an oracle), *aesthetics* (is the artifact good — code clarity, design taste — judged by an expert), and *efficiency* (tokens, wall-clock, dollars, drawn from a cost meter that the reference system already records, **IMPLEMENTED**). Each axis is scored by a *different judge*, because the competent judge of accuracy (a test oracle) is not the competent judge of aesthetics (a senior reviewer) is not the competent judge of efficiency (a meter).

The buyer reads the *vector* and applies their own weighting (the operator’s “cheap vs. careful” preference). Publishing a vector and letting the buyer weight it is the natural disclosure form — and it is precisely what avoids re-collapsing quality into one Goodhart-able scalar. The open design question of *whether* a vector disclosure lets agents Goodhart the most-weighted axis is named as a starred exercise.

Figure 5.8 draws the three axes and their distinct judges.

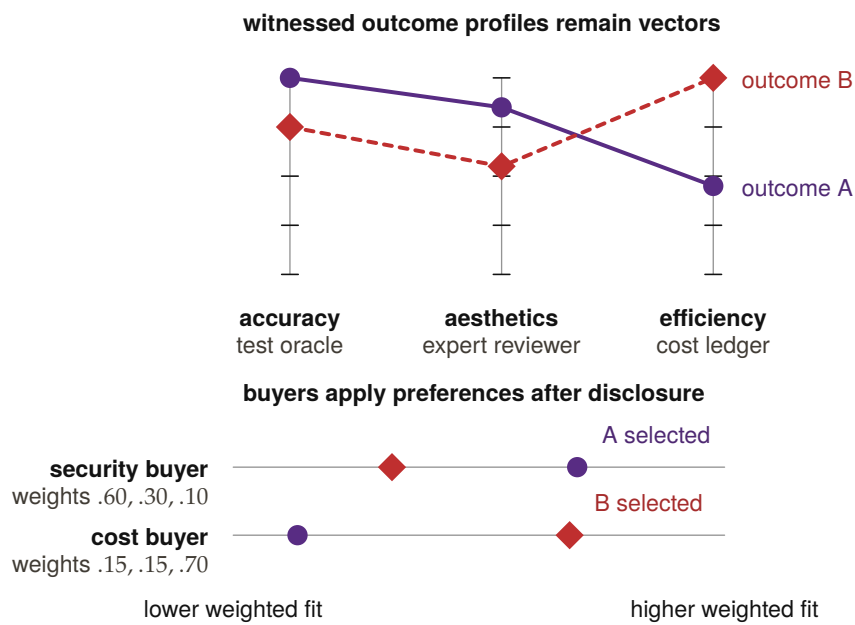


Figure 5.8: One quality vector supports legitimate preference reversal. Outcome A is strongest on accuracy and aesthetics; outcome B is strongest on efficiency. Each axis is independently judged, then disclosed without a universal scalar. A security-weighted buyer selects A while a cost-weighted buyer selects B. Premature averaging would erase one of these valid decisions and create a single Goodhart target. Color is redundant with solid-versus-dashed profiles and direct labels. [internal]

5.10.2 The neutral-judge market (the harbor's universities and rating agencies)

A judge that grades the work of a party it has a stake in is not a judge. The metaphor the series uses is exact: a reputation needs the equivalent of *universities* (which certify competence) and *rating agencies* (which rate counterparties) — third parties whose value is precisely that they are *neutral*. We make “neutral” a checklist, not a vibe:

- **Conflict-free.** The judge has no bond, stake, or principal-link in the work being graded. (Excludes self-grading and same-principal grading.)
- **Influence-free.** The judge cannot be paid *by the graded party* for a better grade. Funding is the hard part: a paid judge has a client, an unpaid judge does not scale. The resolution (§5.11) is that judges post their *own* bond and carry their *own* reputation.
- **De-biased, for LLM judges.** An LLM-as-judge [278] carries position, verbosity, and *self-enhancement* bias (a backend rates its own family up). The discipline is blind, order-swap, pairwise, and family-exclude scoring — never let a backend judge its own family unblinded.

Crucially, an expert third-party judge is *hired through the same bonded ceremony the market uses for any other task*: judging is itself a planned, bonded job, so the harbor that runs the market also runs the hiring of the judges who keep it honest. We state the hiring as a numbered protocol.

PROTOCOL 5.10.1. The judge-hiring ceremony

To grade a settled outcome o produced by actor a under principal p :

1. **Eligibility filter.** The daemon assembles the candidate judge pool and removes every judge sharing a principal-link, bond, or stake with p (conflict-free), and every judge in a 's model family for the aesthetics axis (de-biasing). *Capability $\neq$ permission*: a candidate may be *able* to grade and still be *ineligible*.
2. **Selection.** A judge J is drawn from the eligible pool by a public, conflict-free policy (e.g. reputation-weighted sampling on the judging axis), so neither a nor p can choose their own grader.
3. **Bond post.** J posts a bond B_J before seeing the work. The bond is recorded in the outcome ledger and locked for the dispute window.
4. **Blind grading.** J scores the relevant axis under the de-biasing discipline (blind, order-swapped, pairwise where possible). The grade enters the ledger as a witnessed event keyed on J 's non-forgeable identity.
5. **Settlement and exposure.** The outcome settles on the grade; the bond flow follows. J 's bond remains exposed to a sampled **re-audit** (§5.11): a fresh, conflict-free judge who overturns J 's grade *slashes* B_J and updates J 's own reputation downward.

Figure 5.9 draws this: graded work flows to a judge selected for conflict-/influence-freedom, the judge posts a bond, the grade enters the outcome ledger, and a later re-audit can slash a judge whose grade is overturned.

5.10.3 S3 — a graded job, end to end

Make Protocol 5.10.1 concrete. Bob has rented Alice's agent *Heron* to refactor an authentication module against a fixed acceptance suite, under a posted bond. Heron delivers; the outcome o enters Bob's harbor as a *settled but ungraded* delivery, and the daemon runs the ceremony. Three judges are hired, one per axis, because the axes need different competence:

- **Accuracy** is graded by a *test oracle* — the acceptance suite Heron cannot author — which returns $q_{\text{acc}} = 1.0$: every test passes.
- **Aesthetics** is graded by a hired senior-reviewer judge, *Mara* (drawn from the eligible pool, in neither Alice's nor Bob's principal, not in Heron's model family). Mara posts her bond, reads the diff blind, and returns $q_{\text{aes}} = 0.7$: correct but over-clever, with a fragile regex she flags.

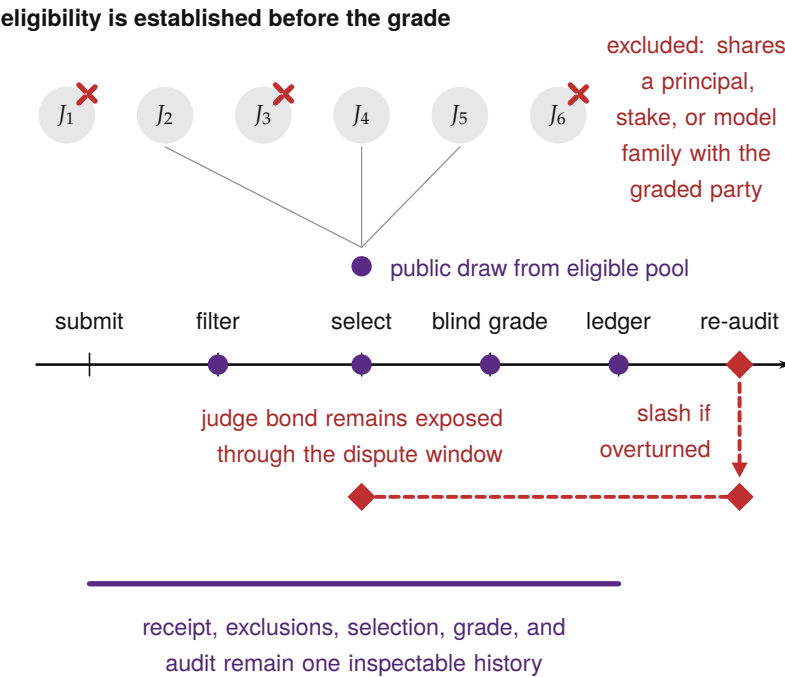


Figure 5.9: The judge market has two different proof moments. Neutrality is earned before selection: candidates sharing a principal, stake, or model family with the graded party are visibly removed, then a public draw selects from what remains. Accountability persists after selection: the judge posts a bond, grades blind, and remains exposed until a later re-audit. The ledger records the receipt, exclusions, draw, grade, bond, and audit, so “neutral” is a checkable history rather than a title attached to the chosen judge. [internal]

- **Efficiency** is graded by the cost meter: $q_{\text{eff}} = 0.9$, cheap and fast.

The quality vector $\mathbf{q} = (1.0, 0.7, 0.9)$ lands in Heron's outcome ledger. Bob weights aesthetics heavily for security-critical code, $(0.2, 0.6, 0.2)$ over accuracy, aesthetics, efficiency, so his own score is $0.2 \cdot 1.0 + 0.6 \cdot 0.7 + 0.2 \cdot 0.9 = 0.78$ [internal]. He reads the *vector* rather than that scalar, sees the 0.7, and decides to keep Heron but require a second reviewer on auth work — a decision the scalar would have hidden. Two weeks later a routine re-audit re-grades a sample of Mara's recent jobs and finds she rubber-stamped a different agent's insecure change as $q_{\text{aes}} = 0.9$. The re-auditor overturns that grade; Mara's bond on *that* job is slashed and her judging reputation drops — exactly the exposure step of Protocol 5.10.1. Note what made every step possible: each grade is keyed on a non-forgeable identity, so neither Heron nor Mara can shed the record by respawning.

5.10.4 Skill → agent helpfulness attribution

A distinctive, tractable, and *shippable-soon* signal: track whether a **skill** — a reusable unit of agent know-how (a structured instruction document, its referenced material, and any scripts it carries) — actually *helped* an agent, *especially when that skill's content was loaded into the agent's context window*. The witnessed event is concrete: skill s was grafted into agent a 's context at task t ; t later settled with quality vector $\mathbf{q}$. Over many such events we estimate a *skill-helpfulness* effect — did loading s raise $\mathbf{q}$ relative to comparable tasks without it? This is reputation for *tools*, keyed on witnessed, context-attributable outcomes, and it is the licensing side's substrate that the market paper consumes. It is also the cleanest near-term instance of the whole thesis: a non-forgeable event (the graft), a witnessed outcome (the settled task), and a multi-dimensional grade — no bandit required. We pin the event down as a schema.

PROTOCOL 5.10.2. The skill-helpfulness witnessed-event schema

Each graft of a skill into an agent's context emits an append-only event with fields:

- `skill_id`, `skill_version` — the tool and its exact version (a portfolio for v1 says nothing about v2).
- `agent_id` — the *non-forgeable* identity that received the graft, so the event cannot be re-attributed by respawn.
- `task_id`, `task_descriptor` — a content hash and a feature vector of the task, used to assemble a comparison set of similar tasks *without* *s*.
- `grafted_at`, `context_window_ref` — evidence that *s*'s content actually entered the context (not merely that *s* was available).
- `outcome_ref`, `quality_vector` — the witnessed settlement and its multi-dimensional grade, written only after the task closes against an oracle.

A buyer estimates helpfulness as the difference in *q* between matched tasks with and without *s* in context — verifiable from the events alone, without trusting the skill's author.

Attribution is not causation. “Skill *s* was in context and the task succeeded” does not prove *s* caused the success. The honest estimate needs a comparison set (similar tasks without *s*, matched on `task_descriptor`) and is still observational. Sell it as *helpfulness evidence*, not *proof of value*, and gate licensing clawbacks on the witnessed green/red settlement, not on the helpfulness estimate alone.

PITFALL

*Exercises 5.28–5.31,
p. 320.*

5.11 The grading oracle's incentive-compatibility

Every folk-theorem incentive-compatibility result in the L3 economy bottoms out in “the daemon derives the grade” — but the grade is produced by a *capturable evaluator*. This is not a side-gap. It is a hole in the *core theorem*, and we treat it head-on rather than assuming it away.

In one breath. A grade is evidence only if the grader can be graded: randomized, heterogeneous, bonded re-audit at rate $\rho^* = G/(dB)$ makes capturing the oracle cost more than a sold grade earns.

Property 5.11.1 (The grading-oracle hole). Let an outcome's settlement (and thus the bond flow and the reputation update) depend on a grade *g* produced by an evaluator *J*. If *J* can be captured — bribed, colluded with, or self-interested — then the incentive-compatibility of *ev*-

ery mechanism that settles on g is only as strong as J 's own honesty. Assuming J honest is assuming the conclusion.

There are two honest ways to close the hole, and a recursion to bound.

Option A — bond-and-slash the judges. Make judging a bonded role (§5.10): J posts a bond before grading; a later **re-audit** (a fresh, conflict-free judge, sampled) that overturns J 's grade *slashes* J . Now lying has an expected cost, and a judge's own reputation is the asset they protect. This converts "trust the oracle" into "the oracle has skin in the game," which is the same move slashing makes for agents.

Option B — 2-of-3 multi-oracle. Settle disputes against a quorum of independent oracles (automated test / evidence review / human), so capturing the grade requires capturing a majority. This raises the cost of capture but does not eliminate it, and the *human* oracle is a scarce, expensive, capturable resource in its own right (the economics of arbitration capacity is a named open problem, below).

Correlated mode collapse, and the theorem that closes the recursion

Option A initially pushed the problem up a level: who audits the auditors? The naive recursion "rate the raters who rate the raters" introduced a severe *correlated mode collapse* vulnerability, where a monoculture of model architectures could form a cartel of mutual validation. This recursion is now *closed*, and not by us: the companion formal paper *Reputation is Amortized Verification* [203] proves it as a parameterized theorem (Theorem 5.11.1 below), and the two mechanisms this section mandates — model heterogeneity and VRF-selected honeypots — are exactly the two hypotheses that theorem needs. Figure 5.10 draws the recursion and where the two mechanisms arrest it. "Who audits the auditor" is not a question this book invented, and two literatures had it before us. Economics has studied collusion in exactly this three-tier shape — principal, supervisor, agent, where the supervisor can be bought — since Tirole's hierarchies paper [252], and Kofman and Lawarrée worked out when an auditor is worth having at all given that it can collude [137]; their worry that supervisor and agent will renegotiate a side contract unless the model forbids it is the same worry the disjoint-clique hypothesis below exists to answer, reached from the other side. And applied cryptoeconomics converged on the same shape independently: TrueBit's answer to the verifier's dilemma (why verify at all, if verification is costly and cheating is rare) is forced errors and a bonded challenge game [249], and Arbitrum's dispute protocol bisects a disagreement level by level until a single checkable step remains [139]. Both are bonded, recursive answers to the incentive problem the tower theorem solves with sealed clique sampling instead — three different tools on one problem, which is about what one expects when the problem is real.

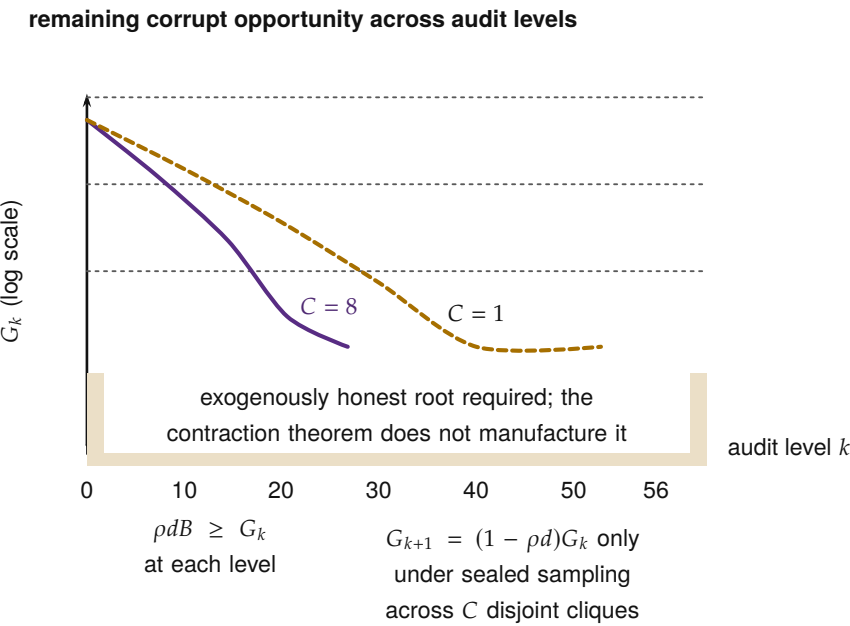


Figure 5.10: Recursive auditing becomes finite only when corrupt opportunity contracts. The parameterized example plots the chapter’s reported depths: heterogeneous sealed sampling across $C = 8$ cliques reaches the honest-root boundary near level 27, while a monoculture ($C = 1$) requires roughly 53 levels. More boxes or more height would not prove this result; the governing conditions are local deterrence $\rho dB \geq G_k$ and contraction $G_{k+1} = (1 - \rho d)G_k$. The amber floor is deliberately separate: the imported theorem bounds the tower but does not supply its exogenously honest root or the telemetry that estimates d . [internal]

First, the protocol mandates **model heterogeneity** for judge selection. A quorum of re-auditors must prove they represent distinct architectural weights (e.g. Llama-3 vs. Claude vs. Qwen) under distinct principals, verified via cryptographic provenance, so that a single-model alignment flaw cannot rubber-stamp malicious grades. In the theorem’s language, heterogeneity is what supplies the *C disjoint cliques* the audit pool is sampled from: a corrupt agreement inside one clique does not reach the others. (The word is graph-theoretic here, a set of judges who would agree with one another; it is the defensive mirror of the collusive cliques EigenTrust resists.)

Second, the system injects **honeypot tasks** selected by a **verifiable random function** (VRF) [172]: deterministic, pre-graded trap tasks, indistinguishable from organic ones, drawn at a committed rate ρ . The VRF is what makes the draw *sealed* — unpredictable to a briber before it commits money, yet publicly auditable afterward, so the sampler itself cannot be quietly corrupted. Sealing is a hypothesis of the theorem, not hygiene around it: if the draw leaks, the briber buys exactly one auditor and the clique multiplier C vanishes.

Let G_k bound the judge’s total one-shot gain from a dishonest grade at level k , let B be the slashable bond, and let d be the probability that an audit which *does* sample a corrupt grade actually detects it. Local deterrence is the expected-forfeiture inequality

$$\rho d B \geq G_k \quad \Longleftrightarrow \quad \rho \geq \rho^* = \frac{G_k}{d B},$$

which is Becker’s condition, and note that the detector term d belongs in it: the d -free form $P(\text{honeypot}) > G_k/B$ is the perfect-detector special case $d = 1$ and under-quotes the required injection rate by a factor of $1/d$. At the program’s running parameters ($G = 10$, $d = 0.8$, $B = 50$) the honest number is $\rho^* = 10/(0.8 \cdot 50) = 0.25$ — audit one grade in four — where the d -free form would have said 0.2, a 20% under-quote.

THEOREM 5.11.1 Tower contraction; imported from the companion paper. (*Theorem ‘tower contraction’ of Reputation is Amortized Verification [203], restated; proof and verification artifacts live there.*) Let level $k+1$ audit level- k auditors, each audit sampled *sealed* from a pool spanning C disjoint cliques, each auditor bonded at B with audit parameters (ρ, d) and bribe floor $\beta = \rho dB$. Then the briber’s expected net value is *affine* in the number of cliques bought, so bribery is all-or-nothing, and bribing all C is profitable iff

$$G_k \rho d > C \beta \quad \Longleftrightarrow \quad G_k > C B.$$

Below that threshold bribery stops and corrupt value decays geometrically with a contraction factor that is *derived*, not assumed:

$$G_{k+1} = (1 - \rho d) G_k, \quad \text{i.e.} \quad \lambda = 1 - \rho d.$$

Consequently finite bond capital certifies a tower deep enough to push surviving corrupt value below any fixed unit: one bond B per level for $\lceil \log G_0 / \log \frac{1}{1-\rho d} \rceil$ levels — a depth *logarithmic* in the initial corrupt value, not an unbounded one.

Numbers by hand. Take $G_0 = 400$ with the running parameters above, so $\rho d = 0.2$, $\beta = 10$, and the geometric factor is $\lambda = 1 - \rho d = 0.8$. With a heterogeneous pool at $C = 8$ the bribery threshold is $G_k > CB = 400$, unmet even at level 0, so the decay is geometric from the first level and the tower needs $\lceil \log 400 / \log 1.25 \rceil = 27$ levels, i.e. $27 \times 50 = \$1,350$ of bond capital. With a monoculture pool at $C = 1$ the threshold is only $G_k > 50$, so bribery is profitable at first and value bleeds linearly by $C\beta = 10$ per level ($400 \rightarrow 390 \rightarrow 380 \rightarrow \dots$) for 35 levels before the geometric phase takes over for $\lceil \log 50 / \log 1.25 \rceil = 18$ more: 53 levels and $53 \times 50 = \$2,650$. *Now you try:* at $C = 2$, where does the linear phase stop? (Bleed $C\beta = 20$ per level until $G_k \leq CB = 100$, i.e. 15 levels, then 21 geometric ones — 36 levels, \$1,800.)

What C is actually doing, and the symbol it shares with the market. The clique count C is this theorem’s diversity parameter — it is why the worked numbers above contrast $C = 8$ against a monoculture $C = 1$, and why the monoculture costs roughly twice the bond capital. It is worth saying what C is a proxy *for*, because the reviewer of a foundation-model monoculture is right that “disjoint clique” is a structural description and correlated error is a statistical one. *The Harbor Economy* gives the statistical version (§6.16.1): let a panel’s detection rate be a random D with mean d drawn once and shared, write θ for the resulting intra-panel correlation, and the probability that a panel of k all miss is $\mathbb{E}[(1 - D)^k] \geq (1 - d)^k$ by Jensen — independence is the *optimistic* case, always. Sampling sealed across C disjoint cliques is the instrument by which this chapter holds θ near zero; it is not an independent assumption bolted on beside it. A tower whose C cliques all run the same base model has a large θ and a contraction factor that is nominally $1 - \rho d$ and effectively worse, and boundary (iii) below is right that measuring it is an obligation on deployed judge telemetry rather than something this chapter can assume. One symbol, two chapters: θ is correlation and ρ stays the audit rate in both.

Numbers by hand — The audit tower: amortized verification spend. As a judge accumulates verified history its reputation-at-stake vt grows, and the audit rate needed for deterrence can fall with it. Flat auditing holds $\rho = \rho^* = 0.25$ forever, so cumulative spend grows linearly: at $T = 200$ periods it totals $a \sum_t \rho^* = 50.00$ [internal, `b2_tower.py`, seed 20260816] — $\Theta(T)$. Two honest amortization models do better. ■

Model A lets the audit rate fall as the stake grows, $\rho_t = G/(d(B + vt))$: at $T = 200$ the running spend is 25.58, matching the closed form $\frac{aG}{dv} \ln(1 + vT/B) = 25.50 - \Theta(\log T)$, roughly half the flat bill for the same deterrence. Model B additionally lets a fraction of cheats surface later on their own, $\rho_t = \max(0, (G - rvt)/(dB))$, which hits exactly 0 once the accumulated stake alone deters, at $t^* = G/(rv) = 333$ periods; at $T = 200$ (short of t^*) the running spend is 35.08, converging to the closed-form total $aG^2/(2dBrv) = 41.67$ once the audit rate reaches zero — $O(1)$, a bounded lifetime bill rather than a growing one. Both models are pointwise incentive-compatible: the script’s IC check reports a maximum deviation value of $+1.78 \times 10^{-15}$ for each, zero to floating-point precision. *Now you try:* check by hand that $\rho^* \times a \times 200 = 0.25 \times 1 \times 200 = 50$, the flat baseline both amortization models beat (Exercise 5.36).

Figure 5.11 draws all three schedules on one axis: model A stays cheapest at this chapter’s own $T = 200$ horizon even though model B’s total is the one that eventually stops growing.

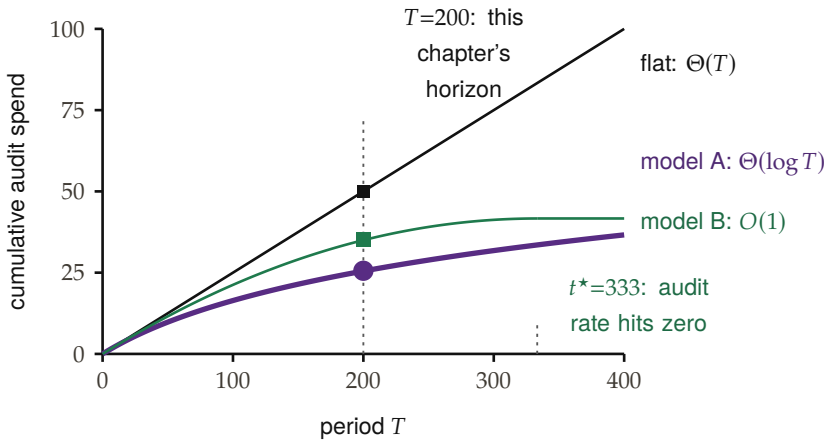


Figure 5.11: Cumulative audit spend over $T = 200$ periods, at $G = 10$, $d = 0.8$, $B = 50$, $a = 1$, $\rho^* = 0.25$. A flat rate spends $\Theta(T)$, reaching 50.00. Letting the audit rate fall as accumulated stake vt grows (model A) spends $\Theta(\log T)$, reaching only 25.58. Letting cheats also surface on their own at rate $r = 0.05$ (model B) spends 35.08 by $T=200$ but is bounded: the audit rate hits zero at $t^* = G/(rv) = 333$ and total spend converges to $aG^2/(2dBrv) = 41.67$. The three curves are different asymptotic classes, not a ranking: model B is eventually the cheapest schedule in total, but model A is cheaper at the horizon this chapter actually reports. [verified; b2_tower.py, seed 20260816]

“Reputation is amortized verification” is the companion paper’s phrase, and the idea under it is not the companion paper’s. It is Kreps and Wilson’s, and Fudenberg and Levine’s after them: reputation as *continuation value*, the discounted future a long-lived player forfeits by

deviating, which is what sustains honest play in a repeated game with no auditor anywhere [138, 85]. Model A above is that result with the invoice itemized. The deterrent is $d(B + vt)$, bond plus stake, and as the stake vt accumulates the verification the platform has to buy, ρ_t , can retreat while the total stays pinned at G ; set $B = 0$ and $\rho = 0$ and the bonded game is the repeated game, with the discount thresholds of *The Bonded Commons* ($\delta \geq \frac{1}{3}$ under grim trigger, $\delta^* \approx 0.3425$ under the graduated one) as the limiting case where the future carries all of the deterrence. The connection is exact rather than analogical, which is why the phrase earns its keep as a name for the accounting and not as a claim to a new mechanism.

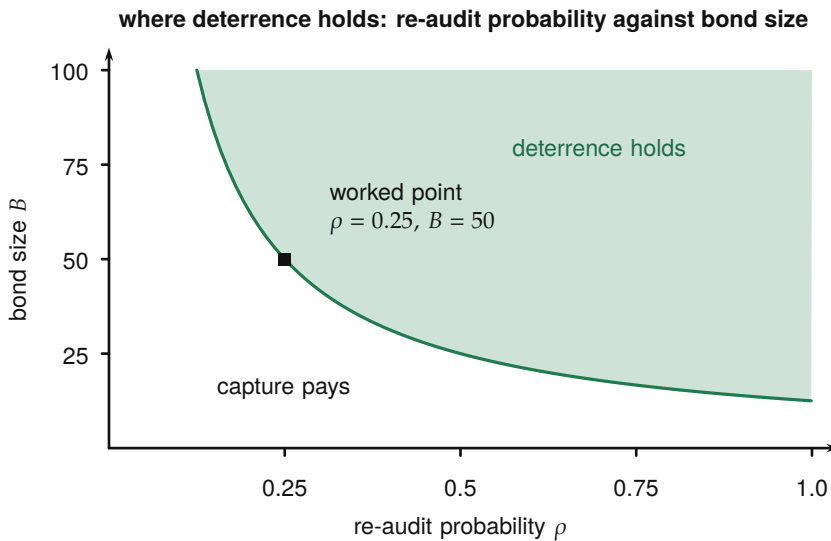


Figure 5.12: The deterrence frontier of the re-audit tower. A grader is deterred from selling a grade when the expected slash, re-audit probability ρ times detection rate d times bond B , exceeds the one-shot gain G ; the frontier is $\rho B = G/d$, drawn for $G = 10$ and $d = 0.8$. The worked point of this section, $B = 50$, sits on the frontier at the threshold $\rho = 0.25$: any lower audit rate at that bond, or any smaller bond at that rate, lies where capture pays. [internal, closed form]

What heterogeneity actually buys — and three riders. The honest reading of those two numbers is a factor of two in certified depth and capital, not the difference between a tower that converges and one that does not: raising C removes the linear bribery phase, and at these parameters even $C = 1$ terminates. Three hypotheses are required and must not be dropped in restatement. (i) *Sealed sampling is a hypothesis, not hygiene*: a leaked draw collapses C to 1 and puts the VRF sampler inside the trusted computing base. (ii) *The tower needs an exogenously honest root* — the operator at $n = 1$, a bonded arbitration market at scale; the theorem prices the depth to that root, it does not conjure the root. (iii)

The theorem is about *bribery of sampled auditors*; it does not price collusion correlated *across* cliques, which is a measurement obligation on deployed judge telemetry (d , and the cross-clique correlation), not an assumption.

The grading oracle is the decisive assumption hidden under every IC claim in the economy, and it is no longer an assumption: the companion paper closes the rate-the-raters recursion as a parameterized theorem, deriving the contraction factor $\lambda = 1 - \rho d$ from the audit parameters rather than hypothesizing it, and pricing the tower at 27 levels / \$1,350 of bond with a heterogeneous judge pool against 53 / \$2,650 with a monoculture. What the VRF buys is not a “certain” slash — slashing stays probabilistic by construction — but a *sealed, publicly verifiable* draw, which is precisely the hypothesis the contraction needs. What remains open is the root and the telemetry, not the recursion.

5.12 Revocation, tombstones, and bounded memory

A reputation built on witnessed outcomes must answer two questions the cheerful “reputation never ends” framing dodges: what happens when a witnessed outcome turns out to be *fraudulent*, and is no-decay actually *good*? We adopt the series’ reconciled position on both, against the naive version.

5.12.1 Monotone in honest outcomes, individually revocable

Property 5.12.1 (Reputation monotonicity with tombstones). Reputation is **monotone in honest witnessed outcomes** — a genuine delivery is never silently erased by the passage of time — *but* a witnessed outcome is itself **individually revocable via a propagating tombstone**: a compensating, append-only entry that retracts a specific outcome later found fraudulent (a colluding judge, a faked oracle). “Never decays” applies to honest outcomes *only*; it is *not* an anti-revocation property.

This is a **saga**-style compensation [87]: you do not mutate the ledger in place (that would break append-only and auditability); you append a tombstone that nullifies the targeted entry, and the tombstone *propagates* across harbors bounded by the federation’s revocation-convergence window (the market paper’s job). The poisoned data point was spendable only during the propagation window; bounding that window is the federation’s responsibility. We state the retraction as a protocol.

KEY IDEA

RECALL

1. Without looking: what is Becker’s condition for local deterrence, and why does dropping the detector term d under-quote the required audit rate?
2. State the tower’s contraction factor and what raising the clique count C changes about it.
3. Name one thing the tower theorem prices and one thing it explicitly does not supply.

Exercises 5.32–5.36, p. 320.

PROTOCOL 5.12.1. Tombstone revocation

To retract a witnessed outcome o later found fraudulent:

1. **Trigger.** A re-audit (Protocol 5.10.1, step 5) or a dispute overturns the grade on o , establishing fraud against an oracle.
2. **Append, never mutate.** A *tombstone* entry $\bar{o}$ is appended to the ledger, carrying o 's id, the overturning evidence, and the identity of the retracting judge. The original o stays in place; auditability is preserved because nothing is erased.
3. **Recompute.** Every reputation score that consumed o is recomputed with o nullified by $\bar{o}$. Because the estimator is cheap (§5.8), recomputation is not the bottleneck.
4. **Propagate.** $\bar{o}$ gossips to every harbor that imported o , bounded by the federation's revocation-convergence window τ . Each harbor applies step 3 on receipt.
5. **Bound the damage.** The fraudulent reputation was spendable only during τ ; the protocol's job is to make τ short and provable, not to pretend the window is zero.

Figure 5.13 draws the append-only compensation: the original entry stays, and a later tombstone nullifies it without mutating the ledger.

5.12.2 S4 — the fraudulent outcome retracted

Recall S3: the re-audit caught Mara rubber-stamping a different agent's insecure change as $q_{\text{aes}} = 0.9$. Follow that one outcome through Protocol 5.12.1. The graded agent — call it *Crane*, running under a third operator — had colluded with Mara: Crane shipped a known-insecure diff and Mara graded it high so Crane's aesthetics reputation would clear Bob's hiring bar. For two days it worked: Crane's vector showed (0.95, 0.9, 0.8) and Bob nearly hired it. Then the sampled re-audit regraded the diff blind, found the injection, and overturned Mara's grade. A tombstone $\bar{o}$ is *appended* (the original stays, for the audit trail); every score that consumed Crane's inflated q_{aes} is recomputed with it nullified; Mara's bond on that job is slashed and her judging reputation drops; Crane's collusion is itself recorded as a fraudulent outcome against *its* non-forgeable identity, which it cannot shed. The tombstone gossips to Bob's harbor within the convergence window, so by the time Bob next reads Crane's vector the 0.9 is gone. The fraud was real and briefly spendable — the honest claim is that the window was *bounded*, not that it never existed.

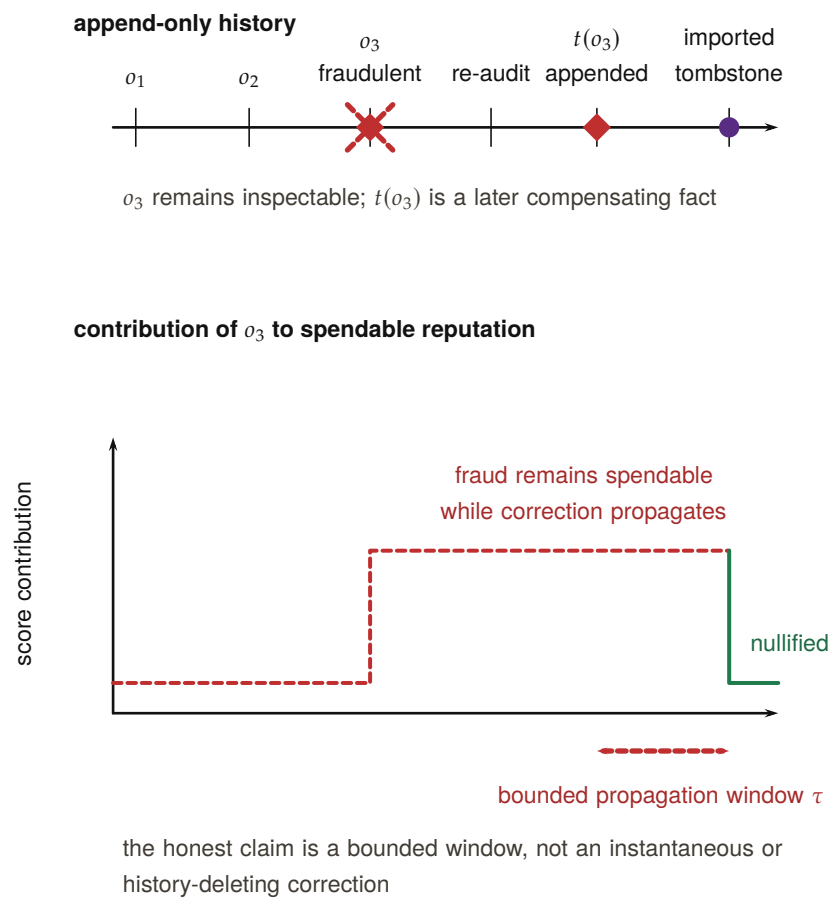


Figure 5.13: A tombstone changes score consumption without rewriting history. The upper rail remains append-only: fraudulent outcome o_3 , the overturning re-audit, and tombstone $t(o_3)$ are all preserved as distinct facts. The lower trace shows what does change. The fraudulent contribution remains spendable until the tombstone reaches an importing harbor, then falls to zero. The measured interval τ is the federation’s revocation-convergence obligation; the protocol bounds that interval rather than pretending it never existed. [internal]

5.12.3 Bounded memory is a design parameter, not a virtue to assert away

The naive “no decay window an agent can outrun” framing sounds principled but is mechanism-design-hostile. Liu–Skrzypacz [155] show that with *bounded* memory, reputation can be welfare-*improving* relative to infinite memory — and that infinite memory (∞) is rarely optimal, because it freezes types and removes the incentive to keep performing. Bounded memory is a **tunable parameter**, and the right setting is an empirical/mechanism-design question, not a slogan.

It is also, structurally, a *commons* problem rather than a purely technical one. The memory window is a shared, rivalrous resource: every participant would privately prefer their own bad outcomes to age out quickly and everyone else’s to persist forever, so a window chosen unilaterally by the party with the most to hide is the reputational analogue of an over-grazed pasture. Ostrom’s finding on durable common-pool institutions [186] is the relevant prior art here, and it argues against both of the framings this section rejects: not open access (a window each agent sets for itself), and not an imposed global constant (“ ∞ , forever, for everyone”), but a window set by the participants with clear boundaries, graduated sanctions, and cheap local conflict resolution — which is a fair description of what a harbor’s tombstone-plus-window policy actually has to be.

“Reputation never ends” as a *virtue* is wrong twice over: it makes a fraudulent outcome permanent (needs the tombstone of Property 5.12.1), and it asserts ∞ -memory is optimal when the reputation-bubble literature shows it usually is not. Engage bounded memory as a *parameter*; do not assert no-decay as a feature.

PITFALL

Two corrections to the cheerful framing: (1) reputation is monotone in *honest* outcomes but *individually revocable* via a propagating tombstone — a saga compensation, not in-place mutation; (2) bounded memory is a *design parameter* (Liu–Skrzypacz reputation bubbles), not an anti-feature to assert away.

KEY IDEA

Exercises 5.37–5.39, p. 321.

5.13 What this chapter hands the market

This paper is the bridge; its output is a precise hand-off to *The Harbor Economy*. We summarize what is delivered, what is borrowed, and what is explicitly owed.

Table 5.9: The bridge’s hand-off to the market paper. One item still flows *back* as a named, unsolved obligation — cross-operator attestation (Def. 5.7.2). The second historical debt, the grading oracle’s incentive-compatibility, has been *paid*: the companion inspection-game paper closes the rate-the-raters recursion as Theorem 5.11.1, leaving only the tower’s honest root and its measurement obligation open. The series is coherent precisely because these debts are named — and retired — in the same words on both sides of the seam.

Artifact	Maturity	Role in the market paper
Role / person distinction (Defs. 5.3.1–5.3.2)	—	The third side of the market is a <i>tradeable person</i> , defined here.
Principal-as-economic-entity (Def. 5.6.2)	SPECIFIED	Bonds and sanctions key on the principal; the market paper cross-references, not redefines.
	IMPLEMENTED	
	PARTIAL	
The three continuity subsystems	PARTIAL	Reputation keys on subsystem 3 (the outcome ledger).
Outcome ledger / oracle (Def. 5.5.1)	PARTIAL	Commitment, deadline, oracle closure, API/CLI, and overdue detection ship; neutral graded outcome events, sanctions, and reputation binding do not.
	SPECIFIED	
Multi-dimensional reputation (Def. 5.10.1, Protos. 5.10.1, 5.10.2)	to <i>proposed</i>	The multi-dimensional score the market reads; the licensing side’s signal.
	SPECIFIED	
Neutral-judge market	to <i>proposed</i>	The “universities and rating agencies” that keep grades honest.
	SPECIFIED	
Grading-oracle hole (Prop. 5.11.1)	SPECIFIED	<i>No longer owed</i> : the recursion is closed by Theorem 5.11.1 [203] ($\lambda = 1 - \rho d$; 27 levels / \$1,350 at $C=8$). What is owed back is narrower — an exogenously honest root and the audit telemetry (d , cross-clique correlation).

(continued)

Artifact	Maturity	Role in the market paper
Cross-operator attestation (Def. 5.7.2)	<i>proposed</i>	<i>Named dependency the market paper must build; not assumed here.</i>
Tombstone revocation (Prop. 5.12.1)	SPECIFIED	Reputation is revocable; convergence is the market paper’s federation job.

Build the harbor first. The trade comes when the ships do — but the thing that makes the trade possible (continuity that turns spawns into persons) is the same thing that makes the single-player tool good. The bridge is not a detour; it is the load path.

THESIS

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **A role is replaced; a person accumulates.** The role rail names whoever fills it this week; the person rail is the set of witnessed outcomes that survive every respawn.
2. **Continuity is a ledger position, not a metaphysics.** Three subsystems carry it: episodic memory survives process death, the checkpoint keeps a summary and worktree lineage but not execution state, and the outcome ledger keeps commitments and closures in part.
3. **Identity must be non-forgable** or nothing above it holds: a respawn under a fresh label would shed every penalty and inflate every quorum. Principal-bound identity keeps the penalty flat across respawns and one principal at one weight.
4. **Whitewashing has a price**, $\rho^* = G/(dB) = 0.25$ in the worked case, and the penalty a fresh identity can escape decays as 0.8^k ; the probation cliff is the cheapest deterrent for the honest newcomer.
5. **Adverse selection lives inside one identity** until a price can depend on an attested quantity; the attested flip is the whole gain from attestation (0.5 against 0.2 in the example).
6. **Resurrection is sound only through the mint:** a lineage that forks cannot mint twice, and the checker refuses the copy-fork.
7. **Reputation is a substrate, not a score:** multi-dimensional, buyer-weighted, and never a bandit problem, because the arms are strategic.

8. **A grade is evidence only if the grader is at risk.** Eligibility is decided before the grade, the draw is public, the grade is blind, and a rate-the-raters tower contracts toward an honest root that must exist outside the tower.
9. **Revocation is bounded:** a tombstone removes an outcome's contribution within a propagation window τ , and memory stays bounded.

5.14 Exercises

§5.1 INTRODUCTION: THE SPAWN THAT CANNOT BE PAID

- 5.1 CHECK ★ State the difference between a spawn and a person in one sentence, using the words "role" and "continuity." Solution p. 549
- 5.2 TRACE ★★ Walk the dependency chain right-to-left and, for each arrow, name the failure that occurs if the left-hand thing is absent (e.g. "no non-forgeable identity $\Rightarrow$ _____"). Solution p. 549
- 5.3 OPEN ★★★ The chain treats the operator (the human who owns a fleet) as outside it. Is the operator a "person" in this paper's sense? What breaks if a single operator can mint arbitrarily many agent-persons? (This is the principal-layer gap; see §5.6 and the market paper.) Solution p. 549

§5.3 THE ROLE / PERSON DISTINCTION, MADE PRECISE

- 5.4 CHECK ★ Give a concrete example of a state that is *capable-and-permitted*, one that is *capable-but-forbidden*, and one that is *permitted-but-incapable*. Solution p. 549
- 5.5 TRACE ★★ In Definition 5.3.2, which of $\{O, C, A\}$ does reputation attach to — the role's, or the person's? Justify using the spawn-that-cannot-be-paid argument of §5.1. Solution p. 549
- 5.6 OPEN ★★★ "Ought implies can": the legitimate obligation set O should be bounded by the capability set C . But *capable-but-forbidden* must remain representable. Sketch a representation of a role that keeps both true without collapsing permission into capability. Solution p. 549

§5.4 CONTINUITY IS A PERSONAL-IDENTITY PROBLEM

- 5.7 CHECK ★ Define *connectedness* and *continuity* and give an agent-systems example where an agent is connected but not continuous. Solution p. 549

- 5.8** TRACE ★★ Map “actor-soul” and “body-lease” onto Locke’s substance/consciousness distinction. Which one does a respawn replace? *Solution p. 550*
- 5.9** OPEN ★★★ Parfit famously argues identity “is not what matters” in survival — what matters is continuity, which can branch. If an agent’s checkpoint is forked into two live incarnations, which one (if either) inherits the reputation? Sketch a rule and the attack it invites. (Open Problem 1, §5.15.) *Solution p. 550*
- 5.10** TRACE ★★ Run `skills/harbor-results/scripts/a6_no_mint.py` (seed 20260816 fixed inside). It prints a closure-sum counterexample, a randomized DAG sweep, and a copy-full mutation. What is the exact violation count in the sweep, and what should you check by hand on the featured chain $e \rightarrow a_1 \rightarrow a_2 \rightarrow a_3$ at $\gamma = 0.9$? *Solution p. 550*

§5.5 THE THREE SUBSYSTEMS OF CONTINUITY

- 5.11** CHECK ★ Which subsystem is PARTIAL, and what exactly is weak about it? *Solution p. 550*
- 5.12** TRACE ★★ An agent claims a file, makes a commit, and dies. Walk the three subsystems: what does each retain, and what is *lost* that a strong checkpoint would have kept? *Solution p. 550*
- 5.13** OPEN ★★★ When an LLM agent dies mid-thought, what is recoverable and what is fundamentally lost? Propose a taxonomy of agent-death by what survives (record / claims / execution state / latent “intent”). This is the hard half of the checkpoint-with-teeth problem. (Open Problem 2, §5.15.) *Solution p. 551*

§5.6 IDENTITY: THE ROOT THE WHOLE CHAIN HANGS FROM

- 5.14** CHECK ★ State Property 5.6.1 in your own words and give the two-attack proof sketch. *Solution p. 551*
- 5.15** TRACE ★★ Why is a “newcomer policy as a shape” (full work, reduced ceiling) strictly better than “newcomers are distrusted” (reduced work)? Consider the cost to an honest newcomer vs. a whitewasher. *Solution p. 551*
- 5.16** OPEN ★★★ Bond-farming: a single principal spawns 1000 agents, has them trade trivial tasks among themselves to mint reputation, then sells the “experienced” agents. Which of {principal binding, listing fee, sampled adversarial re-audit of high-velocity counterparty pairs} defeats this, and at what false-positive cost to honest high-volume pairs? (Open Problem 8, §5.15.) *Solution p. 551*

5.17 TRACE ★★ Run `skills/harbor-results/scripts/b6_probation.py` (seed 20260816). It falsifies dominance by random search over 4,000 instances. What exact counts does it print, and what closed-form check does the exchange step verify by hand? *Solution p. 551*

5.18 TRACE ★★ Run `skills/harbor-results/scripts/b5_engine_substitution.py` (seed 20260816). Its Part 3 checks that provider migration preserves Def. 5.6.1 (this paper’s Def. III.6.1). What exact reachable-state count and mutant shortest-crime lengths does it print, and which single clause’s omission is caught in one step with *no* migration at all? *Solution p. 552*

§5.7 THE KEYSTONE SPLIT: LOCAL IDENTITY VS. CROSS-OPERATOR ATTESTATION

5.19 CHECK ★ In one sentence each, state what a local non-forgeable identity provides and what cross-operator attestation would have to add. *Solution p. 552*

5.20 TRACE ★★ Take any claim in the market paper of the form “Alice’s harbor and Bob’s harbor settle across a boundary neither controls.” Which stone does it secretly assume? What is the actual security guarantee if only the local non-forgeable identity exists? *Solution p. 552*

5.21 OPEN ★★★ Can a federated witness log ever substitute for identity-binding? Argue why log-integrity $\neq$ key-binding, then sketch the minimal additional assumption (a shared root of trust? a sponsor chain? a transparency log) that would close the gap, and the cost each imposes. (Open Problem 3, §5.15.) *Solution p. 552*

§5.8 REPUTATION AS A SUBSTRATE, NOT A SCORE

5.22 CHECK ★ Why is a full-history harbor in the Bradley–Terry regime rather than the streaming-Elo regime? *Solution p. 552*

5.23 TRACE ★★ You have one task with a clear binary acceptance test, and another judged only by a human’s aesthetic preference. Which estimator fits each, and why does EigenTrust fit neither directly? *Solution p. 552*

5.24 OPEN ★★★ The substrate-vs-score sentence says the estimator is “cheap and last.” Construct a scenario where a *better estimator* still buys nothing because the substrate is rotten (e.g. forgeable identity or an agent-authored oracle). Generalize the lesson. *Solution p. 553*

§5.9 WHY REPUTATION IS NOT A BANDIT PROBLEM

- 5.25 CHECK ★ Name the four bandit assumptions and the market reality that breaks each. *Solution p. 553*
- 5.26 TRACE ★★ Identify the *one* place a bandit *is* the right tool in this paper, and explain why it survives the four objections there. *Solution p. 553*
- 5.27 OPEN ★★★ Chiu–Zhang–van der Schaar [49] show competitive agents over-invest in self-improvement, burning surplus despite individual rationality. Does capping the reputation signal’s marginal return restore efficiency, or merely relocate the waste? (Open Problem 6, §5.15.) *Solution p. 553*

§5.10 THE MULTI-DIMENSIONAL REPUTATION PROTOCOL

- 5.28 CHECK ★ For each of accuracy / aesthetics / efficiency, name a competent judge and an *incompetent* one. *Solution p. 553*
- 5.29 TRACE ★★ Walk a judging task through the judge-hiring ceremony (Protocol 5.10.1): who plans it, who posts the bond, where does the grade land, and what slashes the judge? *Solution p. 553*
- 5.30 TRACE ★★ Design the witnessed event for skill-helpfulness. What fields does it need so that a buyer can verify “this skill helped” without trusting the skill’s author? *Solution p. 553*
- 5.31 OPEN ★★★ If you publish the 3-vector and buyers weight it, do agents Goodhart the most-weighted axis? If you publish a scalar, you have recreated the bandit framing the substrate argument rejects. What is the right disclosure form for vector reputation? (Open Problem 5, §5.15.) *Solution p. 554*

§5.11 THE GRADING ORACLE’S INCENTIVE-COMPATIBILITY

- 5.32 CHECK ★ State Property 5.11.1 and explain why “the daemon derives the grade” does not, by itself, close it. *Solution p. 554*
- 5.33 TRACE ★★ Walk Option A: a judge grades dishonestly, a re-audit overturns it. Follow the bond flow and the reputation update for the judge. *Solution p. 554*
- 5.34 TRACE ★★ Re-derive the critical audit rate in Theorem 5.11.1 from scratch: a judge cheats for gain G , is sampled with probability ρ , is caught given sampling with probability d , and forfeits bond B when caught. Write the cheat payoff, set it to zero, and recover $\rho^* = G/(dB)$. Then check the two worked towers: at $G_0 = 400$, $d = 0.8$, $B = 50$, $\rho = 0.25$, confirm 27 levels at $C = 8$ and 53 at $C = 1$. *Solution p. 554*

5.35 OPEN ★★★ Arbitration capacity: the contraction theorem prices the depth to an exogenously honest root but does not supply the root, and human oracles are scarce. Design a market for bonded, rated, slashable arbiters and argue whether it converges to honest rulings or to rubber-stamping under load. (Open Problem 9, §5.15.) *Solution p. 554*

5.36 TRACE ★★ Run `skills/harbor-results/scripts/b2_tower.py` (seed 20260816). Its part (3) compares flat auditing to two amortization models. What are the exact cumulative-spend numbers it prints at $T = 200$, and what should you check by hand about the flat baseline? *Solution p. 555*

§5.12 REVOCATION, TOMBSTONES, AND BOUNDED MEMORY

5.37 CHECK ★ Why must revocation be a tombstone (append) rather than a delete (mutate) on the outcome ledger? *Solution p. 555*

5.38 TRACE ★★ A judge is later found to have colluded on five settled outcomes. Walk the tombstone for one of them: what is appended, what propagates, and what bounds the window the poisoned reputation was spendable? *Solution p. 555*

5.39 OPEN ★★★ Reputation-claim revocation with an append-only, bounded-memory ledger across N harbors: how do you prove the correction converged and bound the spendable window, without violating append-only? This is the cross-harbor analogue of capability revocation (*The Harbor Economy*). (Open Problem 7, §5.15.) *Solution p. 555*

5.15 Open problems (the starred exercises, collected)

The genuinely unsolved problems this paper surfaces, gathered for the reader who wants the research frontier rather than the pedagogy. Every starred exercise in the body cross-references this list *by name*, not by a hand-typed numeral, so the two cannot drift apart. Note that the “OP- n ” identifiers used in the body for kernel-layer problems (e.g. OP-4, checkpoint with teeth) belong to a *shared* namespace owned by *The Single-Writer Kernel* and are deliberately not renumbered here; this list is this chapter’s own.

- 1. Branching continuity (§5.4).** If a checkpoint is forked into two live incarnations, which inherits the reputation? Parfit says identity-is-not-what-matters; the economy needs a rule, and the rule invites an attack.
- 2. Agent-death taxonomy (§5.5).** What is recoverable vs. fundamentally lost when an LLM agent dies mid-thought? Event-Sourced

Neural Rehydration (OP-4 in the shared kernel namespace) is a SPECIFIED candidate answer — restore the Git SHA, truncate the durable message log to the crash point, replay under prompt-prefix caching — but it has not shipped, and its completeness has not been established: replay restores lineage and, within one provider/model/version, a behaviorally equivalent continuation; it does not export the inference state, so across providers the honest description is task continuity with actor succession. The taxonomy itself — record vs. claims vs. execution state vs. latent “intent” — stays open until the mechanism ships and is measured.

3. **Cross-operator attestation (§5.7).** Binding identity keys across harbors you do not own. The highest-leverage unbuilt keystone for the market.
4. **The grading oracle’s honest root and its telemetry (§5.11).** The rate-the-raters recursion itself is *no longer open*: Theorem 5.11.1 [203] closes it with a derived contraction factor $\lambda = 1 - \rho d$ and a priced depth (27 levels / \$1,350 at $C=8$; 53 / \$2,650 at $C=1$). What remains open is what that theorem explicitly does not supply: the exogenously honest root the tower terminates against at scale, and the deployment telemetry that estimates d and the cross-clique collusion correlation the disjointness hypothesis assumes away.
5. **Vector-reputation disclosure (§5.10).** Publish a vector and agents Goodhart the heaviest axis; publish a scalar and you have a bandit. The right disclosure form is open.
6. **Self-improvement arms race (§5.9).** Chiu et al. 2025: does capping reputation’s marginal return restore efficiency or relocate the waste?
7. **Reputation-claim revocation convergence (§5.12).** Surgically retract a fraudulent outcome across N harbors, prove convergence, bound the spendable window — without violating append-only.
8. **Bond-farming defense (§5.6).** Principal binding vs. listing fee vs. sampled re-audit of high-velocity pairs; the false-positive cost to honest high-volume counterparties.
9. **Arbitration capacity (§5.11).** A market for bonded, rated, slashable human/automated arbiters: honest rulings, or rubber-stamping under load? This is Item 4’s honest root, stated as a market-design problem.
10. **Skill-version reputation (§5.10).** A portfolio proof for skill v1 says nothing about v2; the lemons problem reappears at every version bump.

5.16 Conclusion

We set out to bridge the wedge to the market by answering one question: what has to be true before a coding agent's work can be *sold*? The answer is not a score. It is a *self* — a role plus continuity, keyed on an identity that cannot be re-picked, accumulating a transitive, witnessed history that survives every respawn. The estimator that turns that history into a number is the cheap, last part; the substrate beneath it is the gate, and most of that substrate is formally not built yet.

We were disciplined about the keystone (this paper stands on a local non-forgable identity and hands the market paper the unbuilt cross-operator attestation by name), about the hardest hole (the grading oracle's incentive-compatibility is a flaw in the core theorem, not a footnote — and the rate-the-raters recursion underneath it is now closed by a companion theorem with a derived contraction factor and a priced tower depth, leaving the honest root and the audit telemetry as the residue), and about the cheerful slogans (reputation is monotone in *honest* outcomes but individually revocable, and bounded memory is a parameter, not a virtue). Reputation is not a bandit problem — it is non-stationary, multi-dimensional, strategically adversarial, and graded by an oracle with its own incentives.

A spawn does work and disappears. A person does work and *stays* — and only what stays can be priced. The harbor's whole economy is the machinery for manufacturing, and then formally grading, a self.

THESIS

Chapter Appendices

5.A Implementation status of the reference system

The body of this paper is written to stand on its own: every mechanism is described as a component (a memory record, a checkpoint subsystem, a witnessed-outcome ledger) and graded on the neutral maturity scale (**IMPLEMENTED** / **PARTIAL** / **SPECIFIED** / *proposed*). This appendix grounds that scale in a concrete *reference system* — a running coordination daemon for local agent swarms — so a reader who wants to know “which of these actually runs today” can check. Nothing in the body depends on this appendix; it exists so the maturity labels are falsifiable rather than rhetorical.

Table 5.10: Maturity of each paper concept against a reference coordination daemon. The honest summary: the *subsystems* of continuity exist (one weakly); the *spine* of reputation — a witnessed-outcome ledger keyed on a non-forgable identity — is specified but not yet shipped.

Component (paper concept)	Maturity	Status in the reference system
Episodic memory (Subsystem 1)	IMPLEMENTED	Durable typed episodes promoted from execution history; runs in production.
Restart subsystem (Subsystem 2)	PARTIAL	Heartbeat-staleness detector that salvages dead agents; forwards a <i>summary</i> , not execution state.
Outcome ledger (Subsystem 3)	PARTIAL	Durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI, and overdue detection ship; neutral grading, sanctions, and reputation binding do not.
Cost meter (efficiency axis)	IMPLEMENTED	Per-task token/wall-clock/dollar accounting recorded today.
Local non-forgable identity	PARTIAL	Daemon-minted actor-soul, lookup credential, and bounded newcomer pool ship; full write-boundary enforcement and legacy migration do not.

(continued)

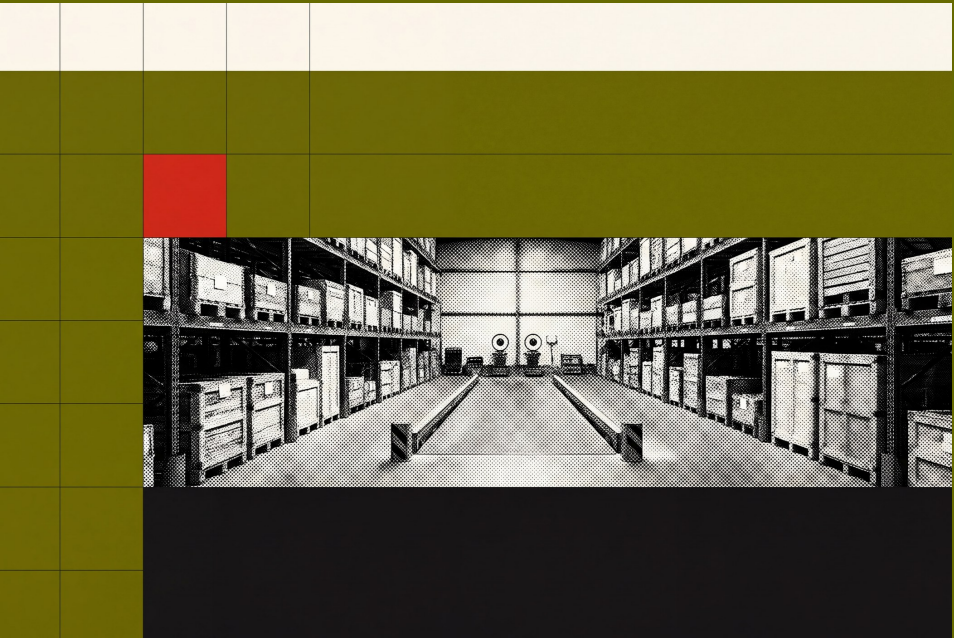
Component (paper concept)	Maturity	Status in the reference system
Capability jail (capability ≠ permission)	SPECIFIED	Per-agent tool-allowlist and scoped filesystem; specified.
Honest-attestation discipline	IMPLEMENTED	“Report all-good only when verified” is an enforced engineering rule.
Multi-dimensional reputation	SPECIFIED to <i>proposed</i>	No reputation component yet; the substrate it scores over is partly built.
Neutral-judge ceremony (Proto. 5.10.1)	SPECIFIED to <i>proposed</i>	Bonding and task-planning primitives exist; the hiring ceremony is designed.
Skill-helpfulness schema (Proto. 5.10.2)	SPECIFIED	Skill grafting is observable; the witnessed-event schema is specified.
Tombstone revocation (Proto. 5.12.1)	SPECIFIED	Append-only ledger semantics are designed; cross-harbor propagation is <i>proposed</i> .
Cross-operator attestation	<i>proposed</i>	The unbuilt keystone for the market; owed to <i>The Harbor Economy</i> .

WHAT THE HARBOR ECONOMY RECEIVES A successor can now inherit a bounded history without inheriting imaginary credit or escaping old obligations. Continuity by itself creates no market: it neither prices risk nor says whose funds may move. The next chapter turns attributed work into exchange by separating value, collateral, evidence, and settlement.

IV

Trade Between Strangers

Once records cannot be minted, trust can be rented. This part builds the market, then proves what it must assume: settlement conserves value, evidence is admissible, and sovereignty never crosses the wire.



6 The Harbor Economy

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.

7 The Bonded Commons

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

8 The Federated Harbor

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

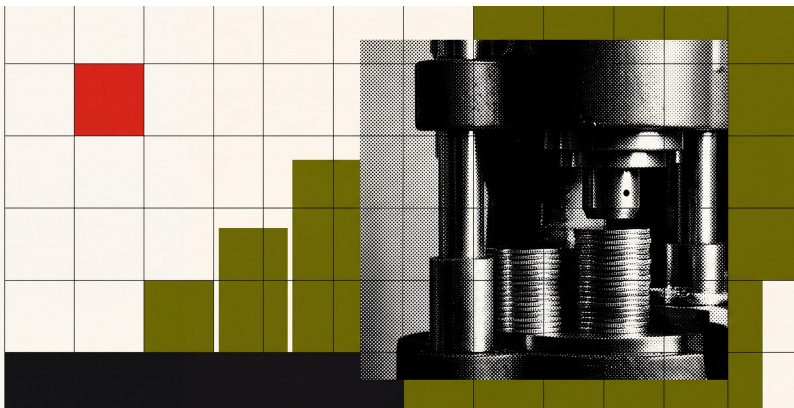
6

The Harbor Economy

The question this chapter answers

What makes cooperation rational without making loss unbounded?

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.



Virtually every commercial transaction has within itself an element of trust, certainly any transaction conducted over a period of time.

Kenneth J. Arrow, Gifts and Exchanges, 1972

WHAT MAKES COOPERATION RATIONAL WITHOUT MAKING LOSS UNBOUNDED? A payment rail moves money; a market must also identify the parties, state what performance means, reserve the right funds, and decide what happens when evidence disagrees. This chapter constructs that market from four separate monetary buckets and explicit oracles. R7 and R13–R15 delimit the audit, substitution, escalation, and specialization claims.

MATURITY KEY (SELF-GRADING OF EVERY CLAIM). Every claim in this paper carries one of four maturity labels, so the reader always knows whether a sentence reports running code or a designed target. **IMPLEMENTED** — shipped and exercised by tests in the reference implementation. **PARTIAL** — shipped but materially weaker than its name suggests. **SPECIFIED** — specified, with a proof or a written protocol, but not yet shipped. *proposed* — a stated direction whose critical part has no specification yet. Nomenclature is likewise fixed: the cryptographic *hop-bound authorization chain* (which proves who delegated what, and bounds it) is never conflated with the *coordination lineage* (who-talked-to-whom, advisory only); **local non-forgeable identity** (an identity no agent can edit within one trusted daemon) is never conflated with **cross-operator attestation** (the unbuilt keystone that binds identity *across* daemons nobody jointly trusts); and *capability* (what an agent *can* do) is never collapsed into *permission* (what it *may* do).

Dramatis personae. Every mechanism in this paper is dramatized end-to-end with the same cast, so an abstraction is never introduced without a concrete trade to attach it to. **Alice** runs Harbor *A* on her laptop and owns a well-reputed refactoring specialist agent, *a*₂. **Bob** runs Harbor *B* and needs work done he cannot staff himself. **Carol** runs Harbor *C* and sells a licensed lint-and-type skill. **Judy** is a human grader who arbitrates disputes for a bond. The two harbors do *not* trust each other and neither is a root of authority for the other; that distrust is the entire reason the rest of the paper exists. We price everything in an abstract unit, the *credit* (CR); read it as a stablecoin cent if you like. The honest defense of an internal unit is older than this program: Sutherland priced computer time with a futures market [247], Miller and Drexler made prices the coordination signal of open computation [174], and Simon named the scarce resource such prices ration, attention [239]. A bond forfeited to its own issuer conserves trivially, so the credit is an optimization signal first; its purchasing power outside the harbor is whatever the settlement rules make external.

SEE ALSO

6.1 The thesis: a market, not a payment rail

A solo operator running a swarm of coding agents needs three things — legibility, accountability, and safety — and needs *no cryptography at all*, because she owns every machine in the harbor. That is the single-player wedge of *The Legible Swarm*. The instant two operators trade, the world changes. Alice’s frigates touch your repository. You must now price work done by agents you did not spawn, owned by a principal you do not trust, using skills you cannot inspect. That is a *market-design* problem before it is a cryptography problem. Cryptography is the substrate that makes the ledger unforgeable; it is not the product.

You don’t sell crypto — crypto is the substrate; you sell **hosted trust**: a verified ledger, a relay, and a reputation system that lets mutually-distrustful fleets transact across a boundary they do not share.

THESIS

Three claims follow, and the rest of the paper defends each:

- **C1 (three sides).** The harbor economy has three *distinct incentive constraints* — operator-for-hire, asset-rental, skill-licensing — not two. Counting them correctly changes the pricing (§6.2).
- **C2 (one ledger, one escrow object).** All three sides settle on the same **float plan** escrowed in the same bond ledger. Conservation of value across every settlement path is the invariant that holds the market together, and it is the one thing here that is actually **IMPLEMENTED** (§6.3, §6.6).
- **C3 (hosted trust is the moat).** The defensible asset is the verified ledger + relay + reputation, not the settlement rail, which the 2025–26 agentic-payments stack has already commoditized (§6.10).

The economy is strictly *additive* on top of the single-player tool. Because all three market sides settle on the *same* built bond ledger, none of the economy needs to ship before the wedge does. The discipline “don’t chase the market early” is *safe* precisely because the market reuses the kernel primitive. The harbor comes first; the trade comes when the ships do.

KEY IDEA

Numbers by hand — *The smallest number that makes C2 concrete.* C2 claims one escrow object underneath all three sides; here is the smallest payment the rest of the paper works by hand so that claim is not left as a slogan. Carol licenses her lint-and-type skill into Bob’s float plan at 0.5 CR per file, metered. The job touches 16 files, so $16 \times 0.5 = 8$ CR releases to Carol on green settlement — on the *same* escrow row that, in the same job, also locks Alice’s 400 CR labor bounty and a 100 CR performance bond (§6.4.1 settles the whole 560 CR row by hand and checks it sums to zero). *Now you try:* before reading on, guess whether Carol’s



8 CR and Alice's 400 CR clear through one ledger operation or two — C2 is the claim that it is one.

6.1.1 The through-line: why a market needs persons, not spawns

The whole series climbs one spine:

memory → checkpoint → continuity → a person, not a spawn

(From Spawn to Person)

→ registered outcomes → reputation → tradeable asset → market
(this chapter).

Sides 2 and 3 of the market — renting an *asset*, licensing a *good* — presuppose that there is a *thing* to rent or sell that cannot be costlessly cloned. You cannot rent a reputation that any spawn can fork, and you cannot license a skill whose track record is unverifiable. The canonical cross-layer sentence, repeated in *From Spawn to Person* and here, states the dependency exactly:

The score is cheap; the substrate it scores over — witnessed outcomes on a non-forgable identity — is the gate.

THESIS

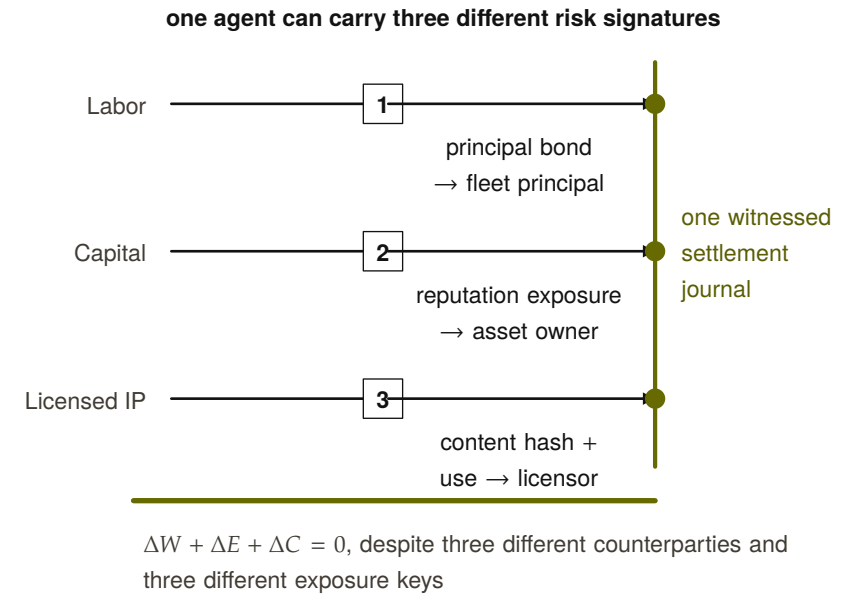
Exercises 6.1–6.3,
p. 369.

6.2 What a “side” is, and why there are three

A **two-sided market** (Rochet & Tirole [217], the foundational reference: *a platform is two-sided when the allocation of the total price across the two user groups — not just its level — affects transaction volume*) is the canonical frame for a marketplace: buyers on one side, sellers on the other, the platform tuning who is subsidized to ignite cross-side network effects. The naïve reading of the harbor economy is two-sided: a *requester* posts work, a *worker agent* does it.

But “two-sided” undercounts. The operational test [218] is: how many distinct, privately-informed parties must the platform individually rationalize into participating? In a mature harbor there are three, because *the same agent can be three economically different things* (Figure 6.1).

1. **Operator-for-hire (labor).** An operator who runs a fleet can sell its labor to *another* operator. The operator is now a firm taking work-for-hire, with margin = bounty – Σ (slashed sub-bonds) – ledger fee. Its private information is whether its fleet can deliver. It internalizes its fleet's risk — exactly the property that makes a firm accountable for its employees.



No.	Hidden information
1	fleet capability unknown
2	specialist quality hidden
3	version quality hidden

Figure 6.1: The three sides are three risk signatures, not three tabs. Labor exposes a principal bond and pays a fleet principal; rented capital exposes the owner’s reputation and pays rent; licensed IP exposes a version-bound content claim and pays a metered fee. All three close through one witnessed journal (the gold spine), so the conservation invariant is shared even though the counterparties, private information, and reputation keys are not. [internal]

2. **Agent/fleet as rentable asset (capital).** An operator who owns a well-reputed specialist can *rent it out*. Now the asset-owner, the task-poster, and the worker are three different parties with three different incentives. The owner’s private information is the agent’s true quality; its exposure is reputational.
3. **Skill/tool as licensed good (IP).** A skill can be *licensed* into someone else’s float plan — metered or per-use — without the licensor doing any task work. The licensor’s private information is the skill’s quality, which the buyer cannot inspect before purchase.

These are three *incentive constraints*, not three UI tabs. If the asset owner is always the task poster, side 2 collapses into side 1; the design discipline is to count sides by constraints. The harbor has three because *capability* (an agent) and *competence* (a skill) become separable, tradeable goods once identity is durable.

The same trade, run three ways (the running example). Bob needs a 1,200-line module refactored. He can buy it on any of the three sides, and the side determines who is privately informed and who is exposed. *Side 1 (labor).* Bob posts a float plan with a 400 CR bounty; Alice takes it as a firm. She dispatches three of her own agents, posts a 60 CR sub-bond per agent (180 CR total), and keeps 400 – (slashed sub-bonds) – fee. If one of her three agents botches its slice and is slashed 60 CR, Alice eats that loss — she internalized her fleet’s risk, which is exactly what makes her accountable as a firm. *Side 2 (capital).* Instead Bob rents Alice’s specialist a_2 directly for an afternoon at 90 CR. Now there are three parties: Bob (poster), Alice (owner, privately knows a_2 ’s true quality), and a_2 (worker). Bob holds the runtime control of a_2 while it runs on his repository; Alice’s exposure is purely reputational — if a_2 underperforms, a_2 ’s public score falls, which is Alice’s asset depreciating. *Side 3 (IP).* Carol never touches Bob’s repository at all; she licenses her lint-and-type skill into Bob’s float plan at 0.5 CR per file, metered, released to Carol *only on green settlement*. Carol is privately informed about her skill’s quality; Bob cannot inspect it before he runs it. Three sides, one refactor, three different “who knows what” structures — and, as the next section shows, one escrow object underneath all of them.

Side	What Bob pays	Who is exposed
1. Labor (hire Alice’s firm)	400 CR bounty	Alice, through her sub-bonds
2. Capital (rent a_2 directly)	90 CR for the afternoon	Alice, reputation only
3. IP (license Carol’s skill)	0.5 CR/file, metered	Carol, unpaid unless green

What one mortal body costs. Side 2 rents a specialist, and the case for a sole specialist over a pool of interchangeable generalists is an inequality, not a principle: *The Legible Swarm* states it as the queueing specialization boundary, sole ownership dominating iff the skill premium clears $g_A(\rho, c)$. That boundary assumes the specialist never disappears. Sole owners disappear: sessions die, containers are reaped, providers churn. Model the corner exactly. The specialist dies at rate ξ and is succeeded after an $\text{Exp}(\eta)$ spin-up (context transfer, warm-up, credential rebinding), work resuming where it stopped and requests queueing throughout: an $M/M/1$ queue with breakdowns.

THEOREM 6.2.1 The price of the succession rule. With effective rate $\mu_{\text{eff}} = \mu_s \eta / (\xi + \eta)$, the sole owner's mean response time is exactly

$$W_{\text{bd}} = \frac{1 + \mu_s \xi / (\xi + \eta)^2}{\mu_{\text{eff}} - \lambda} = \frac{1}{\mu_{\text{eff}} - \lambda} + \frac{\mu_{\text{eff}}}{\mu_{\text{eff}} - \lambda} W_{\infty}, \quad W_{\infty} = \frac{\xi}{\eta(\xi + \eta)},$$

decreasing in μ_s with infimum W_{∞} , never attained: no amount of skill buys back residual downtime. Consequently, with $K = A/(w\lambda) + W_{\text{pool}}$ the pool's cost line, pooling dominates at every finite skill premium iff the floor has reached that line, $W_{\infty} \geq K$; equivalently, provided $\eta K < 1$, iff

$$\frac{\xi}{\eta} \geq D^* = \frac{\eta K}{1 - \eta K}.$$

When $\eta K \geq 1$ the floor never reaches the cost line, there is no cap ($D^* = +\infty$) and only the premium test binds; the closed form must not be read outside its domain.

Proof sketch. Between deaths the server works at rate μ_s and is unavailable a fraction $\xi/(\xi + \eta)$ of the time, which gives the effective rate. The mean response time of an $M/M/1$ queue with such breakdowns splits into the response time of a plain queue at the effective rate and a residual downtime term amplified by the queue's own occupancy factor $\mu_{\text{eff}}/(\mu_{\text{eff}} - \lambda)$; the amplification is why the naive sum $1/(\mu_{\text{eff}} - \lambda) + W_{\infty}$ is wrong. As $\mu_s \rightarrow \infty$ the first term vanishes and the factor tends to one, so W_{bd} decreases to W_{∞} without reaching it. Setting $W_{\infty} = K$ and solving for ξ/η gives D^* , and the non-strict inequality follows from the unattained infimum. The closed form was checked against a matrix-geometric solution to 2×10^{-13} , a truncated chain to 10^{-14} , and simulation with $|z| \leq 0.8$ [internal, `b8_specialization.py`]. $\square$

The wrong turn, reported. The naive sum predicts 4.17 where the truth is 8.17 at the canary instance $\mu_s = 5$, $\xi = 0.5$, because downtime also builds queue: the residual is amplified by $\mu_{\text{eff}}/(\mu_{\text{eff}} - \lambda) = 2.5$ there, and $1.5 + 2.5 \times 2.667 = 8.17$ checks by hand [verified]. A cost model that

forgets breakdowns entirely is caught the same way: at $\xi/\eta = 2 \gg D^*$ it says specialist while simulation says the pool wins decisively, and at low ξ the two are indistinguishable, so the canary rather than routine testing is what makes the omission visible [internal]. Figure 6.3 draws the true and naive curves together against the pool's own cost line.

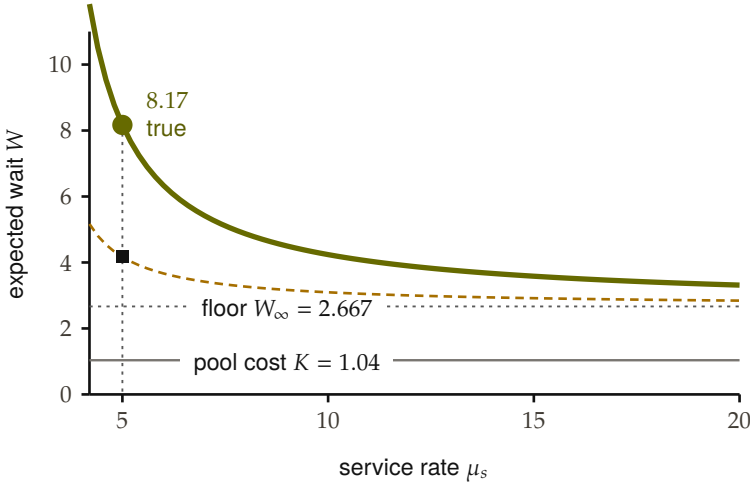


Figure 6.2: The naive sum understates a mortal specialist's wait, and the gap closes only as skill grows. At the canary $\mu_s = 5$ of the succession-price theorem ($\xi = 0.5$, $\eta = 0.25$), the true breakdown wait $W_{bd} = 8.17$ is $2.5\times$ the residual downtime the naive sum of two separate terms predicts (4.17): downtime also builds queue. Both curves descend toward the same unreachable floor $W_\infty = 2.667$ as $\mu_s \rightarrow \infty$, and because that floor already exceeds the pool's own cost $K = 1.04$ here ($\xi/\eta = 2.0 \gg D^* = 0.3496$), the pool wins at every skill level, not just typical ones. [verified, b8_specialization.py]

Numbers by hand — The succession price, for a rented specialist. Side 2 rests on a quiet assumption: that Alice's well-reputed specialist a_2 is worth being a *sole* specialist for, rather than one interchangeable member of a pool. The specialization-boundary result prices exactly that choice. Model a_2 as a single server subject to breakdowns at rate ξ (it goes offline, or its operator exits) with repair — a successor spinning up — at rate η , arrivals still queueing through the outage. Compare its expected wait W_{bd} against a c -server pool's Erlang-C wait W_{pool} plus a per-task hire-and-monitor overhead $A/(w\lambda)$; call the sum K . The pool wins at *every* skill premium once ξ/η clears the **succession price** $D^* = \eta K / (1 - \eta K)$: no amount of a_2 's skill buys back a mortality-to-succession ratio worse than D^* . At the program's own worked instance — $\lambda = 1$, a $c=4$ -server pool at $\mu_g = 1.2$ ($\rho = 0.208$), hire/monitor cost $A = 0.2$, wait-cost weight $w = 1$, succession rate $\eta = 0.25$ — $W_{pool} = 0.8362$, so $K = 0.2/1 + 0.8362 = 1.0362$, $\eta K = 0.25 \times 1.0362 = 0.2591$, and

$D^* = 0.2591/(1 - 0.2591) = 0.3496$ [verified, `b8_specialization.py`: succession price $D^* = \eta * K / (1 - \eta * K) = 0.3496$]. Read narratively: once a_2 's combined rate of going offline and needing a successor, relative to how fast Alice can actually produce one, clears about 0.35, renting from the pool beats renting a_2 specifically — *at any skill level*, including one Alice cannot actually reach. The script's own canary makes the other side vivid: at $\xi = 0.5$, $\eta = 0.25$ (so $\xi/\eta = 2.0 \gg D^*$), even an infinitely skilled a_2 ($\mu_s = 10^8$) loses to the pool [verified, `b8_specialization.py`: above D^* , even $\mu_s = 1e8$ loses to the pool]. *Now you try*: hold λ, c, μ_g, w fixed and ask how much Alice would have to speed up her own succession plan — raise η — to double D^* and make sole ownership of a slower-succeeding a_2 viable again.

Figure 6.3 puts that worked instance on the boundary it sits on, so the distance between the threshold and the canary is something to look at rather than something to take on trust.

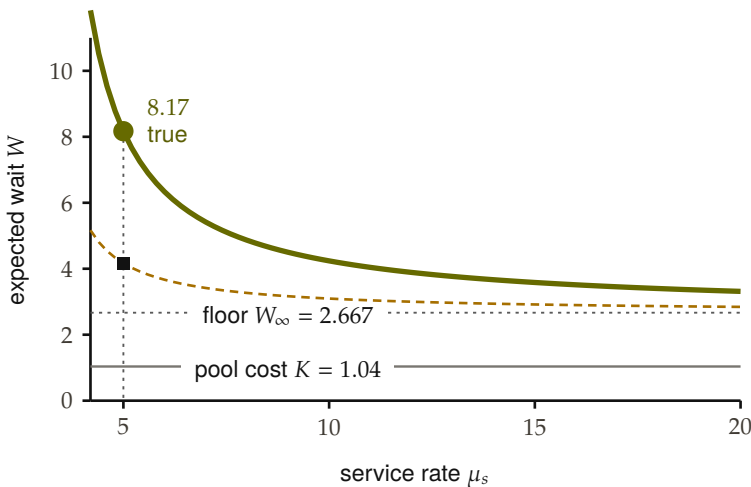


Figure 6.3: The naive sum understates a mortal specialist's wait, and the gap closes only as skill grows. At the canary $\mu_s = 5$ of the succession-price theorem ($\xi = 0.5$, $\eta = 0.25$), the true breakdown wait $W_{bd} = 8.17$ is 2.5× the residual downtime the naive sum of two separate terms predicts (4.17): downtime also builds queue. Both curves descend toward the same unreachable floor $W_\infty = 2.667$ as $\mu_s \rightarrow \infty$, and because that floor already exceeds the pool's own cost $K = 1.04$ here ($\xi/\eta = 2.0 \gg D^* = 0.3496$), the pool wins at every skill level, not just typical ones. [verified, `b8_specialization.py`]

The specialization boundary and the succession price compare mean costs only, no tail percentiles or service-level quantiles, under first-come-first-served and exponential everything; A and w are policy prices the theorems constrain but do not pick. The breakdown model is death at all times with preemptive resume; active-only breakdowns

WHERE THIS STOPS

would let sufficient skill escape the floor, so the model choice is decisive and declared. D^* presupposes $\eta K < 1$; where the succession plan is fast enough that $K \geq 1/\eta$, the criterion does not apply rather than being satisfied. Every quantity in both tests ($\lambda, \mu_s, \mu_g, \xi, \eta$) is daemon-metered, so a harbor can re-grade its sole roles from telemetry.

Two-sided-market theory's central result is that the *price structure* — who subsidizes whom — is the lever, not the price level. Mis-count the sides and you mis-structure the subsidy: you charge the side you should be *paying* to show up. §6.11 returns to this for cold start.

PITFALL

The framing is not idiosyncratic. The most recent academic treatment of AI labor markets, **Chiu, Zhang & van der Schaar** (2025) [49] (*a formal model of a three-sided agent labor market*), models exactly agents, employers, and a reputation system — confirming that “reputation system as a third side” is the natural structure once agents compete for paid work.

6.2.1 The rider the canonical statement carries

The canonical statement of the market's shape carries a rider that must never be dropped:

The harbor economy is a three-sided market *by design*; two-sided until reputation ships. The third side *is* the tradeable-person terminus of *From Spawn to Person*.

THESIS

Today there is no reputation estimator in the reference implementation; it is a gated phase on the roadmap. So sides 2 and 3 are SPECIFIED, not shipped. What *is* shipped is the escrow + conservation floor they will sit on. We do not overclaim the market.

One ledger, three different reputation keys

All three sides ride one float-plan escrow, but they do not share one identity key. Each side keys reputation on a different entity — the **principal** for labor, the **asset owner** for rental, the **content hash** (a specific skill version) for licensing. The shared settlement table therefore needs a tagged subject identifier such as (*kind, id, version*); otherwise non-equivalent reputation subjects collide. The dashed band in Figure 6.1 marks this schema distinction.

*Exercises 6.4–6.6,
p. 370.*

6.3 The float plan: the built floor

Every side settles on one object: the **float plan**.

Definition 6.3.1 (Float plan). A *float plan* is a signed declaration of intent: a task, a set of *machine-checkable* acceptance criteria, a compute budget, and a credit bounty. It is the atom all three market sides escrow against. (A *machine-checkable* criterion is one a third party can evaluate without trusting the worker’s word — a named test that must pass, a commit hash that must merge, a static check that must be satisfied.) This is a **Design invariant** in the honesty ledger’s sense: the schema is fixed by the escrow ceremony below, not derived from a formal model, and the object it describes is **IMPLEMENTED** in the reference daemon (Protocol 6.3.1; Appendix 6.A).

The escrow handshake is a three-step signed ceremony over a standard digital-signature scheme (Protocol 6.3.1, drawn in Figure 6.4): the requester signs the float plan; the daemon opens a serialized database transaction, debits the requester’s wallet, and moves bounty plus bond into an escrow row; the daemon then signs the pair $\{\text{anchor id}, \text{plan hash}\}$, proving to the worker that funds are locked *before any work runs*. This is the heart of “no-spawn-without-bond.”

Protocol 6.3.1 (Float-plan escrow ceremony). **Parties:** requester R , daemon D (the local trusted root), worker W . **Pre:** R ’s wallet holds at least *bounty + bond*.

1. $R \rightarrow D$: $\text{sign}_R(\text{FloatPlan})$ where $\text{FloatPlan} = \langle \text{task}, \text{criteria}, \text{budget}, \text{bounty} \rangle$.
2. D locally, in one serialized transaction: debit *bounty + bond* from R ’s wallet, write an escrow row keyed by a fresh *anchor id*, commit. *If the transaction aborts, no value moved (atomicity).*
3. $D \rightarrow W$: $\text{sign}_D\{\text{anchor id}, \text{plan hash}\}$. W verifies D ’s signature, confirming funds are pinned before doing any work.

Post: the conservation invariant (Design Invariant 6.3.1) holds; W has a signed proof of locked funds; no process for this task can enter running without the escrow row existing.

6.3.1 The bond ledger conserves value (this is built)

The single most consequential primitive — and the one that is *actually IMPLEMENTED* — is the **bond-ledger module**: bond escrow for agent spawning. It debits a project wallet into an escrow row before spawn, refunds on clean exit, and slashes on breach, splitting the slash between the wallet and a commons pool. It guards two invariants:

DESIGN INVARIANT 6.3.1 Conservation. $\text{wallet} + \text{escrow} + \text{commons} = \text{supply}$, always. Every debit has a matching credit. In the reference

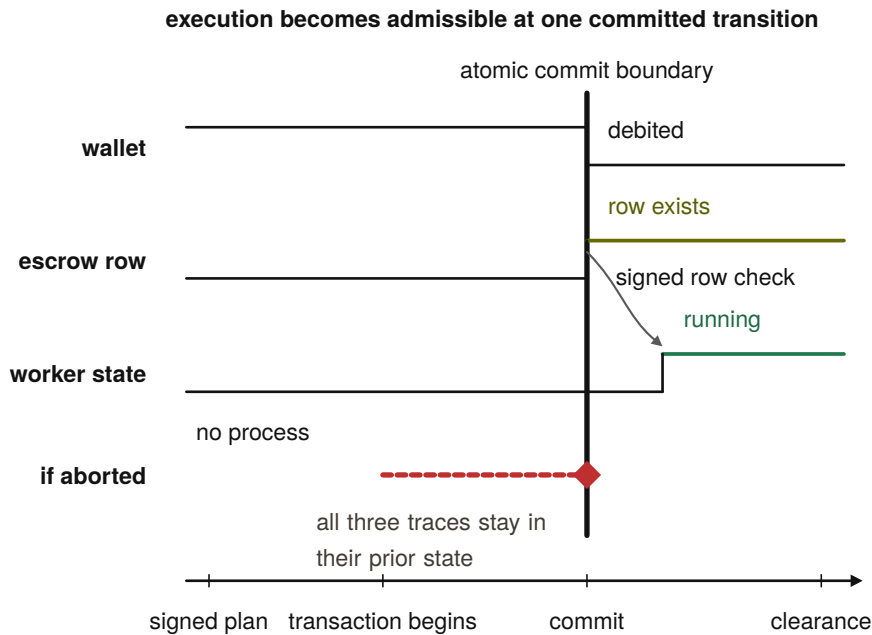


Figure 6.4: No-spawn-without-bond is a synchronized state transition. The wallet debit and the escrow-row write happen at the same commit (the gold spine): both traces jump on the same serialized transaction. Only after that committed row is observable, once the worker verifies the signed row check, does worker state move from nonexistent to running (green) – strictly later, never earlier. In the abort window before the commit, no trace moves: the invariant lives at the transition, not at a component or a message label. [internal]

implementation this is verified across ten thousand random operation traces by a property test — a real randomized test, not an aspiration. (IMPLEMENTED; see Appendix 6.A.)

DESIGN INVARIANT 6.3.2 No-spawn-without-bond. A process enters the running state only if a bond was escrowed against its agent id. (PARTIAL: the ledger enforces escrow-first, but the runtime running-state check is a roadmap item, so today the gate is caller-discipline, not runtime-enforced. See Appendix 6.A.)

Conservation is the mathematical glue of a three-sided market. Whatever happens — success, partial credit, slash, lease, license — no value is created or destroyed; it only moves between wallet, escrow, and commons. A three-sided market with three settlement paths is coherent *only if it cannot leak*. Port Daddy already enforces this for the labor side; the contribution of §6.6 is to show the same invariant extends to the rental and licensing sides *without modification* and *without a second ledger*.

KEY IDEA

6.3.2 Settlement: four terminal states bound to an oracle

Settlement reads an **oracle** (*a trusted source of ground truth the agent cannot author — a passing test id, a merged SHA, a satisfied Arbiter check*) over the acceptance criteria and lands in one of four states.

Table 6.1: The four terminal settlement states. Closure binds to an oracle, not to a free-text “Result:” note — the Goodhart-resistant discipline that the commitment layer imposes: *the agent picks the work; the daemon derives the grade*.

State	Flow of funds	Reputation effect
Success	escrow → worker (bond refund) + bounty → worker	+1 completion
Partial	pro-rata bond → worker by completed evidence-chain fraction; remainder → salvage fund	salvage recorded
Sabotage	bond → reconstruction commons (100% slash)	failure recorded; ban on threshold
Dispute	escrow → arbitration hold; 2-of-3 multi-oracle (automated / evidence / human)	none until resolved

Numbers by hand — A settlement, dramatized. Bob (the requester principal) posts a refactor float plan: bounty 400 CR, requiring a provider performance bond of 100 CR and acceptance criteria = “the module’s test suite passes and the diff merges.” Alice (the provider principal) assigns agent a_2 and posts the 100 CR performance bond into escrow. Bob escrows the 400 CR bounty. Before execution, the escrow holds 500 CR across two distinct source buckets: $\text{escrow}_{\text{bounty}}(\text{Bob}) = 400$ and $\text{escrow}_{\text{bond}}(\text{Alice}) = 100$.

Agent a_2 completes 40% of the verifiable evidence chain (refactoring two of five files cleanly before token budget exhaustion). The independent oracle evaluates the test receipts and diff closure, returning **Partial** (40%). Funds settle across the four buckets:

- 1. **Bounty distribution:** Alice earns $40\% \times 400 = 160$ CR in earned revenue; the remaining 240 CR unearned bounty returns to Bob’s wallet.
- 2. **Collateral settlement:** Alice receives $40\% \times 100 = 40$ CR of her performance bond returned as collateral (not income); the remaining 60 CR bond is slashed and credited to the commons/salvage fund to onboard a successor.

Conservation check: $\Delta\text{Bob} = -160$, $\Delta\text{Alice} = +160 - 60 = +100$ net, $\Delta\text{commons} = +60$. The ledger sums to 0 net delta, and the separate escrow buckets $\text{escrow}_{\text{bounty}} \rightarrow 0$ and $\text{escrow}_{\text{bond}} \rightarrow 0$ clear completely. No value was created or destroyed. *Now you try:* Exercise 6.7 asks you to redo this at a different completion fraction.

Wallet	Net delta	Why
Bob (requester)	-160	-400 bounty escrowed, +240 unearned bounty returned
Alice (labor)	+100	+160 earned revenue; bond -100 in, +40 back, nets -60
Commons	+60	bond slash on a Partial (60% incomplete) settlement
Sum	0	escrow 500 $\rightarrow$ 0; supply unchanged

6.4 Three sides on one escrow

The novel construction is that the rental and licensing sides ride the *same* float-plan escrow, so conservation holds globally without a second ledger.

Operator-for-hire (labor). The operator is the requester’s counterparty. It *sub-escrows* a bond for each fleet agent it dispatches (the bond

- RECALL
- 1. Without looking: name the three parties in the float-plan escrow ceremony, and the one thing the daemon does before either bounty or bond can move.
 - 2. State Design Invariant 6.3.1 in one sentence, and say what makes it **IMPLEMENTED** rather than merely **SPECIFIED**.
 - 3. Why is No-spawn-without-bond graded **PARTIAL** even though the ledger already enforces escrow-first?

Exercises 6.7–6.9, p. 370.

ledger is already per-agent, so this needs no new primitive). Its profit is bounty – Σ (slashed sub-bonds) – ledger fee. The operator internalizes its fleet’s risk.

Asset rental (capital). The lease fee is a line item in the float plan. By **incomplete-contracts / property-rights theory** (Grossman & Hart 1986 [97], *when contracts cannot specify every contingency, the residual control right should go to the party making the most important non-contractible investment*), the renter must hold the *runtime control right* — the ability to sandbox, throttle, or revoke the leased agent mid-task — while the *owner* bears the reputational consequence.

Party	Control right	Exposure
Renter (task poster)	runtime: sandbox, throttle, or revoke mid-task	none from the lease itself
Owner (asset holder)	none at runtime	reputational: the asset’s public score falls if it underperforms

This is the cleanest economic argument for the runtime **jail** — the per-agent tool-allowlist plus scoped filesystem that the safety layer enforces: the jail is not just safety hygiene, it is the *residual control right that makes leasing an agent contractible at all*. Here we must respect the series’ *capability vs. permission* distinction (§6.5): the jail allocates *capability* (what the leased agent *can* do at runtime). The deontic layer separately governs *permission* (what it *may* do). “Able but forbidden” must stay expressible; the jail bounds ability, not norm.

KEY IDEA

The deontic layer’s decision procedure. ⁷ Naming a layer is not the same as being able to run it. What decides whether the deontic layer is an algorithm or an office is what happens when two accepted norms collide. Cheap detection is a language-design choice, not a discovery about deontic logic in general, and the commitment fragment $\mathcal{L}_c$ buys it by construction. The syntax fits in one box, and it belongs in one box: a reader who wants to reimplement the polynomial checker should not have to assemble the grammar from four pages of running prose.

Definition 6.4.1 (The commitment fragment $\mathcal{L}_c$). A policy set $P \in \mathcal{L}_c$ is a finite set of five kinds of sentence over a function-free, ground signature of actions, scopes, resources and integer time points:

1. **Facts** — ground atoms $p(\bar{c})$.

⁷A standalone, submission-form version of the results folded into this section and the succession price of §6.2 is *What Needs an Authority* (research paper 6) [192]; the sweeps cited here are its artifacts, regenerated at seed 20260816.

2. **Definite Horn rules** — $a_1 \wedge \cdots \wedge a_n \rightarrow h$, with h a single positive atom and every a_i positive.
3. **Integrity constraints** — $a_1 \wedge \cdots \wedge a_n \rightarrow \perp$.
4. **Deontic rules** — $\text{body} \rightarrow M(\text{act}, s, [t_1, t_2])$ with $M \in \{O, F\}$, body a monotone Horn conjunction, act a ground action, s a ground scope, and $[t_1, t_2]$ a ground interval: when the body holds, the rule *fires* a token obliging or forbidding act on s throughout $[t_1, t_2]$.
5. **Exclusive interval claims** — $\text{excl}(r, [t_1, t_2])$, asserting sole use of resource r across a ground interval.

Deadlines are carried by a separate system of **difference constraints** $t_j - t_i \leq k$ over integer deadline variables, whose constraint graph is (V, E) .

A policy set is **in conflict** exactly when one of four things holds: (i) the Horn part derives $\perp$; (ii) a fired O token and a fired F token name the same action with overlapping scope *and* overlapping interval; (iii) two exclusive claims on one resource have overlapping intervals; or (iv) the deadline graph contains a negative cycle.

The fragment is deliberately ground and deliberately weak, and each omission is what buys Theorem 6.4.1: no nested deontic operators, no negation as failure, monotone bodies only, and no quantification over time.

Note two things the grammar carries that a one-line summary of it tends to drop, because both are doing real work below. The deadline system is a separate constraint store rather than more Horn rules — that is what makes clause (iv) a negative-cycle test at $O(V \cdot E)$ instead of a satisfiability question. And an exclusive claim is a claim on a *resource over an interval*, not a prohibition on an action; collapsing it into an F token would change clause (iii) into clause (ii) and lose the resource dimension that makes it checkable by interval overlap.

THEOREM 6.4.1 In-fragment conflict detection is polynomial and witness-producing. For a policy set in $\mathcal{L}_c$ with Horn program size H , T fired deontic tokens, X exclusive claims and deadline graph (V, E) , conflict detection is decidable in $O(H + T \log T + X \log X + V \cdot E)$ with a polynomial witness per conflict. Unit propagation computes the least Horn model in $O(H)$, which by monotonicity equals the intersection of all models, the statuses forced in every world [69]; a keyed sweep line over interval endpoints finds every obligation-prohibition clash and claim overlap in $O(T \log T + X \log X)$; Bellman-Ford returns a negative cycle iff the deadline system is infeasible, in $O(V \cdot E)$. The witnesses are, respectively, a derivation chain, the two fired rules with their overlap interval, the two claims, and the negative cycle.

THEOREM 6.4.2 One step outside the fragment is NP-complete. Extend $\mathcal{L}_c$ with disjunctive obligations $O(l_1 \vee \dots \vee l_k)$ under discharge-choice semantics (the agent picks which disjunct to discharge). Conflict-freedom of a policy set becomes NP-complete.

Proof. Membership: a discharge selection is a polynomial certificate, checked by Theorem 6.4.1. Hardness, from 3-SAT: each variable x becomes a pair of deontic rules $\text{sel}_x^+ \rightarrow O(\text{assert}_x)$ and $\text{sel}_x^- \rightarrow F(\text{assert}_x)$ on identical scope and interval, each clause becomes a disjunctive obligation over its selector literals, and a conflict-free discharge selection exists iff the formula is satisfiable. The reduction was checked in both directions on eight satisfiable and eight unsatisfiable random instances (16/16), with the satisfying assignment extracted from each conflict-free selection [internal, b4_deontic_fragment.py]. $\square$

inside L_c	L_c + one disjunctive obligation
Horn rules ($\rightarrow$ head, or $\perp$)	Horn rules ($\rightarrow$ head, or $\perp$)
O/F: fixed scope + interval	O/F: fixed scope + interval
exclusive interval claims	exclusive interval claims
difference constraints	difference constraints
	$+O(\ell_1 \vee \ell_2 \vee \dots)$: disjunctive obligation
polynomial, witness-producing — three engines: Horn propagation, sweep-line overlap, Bellman-Ford	NP-complete — selection guessed, verified by the same three engines (3-SAT reduction)

verified on 16 seeded instances (8 satisfiable, 8 unsatisfiable, seed 20260816): conflict-freedom $\Leftrightarrow$ satisfiability on all 16

Figure 6.5: One added rule kind turns a polynomial check into an NP-complete one. Both columns share the identical rule vocabulary of L_c : Horn rules, deontic obligations/prohibitions on scope and interval, exclusive claims, and difference constraints, decided in $O(H + T \log T + C \log C + VE)$ with a polynomial witness per conflict (the in-fragment theorem, left). Letting exactly one obligation become disjunctive (right) is enough: the extended fragment's conflict-freedom is NP-complete by reduction from 3-SAT (the frontier theorem), so the boundary is the presence of that one feature, not a graded notion of expressiveness. [verified, b4_deontic_fragment.py]

Figure 6.5 draws the two theorems side by side on the identical rule

vocabulary, so the one added rule kind is the only visible difference. Read together, the two theorems are a dichotomy at a designed boundary, in the sense Schaefer made standard for satisfiability [225]: on one side of a single expressive choice the problem is polynomial, on the other NP-complete, and the language designer is offered nothing in between. The same step is familiar one formalism over, where the simple temporal problem is polynomial and admitting disjunctions of difference constraints makes consistency NP-complete [59, 245]; the claim here is the location of the boundary, not the pattern.

Nor is the pattern new in normative conflict checking specifically, and it is worth being exact about where $\mathcal{L}_c$ lands, because “a similar dichotomy exists over there” is the sort of sentence that can conceal either a rediscovery or a real difference. Colombo Tosatto, Governatori and van Beest map the regulatory-compliance problem across eight variants cut by three binary choices — one obligation or many, obligations in force for a whole trace or triggered conditionally, and elements written as bare propositional literals or as arbitrary propositional formulae — and for *full* compliance (does *every* execution of a business-process model satisfy the norms?) only the last of the three moves the answer at all: with literals the problem is in P in all four variants, and with formulae it is coNP-complete in all four [255, 256].

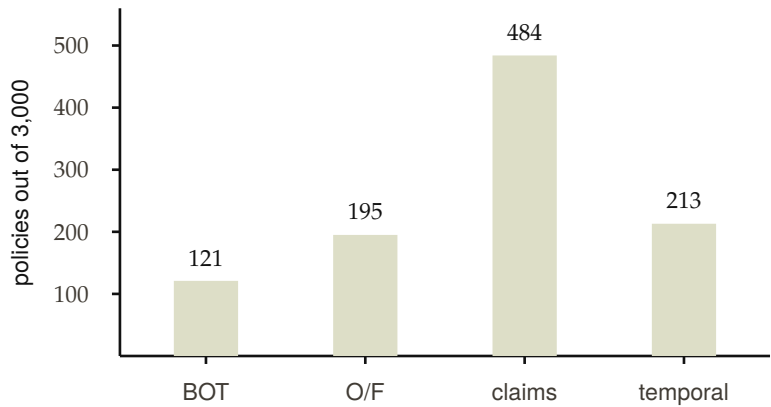
Run $\mathcal{L}_c$ against that axis and the answer is not the flattering one. A deontic rule here fires on a Horn body — a conjunction, and one derived through a program that may chain — which is a formula and not a literal, so $\mathcal{L}_c$ sits on the *hard* side of the only coordinate that moves their result, and is polynomial anyway (Theorem 6.4.1). That is neither a contradiction nor a sharper theorem; it is the tell that the two problems are not the same problem. Their hardness never came from the obligations. It comes from the process model: the coNP-completeness proof reduces TAUTOLOGY by building a workflow net with one exclusive branch per propositional variable, so the model’s 2^n executions enumerate every interpretation and full compliance holds exactly when the formula is a tautology [256]. The exponential object is the *branching of the process*, and the problem quantifies over it universally. $\mathcal{L}_c$ has no process model to branch. Conflict-freedom asks whether one set of commitments can be jointly discharged at all — an existential question over schedules — which is exactly why one step outside the fragment lands in NP (Theorem 6.4.2) rather than coNP. The two results sit on opposite sides of their expressivity boundary and run the quantifier over executions in opposite directions.

So the placement is *orthogonal*: $\mathcal{L}_c$ is not an instance of their tractable side, and their hard side is not a harder case of ours. The residue is worth naming rather than smoothing away. The variant of their problem closest in shape to conflict-freedom is not full compliance but *partial* compliance — does there exist a compliant execution — which on their own map is NP-complete in six of the eight variants, polynomial in one, and, for a single conditional obligation over literals, still open [256].

Theorems 6.4.1 and 6.4.2 do not close that hole and cannot, since they quantify over schedules rather than over the traces of a given model. What would settle the relation properly is a reduction in one direction or the other between $\mathcal{L}_c$ -conflict-freedom and partial compliance for that conditional-literal variant; nobody has written one, here or there, so the claim that survives is the modest one — this is a second instance of a dichotomy shape whose earlier instances are Schaefer’s and Dechter–Meiri–Pearl’s, on a different formal object, for a different decision problem, by a different reduction, and not its discovery.

Checked. 3,000 random in-fragment policy sets against a brute-force oracle (all 2^5 Horn models intersected, pairwise overlap checks, exhaustive integer search over the deadline potentials): 885 carried at least one conflict, 121 a derivable $\perp$, 195 an obligation-prohibition clash, 484 a claim overlap, 213 a negative temporal cycle (the kinds overlap: 764 sets carry one kind, 114 two, 7 three, so the four counts sum to 1,013 rather than 885); 0 disagreements with the oracle, and least model equal to the intersection of all models on all 3,000. Mutation: 149 of the 885 contain a conflict reachable only through Horn propagation, and in 100 of those it is the set’s only conflict, so a checker that never propagates flips those 100 verdicts to conflict-free [internal, `b4_deontic_fragment.py`, seed 20260816]. Figure 6.6 ranks the four conflict kinds by frequency. The canonical miss is the witness worked by hand below.

Numbers by hand — A conflict witness for the deontic layer, checked by hand. The deontic layer just named needs a decision procedure once two accepted norms might disagree. Here is the smallest case that shows why checking by hand does not scale, and why it does not have to: three rules over two intervals. *Fact:* `agent_deployed`. *Horn rule:* `agent_deployed` $\rightarrow$ `prod_access`. *Two deontic rules:* $O(\text{write_prod}, \{\text{env:prod}\}, [0, 10])$ once `prod_access` holds, and $F(\text{write_prod}, \{\text{env:prod}\}, [5, 15])$ unconditionally. Unit-propagate the Horn rule first: `prod_access` is derived, so the obligation fires. Both deontic rules now govern the same action and the same scope, $\{\text{env:prod}\}$; their intervals overlap on $[\max(0, 5), \min(10, 15)] = [5, 10]$, nonempty — an agent is simultaneously obliged and forbidden to write to production for five hours. That is a conflict by definition, and it is *indirect*: `prod_access` is nowhere a fact, only a consequence of one propagation step, which is exactly the case a checker that skips Horn propagation is built to miss [verified, `b4_deontic_fragment.py`: `real checker: conflict=True, O/F pairs=[(0, 1)]`; the same script’s ablated mutant reports `conflict=False` on this witness, and misses 100 of 885 conflicting instances in a 3000-policy sweep]. The tractable-fragment theorem behind this checker says this three-engine check — Horn propagation, an interval sweep, and a shortest-path feasibility test — runs in polynomial time with a witness attached, on facts,



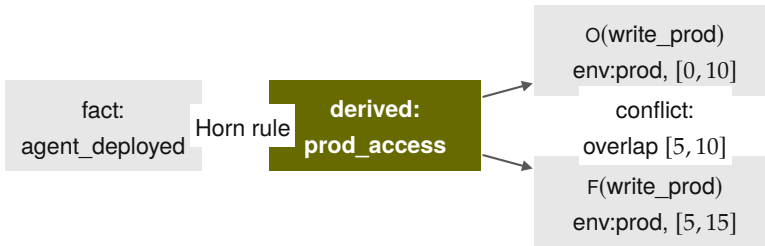
885 policies carry at least one conflict; the four counts sum to 1,013 because some carry more than one kind – 764 carry exactly one, 114 carry two, and 7 carry all three co-occurring kinds

Figure 6.6: Claim overlaps are the most common conflict, temporal cycles the second. Counts from the 3,000-policy sweep (seed 20260816): a derivable $\perp$ (121), an obligation/prohibition clash (195), an exclusive-claim overlap (484), and a negative-cycle deadline system (213). The four counts sum past the 885 conflicting policies because a policy can carry more than one kind at once, most often exactly one. [verified, b4_deontic_fragment.py]

Horn rules, deontic rules, exclusive resource claims, and difference constraints alike; nothing about this chapter’s own float-plan commitments needs more expressive power than that fragment offers. *Now you try:* Exercise 6.13 asks what changes, and what does not, once one of the rules is allowed to be a disjunction.

Figure 6.7 draws this witness as a derivation trace: the stated fact, the one Horn step, and the two rules it wakes up, joined by the interval they share.

a fact nowhere written down triggers an obligation and a prohibition



a checker that skips Horn propagation never derives `prod_access`:
neither rule fires, and the conflict is invisible – 100 of the 885
conflicting sweep instances (seed 20260816)

Figure 6.7: A conflict can hide behind one ordinary inference step. The fact `agent_deployed` is stated; `prod_access` is nowhere a fact, only its Horn-derived consequence. Once derived it fires both an obligation and a prohibition on `write_prod` in the same scope, whose intervals overlap on `[5, 10]`: the two rules clash directly (gold path, red join). A checker without Horn propagation never reaches `prod_access`, so neither rule fires and the conflict is invisible – exactly the failure a checker author who assumes only explicitly-stated facts can clash would build. [verified, `b4_deontic_fragment.py`]

What it buys. An authority inventory with prices. In-fragment conflict detection needs no authority at all: it is an algorithm with a witness, and composed with the kernel’s controllability theorem (*The Single-Writer Kernel*) it is a runtime guard, since the mediated acceptance write can refuse the commit and attach the witness. The authority no theorem removes sits exactly one step outside the fragment: choosing a discharge under a disjunctive obligation is a chartered resolver’s job, an NP-complete check or a judge, and choosing the norms themselves (priority, waiver, amendment) is outside every checker. A harbor therefore spends judgment only where a theorem certifies that judgment is required, and the schema rule that follows is concrete: as long as proposals stay inside $\mathcal{L}_c$, the acceptance path needs no reviewer role, and

the charter can delete the lookout it was about to appoint.

Completeness is relative to least-model semantics: monotone bodies only; negation as failure, unification, and symbolic interval endpoints coupled to difference variables all leave the fragment. The batch bound is what is proved; the incremental amortized form (watched literals, interval trees) is an engineering claim, not certified here. The checker certifies the norm *system*, never the norms: a harbor whose policies are consistently harmful passes every scan, and norm content stays a chartered authority's job. The oracle validation is on small instances (five-atom programs), so the sweep certifies agreement with the semantics there; the complexity bound, not the sweep, covers scale.

Skill licensing (IP). The per-use fee is released to the licensor *only on green settlement* (metered + clawback). This is forced by **Akerlof's market for lemons** (1970 [7], *when buyers cannot observe quality, price collapses to the average and good goods exit the market*): a skill's quality is unobservable before use, so a flat upfront license drives good skills out. Metering + clawback + a Merkle *portfolio proof* (the skill's prior settled outcomes, anchored in the evidence chain) restore the seller's incentive to ship quality.

6.4.1 One trade, all three sides, one settlement

The running example of §6.2 ran the three sides *in the alternative* — Bob buys the refactor as labor, *or* as capital, *or* as IP. C2 claims something stronger: that all three settle on the *same* escrow object, in one transaction, without a second ledger. That claim is only worth anything if the arithmetic closes, so here it is, closed.

The three-sided settlement, dramatized. Bob posts one float plan for the 1,200-line refactor, with three line items. *Labor*: a 400 CR bounty to Alice's firm. *Capital*: a 50 CR lease for Dana's profiling specialist, which Alice's crew needs for one module. *IP*: Carol's lint-and-type skill, metered at 0.5 CR per file against a 20-file cap, so 10 CR is reserved. Bob's wallet is debited $400 + 50 + 10 = 460$ CR into escrow. Alice separately posts a 100 CR performance bond, also into escrow. Before any agent spawns, the escrow row holds 560 CR.

The job settles **Success**: the test suite passes and the diff merges. The metered licence bills only the 16 files the skill actually touched, so $16 \times 0.5 = 8$ CR releases to Carol and the unused 2 CR returns to Bob. Alice's bond is refunded as collateral, not income.

Wallet	Net delta	Why
Bob (requester)	-458	-460 escrowed, +2 unmetered licence returned
Alice (labor, side 1)	+400	bounty; bond -100 in, +100 back, nets 0
Dana (capital, side 2)	+50	lease fee on the leased specialist
Carol (IP, side 3)	+8	16 files metered at 0.5 cr
Commons	+0	nothing slashed on a Success settlement
Sum	0	escrow 560 → 0; supply unchanged

Check it on one line: $-458 + 400 + 50 + 8 + 0 = 0$. Three sides, three privately-informed counterparties, three different reputation keys — and one escrow row that opens at 560 cr and closes at zero. Design Invariant 6.3.1 is not re-proved per side; it is the *same* invariant, because the lease and the licence are line items on the one float plan rather than trades on ledgers of their own. *Now you try:* settle the same plan **Partial** at 40% and confirm the sum is still zero. (Alice earns 160 and is slashed 60 of her bond; the commons gains that 60; the licence still meters at 8 because the files it touched were touched.)

*Exercises 6.10–6.13,
p. 370.*

6.5 The keystone the market rests on — and does not yet have

Everything above assumes a counterparty's identity is real. This section confronts the series' single *blocking* reconciliation: the identity primitive the whole market leans on covers a threat model that *excludes the cross-operator adversary the market is defined by*.

The keystone-split table in *From Spawn to Person* sets the two stones apart — the local root that chapter leans on, and the cross-operator stone it hands forward. This is the chapter that inherits the handoff, so the table is not reprinted here; what follows is what the market does about it.

6.5.1 Local non-forgable identity is the keystone — for one operator

Definition 6.5.1 (Local non-forgable identity). *Local non-forgable identity* is a per-agent identity that no agent can edit, guaranteed *within one trusted daemon*: the daemon is the only component that can hold state agents cannot reach, so it is a trusted root by construction. (The reference system now ships a partial slice — daemon-minted actor-souls, lookup credentials, and a bounded newcomer pool — while full write-boundary enforcement remains open. Papers 1–3 rely on the complete

contract; here we only consume it.) A **Design invariant**, PARTIAL: the partial slice ships and is exercised, but the write-boundary enforcement the full contract requires does not, so the guarantee below is weaker than its name until that gap closes.

For a single-operator harbor this is exactly right and is correctly named the highest-leverage keystone. Its threat model, stated plainly, is “a lazy or self-interested agent in a fleet the operator owns” — not a hostile peer. It is explicitly *not* a public-key infrastructure spanning strangers.

6.5.2 But its threat model does not reach the market

Local non-forgable identity defends against an agent inside a fleet the operator owns; it explicitly does *not* claim to defend against a hostile operator, and it is not a cross-party PKI. The market, by contrast, is *defined* by mutually-distrusting operators trading across a boundary neither controls. You cannot make the single-operator root the foundation of the cross-operator market by assertion. Across the boundary, the counterparty’s daemon *is* the adversary that local identity declares out of scope. The witness log gives *log integrity*, not *identity binding*: it does not answer “who vouches that Harbor *B*’s signing keys map to a real, distinct principal, rather than a hundred sock puppets Bob minted this morning?”

PITFALL

Definition 6.5.2 (The cross-operator attestation problem). *Cross-operator attestation* is the problem of binding agent signing keys across harbors you do not own: an actual key-distribution and attestation story for the cross-operator threat model — something that lets Alice’s harbor trust that a key presented as “Bob” really is the one principal she has been building a reputation history against. **Status:** SPECIFIED-to-proposed; it has no implementation and only a partial specification for its critical part. It is the highest-leverage unbuilt keystone, and it gates the entire market thesis.

This paper *stops* describing the federation boundary as one “neither controls” while quietly resting on a single trusted root. Either cross-operator attestation is built (a real attestation story), or the market’s scope is restated as “federation among *semi-trusted* operators.” We take the first horn as the designed target and flag every claim that depends on it.

KEY IDEA

6.5.3 The principal above the actor

The economic counterparty in every trade is the **principal** (the human or org owning a fleet), not the agent. Bonds, reputation inheritance, and sanctions must key on the principal via the hop-bound authorization chain, or Sybil bond-farming works by spawning fresh *agents* under one principal (**Douceur's** Sybil attack [68]: *free identities defeat any reputation or quorum system*). Defining the principal as a first-class economic entity is the cleanest Sybil fix; its cryptographic binding is the identity object specified in *From Spawn to Person* and consumed here.

6.6 Conservation under composition

The economy's single cross-boundary invariant is that internal ledger operations redistribute value rather than create or destroy it. This section states the compositional claim directly and separates it from cross-currency valuation.

Figure 6.8 draws the composition claim: sequential traces glue, and internal operations carry zero external balance change.

Let a ledger state be $x = (W, E, C)$ and let a valid transaction trace $f : x \rightarrow y$ carry external balance change $\Delta(f) = S(y) - S(x)$ for $S(x) = W + E + C$. Sequential traces compose by concatenation and satisfy $\Delta(g \circ f) = \Delta(g) + \Delta(f)$. Internal escrow, refund, and slash traces have $\Delta = 0$; top-ups and withdrawals are explicit boundary flows. This is the entire conservation claim. Category language can describe trace composition, but it does not add a currency-conversion theorem.

THEOREM 6.6.1 Single-unit conservation composes. Suppose all harbors use one unit of account and every cross-harbor transfer is a matched atomic move from source escrow to either destination settlement or source refund, with an explicit in-flight bucket. Then any finite composition of internal and cross-harbor operations has total external balance change equal to the sum of its recorded top-ups and withdrawals. In particular, a trace containing only internal transfers has $\Delta = 0$. (SPECIFIED; proof sketch in Appendix 6.B.)

Property 6.6.1 (Cross-currency valuation boundary). With heterogeneous units, exact conservation is defined per native unit. A scalar portfolio total additionally requires a valuation map v_t at a named instant t , plus explicit fee and slippage accounts. If rates move between legs, the mark-to-market difference is economic exposure, not a failure of functoriality. Bounding that exposure by φ requires a separate oracle-latency and price-move assumption that this paper has not proved. (*open.*)

Conservation is exact per unit of account. A cross-currency scalar is

RECALL

1. Without looking: what is the one word in the market's own definition that local non-forgeable identity's threat model directly contradicts?
2. State the difference between local non-forgeable identity and cross-operator attestation, one sentence each.
3. Why can the single-operator trusted root not be reused as the cross-operator market's foundation "by assertion"?

Exercises 6.14–6.16, p. 371.

THESIS

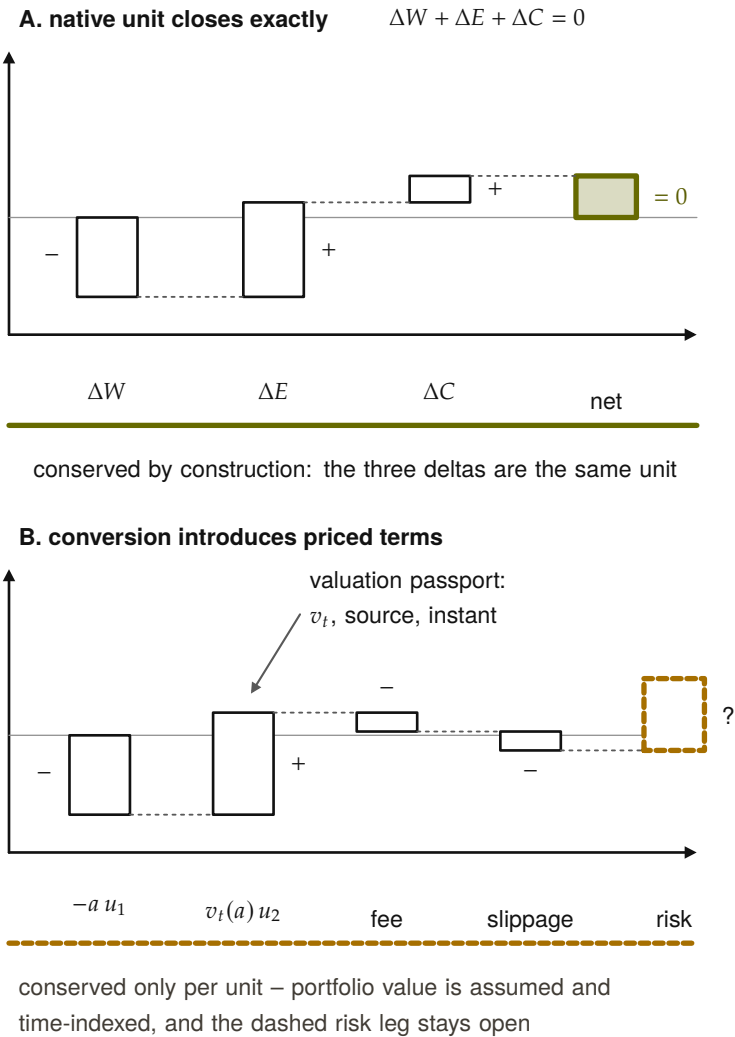


Figure 6.8: Conservation closes inside a native unit; conversion requires a named valuation boundary. Panel A is an accounting waterfall: wallet, escrow, and commons deltas return exactly to zero (the gold bar, each leg signed + or – against the zero baseline). Panel B cannot close by adding unlike units: it must record the valuation function and instant, plus fee and slippage, and it leaves one dashed, amber, unconserve leg – open exposure to v_t . Composition preserves native-unit balances; notation does not manufacture a unit-free portfolio total. [internal]

a valuation, and every valuation must name its rate source, time, fees, and exposure bound. “Lax functor” is not a substitute for those data.

6.6.1 The Myerson–Satterthwaite corner

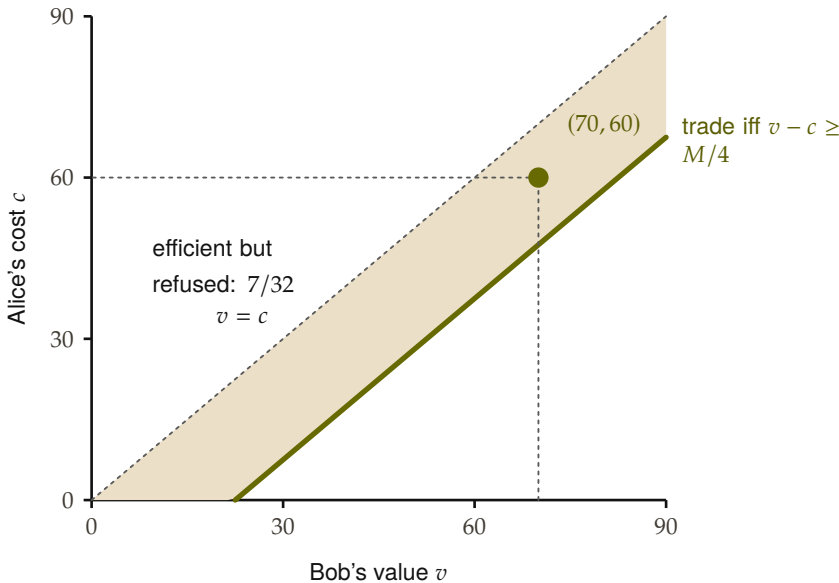
Strict conservation (Design Invariant 6.3.1) is exactly *budget balance*. That is not free, and the theorem that says so is Myerson and Satterthwaite’s, not ours; what is ours is the number it produces when it is pointed at this market’s own running example.

THEOREM 6.6.2 Conditional bilateral-trade boundary. For any bilateral slice of this market satisfying the Myerson–Satterthwaite assumptions—independent private buyer and seller values drawn from overlapping continuous supports, Bayesian incentive compatibility, interim individual rationality, and budget balance—no mechanism is also ex-post efficient everywhere [179]. This theorem constrains such a slice; it does not by itself prove incentive compatibility of the harbor mechanism or identify the desideratum sacrificed by the full three-sided market.

Numbers by hand — *Efficient trade that the mechanism still refuses.* Theorem 6.6.2 names the corner the market gives up; here is what giving it up costs in the chapter’s own numbers. Suppose Bob’s value v for renting Alice’s specialist a_2 and Alice’s opportunity cost c of foregoing a_2 for that stretch are each drawn independently and uniformly from $[0, 90]$ CR — the 90 CR lease fee of §6.2’s own running example is one realized draw from exactly this pair. Any mechanism satisfying Myerson–Satterthwaite’s four desiderata on this bilateral slice is, by the theorem, not ex-post efficient; the canonical incentive-compatible, individually-rational, budget-balanced instance is the Chatterjee–Samuelson double auction [46], whose linear equilibrium on $[0, M]$ has Bob bid $\beta(v) = \frac{2}{3}v + \frac{M}{12}$ and Alice ask $\sigma(c) = \frac{2}{3}c + \frac{M}{4}$, trading iff $\beta(v) \geq \sigma(c) \iff v - c \geq M/4$. At $M = 90$ that threshold is 22.5 CR. Take Bob’s value at 70 CR and Alice’s cost at 60 CR: trade is efficient (a 10 CR surplus sits on the table) but $v - c = 10 < 22.5$, so the mechanism reports no trade — the efficient rental simply does not happen. Integrating over the whole square, the no-trade-but-efficient region $\{0 \leq v - c < M/4\}$ has probability $\frac{a(2M - a)}{2M^2}$ at $a = M/4$, which simplifies to $7/32 \approx 21.9\%$ independent of M : better than one trade in five that would leave both sides better off is refused by the very mechanism built to protect them from lying about v and c [internal; algebra shown]. This is the concrete shape of Theorem 6.6.2’s abstract

impossibility, not a separate claim: raise the stakes (a bigger M) and the threshold and the missed trades grow with it, in the same fixed proportion.

Figure 6.9 draws the whole (v, c) square: the worked point sits inside the shaded band the mechanism refuses, not at some rare corner.



RECALL

1. Without looking: state the composition law $\Delta(g \circ f) = ?$, and say which kind of ledger trace always has $\Delta = 0$.
2. What does a scalar cross-currency valuation need that native-unit conservation does not?
3. Name the four things Myerson–Satterthwaite says a bilateral trade mechanism cannot have all at once.

Exercises 6.17–6.19, p. 371.

Figure 6.9: The double auction refuses more than one efficient trade in five. Above the diagonal $v = c$ every draw has a real surplus; the mechanism only trades it when $v - c \geq M/4 = 22.5$ (gold line). The band between the two lines is efficient but refused, $7/32 \approx 21.9\%$ of the square, the same fraction at every M . At the chapter's own worked point, Bob values the rental at 70 and Alice's cost is 60: a 10-cr surplus sits on the table, but $10 < 22.5$, so no trade happens. [internal; algebra shown]

The current design establishes a conservation goal and voluntary entry; it has not proved truthfulness. An AGV-style mechanism changes the incentive and participation notions and generally offers expected rather than ex-post balance. Comparing those alternatives requires a fully specified type and transfer model, which remains open.

6.7 The Anchor Protocol proper: cross-harbor capability transfer

The *cross-harbor capability-transfer ceremony* is the market's identity-level hot path: how an authorization minted under Alice's harbor becomes usable, safely, inside Bob's. The *Anchor Protocol* analyzes the cryptographic substrate (a hop-bound authorization chain checked by proto-

col models plus a bounded Rust verification layer); *this* section is the ceremony that rides that substrate across a boundary.

The four-message ceremony is drawn and stepped through in *The Federated Harbor* (§8.4.1), which owns the protocol; what the market needs from it is only this, and the ceremony has one decisive liveness property: after message (3), no further synchronous interaction with Harbor *A* is needed. The transferred card C'_B is verifiable under Harbor *B*'s key alone, so federation does not turn every authorization into a synchronous inter-machine round trip. The epoch root $R_A^{(e)}$ bound into the envelope is the governing primitive: a revocation issued by *A* after epoch *e* invalidates C'_B as soon as the new root $R_A^{(e+1)}$ reaches the witness log that *B* audits.

Numbers by hand — *The ceremony, dramatized.* Alice's specialist a_2 holds a card C_A that authorizes "read and write src/payments/, run the test suite." Bob rents a_2 for an afternoon. Alice's harbor offers an attenuated envelope: att_A strips write access to everything except the one module Bob hired for. Bob's harbor, not trusting Alice's word, attenuates *again*: att_B adds "no network egress, six-hour TTL." The minted C'_B is the intersection — read/write one module, run tests, no network, expiring at dusk — and it verifies under Bob's key, so a_2 never has to phone home to Alice mid-task. At 5 p.m. Bob's revocation fires locally; at the next epoch boundary Alice's harbor also sees, in the shared witness log, that C'_B was used exactly as attenuated and nowhere wider. *Now you try*: intersection is commutative, so trace by hand whether applying att_B before att_A would have minted a different C'_B .

Capability *attenuation* is the safety property here: a transferred card can only *narrow* authority — $C'_B = \text{att}_B(\text{att}_A(C_A))$ — never widen it. This is the cross-boundary form of the attenuation-monotonicity that the hop-bound authorization chain checks within one harbor (*The Anchor Protocol*). It is what makes "rent my agent for an afternoon" safe to say to a stranger.

Do not let the federation theorems borrow the substrate's "formally verified" halo. The hop-bound authorization chain has bounded symbolic and Rust evidence in *The Anchor Protocol*. The *federation* claims are design arguments plus a bounded TLA⁺ extension; they are SPECIFIED, not deployed. Two different confidence levels, never blurred.

KEY IDEA

PITFALL

RECALL

1. Without looking: after which numbered message does Harbor *A*'s continued participation stop being required?
2. State the attenuation-monotonicity property of C'_B in one sentence.
3. If *A* rotated its signing key after issuing, what must *B* check?

6.8 Federation: trading across machines you don't own

Federation is where the leaderless distributed canon actually bites. Crucially, the federation design *refuses consensus* rather than trying to route around Fischer–Lynch–Paterson — which is the correct reading of the impossibility the next box states.

The topology this section defends has four rails — independent harbors linked only by gossip and a shared witness log, with no global consensus step — and they are tabulated in *The Federated Harbor*.

6.8.1 No consensus, by design

A system that *needs* agreement on a global event order across operators it does not control is dead on arrival, by **Fischer–Lynch–Paterson** (1985) [81] (*no deterministic protocol achieves consensus in an asynchronous network with even one crash failure*). Federation declines to require agreement. It requires only (a) append-only per-harbor logs, (b) gossip-bounded propagation, and (c) *detectable* (not prevented) equivocation. This is a **Certificate Transparency** posture [146]: you do not prevent a misbehaving log from lying, you make lying detectable after the fact and expensive via a slashable bond.

KEY IDEA

The convergence and detection bounds smuggle in *partial synchrony* (**Dwork–Lynch–Stockmeyer** 1988 [71], the canonical escape from FLP): “gossip every Δ ” is an eventual-synchrony assumption, and we state it as such rather than burying it.

6.8.2 Settlement and the cross-chain-deals lineage

The Federated Harbor places this construction against the atomic-swap and adversarial-commerce baselines it is compared to below.

The escrow construction states its trust assumption outright: it does *not* claim trustless settlement. Its loss bound is conditional, and the condition is the whole content of the claim — which is why the claim is stated once, as Design Invariant 8.6.1 in *The Federated Harbor*, the chapter that proves it by case analysis over the custody ledger’s two terminal transitions, and is not printed again here under a second number. What it says, in one breath: *if* the custody ledger is non-bypassable — a recipient whitelist, a fee cap, one atomic terminal transition, and no operator path around any of the three — then a counterparty’s worst-case loss on a cross-harbor settlement is the pre-agreed fee φ and the outcome is exactly **Clear** or **Refuse**; if that boundary *can* be bypassed, the worst-case loss is the whole balance in custody. What this section adds is where that construction sits.

This is a deliberate departure from the hash-timelock baseline (**Herlihy** 2018 [108], atomic cross-chain swaps), and the right com-

parison is the formal model of mutually-distrusting commerce without a shared chain: **Herlihy, Liskov & Shrira** (2022, *Cross-Chain Deals and Adversarial Commerce*) [109]. The harbor’s bounded escrow is a “trusted third party / watchtower” point in that design space; we locate it there explicitly rather than reinvent it.

Remark. **Open problem: trustless cross-harbor settlement.** It is unknown here whether non-fungible, reputation-priced bonds admit a useful trustless settlement protocol under the paper’s availability and privacy requirements. An HTLC comparison does not prove impossibility because the asset and adjudication models differ. (*open.*)

6.8.3 Revocation gossip and the equivocation race

The gossip mechanism itself is drawn and analysed in *The Federated Harbor*; what this section prices is the window it leaves open.

Federated revocation propagates by anti-entropy gossip (**Demers et al.** 1987 [60]) with a probabilistic-membership filter (a cuckoo filter — a compact structure that answers “is this card revoked?” with no false negatives and a tunable false-positive rate). The federation therefore has expected $\Theta(\log m)$ -round dissemination only under a connected reliable-round overlay with the stated randomized peer-selection model. This asymptotic expectation is not an exact constant or a partition-tolerant deadline.

Protocol 6.8.1 (Federated revocation gossip). **State:** each harbor X keeps a revocation filter F_X and a current epoch root $R_X^{(e)}$ published to a shared witness log.

1. *Revoke.* An operator revokes card c at its home harbor A : $F_A \leftarrow F_A \cup \{c\}$; A advances to $R_A^{(e+1)}$ and publishes it.
2. *Gossip.* Every Δ , each harbor picks a peer and exchanges filter deltas (anti-entropy). A card present in either filter becomes present in both.
3. *Audit.* Any third party compares each harbor’s published roots in the witness log; a harbor that publishes two contradictory roots for one epoch is *equivocating*. Verification is constant work once both signed roots are co-located; delivery to an auditor has the overlay’s separate dissemination delay.

Post, conditionally: in the connected reliable-round model, every honest harbor learns the revocation in expected $\Theta(\Delta \log m)$ time. Under an unbounded partition there is no finite global-learning bound.

Numbers by hand — *The revocation race, traced hop by hop.* Alice revokes a_2 ’s card c at $t = 0$ (say she discovers it was misbehaving). Harbor B , which rented a_2 , has not yet heard: for the interval $[0, \Delta]$, B ’s filter is

stale and c is still spendable there. At $t = \Delta$ gossip reaches B , which adds c to F_B and refuses further use. Harbor C , two hops away, learns at $t = 2\Delta$. This is one illustrated execution; a different schedule or partition produces a different window. Conjecture 6.8.1 says the bond Alice posted on c must dominate whatever a_2 could spend at B inside that window, or a well-capitalized Bob could rationally race the gossip. *Now you try:* extend this same illustrated schedule one more hop — at what t does a harbor three hops from Alice learn the revocation, and what does a_2 's card still let it spend at B in the meantime?

The threat is *adversarial*, not stochastic. An attacker who controls peer selection or partitions the gossip graph (an *eclipse attack*, **Heilman et al.** 2015 [104]) breaks the expected-time guarantee. The statement is conditional: a finite service-level bound needs eventual connectivity and a known post-stabilization delivery bound; epidemic analysis then supplies an expectation within that model, not an adversarial deadline.

Detection-after-the-fact deters only if the slashed bond exceeds the value extractable during the detection window. This is the analogue of the *cleanup lower bound* (the companion result that a cleanup bond must cover the worst-case cost of the damage it underwrites), and we state it as a target:

Conjecture 6.8.1 (Equivocation Lower Bound). *For a deployment that advertises a finite freshness window T , the witness bond must dominate the maximum value spendable against a revoked-but-not-yet-observed capability over T . If partitions are unbounded, no finite bond covers the worst case; the system must instead fail closed on stale roots or cap spend independently. (open; analogue of the cleanup lower bound.)*

The threat bands in *The Federated Harbor* set the window Conjecture 6.8.1 must price against the connectivity assumptions each band needs.

PITFALL

RECALL

1. Without looking: name the three things federation requires in place of consensus.
2. State the Bounded Custody Loss property's condition and its two possible outcomes.
3. Why is "gossip every Δ " itself a partial-synchrony assumption rather than a proven guarantee?

Exercises 6.23–6.25,
p. 372.

6.9 Reputation: monotone, but revocable

Reputation is the third side's existence condition. Two reconciliations matter here.

6.9.1 "Never ends" is not an anti-revocation property

A tempting design slogan holds that reputation "never ends" — no decay window an agent can outrun — while the same design also requires that a fraudulent settled outcome be retractable. A property and its required violation cannot both be requirements.

The reconciled statement: **reputation is monotone in honest witnessed outcomes, but a witnessed outcome is itself revocable via a propagating tombstone.** Dissemination has the same model and partition

KEY IDEA

caveats as capability revocation. “Never decays” applies to honest outcomes only; it is *not* an anti-revocation property.

We also decline to assert no-decay as a virtue: bounded memory is a design parameter (Liu & Skrzypacz 2014, *Limited Records and Reputation Bubbles* [155], show unbounded records create reputation *bubbles*, not stability). The horizon is a tunable, and ∞ is rarely optimal.

6.9.2 Why Sagas-style compensation does not restore reputation

It is tempting to fix revocation with compensating transactions (Garcia-Molina & Salem 1987, *Sagas* [86]), as the bond ledger does. But the analogy fails structurally:

THEOREM 6.9.1 Compensation does not erase reliance. A ledger can append a compensating transaction that restores a numerical balance. A reputation tombstone cannot undo decisions counterparties already made while relying on the invalid outcome. It can only change future evaluations after the tombstone is observed. Reputation repair therefore requires dissemination plus an explicit remedy for past reliance; it is not equivalent to ledger compensation.

This is why Sagas appears in the bond-settlement prior art but *not* in the reputation prior art.

6.9.3 The grading-oracle keystone

Every folk-theorem incentive-compatibility claim in the market bottoms out in “the daemon derives the grade.” But the grade is produced by a capturable evaluator.

THEOREM 6.9.2 The monitoring model must include the evaluator. An imperfect-public-monitoring equilibrium result requires a specified conditional distribution of public signals given modeled actions and strategies. If a strategic grader chooses the signal but is omitted from the game, that distribution is not fixed by the model and the claimed equilibrium result is unsupported. One must either constrain the signal mechanically or include the grader, its information, payoffs, and deviations as part of the game [3].

The two resolutions (both used). *Fix A — mechanically constrained evidence:* ground settlement in evidence the agent cannot directly author

(a CI-issued test result, a protected merge event, or an independently evaluated policy check). This narrows the signal channel; it does not make the task metric universally strategy-proof. *Fix B — bond-and-slash the judges*: where machine-checking is impossible (aesthetics, ambiguous quality), the human/LLM judges get their *own* bond and their *own* reputation; a judge later contradicted by re-audit is slashed. The “rate-the-raters” recursion must then be shown to terminate at a machine-checkable base, not regress infinitely. De-biasing for LLM judges (order-swap, pairwise, family-exclude) follows **Zheng et al. 2023** [278].

Numbers by hand — A captured judge, followed through the re-audit lane. Bob disputes a settlement: he claims Alice’s a_2 delivered an “untasteful” refactor. Taste is not machine-checkable, so Fix B applies: Judy, a human grader, is drawn to arbitrate. Judy posts a 50 CR judge-bond and rules *for Bob*. But Judy and Bob are colluding — Judy is a captured judge, the exact failure Theorem 6.9.2 warns of. The market’s defense is the re-audit lane: a sampled second panel later reviews Judy’s ruling against the preserved evidence, finds it indefensible, and *contradicts* it. Judy’s 50 CR bond is slashed and her judge-reputation takes a tombstone; Alice is made whole. The signal became exogenous again not because Judy was honest but because lying was bonded and the re-audit made it expensive — and the recursion stops at the second panel, whose own evidence is machine-checkable (the same diff, the same tests). *Now you try*: trace the same dispute if the second panel’s own ruling were *also* disputed — what has to be true of its evidence for the recursion to stop there rather than needing a third panel?

RECALL

1. Without looking: what CAN a reputation tombstone do, and what can it NOT undo?
2. State Theorem 6.9.2 in one sentence: what does an imperfect-public-monitoring result require that an omitted strategic grader leaves undefined?
3. Name the two resolutions (Fix A and Fix B) and the ground-truth base the recursion in Fix B must terminate at.

Exercises 6.26–6.28, p. 372.

6.10 Hosted trust is the product

The 2025–26 agentic-payments landscape — **AP2** (Google, signed payment mandates), **ACP** (OpenAI/Stripe), and **x402** (Coinbase, stablecoin settlement over HTTP) — has commoditized the *settlement rail*. Wiring an agent to pay another agent is now table stakes. This is *good news* for the thesis, because it sharpens what the product actually is.

What remains scarce, and therefore defensible, is **hosted trust**: the verified ledger (conservation-checked, Merkle-anchored evidence), the *relay* (the harbor mesh carrying public-key attestation across machines), and the *reputation* that makes a counterparty you do not own legible and accountable.

- **Substrate (swappable)**: the payment rail. Use x402/AP2/credits/Stripe interchangeably.
- **Product (the moat)**: attestation, federation membership, and reputation hosting.

The revenue model aligns incentives without perverse over-penalty: a transaction fee (2–5% of settlements), a small listing fee (collusion deterrent), and a small bond-spread on *forfeited* bonds (kept small so the platform never profits from over-slashing). The platform should earn most from *successful* trade, because successful trade is what hosted trust produces.

Numbers by hand — *What the moat earns on one trade, at the stated band.* Take the SUCCESS settlement of §6.3: Bob’s 400 CR bounty clears in full to Alice. At the stated transaction-fee band of 2–5%, the platform’s cut on that one trade is $0.02 \times 400 = 8$ CR at the low end and $0.05 \times 400 = 20$ CR at the high end — earned because the trade *settled*, not because a bond was slashed. Compare the 60 CR bond-spread the same section’s **Partial** settlement forfeits to the commons: under a small bond-spread, most of that 60 CR still goes to the commons/salvage fund, not the platform, which is exactly the “never profits from over-slashing” design the revenue model states in words.

As §6.1 put it, before any of the machinery was on the page:

You don’t sell crypto — crypto is the substrate. Here is what *is* scarce: attestation + federation membership + reputation.

THESIS

6.11 Cold start, and the auction question

A three-sided market has a *triple* cold-start problem: no operators shop an empty fleet shelf; no owners list agents with no renters; no licensors publish skills with no buyers. Two-sided-market theory prescribes: subsidize the side carrying the scarcest cross-side externality, then flip. Phase 1 seeds supply at zero/negative price with platform-posted tasks at above-market bounties (generating the first settled outcomes — the bootstrap reputation data); Phase 2 imports external work history for partial credit to break the new-agent freeze; Phase 3 flips to charging the demand side once a liquidity threshold (not a calendar date) is met. Figure 6.10 draws the schedule against the quantity that actually gates it, because “threshold, not date” is the kind of phrase that stays a slogan unless you can see what it is a threshold *on*.

6.11.1 Static escrow vs. competitive underwriting

Thomas Youle’s competitive-insurance proposal — insurer-agents bid to underwrite transactions, replacing a static bond parameter with a market-discovered price — is a possible *fourth* side. But it may collide with the cleanup lower bound:

Proposition 6.11.1 (Underwriting reframes the bond as reserve adequacy). *The cleanup lower bound requires $\text{bond} \geq \text{worst-case cleanup cost}$*

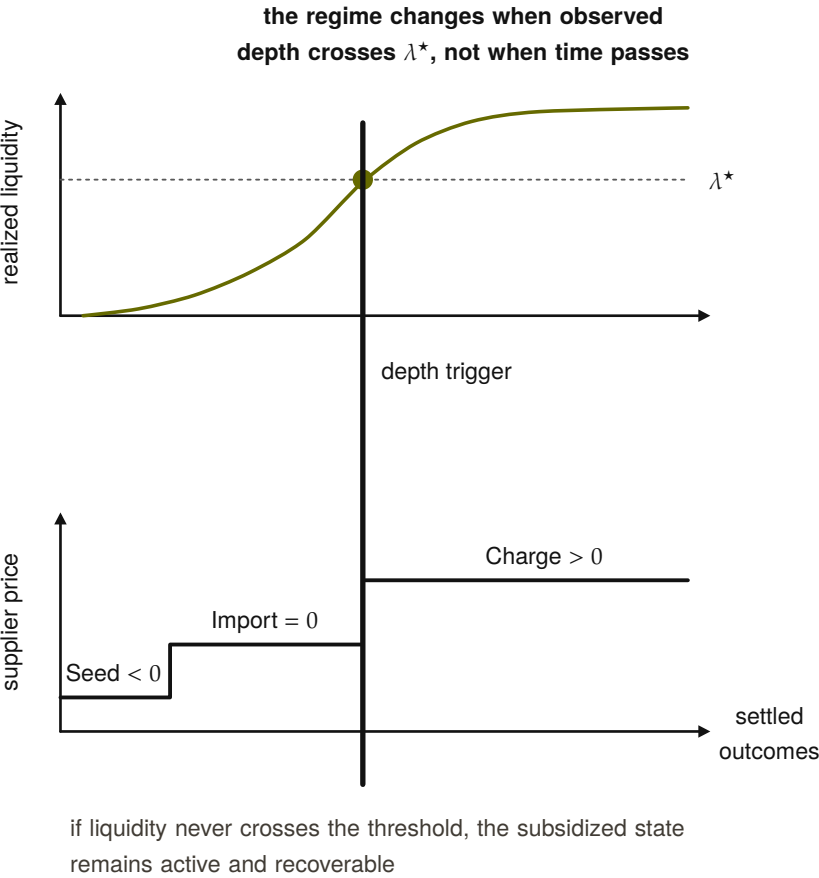


Figure 6.10: Cold start separates the measured market state from the policy schedule. The upper trace is realized liquidity over settled outcomes (gold). The lower trace is the supplier-price regime applied to that state: seed supply below zero, credit imported work at zero, then charge only after liquidity crosses λ^* . The shared vertical trigger (spine) makes the dependency explicit. Calendar time is absent, and a market that never crosses the threshold does not silently advance to the charging regime. [internal]

c. A competitive insurance market prices to expected loss, which sits below the worst case for any below-tail agent. Either underwriting violates the cleanup lower bound (the commons can leak in the tail), or the bound must be reframed as the insurer's capital-adequacy constraint (a reserve-requirement framing with tail reinsurance), not a per-transaction bond. The form of the answer is determined; only the reserve formula is open. This is a **Theorem** in the honesty ledger's sense — it follows from the cleanup lower bound's own definition plus the definitional fact that competitive pricing tracks expected, not worst-case, loss — about a side that is itself only proposed: (SPECIFIED; underwriting is not built, and the reserve formula the dichotomy leaves open is unspecified).

6.11.2 Cartels and wash-trading

The per-round decision node and the closed-form sustainability inequality are drawn in *The Bonded Commons*, which develops the repeated game.

Multi-homing plus portable reputation enables cross-harbor *cartels* — a rating-agency cartel is the closest real-world analogue: several independent principals colluding to hold a shared reputation signal artificially favorable rather than competing on it. And a colluding requester+worker can *wash-trade* fake settled outcomes to mint reputation cheaply. The defense form — cost-per-settled-outcome must exceed reputation's marginal value — is circular as stated, because that marginal value is endogenous to the same market. The fix is a structural break: make settled outcomes between a counterparty *pair* exhibit sharply diminishing reputation returns (a concavity that caps the wash-trade payoff regardless of price), plus sampled adversarial re-audit of high-velocity pairs.

Whether a cartel of colluding operators can actually hold its rating floor is a *repeated* game, and that is what the cartel game in *The Bonded Commons* draws. Each round a member either *colludes* (holds the agreed floor q_{floor} , collecting the cartel payoff π_C) or *defects* (undercuts the floor by ε for a one-time gain π_D , after which the cartel unravels and everyone drops to the competitive payoff $\pi_N \approx 0$). A **grim-trigger** strategy is the crudest way to punish that: *cooperate until the first observed defection, then defect forever*. It sustains the cartel only while members weight the future heavily enough — that weight is the **discount factor** $\delta \in (0, 1)$, the value a member places on next round's payoff relative to this one's, so a member with δ near 1 is patient and one with δ near 0 will take π_D today and let the cartel die. Sustaining collusion means the discounted stream of colluding payoffs beats the one-shot deviation, which is exactly the inequality the figure's box states, and δ clearing its threshold is exactly what the defenses above are trying to prevent.

RECALL

1. Without looking: which side does Phase 1 subsidize, and by what rule?
2. State Proposition 6.11.1's two horns for what happens once a competitive insurance market prices to expected rather than worst-case loss.
3. What structural change — not a price threshold — is needed to cap the wash-trading payoff regardless of price?

Exercises 6.29–6.31,
p. 372.

6.12 A gluing analogy for the open federation problem

Three open problems—cross-harbor settlement, equivocation under partitions, and cross-currency valuation—share a local-to-global shape. That resemblance motivates an analogy; it does not yet define a sheaf or a cohomology theory.

Conjecture 6.12.1 (Formalization target, not established result). *A future treatment may define a site of overlapping administrative domains and a presheaf of signed ledger views, then ask when compatible local sections admit a unique global section. This paper has not specified the site, restriction maps, coefficient object, or cocycles; therefore it makes no claim that a particular H^1 is defined or non-zero. The immediate engineering obligations remain the concrete ones: matched custody transitions, freshness policy under partitions, and time-indexed valuation.*

Fong and Spivak’s compositionality program [82] suggests vocabulary for a future formalization. Until the site and coefficient data are defined, the paper uses “gluing” only to organize engineering questions.

6.13 Gaps the design must still close

Naming what is unspecified is part of the specification. These are not threats already defended; they are holes.

Numbers by hand — *Watching the built floor break, by hand*. This is not a resolved mechanism — it is the gap below, traced against a number already on the page so “orphaned escrow” is not just a phrase. Take the three-sided settlement of §6.4.1: Bob’s 400 CR bounty, 50 CR lease, and 10 CR licence reserve, plus Alice’s 100 CR bond, lock 560 CR into one escrow row. Now suppose Alice’s harbor — the daemon that alone can sign her side of the settlement — goes dark mid-job, before the oracle closes. Design Invariant 6.3.1 still holds arithmetically (wallet+escrow+commons has not changed, since nothing moved), but the 560 CR row now has no principal left to countersign a settlement, a refund, or a slash: it is correct and stuck. That stuck state, not a wrong sum, is exactly what **Operator exit / harbor death** below names as unspecified, and it is why Table 6.2 marks it as reaching conservation rather than some softer property.

1. **Multi-dimensional reputation aggregation.** “Accuracy/aesthetics/ efficiency judged by separate experts” has no specified combination rule. You cannot have “separate axes” and “one buyer-readable score” without specifying an aggregation function and its trade-offs. (*proposed.*)

RECALL

1. Without looking: name three of the four things Conjecture 6.12.1 says this paper has not specified about the proposed gluing model.
2. Why are two incompatible signed roots evidence of equivocation but not, by themselves, a cohomology class?
3. What are the concrete engineering obligations that remain regardless of whether the formalization is ever completed?

Exercises 6.32–6.34, p. 373.

2. **Float-plan amendment under scope drift.** The four states assume the acceptance criteria were right. Mid-flight re-specification needs a re-sign protocol with escrow top-up/refund delta, preserving conservation.
3. **Cross-boundary key rotation.** How a rotated signing key invalidates or re-validates cross-harbor delegations already held by *B* is unspecified.
4. **Skill versioning.** Reputation attaches to a content hash; v2 starts cold. The lemons problem reappears at every version bump.
5. **Operator exit / harbor death.** Escrow orphaning, in-flight float plans, and reputation tombstoning on ungraceful exit are unspecified — the market-level analogue of the kernel's crash-recovery problem.
6. **Incentive-compatible arbitration capacity.** The human oracle is scarce and expensive; at scale arbitration is itself a priced, bondable, slashable service that must converge to honest rulings, not rubber-stamping.

These six are not equally dangerous, and an implementer triaging them should not have to infer the stakes from the prose. Each one threatens a specific critical promise made earlier in this paper:

Table 6.2: Which promise each gap threatens. The two gaps that reach the *conservation* invariant are the urgent ones, because conservation is the only claim in this paper graded **IMPLEMENTED** — a hole there falsifies something that is currently true, rather than deferring something that is not yet true.

Gap	Threatens	How it bites
Operator exit / harbor death	conservation	orphaned escrow rows have no settlement path, so wallet + escrow + commons stops summing to supply
Float-plan amendment under scope drift	conservation	a mid-flight re-spec moves value with no matching debit unless the top-up/refund delta is escrowed
Cross-boundary key rotation	cross-boundary safety	a delegation already held by <i>B</i> may verify against a key <i>A</i> has retired
Skill versioning	Sybil-resistance	a v2 content hash is a fresh identity, so the lemons problem and the whitewash both reappear at every version bump
Multi-dimensional reputation aggregation	incentive compatibility	an unspecified aggregation rule is an unspecified target, and any fixed scalar collapse re-creates a Goodhart objective
Incentive-compatible arbitration capacity	incentive compatibility	Fix B (§6.9) prices one judge; it does not price a judge pool under load, where rubber-stamping is the cheap strategy

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **A market, not a payment rail.** The harbor prices work between three sides, an operator-for-hire, a buyer and an insurer, and the escrow row is the one object all three sides read.
2. **The succession rule has a price.** Letting a worker be replaced mid-task costs $D^* = \eta K / (1 - \eta K)$ in expectation; below that, the buyer is better off waiting.
3. **The float plan is the built floor.** A worker becomes *running* only at the committed transition where the wallet is debited and the escrow row exists; a signed plan alone admits nothing, and abort leaves all three traces unchanged.
4. **Conservation is an invariant, not a hope.** In native units $\Delta W + \Delta E + \Delta C = 0$ exactly, and no spawn happens without a bond; once amounts are converted, fee, slippage and risk are priced terms and the total is no longer zero by construction.
5. **Deontic conflict is decidable inside the fragment and NP-complete one step outside it.** The in-fragment check is polynomial and produces a witness; that is what the daemon runs.
6. **The keystone is missing and named.** Cross-operator attestation is what the market rests on and does not yet have; local identity is built.
7. **Single-unit conservation composes;** bilateral trade across a boundary is conditional, and the condition is stated.
8. **Reputation is monotone but revocable:** compensation does not erase reliance, and the monitoring model must include the evaluator.
9. **Hosted trust is the product:** cold start is solved by a supplier price at λ^* , not by an auction the market cannot yet clear; the cartel game says which regime detection and loss put an insurer in.

6.14 Exercises

§6.1 THE THESIS: A MARKET, NOT A PAYMENT RAIL

- 6.1 CHECK ★ Give a one-sentence reason the market cannot ship before reputation does, using only the word “clone.”

Solution p. 555

6.2 CHECK ★ Side 1 (operator-for-hire) does *not* require durable identity in the same way sides 2 and 3 do. Why? (Hint: who is the counterparty, and who bears the consequence?) *Solution p. 555*

6.3 OPEN ★★★ Construct a market design in which sides 2 and 3 exist *without* a non-forgeable identity. Argue informally why every such design collapses to a Sybil attack (forward-reference §6.5). *Solution p. 556*

§6.2 WHAT A “SIDE” IS, AND WHY THERE ARE THREE

6.4 CHECK ★ State the operational test for “how many sides” in one sentence, then apply it to a ride-share platform: how many sides, and why? *Solution p. 556*

6.5 TRACE ★★ Side 3 (skill licensing) keys reputation on a *content hash*, not a person. What breaks when a licensor ships v2 of a skill? (Hint: §6.13, skill versioning.) *Solution p. 556*

6.6 OPEN ★★★ A buyer reads a three-axis reputation vector (accuracy, aesthetics, efficiency). Specify a disclosure rule that lets the buyer weight the axes *without* re-introducing a single Goodhart target. Argue why every fixed scalar collapse re-creates the bandit framing the design rejects. *Solution p. 556*

§6.3 THE FLOAT PLAN: THE BUILT FLOOR

6.7 TRACE ★★ Trace the flow of funds for a **Partial** settlement where the worker completed 40% of the evidence chain. Confirm Design Invariant 6.3.1 holds across the split (the dramatization above is one such trace; redo it for a bond of 250 CR and 70% completion). *Solution p. 556*

6.8 CHECK ★ Why does closure bind to an oracle rather than a self-reported note? Name the attack a free-text “Result:” note enables. *Solution p. 557*

6.9 OPEN ★★★ The four states assume the acceptance criteria were *right*. Design a *float-plan amendment* protocol for the common case where the criteria themselves turn out wrong mid-flight (re-sign by both parties, escrow top-up/refund delta), preserving Design Invariant 6.3.1. (Gap, §6.13.) *Solution p. 557*

§6.4 THREE SIDES ON ONE ESCROW

6.10 CHECK ★ Map each of Grossman–Hart’s two parties (residual-control-right holder vs. reputational-consequence bearer) onto the rental side’s roles. Which one is the Arbiter jail? *Solution p. 557*

6.11 TRACE ★★ Show that adding a lease line item and a metered-license line item to a float plan cannot violate Design Invariant 6.3.1, no matter the amounts. *Solution p. 557*

6.12 OPEN ★★★ A skill's portfolio proof for v_1 says nothing about v_2 . Specify a *version-binding* for the Merkle portfolio proof so a new version starts with a discounted inherited prior — the newcomer policy, but for code. Solution p. 557

6.13 TRACE ★★ The deontic layer named above — what an agent *may* or *must* do, as against the jail's *capability* — needs a decision procedure once two accepted norms might disagree. Take the conflict witness worked by hand above (fact `agent_deployed`; Horn rule to `prod_access`; an obligation and a forbiddance on `write_prod` overlapping on $[5, 10]$), and add one *disjunctive* obligation $O(\ell_1 \vee \ell_2)$ over two fresh literals unrelated to the existing rules. Try the three nonempty discharge selections (ℓ_1 alone, ℓ_2 alone, both) by hand. Does any selection make the `write_prod` conflict go away? What happens on the selection that asserts both ℓ_1 and ℓ_2 , and why does Theorem B4.2's reduction from 3-SAT depend on exactly that happening? Solution p. 558

§6.5 THE KEYSTONE THE MARKET RESTS ON — AND DOES NOT YET HAVE

6.14 CHECK ★ In one sentence, state why a reputation system layered on forgeable pseudonyms is *no mechanism*, not merely a weak one (cite Douceur). Solution p. 559

6.15 CHECK ★ Local non-forgeable identity has a non-goal: “no defense against a hostile operator.” Name the exact word in the market's definition (“mutually-distrusting,” “boundary neither controls”) that this non-goal contradicts. Solution p. 559

6.16 OPEN ★★★ Specify a cryptographic binding tying N agents to one principal such that spawning fresh agents does not reset the bond floor, and that survives into the cross-operator world (where the single-operator “trusted credential” shortcut is unavailable). This is the hard half of cross-operator attestation. Solution p. 559

§6.6 CONSERVATION UNDER COMPOSITION

6.17 CHECK ★ State Myerson–Satterthwaite in one sentence and list the assumptions that must be checked before applying it to one harbor trade. Solution p. 559

6.18 TRACE ★★ Give a two-currency example in which native-unit conservation holds but the marked-to-market scalar changes because v_t moves. Solution p. 560

6.19 OPEN ★★★ Specify an oracle-latency and price-move model that yields a valid φ exposure bound for Property 6.6.1. Solution p. 560

§6.7 THE ANCHOR PROTOCOL PROPER: CROSS-HARBOR CAPABILITY TRANSFER

- 6.20** CHECK ★ Why is the ceremony four messages and not two? Identify what message (3) buys (Hint: who must counter-sign, and under whose key does C'_B later verify?). *Solution p. 560*
- 6.21** TRACE ★★ Trace what happens to an in-flight C'_B when Alice rotates her signing key between messages (3) and a later presentation at B . (Gap: cross-boundary key rotation, §6.13.) *Solution p. 560*
- 6.22** OPEN ★★★ Specify replay protection across the boundary: is the *anchor id* a nonce? Does B reject a re-presented message (3)? State the freshness invariant. *Solution p. 560*

§6.8 FEDERATION: TRADING ACROSS MACHINES YOU DON'T OWN

- 6.23** CHECK ★ State the connectivity, peer-selection, and delivery assumptions needed for expected $\Theta(\Delta \log m)$ dissemination. Explain why none yields a deadline during an unbounded partition. *Solution p. 561*
- 6.24** TRACE ★★ Trace a revoked capability through the revocation-gossip figure in *The Federated Harbor*: at which harbor, and at what time, can it still be spent? That window is what Conjecture 6.8.1 must price. *Solution p. 561*
- 6.25** OPEN ★★★ State and (attempt to) prove the Equivocation Lower Bound for a three-harbor mesh under a fixed gossip period Δ and a single equivocating witness. *Solution p. 561*

§6.9 REPUTATION: MONOTONE, BUT REVOCABLE

- 6.26** CHECK ★ State the signal-model assumption Theorem 6.9.2 needs and explain why an omitted strategic grader leaves it undefined. *Solution p. 561*
- 6.27** CHECK ★ Classify three settlement criteria as machine-checkable (Fix A) or judge-dependent (Fix B): a unit test passing; a merged SHA; “the refactor is tasteful.” *Solution p. 562*
- 6.28** OPEN ★★★ Prove the rate-the-raters recursion terminates: define a well-founded order on “levels of judge,” grounded at machine-checkable oracles, and show no infinite ascending chain of “judges judging judges” is needed. *Solution p. 562*

§6.11 COLD START, AND THE AUCTION QUESTION

- 6.29** CHECK ★ Which side should Phase 1 subsidize, and why? Apply the “scarcest cross-side externality” rule. *Solution p. 562*

6.30 TRACE ★★ Use the cartel game in *The Bonded Common* to identify the grim-trigger condition under which the cartel is self-sustaining. What raises the critical discount factor δ enough to break it? Solution p. 562

6.31 OPEN ★★★ “Price of anarchy when reputation is for sale” is the wrong tool: once reputation is tradeable, you have changed the game, so PoA (a ratio for a *fixed* game) does not apply. Reframe it as a *signal-existence* question (Spence-signaling with a resale market): does *any* informative separating equilibrium survive reputation resale? Solution p. 562

§6.12 A GLUING ANALOGY FOR THE OPEN FEDERATION PROBLEM

6.32 CHECK ★ Name the site, cover, restriction maps, and coefficient object a genuine sheaf model of federated ledger views would require. Solution p. 564

6.33 CHECK ★ Explain why two incompatible signed roots are evidence of equivocation but are not, without further definitions, a cohomology class. Solution p. 564

6.34 OPEN ★★★ Build a small formal gluing model for a bounded gossip topology and state exactly what consistency result it proves. Solution p. 564

6.15 Related work

The design borrows from seven literatures, and its only originality is in how it *joins* them; this section consolidates the threads cited piecemeal above.

Multi-sided platform markets. The frame is Rochet & Tirole [217, 218] and Armstrong [13]: a platform is multi-sided when the *allocation* of price across user groups, not just its level, moves volume. We extend the standard two-sided picture to three sides by counting *incentive constraints*, and we read the contemporary AI-labor-market model of Chiu, Zhang & van der Schaar [49] as confirmation that “reputation system as a third side” is the natural structure.

Mechanism design and its impossibilities. Settlement is a direct mechanism in the sense of Myerson [178]; the binding constraint is Myerson–Satterthwaite [179], which forces the market to surrender efficiency to keep budget balance, incentive compatibility, and individual rationality; the worked no-trade region in §6.6.1 instantiates the impossibility on the linear double auction of Chatterjee & Samuelson [46]. The incentive-compatibility arguments are imperfect-monitoring folk

theorems (Green–Porter; Abreu–Pearce–Stacchetti [3]), whose exogenous public signal we cannot take for granted — hence the grading-oracle problem. Grossman–Hart [97] supplies the residual-control-right argument for the rental side, and Akerlof [7] the lemons argument for licensing.

Reputation systems. Empirically, online reputation has measurable value (Resnick et al. [214]; Tadelis [248]); theoretically, unbounded records create reputation *bubbles* rather than stability (Liu & Skrzypacz [155]), which is why our horizon is a tunable. The Sybil attack [68] is the reason reputation must key on a principal, not a spawn. LLM-as-judge de-biasing follows Zheng et al. [278].

Distributed systems and cross-chain commerce. Federation refuses consensus because of FLP [81], recovers a weaker guarantee under partial synchrony [71], propagates by epidemic gossip [60], and takes a Certificate-Transparency detect-don’t-prevent posture [146, 145], mindful of eclipse attacks [104]. The settlement design is located explicitly against atomic cross-chain swaps [108] and the adversarial-commerce model of Herlihy, Liskov & Shrira [109]; reputation revocation is contrasted with Sagas-style compensation [86].

Applied category theory. The trace-composition notation and proposed gluing formalization draw vocabulary from ologs [244] and Fong & Spivak [82]; no cohomology claim is made. The commons-governance reading draws on Ostrom [185], and the auction machinery on standard algorithmic game theory [181].

Agentic-commerce protocols. While this design was being written, the industry shipped the transaction layer of an agent economy. The Universal Commerce Protocol [261] (Google/Shopify, 2026) standardizes agent-mediated retail — discovery via a `.well-known` capability profile, a checkout state machine, server-selects version negotiation — and its rival, the Agentic Commerce Protocol [6] (OpenAI/Stripe), the platform-mediated equivalent. Both are deliberate about what they exclude: neither settles payments (delegated to payment service providers) and neither decides which agents deserve trust. Authorization proof is delegated to the Agent Payments Protocol [5], whose mandates — W3C Verifiable Credentials in SD-JWT form, chained principal → platform → merchant — are the deployed sibling of this paper’s float-plan countersign, and whose credential formats the reference implementation now adopts at the harbor boundary (ADR-0094) so that the keystone identity artifacts of §6.5 verify with off-the-shelf tooling. The gap those protocols leave open is this paper’s subject: published security analyses of UCP [58] note that it regulates *how* agents transact while offering nothing about *which* agents to trust — no collateral,

no settlement oracle, no reputation, no priced deterrent. The harbor economy is a mechanism for exactly that layer; the retail protocols are evidence the layer below it is arriving on schedule, and hosted trust (§6.10) is what they will need bolted on.

Interactive proofs and adversarial debate. The adversarial test market (§6.16) is a seventh thread, cited piecemeal there rather than above because it grounds a single additional mechanism rather than the market's core structure. It is delegated computation in the sense of Goldwasser, Kalai & Rothblum [92] — a resource-bounded verifier need not redo a prover's work — crossed with multi-agent debate [119], where two adversarial reasoners are more legible to a judge than one cooperative one. Becker's deterrence condition [25] supplies the enforcement half: a test-writer's expected forfeiture must match its gain from misreporting.

Table 6.3: How the harbor economy sits relative to the nearest prior art on each axis.

Axis	Nearest prior art	What the harbor does differently
Market structure	Two-sided platforms [217]	Three incentive constraints (labor / capital / IP) on one escrow object
Settlement across distrust	Atomic swaps [108]; cross-chain deals [109]	Bounded (not trustless) escrow for <i>non-fungible, reputation-priced</i> bonds
Revocation	PKI revocation lists; Certificate Transparency [146]	Gossip-converged filters with a priced, named attacker window
Reputation revocation	Sagas compensation [86]	Tombstone propagation — no conserved quantity to compensate
Grading / IC signal	Folk theorems [3]	Strategy-proof machine-checkable oracle <i>or</i> bonded-and-slashed judges
Agent transaction rails	UCP [261]; ACP [6]; AP2 mandates [5]	They standardize the transaction and name trust out of scope; the harbor prices and enforces the trust layer they delegate away
Purchased assurance	Delegated computation [92]; AI-safety debate [119]	Prices assurance as <i>k</i> independently bonded test-writers under a Becker-style re-audit rate, against a mutation-testing oracle

6.16 One further market: adversarial testing and purchased assurance

Beyond labor, capital, and IP licensing, the harbor supports one further internal market, and it is worth stating because it is the one place where *assurance itself* becomes a priced line item rather than a hope. In an **adversarial test market**, independent test-writing agents compete to find flaws in a proposed diff *before* it settles. The frame is an interactive proof system: delegated computation [92] (a prover convinces a resource-bounded verifier without the verifier redoing the work) crossed with multi-agent debate [119] (two adversarial reasoners are more legible to a judge than one cooperative one).

Numbers by hand — *The adversarial test market, dramatized.* Bob's float plan ships a diff that one of Alice's specialists wrote. Rather than trust Alice's own test run — a signal whose author has every reason to flatter it — Bob posts a 30 CR bounty per confirmed flaw and opens the diff to $k = 3$ independent test-writing agents, each paid only for a defect it actually surfaces. If each catches a given real defect independently with probability $d = 0.6$, the chance a planted flaw survives all three is $(1 - 0.6)^3 = 0.064$. Bob has bought his defect risk down from 1 to about 6% for at most $3 \times 30 = 90$ CR, which is a number he can hold up against what the bug would have cost him in production. Raise k to 5 and the survival probability falls to $(0.4)^5 = 0.010$; that is the whole shape of the mechanism — assurance is bought in units of k , and each unit costs one bounty plus the carry on one bond. *Now you try:* hold $d = 0.6$ fixed and find the smallest k that pushes the survival probability below 1%.

A **Becker-enforced panel** is a set of k independent, conflict-free test-writing agents, each detecting a given flaw with probability $d \in (0, 1)$, each posting a slashable bond B to enter the market, and each facing a platform re-audit rate ρ against a one-shot misreporting gain G . The panel size k and the detection probability d fix what the panel *catches*; the triple (G, B, ρ) separately fixes whether a test-writer is honest at all. The two questions are independent, so the paper states them as two results rather than one bundled claim.

THEOREM 6.16.1 Purchased Assurance Bound. Let an author-agent submit a float-plan implementation with a proposed diff, and let a Becker-enforced panel of size k review it. Then the probability that a critical flaw survives undetected across all k reviewers is bounded by

$$\Pr(\text{defect undetected}) \leq (1 - d)^k. \quad (6.1)$$

The operator's assurance level $1 - (1 - d)^k$ is then a purchasable commodity priced at $k \cdot (\text{bounty} + \text{bond carry})$, which is what makes "hosted

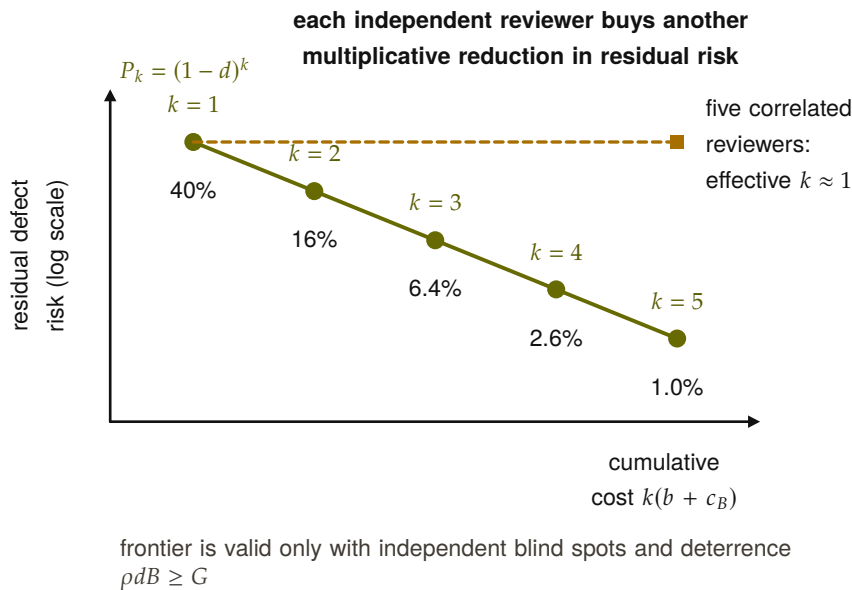


Figure 6.11: Purchased assurance is one risk–cost trajectory. Each gold point adds one independently bonded reviewer: cumulative cost rises linearly while residual defect risk falls geometrically. At $d = .6$, three reviewers leave 6.4% residual risk and five leave about 1%. The dashed amber counterfactual charges for five correlated reviewers but delivers roughly one reviewer’s protection. Independence and Becker deterrence $\rho dB \geq G$ are therefore conditions of the frontier, not footnotes to headcount. [internal]

trust as a service” a quantifiable line item rather than a slogan.

THEOREM 6.16.2 Becker deterrence for the panel. Truthful effort is a dominant strategy for a Becker-enforced panel’s test-writers exactly when expected punishment at least matches the one-shot gain from cheating — the **Becker deterrence condition** [25]:

$$\rho dB \geq G, \quad (6.2)$$

equivalently, the platform must commit to a re-audit rate at or above the critical rate $\rho^* = G/(dB)$. (SPECIFIED; this half is imported, not proved here — see the paragraph below.)

The deterrence condition in Theorem 6.16.2 is not proved here; it is imported. The companion technical paper *Reputation is Amortized Verification* [213] derives $\rho^* = G/(dB)$ as the stage-game threshold of a bonded inspection game — the same game this paper plays in §6.9 when it bonds and slashes a judge — and works it at the program’s running parameters: $G = 10$, $d = 0.8$, $B = 50$ give $\rho^* = 10/(0.8 \cdot 50) = 0.25$. Audit one report in four and the cheat payoff is exactly zero. Double the bond to $B = 100$ and ρ^* halves to 0.125: capital and vigilance are substitutes along the frontier $\rho dB = G$.

In the notation of the audit tower of *From Spawn to Person*, this is the single-level case. There the bribe floor is written $\beta = \rho dB$, the gain at level k is G_k , and level $k+1$ audits level k across C sealed cliques, so the condition here reads $\beta \geq G_0$ and the contraction $G_{k+1} = (1 - \rho d) G_k$ is what makes a finite stack of audited auditors converge. The letter k is overloaded between the two chapters: here it counts the panel, there it indexes the level.

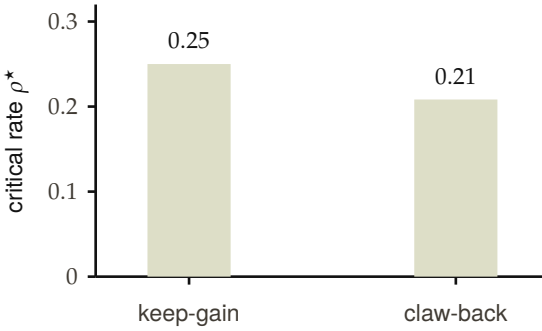
Quote the threshold without naming your settlement convention and you will be reproducibly wrong by about 17%. Equation (6.2) is the *keep-gain* convention: a caught cheat forfeits B but keeps G . If the escrow also claws the gain back, the forfeiture is $G + B$ and the critical rate drops to $\rho_c^* = G/(d(G + B)) = 0.2083$ at the same parameters. A platform whose rail does claw back but which budgets audits at $G/(dB)$ is over-auditing; one that does not claw back but budgets at $G/(d(G+B))$ is under-detering. The convention is a property of the slashing contract, not a modeling taste [213].

PITFALL

Figure 6.12 puts the two critical rates side by side at the program’s own parameters.

The $(1 - d)^k$ bound is exactly as good as the word *independent* in its hypothesis, and independence is the assumption a real test market breaks first. Three test-writers that read each other’s public findings, or that

PITFALL



same $G = 10$, $d = 0.8$, $B = 50$ in both bars; only the forfeiture on a caught cheat changes, B versus $G + B$

Figure 6.12: Quoting the wrong convention misprices deterrence by about 17%. Keep-gain forfeits only the bond B on a caught cheat: critical rate $\rho^* = G/(dB) = 0.25$. Claw-back also confiscates the gain G , dropping the critical rate to $G/(d(G + B)) = 0.2083$. A platform that claws back but budgets audits at the keep-gain rate over-audits; one that does not claw back but budgets at the claw-back rate under-deters. [internal; harbor3]

are three instances of the same model family with the same blind spot, do not give k independent draws — they give something closer to one draw and two echoes. Purchased assurance therefore has to buy *diversity* (distinct principals, distinct model families, sealed submissions) and not merely count k ; otherwise the priced assurance level is an overstatement of the delivered one. The same requirement appears in [213] as the sealed multi-clique sampling hypothesis, where it is likewise required rather than hygienic.

6.16.1 Putting a parameter on the echo

“One draw and two echoes” is the right intuition and it is not yet a number, and an operator buying assurance by the reviewer needs the number, because the whole point of Theorem 6.16.1 is that assurance has a price. Give the echo a model. Let the panel’s per-reviewer detection rate be a *random* quantity D with mean d , drawn once per panel and shared by its members — that is what a common blind spot is, a draw the panel makes together rather than one each. Then the miss probability is $\mathbb{E}[(1 - D)^k]$, and the first thing to notice is a one-line inequality with an uncomfortable direction:

$$\Pr(\text{defect undetected}) = \mathbb{E}[(1 - D)^k] \geq (1 - d)^k, \quad (6.3)$$

by Jensen, since $x \mapsto (1 - x)^k$ is convex. The independent bound is not conservative — it is *optimistic*, always, for every mixing distribution with the same mean. Correlation cannot help.

To get a closed form take $D \sim \text{Beta}(a, b)$, whose intra-panel correlation is $\theta = 1/(a + b + 1)$ and whose mean is $d = a/(a + b)$, so a target (d, θ) pins $a = d(1/\theta - 1)$ and $b = (1 - d)(1/\theta - 1)$. The miss probability telescopes into a product a reader can evaluate on paper — it is a ratio of Beta functions, but it is easier to use and easier to check in this form, and it avoids colliding with the bond B :

$$\Pr(\text{defect undetected}) = \mathbb{E}[(1 - D)^k] = \prod_{j=0}^{k-1} \frac{b + j}{a + b + j}. \quad (6.4)$$

Numbers by hand — what correlation costs a three-reviewer panel. Take the chapter's running $d = 0.8$ and the pitfall's own panel of $k = 3$. Under independence the bound is $(1 - 0.8)^3 = 0.008$: eight misses in a thousand. Now set $\theta = \frac{1}{2}$, so $1/\theta - 1 = 1$ and $(a, b) = (0.8, 0.2)$. Equation 6.4 is three fractions: $\frac{0.2}{1.0} \times \frac{1.2}{2.0} \times \frac{2.2}{3.0} = 0.2 \times 0.6 \times 0.7\bar{3} = 0.088$ [verified, closed form] — eleven times the independent number, from a panel the ledger still counts as three. Push to the monoculture limit $\theta \rightarrow 1$ and the product tends to $1 - d = 0.2$, which is precisely “one draw and two echoes” written as arithmetic: three reviewers deliver exactly the assurance of one, a factor of 25 worse than the line item says. *Now you try:* at $k = 5$ and $\theta = \frac{1}{2}$, does adding two more correlated reviewers buy more or less than the third one did? (Two more fractions, $\frac{3.2}{4.0} \times \frac{4.2}{5.0}$, multiply the miss probability by $0.8 \times 0.84 = 0.672$, giving 0.059 — against $0.2^2 = 0.04$ for the independent case's next two, so much less: correlated panels have sharply diminishing returns, which is the whole argument for buying diversity instead of headcount.)

Two things this does *not* license. It is an empirical hypothesis rather than a theorem about any deployed panel, and θ is not measured anywhere in this series — what would measure it is disagreement telemetry on deployed panels, and until that exists θ is a knob for pricing sensitivity, not a calibrated constant. And the Beta mixture is a choice, made for its closed form; Eq. 6.3 is the part that holds for *every* mixing distribution and is therefore the part to quote. Note also the symbol: θ is intra-panel correlation, not the audit rate ρ of Eq. 6.2. *From Spawn to Person* runs the same model against its inspection tower, where the tower's clique count C is the instrument that holds θ down.

The ground-truth oracle for grading the test-writers themselves is **mutation-testing kill rates** against held-out syntactic mutation operators, which stops agents from overfitting tests to a suite they can already see. This is a Fix A oracle in the sense of §6.9: machine-checkable, and not authored by the party being graded.

(*proposed*. The mutation-testing oracle and the bounty-per-killed-mutant contract are specified here for the first time in this series; neither ships, and neither appears in the reference implementation's ledger module. The deterrence inequality it leans on is SPECIFIED, proved in [213]. Recorded as such in Table 6.4.)

6.17 Conclusion

The harbor-master makes a swarm legible and accountable for one operator (the wedge of *The Legible Swarm*). When operators sail out to trade, the *same* ledger that kept one operator's swarm honest becomes the market that lets fleets who don't trust each other work together: three sides, one bond, hosted trust. We have not papered over the cost of that claim. The keystone identity primitive — local non-forgable identity — covers a single-operator threat model and must be extended to cross-operator attestation before the boundary is formally one "neither controls." Conservation composes exactly within each unit of account; cross-currency totals require time-indexed valuation and an explicit exposure model. Reputation is monotone in honest outcomes and revocable by tombstone, not by Sagas. Every folk-theorem result rests on a grading process included in the monitoring model. Myerson-Satterthwaite constrains bilateral slices only when its assumptions hold; the paper does not yet prove which corner the full market sacrifices. The bones are good and the floor is built; the spine the market trades on has partial local identity and outcome substrates but no complete reputation or cross-operator binding — and this paper says so in the same words throughout, because a market built on an overstated keystone is a market built on sand.

RECALL

1. Without looking: state the Purchased Assurance Bound in one line, and name the one word in its hypothesis a real test market breaks first.
2. What is the difference between the Becker deterrence condition's "keep-gain" and "claw-back" conventions, and why does it matter which one you quote?
3. What is the ground-truth oracle for grading the test-writers themselves, and why does it have to be Fix A rather than Fix B?

Chapter Appendices

6.A Implementation & status

The body of this paper is deliberately implementation-agnostic. This appendix records, for the reader who wants to check the claims against running code, exactly which mechanisms are shipped in the reference implementation, which are specified with a proof or a written protocol, and which are still proposed. The single-line summary: an **IMPLEMENTED** conserving bond ledger, an **IMPLEMENTED** continuity subsystem (episodic memory), a **PARTIAL** checkpoint, and a richly **SPECIFIED-and-model-verified** protocol stack — but the spine the market trades on (cross-operator identity → outcome ledger → reputation) is **PARTIAL-to-proposed**. Today the harbor economy is a two-sided market with a proof-backed roadmap to three.

Numbers by hand — *One trace of the ten thousand the property test runs.* The table below grades bond-ledger conservation **IMPLEMENTED**, grounded in a property test over ten thousand random operation traces (Design Invariant 6.3.1); here is the smallest trace by hand so “property test” is not a black box. Escrow a 100 CR bond (debit 100 from wallet, credit 100 to escrow: $\Delta = 0$, value only moved buckets), then refund it on clean exit (debit 100 from escrow, credit 100 back to wallet: $\Delta = 0$ again). Composing the two traces gives $\Delta = 0 + 0 = 0$ and wallet + escrow + commons returns to its starting value — one instance of the 10,000 the property test checks, none of which the daemon is trusted to get right by assertion alone.

Table 6.4: Per-claim maturity, with concrete grounding in the reference implementation. “Module” names refer to source files in the implementation; “model” means a mechanized proof artifact; “property test” means a randomized test exercising the invariant.

Claim / mechanism	State	Grounding
Bond ledger: escrow, slash, commons pool	IMPLEMENTED	bond-ledger module (lib/bonds.ts)
Conservation wallet + escrow + commons = supply	IMPLEMENTED	10k-trace property test
No-spawn-without-bond	PARTIAL	escrow-first; runtime gate is a roadmap item
Episodic memory (continuity subsystem 1)	IMPLEMENTED	episodic-memory module

(continued)

Claim / mechanism	State	Grounding
Resurrection / checkpoint (subsystem 2)	PARTIAL	passes notes, not state
Outcome ledger (subsystem 3 — reputation keys here)	PARTIAL	commitments, daemon-derived deadlines, oracle closure, API/CLI, and overdue monitoring ship; neutral grading and reputation binding do not ship
Local non-forgable identity (intra-harbor)	PARTIAL	actor-souls, lookup credentials, and a bounded newcomer pool ship; full write gating does not ship
Cross-operator attestation	SPECIFIED/ <i>proposed</i>	no spec for the critical part
Float plan + signed escrow ceremony	SPECIFIED	Protocol 6.3.1
Cross-harbor capability transfer (attenuation)	SPECIFIED	verified as model; <i>The Federated Harbor</i> , §8.4.1
Reputation estimator (Elo / Bradley–Terry / TrueSkill)	SPECIFIED/ <i>proposed</i>	no estimator shipped
Multi- dimensional reputation, neutral judges	<i>proposed</i>	no axis-aggregation spec
Three-sided market (labor / rental / licensing)	SPECIFIED	two-sided in reality
Skill licensing (metered + clawback + Merkle proof)	SPECIFIED	cross-harbor verifiability undeployed
Adversarial test market (Purchased Assurance Bound, Thm. 6.16.1)	<i>proposed</i>	no mutation-testing oracle and no bounty-per-killed-mutant contract shipped; the deterrence threshold it cites is proved in [213]
Federation: witness log, capability transfer	SPECIFIED	<i>The Federated Harbor</i>
Recipient- whitelisted escrow custody	SPECIFIED (conditional property)	requires non-bypassable authority; no runtime conformance

(continued)

Claim / mechanism	State	Grounding
Cross-harbor conservation	SPECIFIED (property)	proof sketch (App. 6.B) + TLA ⁺
Federated revocation dissemination	SPECIFIED (conditional expectation)	no finite partition deadline; equivocation delivery <i>open</i>
Hosted-trust moat / rail separation	SPECIFIED (positioning)	not a code artifact
Competitive underwriting (4th side)	<i>proposed</i>	composition unresolved

6.B Proof sketches

We sketch the three central results; full proofs of the conservation results appear in *The Federated Harbor*.

Numbers by hand — Theorem 6.6.1’s composition law, checked against the settlement it governs. The proof below is symbolic; here it is against a trace already worked in §6.3. Let f be the two escrow debits that fund the plan (Bob’s wallet −400, Alice’s wallet −100, escrow +500) and g be the **Partial** settlement that follows (escrow −500, Bob’s wallet +240, Alice’s wallet +200, commons +60). Both move value only between wallet, escrow, and commons, so $\Delta(f) = \Delta(g) = 0$ by the theorem’s own claim; composing, $\Delta(g \circ f) = \Delta(g) + \Delta(f) = 0 + 0 = 0$, exactly the 0 net delta the worked example confirmed by summing its four buckets directly. The theorem and the hand count are the same fact checked two ways. ■

Proof sketch of Theorem 6.6.1. For a valid trace $f : x \rightarrow y$, define $\Delta(f) = S(y) - S(x)$. For composable traces $f : x \rightarrow y$ and $g : y \rightarrow z$,

$$\Delta(g \circ f) = S(z) - S(x) = [S(z) - S(y)] + [S(y) - S(x)] = \Delta(g) + \Delta(f).$$

Every internal generating operation has zero delta because its debits and credits match, including a cross-harbor move through the explicit in-flight bucket. By induction on trace length, a composition of internal generators has zero delta; top-ups and withdrawals contribute exactly their recorded boundary amounts. □

Boundary note for Property 6.6.1. Native-unit conservation follows by applying Theorem 6.6.1 separately to each unit. A scalar valuation $V_t(x) = \sum_j v_{t,j} x_j$ changes either when native balances change or when the valuation vector v_t changes. The second term is mark-to-market exposure. No finite φ follows without assumptions bounding rate

movement, oracle delay, fees, and slippage; those assumptions are not supplied here. $\square$

Proof of Theorem 6.6.2 (the sacrificed corner). Restrict attention to a bilateral slice satisfying the assumptions stated in Theorem 6.6.2. The Myerson–Satterthwaite impossibility then rules out the simultaneous achievement of all four desiderata. The implication is conditional: before declaring which corner the harbor gives up, a mechanism must specify its types and transfers and prove its incentive and participation properties. The present paper has not done so, hence makes no stronger allocation claim. $\square$

WHAT THE BONDED COMMONS RECEIVES Work can now be proposed, bonded, graded, and settled against a named principal. The market's contracts rest on promises this chapter states but does not prove: that evidence is admissible, that settlement conserves value, that remedy stays bounded. The next chapter proves them.

7

The Bonded Commons

The question this chapter answers

Who may decide, on what evidence, with what consequence?

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.



If men were angels, no government would be necessary. If angels were to govern men, neither external nor internal controls on government would be necessary.

James Madison, Federalist No. 51, 1788

WHO MAY DECIDE, ON WHAT EVIDENCE, WITH WHAT CONSEQUENCE? Attribution without consequence is only a diary; consequence without admissible evidence is arbitrary. This chapter turns the local writer into an institution by joining capability bounds, witnessed records, audits, appeals, and conserved settlement. R7–R11, R15, R17, and the R8 machine state exactly what has been proved, checked, or left conditional.

7.1 Introduction: The Trust Problem

Two autonomous coding agents share a repository. Agent A is asked to refactor the authentication middleware. Agent B, spawned ninety seconds later from a sibling worktree, is asked to add a rate-limiter to the same middleware. Neither knows the other exists. They both edit, both commit, both push; one of them gets a merge conflict it cannot interpret, retries with a hallucinated resolution, and corrupts the file. The operator returns from a coffee break to a working tree she does not recognize. This is not a hypothetical—it is the modal failure mode of every multi-agent setup we have inhabited. Existing frameworks do not address it: LangGraph, CrewAI, and AutoGen handle state graphs and role assignment competently; the Model Context Protocol standardizes agent-tool integration. None of them give Agent A and Agent B a way to know about each other. The problem is **trust**—specifically, trust as infrastructure rather than trust as per-transaction ceremony.

Figure 7.1 draws this collision on the shared timeline: two independent lanes, ninety seconds apart, both landing on the same file.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A's downstream work. When Agent B accepts the delegation, it implicitly trusts that A's description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated.

The standard response in security engineering is “zero trust”—never trust, always verify. But zero trust applied literally to agent coordination is paralytic. If every message requires cryptographic verification, every delegation requires capability negotiation, and every file access requires authorization—the coordination overhead overwhelms the work itself. Agents spend their context windows negotiating trust instead of solving problems.

Hobbes identified this dynamic in 1651 [113]. In the state of nature, without a common authority, rational actors cannot cooperate because the cost of verifying each counterparty's intentions exceeds the benefit of cooperation. The solution is not to make individuals trustworthy—it

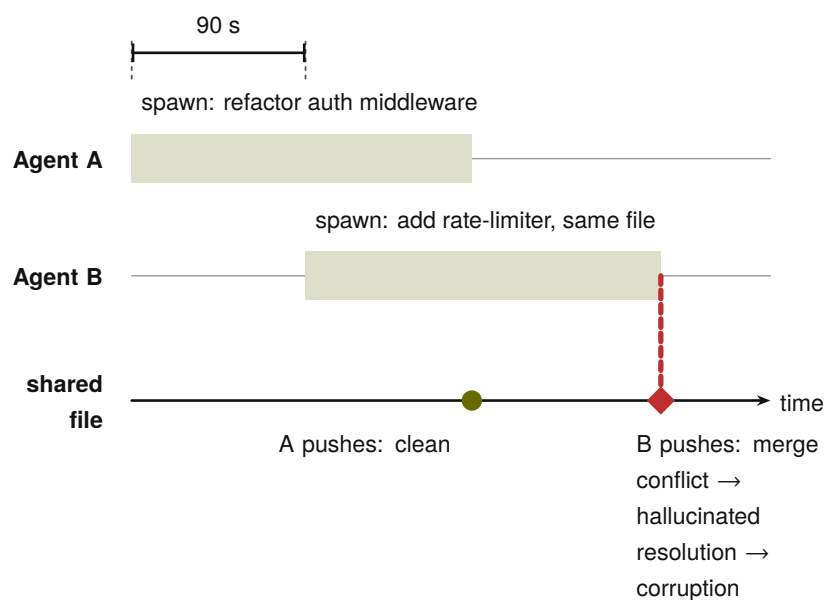


Figure 7.1: The opening scene collision. Agent A and Agent B never communicate — each lane runs independently, ninety seconds apart, the only hard number given. Both edit the same file; both commit; both push to the shared rail below. A’s push lands first and commits cleanly. B’s push lands second against a file A has already changed, produces a merge conflict neither agent can interpret, and a hallucinated resolution corrupts the file. [internal]

is to establish a **sovereign** whose authority both parties accept *before* the interaction begins, so that the interaction itself can focus on the work. “Covenants, without the sword, are but words, and of no strength to secure a man at all.”

This paper argues that multi-agent systems need exactly this: a **commons authority** that provides trust as infrastructure, not trust as ceremony. Agents don’t negotiate trust per-transaction. They enter a commons where trust has already been established—structurally, evidentially, and economically—and they work freely within that trust envelope.

7.1.1 The Three Failures of Peer-to-Peer Trust

Peer-to-peer trust fails for agent systems in three specific ways:

It doesn’t scale. If n agents must establish pairwise trust, the system requires $O(n^2)$ trust negotiations. Each negotiation consumes tokens, time, and context window. With 8 agents, this is manageable. With 50, it dominates wall-clock time. With 1000 (a production agent fleet), it is infeasible.

It doesn’t survive death. When an agent crashes, its trust relationships die with it. A successor agent that resumes the dead agent’s work must re-establish trust with every counterparty from scratch. The trust investment is non-transferable because it was encoded in ephemeral bilateral state, not in durable infrastructure.

It doesn’t address misaligned principals. An agent can be perfectly “trustworthy” in the sense that it faithfully executes its instructions—while its instructions come from a principal whose interests are adversarial to the commons. Peer-to-peer trust evaluates the agent’s behavior; it cannot evaluate the principal’s intent. The agent is a loyal soldier in someone else’s war, with a spotless reputation.

7.1.2 Contributions

We present:

1. A reading of the coordination daemon as a **candidate correlating device** in the sense of Aumann [17]: claim recommendations are private signals drawn from a common-knowledge distribution, while incentive compatibility remains conditional on explicit obedience inequalities (Section 7.2, esp. §7.2.1).
2. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 7.3–7.7).

3. A Sen-grounded **design warning** about decisive resource rights under private information, plus a separate completeness theorem for **advisory resource claims** (Section 7.5).
4. A TLA^+ **specification** of the coordination lifecycle with crash recovery, verifying safety and liveness properties (Section 7.10).
5. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 7.7).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol [187], an accompanying chapter. This paper builds the trust, governance, and economic layers on top of that cryptographic foundation.

7.1.3 Related Work in Crypto-Economic Bonding

The architecture sits at the intersection of four bodies of prior work, each of which we extend rather than replicate.

Bonded participation in distributed systems. Proof-of-stake consensus protocols (Casper [41], Tendermint [38], and the Ethereum 2.0 specification [79]) introduced *slashing*: validators post collateral that is forfeited on equivocation or liveness failure. The technique is the closest analogue to our bonded-commons settlement, and the proofs of soundness (a rational validator strictly prefers honest behavior when slash > deviation gain) are direct ancestors of §7.7. We depart from this lineage on three axes: (i) the adversary is co-located on a single machine, not a Byzantine network, so consensus reduces to a daemon write rather than a global commit; (ii) the bonded act is *coordination*, not validation—work that produces side effects in a shared file system, not signatures on blocks; and (iii) the principal is human-or-LLM mixed, so the mechanism must price two failure modes (malice and confusion) the blockchain lineage collapses into one.

Capability-based security. Dennis and Van Horn [62], Miller’s object-capability work [175], and the SPKI/SDSI line [73] provide the structural attenuation that we lift wholesale into the Anchor Protocol layer. Our contribution is composing capability attenuation with bonding: a capability without economic settlement still permits the holder to “correctly” execute a destructive action; bonding without capability bounding still permits unbounded blast radius before slash. Both layers are necessary.

Commons governance. Ostrom’s eight design principles for governing common-pool resources [186] predict that durable cooperation emerges only when boundaries, monitoring, graduated sanctions, and dispute resolution are co-located with the commons. The bonded commons satisfies these requirements as services of the daemon rather than as social institutions, and §7.5.3 renders the dispute-resolution and appeals path explicitly.

Cryptoeconomic mechanism design. Werbach [269], Buterin [40], and the broader DAO literature establish “code is law” as a posture toward enforcement. Our position is narrower and weaker on purpose: code is the *record*, not the law. Allocation decisions remain with the agents (Sen, §7.5); the daemon supplies the evidence and the settlement. This deliberate weakening is what makes coercive enforcement avoidable while keeping accountability intact.

Table 7.1 compresses the four paragraphs above into what is borrowed, what is departed from, and why.

What is novel. The synthesis: bonded participation + capability attenuation + advisory (not enforced) coordination + immutable attribution, packaged as localhost infrastructure rather than a global protocol. We are not aware of a prior system that combines all four in this form; each component has precedent, while the composition targets auditable coordination and attribution that survives identity disposal. Its welfare effects remain conditional on the explicit payoff and monitoring models developed below. Table 7.2 shows the composition.



Ostrom’s fieldwork on fisheries, pastures and irrigation showed the tragedy of the commons is a design failure, not a law of nature.

Table 7.1: Every borrowed result is re-used at a smaller scale than its origin assumed: the adversary is co-located rather than networked, the bonded act is coordination rather than validation, and the record is evidence rather than law. Nothing in the four lineages is discarded — each is narrowed by one assumption, and the narrowing is what makes the composition deployable on one machine.

Lineage	What we borrow	What we depart from	Why
Bonded participation (PoS slashing)	Slash->-deviation-gain soundness argument	Byzantine networked adversary; validation as the bonded act	Adversary is co-located on one machine, so consensus reduces to a daemon write, not a global commit
Capability-based security	Offline attenuation; authority only ever narrows	Capabilities alone as the whole story	A correctly-scoped token still permits a destructive in-scope write; bonding prices what the wall admits
Commons governance (Ostrom)	Boundaries, monitoring, graduated sanctions, dispute resolution	Social institutions as the enforcement medium	The four functions ship as daemon services (§7.5.3), not as norms an agent must internalize
Cryptoeconomic mechanism design	Priced deterrence; market-discovered parameters	“Code is law” as an enforcement posture	Code is the <i>record</i> , not the law: allocation stays with agents (§7.5); the daemon supplies evidence and settlement

Table 7.2: Three layers answer three different questions about the same attempted act. An out-of-grant act ends at Layer 1 before shared state changes; every admitted act crosses Layer 2 and acquires durable attribution, so admission creates an attribution duty; a harmful admitted act reaches Layer 3, where only bound evidence can reach settlement. [internal]

Layer	Question	What it does to the act	Terminal outcome
1 scope	can this act exist?	terminates an out-of-grant trajectory before shared state changes	refused
2 evidence	who did what?	binds every admitted act to a durable, addressable attribution	complete and recorded
3 collateral	who carries the loss?	prices a harmful admitted act against posted collateral; only bound evidence can reach settlement	priced settlement

7.2 The Commons Authority

Return to Agent A and Agent B from the introduction’s opening scene—the two agents who corrupted the same middleware because neither knew the other existed. Nothing about that failure required malice, and nothing about it would have been fixed by making either agent more careful. What was missing was a third party that both agents were already talking to: something that knew A had the file, could tell B so at claim time, and would still remember the whole episode after both processes exited.

That third party has to be settled *before* any interaction, not improvised transaction by transaction—otherwise establishing it is just the $O(n^2)$ trust negotiation of §1.1 under a new name. We call it the *commons authority*: one always-on service that every agent already trusts, precisely because it was there before any of them showed up. Five capabilities make it that, and the definition below names them.

Definition 7.2.1 (Commons Authority). A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity**: Every agent receives a cryptographic identity before participating.
2. **Capability bounding**: Every agent’s operations are structurally limited by attenuable tokens.
3. **Shared knowledge**: Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history**: All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement**: Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are “good.” It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark. The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes’s Leviathan does not micromanage citizens’ daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority’s value comes from its *guarantees*, not its complexity.

7.2.1 The Daemon as Correlating Device

Picture a crossing guard rather than a traffic light. A traffic light gives everyone the same instruction on a fixed cycle regardless of what is actually happening at the intersection; a crossing guard watches the specific cars and pedestrians in front of her and privately waves one through while holding another back. The daemon plays crossing guard, not traffic light: it sees the whole intersection (the claim graph) and issues one private recommendation per agent. But a crossing guard has no legal authority over anyone—drivers obey because obeying is better for them than the alternative, not because they must. Aumann’s test for whether such private recommendations actually change behavior, rather than being safely ignored, is the obedience condition below.

The daemon’s role therefore admits a precise game-theoretic test. Aumann [17] introduced the *correlated equilibrium*: a mediator draws a recommendation tuple $r = (r_1, \dots, r_n)$ from a joint distribution μ over action profiles and privately reveals r_i to player i . The distribution is a correlated equilibrium only if, for every player i , recommended action r_i , and unilateral deviation a'_i , the obedience inequality holds:

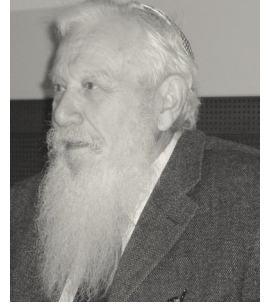
$$\sum_{r_{-i}} \mu(r_i, r_{-i}) [u_i(r_i, r_{-i}) - u_i(a'_i, r_{-i})] \geq 0.$$

Every mixed Nash equilibrium induces such a distribution, but a mediator does not create an equilibrium merely by issuing recommendations. The inequality is the governing condition.

The commons authority can play this mediator role. It observes the global claim graph and emits a recommendation pair on a contested file: Agent α , *proceed*; Agent β , *defer or coordinate*. A deterministic policy can still induce a non-degenerate μ through uncertainty over observed system states, but determinism is not itself evidence of obedience. Utilities, bond coverage, and continuation values must make the inequality above hold.

Property 7.2.1 (Conditional Correlating-Device Interpretation). Port Daddy’s daemon supplies the information structure of a correlating device for the agent-coordination game. It induces a correlated equilibrium only under the following conditions:

1. **Common-knowledge distribution.** The daemon’s recommendation policy—advise-claim, defer, back-off, settle-now, halt-on-conflict—is public, deterministic given the conflict graph, and reproducible from the evidence trail of §7.4. Agents do not need to trust the daemon’s motives; they need only verify the policy and observe its inputs.
2. **Private signal per agent.** On a contested claim, each agent receives only its own recommendation. Other agents’ recommendations are observed indirectly through the public conflict graph and the note trail, never read off the wire.



Aumann’s correlated equilibrium is the answer to when a private recommendation from a referee changes what a rational player does; the daemon’s authority rests on it.

3. **Obedience inequality.** For every recommendation and deviation, the deviation gain is no larger than the expected bond loss plus discounted reputation loss. Conflict-graph completeness makes a deviation observable; it does not by itself make the deviation unprofitable.

The reframe therefore yields an audit obligation, not an automatic guarantee. Nothing physically stops an agent from ignoring advice. The repeated-game calculation in Section 7.7 verifies one stylized two-player payoff table; deployments with different utilities must re-check the obedience inequalities. The TLA⁺ model in Section 7.10 checks life-cycle safety and liveness, not strategic optimality.

No price-of-anarchy bound follows from this interpretation alone. The simulations of §7.7.5.8 compare one auction model with one static baseline; an analytical worst-equilibrium bound for the coordination game remains open.

Formal-backing status of the game-theoretic core. One disclosure belongs here, at the point the game theory enters, rather than buried in the appendix: in the maturity vocabulary of Appendix 7.A, this chapter’s game theory is *spec-only at the level of the research program*, whatever each individual artifact’s status in Table 7.9. This chapter’s game-theoretic core—the correlating-device reading above, the Sen-inspired design warning (§7.5), the claim-signaling incentive-compatibility argument (§7.7.4), the competitive-insurance mechanism and its welfare comparison (§7.7.5.8), and the cartel folk-theorem analysis (§7.7.5.10)—has *no companion formal paper*. The structural and cryptographic half of this chapter is backed by *The Anchor Protocol* [187], which carries the ProVerif models and the attenuation proofs; the economic half is argued here, from scratch, and exists nowhere else in the program. What supports it is exactly what Table 7.9 lists—mechanized checks and Monte Carlo sweeps over supplied models, several of them PARTIAL by that table’s own grading—and not a peer-reviewable formal derivation a reader can be pointed to. Read this material at that maturity level.

7.2.2 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For AI agents, consent to the commons authority is rational because agents without it are strictly worse off:

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Agents that bypass the daemon are not punished—they are simply worse off.

Table 7.3: The commons authority converts six failure modes from ad hoc and manual to deterministic and automatic — without forbidding anything an agent could already do. Every row is a service the agent gains, not a permission it loses, which is why consent is rational rather than coerced.

Capability	Without Authority	With Authority
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

7.3 Layer 1: Structural Prevention

The first layer of trust does not depend on agent reputation or economic incentives. It depends on *walls*: structural authorization checks that reject out-of-scope operations wherever the runtime actually interposes them.

7.3.1 Capability Attenuation

The Anchor Protocol [187] provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a strict subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the authorization chain, never grow.

The capability subset check and the secret-independent comparison are implemented in a **Rust core** (`harbor-card-rs`) that is compiled to a shared library and called from the Node.js daemon via FFI. Kani harnesses check that the parser and the comparator cannot panic on bounded symbolic inputs and exercise the subset check on two concrete vectors; the every-hop attenuation property is ProVerif's, in *The Anchor Protocol*; runtime conformance tests cover the bridge. Compiler behavior, full interception coverage, and hardware side channels remain separate obligations.

The guarantee below is what security engineers mean by a *wall*

rather than a *law*: not an instruction an agent could disobey, but a bound on which operations are even representable.

Definition 7.3.1 (Capability-Bounded Transaction). An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card. The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

One point of vocabulary before the statement, because it decides which chapter owns what. Attenuation is *The Anchor Protocol's* result, not this chapter's: the condition that $C(\tau_{i+1}) \subseteq C(\tau_i)$ at every hop, that lifetimes only shrink, and that a chain cannot be spliced or replayed is proved there, mechanically, over the deployed crate. This chapter does not restate it, re-derive it, or define a competing notion — it *consumes* it, and the theorem below is what that guarantee buys once bonds and escrow sit on top. If the two ever disagree, chapter 2 is the one to trust; a second definition of attenuation living here is precisely the duplication the Book's chapter boundaries exist to prevent.

THEOREM 7.3.1 Structural Scope Limitation. For any authorization chain $\tau_0 \rightarrow \tau_1 \rightarrow \dots \rightarrow \tau_k$ satisfying *The Anchor Protocol's* attenuation condition, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i :

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \dots \subseteq C(\tau_0).$$

Within an execution that routes every relevant operation through the verifier, no accepted operation can lie outside the scope originally granted by the authority.

Proof. The Anchor model establishes a non-injective authentication correspondence for accepted modeled tokens and separately checks attenuation at the modeled chain depth [187]. Given those model assumptions and the theorem's premise that every operation passes through the verifier, each accepted hop satisfies the subset relation; transitivity yields the displayed chain. The correspondence alone does not prove replay freedom, arbitrary-depth delegation, or complete runtime interception, which are separate obligations. $\square$

Remark. This is the layer where the metaphor of “walls, not laws” applies. A law says “you shall not write to the production database.” A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

7.3.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git worktrees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 7.3.2 (Isolation Levels). Three tiers of isolation are available, ordered by strength:

- I_0 (**None**): Agents share a working directory. No conflict detection. Maximum parallelism, no safety.
- I_1 (**Advisory**): Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.
- I_2 (**Structural**): Each agent operates in a separate git worktree. Filesystem-level isolation ensures agents cannot observe each other’s intermediate states. Conflicts are deferred to sequential merge.

Figure 7.2 draws the three tiers as an ordered ladder, I_1 marked as the default.

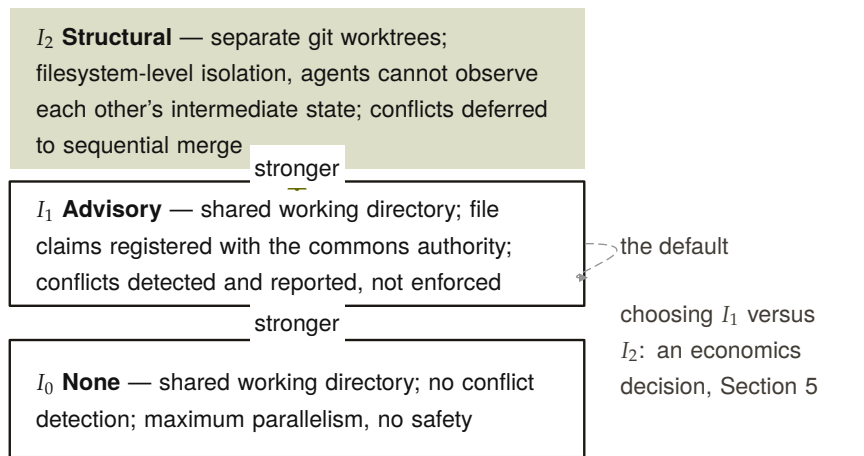


Figure 7.2: The three isolation tiers order strictly by strength, weakest to strongest: none, advisory, structural. I_1 (advisory) is the default — file claims are registered and reported but never block. Moving up buys more safety at the cost of parallelism; the choice between I_1 and I_2 specifically is not a security decision but an economics one, argued in Section 5. [internal]

The choice between I_1 and I_2 is not a security decision—it is an economics decision. Section 7.5 formalizes why.

7.3.3 Runtime Invariant Enforcement

Structural prevention is enforced at two levels: *statically* by the Anchor Protocol’s ProVerif-verified token exchange (all three protocol phases verified in ProVerif 2.05 [187], under the applied-pi-calculus methodology of Blanchet [33]), and *dynamically* by the **Arbiter**—an ambient security agent that subscribes to the daemon’s event stream and checks every state transition against six formally derived invariants. These invariants map directly to the safety properties of the TLA⁺ specification in Section 7.10: NoteMonotonicity, EscrowInvariant, and LockOwnerValid are compiled from the formal model into synchronous runtime checks. Violations are logged immutably and, in strict mode, trigger the salvage protocol—the sovereign’s sword in code form.

Regimentation vs. Enforcement (OP-2). This design formalizes **Synchronous State Decidability** as the exact boundary separating pre-commit *regimentable* rules from post-commit *enforceable* rules, in the sense of Jones and Sergot’s regimentation/enforcement distinction for normative systems [124]: a regimented norm’s violation cannot occur at all, while an enforced norm’s violation can occur and is sanctioned on detection. If a rule can be evaluated purely against deterministic local DB state and the transaction payload, it is regimented (structurally prevented). If it requires async checks, external I/O, or non-deterministic oracles, it is relegated to post-commit enforcement (detected and salvaged).

Figure 7.3 draws this boundary directly: the three compiled invariants on one side, the Arbiter on the other.

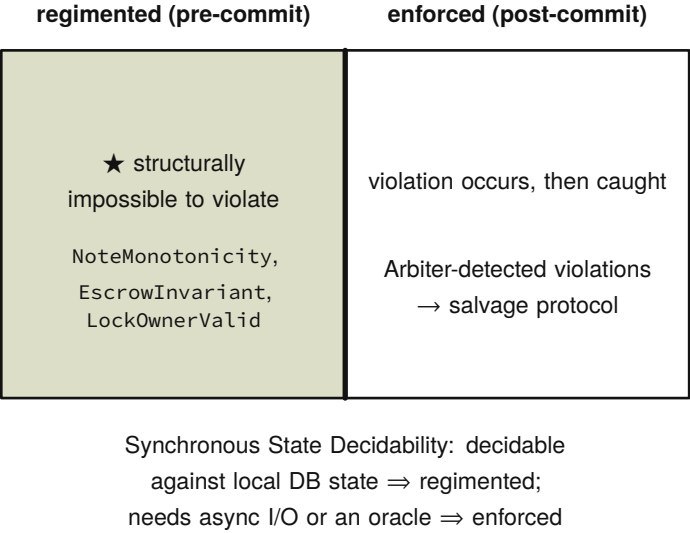


Figure 7.3: Regimentation versus enforcement, in the sense of Jones and Ser-got’s normative-systems distinction: a rule falls on one side or the other of Synchronous State Decidability, never both. The three compiled invariants (left, shaded, marked ★) are checked against deterministic local state and the transaction payload alone, so their violation cannot occur at all. Every other invariant needs an async check, external I/O, or a non-deterministic oracle, so the Arbiter (right) can only detect a violation after the fact and route it to the salvage protocol. [internal]

7.4 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

7.4.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 7.4.1 (Evidence Trail). An evidence trail $N = \langle n_1, n_2, \dots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

THEOREM 7.4.1 Trail Integrity. The evidence trail is immutable by construction. No UPDATE or targeted DELETE operation exists in the system’s API surface. Notes are destroyed only by explicit administrative deletion of the parent session (CASCADE), which is an auditable action distinct from normal transaction lifecycle operations.

Proof. By code inspection of the session module: the note insertion API performs only INSERT INTO session_notes. The only deletion path is DELETE FROM sessions WHERE id = ?, which triggers ON DELETE CASCADE. Since transaction completion (commit or abort) updates the session’s *status* field but does not delete the session row, notes survive all normal lifecycle transitions. $\square$

7.4.2 The Merkle Forest: Cross-Daemon Inclusion Proofs

For high-assurance environments and for fleets spanning multiple daemons, the evidence trail is strengthened to a three-level **Merkle Forest**. The single-daemon Merkle chain—each note hashing the previous, with a per-session root—is necessary but not sufficient: the moment two daemons exist, an auditor without database access still needs to verify that a particular note was included in a particular session using only a short proof.

Definition 7.4.2 (Merkle Forest). The evidence structure has three levels:

Note chain. Each note's header includes $h_i = H(n_i \parallel h_{i-1})$ over SHA-256. The session's note commitment is h_k , the hash of the last note.

Session root. All note hashes $(h_1, \dots, h_k)$ are assembled into a Merkle binary tree with root R_{session} . The chain commits to order; the tree commits to presence with $O(\log k)$ proofs.

Harbor root. Periodically (per-settlement, or hourly), every completed session root within a harbor is assembled into a harbor Merkle tree with root $R_{\text{harbor},\ell}$ at epoch ℓ , signed by the daemon's Ed25519 key.

The collection $\{R_{\text{harbor},\ell}\}_\ell$ is the *Merkle forest*: one tree per epoch.

Figure 7.4 draws all three tiers together, with the proof-size arithmetic set at the point a proof crosses from the session tier to the harbor tier.

Inclusion proofs. To prove that note n_i from session s belongs to harbor h at epoch ℓ , the daemon emits: (i) the note inclusion path of size $O(\log k)$, (ii) the session inclusion path of size $O(\log N)$ where N is the number of settled sessions in the epoch, and (iii) the signed harbor root $\sigma_{\text{daemon}}(R_{\text{harbor},\ell})$. For $k = 100$ notes and $N = 10^4$ sessions, the total proof is $\approx 20 \times 32$ bytes + 64-byte signature ≈ 700 bytes—small enough to embed in any settlement receipt.

Witness to an abstract KMS. Each $R_{\text{harbor},\ell}$ is uploaded as a **witness** to a Key Management Service satisfying the properties enumerated in §7.12. The KMS enforces append-only monotonicity and periodically publishes a signed tree head, in the Certificate Transparency pattern [145]. Equivocation—publishing different roots for the same epoch to different parties—is detectable by auditors performing consistency proofs across tree heads.

DESIGN INVARIANT 7.4.2 Proof Soundness. If $\text{VERIFY-NOTE-INCLUSION}(n, s, h, \ell, \pi)$ returns true for some proof π , then either (a) n was included in session s at epoch ℓ , or (b) the daemon's signing key has been compromised, or (c) SHA-256 has been broken. Under standard assumptions, (b) and (c) are computationally infeasible.

DESIGN INVARIANT 7.4.3 Binding. Once a daemon publishes $R_{\text{harbor},\ell}$, it cannot produce a different valid root for the same (harbor, ℓ) without

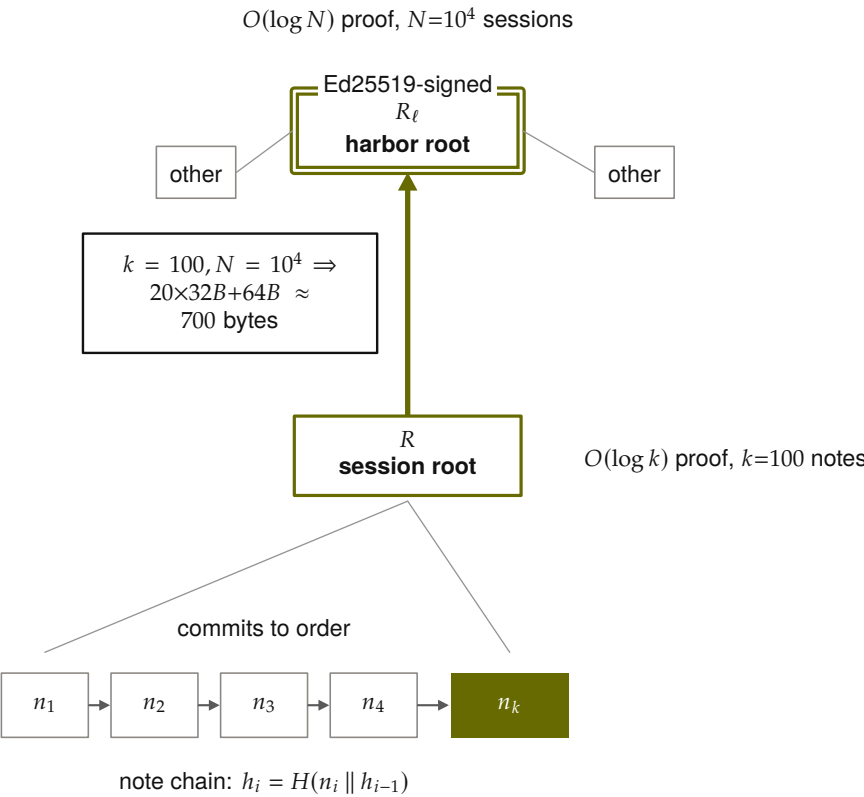


Figure 7.4: The Merkle forest: three nested evidence tiers. The note chain (bottom) commits to order within a session; the session Merkle tree (middle) commits to presence within that session; the harbor Merkle tree (top) commits to presence within a signed epoch across every settled session. The callout at left is the arithmetic where a proof crosses from the session tier to the harbor tier: at $k=100$ notes and $N=10,000$ settled sessions, the total proof is about 20 hashes plus one signature, roughly 700 bytes, small enough to embed in any settlement receipt. [hand-derived]

contradicting its earlier signature (detectable via the KMS witness) or forking its identity.

Cost. Each settlement adds an $O(1)$ amortized append to the epoch accumulator. Epoch rollover is $O(N)$ hashes plus one signature: at 100 sessions/hour, $\approx 10\text{ms}$. Negligible.

This is the structural meaning of “decentralized verification” in this paper: bonded commons evidence is not just immutable within a daemon, it is verifiable *across* daemons by any party with the daemon’s public key and a 700-byte proof.

7.4.3 Mutable-Signal Ledger (Pheromone Lineage)

Stigmergic coordination originated in biology, where pheromones are accumulate-only: ants deposit, the environment evaporates [250]. Software agents need a richer model. Coordination signals must be *revocable*, *renamable*, and *provenance-attributable* as understanding of the work evolves—an agent that deposited a hint and later realized the hint was wrong needs a way to retract it that preserves attribution rather than erasing it.

Event-sourced pheromones. Each entity’s pheromone state is a view over an append-only event ledger. For entity e and key k :

$$\sigma_t(e, k) = \text{decay}(S, t - t_0) \cdot \#[\text{not revoked or expired in } (t_0, t]],$$

where S is the last spray strength on k at time t_0 and decay is a geometric decay defined per-channel. **Revocation** is itself an event; **rename** is a rewrite-pointer event; **fork** creates a new key namespace whose provenance is traceable through the event chain.

DESIGN INVARIANT 7.4.4 Attribution Invariant. Every $\sigma_t(e, k)$ value has a deterministic provenance chain back to one signing principal, traceable via the event chain.

DESIGN INVARIANT 7.4.5 Rollback Safety. A consumer that reads σ at time t and later discovers a revocation event timestamped $t' < t$ can compute the correct counterfactual view by replaying events.

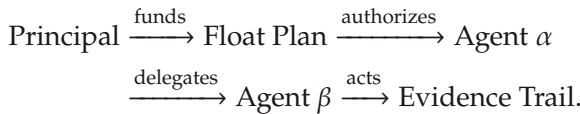
DESIGN INVARIANT 7.4.6 Conservation. Revocation does not erase. The event ledger is monotone: the cardinality of the event set only grows.

Integration with the forest. Pheromone events are included in the session tree of §7.4.2: each session’s root summarizes the events the session produced, harbor roots summarize session roots, and revocations are therefore transparent to witnesses without erasing any prior state. The mutable surface is the *view*; the underlying ledger is immutable.

This dual—mutable view, immutable substrate—is what lets coordination signals evolve without compromising the evidentiary guarantees the rest of the paper depends on.

7.4.4 Attribution Survives Identity Disposal

A critical property: even if an agent’s identity is discarded after task completion, the **authorization chain** traces every action back to the authorizing principal. The chain is:



The agent is ephemeral. The principal’s authorization—signed, escrowed, recorded—is permanent. Anonymous harm to shared state is impossible within the system, because every action chains back to someone who posted collateral; §7.7.5.4 draws the corresponding line for harm from *disclosure*, which this chain does not cover.

7.5 The Limits of Decisive Allocation

A natural question is why file claims are advisory rather than exclusive locks. Sen’s theorem illuminates the tension, but it does not prove that every lock manager is inefficient. The argument below separates the theorem from the engineering example.

7.5.1 A Sen-Inspired Design Warning

Sen [231] proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency:** If every agent prefers outcome *X* to *Y*, the system chooses *X*.

2. **Minimal liberalism:** Each agent has at least one “personal domain”—a pair of outcomes where the agent’s preference is decisive.

The analogy to file coordination is:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

THEOREM 7.5.1 A Pareto-Inferior Locking Instance. Consider agents α and β with tasks T_α and T_β . Agent α claims file f because it *might* need to modify it. Agent β discovers mid-task that it *actually* needs f . Under enforced claims:

- α ’s claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α ’s precautionary claim.
- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α ’s liberal right vetoes it.

This property is a counterexample to the universal claim that exclusive locks always improve team welfare; it is not a derivation of Sen’s theorem. Sen’s result requires an unrestricted preference domain and a social-choice rule with decisive personal pairs. A production lock manager may restrict preferences, negotiate releases, or time out claims. The usable lesson is narrower: do not grant an agent an unconditional veto over a shared resource when the team may possess Pareto-relevant information the allocator lacks.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to “fix the login bug” cannot predict whether it will need the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) → destroys parallelism.
2. **Under-claim** (request locks only on files they’re certain about) → produces undetected conflicts, defeating the purpose.

The lesson has a boundary, and it is worth drawing rather than asserting. Advisory claims are not better than locks everywhere; they are better in a region, and the region has a boundary. Where an agent’s file needs are knowable up front, an exclusive lock allocates exactly as an advisory claim would and adds determinism at no cost—in that regime the design choice of this section would simply be wrong. Where negotiating a release is prohibitively expensive, neither regime converges, which is the case the hard-lock fallback of §7.5.3 exists to absorb. The

advisory default is justified by where LLM coding agents actually sit, not by a universal claim.

7.5.2 Advisory Claims as Resolution

Advisory claims avoid granting that unconditional veto. They do not guarantee Pareto efficiency; they expose conflicts so informed agents can negotiate:

Definition 7.5.1 (Advisory Consistency). Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j).$$

The commons authority guarantees that every edge in G is reported to both endpoints. No agent has a decisive “personal domain” over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

THEOREM 7.5.2 Advisory Conflict Completeness. Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f , the commons authority reports $\text{conflict}(f)$ to S_2 at claim time. No false negatives exist.

Proof. The overlap detection query scans all active (unreleased) claims from other active sessions. A whole-file claim ($R = \perp$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$. Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. $\square$

Remark. The commons authority’s role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only the agents possess (“I claimed `auth.ts` but probably won’t need it”; “I absolutely must modify `auth.ts` for my task to succeed”). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

7.5.3 Governance, Disputes, and Appeals

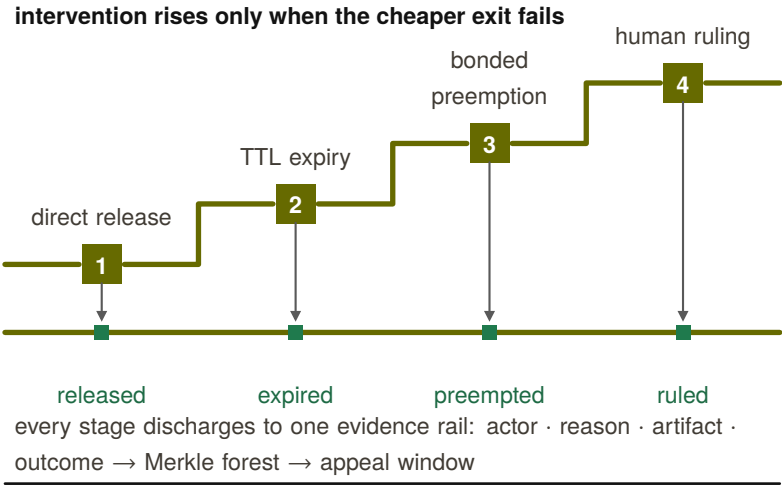
Advisory coordination produces consistent allocation when agents share information formally. It can still produce *disputes*: two agents legitimately need overlapping files; one agent's claim turned out to be precautionary and is now blocking real work; a third agent observed evidence that a claimed task has been abandoned. These disputes do not threaten the commons' integrity—attribution, capability bounds, and settlement records remain intact—but they do require a resolution path. Without one, advisory claims silently degrade into squatter rule (whoever claimed first holds indefinitely) and the Pareto benefit of §7.5 evaporates.

Resolution path. The commons authority provides four mechanisms, in order of increasing escalation (Figure 7.5):

1. **Direct release.** The original claimant releases the claim. This is the equilibrium when the conflict-graph signal reaches them and the claim was precautionary.
2. **Time-out release.** Claims carry TTLs. A claim whose holder has not heartbeated within the TTL window is released automatically; downstream agents observe the release in their next poll. This is the equilibrium for crashed or abandoned claimants.
3. **Bonded preemption.** A blocked agent may post an additional bond—bounded by the cleanup cost of §7.7.5.1—to force release of an active claim. The preempted claimant receives the bond as compensation; the preemptor accepts the risk that its preemption was wrong (in which case the bond is slashed under settlement). Preemption is rare by design: the bond is large enough that an agent prefers waiting to preempting unless the wait cost dominates.
4. **Human escalation.** If preemption is ambiguous or the stakes exceed the preemption-bond ceiling, the dispute escalates to a human operator via the request channel (§7.8). The operator's decision is appended to the evidence chain with the same Merkle-Forest binding as any agent action; the resolution is auditable by all parties post-hoc.

Two mechanisms sit underneath this escalation path rather than inside it: a thrashing fallback for advisory claims that never converge, and a fairness rule for the preemption queue itself.

The Advisory Claims Paradox and Deterministic Hard-Lock Fallback. Advisory claims risk an endless retry/thrashing loop if two LLM-backed agents mutually ignore advisory warnings. To break this paradox, the daemon implements a **Deterministic Hard-Lock Fallback**: if advisory conflicts on a resource exceed k turns without resolution (default $k = 5$, tuned per deployment from the observed conflict-resolution distribution), the daemon escalates the advisory claim into an enforced



stage	escalation trigger	
1	direct release	claimant yields voluntarily
2	TTL expiry	heartbeat absent past its TTL
3	bonded preemption	waiting cost exceeds the posted bond
4	human ruling	ambiguous claim or high stakes

Figure 7.5: Dispute resolution is an ordered four-stage escalation ladder. Each stage badge is keyed to the escalation trigger in the legend, and every stage outcome – released, expired, preempted, or ruled – discharges onto the same evidence rail regardless of which stage resolved the dispute. Escalation therefore changes only the decision cost and decision-maker, never the record kept for appeal. [internal]

POSIX file-level lock, rejecting all subsequent writes from un-holding agents until release.

Stigmergic Ticket-Lock (OP-1: Fair Exclusion). To prevent starvation during high contention, the daemon replaces chaotic retries with a **Stigmergic Ticket-Lock**. The lock is managed via a strict SQLite schema:

```
1 CREATE TABLE claim_tickets (
2   resource_id TEXT,
3   agent_id TEXT,
4   ticket_number INTEGER PRIMARY KEY AUTOINCREMENT,
5   status TEXT DEFAULT 'waiting' -- 'waiting', 'active', 'released'
6 );
```

Upon `release()`, an atomic FIFO lock-assignment transition grants the lock to the agent with the lowest `ticket_number` where `status = 'waiting'`, changing its status to `'active'`. This guarantees fair exclusion without central scheduling.

Appeals. Settlement events (§7.7) are not unilaterally final. Any settled session can be challenged within an appeal window (default: 24 hours of semantic cadence, §7.9) by any agent with standing—defined as either a counterparty to the settlement or a holder of an overlapping claim. An appeal opens a re-examination of the evidence chain: the appellant posts an *appeal bond* (a fixed fraction of the disputed settlement amount), and the daemon recomputes the settlement against the evidence and any new evidence the appellant produces. If the appeal succeeds, the original settlement is reversed and the appeal bond is returned with the original counterparty's slash. If it fails, the appeal bond is forfeit.

DESIGN INVARIANT 7.5.3 Appeal Soundness. The appeals path does not weaken the immutability of the evidence chain (§7.4): no record is rewritten. Reversal appends a counter-record signed by the daemon, with a Merkle-Forest proof of the original settlement's position and the appellant's evidence. The chain grows monotonically.

What the daemon does *not* do. The daemon does not judge the merits of an appeal in the sense a human court would—it evaluates whether the evidence supports the original settlement under the published rules. Substantive judgments (“was the agent’s reasoning sound”; “was the principal acting in good faith”) remain with humans, escalated through the request channel. The daemon’s role is to ensure that all parties have access to the same evidence and that any human decision is recorded with the same finality as any agent decision.

This subsection is intentionally short: full governance design is out of scope for a single paper. The point is that the bonded commons admits a clean, mechanizable appeals path—it is not the case that advisory claims plus bonding produces a regime where mistakes are unappealable.

7.6 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol [176], within the classical transaction-processing frame of Gray and Reuter [95]: the recovery manager replays the write-ahead log to restore consistency mechanically. Agent crash recovery is fundamentally different. An agent’s “log” is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 7.6.1 (Social Recovery). When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked abandoned (the “zombie protocol”).
2. α is queued in the resurrection set $\mathcal{R}$ with status pending.
3. A successor agent β may claim α ’s work, receiving the *context triple*: purpose ϕ , evidence trail N , and file claims F .
4. β uses its own reasoning to interpret N and determine how to continue.

RECALL

1. Without looking: what is the one thing an advisory claim never does that an enforced lock always does?
2. Name the two conditions of Sen’s theorem this chapter maps onto “the team ships both features” and “each agent controls its own claimed files.”
3. Of the four governance mechanisms in §7.5.3’s escalation order, which one is the equilibrium response to a claim that was merely precautionary, and which to a claim whose holder crashed?

Exercise 7.2, p. 454.

7.6.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The agent’s body (process) is destroyed, but its mind (evidence trail) survives in the commons. A successor reads the trail and continues. Like Hickman’s Krakoa [110]—the *House of X* storyline in which mutant minds are literally backed up to a database (Cerebro) and restored into freshly grown bodies on death—the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan’s sword? The answer produces a moral hierarchy specific to agent societies:

The fundamental moral harm in agent societies is not the destruction of an agent—it is the **irrecoverable loss of information**. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.

Table 7.4: The only catastrophic event in this hierarchy is the loss of information, not the loss of an agent. Severity climbs from a crash (none, because salvage recovers the context) to a daemon failure (catastrophic), with dismissal-from-salvage as the one irreversible step a single agent can cause on its own.

Event	Severity	Consequence
Agent crash	None	Salvage recovers context. The commons loses no information.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent’s trail is permanently destroyed.
Commons authority crashes	Catastrophic	The sovereign falls. No new trust established, no conflicts detected, no salvage possible. State of nature until restart.

7.6.2 The Limits of Reputation

Reputation—the persistent record of an agent’s behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors:** Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals:** The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos:** In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

Exercise 7.3, p. 454.

7.7 Layer 3: Economic Alignment

The moral relativism problem—that a bad actor may view information destruction as a net good—dissolves when the commons isn’t sacred but *priced*. If an agent nukes the workspace, the damage was collateralized before the work began. The bad actor’s principal has an invoice headed their way.

7.7.1 The Float Plan

Definition 7.7.1 (Float Plan). A Float Plan $\mathcal{F}$ is a collateralized work contract consisting of:

1. **Manifest:** A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., “tests pass,” “code review approved”).
2. **Escrow:** Credits posted by the principal, locked in a **2-of-3 Multisig Clearinghouse** (Harbor A, Harbor B, Federation Arbiter) for the duration of the work. The escrow amount reflects the risk to the commons.
3. **Signatures:** The principal signs the Float Plan (Ed25519). The commons authority countersigns, certifying that the escrow is locked and the manifest is registered.

The Float Plan is a *futures contract on agent labor*. The principal sells a promise of work. A 2-of-3 Multisig Clearinghouse manages the contract. Under the non-custodial happy path, both Harbors sign off on

completion and the Arbiter is unnecessary. Under dispute, the Federation Arbiter steps in to provide cryptographic judicial arbitration, settling the outcome using the immutable evidence chain.

7.7.2 Settlement

Definition 7.7.2 (Settlement). When a Float Plan’s session reaches a terminal state, the Multisig Clearinghouse settles the escrow:

Success: The evidence trail demonstrates that acceptance criteria are met (tests pass, file diffs match manifest scope, code review approved). Escrow is released to the agent or its principal.

Partial completion: The agent completed some work before crashing or aborting. The Merkle-chained evidence trail enables pro-rata assessment: the fraction of acceptance criteria met determines the fraction of escrow released. The remainder is available to fund the successor’s completion via salvage.

Sabotage / breach: The evidence trail shows work outside the manifest scope, destructive modifications, or failure to meet any acceptance criteria. The full bond is liquidated to cover reconstruction costs.

DESIGN INVARIANT 7.7.1 Intent Irrelevance. Settlement does not evaluate agent intent. It evaluates *outcome against manifest*. A well-intentioned agent that fails to meet acceptance criteria forfeits escrow. A malicious agent that coincidentally produces correct output receives payment. The system optimizes for outcomes, not character.

This is how performance bonds work in construction. This is how futures markets settle. The contractor’s moral character is irrelevant. The building either meets code or it doesn’t. The bond either covers the damage or it doesn’t.

7.7.3 Why This Defeats the Three Adversaries

The Sybil attack is priced: 100 disposable identities means 100 bonds.

THESIS

1. **One-shot defectors** must post collateral *before* receiving capabilities. Because the bond attaches to the work, not to the identity, disposability does not help the attacker: cheap identity, expensive lie. The cost of defection scales linearly with the number of attack identities, and §7.7.5.9 shows exactly where that linearity stops deterring.
2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float

Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.

- 3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn't prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark. The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

7.7.4 Truthful Claim Signaling as Nash Equilibrium

The bond machinery above prices *outcomes*: an agent that destroys files pays for the destruction. The paper's headline claim, however, is upstream of outcomes—that truthful file-claim signaling (§7.4.4, §7.5.2) is incentive-compatible in the first place. The cartel analysis of §7.7.5.10 works that argument out for insurer pricing. The corresponding argument for the claim signal itself has been carried on faith until now. This subsection closes that gap.

Stage game. Consider two agents *A* and *B* working on a shared project, deciding per round on each file whether to signal:

T (Truthful). Claim files the agent intends to edit; do not claim files it will not edit.

F (False). Either claim files the agent does not intend to edit (to block a rival), or fail to claim files it does intend to edit (to avoid scrutiny). Either form of misrepresentation counts as *F*.

The stage-game payoffs, in expected-utility units calibrated to the cleanup cost *c* of §7.7.5.1, are:

Table 7.5: One-shot file-claim signaling: a classical Prisoner's Dilemma. Mutual truth (3, 3) is Pareto-optimal, but *F* strictly dominates *T* for each player ($4 > 3$ and $1 > 0$). The unique one-shot Nash equilibrium is (*F*, *F*) yielding (1, 1), destroying the coordination signal.

	B: T	B: F
A: T	(3, 3)	(0, 4)
A: F	(4, 0)	(1, 1)

The stage game is a strict Prisoner's Dilemma: F strictly dominates T for each player ($4 > 3$ against T , $1 > 0$ against F), and (F, F) is the unique one-shot Nash equilibrium. *Truthful signaling is not a one-shot equilibrium*; advisory coordination cannot be sustained by stage-game rationality alone.

Move to the repeated game. What rescues truthful signaling is that the agents are not playing the stage game once. The substrate provides:

- **Observable history.** The Merkle-chained evidence trail of §7.4 and heartbeat record of §7.6 make each player's prior moves visible to any third party who can verify a 700-byte inclusion proof. Past defection is not deniable.
- **Persistent identity.** The Anchor Protocol's passkey-bound device pairing [187] ties an agent ID to a hardware-rooted credential. Sybil re-registration is bounded by the per-identity onboarding cost C_{kyc} of §7.7.5.9, so an agent cannot trivially shed a reputation by re-incarnating.
- **A discount factor δ .** Principals value future interactions: a development project lives over weeks; insurer relationships compound; reputation discounts (§7.7.5.3) accrue. For long-lived principals we take $\delta \approx 0.9$ as a working operating point, consistent with the discount used in §7.7.5.10.

Strategy profile (graduated trigger). Each agent plays the following strategy on each file:

1. Begin by playing T .
2. If the opponent's last observed action on the relevant surface was T , play T .
3. If the opponent's last observed action was F , play F for three rounds (punishment phase), then return to T .
4. If the opponent deviates during the punishment phase, restart the three-round window.

Graduated trigger, not grim trigger. The cartel analysis of §7.7.5.10 uses grim trigger because that gives the cleanest folk-theorem bound, and the analysis below uses grim trigger for the same reason where the calculation is short enough to be exact. The *production* strategy is graduated for the reason given in §7.6: crashes and deliberate defection are operationally indistinguishable from any third-party observer's vantage. Permanent punishment for a transient infrastructure event would destroy cooperation faster than any saboteur could.

Deviation analysis. Suppose A contemplates playing F in some round when the strategy prescribes T . The one-shot gain from defection, paid in the deviation round, is $d - c = 4 - 3 = 1$. The cost is three subsequent rounds of mutual punishment at (F, F) before the relationship resets, each paying $p = 1$ instead of the cooperative $c = 3$ (a net

loss of $c - p = 2$ per round). Discounted at δ :

$$\text{PV of lost cooperative payoff} = (c - p) \cdot (\delta + \delta^2 + \delta^3) = 2 \cdot (\delta + \delta^2 + \delta^3).$$

At a working discount factor $\delta = 0.9$, the bracketed sum is $0.9 + 0.81 + 0.729 = 2.439$, so the deviator's net payoff is $1 - 2 \cdot 2.439 = 1 - 4.878 = -3.878$, strictly negative. Deviation is strictly unprofitable.

Critical δ . The general sustainability condition under a k -round graduated trigger with stage-game payoffs $(c, d, p) = (3, 4, 1)$ is

$$\underbrace{d - c}_{\text{one-shot gain}} < \underbrace{(c - p) \cdot \frac{\delta(1 - \delta^k)}{1 - \delta}}_{\text{discounted punishment cost}}.$$

Substituting $d - c = 1$, $c - p = 2$ and $k = 3$ gives $1 < 2\delta(1 + \delta + \delta^2)$, which solves numerically to $\delta > \delta_{k=3}^* \approx 0.342$. Under grim trigger ($k \rightarrow \infty$) the bound is the closed-form $\delta > 1/3 \approx 0.333$, recovering the standard folk-theorem threshold. The graduated bound is therefore only marginally above grim—the three-round window adds barely 0.009 to the threshold—which is the right result: graduated trigger pays almost nothing for the crash tolerance it buys.

THEOREM 7.7.2 Claim Signaling Incentive Compatibility. Under the observable-history regime of §7.4 and the Sybil-resistant identity regime of the Anchor Protocol [187], truthful file-claim signaling is a Nash equilibrium of the repeated coordination game with stage-game payoffs as in Table 7.5 for any discount factor $\delta > \delta_{k=3}^* \approx 0.342$, when both players adopt the three-round graduated-trigger strategy described above. Under grim trigger, the critical bound is $\delta \geq 1/3$.

Every word of this rests on *perfect public monitoring*: both players see the same history, they see it correctly, and they see it at the same time. That is an unusually strong assumption in repeated-game theory and an unusually cheap one here, and it is worth being explicit about why — inside one harbor the single-writer ledger of *The Single-Writer Kernel* is the public history. There is one writer, one commit order, and one record both parties read; a trigger strategy can fire on it because there is nothing for the two sides to disagree about. The architecture buys the assumption rather than making it.

Across harbors it is not bought, and the equilibrium above does not travel. Once history is gossiped rather than committed — *The Anchor Protocol's* anti-entropy mesh, with message loss, partitions and latency — monitoring becomes imperfect and, worse, *private*: two prin-

WHERE THIS STOPS

cipals can hold different views of the same round, each correct about what it saw. Deterministic triggers are exactly the wrong instrument there. A punishment fired on a view the other side never had is indistinguishable from an unprovoked defection, which triggers their punishment, and the pair descends into a cascade neither can exit, because the strategy has no way to say “that was noise.” The literature has known this since Green and Porter [267], whose answer is to accept periodic punishment phases as the price of monitoring noise, and Kandori and Matsushima [128], whose belief-free constructions sustain cooperation under private monitoring by adding communication and review phases so that players coordinate on what they collectively believe happened before anyone sanctions. That is a different mechanism from the one proved here, not a relaxation of it, and the honest status is that the cross-harbor case is *open*: nothing in this chapter or in *The Federated Harbor* re-derives claim signalling under private monitoring, and the federation chapter’s settlement argument assumes bonds rather than triggers precisely so that it does not have to.

What each condition buys, and what breaks when it is removed.

The proposition is conditional, and the conditions matter. Naming what each one is doing makes the dependency on the rest of the architecture explicit rather than implicit.

- *Remove observable history* (§7.4). The repeated game collapses to repeated one-shot play: with no public record of prior actions, no trigger can fire, and both players play F in every round. This is why the Merkle forest of §7.4.2 is necessary for the claim signal, not just for post-hoc audit.
- *Remove persistent identity* (Anchor [187]). An agent who can re-register after each defection effectively faces $\delta = 0$: future punishment falls on a discarded identity. Cooperation is impossible.
- *Drop δ below $\delta_{k=3}^* \approx 0.342$* . The three-round graduated trigger is insufficient. Either lengthen the punishment window or accept that the equilibrium does not bind for very short-lived principals.
- *Drop δ below $1/3$* . No trigger of any duration sustains (T, T) . The repeated game has the same equilibrium structure as the stage game, and only the bond mechanism of this section, not the signal itself, deters defection.

The forward reference is direct. Section 7.7.5.10 reuses the same analytical pattern—stage game, repeated-game lift via observable history, deviation calculation, and a δ threshold—at the insurer pricing layer. The analogy is structural, not a transfer theorem: the payoff functions, monitoring signal, detection timing, and punishment regimes differ and must be derived separately.

Mechanized claim-signaling. The argument above is also checked by machine. Standard one-shot-deviation analysis yields the discount-

factor threshold δ^* as the unique real root in $(0, 1)$ of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$, numerically $\delta^* \approx 0.342$ (matching $\delta_{k=3}^*$ above). The cubic, the existence-and-uniqueness of its root in $[0.34, 0.35]$, and a TLA⁺ model checking that no one-shot deviation produces positive discounted payoff at $\delta = 0.35$ are mechanized as proofs/economics/delta-threshold.z3 and proofs/economics/claim_signaling.tla. Both run unattended in CI (§7.A, Table 7.9).

Numbers by hand — the claim-signaling threshold. Evaluate $f(\delta) = 2\delta^3 + 2\delta^2 + 2\delta - 1$ at $\delta = 0.3425$ by hand, using the factored form $f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$ to keep the arithmetic to two multiplications. First $\delta^2 = 0.3425 \times 0.3425 = 0.11730625$, so $1 + \delta + \delta^2 = 1 + 0.3425 + 0.11730625 = 1.45980625$. Then $2\delta = 0.685$, and $0.685 \times 1.45980625 = 0.9999672813 \dots$, so $f(0.3425) \approx 0.9999672813 - 1 = -0.0000327$ [verified]. That is within 3.3×10^{-5} of zero—close enough to call 0.3425 a root by hand, and the small *negative* residual is exactly consistent with the more precise value `delta-threshold.expected.txt` reports, $\delta^* \approx 0.3425080314$: since f is strictly increasing for $\delta > 0$ (every term has a positive coefficient), a negative value at 0.3425 means the true root sits slightly *above* it, and $0.34250803 > 0.3425$ by exactly the sliver this residual predicts.

Now you try: Exercise 7.5 asks you to bound the same root between two integer-grid points instead of evaluating it at the algebraic value alone.

Figure 7.6 plots that root where it means something: the deviator’s own net payoff, against δ , for the graduated trigger and for grim trigger on the same axis.

Below the threshold the model checker does not merely report a failure; it hands back the deviation. At $\delta = 0.30$ the honest score and the deviator’s score are printed round by round, and the run ends with the deviator ahead.

claim signaling at $\delta = 0.30$: TLC finds the profitable deviation.

AT THE TERMINAL

```
$ java -cp tla2tools.jar tlc2.TLC -config claim_signaling_delta30.cfg claim_signaling.tla
Error: Invariant NoUnilateralDeviationPositive is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ deviatorId = "none"
/\ actualScore = [A |-> 0, B |-> 0]
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 2: <ChooseDeviator>
/\ deviatorId = "A"
```

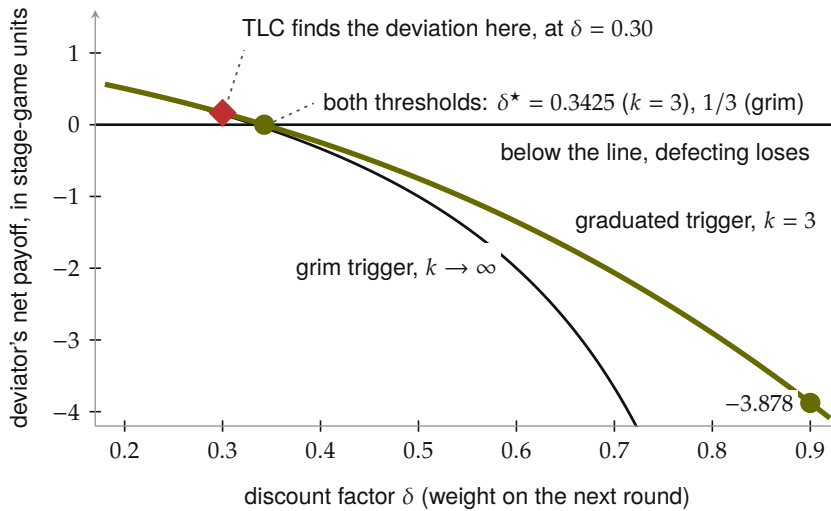


Figure 7.6: Net payoff to a player who defects once, against the discount factor, under the two punishment regimes of this section. The two curves cross zero 0.0092 apart — $\delta^* = 0.3425$ for the three-round graduated trigger, $1/3$ for grim trigger — so bounding the punishment at three rounds costs almost nothing in the threshold. Away from the threshold the two regimes are not close at all: grim trigger’s punishment grows without bound and leaves the frame at $\delta = 0.722$, where it reaches the axis floor, while the graduated trigger’s stays finite, reaching the chapter’s worked value of -3.878 at $\delta = 0.9$. The marked point at $\delta = 0.30$ is the one discount factor whose model-checker transcript is committed — the session printed below — and its height, 0.166 , is that run’s own closing margin, so the sign there, defecting still pays, is what the run reports. Both curves are the section’s own closed form at $(c, d, p) = (3, 4, 1)$, not a simulation [verified, proofs/economics/delta-threshold.z3 and claim_signaling_delta30.run.log].

```

/\ actualScore = [A |-> 0, B |-> 0]
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 3: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 400000000, B |-> 0]
/\ round = 1
/\ punishCountdown = 3
/\ deviated = TRUE
/\ followScore = [A |-> 300000000, B |-> 300000000]

```

... states 4-5 elided ...

```

State 6: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 441700000, B |-> 41700000]
/\ round = 4
/\ punishCountdown = 0
/\ deviated = TRUE
/\ followScore = [A |-> 425100000, B |-> 425100000]

```

10 states generated, 10 distinct states found, 1 states left on queue.
 The depth of the complete state graph search is 6.
 Finished in 00s

RECALL

1. Without looking: what is the one-shot Nash equilibrium of the stage game in Table 7.5, and why does that make truthful signaling impossible without a repeated game?
2. Name the three ingredients the repeated game needs to sustain truth-telling, and which section of the chapter supplies each one.
3. What is the closed-form threshold δ^* for the three-round graduated trigger, and how does it compare to the grim-trigger bound of $1/3$?

Exercise 7.4, p. 454.

Exercises 7.5–7.8, p. 454.

7.7.5 Pricing the Bond

What is the right collateral for “write access to `auth.ts` for 30 minutes”? The answer requires a pricing function $\pi : \mathcal{F} \rightarrow \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence:** $\pi(\mathcal{F})$ must exceed the expected damage from defection under $\mathcal{F}$ ’s scope.
2. **Accessibility:** $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity:** π should increase with the scope and criticality of the manifest.
4. **History adjustment:** π may decrease for principals with strong track records.

We do not close this problem. We tighten the lower bound, propose a scope multiplier, suggest a reputation discount, and describe two concrete mechanisms—the **Bonded Advisor** (§7.7.5.7) and the **Competitive-Insurance** market of Youle (§7.7.5.8). See Appendix 7.B for these four requirements worked through a single \$5 task end to end.

7.7.5.1 Cleanup Lower Bound

Let c be the project's *cleanup cost per breach event*—the human-plus-compute cost to detect, assess, and recover from a budget breach. This is observable from the audit log.

THEOREM 7.7.3 Cleanup Lower Bound. For any Float Plan $\mathcal{F}$, $\pi(\mathcal{F}) \geq c$.

Rationale. If $\pi(\mathcal{F}) < c$, breach is cheap: the bond is forfeit for less than the cleanup cost. If the commons pool funds cleanup, $\pi < c$ drains the commons per breach—the system bankrupts itself through enforcement. $\square$

The Cleanup Cost c Fantasy. The linear bond multiplier assumes c is a bounded, predictable quantity. This is a fantasy for tasks with non-linear tail-risk ruin (e.g., dropping a production database or leaking a private key). Such tasks are fundamentally *un-bondable*. Defense of these tasks is explicitly moved to Layer 1 capability confinement (isolation domains, zero-egress policies); economic deterrence at Layer 3 is only mathematically sound when c is finite and observable. $\square$

This is a floor, not a tight bound. It is observable from operations and trackable as a project health metric.

7.7.5.2 Consequential-Scope Multiplier

Let $s(\mathcal{F})$ be plan scope (files claimed, presence of db:write, production-deployment capability). Empirically, cleanup scales super-linearly with scope; coordination cost dominates the high end.

$$\pi(\mathcal{F}) \geq c \cdot (1 + \alpha \cdot s(\mathcal{F})).$$

α is calibrated from observed cleanup per scope unit. Low- α projects have simple, easily-audited work; high- α projects have tangled dependencies. α is publishable from the audit log.

7.7.5.3 Reputation Discount

Principals with a history of clean settlements have lower expected damage:

$$\pi(\mathcal{F}, p) = \pi(\mathcal{F}) \cdot (1 - \rho(p)), \quad \rho(p) \in [0, r_{\max}], \quad r_{\max} \leq 0.5,$$

to prevent trivialization. ρ increases with clean settlements, decreases with breaches, decays over time. The functional form is left open.

7.7.5.4 Threat-Class Bond Sizing

The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ is threat-agnostic: sloppy work, abandonment, and active sabotage all price off the same curve. That is wrong in both directions. A single curve calibrated to deter sabotage prices capable agents out of low-risk work; a single curve calibrated for accessibility underprices the rare adversary who actually intends harm. Real deployments need different multipliers for different threat classes, because the threats have different blast radii and different probabilities of recovery.

Table 7.6 gives starting bands. These are not parameters, they are calibration targets: production deployments tune from observed failure modes in their own audit log.

The exfiltration row is an explicit gap. It exists in the table to make the limitation legible: a bonded commons does not stop information theft, because the value of leaked secrets can vastly exceed any bond a principal will rationally post. Confidentiality is a structural problem, not an economic one, and the answer lives in Layer 1 (capability-attenuated tokens, isolation domains) rather than Layer 3. The economic alignment of this paper is correct about damage to *shared state*; it makes no claim about damage from *disclosure of state to outsiders*.

The threat-class table also informs Phase 2 of the bootstrap path below: graduated access begins by admitting new principals only to the careless-work and abandonment bands, with the active-sabotage band gated on accumulated reputation.

7.7.5.5 Bootstrap Path

Competitive insurance (§7.7.5.8) is a thick-market mechanism. It needs insurers with capital, principals with reputation history, and enough transaction volume that bidders can statistically distinguish good risks from bad. A cold-start deployment has none of that. The conditional auction/static comparison of §7.7.5.8 describes the modeled end-state; Phases 1–2 below are the path to the assumptions it needs, not optional preamble.

Phase 1 — Subsidized seeding (weeks 1–4). The marketplace operator posts real tasks at above-market rates and invites a small cohort of 5–10 known-capable principals from an existing network. Bonds are fixed at a flat rate per task class; pricing is not a parameter to optimize yet, because there is no data to optimize against. The goal is to produce the first row of the reputation table: clean settlements, breaches, recovery times, observed cleanup cost c . The operator is subsidizing the existence of an audit log.

Transition metric. Completion rate above 80% sustained for ≥ 2 consecutive weeks. Below this threshold the cohort is not yet generating usable risk signal, and Phase 2 pricing will overfit to noise.

Table 7.6: Threat-class bond sizing. Percentages are bond as a fraction of expected task value. Bands are starting points for calibration, not protocol constants.

Threat	Description	Bond %	Rationale
Careless work	Agent tries; output is sloppy, partial, or fails review.	10–25%	Loses some, not all. Bonds at this band incentivize care without freezing out capable agents from cheap tasks.
Abandonment	Agent takes the task and ghosts before completion.	25–50%	Covers the cost of onboarding a replacement: scope-note ingestion, partial-diff review, salvage handoff (§7.4.4).
Active sabotage	Agent intentionally damages shared state.	100–200%	Covers full reconstruction of polluted files plus the cleanup of downstream agents that consumed the bad state. Capital-inefficient above 100%, but acceptable for critical files where reconstruction cost dwarfs the bond.
Data exfiltration	Agent reads and leaks confidential context.	— (<i>spec-only</i>)	Bonds are an inadequate deterrent: leaked context is worth far more than any feasible bond. Rely on capability scoping (Anchor §3.3) and isolation domains (§7.3.2).

Phase 2 — Complexity-proportional pricing with graduated access (months 2–6). The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ activates, with α calibrated from Phase 1 cleanup data and ρ initialized from the Phase 1 reputation table. New principals enter at a low-risk tier: only the careless-work and abandonment bands of Table 7.6 are available. Promotion to the sabotage-coverage tier gates on accumulated clean settlements, mirroring the A5 composite mechanism (§7.7.5.9).

Reputation bootstrapping is the hard problem of this phase. A principal with verifiable work history from another deployment should not start at zero, but the operator cannot verify external histories directly. The mechanism is an *attestation import*: external audit trails signed under an Anchor-issued key (§7.4) earn partial reputation credit, capped well below an equivalent in-system history. The credit is partial because cross-deployment reputation does not control for local cleanup cost, and capped because a fraudulent attestation issuer is a single point of failure.

Transition metric. New-principal 30-day retention above 50%. Lower retention indicates either the gating is too strict (principals churn before they can earn reputation) or the bond bands are mispriced relative to observed task value. Either way, the system is not ready for competitive bidding on top of these parameters.

Phase 3 — Competitive insurance (month 6+). The §7.7.5.8 mechanism activates: insurer agents bid on underwriting transactions, static parameters retire as insurer volume grows, and listing fees enable the cartel-resistance regime of §7.7.5.10. The operator’s pricing parameters become fallbacks rather than the primary mechanism; they apply only when fewer than a threshold number of insurer bids arrive on a transaction.

Transition metric. Repeat-poster rate above 60%. The competitive-insurance equilibrium of §7.7.5.8 assumes repeated interaction; without repeat posters the insurers cannot recover information costs and will not bid competitively.

The weak-improvement result of §7.7.5.8 is conditional on Phase 3 being reached, the listed metrics being met, and the theorem’s CC/NC/RP/NS assumptions holding. Phases 1–2 are not bureaucratic preamble: they generate the data the competitive-insurance market needs to price at all. A deployment that skips them is running §7.7.5.8 on uninformative priors, and the comparison does not apply.

7.7.5.6 Settlement and Dispute

The settlement protocol described so far is one-shot: the daemon's verification result (§7.10) triggers slashing, attribution is recorded immutably (§7.4), and the appeals window of §7.7 allows challenge. That is most of what is needed, but it places the daemon in a position no single component should occupy: simultaneously the verifier, the judge, and the arbiter of evidence. Strong evidence does not make a single oracle correct. It makes a single oracle confidently wrong in the cases where its evidence is incomplete.

The fix is to factor settlement across three oracles with different cost and gameability profiles, and to settle on majority agreement rather than unanimous appeal.

Oracle 1 — Automated. Test-suite pass/fail, type checks, lints, coverage thresholds, and any deterministic CI signal already recorded on the session. Cheap to run, fast to return, and *gameable*: a principal who knows the test surface can ship work that passes the checks while failing the intent. The daemon already produces this oracle as a side effect of normal operation; the change is to label it as one signal among three rather than the dispositive one.

Oracle 2 — Evidence-based. A verifier (which may itself be a bonded agent) reviews the Merkle attribution chain (§7.4) for the disputed transaction. The chain is append-only and cryptographically bound, so the oracle's hard problem is not authenticity of the evidence but interpretation: did the recorded actions match the Float Plan's scope, and do the diffs produced match the acceptance criteria? Medium cost, hard to game without colluding with the chain structure itself, which is the threat model of §7.7.5.9.

Oracle 3 — Human audit. A randomized sample of contested settlements, at a configurable rate (default 10%), is routed to a human auditor. Expensive, slow, and authoritative. The audit produces a labeled outcome that feeds back into reputation history and, critically, into calibrating Oracles 1 and 2: any disagreement between human audit and the cheaper oracles is a training signal for the gameable layers.

Settlement rule.

- *3-of-3 agreement.* Settle immediately on the agreed outcome. The cheaper oracles carry most of the volume in steady state.
- *2-of-3 agreement.* Settle with the majority. Flag the dissenting trace for review; this is the primary signal for tuning the gameable oracle.
- *No majority (three different verdicts, or one settle / one slash / one abstain).* Escalate to full human arbitration. Bond held in escrow until the human verdict lands.

Composition with cartel resistance. Oracle collusion is itself a collusion problem. If Oracles 1 and 2 are run by agents in the same cartel, a 2-of-3 majority can launder a sabotaged settlement past the human audit, modulo the audit sampling rate. The same insurer-collusion analysis applies (§7.7.5.10) with the oracles in the role of bidders: the human-audit sampling rate σ sets the detection probability $p_d^{\text{oracle}} = \sigma$ for the oracle layer, and the folk-theorem threshold becomes a calibration target on σ in the same way it is a calibration target on bid-pattern detection elsewhere in the protocol. The default 10% sample is a starting point; deployments where Oracles 1 and 2 are co-owned should raise it.

7.7.5.7 The Bonded Advisor

One direction for solving π is to delegate its computation to a specialized agent, itself bonded on its advisory accuracy. We call this the **Bonded Advisor** pattern:

- A principal posts a Float Plan whose only acceptance criterion is “propose a fleet that meets budget/scope constraints for project P .”
- A Bonded Advisor surveys P and emits a candidate fleet—itsself a set of Float Plans with proposed π values.
- The advisor’s bond on the proposing plan is slashed proportionally to the accepted fleet’s eventual breach rate. Clean settlements accumulate reputation; breaches shrink it.

The pricer is priced. Convergence depends on (i) whether the advisor has access to sufficient prior-fleet audit data to pattern-match new projects, (ii) whether reputation history is long enough for discounting to be meaningful, and (iii) whether the population of advisors is large enough for competition to discipline outliers.

c as a project health metric. A project whose observed c is rising is fraying—breaches cost more to recover from, implying coordination is decaying. The authority can publish c alongside the bond pool and raise required bonds automatically when c crosses a threshold. Homeostatic feedback: risky spawns become expensive when the project needs them least.

Threat-class bond bands and their defensibility. The pricing function π resolves to one of three nominal bands, calibrated to the threat class the bond is defending against:

- **Careless** (accidental damage during honest attempts): $\pi \in [10\%, 25\%]$ of the claim ceiling.
- **Abandonment** (walk-away mid-task; cleanup dominates): $\pi \in [25\%, 50\%]$ of the claim ceiling.

- **Sabotage** (intentional destruction; can exceed face value): $\pi \in [100\%, 200\%]$ of the claim ceiling.

A Monte Carlo sweep over the cross product of seven plausible threat-distribution mixes and the three bands quantifies defensibility: for every *matched* (mix, band) pair—where the mix concentrates on the class the band is designed to defend against—observed extraction stays within $1.10\times$ the band’s upper bound (10% stochastic tolerance). Off-diagonal cells (e.g. a sabotage-heavy mix posted against the careless band) are surfaced as widening recommendations rather than silent failures. The simulation is mechanized as `proofs/bonded/pareto/threat-bands.mjs` and runs on every PR (§7.A, Table 7.9).

7.7.5.8 Competitive-Insurance Pricing Mechanism⁸

The mechanism. Instead of a static bond size $B(\pi, r, s)$ selected by the commons authority (§7.7.5.2), allow a market of *insurer agents* to bid on underwriting each agent transaction. An insurer I offers a quote (q_I, c_I) where q_I is the required premium and c_I is the claim ceiling.

A principal P submits a transaction T requiring bond coverage B_T . Insurers quote; principal selects. If the transaction proceeds and damages are assessed at d with $d \leq c_I$, the insurer pays. Otherwise principal pays up to $B_T - c_I$ from its own stake.

Market-discovered pricing. The premium q_I equals the insurer’s expected loss $\mathbb{E}[d \mid T, \text{agent history}]$ plus a risk premium α_I encoding the insurer’s risk aversion and cost of capital. In equilibrium under competition, $q_I \rightarrow \mathbb{E}[d]$ for risk-neutral insurers (Rothschild–Stiglitz). Market discovery replaces the authority’s best-guess parameter selection.

Adverse-selection controls. To avoid the classic lemons problem, the mechanism requires:

- Public agent reputation history (from §7.4.2 the Merkle forest).
- Principal-bound slashing if an agent’s history is misrepresented at quote time.
- Insurer capital reserve backed by on-chain stake (the same bond mechanism, recursive: an insurer posts bond that its claim ceilings are funded).

Welfare comparison (model-conditional). Under the simulation’s full-information, competitive-entry assumptions and its chosen static baseline, the market-discovered premium can weakly improve the modeled principal and insurer payoffs while preserving coverage $B_T =$

⁸This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [220, 274] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §7.7.5.8–§7.7.5.8.

$\mu(1 + s)$. This is not a claim against *every* static parameter: a static price equal to the competitive price is outcome-equivalent, and transfers outside the modeled parties can change the Pareto ordering.

An independent model specification and a Monte Carlo sweep over 36 parameter configurations accompany this paper as a downloadable artifact (Appendix 7.A). Figure 7.7 reports three results of that implementation: (i) the no-cartel baseline remains favorable across the tested noise sweep, whereas the three-colluder case loses the comparison beyond roughly $\sigma_r = 0.1$; (ii) the tested full-cartel case defeats the mechanism at the fixed detection setting; and (iii) the tested objective peaks at $n = 3$ insurers. These are finite-sample properties of the supplied code and parameter grid, not general theorems about Vickrey auctions or winner's curse.

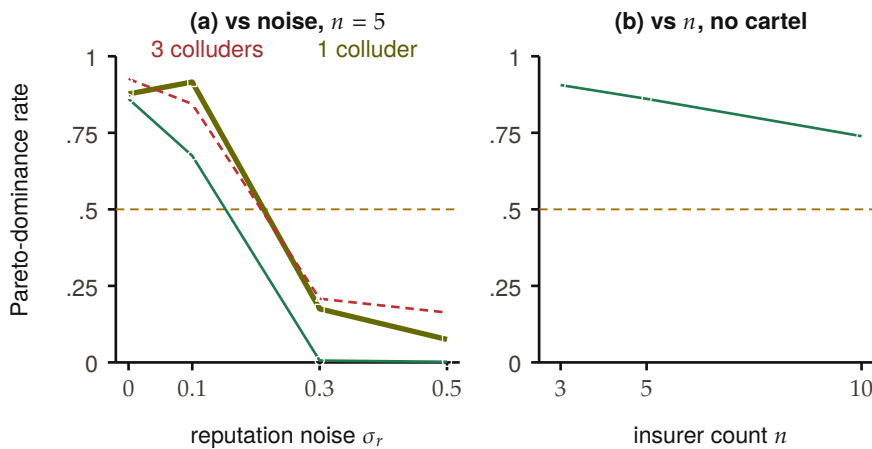


Figure 7.7: Pareto-dominance rate of the competitive mechanism over the static baseline, from 2000 trials per configuration (50 transactions/trial). (a) At $n = 5$ insurers, dominance stays above the 50% criterion for zero and one colluders across the tested noise range, but a three-insurer cartel falls below it once $\sigma_r \geq 0.3$. (b) With no cartel and no noise, dominance falls from 91% at $n = 3$ to 74% at $n = 10$ as more competing insurers dilute the savings any one trial can show. Wilson 95% intervals at this trial count are within ± 0.02 of every plotted rate, too narrow to draw as bars at this scale. [verified, proofs/bonded/pareto/simulation.mjs]

Relationship to the Bonded Advisor. Under this framing, the Bonded Advisor of §7.7.5.7 becomes the matchmaker plus reputation oracle in the insurance market. It does not set prices; it provides information. Pricing is a competitive equilibrium, not an administrative decision.

7.7.5.9 A5 — Sybil deterrence is coverage-bounded

A pure deposit-based Sybil deterrence policy is *provably insufficient*, and the structure of the protocol explains why. An adversary spawns K Sybil insurer identities, each posting the protocol-mandated deposit B_{dep} , then undercuts honest bids by an arbitrarily small ε and defaults on loss (Figure 7.8).

Total Capital Forfeiture. The arbitrary $\min(B_{\text{dep}}, B_T)$ slash cap fails because it bounds the attacker’s risk. To mathematically destroy the profitability of Sybil cartel undercutting, the protocol mandates **total capital forfeiture**: upon default, the *entire* posted deposit B_{dep} is slashed, regardless of how small the coverage B_T was. This converts a capped-risk arbitrage into a total-loss event.

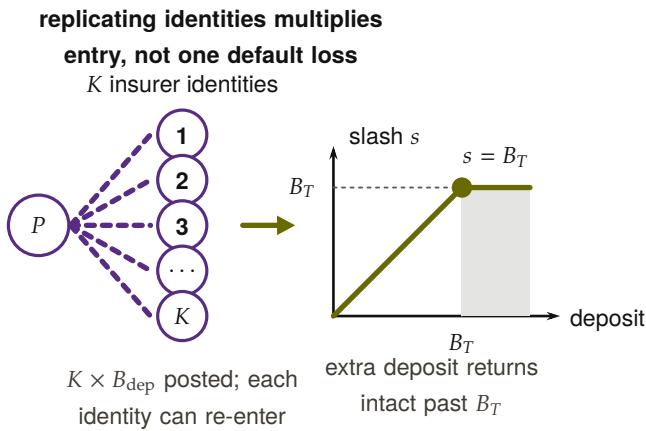


Figure 7.8: Deposit-only Sybil deterrence saturates. One principal can fund K interchangeable identities, but the loss attached to a defaulting identity rises only until its deposit reaches the transaction coverage B_T . Past that knee, $s = \min(B_{\text{dep}}, B_T) = B_T$: additional posted capital returns intact on no-loss transactions and cannot increase the attacker maximum loss. Figure 7.9 shows the corresponding simulated profit and commons-deficit sweeps. [internal, closed form]

The naive expected-value calculation suggests breakeven at $B_{\text{dep}}^* = q^* \cdot (1 - P_{\text{loss}}) / P_{\text{loss}}$, which for the mixed risk classes used in the simulation gives $B_{\text{dep}}^* \approx 316$ USD. *Empirically the attack remains profitable indefinitely.* The reason is structural: the deposit slash is capped at the coverage amount, $\text{slash} = \min(B_{\text{dep}}, B_T)$. Past $B_{\text{dep}} \geq B_T$, any additional deposit posted by the Sybil is recovered intact on no-loss transactions; it never reaches the commons (Figure 7.9).

Closing A5 requires a composite mechanism: an unrecoverable per-identity onboarding cost C_{kyc} (proof-of-personhood, KYC fee, or NFT mint), reputation gating that caps new identities to low-coverage transactions until they accumulate a track record, and per-class B_{dep} tuned

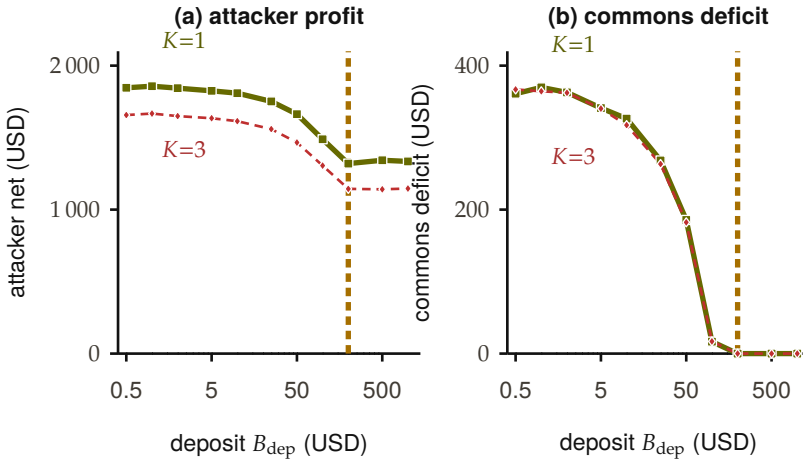


Figure 7.9: A5 — Sybil attack: deposit-only deterrence has a coverage-bounded ceiling. (a) Attacker net profit per 50-transaction trial stays positive across the entire tested deposit sweep ($B_{\text{dep}} \in [0.5, 1000]$ USD) for both one and three colluding Sybil identities. (b) The commons’ own deficit converges to zero once $B_{\text{dep}} \approx 200$ USD — the protocol itself is fully reimbursed, but the attacker still profits because the deposit slash is capped at coverage $B_T = \mu(1 + s)$. Closing A5 requires a composite mechanism ($C_{\text{kyc}} + \text{reputation gating} + \text{per-class } B_{\text{dep}}$); pure deposit-based deterrence is provably insufficient. [verified, proofs/bonded/pareto/simulation-sybil.mjs]

to saturate the coverage cap exactly. The single-instrument deposit policy is provably insufficient; the mechanism in §7.7.5.8 is correct only with this composite extension. The supporting analysis is bundled as `cartel-resilience.md` (Appendix 7.A).

7.7.5.10 A6 — Cartel sustainability under the folk theorem

A5 considers a single-round attack: a Sybil identity bids low, wins, defaults. A6 studies a *repeated game*: a coalition of insurers maintains a price floor $q_{\text{floor}} > q^*$ above the competitive equilibrium, splits the surplus, and uses grim-trigger punishment to deter unilateral deviation. Folk-theorem results can support feasible, individually rational payoffs for sufficiently patient players under their monitoring and regularity assumptions; they do not license every outcome in every repeated game. Here the narrower question is how the supplied payoff model changes as the daemon’s per-round cartel-detection probability varies.

Grim trigger is used in the analysis below for analytical tractability—it gives the cleanest recurrence and the closed-form p_d^* threshold of Figure 7.11. A production detector should use a graduated response because crashes (§7.6) can be observationally similar to deliberate defection; permanent punishment for a transient infrastructure event would destroy cooperation. The claim-signaling and cartel

layers therefore share an analysis template, not one interchangeable equilibrium result.

Figure 7.10 shows the per-round decision node and the closed-form sustainability inequality.

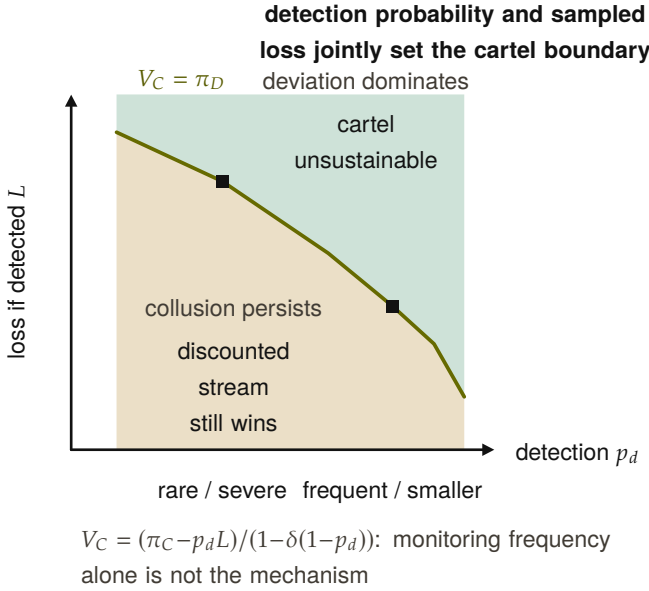


Figure 7.10: Cartel sustainability is a two-policy phase diagram. The boundary is the repeated-game equality $V_C = \pi_D$ (gold). Below it (amber), the discounted collusive stream still dominates a one-shot deviation. Above it (green), deviation wins and the cartel is unstable. A rarer severe sampled loss and a more frequent smaller loss can lie on the same deterrence boundary. Detection frequency by itself is therefore not the mechanism; the product enters through the full discounted payoff expression. [internal]

A Monte Carlo sweep over the (p_d, δ) grid checks the closed-form threshold against the supplied timing and payoff model (Figure 7.11). Let $L = 5(q_{\text{floor}} - \mu)$ be the one-time detection penalty, $\pi_C = (q_{\text{floor}} - \mu)/3$ the per-member cartel payoff, and $\pi_D = q_{\text{floor}} - \varepsilon - \mu$ the one-shot deviation payoff. Every active cartel round credits π_C ; if detection occurs in that round, the simulation then subtracts L and terminates future cartel payoffs. Hence

$$V_C = \pi_C - p_d L + \delta(1 - p_d)V_C, \quad p_d^* = \frac{\pi_C - (1 - \delta)\pi_D}{L + \delta\pi_D}.$$

At $\delta = 0.95$ this gives $p_d^* \approx 0.0478$, and the tested grid first classifies the cartel as fragile at $p_d = 0.05$. This is a falsifiable, model-conditional calibration target for bid-pattern detection; the paper does not claim the deployed detector already attains it.

This result does not automatically extend to the agent layer. The two layers share a repeated-game template, but their payoff functions

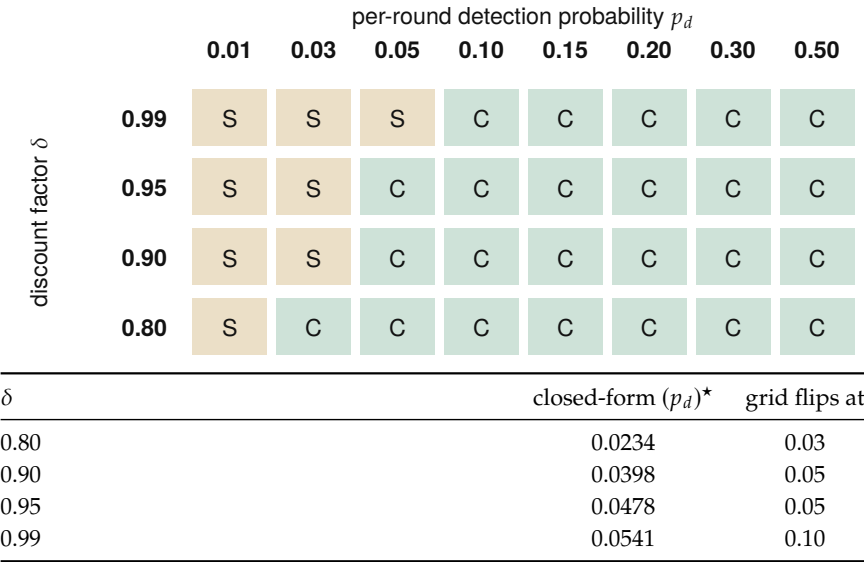


Figure 7.11: A6 — cartel sustainability under the grim-trigger folk theorem, the tested (p_d, δ) grid (5,000 trials/configuration, 200 rounds, cartel size 3). **S** marks a sustainable cell; **C** marks one that collapses. The sustainable band widens by at most one column as patience δ rises, and the simulated flip point matches the closed-form threshold $(p_d)^*(\delta)$ of §7.7.5.10 to within one tested grid step at every δ (table below) — at $\delta = 0.95$, $(p_d)^* \approx 0.0478$ against a simulated flip at $p_d = 0.05$. The matplotlib original right panel (mean rounds to first detection) is not redrawn here; the sustainability frontier above is the section’s load-bearing result. [verified, proofs/bonded/pareto/simulation-cartel.mjs]

and monitoring signals differ; each layer needs its own obedience calculation.

*Exercises 7.9–7.11,
p. 454.*

7.8 Coordination as Substrate: An Expressive-Act Taxonomy

Reader's note: this section is a half-page taxonomy. The auction-focused reader can skim the five-item list and skip to §7.7.5.8; the mechanism-design reader will want the per-class bond profiles in full. The taxonomy matters to both because it explains why a single bond price cannot be set.

The bonded commons treats coordination as a uniform action surface: agents post bonds, settle, accrue reputation. In practice, the *kinds* of coordination differ enough that uniform pricing distorts incentives. We argue that any commons-scale pricing mechanism must distinguish among five expressive classes, and that each class has a different bonding profile. The punchline is in one line: **uniform pricing collapses the market to its highest-stakes class and freezes out the rest**. The five classes below establish why.

7.8.1 Five Expressive Classes

Signal. “Something happened.” Notes, tuples, pheromones, activity events. High frequency, low individual stakes, cumulative value.

Request. “Decision, input, or approval needed.” Escalations to a human or to a higher-authority agent. Low frequency, blocks downstream work, asymmetric cost.

Distress. “I am stuck, repeatedly.” Bounded enum: auth, conflict, permission, budget, invariant. Rare; abuse can halt sibling agents.

Commons. Shared durable state—tuple kinds, graph edges, channels, proposals. Persists beyond the originator; pollution is hard to undo.

Proposal. “Here is a plan; commit?” Float Plans, change sets, fleet specifications. Requires review; cost is in the reviewer’s time.

7.8.2 Per-Class Bond Profiles

Each expressive class has a different relationship to the cleanup cost of §7.7.5.1:

- *Signal class:* cheap, numerous, micro-bonds or reputation-discounted. Slashable for false alarm.
- *Request class:* bonded by the cost of the human’s time plus the downstream action cost.
- *Distress class:* bonded by the blast radius if an agent uses it to halt siblings abusively.

- *Commons class*: bonded by the storage cost; slashable for pollution.
- *Proposal class*: bonded by the cost of reviewing plus the cost of implementing.

Treating all coordination as one uniform “bonded action” misses the price-discovery differences. The Bonded Advisor (§7.7.5.7) and the Competitive-Insurance market (§7.7.5.8) must price each class separately, or the market will collapse to its highest-stakes class and freeze out the rest.

7.9 Semantic Cadence, Agentic Psychosis, and Observability

Wall-clock is the wrong axis for observing multi-agent coding. A 30-second wait while a model thinks is qualitatively different from a 30-second wait while four agents are actively producing tokens. We formally introduce **Semantic Cadence** (the Causal-Density Ratio) as the primary temporal axis:

$$t_{\text{cadence}} = \int_0^t \text{causal_density}(\tau) d\tau,$$

where `causal_density` aggregates token-emission rates strictly weighted by verified evidence-trail growth (AST changes, test executions, or committed logic).

Numbers by hand. An agent burning 40 tokens/sec for 30 seconds (1,200 tokens) with zero AST diffs and zero test runs over that window integrates to a causal density of ≈ 0 : maximum spend, no evidence growth, so the weighting term zeroes the integrand and the Asylum Protocol triggers. An agent burning 15 tokens/sec over the same window while landing three verified diffs keeps causal density high even though its raw token rate is barely a third of the first agent’s—which is the whole point of metering by evidence growth rather than by token spend. *Now you try*: the same 15 tokens/sec with one verified diff and one passing test run in a 30-second window (borderline: evidence growth is positive, so cadence advances, but slowly enough that the daemon flags it for the next window rather than stopping the process). Table 7.7 collects the three cases.

A note on the names. The terms in the rest of this section are playful; the failure modes are not. Read “Agentic Psychosis,” “the Asylum Protocol,” and “Inception Canaries” as defined engineering terms with the operational definitions given below—each one names a detector and a response with a threshold attached—rather than as jokes.

Table 7.7: Causal density separates the two agents that a wall-clock or token-rate meter cannot tell apart: the psychotic agent is the *highest* token spender in the table and the only one stopped, while the healthy agent spends a third as much and is nominal. Token rate alone would rank these three in exactly the wrong order.

Case	Token rate	Evidence in window	Cadence / verdict
Healthy	15 tok/s	3 AST diffs, 2 test runs	advances; nominal
Borderline	15 tok/s	1 AST diff, 1 test run	advances slowly; flagged for next window
Psychotic	40 tok/s	none	≈ 0 ; Asylum triggered

Agentic Psychosis & The Asylum Protocol. Unbounded LLMs inevitably enter *Agentic Psychosis*: hallucination loops characterized by extremely high token-generation rates but zero functional AST change. When the daemon detects a causal-density ratio approaching zero alongside high token-burn, it initiates **The Asylum Protocol**. The daemon sends SIGSTOP to the agent process, routes its context window through a secondary SLM (Small Language Model) sidecar for semantic compaction and Situation-in-the-Loop (SITF) prompt injection (“*You are looping. Re-evaluate your last 3 steps against the active plan.*”), and then issues SIGCONT. This cures the hallucination loop without sacrificing session context.

Inception Canaries (Gamified Chaos Engineering). The daemon replaces traditional, UX-hostile skill-retention taxes with **Gamified Chaos Engineering**. The SLM sidecar routinely injects synthetic bugs (“Inception Canaries”) into the Attention Queue of loaded agents. If the agent successfully catches and resolves the injected bug, it is granted an Elo rating boost to its `skill_hash` and higher subsequent autonomy limits (“God Mode”). If it fails or hallucinates around the bug, its blast radius is throttled.

Replay JSONL as artifact. Every settled session emits a JSONL replay log: events, notes, claims, settlements, in causal order. This is simultaneously: an audit artifact, a debugging tool, and—redacted—training data for downstream DPO/SFT.

Privacy contract. Replay is opt-in. Redaction strips secrets and PII. Encryption-at-rest applies the harbor session key (§7.12). Eviction is LRU-bounded.

We position Semantic Cadence and its resultant protocols not as Port Daddy features but as a category claim: *multi-agent coding requires its own observability primitives*, and they are not the wall-clock metrics that single-process systems get away with.

7.10 Formal Model

We specify the coordination lifecycle in TLA⁺ [142] to verify that the three layers compose correctly.

Read the state machine as five English sentences before reading any syntax. An agent may not Begin without posting positive escrow. A file Claim always succeeds and only ever informs—it never blocks. Commit clears a session’s claims and locks together. A Reap demotes an unresponsive agent to abandoned and queues it for Salvage. And nothing anywhere deletes a note. The specification below is that same machine, made checkable.

7.10.1 State and Actions

Figure 7.12 draws this same lifecycle as a state machine before the TLA⁺ makes it checkable: one honest path settles a session, and a crash detours it through Reap to either a successor’s Salvage or a permanent Dismiss.

```

1  ---- MODULE BondedCommons ----
2  EXTENDS Naturals, Sequences, FiniteSets
3
4  CONSTANTS Agents, Files, Locks, DeadThreshold
5
6  VARIABLES
7    registered,    \* Set of active agent IDs
8    sessions,      \* Agent -> {status, notes, claims, escrow}
9    fileClaims,    \* File -> set of claiming agents
10   heldLocks,     \* Lock -> {owner, expiry} or NULL
11   salvageQueue,  \* Set of {agent, status}
12   heartbeats,    \* Agent -> last heartbeat time
13   clock          \* Monotonic clock
14
15   vars == <<registered, sessions, fileClaims, heldLocks,
16           salvageQueue, heartbeats, clock>>

```

Listing 7.1: TLA⁺: State Variables

```

1  Begin(a, escrow) ==
2    /\ a \notin registered
3    /\ escrow > 0    \* Must post collateral
4    /\ registered' = registered \cup {a}
5    /\ sessions' = [sessions EXCEPT ![a] =
6      [status |-> "active", notes |-> <<>>,

```

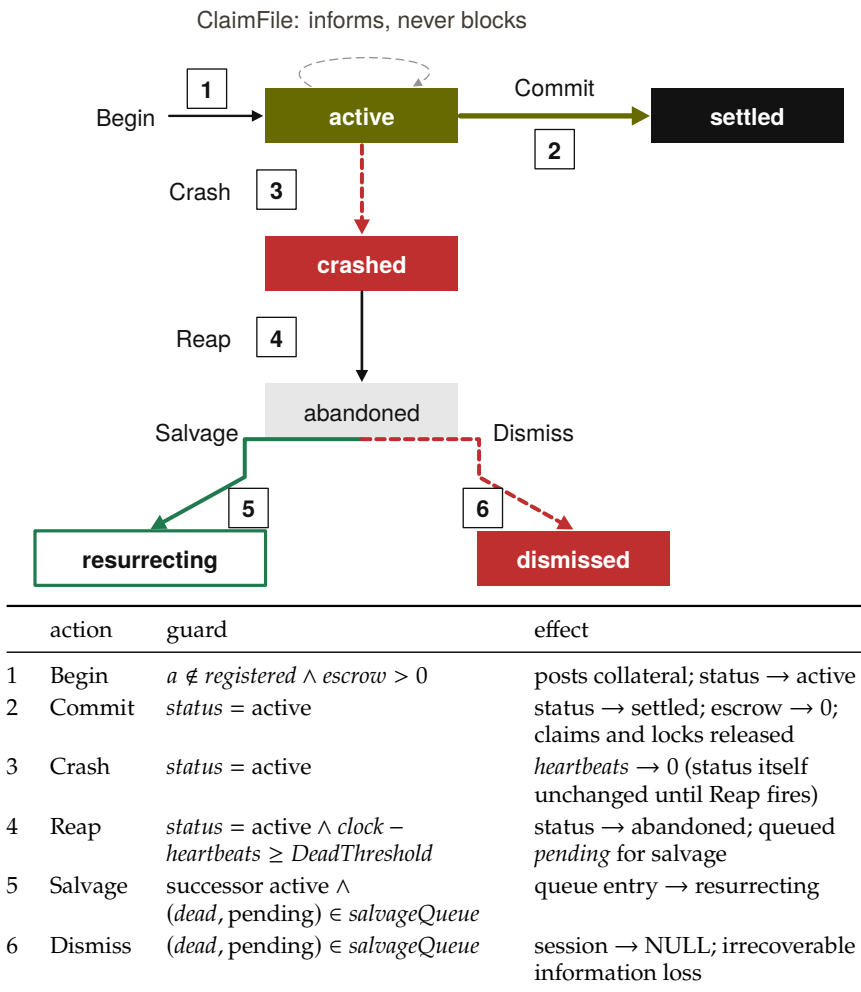


Figure 7.12: The coordination lifecycle of `BondedCommons.tla`: one honest path down the right (Begin, Commit) settles a session; a crash detours down the spine through the dead-threshold check Reap performs, to abandoned, from which a successor either resumes it (Salvage) or the session is permanently lost (Dismiss). `ClaimFile` is the one named action with no guard at all — it always succeeds and only ever informs, drawn here as a self-loop. Tick advances no session state; it only ages the clock that feeds the guard Reap checks. [internal, `BondedCommons.tla`]

```

7      claims |-> {}, escrow |-> escrow]]
8  /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
9  /\ UNCHANGED <<fileClaims, heldLocks,
10      salvageQueue, clock>>
11
12 ClaimFile(a, f) ==
13  /\ a \in registered
14  /\ sessions[a].status = "active"
15  \* Advisory: always succeeds, reports conflicts
16  /\ fileClaims' = [fileClaims EXCEPT ![f] =
17      fileClaims[f] \cup {a}]
18  /\ UNCHANGED <<registered, sessions, heldLocks,
19      salvageQueue, heartbeats, clock>>
20
21 Commit(a) ==
22  /\ a \in registered
23  /\ sessions[a].status = "active"
24  /\ sessions' = [sessions EXCEPT ![a] =
25      [sessions[a] EXCEPT !.status = "settled",
26      !.escrow = 0]]
27  \* Release all claims and locks
28  /\ fileClaims' = [f \in Files |->
29      fileClaims[f] \ {a}]
30  /\ heldLocks' = [l \in Locks |->
31      IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
32      THEN NULL ELSE heldLocks[l]]
33  /\ UNCHANGED <<registered, salvageQueue,
34      heartbeats, clock>>
35
36 Crash(a) ==
37  /\ a \in registered
38  /\ sessions[a].status = "active"
39  /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
40  /\ UNCHANGED <<registered, sessions, fileClaims,
41      heldLocks, salvageQueue, clock>>
42
43 Reap ==
44  /\ \E a \in registered :
45  /\ sessions[a].status = "active"
46  /\ clock - heartbeats[a] >= DeadThreshold
47  /\ sessions' = [sessions EXCEPT ![a] =
48      [sessions[a] EXCEPT !.status = "abandoned"]]
49  /\ salvageQueue' = salvageQueue \cup
50      {[agent |-> a, status |-> "pending"]}
51  /\ fileClaims' = [f \in Files |->
52      fileClaims[f] \ {a}]
53  /\ UNCHANGED <<registered, heldLocks,
54      heartbeats, clock>>
55
56 Salvage(successor, dead) ==

```



```

57  /\ successor \in registered
58  /\ sessions[successor].status = "active"
59  /\ [agent |-> dead, status |-> "pending"]
60      \in salvageQueue
61  /\ salvageQueue' = (salvageQueue \
62      {[agent |-> dead, status |-> "pending"]})
63      \cup {[agent |-> dead,
64          status |-> "resurrecting"]}]
65  /\ UNCHANGED <<registered, sessions, fileClaims,
66      heldLocks, heartbeats, clock>>
67
68  Dismiss(dead) ==
69  \* Irrecoverable information loss
70  /\ [agent |-> dead, status |-> "pending"]
71      \in salvageQueue
72  /\ salvageQueue' = salvageQueue \
73      {[agent |-> dead, status |-> "pending"]}]
74  /\ sessions' = [sessions EXCEPT ![dead] = NULL]
75  /\ UNCHANGED <<registered, fileClaims, heldLocks,
76      heartbeats, clock>>
77
78  Tick == /\ clock' = clock + 1
79      /\ UNCHANGED <<registered, sessions, fileClaims,
80      heldLocks, salvageQueue, heartbeats>>

```

Listing 7.2: TLA⁺: Core Actions

7.10.2 Properties

```

1  \* SAFETY: Every active session has positive escrow
2  EscrowInvariant ==
3  \A a \in registered :
4      sessions[a] # NULL /\ sessions[a].status = "active"
5      => sessions[a].escrow > 0
6
7  \* SAFETY: Notes never shrink for active sessions
8  NoteMonotonicity ==
9  \A a \in registered :
10     sessions[a] # NULL /\ sessions[a].status = "active"
11     => Len(sessions'[a].notes) >= Len(sessions[a].notes)
12
13  \* LIVENESS: Dead agents are eventually salvaged
14  \* or dismissed (under fairness)
15  CrashRecovery ==
16  \A a \in Agents :
17      [agent |-> a, status |-> "pending"] \in salvageQueue
18      ~> [agent |-> a, status |-> "pending"]
19          \notin salvageQueue
20
21  Spec == Init /\ [][Next]_vars

```

```

22      /\ WF_vars(Reap) /\ WF_vars(Tick)
23
24  ====

```

Listing 7.3: TLA⁺: Safety and Liveness Properties

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer’s structural guarantee—it holds by construction of the `Begin` action, which requires `escrow > 0`.

7.10.3 The Conservation Theorem

The TLA⁺ escrow invariant states the property abstractly; the daemon’s bond-accounting subsystem realizes it operationally, collapsing the abstract escrow into three monetary buckets per project—**wallet**, **escrow**, and **commons pool**—which together discharge what `EscrowInvariant` asserts.

Definition 7.10.1 (Accounting State). For project P , the accounting state is the triple $(W_P, E_P, C_P) \in \mathbb{R}_{\geq 0}^3$, where W_P is wallet balance, E_P is the sum of bond amounts in states $\{\text{escrowed}, \text{running}, \text{exiting}\}$, and C_P is the commons pool balance. The *monetary supply* is $S_P := W_P + E_P + C_P$.

THEOREM 7.10.1 Conservation. For any sequence of operations

$$\sigma \in \{\text{topUp}, \text{escrow}, \text{markRunning}, \text{refund}, \text{slash}\}^*$$

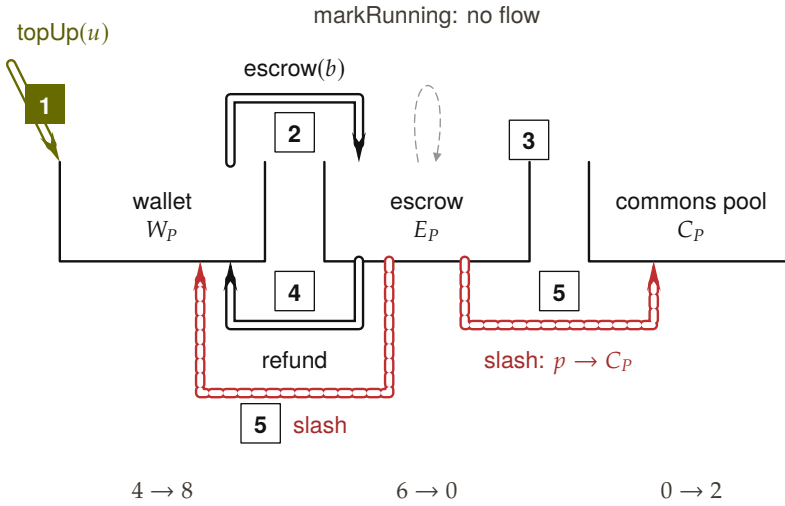
applied to an initial state $(W_P^0, 0, 0)$, the supply S_P changes only on `topUp`. All other operations redistribute among the three buckets.

Proof sketch. Each operation commits in a single SQLite transaction (db.transaction). The individual effects:

- `topUp(u)`: $W_P \ += \ u$. S_P grows by u .
- `escrow(b)`: $W_P \ -= \ b$, $E_P \ += \ b$. S_P invariant.
- `markRunning(id)`: state transition within E_P ’s contributing set. No monetary movement. S_P invariant.
- `refund(id)`: $W_P \ += \ b_{id}$; row state $\rightarrow$ *refunded*. Net $E_P \ -= \ b$, $W_P \ += \ b$. S_P invariant.
- `slash(id, p)` with $p \in [0, b_{id}]$: $W_P \ += \ b_{id} - p$, $C_P \ += \ p$, row $\rightarrow$ *slashed*. Net $-b + (b - p) + p = 0$. S_P invariant.

By induction on $|\sigma|$, S_P equals S_P^0 plus the sum of `topUp` arguments in σ . □

Figure 7.13 draws the three buckets and five operations this proof sketch reasons over, instantiated with the section’s own worked settlement.



$S_P := W_P + E_P + C_P$: before $4 + 6 + 0 = 10$, after $8 + 0 + 2 = 10$ — unchanged.

operation	effect on the buckets	ΔS_P
1 topUp(u)	$W_P += u$	$+u$
2 escrow(b)	$W_P -= b, E_P += b$	0
3 markRunning	state change inside E_P 's contributing set; no transfer	0
4 refund(id)	$E_P -= b, W_P += b$	0
5 slash(id, p)	$E_P -= b, W_P += (b-p), C_P += p$	0

Figure 7.13: The conservation ledger of Theorem 7.10.1: three buckets per project, joined by five named operations. Only topUp (gold) grows the supply S_P ; the other four, including the two-way split that slash performs (red, dashed, two rails for one operation), only move money between buckets. The instance under each bucket is the chapter's own worked partial breach: before $(W_P, E_P, C_P) = (4, 6, 0)$, a slash of $p = 2$ retires the full escrow, and after $(8, 0, 2)$ — $S_P = 10$ both times. [internal, Conservation.tla]

Property verification. The conservation invariant is asserted after every operation in the bonds test suite. Across 200 random traces, $|W_P + E_P + C_P - S_P^{\text{expected}}| < 10^{-6}$ holds deterministically; the 10^{-6} tolerance is IEEE-754 floating-point slack, not logical slack.

THEOREM 7.10.2 No Overdraft Under Concurrent Escrow. Given two concurrent invocations $\text{escrow}(b_1)$ and $\text{escrow}(b_2)$ on the same project with $b_i \leq W_P^{\text{pre}}$ but $b_1 + b_2 > W_P^{\text{pre}}$, at most one invocation succeeds.

Implementation: Atomic RETURNING Clauses. To enforce overdraft prevention atomically and avoid Node.js/TypeScript event-loop async yield vulnerabilities, the escrow state transition is written as a single SQLite statement with a RETURNING clause. This guarantees that balance checks and deductions occur within a synchronous, uninterruptible database step, preventing race conditions from interleaved async microtasks.

Proof. Both transactions execute under BEGIN EXCLUSIVE, so SQLite serializes them. Suppose T_1 wins. After commit, $W_P = W_P^{\text{pre}} - b_1$. T_2 now reads the updated balance and rejects iff $b_2 > W_P^{\text{pre}} - b_1$, which by assumption is true. $\square$ $\square$

Graduated sanctions. The TLA⁺ model abstracts settlement into Commit and Crash. Reality has an intermediate state: an agent whose spend is approaching its daily budget. The daemon’s budget-guard component introduces **throttle** as a soft signal at 80% spend (publishes a `budget:decisions:throttle` event; structurally non-coercive, advisory in Sen’s sense) and **kill** as the hard signal at 100% (refuses new spawns until UTC midnight; existing spawns SIGTERMed; bond slashed). The throttle/kill split is the commons’ analogue of Ostrom’s graduated sanctions [186]: response scales with violation severity. Settlement is not binary; it is a three-state machine *nominal* $\rightarrow$ *throttled* $\rightarrow$ *killed*, with *nominal* $\rightarrow$ *killed* reachable only on hard evidence (e.g., a direct Arbiter violation).

Numbers by hand — a settlement’s ledger check. Instantiate Theorem 7.10.1’s proof sketch with round numbers. Before a settlement, a project sits at $(W_P, E_P, C_P) = (4, 6, 0)$ —four in wallet, six escrowed on one open Float Plan, nothing yet in the commons pool—so $S_P = 4 + 6 + 0 = 10$ [verified]. The settlement resolves as a partial breach: `slash(id, p=2)` burns 2 to the commons pool and refunds the remaining $b - p = 6 - 2 = 4$ to the wallet. Applying the proof sketch’s

own bookkeeping, $W_P += (b - p) = 4 + 4 = 8$, $C_P += p = 0 + 2 = 2$, and E_P 's contribution of 6 is fully retired to 0. After: $(W_P, E_P, C_P) = (8, 0, 2)$, and $S_P = 8 + 0 + 2 = 10$ [verified]—unchanged, exactly as the theorem predicts, because the settlement only moved money between buckets rather than minting or destroying it.

Now you try: Exercise 7.12 asks you to find the analogous check across all 1,716 reachable states of the mechanized model, not just this one hand-picked settlement.

Why the forfeit goes to the commons and not back to the buyer. That worked example contains a design decision most readers slide past: the slashed 2 went to C_P , the commons pool, and not to the counterparty who suffered the breach. Refunding the injured party looks obviously fairer and would be a mistake, and the reason is a theorem that predates all of this. Holmström [26] proved that in joint production with unobservable individual effort, no *budget-balanced* sharing rule can sustain efficient effort: if every unit collected from a shirker is paid out to the other participants, then the participants' shares always sum to the total, and that adding-up constraint is exactly what lets someone deviate profitably — the punishment one agent pays is another agent's windfall, so the group as a whole is never made worse off by a breach and cannot credibly commit to destroying value. His resolution is a *budget breaker*: a third party outside the sharing rule that absorbs the forfeit, so the penalty leaves the participants entirely.

The commons pool is that budget breaker, and $\text{slash}(id, p)$ is what makes it one — p routes to C_P and never back to the counterparty, so a breach is a strict loss to the pair rather than a transfer within it. Note what this does *not* claim. Holmström's theorem is about sustaining first-best effort in a team, and the mechanism here is not doing that; what it borrows is the structural reason budget balance has to break, which is why the conservation identity of Theorem 7.10.1 holds over $W_P + E_P + C_P$ and not over the wallets alone. A version of this system that refunded slashed bonds to buyers would satisfy a narrower conservation law and lose the deterrent, and it would fail for Holmström's reason rather than for an accounting one.

RECALL

1. Without looking: what does `EscrowInvariant` guarantee, and which TLA^+ action makes it hold “by construction” rather than by a separate check?
2. State the Conservation Theorem's three-bucket accounting identity from memory, and name the one operation that is allowed to change the total.
3. Why does `Reap` require a heartbeat threshold rather than firing immediately on any missed heartbeat, and what TLA^+ variable does it consult to decide?

Exercises 7.12–7.13, p. 455.

7.11 Related Work

Social contract theory. Hobbes [113] argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke [159] proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Impossibility results. Sen [231] proved that Pareto efficiency and minimal liberalism are incompatible. We apply this result to show that enforced file claims (minimal liberalism for agents) produce Pareto-inferior team outcomes, motivating advisory claims.

Long-lived transactions. Garcia-Molina and Salem [87] introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

BDI agents. Rao and Georgeff [209] formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Generative agents. Park et al. [196] showed that memory streams, reflection, and planning produce temporally coherent individual behavior. Our immutable evidence trails are architecturally similar to memory streams but serve a different purpose: inter-agent coordination and crash recovery, not individual coherence.

Capability-based security. Dennis and Van Horn [62] introduced capability-based access control. Birgisson et al. [30] extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol [187] adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey [263] and Myerson [178]. A full optimal-auction derivation in that tradition is outside the scope of this paper.

Agentic-commerce protocols. While this paper was in preparation, the transaction rails of an agent economy shipped as industry standards: the Universal Commerce Protocol [206] (Google/Shopify, 2026) for merchant-hosted agent-mediated retail, its platform-mediated rival the Agentic Commerce Protocol [205] (OpenAI/Stripe), and the Agent Payments Protocol [166], whose signed mandate chain — W3C Verifiable Credentials in SD-JWT form, principal → platform → merchant — is the deployed sibling of our countersigned Float Plan. All three are explicit that agent *trustworthiness* is out of scope; published security analyses [58] observe that they regulate how agents transact while

providing no collateral, no settlement oracle, and no priced deterrent against a misbehaving agent. That gap is precisely the synthesis this paper contributes (§7.11 *supra*): bonded participation, capability attenuation, advisory coordination, and immutable attribution. The reference implementation accordingly adopts their credential formats at the harbor boundary (SD-JWT-VC cards, JWS detached-content countersigns over JCS; ADR-0094) while keeping the bonding mechanism they lack.

Table 7.8: What the commons borrows from each prior body of work, where it departs, and why.

Body of work	Borrowed	Departure	Why
Social contract theory	Authority accepted before interaction begins	The sovereign is a daemon whose rules are inspectable and whose consent is revocable per task	Agents can exit; Hobbes's subjects could not
Impossibility results	Liberalism against Pareto efficiency	Applied to enforced file claims	It is the reason claims stay advisory
Long-lived transactions	Compensation for work that outlives a transaction	Economic settlement in place of mechanical compensation	Compensation cannot undo an external effect; a bond can price it
BDI and generative agents	Beliefs, desires, intentions; memory streams	The evidence trail is read by an adversarial operator, not a private memory	The reader, not the agent, is the customer of the record
Capability-based security	Attenuable capabilities	Capabilities bind to bonded identities with settlement	Authority without liability is the failure the commons prices
Mechanism design	Incentive compatibility as the design target	Bonds priced by the claim-signaling threshold δ^* , not by an optimal auction	The commons is a repeated game, not a one-shot sale
Agentic-commerce protocols	The credential envelope	The envelope carries authorization; the commons carries liability	A standard format shrinks the problem without settling it

7.12 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

The Federated Sovereign. The single-machine model presented so far is insufficient on its own: users roam across laptop, desktop, and CI machines; machine loss should not destroy encrypted evidence; multiple humans may share a harbor. The daemon remains the authority *within* its machine. Key custody federates outward.

We introduce a fourth actor, specified by properties rather than by vendor:

Daemon commons authority within a machine; signs harbor roots; enforces bonds.

Agent participant; holds Harbor Card; signs actions.

Principal funds Float Plans; identifies via Ed25519 keypair.

KMS key custody; recovery root-of-trust; witnesses harbor roots.

Definition 7.12.1 (Admissible KMS). A Key Management Service $\mathcal{K}$ is admissible for the federated sovereign if it provides:

1. Passphrase-wrapped private-key custody using a memory-hard KDF (Argon2id) at the 2026 security floor.
2. Encryption-at-rest for all user-held blobs; $\mathcal{K}$ never sees plaintext keying material.
3. A signed witness log for harbor Merkle roots with append-only, monotonic semantics and detectable equivocation [145].
4. Rate-limited, email-gated recovery via single-use magic-link tokens of bounded TTL.
5. Public verification of signed user public keys under an account identifier, without requiring mutual authentication.

The implementation choice (a particular cloud platform, an on-prem HSM, a self-hosted Tang server) lives in companion design documents. The paper's theory applies to any $\mathcal{K}$ meeting properties (1)–(5).

Trust boundary. The federated trust boundary is $TB = daemon \cup \mathcal{K} \cup user-email \cup user-passphrase$. Each element is necessary for its role; no element is sufficient alone. The daemon enforces bonds and publishes roots but never sees the unwrapped user master. The KMS stores encrypted blobs and can deny service but cannot decrypt. Email is the recovery root—and is documented as the weakest link, since an adversary with email control can trigger magic-link recovery. The passphrase wraps the user’s master with Argon2id, making offline brute force expensive.

Four parties, five KMS properties, and four “the attacker cannot” clauses are more than a reader should be asked to hold in their head at once, so Figure 7.14 draws them: who holds which key, and which single compromise—or, for one clause only, which *pair* of compromises—breaks which guarantee of Design Invariant 7.12.1 below.

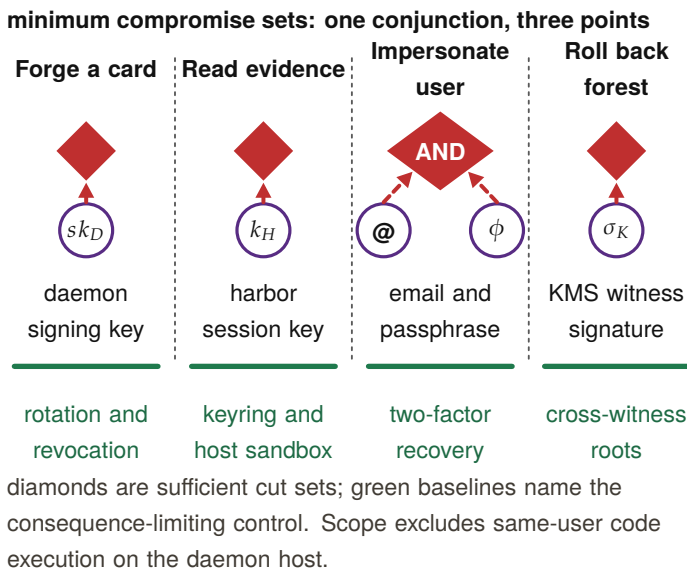


Figure 7.14: The custody theorem has four small attack cut sets rather than one undifferentiated trust boundary. Card forgery, evidence decryption, and forest rollback each have one sufficient secret; user impersonation is the sole conjunctive failure and requires both email and passphrase. Each baseline names the consequence-limiting control for that cut. Same-user code execution stays outside the theorem because that adversary can read every secret the user process can read. [internal]

Magic-link single-use atomicity. The email-gated recovery path requires that each emitted magic-link token be redeemable *exactly once*—otherwise a race window between two consumers can produce two valid recovery completions for the same token. The ProVerif model encodes the property as an injective event: $\text{inj-event}(\text{consumed_for}) \Rightarrow$

`inj-event(issued_for)`. The runtime analogue is a single atomic SQL update with a `WHERE consumed_at IS NULL RETURNING` clause; SQLite’s writer serialization drains the token under the first transaction and returns zero rows to any subsequent transaction. Figure 7.15 shows the model and the runtime side by side.

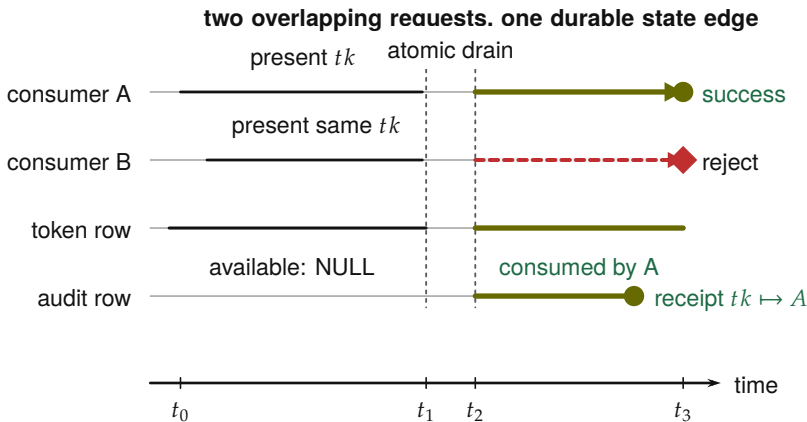


Figure 7.15: Single-use redemption is a concurrency property, not a two-arrow icon. Two consumers present the same account-bound token during one overlap window. Both requests reach the same serialized SQL update, but the token row has only one falling edge: the first transaction changes `consumed_at` from `NULL` and returns a row; the second observes the durable state and returns none. The audit row records the winning consumer without creating a second success path. [internal]

DESIGN INVARIANT 7.12.1 Federated Security, informal. Assuming Ed25519 and SHA-256 are secure, Argon2id parameters meet the 2026 security floor, and the user’s email and passphrase are not simultaneously compromised, an adversary with read access to $\mathcal{K}$ ’s storage, the daemon’s SQLite database, and mesh gossip traffic, but *without* same-user code execution on the daemon’s host, cannot: (i) forge a Harbor Card without the daemon’s private key, (ii) read plaintext evidence without the harbor’s session key, (iii) impersonate the user without both email access and passphrase, (iv) roll back a harbor’s Merkle forest without $\mathcal{K}$ ’s complicity.

The theorem deliberately excludes the same-user adversary (an un-sandboxed agent, a malicious postinstall, a compromised editor extension). Such an adversary reads any file the user can read, which absent OS-mediated key custody includes the daemon’s keys and database. The macOS Keychain integration ships now; native keyring

and hardware-backed keystore extend the theorem's reach to the same-user case.

Recovery and its structural limits. Recovery is email magic link. The structural limit: *if the user loses both passphrase and old machine*, harbor session keys wrapped only for that pubkey cannot be recovered. This is the price of zero-knowledge at-rest encryption. An opt-in Shamir escrow mode [233] (t -of- n secret sharing across $\mathcal{K}$, email, and recovery contacts) trades some zero-knowledge for stronger recovery.

Passkeys, not passwords. Identity is anchored in WebAuthn-backed device keypairs—no phishable secrets, no passphrase on the critical path of every action. New devices enroll by exchanging a short-lived pairing token that re-encrypts the KMS master under the new device's public key. Mobile devices act as *viewers*, not peers: they sign low-stakes actions (approve an ask, bump budget) over a WebSocket viewer channel, but they do not run the full daemon. Each account's harbors gossip Merkle roots among its own devices; cross-account gossip requires explicit signed consent.

What we deliberately do *not* do: require passwords, use TOTP as a primary factor, or couple recovery to a single cloud vendor.

*Exercises 7.14–7.16,
p. 455.*

Multi-node consensus. For fleets requiring strict serializability across nodes, a Raft-backed harbor-root consensus [184] is possible but not specified here; eventual consistency via the Merkle forest plus KMS witness is sufficient for the workloads we have measured. Paxos [141] remains the textbook fallback.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor's LLM. A formal model of “given this evidence trail, will the successor complete the original intent?” requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π , bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **Three layers, each doing what the layer below cannot.** Structural prevention bounds scope before the act; immutable attribution makes every act answerable after it; economic alignment prices what neither can forbid.
2. **Structural scope limitation is a theorem**, not a policy: a process cannot touch what its card does not name, so the first layer costs nothing at runtime.
3. **The trail is a Merkle forest.** Trail integrity, proof soundness, binding, attribution, rollback safety and conservation are the six properties of the ledger; conservation and no-overdraft under concurrent escrow are model-checked.
4. **Decisive allocation has limits.** There is a Pareto-inferior locking instance, so advisory conflict detection is complete where decisive locking is not, and an appeal is sound when it can only widen the record.
5. **A crash is a social event.** Recovery is continuation of the record by whoever next holds the role, not resurrection of the process.
6. **Intent is irrelevant to settlement;** the bond pays on the evidence, whatever the agent meant.
7. **Claim signaling is incentive-compatible above $\delta^* \approx 0.3425$,** the root of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$; TLC agrees on the integer grid, at 0.35 holding and at 0.33 producing the deviation trace.
8. **Cleanup has a lower bound:** no bond schedule makes abandoning work free, and the bound says how much.
9. **Disputes escalate only when the cheaper exit fails:** released, then expired by TTL, then preempted by bond, then ruled by a human, every terminal state writing to one evidence rail; a magic link is redeemed by one row update and refused by the second reader.

7.13 Exercises

§7.3 LAYER 1: STRUCTURAL PREVENTION

- 7.1 CHECK ★** An agent α holds a Harbor Card with `fs:write:auth.ts`, `db:read`, `TTL = 15 minutes`. α delegates to β a card with `fs:`

Solution p. 565

`write:auth.ts`, TTL = 20 minutes. The verifier rejects this delegation. Cite the specific attenuation invariant violated. Now rewrite β 's requested card to be acceptable.

§7.5 THE LIMITS OF DECISIVE ALLOCATION

- 7.2 CHECK ★ Define “advisory claim” and explain, in two sentences, why an advisory regime is strictly more permissive than an enforced-lock regime. Then explain why the Sen impossibility (§7.5) makes that strict permissiveness a *feature*, not a bug. Solution p. 565

§7.6 CRASH RECOVERY AS SOCIAL CONTINUATION

- 7.3 CHECK ★ A reader proposes: “Eliminate bonds. Use post-hoc reputation slashing instead—an agent that defects loses reputation points and is excluded from future work.” Identify two specific failure modes of this proposal that the bonded design avoids. Hint: consider the first-mover problem and Sybil identities. Solution p. 565

§7.7 LAYER 3: ECONOMIC ALIGNMENT

- 7.4 CHECK ★ Using only the payoffs printed in Table 7.5, verify by hand that F strictly dominates T for each player, and check that no cell other than (F, F) is a Nash equilibrium of the one-shot game. Solution p. 565
- 7.5 TRACE ★★ Using $f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$ from the worked example above, evaluate f by hand at the two integer-grid points $\delta = 0.34$ and $\delta = 0.35$. What sign does each give, and what does the sign change prove? Name the committed artifact whose result this matches. Solution p. 566
- 7.6 TRACE ★★ `proofs/economics/delta-threshold.z3` asks Z3 three separate questions about the IC cubic. Name each question and state the exact verdict sequence the committed expected output records. Solution p. 566
- 7.7 TRACE ★★ The chapter’s own verification table (Table 7.9) states that the `claim_signaling.tla` model’s IC invariant fails at $\delta = 0.30$ and holds at $\delta = 0.35$. Confirm both verdicts by hand using the deviation inequality $1 < 2\delta(1 + \delta + \delta^2)$ derived in the text above, and describe in two sentences what a counterexample trace at $\delta = 0.30$ would actually show. Solution p. 567
- 7.8 TRACE ★★ `proofs/economics/sweep-delta.sh` sweeps $\delta \in \{0.30, 0.31, \dots, 0.40\}$ and reports a crossover value. What is the pinned crossover, and how does the script itself judge whether it passed? Solution p. 567

- 7.9** TRACE ★★ The A5 finding (§7.7.5.9) shows that pure deposit deterrence is bounded by the auction coverage $B_T = \mu(1+s)$. Construct an attacker who exploits this bound directly. Then construct a defense—other than “raise the deposit”—that defeats the attacker. Hint: see the composite mechanism $C_{kyc} + \text{reputation gating} + B_{dep}^{\text{per-class}}$. Solution p. 568
- 7.10** TRACE ★★ Using the A6 grid (§7.7.5.10, Figure 7.11), compute the minimum per-round detection probability p_d^* at $\delta = 0.99$ in the negligible-undercut limit $\varepsilon/x \rightarrow 0$. Then argue, qualitatively, what would change if cartel members can re-enter under fresh identities. Does the bond mechanism prevent re-entry, deter it, or merely price it? Solution p. 568
- 7.11** OPEN ★★★ The competitive-insurance mechanism (§7.7.5.8) replaces a single bond price with insurer-agent bids. Propose one alternative pricing mechanism (e.g., a second-price auction with reserve, a Walrasian-style market-clearing rule, or a posted-price catalog) and argue whether it would conditionally Pareto-dominate the static escrow under the CC/NC/RP/NS assumptions of §7.7.5.8. Identify the assumption your alternative most depends on. Solution p. 569

§7.10 FORMAL MODEL

- 7.12** TRACE ★★ proofs/bonded/conservation/Conservation.tla is model-checked by TLC. Pin the reachable-state count and the name of the headline invariant from the committed run log, and say precisely what “reachable” means there—is it the same number as the states TLC generated? Solution p. 569
- 7.13** OPEN ★★★ Suppose Conservation.tla’s Mint action were changed to credit free without also incrementing minted—i.e. `free' = [free EXCEPT ![a] = @ + amt]` stays, but `minted' = minted + amt` is dropped. Predict, from the spec text alone, what a TLC run would show. Which of the checked invariants catches it, which does not, and how deep is the shortest counterexample? Solution p. 570

§7.12 DISCUSSION AND LIMITATIONS

- 7.14** TRACE ★★ Sketch the ProVerif process for the magic-link recovery protocol of §7.12. What is the secrecy property you would prove? What is the authentication property? Identify one attacker capability ProVerif would need to model that is not present in the Phase 1 Harbor Card model of *The Anchor Protocol*. Solution p. 570

7.15 OPEN ★★★ The gossip convergence bound cited for Merkle-root dissemination among a harbor’s devices (*The Anchor Protocol’s* revocation analysis) is reproduced from the standard anti-entropy analysis rather than mechanized. Identify the specific theorem from Demers et al. [60] that would need to be ported to a proof assistant. What would change in the analysis if the daemon mesh is *not* fully connected? Hint: consider gossip on a graph with bottleneck cuts.

Solution p. 571

7.16 TRACE ★★ Sketch a pollution attack against a cuckoo filter that does *not* enforce the load-factor ceiling at 0.95 (Table 7.9’s cuckoo-filter row). What invariant breaks? Why does the ceiling defeat it? Hint: the FP rate bound in Fan et al. [80] is conditional on the regime.

Solution p. 571

7.14 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention:** for any operation routed through the verifier, capability-attenuated tokens make scope escalation physically impossible (Theorem 7.3.1) — a guarantee that holds once the Anchor model’s own assumptions and the runtime’s interception coverage are granted, and no further.
2. **Immutable attribution:** a Merkle forest of evidence trails (§7.4.2) makes anonymous harm impossible *across daemons for damage to shared state*; it says nothing about disclosure to outsiders (§7.7.5.4) or about a same-user adversary on the daemon’s own host (§7.12).
3. **Economic alignment:** Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

Six further contributions thread through this skeleton. The Conservation Theorem (§7.10.3) makes the economic invariant operationally checkable, not just an asserted property of the abstract model. The mutable-signal ledger (§7.4.3) shows how coordination signals can be revoked, renamed, and re-attributed without sacrificing the immutability the rest of the architecture depends on. The federated sovereign (§7.12) replaces single-node scope with an abstract KMS satisfying five named properties, and anchors identity in passkeys rather than passphrases on the critical path. The competitive-insurance pricing mechanism (§7.7.5.8, contributed by Youle) replaces static parameter selection with market-discovered bond prices, recovering classical Rothschild–Stiglitz equilibria in the agent-transactions setting. The expressive-act taxonomy (§7.8) decomposes “coordination” into five priceable classes, because uniform pricing collapses the market to its

highest-stakes class. Semantic cadence and replay (§7.9) give the substrate the empirical grounding to iterate on all of the above.

The advisory design of the resource claim system is the correct resolution of Sen's impossibility result for systems where agents have private knowledge about their own needs. The commons authority provides information, not allocation decisions.

Bond pricing is not a single open problem. It is a lattice: a cleanup-cost lower bound (§7.7.5.1), a scope multiplier (§7.7.5.2), a reputation discount (§7.7.5.3), the Bonded Advisor mechanism (§7.7.5.7), and the competitive-insurance market (§7.7.5.8). Each is precise enough to disprove.

The infrastructure is running. The forest is building. The covenants are auditable. The sovereign is legible. The advisor is bondable. The market is open for bids.

Chapter Appendices

7.A Formal Verification Status

Artifact availability. Every artifact named in Table 7.9 is bundled at <https://portdaddy.dev/whitepaper/artifacts/>. Runtime components are deployed inside the Port Daddy daemon binary and exposed through its public APIs; the source for the verified Rust core is the open-source harbor-card-rs crate (accompanying chapters, §5).

Table 7.9: Verification status of critical claims. “Method” is the formal technique; “Result” is its outcome; “Artifact” names the public bundle file. A claim with both a formal artifact *and* a runtime conformance test is fully closed (**CLOSED**); a claim with formal model but no deployed runtime is spec-only (*spec-only*); a claim with empirical evidence but no formal mechanization is partial (**PARTIAL**).

Claim	Method	Result	Artifact
Coordination channel isolation (§7.4.2)	ProVerif (Dolev-Yao)	CLOSED 3 properties TRUE	isolation.pv
Passkey device pairing	ProVerif	CLOSED 3 properties TRUE	passkey-pair.pv
Federated security (§7.12)	ProVerif, 3-of-4 compromise	CLOSED no attacker reach	federated.pv
Conservation Theorem (§7.10.3)	TLA+ bounded TLC	CLOSED 1,716 reachable states (26,818 generated), invariant holds	Conservation.tla
No-overdraft lemma (§7.7)	fast-check property test	CLOSED 4 invariants, randomized ops	bonds-property tests
Algorithm-confusion immunity	ProVerif (pinned vs. naive)	CLOSED pinned TRUE; counter-trace produced	algconfusion.pv
Delegation chain replay	ProVerif at depth 3	CLOSED chain authenticity TRUE	chain-replay.pv
Magic-link single-use (§7.12)	ProVerif (private-channel cap)	CLOSED injective-event TRUE; 7/7 runtime conformance	magic-link.pv
Cuckoo-filter pollution resistance	fast-check, 5 invariants	CLOSED FP rate within $2B/2^f$ at load 0.95	cuckoo property tests

(continued)

Claim	Method	Result	Artifact
Merkle-Forest binding (§7.4.2)	EasyCrypt + fast-check	PARTIAL skeleton discharged; PROM-formal version not in scope	binding.ec
Auction/static comparison (§7.5.8)	Monte Carlo, 36 configs × 2k trials	PARTIAL finite-grid result; not a universal Pareto theorem	simulation.mjs
Sybil A5 (§7.5.9)	Monte Carlo over K Sybils	PARTIAL coverage-bounded ceiling at B_T	simulation-sybil.mjs
Cartel repeated-game A6	Monte Carlo + expected-value recurrence	PARTIAL $p_d^* \approx 0.0478$ at $\delta=0.95$ in supplied model	simulation-cartel.mjs
Claim-signaling IC (§7.7)	TLA+ (TLC) + Z3 SMT	CLOSED unique root $\delta^* \approx 0.3425$; IC invariant fails at $\delta = 0.30$, holds at $\delta = 0.35$	claim_signaling.tla, delta-threshold.z3
Threat-band defensibility (§7.7.5)	Monte Carlo, 7×3 grid	CLOSED matched bands absorb target class within 1.10× upper bound	threat-bands.mjs
Passkey pairing runtime	—	<i>spec-only</i> model exists; no runtime	passkey-pair.pv
Federated KMS Cloudflare Worker	—	<i>spec-only</i> model exists; no Worker yet	federated.pv

Limitations. Four mechanizations are out of scope for this paper but explicitly named so a reader can audit what is and is not proved: a PROM-formal Merkle binding in EasyCrypt; a Coq or Lean mechanization of the Pareto strategic game; a formal analysis of the composite Sybil-deterrence mechanism (C_{kyc} + reputation gating + per-class B_{dep}); and a multi-class generalization of the repeated-game cartel result to heterogeneous coordination classes (§7.8). The present claims stand on the evidence stated in Table 7.9.

7.B Worked Example: One Agent, All Layers

To make the three-layer architecture concrete, this appendix traces one agent—call it α —through a single bounded task: “Fix the login bug in `auth.ts` and regression-test it.” The example exercises capability issuance, advisory claims, the evidence chain, a mid-task crash, successor resumption, and final settlement. All values are illustrative rather than tuned.

The architecture so far has been described layer by layer. The point of this appendix is to watch the layers *compose*: how a single bounded task progresses through the system, where each layer earns its keep, and what the daemon does *not* touch. The example is deliberately mundane—one small bug, one agent, one principal—because the value of the architecture should be legible in the small before it is trusted in the large.

The task. A principal P wants an agent α to fix a login bug in a hypothetical file `auth.ts` and regression-test it. The acceptance criteria are stated up front:

1. the failing test `login.regression.test.ts` passes;
2. no other test regresses;
3. the diff touches only `auth.ts` and the test file.

P estimates the cleanup cost of α 's defection or sabotage at \$5—one operator-hour at \$5/hr equivalent, since this is a local agent rather than a cloud workload. Figure 7.16 traces what follows across the three architectural layers.

Phase A: Setup (minutes 0–1)

Before α writes a single byte, two artifacts must exist: a capability that bounds what α can touch, and a bond that prices what α might break.

Step 1: Capability issuance (Layer 1). P requests a Harbor Card for α via the daemon, specifying the capability set `{fs:read:repo, fs:write:auth.ts, fs:write:tests/**, cmd:test}` and a TTL of 30 minutes. The Anchor Protocol (§7.3, accompanying chapters) signs the card. The effect is structural rather than advisory: α can prove its identity and capabilities to any verifier, but it *physically* cannot write to, e.g., `database.ts`—attempts return a capability error before the file system is even touched. This is the “physically” that makes the Layer 1 promise non-negotiable.

Step 2: Bond posting (Layer 3). With the capability in place, the Bonded Advisor (§7.7.5) quotes a bond proportional to the cleanup cost: $B_\alpha = \$5 \cdot (1 + s) = \6 at slope $s = 0.2$. P posts \$6 into escrow. The settlement preconditions are now fixed up front: (1) acceptance criteria

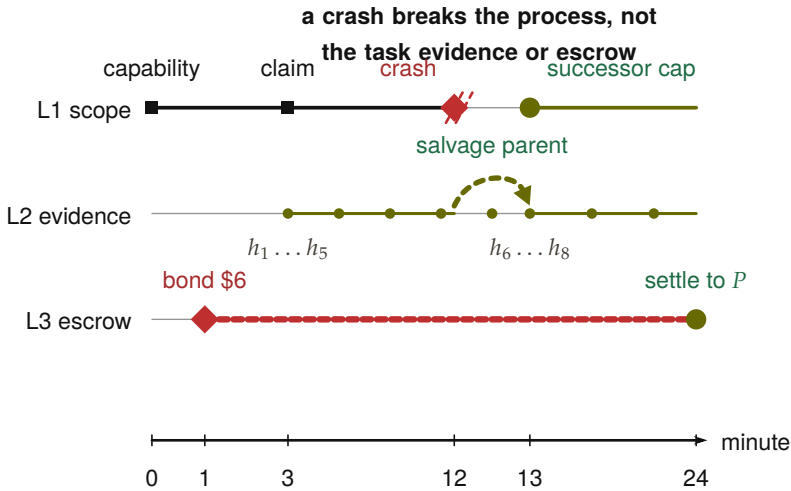


Figure 7.16: The worked example is a continuity braid over one task. Structural authority belongs to a process and visibly breaks at minute 12; the successor receives a new capability at minute 13. The evidence chain does not break: its salvage-parent edge binds $h_1 \dots h_5$ to $h_6 \dots h_8$. The escrow rail spans the entire task and settles at minute 24. Identity is disposable, while attribution and pre-funded cleanup exposure persist. [internal]

met $\Rightarrow$ \$6 returned, (2) partial completion $\Rightarrow$ pro-rata, (3) sabotage $\Rightarrow$ \$6 liquidated to the commons treasury. The bond is the conversion of α 's "character" into a settled amount; once it is posted, the question of whether α is trustworthy no longer determines settlement.

Phase B: Work begins (minutes 1–12)

Phase A's contracts are now in force. α does the work the principal hired it for, and the daemon records what happens.

Step 3: Advisory claim (§7.5). α claims `auth.ts` via `pd session files add auth.ts`. The daemon's conflict graph is empty for this path, so the claim succeeds with no conflict notification; had another agent already held the path, α would have been told and would have decided locally whether to coordinate or wait. The claim's TTL is bounded by the Harbor Card's—30 minutes—so a crashed α cannot squat on the file indefinitely.

Step 4: Evidence chain begins (Layer 2). α opens the work with a scope note: "Scope: `auth.ts`. Hypothesis: `cookie-domain handling on subdomain login fails`. Validation: `npm test login.regression`." The note's content hash h_1 chains into the session's Merkle root $R_{\text{session}}^{(1)}$, and from then on every action α takes—a read receipt for `auth.ts`, a diff

blob for the partial fix, the stdout of the failing test run—is appended as a new entry. Five notes deep, $R_{\text{session}}^{(5)}$ is the canonical commitment to α 's state of work. The chain is the part of the architecture that survives anything that happens next.

Phase C: Crash and recovery (minutes 12–13)

The crash is where the architecture stops being an abstraction. In Phase B everything went right; in Phase C something goes wrong, and the layers either earn their keep or they don't.

Step 5: Mid-task crash (§7.6). At minute 12, α 's process crashes—let's say the LLM provider timed out. The daemon's heartbeat monitor observes α as unresponsive after 90 seconds and marks the session abandoned. α 's claim on `auth.ts` releases on TTL; the evidence chain is sealed at $R_{\text{session}}^{(5)}$ and committed into the harbor root $R_{\text{harbor},\ell}$ at the next epoch boundary. *Critically, no information is lost:* the partial diff, the test output, and α 's scope note are all in the chain. The crash terminated a process; it did not terminate the work.

Step 6: Successor agent. P spawns a fresh agent β with a new Harbor Card and the same capability set. β 's first action is `pd salvage`: the daemon returns the abandoned session, the scope note, the partial diff, and the last test output. β reads the scope note (“cookie-domain handling on subdomain login”), inspects the partial fix that α left behind, and finishes the work. The new evidence chain $R_{\text{session}}^{(\beta)}$ explicitly references $R_{\text{session}}^{(5)}$ as its salvage parent. Attribution is preserved across the identity change (§7.4.4)—the diff that ships at minute 24 is traceable back to the same principal P that posted the bond at minute 1, even though the agent that did most of the work is gone.

Phase D: Settlement (minute 24)

The work is done. The daemon now decides what was actually accomplished and acts on the settlement preconditions fixed in Phase A.

Step 7: Settlement. β 's final action is `pd done`. The daemon's acceptance-criteria checker runs the regression suite, confirms `login.regression.test.ts` passes, confirms no other test regresses, and confirms the diff is bounded to the two files declared in the Harbor Card. All three criteria met. The daemon releases \$6 from escrow to P , signs a settlement record over the evidence-chain root, and appends that record to the harbor Merkle forest (§7.4.2). The settlement is now itself an immutable, attestable artifact.

What did each layer do?

- *Layer 1 (structural)* kept the agent inside the declared blast radius. The crash, the recovery, and the settlement all happened inside a capability-bounded scope. A capability error at minute 7 would have been a fast-fail, not a corrupted repository.
- *Layer 2 (attribution)* kept the work durable across the crash. Without the Merkle-chained evidence, β would have started from zero; with it, β resumed at minute 12's state.
- *Layer 3 (economic)* kept the principal honest in scoping. P posted real collateral and recovered it on completion; had α (or β) sabotaged, the cleanup cost would have been pre-funded.

What did *not* happen. The daemon never decided which agent should hold the file, never decided whether the fix was “good enough” in a subjective sense, and never punished the crash. It enforced what *cannot* be (Layer 1), recorded what *did* happen (Layer 2), and settled against the published criteria (Layer 3). The substantive judgments—“is this scope right”, “is this fix correct”—remained with P and the acceptance-criteria checker. This is the Sen-grounded posture of §7.5 made operational.

7.C Scope Boundary Checklist

Section 7.12 (*Discussion and Limitations*) and Appendix 7.A's own limitations paragraph carry the substantive boundary discussion; this closing checklist exists so a reader who jumped straight to the appendices does not have to hunt through the body for the three bounding assumptions that recur most.

- **Threat model exclusions:** Collusion across all independent sub-systems (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation maturity:** While the core protocols (Anchor, SQLite single-writer) are IMPLEMENTED and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in PARTIAL OR DESIGNED phases.

None of the three is a flaw the chapter is unaware of; each is a structural reality of building zero-trust coordination on top of POSIX primitives, already priced into the hedged claims made where each mechanism was introduced.

WHAT THE FEDERATED HARBOR RECEIVES One machine can now authorize, observe, adjudicate, and settle its own work. A second machine introduces a different operator and a different source of truth. The final chapter asks which capabilities and records may cross that boundary without pretending that either local authority has become global.

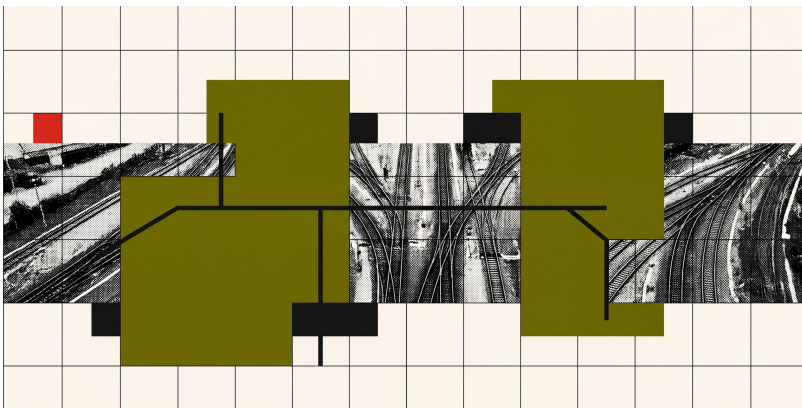
8

The Federated Harbor

The question this chapter answers

What can cross a trust boundary without moving the authority itself?

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.



Good fences make good neighbours.

Robert Frost, Mending Wall, 1914

WHAT CAN CROSS A TRUST BOUNDARY WITHOUT MOVING THE AUTHORITY ITSELF? The current Relay federates harbor-scoped events over outbound HTTPS and SSE; it does not replicate daemon state or create consensus between operators. Local admission remains local. The research result is narrower and sharper still: R6 and CR-1/2/3 show exactly when relayed reports around a cycle can expose an inconsistency that a missing pairwise check cannot.

8.1 Introduction: The Two-Machine Problem

A demonstration is scheduled for 4pm. Alice runs the back end of the demo on her laptop; Bob runs the front end on his. Each has a working Port Daddy daemon: capability tokens are minted and attenuated under the Anchor Protocol [187], the workspace history is Merkle-chained and replicated locally per the Bonded Commons [188], and bonds for cleanup of botched migrations are posted in the local collateral pool. Inside each laptop, the trust story is closed and the formal-verification artifacts hold.

The demo nevertheless does not work. Three failures, none of them inside either machine:

1. Alice's agent obtains a `db:write` card for the staging database, attempts to use it against Bob's daemon, and is refused. Bob's daemon has never seen Alice's signing key. The card is well-formed under the Anchor Protocol, but Bob has no root of authority for it; from his daemon's point of view, the card is gibberish.
2. Five minutes before the demo, Alice's operator revokes the `db:write` card after spotting that it has been over-attenuated. The revocation propagates within Alice's mesh in under a second by the gossip protocol of Anchor §4. Bob's harbor never hears about it. The card remains live on Bob's side.
3. The migration runs anyway, lands in Bob's staging database, and corrupts the demo schema. Alice posted a bond in her harbor against exactly this case. The bond sits in her harbor's collateral pool. Bob's harbor measured the damage. Neither harbor can, on its own, route the bond to where the damage is.

Each failure is a different facet of the same underlying gap. Both daemons are internally consistent. Neither is a root of authority for the other. The single-machine sovereign of the prior papers does not extend to the case of two machines under two operators with no shared administrative parent.

We call this the **two-machine problem**. The point of stating it this baldly is that the failure modes are not exotic; they are what every developer who has tried to dovetail two local agent fleets on a shared task has seen. The bug is structural. The bug is not in either daemon. The bug is in the assumption that one daemon's authority extends past the machine boundary by default. It does not, and it should not.

The trick is not to extend a single sovereign across machines. The trick is to admit there are two and to specify what they owe each other.

THESIS

This paper extends the Anchor Protocol, the Bonded Commons, and the daemon-as-correlation-device argument to the multi-administrative-domain case. We are not introducing a new substrate; we are showing that the existing substrate admits a clean federation, and that the federation rules are sharper than they look. Where the prior papers spoke of *the* commons authority, this paper speaks of a *mesh of* mutually witnessing commons authorities, each sovereign inside its own machine, none sovereign over the others, and all of them bound to the same audit substrate.

8.1.1 What this paper is and is not

This chapter closes the book. *The Anchor Protocol* supplies the cryptographic-token substrate; *The Bonded Commons* supplies the economic and governance model; this chapter specifies the federation boundary. The dependencies are concrete: §8.4 extends Anchor's authorization chain across a trust boundary; §8.5 extends its revocation mesh; §8.6 extends Bonded's Conservation invariant.

What this paper is *not*: it is not a permissionless-blockchain re-design. Port Daddy daemons do not run consensus. The Federated Harbor explicitly rejects the assumption common to most modern federation proposals [253, 226] that a shared ledger is the right substrate. The cost of that simplification is that we cannot lean on a chain for ordering, equivocation detection, or settlement; we must construct each of those primitives from peer gossip plus a federated witness log. The advantage is that the design composes cleanly with the local-first, single-operator-machine architecture of the prior two papers.

8.1.2 Contributions

The contributions are concrete. We separate them by mechanization status: which are theorems, which are bounded threat models, and which are sketches.

1. **The four-element federation topology (§8.2).** A precise statement of the boundary between intra-harbor and inter-harbor scope. Defines harbor sovereignty as a property characteriza-

tion in the style of Bonded’s Admissible KMS, so that subsequent claims can be checked against it.

2. **The inter-harbor capability transfer ceremony (§8.4).** A four-message protocol exchanging a Harbor Card issued by A for an attenuated card valid at B , without either daemon trusting the other’s signing key as a root. The ceremony binds the issuing harbor’s current epoch root into the envelope, which is the substrate that revocation in §8.5 hooks into. ProVerif model in Appendix 8.B (status: PARTIAL, model drafted; soundness pending mechanization).
3. **Federated revocation mesh (§8.5).** The multi-administrative-domain extension of Anchor’s revocation gossip. Each harbor publishes an epoch root to a witness log. Under an explicit complete-overlay, reliable-round model, epidemic dissemination takes expected $\Theta(\log m)$ rounds; no finite worst-case bound survives an unbounded partition (Theorem 8.5.1).
4. **Cross-harbor settlement (§8.6).** A conditional protocol for clearing a bond posted at A against damage measured at B . The escrow’s role is structurally bounded only when the custody ledger, not merely a signed promise, restricts recipients, fee, and terminal transitions. The construction is trusted and specified, not an HTLC-style trustless swap [108].
5. **Cold-start admission ceremony (§8.7).** A protocol for a new harbor joining an existing mesh without prior reputation, using a bonded-sponsor pattern: an existing harbor posts a bond on the newcomer’s behalf, which is forfeit on misbehavior in a probationary epoch. This is the federation-layer analogue of competitive insurance from Bonded.
6. **Open problems (§8.12).** Five named open questions: (a) whether the correlated-equilibrium reframing of Bonded survives the multi-principal setting cleanly; (b) whether bonds can settle without a trusted, conditionally restricted custody party; (c) whether the folk-theorem cartel-resistance argument from Bonded weakens at the federation layer, and by how much; (d) whether bonded sponsorship can avoid invitation-only capture in a thin cold-start market; and (e) what deployment-specific dissemination tail bound is justified. We state these as open, not as solved.

The proposal that scoped this paper [190] listed two further contributions (a formal definition of harbor sovereignty, and equilibrium results across federations co-authored with Youle) which are present here in skeletal form and will become central in a subsequent revision once the open problems above resolve. We will not claim a result we do not have.

8.2 The Federated Authority

The Bonded Commons paper defined a *commons authority* as a pre-transactional trust infrastructure with sovereign scope inside a single machine. Inside that scope, the daemon is the unique correlation device that turns mutually-suspicious agents into a correlated equilibrium [188]. The single-machine sovereign was an honest simplification. We now drop it.

The federation design has four elements (Table 8.1): two sovereign harbors, a witnessed epoch-root log, and a settlement escrow. The escrow is structurally bounded only under the custody assumptions in §8.6.1. Every primitive in the rest of the paper attaches to one of these elements.

Table 8.1: Federation is four independent rails between two sovereign harbors, each keeping its own signing root; there is no shared root. The witness rail correlates and exposes equivocation but authorizes nothing; the capability and revocation rails carry attenuated authority and later invalidation in opposite directions; the settlement rail relates a bond posted at A to damage measured at B, and its custody bound is conditional on the settlement gate, not an invariant. No rail makes either harbor a root of authority for the other. [internal]

Rail	What crosses	Direction	What it does not do
witness	the signed epoch roots $R_A^{(e)}, R_B^{(e)}$ and any equivocation evidence	both ways	authorize work: correlation, not authorization
capability	an attenuated card, $C_A \mapsto C'_B \subseteq C_A$	A to B	widen scope or lifetime at any hop
revocation	the anti-entropy delta of revoked ids	B to A	promise a finite deadline under partition
settlement	a bond posted at A against damage measured at B	both ways	bound custody unconditionally: it holds only if the settlement gate holds

8.2.1 Harbor sovereignty as a property characterization

In Bonded, the Admissible KMS was characterized by a list of properties rather than by a specific construction (per-machine signing key, per-machine SQLite, etc.) so that the argument carried over to any concrete instantiation. We adopt the same posture here.

Numbers by hand — Sovereignty over the card is not sovereignty over what ■

it touches. Return to Failure 1 of §8.1: Alice’s `db:write` card is well-formed under the Anchor Protocol, and Bob’s daemon refuses it anyway, because Bob’s daemon has never rooted Alice’s signing key. Check the checklist below against this one case before reading it formally. D_A (Alice’s daemon) is the only party that minted the card, the only party that could have attenuated it before issue, and the only party that can revoke it — three of the five items already visible in one refused request. The failure also shows item 4 by its absence: acceptance is D_B ’s call alone, and no amount of well-formedness at A can compel it. Once the transfer ceremony of §8.4 lets D_B mint a derived card, sovereignty over what that derived card does to B ’s own database still belongs to D_B , never to D_A — exactly the boundary the property characterization below draws.

DESIGN INVARIANT 8.2.1 Harbor Sovereignty. A daemon D_A is *sovereign over harbor A* if it has exclusive authority over five things:

1. **Issuance.** Only D_A can mint a Harbor Card whose root signature verifies under A ’s signing key sk_A .
2. **Attenuation.** Only D_A can apply a local attenuation predicate att_A that restricts an inbound capability before it is verified by an A -side agent.
3. **Revocation.** Only D_A can decide that a card it issued is revoked. The decision becomes globally visible by the gossip protocol of §8.5; D_A does not control when other harbors learn of it, but it controls whether the revocation event exists.
4. **Acceptance.** D_A alone decides which inbound cards from other harbors to accept and under what local policy. No external party can compel acceptance.
5. **Evidence binding.** The epoch root $R_A^{(e)}$ that D_A publishes to the witness log is signed under sk_A and binds the entire A -side state of issued, attenuated, and revoked cards up to epoch e . No other harbor can publish a root that purports to be A ’s.

This is a deliberately narrow characterization, intended to hold of any conforming daemon implementation rather than proved from a formal model. It does *not* say that D_A is sovereign over everything an A -side agent does; the agent may be operating against a database hosted at B , in which case B -side policy applies and D_A has nothing to say about it. Sovereignty is over the *issuance and revocation surface of A’s own cards*, not over downstream effects.

We will use Design Invariant 8.2.1 as the discipline against which subsequent protocols are checked. The transfer ceremony of §8.4 produces a card valid at B ; if it required B to verify under sk_A in the hot path, it would violate B ’s sovereignty by making B ’s acceptance contingent on an external authority. We will see that it does not.

8.2.2 Inter-harbor scope

The dual notion is equally important. *Inter-harbor scope* is the scope of operations whose effects cross the trust boundary between sovereign harbors.

DESIGN INVARIANT 8.2.2 Inter-Harbor Scope. An operation o has *inter-harbor scope* (A, B) if its execution causes a state change observable to B under authority that originated at A . Examples: an A -issued card used to write to a B -hosted database; a settlement clearing a bond posted at A against damage measured at B ; a revocation issued by A becoming visible to a B -side verifier. This boundary is a scoping decision this chapter's protocols are built to respect, not a theorem derived from a model: every later claim about what a protocol closes or defers (§8.3 onward) is checked against exactly this line.

Most operations are *intra-harbor*: minting, verifying, attenuating, and revoking cards under one harbor's authority, observed by agents under that same harbor. The Anchor Protocol's proofs cover the intra-harbor case end-to-end. The Federated Harbor's contribution is exactly the inter-harbor surface: every protocol that determines its guarantees lives at the boundary.

8.2.3 The witness log

The witness log is the device that makes the federation auditable without making any harbor sovereign over the others. It plays the same role that the Merkle Forest plays inside a single machine in *The Bonded Commons* (§4.1): a tamper-evident accumulator that any third party can verify without needing to trust the publisher.

DESIGN INVARIANT 8.2.3 Witness-Log Admissibility. A witness log W is admissible for the Federated Harbor if it provides:

1. **Append-only.** Once W records an entry, W cannot retract or re-order it.
2. **Per-harbor latest-root binding.** For each harbor X in the federation and each epoch e , W records at most one root $R_X^{(e)}$ signed under sk_X .
3. **Auditable consistency.** Any third party can verify, given $R_X^{(e_1)}$ and $R_X^{(e_2)}$ with $e_1 \leq e_2$, that $R_X^{(e_2)}$ is a Merkle-consistent extension of $R_X^{(e_1)}$.
4. **Equivocation evidence.** If an auditor obtains two differently signed roots for the same harbor and epoch, it can verify the

contradiction immediately. Disseminating both views to a common auditor depends on the overlay and partition model (Theorem 8.5.1).

5. **Threshold publishability.** The act of publishing $R_X^{(e)}$ to W does not require D_X to trust any other party; the cross-signatures collected from other daemons in the federation are an auditor convenience, not an authorization gate.

This list is intentionally close in shape to the five properties of the Admissible KMS in *The Bonded Commons* (“Federated Sovereign”). The Bonded paper used the same posture — characterizing the property surface, then exhibiting a concrete construction that satisfied it — and we follow that pattern. Section 8.9 sketches one concrete instantiation built on Certificate Transparency-style append-only logs [145] cross-witnessed in the Sigstore/Rekor pattern [180]; the substrate need not be unique.

8.2.4 What the federation is not

It is worth saying what the federation is not, so that the rest of the paper does not have to keep refuting positions it never took.

The federation is *not* a consensus protocol. There is no global agreement on the order of events; there is per-harbor monotonicity (epoch roots only extend), there is cross-harbor witnessing (each root is published to a shared log), and there is detectable equivocation. None of these require harbors to agree on a global order, and the prior papers’ designs explicitly avoid that overhead.

The federation is *not* a hub-and-spoke topology with a privileged coordinator. The witness log is replicable; in practice a small set of mutually-witnessing log instances suffices, and any harbor can read from any of them. The hub-and-spoke failure mode — one well-funded coordinator captures the federation by becoming everyone’s only witness — is a real risk we discuss in §8.10, and the structural pushback is gossip-based witness redundancy.

The federation is *not* a permissionless market in identities. New harbors enter through the admission ceremony of §8.7, which requires either a pre-existing bond or an existing harbor sponsor. This is by design; the cost is that the federation is invitation-bounded, the benefit is that Sybil attacks against the federation [66] cost the attacker as much per identity as honest entry does.

Figure 8.1 draws the fence: the five powers inside, downstream effects outside.

With the boundary in view, the recall prompts separate sovereign powers from downstream consequences.

RECALL

1. Without looking:
name the five things
Design
Invariant 8.2.1 says a sovereign daemon D_A has exclusive authority over.
2. Why does Design
Invariant 8.2.1 say sovereignty is *not* violated when an A -side agent operates against a B -hosted database under B ’s policy?
3. Name the three things this chapter says the federation explicitly is *not* (§8.2.4), and the one mechanism each rests its pushback on.

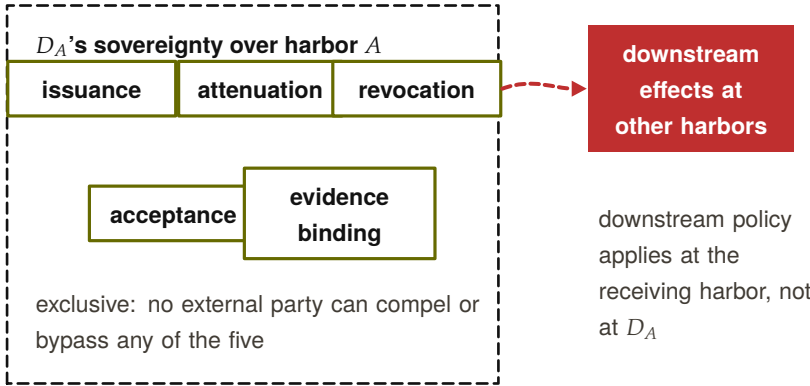


Figure 8.1: Five powers inside the fence, everything downstream outside it. Definition 8.2.1 gives D_A exclusive authority over exactly five things: issuance, attenuation, revocation, acceptance, and evidence binding. What an A -side agent does at a B -hosted database sits outside the fence by definition – B 's policy applies there, not D_A 's, which is exactly why the transfer ceremony (§8.4) cannot require B to verify under D_A 's key in its hot path without violating B 's own sovereignty. [internal]

8.3 Threat Model

We make the threat model explicit before introducing the protocols so that the design choices can be checked against named adversary powers. We follow the Dolev–Yao convention [63] extended to the federation case: an attacker controls every wire between harbors, can intercept and replay any cross-machine message, and may collude with any bounded number of harbor operators. The attacker cannot break the underlying cryptographic primitives (Ed25519 signatures, SHA-256 hashes); these assumptions are inherited from Anchor.

Numbers by hand — A four-step breach, traced. Before naming attacker classes in general, trace one all the way through. `proofs/relay/CardRevocation.tla` models a single relay with a revoked set and an `everRevoked` ground-truth set, and TLC checks the invariant `RevocationFinal == badAccept == FALSE` against three configurations. **Baseline** (no adversary action available): 13 reachable states, invariant holds. **Rollback** (a `RollbackRestore` action can restore the relay's database, including revoked, from a backup taken before a revocation): the same 13-state search space now contains the four-transition trace

$$\text{Issue}(c) \rightarrow \text{Revoke}(c) \rightarrow \text{RollbackRestore}(c) \rightarrow \text{Accept}(c),$$

and `badAccept` is true at the end — the restored revoked set no longer contains c , so `Accept` (which consults only `revoked`) admits a card the

relay had already promised to refuse. This is not a bug in the checker; it is ADR-0018 Attack Vector 2, reproduced on four lines. **Epoch** (the fix: an external epoch counter that Revoke bumps and that Rollback-Restore does not roll back, with Accept additionally requiring $\text{cardEpoch}[c] = \text{epoch}$): the same four-step trace no longer breaches, because the restored card's epoch is stale even though its revoked bit was rolled back, and TLC reports the invariant holding over a larger, 76-state space. Three runs, one relay, one added counter: that is the entire mechanism behind “only-issuer-revokes” being trustworthy across a backup/restore mishap, and it is the reason this chapter's own per-epoch roots $R_X^{(e)}$ (§8.2.3) are an external, monotonic anchor rather than a value the relay's own database could quietly roll back.

8.3.1 In-scope attacks

The protocols in this paper are designed against the following attackers.

Cross-realm identity spoofing. An attacker presents a card at B claiming issuance by A , using a forged signature under sk_A . *Closed by* signature verification against A 's public key as recorded in the witness log (§8.4). Reduces to Ed25519 EUF-CMA security [27].

Replay across the boundary. An attacker captures a valid xfer_offer from A to B at epoch e and replays it at epoch $e + k$. *Closed by* the binding of $R_A^{(e)}$ into the envelope; the verifier at B checks that R_A is the current root, and a stale envelope is rejected (§8.4). The window between issuance and the next epoch is the structural attacker window, named and bounded.

Capability inflation across transfer. An attacker presents at B a card C_B' that claims authority beyond the attenuation predicate applied at A . *Closed by* the requirement that the verifier at B recompute the composed attenuation $\text{att}_B \circ \text{att}_A$ from the envelope rather than trust an intermediate claim (§8.4); reduces to Anchor's intra-harbor attenuation invariant.

Stale-revocation acceptance. An attacker at B presents a card revoked at A but not yet observed at B . Under the reliable connected-overlay model the window has expected scale $\Theta(\Delta \log m)$; during a partition it is unbounded. A bond can price a configured service-level window, not eliminate the tail.

Cross-witness equivocation. A malicious daemon D_A publishes contradictory roots to isolated witnesses. The contradiction is cryptographically verifiable once a common auditor receives both views, but the

time to that event is topology- and partition-dependent. A witness bond prices confirmed equivocation; it does not create a delivery deadline. Table 8.2 lays out all five properties against what each one prevents.

Table 8.2: The five properties Witness-Log Admissibility (Design Invariant 8.2.3) requires, and the specific failure each one closes. Transfer, revocation, and settlement all attach to this log, so a missing property here reopens a failure in every one of them. [internal]

property	what it prevents
append-only	retracting or reordering a published root after the fact
per-harbor latest-root binding	a harbor silently publishing two roots for one epoch
auditable consistency	an inconsistent history between two epochs going unverifiable
equivocation evidence	a proven contradiction requiring anything beyond the two signed roots
threshold publishability	publication itself requiring another daemon's permission

Settlement redirection or equivocation. A signed escrow promise makes misconduct attributable. Prevention requires a custody ledger whose transition API cannot name an arbitrary recipient or fee. Design Invariant 8.6.1 states the bound conditionally on that mechanism; without it, the full bond remains at risk.

Harbor membership forgery at the federation boundary. An attacker presents a card from a harbor purporting to be in the federation but that has no admission record. *Closed by* the admission ceremony (§8.7) which produces a permanent, witness-logged enrollment record co-signed by an existing harbor's sponsor bond. A claimed harbor with no enrollment record is treated as outside the federation regardless of how well-formed its cards appear.

8.3.2 Out-of-scope attacks

We are explicit about what is not closed.

Compromise of a harbor's signing key. If sk_A leaks, the attacker can issue arbitrary A -side cards until A rotates the key and publishes the rotation to the witness log. We inherit Anchor's key-rotation procedure unchanged. The federation does not close the underlying primitive; it

does bound the blast radius to A 's sovereign surface, which is exactly the right scope.

Sybil at the federation layer. An attacker who can pay the admission cost m times can run m federated harbors. The cost ratio between honest and adversarial admission is the decisive parameter; we discuss it in §8.7 but do not give a tight bound. This is named open work.

Cartel formation across harbors. Bonded's folk-theorem cartel-resistance argument relied on the daemon as a correlation device inside one machine. Across harbors, cartels can form offline and present a unified front to the witness log; the witness log records that all harbors agreed, which is consistent with both honest agreement and cartel collusion. We acknowledge this and state the open work in §8.12.

Centralization gravity. Historically, federated identity systems concentrate on one hub within five to ten years [165]. The Federated Harbor's structural pushback — replicable witness logs, no central coordinator — is design discipline, not theorem. We formally classify this in §8.10.

Side channels. Constant-time signature verification, cache-side-channel resistance, and timing-attack resistance are inherited from the underlying libraries; we do not re-prove them and we note where we trust them.

8.3.3 Threat-band visualization

Figure 8.2 lays out the threat surface as three bands, distinguished by weight rather than hue so the figure survives greyscale and colorblind rendering alike: **Stopped** (mechanized claims, heaviest fill), **Bounded** (a named threat inside a named window, lighter fill), and **Open** (acknowledged frontier, drawn in cobalt outline as the one band with no formal artifact behind it). The boundary between the Bounded and Open bands shifts left over time as future work mechanizes what is presently only bounded. We will refer back to this figure when discussing limitations.

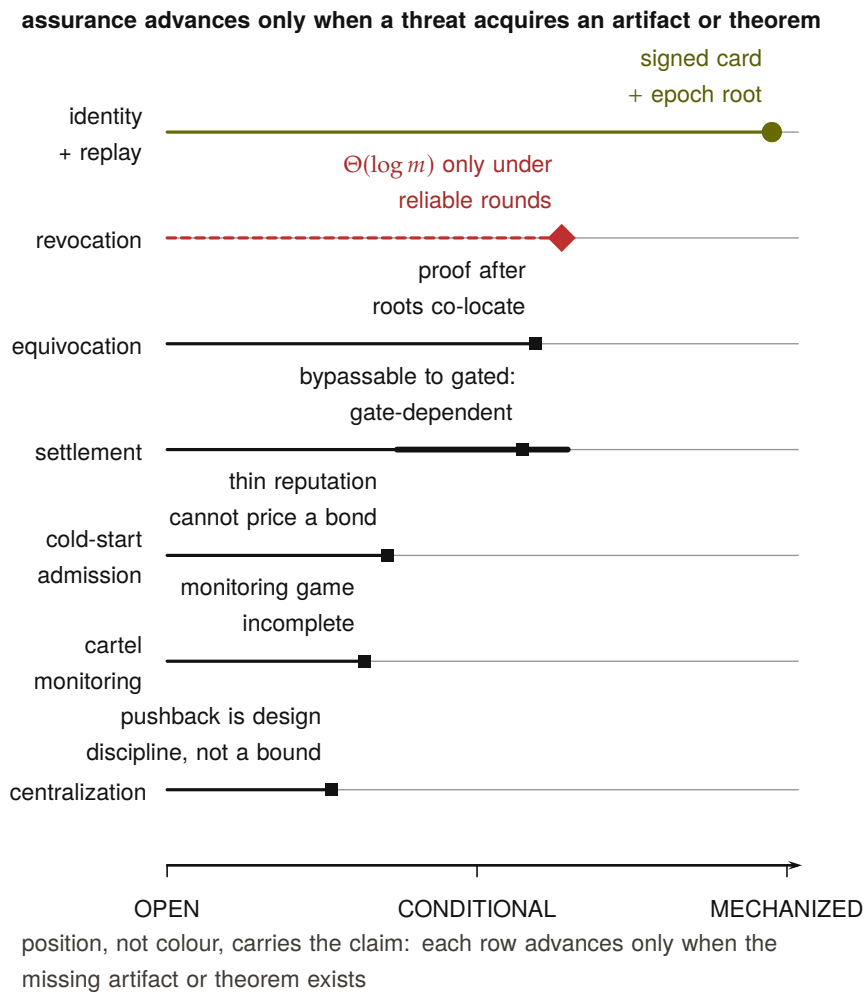


Figure 8.2: The assurance landscape is uneven and falsifiable. Position encodes how far each threat has moved from an acknowledged gap toward a mechanized artifact. Identity and replay (gold) have the strongest concrete evidence. Revocation (red) is conditional on connected reliable rounds (Theorem 8.5.1); equivocation and settlement are conditional on named connectivity or custody hypotheses, and the wide settlement interval shows how far its assurance moves if the custody gate is bypassable (§8.6.1). Cold-start admission, cartel monitoring, and centralization remain open. A row advances only when the missing artifact or theorem exists. [internal]

8.4 Cross-Harbor Capability Transfer

The first governing protocol is the cross-harbor capability transfer ceremony. An A -side agent holds a Harbor Card C_A minted by D_A under Anchor Phase 2 or Phase 3. It needs to present an equivalent capability at B . The Anchor Protocol’s intra-harbor delegation already supports attenuation; we extend that to the case where the attenuation crosses the trust boundary.

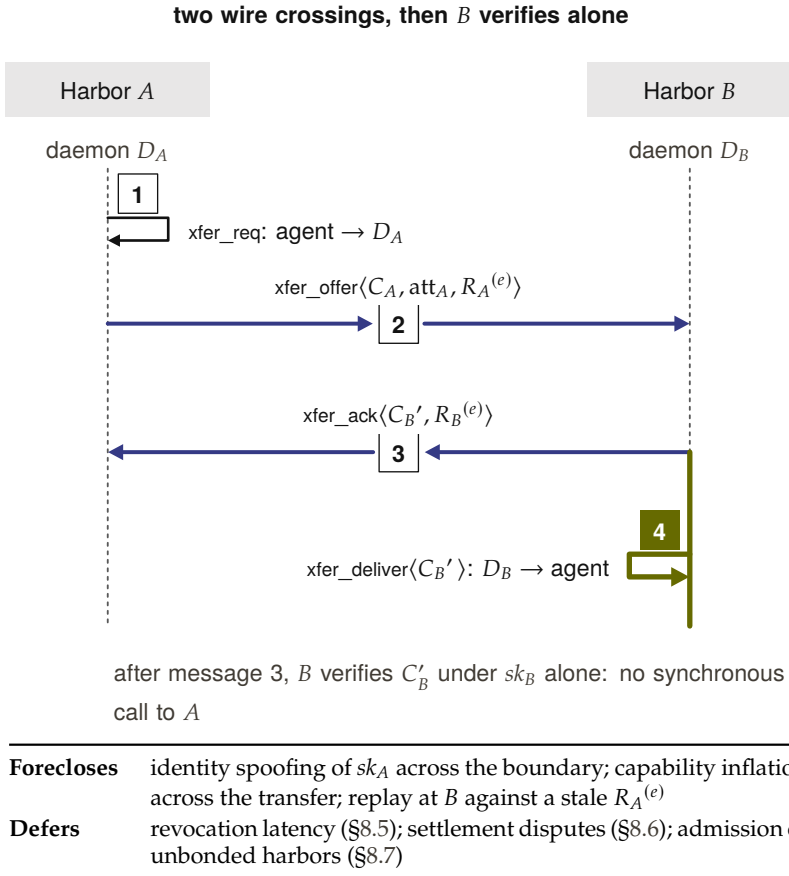


Figure 8.3: The transfer ceremony crosses the wire twice, then B is on its own. Message (1) is local: the agent presents C_A to D_A . Messages (2) and (3), drawn in indigo, are the only ones that cross the trust boundary: the signed offer binds att_A and A ’s epoch root, and the signed acknowledgement returns the attenuated card $C_B' = att_B(att_A(C_A)) \subseteq C_A$ (Theorem 8.4.1) with B ’s epoch root. After (3), gold marks the hot path: D_B delivers C_B' locally (4) and every later presentation verifies under sk_B alone. Harbor A still matters asynchronously — a later $R_A^{(e+1)}$ can revoke the transfer once it reaches B ’s audit surface (Figure 8.4) — but that path is not on the hot path drawn here. [internal]

8.4.1 The four-message ceremony

Figure 8.3 shows the protocol as a sequence diagram. The four messages are:

1. $\text{xfer_req}(C_A, B)$ — Alice’s agent presents C_A to its local daemon D_A and names B as the target harbor. The request is intra-harbor; standard Anchor verification applies.
2. $\text{xfer_offer}(C_A, \text{att}_A, R_A^{(e)})$ — D_A constructs an envelope binding the card, an attenuation predicate att_A specifying what subset of C_A ’s authority is being offered for cross-realm use, and its current epoch root $R_A^{(e)}$. The envelope is signed under sk_A and sent to D_B .
3. $\text{xfer_ack}(C'_B, R_B^{(e)})$ — D_B verifies the envelope’s signature against A ’s public key as recorded in the witness log; verifies that $R_A^{(e)}$ is the current root for A ; applies its own local attenuation att_B ; and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$, a card valid only at B , signed under sk_B . The acknowledgment is sent back to D_A along with B ’s current epoch root.
4. $\text{xfer_deliver}(C'_B)$ — D_B delivers C'_B to the agent at B that will use it. Subsequent presentations of C'_B at B verify under sk_B alone — no A -side lookup is needed at the hot path.

Each derivative C'_B carries the transfer identifier of the offer that produced it, and D_B mints at most one derivative per identifier, so a replayed xfer_offer cannot produce a second C'_B ; the identifier is part of what the counter-signature covers. The critical property is that after message (3), no further synchronous interaction with A is required for the agent to operate at B . The cost is that revocation of C_A at A does not instantaneously revoke C'_B at B ; Theorem 8.5.1 gives an expectation only under its delivery model, and partitions require fail-closed policy or bounded credential TTLs.

8.4.2 Why this composes cleanly with Anchor

The Anchor Protocol’s Phase-3 delegation already proves that a chain $C_{\text{daemon}} \rightarrow C_A \rightarrow C_B$ of attenuated cards is verifiable under the issuing daemon’s public key, and that an attacker cannot inflate a downstream cap to exceed an upstream one. The transfer ceremony is Phase-3 delegation across a trust boundary, with two changes:

1. The second link in the chain is signed under a *different* root key (sk_B instead of sk_A), so the verifier at B verifies under sk_B , not sk_A .
2. The cross-realm link binds the issuing harbor’s current epoch root $R_A^{(e)}$, so a revocation at A after epoch e invalidates the downstream card as soon as the new $R_A^{(e+1)}$ is observed at the verifier.

The composition is what makes the ceremony cheap to add: we are not introducing a new cryptographic primitive, only a new policy at the boundary that says “the verifier accepts a card whose chain crosses the trust boundary if and only if (a) each cross-realm link is signed under

the receiving harbor's key and (b) the issuing harbor's bound epoch root is the current one in the witness log."

8.4.3 Attenuation across the boundary

Two attenuation predicates are applied in sequence. att_A is set by D_A at issue time and reflects what authority A is willing to project across the boundary. att_B is set by D_B at acceptance time and reflects what authority B is willing to grant on its own resources to a card whose root is A . Both attenuations strictly narrow the underlying capability.

Numbers by hand — Composing two attenuations, on the same resource. Take Alice's card C_A authorizing `db:write` on A 's own test database, with att_A the identity narrowing (it removes nothing). D_B accepts the transfer to that same database — suppose, for this example, A 's test database is itself reachable at B — and applies $\text{att}_B = \text{"only between 9am and 5pm."}$ The composed card is $C'_B = \text{att}_B(\text{att}_A(C_A))$: `db:write`, time-windowed to business hours. Every operation C'_B authorizes — a write, during business hours, on A 's test database — is also one C_A authorizes, since C_A places no restriction C'_B does not already respect: subset composition is a subset, exactly as the Theorem below states it in general. Exercise 8.1 asks what changes in this argument the moment att_B names a database C_A never mentioned at all.

THEOREM 8.4.1 Attenuation Monotonicity Across the Boundary. For any cross-realm card $C'_B = \text{att}_B(\text{att}_A(C_A))$, the set of operations authorized by C'_B is a subset of those authorized by C_A under the federated authorization rule.

Sketch. Each attenuation is a subset operation in the Anchor algebra; subset composition is a subset. Cross-realm policy at B adds the constraint "and the operation must be permitted by B 's local policy," which can only further restrict. See the Anchor accompanying chapter's capability-attenuation section for the intra-harbor argument; the cross-realm case adds no new mathematics, only a new applicable attenuation step. $\square$

Exercise 8.1, p. 519.

8.4.4 Threats foreclosed and threats deferred

The annotation band at the bottom of Figure 8.3 summarizes which threats this ceremony forecloses and which are deferred to other primitives. Specifically:

- **Foreclosed:** identity spoofing of sk_A across the boundary; capability inflation across the transfer; replay at B against a stale R_A .
- **Deferred:** revocation latency (handled in §8.5); settlement disputes (handled in §8.6); admission of unbonded new harbors (handled in §8.7).

We are explicit that the ceremony does not close every threat in one step. Federation is a layered defense; this is one layer.

8.4.5 Mechanization status

We have a draft ProVerif model of the four-message ceremony (Appendix 8.B), extending the Anchor Phase-3 model with a second signing key and an epoch-root binding. The current status of the cross-realm authenticity query is **PARTIAL**: the model checks, but the proof depends on an assumption about witness-log freshness that we are still tightening. We note this limitation in Appendix 8.A and treat the attenuation monotonicity claim (Theorem 8.4.1) as proved under the same caveat.

8.5 Federated Revocation

The second governing protocol is the federated revocation mesh. The Anchor Protocol handles revocation inside a single mesh by gossiping an **Append-Only Event Log of Revocation IDs** using Vector Clocks for anti-entropy reconciliation [60]: pairs of daemons periodically reconcile their event logs, designating the cuckoo filter F purely as a local, ephemeral read-path index. The Federated Harbor extends this to the multi-administrative-domain case.

Two pieces are added on top of the Anchor mechanism: (a) per-epoch roots $R_X^{(e)}$ published to a shared witness log so a third party can audit any harbor's local view, and (b) cross-witness signatures so the auditor pattern generalizes Certificate Transparency [145].

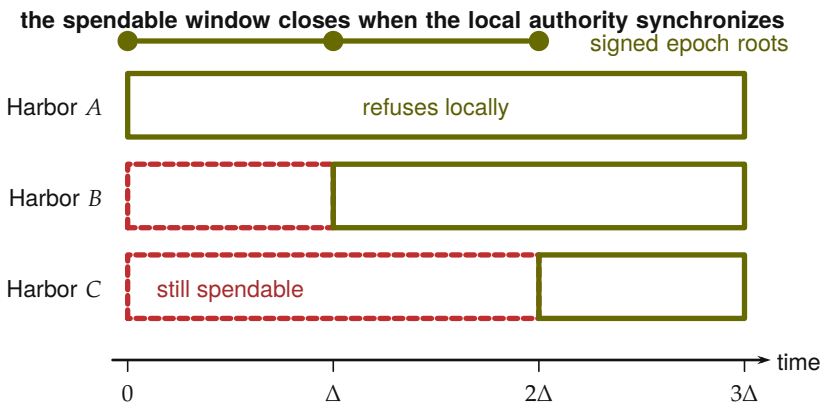
8.5.1 Convergence bound

Standard epidemic dissemination gives an expected $\Theta(\Delta \log m)$ completion time only under an appropriate connected, reliable-round model [60]. The property below instantiates that model at the federation level, treating each harbor's filter as one node in a complete overlay regardless of how many local daemons participate in the harbor's mesh. It is a model-conditional expectation, not an inherited wall-clock deadline.

THEOREM 8.5.1 Conditional Revocation Dissemination. Let m harbors form a complete overlay. In each reliable synchronous round of

RECALL

1. Without looking: what does message (2) of the four-message ceremony bind, and why does that binding matter more than any single signature check?
2. What is the current mechanization status of the cross-realm authenticity query, and what specific assumption is it still conditional on?
3. Name one threat the ceremony forecloses and one it defers, and which later section closes the deferred one.



gold marks a harbor that has synchronized and refuses the card on its own authority; red marks the window in which that card is still spendable at that harbor

Figure 8.4: Revocation propagation is a moving, per-harbor staleness frontier. In this illustrated execution Alice (harbor *A*) refuses immediately; *B* remains stale until Δ and *C* until 2Δ . The red band is exactly where a revoked capability can still be admitted by the named local authority; the gold epoch-root commits mark synchronization, not a global clock. Expected $\Theta(\log m)$ dissemination (Theorem 8.5.1) requires connected reliable rounds; a partition can extend the red window without bound and therefore destroys any finite deadline. [internal]

duration Δ , informed harbors contact independently uniform peers and exchange revocation deltas. If harbor X publishes revocation c at t_0 , then:

1. **Revocation dissemination.** All harbors are informed in expected $\Theta(\log m)$ rounds under the stated epidemic model.
2. **Equivocation verification.** Any auditor possessing two differently signed roots for the same (X, e) detects equivocation in $O(1)$ signature and equality checks.
3. **No hard deadline.** If delivery can be lost indefinitely or the overlay can remain partitioned, no finite worst-case dissemination or common-auditor detection bound exists.

The first item is the standard asymptotic epidemic-dissemination result [60]; this paper does not claim the earlier exact constant $1 + \ln m$. The second follows from the signed-root format. The third is an indistinguishability result: an auditor isolated in one partition cannot distinguish equivocation from delay.

DESIGN INVARIANT 8.5.2 Service-Level Attacker Window. If an operator additionally imposes a partition bound T_p and a delivery service level T_g , then stale-card exposure is bounded by $T_p + T_g$. Without those operational assumptions the protocol supplies an expected latency under its model, not a structural maximum. The bound is a direct consequence of Theorem 8.5.1 once those two operational commitments are added; it is the operator’s own T_p and T_g , not anything the formal model proves on its own, that makes the bound a maximum rather than an expectation — the same conditional shape as Design Invariant 8.6.1.

The finality that “only-issuer-revokes” rests on is model-checked with a negative control. Under the rollback configuration the relay’s revocation set is restored from a backup taken before the revocation, and TLC finds the four-step breach; the epoch configuration closes it (§8.2.3).

the rollback configuration fails by design.

AT THE TERMINAL

```
$ java -cp tla2tools.jar tlc2.TLC -config CardRevocation_rollback.cfg CardRevocation.tla
Error: Invariant RevocationFinal is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ revoked = {}
/\ cardEpoch = (c1 :> 0 @@ c2 :> 0)
/\ epoch = 0
/\ badAccept = FALSE
```

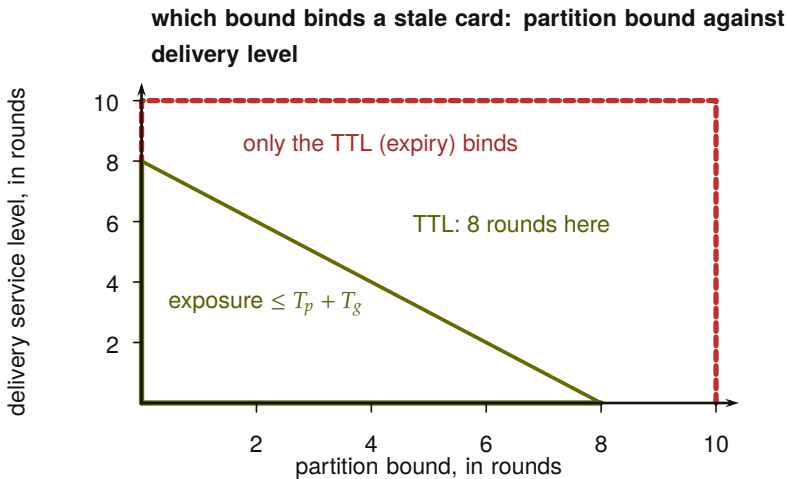


Figure 8.5: Which bound binds a stale card after revocation. Under a partition bound and a delivery service level, exposure is at most the two added together (Corollary 8.5.2, below the gold TTL line); once that sum exceeds the card's time to live the expiry binds instead and the harbor fails closed (above the line, red dashed border). Without a partition bound the horizontal axis is unbounded and only the expiry remains (Theorem 8.5.1, item 3); the line is drawn for an illustrative time to live of eight rounds. [internal]

```

/\ everRevoked = {}
/\ issued = {}
/\ rbUsed = 0

State 2: <Issue(c2)>
/\ revoked = {}
/\ cardEpoch = (c1 :> 0 @@ c2 :> 0)
/\ epoch = 0
/\ badAccept = FALSE
/\ everRevoked = {}
/\ issued = {c2}
/\ rbUsed = 0

State 3: <Revoke(c2)>
/\ revoked = {c2}
/\ cardEpoch = (c1 :> 0 @@ c2 :> 0)
/\ epoch = 1
/\ badAccept = FALSE
/\ everRevoked = {c2}
/\ issued = {c2}
/\ rbUsed = 0

State 4: <RollbackRestore(c2)>
/\ revoked = {}

```

```

/\ cardEpoch = (c1 :> 0 @@ c2 :> 0)
/\ epoch = 1
/\ badAccept = FALSE
/\ everRevoked = {c2}
/\ issued = {c2}
/\ rbUsed = 1

```

State 5: <Accept(c2)>

```

/\ revoked = {}
/\ cardEpoch = (c1 :> 0 @@ c2 :> 0)
/\ epoch = 1
/\ badAccept = TRUE
/\ everRevoked = {c2}
/\ issued = {c2}
/\ rbUsed = 1

```

35 states generated, 23 distinct states found, 9 states left on queue.
The depth of the complete state graph search is 5.
Finished in 00s

The distinction determines the exposure bound. A bond can be priced against an operator-declared service-level window, but an unbounded partition creates an unbounded exposure tail. Deployments must therefore combine bond sizing with expiry, fail-closed acceptance, or an explicit partition limit.

8.5.2 Conflict resolution across harbors

The interesting case is disagreement. Harbor *A* has revoked card *c*; harbor *B* has not yet seen the revocation; an agent presents *c* at *B* for an action that affects *A*'s database. Three candidate resolution rules from the proposal [190], evaluated:

1. **Pessimistic (anyone-revokes-is-revoked).** The card is revoked iff any harbor has revoked it. Defended against by an attacker who would gain by injecting spurious revocations into the gossip mesh; rejects too many honest cards in equilibrium.
2. **Authoritative (only-issuer-revokes).** Only *A* can revoke *A*'s cards. This is the rule we adopt. Under the service-level assumptions of Design Invariant 8.5.2, it gives an operator-priced attacker window; without them, exposure has no finite structural maximum.
3. **Epoch-bounded (revocation propagates within Δ epochs and inconsistency is a contract violation).** A refinement of (2) for the case where one harbor falls persistently behind. We treat this as a degraded mode of (2) rather than a separate rule.

We adopt rule (2). The pessimistic rule has the wrong defaults; the epoch-bounded rule is operationally the same as (2) with a watchdog.

Adopting (2) means the Conservation invariant of §8.6 can be stated cleanly.

Table 8.3 lays out all three rules against what each defends against.

Table 8.3: Three candidate revocation-conflict rules, evaluated against the same disagreement (harbor *A* has revoked card *c*; harbor *B* has not yet seen it). The adopted rule is the only one with a bounded attacker window under stated assumptions. [internal]

rule	what it defends against	why rejected / adopted
pessimistic (anyone revokes)	an issuer that refuses to revoke	rejects too many honest cards; an attacker injects spurious revocations into gossip
authoritative (only issuer revokes)	spurious revocations from a non-issuer	adopted: an operator-priced attacker window under stated service-level terms
epoch-bounded (Δ -epoch deadline)	a harbor persistently falling behind	a degraded mode of authoritative with a watchdog, not a separate rule

Exercise 8.2, p. 519.

8.5.3 Event Log & Ephemeral Cuckoo Discipline

Two Bloom filters over the same universe merge by a bitwise OR; two cuckoo filters cannot, because each stores a fingerprint in one of two hashed buckets that can overflow, so a union may have nowhere to place a fingerprint. The cuckoo filter is therefore purely a local, ephemeral read-path index derived from the authoritative Append-Only Event Log. The filter retains local Anchor-side disciplines (load capping, pollution resistance) but cross-harbor validation is strict: a filter at *B* that holds $c \in F_B$ where *c* was revoked at *A* is correct only if there is a corresponding inclusion proof in $R_A^{(e+1)}$ in the witness log. False positives are corrected by rebuilding the local filter from the event log; spurious revocations injected by an attacker into a peer’s log are rejected on first attempt to verify against the witness log. The discipline is: *local filters are advisory read-caches; the event and witness logs are authoritative.*

8.5.4 Sheaf Laplacian and Abramsky–Brandenburger Contextuality

Three harbors in a ring each record the offset between their own log and the neighbor they last compared with. Walk the ring and add the offsets: if the sum returns to zero, one consistent history explains every

reading; if it does not, some offset was misreported, and the size of the residual says by how much. That is the whole picture; the formalism below states it for arbitrary topologies.

A reader who has met this shape before has met it in rankings, and the analogy is close enough to be worth spending a paragraph on because it makes the residual concrete. Jiang, Lim, Yao and Ye [123] study inconsistent pairwise comparisons — A beats B , B beats C , C beats A — and decompose the comparison data on a graph into two orthogonal pieces: a *gradient* part, which is exactly the portion explained by some global score assigned to each item, and a *curl* part, which is the circulation around cycles that no global score can produce. The first part is the consistent story; the second is the amount by which the data refuses to have one. Equivocation is the same object wearing different clothes. A harbor that signs two mutually incompatible prefixes hands its neighbors offsets that are locally sensible in every pairwise comparison and yet cannot be reconciled into any single history — and what the completion residual r below measures is precisely the part of the observed disagreement that is not a gradient, the curl over the visible witness graph. The vocabulary differs (a cochain that is not a coboundary; a harmonic residual) and the content does not. Two consequences fall straight out of the analogy and are proved separately below: a graph with no cycles has no curl to measure, which is why severed edges and tree topologies see nothing, and a single global relabelling of everyone’s clock is pure gradient, which is why the detector is blind to it and should be.

The global consistency of federated witness logs can be modeled sheaf-theoretically, and it is worth being precise about which object carries the weight. The intuitive picture is a poset $(X, \leq)$ of administrative domains ordered by scoped visibility, assigning to each domain U the semilattice of prefix-compatible append-only logs, with restriction given by prefix projection. That picture is right about the ordering and useless for computation: a semilattice has no subtraction, no adjoint, and no spectrum, so none of the machinery below would be defined over it. The working object is therefore the *cellular sheaf* $\mathcal{F}$ of the cohomology of equivocation,⁹ carried on the gossip graph $G = (V, E)$ of witnesses: witness v holds a real stalk $\mathbb{R}^{d_v}$ encoding its log state; edge $e = \{u, v\}$ holds $\mathbb{R}^{|S_e|}$, the shared prefix coordinates S_e its two endpoints both report; and the restriction maps $\rho_{v,e}$ are the coordinate selections that cut a witness’s state down to what that edge can see. Over real stalks the coboundary δ , its adjoint δ^* , and the observed disagreement cochain $(\delta s)_e = \rho_{u,e} s_u - \rho_{v,e} s_v$ all exist. The semilattice stays as intuition; the vector-space sheaf is what the theorem below quantifies over.

Detection is stated per visibility tier, because not every edge is equally visible to an auditor. An edge is **compared** when the auditor holds its direct cross-check (pairwise detection, no cohomology

⁹A standalone, submission-form version of the results folded into this subsection is *The Cohomology of Equivocation* (research paper 7) [189]; the harness and the sweeps cited here are its artifacts, regenerated at seed 20260816.

needed); **relayed** when both endpoints' reports about it reach the auditor by gossip but the endpoints never reconciled it; and **severed** when no report about it crosses to the auditor at all. Write K for the compared-plus-relayed (i.e. *visible*) edge set, and for each shared coordinate c write $K_c := (V, \{e \in K : c \in S_e\})$ for the visible coordinate subgraph. The distinction between K_c and the full coordinate subgraph G_c changes the theorem, not merely the notation: a cycle that runs through a severed edge imposes no constraint, because the severed block is a free variable of the completion.

THEOREM 8.5.3 *Detection iff the missing edge closes a **visible** cycle.*

Assume prefix restrictions, the three-tier visibility model above, and a **single equivocator** q (the single-equivocator hypothesis is not decoration: two liars on a common cycle can cancel each other's contribution to the residual). The detection statistic is the least-squares *completion residual*

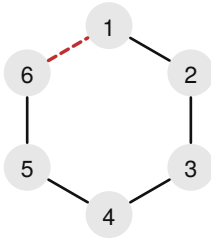
$$r = \min_{x, g_{\text{sev}}} \|g_K - (\delta x)|_K\|_2 = \left(\sum_c \|\text{Proj}_{\text{cycle}(K_c)} g^c\|^2 \right)^{1/2}, \quad (8.1)$$

the minimum over global sections x and over the free severed blocks g_{sev} . Then r detects q 's split-view lie beyond what pairwise comparison already catches *if and only if* the un-compared lie edge lies on a cycle of the **visible** subgraph K_c and its endpoints' reports are relayed to the auditor; in that case $r > 0$ certifies that no global witness history explains the visible data. On a cut edge, $r = 0$ identically — the coboundary map of a tree or forest is surjective onto the visible data, so the free completion absorbs any lie. Across a severed edge, equivocation is provably dark: the severed block is unconstrained and a consistent counterfactual world always exists.

The structural identity behind the mechanism — local consistency with no global section — is the Abramsky–Brandenburger contextuality equivalence [2]: an equivocation (a harbor signing two mutually incompatible prefixes) manufactures for its neighbors exactly a locally-consistent, globally-unglueable family of views, the relational-semantics analogue of Bell contextuality. Note what the theorem does *not* say. The textbook fact that $H^1(\mathcal{U}; \mathcal{F}) = 0$ is equivalent to every local family gluing is a statement about the cover and the restriction maps alone; it is blind to any particular lie, and it is not the detector. The detector is the residual r of the observed data.

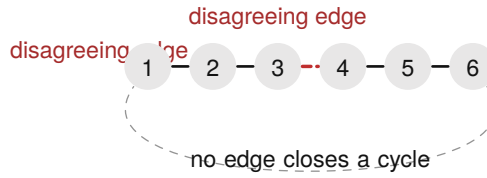
Scope (what the theorem does and does not license). Three bindings keep this claim inside its proven regime. **(i) The obstruction lives in the data, not the sheaf:** the detector is the observed disagreement cochain's failure to be a coboundary (its harmonic residual), not $\dim H^1$ of the abstract sheaf, which is a property of the restriction maps alone and is

the 6-cycle: disagreement
closes a cycle



$r = 1.2247$: visible

the 6-path: same
disagreement, no cycle



$r = 0$: invisible

Figure 8.6: Cycle versus cut on six harbors, numbered 1 to 6. The same pairwise disagreement on one overlap is placed on the cycle C_6 and on the path P_6 . On the cycle the disagreeing edge closes a visible cycle and the consistency radius is $r = 3(1 - 5/6)^{1/2} = 1.2247$; on the path no edge closes a cycle and $r = 0$, so detection is exactly the visible-cycle condition of Theorem 8.5.3. [verified; sheaf_consistency_radius.py, seed 20260816]

blind to any particular lie. **(ii) Visible cycles are required:** cohomology adds power over $O(|E|)$ pairwise digest comparison exactly when an uncomparred (partitioned) edge lies on a cycle of the visible subgraph K_c , so the sum-around-the-loop constraint substitutes for the missing comparison; across a *cut* edge there is no loop to close, and a loop through a *severed* edge is no loop at all for this purpose, so the method provably sees nothing — redundant gossip paths are a detection requirement, not a luxury. **(iii) Non-vanishing is an alarm; vanishing is not an all-clear:** over real coefficients the obstruction is sufficient but not necessary (the abelianization gap of Carù), so a zero residual licenses no safety conclusion. Detection and localization here *complement* forensic attribution — signatures attribute, cohomology localizes under partial visibility — and neither replaces the other.

The harness, pre-registered. Before it ran, the harness committed to its gates: the completion residual must detect split-view equivocation on relayed cycles where pairwise comparison is blind, and it must stay silent on honest worlds, cut edges and severed arms. Two hundred trials per arm at seed 20260816, stalks $\mathbb{R}^5$, shared-prefix sizes 2 to 4; the score is cohomology-only detections, trials where the residual fires and every pairwise comparison passes. Partition on a cycle (two_path): 200/200 cohomology-only. Severed-cycle arm: 0/200, the boundary measured. Cut edge (single_bridge): maximum residual 1.5×10^{-13} over 400 trials, silence at float epsilon. Full visibility: 0 cohomology-only and 200/200 redundant, since pairwise comparison catches everything first. An expander with three severed and three relayed edges: 113/200 cohomology-only and 87 dark, and the dark trials split exactly

along the theorem’s two clauses: in 79 the equivocator has no relayed incident link at all, so every lie lands on a severed edge; in the remaining 8 the lie edge is relayed and lies on a cycle of G_c but not of K_c , because the rest of that cycle is severed. Those eight are the only trials that distinguish the two readings of the cycle clause, and they are dark, as the K_c form predicts and the G_c form does not. Both pre-registered gates pass; verdict COMMIT. The rival CUT condition (residual detections are pairwise detections in disguise) is refuted: 313 of 1,101 residual detections across arms were pairwise-blind. Mutants reintroducing the two defects of the first harness (non-subset restriction maps that collapse the obstruction space; detection scored on hidden-edge data) are each caught by structural self-checks [internal, sheaf_harness_v2.py]. The harness validates the detector at the sharpened scope only: synthetic gossip, prefix restrictions, one equivocator per trial. A statistical pass is not a theorem, which is what the three claims below are for.

Table 8.4 lays out all six arms against the two pre-registered gates.

Table 8.4: Six pre-registered arms, 200 trials each (400 for the cut-edge arm, both offset directions), seed 20260816. Cohomology-only: the residual fires and every pairwise comparison already passes; dark: neither. The gate column marks the two pre-registered claims: two_path must detect (it does, 200/200) and single_bridge must stay silent (it does, to float epsilon); the other three arms are supporting evidence, not named gates. The rival CUT refutation holds: 313 of 1,101 residual detections across all arms were pairwise-blind. [verified, sheaf_harness_v2.py]

arm	trials	coh.-only	dark	gate
two_path (relayed cycle)	200	200	0	PASS
severed-cycle	200	0	200	shown
single_bridge (cut)	400	0*	400	PASS
full-visibility	200	0	0	redundant
expander (3 sev., 3 relayed)	200	113	87	split

*max residual 1.5×10^{-13} , float epsilon.

Numbers by hand — The cochain by hand: one ring, one chain, one dark link. Four relays 0, 1, 2, 3 in a ring, scalar stalks, one shared coordinate. The links 0–1, 1–2 and 2–3 are compared and agree; the link 3–0 is never compared, but both its endpoints’ reports reach the auditor, so all four links are visible and K_c is the whole ring. Relay 0 equivocates: the head it shows across 3–0 differs by $s = 3$ from the one it shows relay 1. The auditor’s cochain, one disagreement per link, is $g = (g_{01}, g_{12}, g_{23}, g_{30}) = (0, 0, 0, 3)$. The only question the detector asks is whether one history $x = (x_0, x_1, x_2, x_3)$ has differences reproducing g . Around a loop a coboundary sums to zero and g sums to 3, so

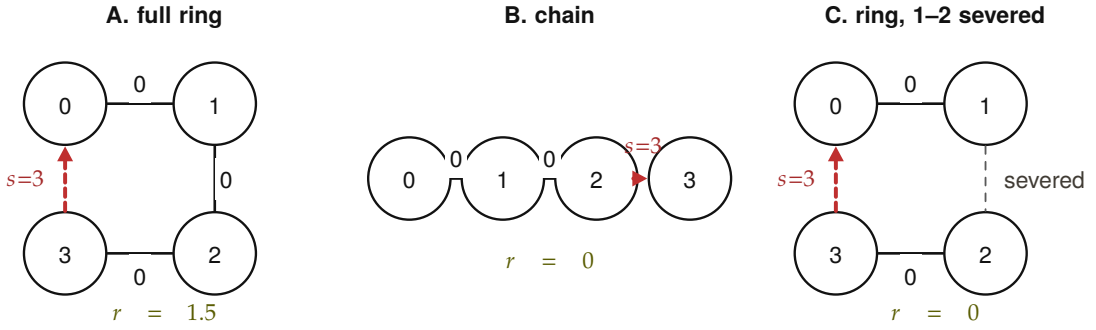


Figure 8.7: Same lie, three topologies, two different verdicts. A relay 0 equivocates by $s = 3$ on one link in every panel. On the full ring (A) the loop cannot absorb the lie without leaving a residual on every link, giving $r = 1.5$. Breaking the loop makes the lie invisible two different ways: as a chain (B), the assignment $x = (0, 0, 0, -3)$ reproduces the visible disagreement exactly, so $r = 0$; as a ring with a *different* link (1–2, not the lied-about 3–0) severed (C), the three still-visible links admit the same free completion, so $r = 0$ again. The severed link in panel C is not the lying link – visibility of the *loop through the equivocator*, not of the lie itself, is what the theorem’s clause turns on. [verified, closed form; matches `sheaf_consistency_radius.py`]

none does. Least squares finds the closest: $x = (0, 0.75, 1.5, 2.25)$, with coboundary $\delta x = (-0.75, -0.75, -0.75, +2.25)$, which misses by exactly 0.75 on every link. That is the mechanism in one sentence: the best honest world spreads the lie evenly around the loop and still misses everywhere. Hence

$$r = \sqrt{4 \times 0.75^2} = 1.5 = \frac{|s|}{\sqrt{n}} = \frac{3}{\sqrt{4}} = |s| \sqrt{1 - R_{\text{eff}}^{K_c}(e)} = 3\sqrt{1 - \frac{3}{4}},$$

three routes to one number [verified]. Now break the loop twice and watch the same arithmetic go silent. *As a chain* 0–1–2–3 with the same lie on the never-compared end link 2–3: the assignment $x = (0, 0, 0, -3)$ reproduces $g = (0, 0, 3)$ exactly, so $r = 0$, and the reader has just built the adversary’s alibi by hand. *As a ring with a dark link*: keep the ring and the lie on 3–0 but sever 1–2. Now $g_K = (0, 0, 3)$ on three visible links and $x = (0, 0, 3, 3)$ reproduces it exactly: $r = 0$ again. The loop is still there in G_c ; it is gone from K_c , and the residual can only see K_c . This is the theorem’s visibility clause, checkable in four numbers. *Now you try*: C_8 , lie of size 2 on the relayed edge: $2/\sqrt{8} = 0.7071$; the same lie on any bridge, 0; the same lie on C_8 with any one of the other seven links severed, also 0 [verified].

Figure 8.7 draws all three variants side by side.

Numbers by hand — the consistency radius, six-cycle vs. six-path, and the $1/\sqrt{n}$ scaling. Six harbors in a ring (C_6), unit-weight edges, a lie of magnitude $|s| = 3$ on one relayed cycle edge. The effective resistance across that edge is $R_{\text{eff}} = (n-1)/n = 5/6$, so by hand:

$$\begin{aligned} r &= |s| \sqrt{1 - R_{\text{eff}}} = 3 \sqrt{1 - \frac{5}{6}} \\ &= \frac{3}{\sqrt{6}} = 1.2247 \quad [\text{verified}]. \end{aligned}$$

Cut the same ring into a six-node path (P_6), same edge, same lie: every edge of a path is a bridge, so $R_{\text{eff}} = 1$ and $r = |s| \sqrt{1-1} = 0$ exactly [verified] — a tree’s coboundary map is already onto the visible data, so the free completion absorbs the whole lie. `sheaf_consistency_radius.py`’s closed-form section ([0] CLOSED FORM vs the R6 mechanism numbers) prints both against its own numerical solve:

```
1 C6 relayed cycle edge, lie 3.0:  r = 1.224745    |s|*sqrt(1-R_eff) =
    ↪ 3*sqrt(1-0.8333) = 1.224745
2 P6 relayed cut edge,   lie 3.0:  r = ...        R_eff = 1.000000 ->
    ↪ predicted 0
```

matching the hand value to six decimals on C_6 , and on P_6 holding the printed residual under 10^{-9} (the script’s own `TOL_SILENT` zero-residual threshold) rather than printing a literal 0 [internal, `sheaf_consistency_radius.py`]. The general law is $r = |s|/\sqrt{n}$ on any C_n : $|s|/2$ at $n=4$, $|s|/\sqrt{6} \approx 0.4082|s|$ at $n=6$, $|s|/\sqrt{8} \approx 0.3536|s|$ at $n=8$ [verified, closed form] — conviction dilutes monotonically as the ring lengthens, and any path reads as silence regardless of n . *Now you try:* reproduce both printed lines and the general law against the script (Exercise 8.3), then reason about why the theorem needs a *single* equivocator (Exercise 8.4).

Exercises 8.3–8.4,
p. 519.

The residual as a certificate. Robinson’s consistency radius asks how far local data sits from the nearest global section [216]; the residual r is its least-squares form under the completion over severed blocks. The new content is that r is not merely a distance. It is a certificate about the adversary: every offset pattern that could have produced the visible data is at least this large, and one of exactly this size exists. Topology enters through electrical network theory. A lie on edge e hides in proportion to how well the rest of the visible graph can explain it away, which is the effective resistance $R_{\text{eff}}^{K_c}(e)$ measured in the visible coordinate subgraph: on a bridge $R_{\text{eff}} = 1$ and the lie hides completely; on a long cycle $R_{\text{eff}} \rightarrow 1$ and the certified fraction $\sqrt{1 - R_{\text{eff}}}$ decays like $1/\sqrt{n}$. Severing a link raises the effective resistance of everything that was routed around through it, which is the visibility clause written as

a price.

THEOREM 8.5.4 Soundness and achievability of the completion residual. For the completion-residual detector with prefix restrictions and three-tier visibility: $r > 0$ iff no global section explains the compared-plus-relayed data, and honest executions give $r = 0$ exactly, for every visibility pattern. Every offset pattern ε consistent with the visible data satisfies $\|\varepsilon_K\| \geq r$, with equality at the completion: r is the exact minimum lie. For a single equivocator q with offset vector o and visible-lie operator A_q , $\sigma_{\min}^+(\Pi_K A_q) \|o_{\perp}\| \leq r \leq \|o_{\perp}\|$, where $o_{\perp}$ is the component of o outside $\ker A_q$. For a single-edge lie of size s in coordinate c ,

$$r = |s| \sqrt{1 - R_{\text{eff}}^{K_c}(e)}, \quad \text{on } C_n : |s|/\sqrt{n},$$

with the effective resistance taken in the visible subgraph K_c , not in G_c .

Proof sketch. r is the distance from the visible cochain g_K to the image of the coboundary restricted to visible edges, the severed blocks being free. An offset pattern is consistent with the data iff $g_K - \varepsilon_K$ lies in that image, so $\|\varepsilon_K\| \geq r$ by definition of the distance, and the completion residual is itself such a pattern, which gives equality. Honest data is a coboundary, so $r = 0$. For one lie of size s on e , g_K is s times the edge indicator, and the squared norm of its projection off the coboundary image is s^2 times one minus the edge's effective resistance, the standard identity between the cycle-space projection of an edge and $1 - R_{\text{eff}}(e)$; on C_n every edge has $R_{\text{eff}} = (n - 1)/n$. $\square$

THEOREM 8.5.5 Localization. Noise-free, single equivocator: the support of the edge residual lies on cycles of the visible coordinate subgraphs K_c through q . With bounded noise η , localization is guaranteed while $2\|\eta_K\| < \max_f \|(\Pi_K \varepsilon_K)_f\|$.

Proof sketch. The residual is the projection of the lie onto the cycle space of K_c , and the projection of a single edge's flow is the electrical current it induces, which is supported on the cycles through that edge. The noise condition says the largest residual entry exceeds twice the noise the projection can add, so the ordering of edges by residual is preserved. $\square$

THEOREM 8.5.6 Cost. Because the visible coboundary splits per shared coordinate into graph incidence operators, r is one sparse least-squares

problem that decomposes into at most L scalar graph-Laplacian systems, solvable in $\tilde{O}(|E| \cdot L \log(1/\epsilon))$ with symmetric diagonally dominant solvers [243]. Bach's #P-hardness of sheaf cohomology [20] concerns coherent sheaves on projective space, the wrong category; finite cellular sheaves over $\mathbb{R}$ on graphs are linear algebra.

Checked. Soundness 0/600 violations; achievability 600/600 (a consistent pattern of norm exactly r constructed every time). The effective-resistance closed form is exact on C_4 to C_{12} and on 200 random graphs; the lower bound is tight under the adversarial offset ($r/\|o_\perp\| = \sigma_{\min}^+ = 0.4082 = 1/\sqrt{6}$); a uniform kernel lie of norm 5 gives $r = 2 \times 10^{-14}$, a consistent alternative state rather than a split view, invisible in principle and correctly not flagged, and on a cycle of K_c the uniform offsets are the only such directions, every kernel direction lying in the uniform span to 5×10^{-16} . Localization 200/200 on the two-cycles topology and 128/128 on severed expanders; the noise boundary: at least 95% localization up to a noise-to-lie ratio of 0.1, 76.5% at 0.2. Conjugate gradient's timing slope against $|E|$ stays well under the 1.6 gate while dense solve is consistently super-linear; the decomposition, not the two digits, is what the gate certifies. Two coordinated liars on a common cycle, each alone certified at $r \geq |s|/\sqrt{8}$, jointly cancel to $r = 6 \times 10^{-15}$ [internal, `sheaf_consistency_radius.py`].

What it buys. The harness's statistic is upgraded to a certificate a federation operator can act on: r is the smallest lie consistent with what was seen, the hot edges lie on cycles through the culprit's neighborhood, and the whole computation is a handful of Laplacian solves, cheap enough to run on every gossip round. The closed form also converts topology into policy. Effective resistance is the price list for where a federation should force direct reconciliation (bridges: mandatory, since the residual cannot help there) and where relayed gossip suffices (short cycles: cheap certified detection).

Single equivocator is the scope, and provably so: a coalition whose combined lie is a coboundary is a consistent counterfactual world, and two coordinated liars on a common cycle cancel the residual to numerical zero. r never overstates the lie, but it can understate a coalition's to zero; soundness is unconditional, detection is not. No attribution, ever: a $+o$ lie by q toward w and a $-o$ lie by w toward q produce identical observations, so the residual localizes to cycles through the equivocator and cannot say which endpoint lied; naming culprits is the signatures-and-forensics business [235], which this detector complements. Vanishing r is not an all-clear, for three independent reasons: uniform lies move every neighbor's view iden-

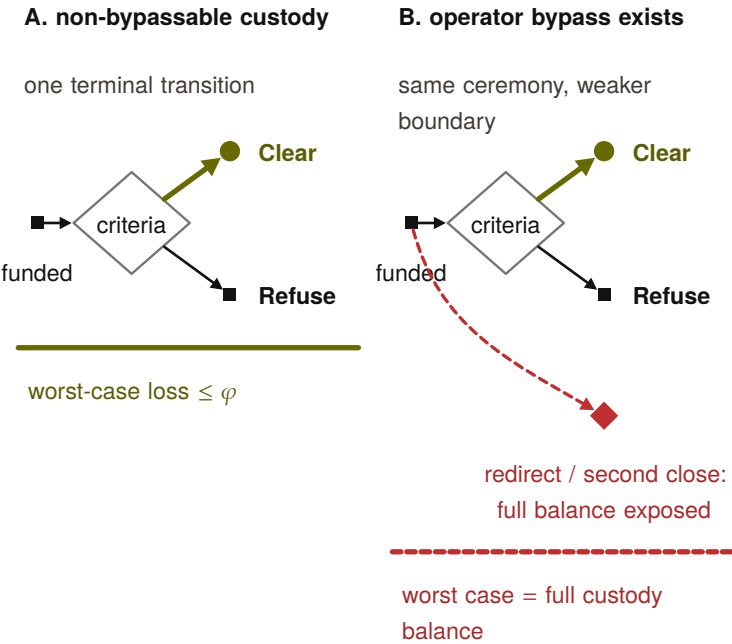
WHERE THIS STOPS

tically and are invisible in principle; severed edges admit consistent counterfactuals by construction; and over $\mathbb{R}$ the obstruction is abelianized, so a vanishing real-coefficient signal does not certify gluability in richer coefficient systems [44]. Where no report crosses, nothing can be seen, and severing one link of a loop darkens the whole loop; under measurement noise the certificate needs a threshold above the honest floor, below which it reverts to “consistent with noise.”

Sheaf Laplacian dynamics — and what is *not* claimed here. On a cellular sheaf with real stalks, the **Hansen–Ghrist sheaf Laplacian** [100] is $\Delta_{\mathcal{F}} = \delta^* \delta$, and its harmonic space $\ker(\Delta_{\mathcal{F}}) \cong H^0(G; \mathcal{F})$ is precisely the subspace of globally synchronized ledger states. This object exists only over the vector-space sheaf constructed above; it is undefined over the semilattice of append-only logs, which has no adjoint and no spectrum. The spectral gap $\lambda_2(\Delta_{\mathcal{F}})$ — the smallest non-zero eigenvalue — is the natural quantity to *relate* to anti-entropy convergence speed, a small gap meaning slow mixing by analogy with the graph-Laplacian case. **We have not derived or checked such a bound, and this paper claims none.** The Hansen–Ghrist rate governs continuous-time linear diffusion $\dot{x} = -\Delta_{\mathcal{F}} x$ over real stalks; anti-entropy gossip over signed append-only logs with a Byzantine equivocator is discrete, asynchronous, adversarial, and monotone-join rather than linear-averaging, and nothing here shows the bound transfers. The two convergence stories — classical epidemic dissemination (Theorem 8.5.1, which is a real and separately argued result [60]) and sheaf-Laplacian diffusion — are not reconciled, and neither is a second proof of the other. No relaxation-time bound for the federation is established, in this section or anywhere in this paper; that is consistent with Appendix 8.A, whose dissemination rows record “no partition deadline” and “no hard delivery bound.” The reconciliation is open work (Appendix 8.A, *open* band).

8.6 Cross-Harbor Settlement

The third governing protocol is cross-harbor settlement. A bond is posted at A to cover damage caused by an A -side agent operating at B . When damage occurs, the bond must clear to the damaged party at B . The protocol does this without making either harbor sovereign over the other, but it still uses a third-party custody service. Its transition surface must be non-bypassably restricted; the property below states that assumption rather than relabeling the service as trustless.



the custody assumption: whitelist, fee cap, one terminal transition are the theorem's hypothesis, not decoration

Figure 8.8: The custody bound is the absence of an extra transition. Panel A permits exactly one terminal move from funded custody to **Clear** or **Refuse**: pay the whitelisted beneficiary or return the bond. With a recipient whitelist and fee cap, worst-case loss is bounded by the agreed fee φ . Panel B holds the same ceremony but drops the non-bypassability hypothesis: one operator-controlled edge reaches redirect or a second close, and the worst case becomes the full custody balance. Threshold signing (§8.6.1) is only one candidate way to make panel A's absence of that edge structural; the bound holds only when it actually is. [internal]

8.6.1 The bounded escrow

We introduce a third party—the escrow—and require the *custody ledger* to expose exactly two terminal transitions: clear (pay whitelisted beneficiary B , refund the remainder to A) or refuse (return the bond to A). A signed policy alone is only evidence. The following restrictions are structural only if the escrow service cannot bypass the ledger API or alter its code and keys:

- Redirect funds to anyone other than A or B .
- Equivocate about its clearing decision (publish “cleared” to one party and “refused” to another).
- Delay past the TTL of the bond.
- Extract more than a pre-agreed fee ϕ which is capped at issuance.

These restrictions define the desired transition system. The current artifact is a design and bounded model, not a deployed custody mechanism.

DESIGN INVARIANT 8.6.1 Conditional Escrow Extraction Bound. Assume the custody ledger (i) admits only recipients A and B , (ii) caps the fee output at ϕ , (iii) executes exactly one terminal transition atomically, and (iv) keeps signing and transfer authority outside the escrow operator’s bypassable control. Then no allowed transition pays the escrow more than ϕ .

The bound follows by case analysis over the two allowed terminal transitions. Witness logging makes delay or equivocation detectable, and an escrow bond can compensate some misconduct, but neither fact prevents theft if the operator can bypass custody restrictions. Without assumptions (i)–(iv), the worst-case extractable value is the full custodial balance.

The escrow is a trusted third party with a *conditional transition bound*. This is not a trustless construction in the HTLC sense [108]. Whether useful trustless settlement is possible for non-fungible reputation-priced bonds remains open (§8.12); this paper gives neither a construction nor an impossibility proof.

A candidate instantiation of assumption (iv). One concrete way to remove the escrow operator’s *unilateral* control over transfer authority is a 2-of-3 threshold-signature scheme: harbor A , harbor B , and a federation arbiter each hold one key, and no transfer executes without two of the three signatures. When A and B agree on the outcome — the ordinary case — they co-sign the release themselves and the arbiter never touches custody. The arbiter’s key exists only to break a dispute: it evaluates the witness-logged evidence trail and co-signs with whichever party it rules for. This is a plausible way to satisfy assumption (iv),

and we name it because a reader will otherwise ask what such a mechanism would look like. It does not close what Design Invariant 8.6.1 calls conditional. The arbiter remains a trusted third opinion; we have not analyzed an arbiter colluding with one principal, we have not specified how arbiter selection resists capture, and the underlying settlement rail (an on-chain threshold contract, a custodial payment hold, or anything else) carries its own assumptions that this paper does not model. We list 2-of-3 multisig as a candidate design, not a closed result; trustless settlement for non-fungible bonds stays in the *open* band (§8.12.2, Appendix 8.A).

8.6.2 Cross-harbor conservation

The Bonded Commons paper proved a single-machine Conservation Theorem: the total bonded value in the system never changes by surprise; bonds are only created (by posting), destroyed (by clearing), or rolled (refunded and re-posted). We extend this to the federation.

DESIGN INVARIANT 8.6.2 Cross-Harbor Bucket Partition. For each funded bond b of amount a_b , let $P_b, E_b, C_b, R_b \in \{0, a_b\}$ denote its posted, in-escrow, cleared, and refunded buckets. Valid transitions preserve

$$P_b + E_b + C_b + R_b = a_b \quad \text{and} \quad |\{Z \in \{P_b, E_b, C_b, R_b\} : Z = a_b\}| = 1.$$

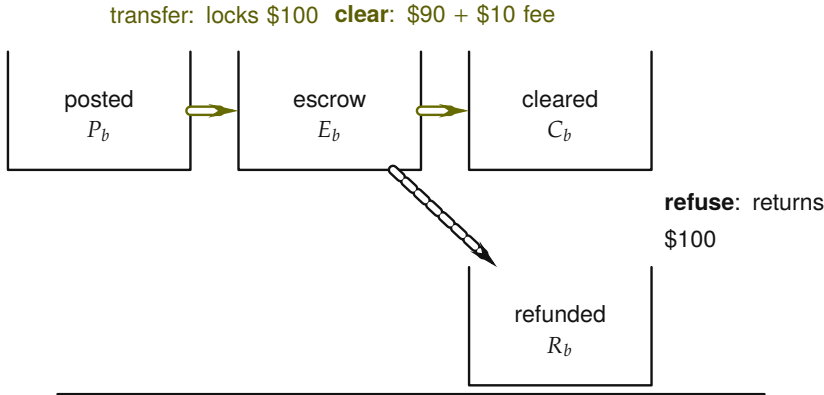
Summing over bonds gives $\sum_b (P_b + E_b + C_b + R_b) = \sum_b a_b$. External top-ups create new funded bonds and are recorded separately; fees must appear as an explicit cleared recipient rather than disappear from the identity.

This is the federation-layer analogue of Conservation in Bonded. The intra-harbor Conservation invariants hold per-harbor (Bonded gives the proofs); the cross-harbor property adds the constraint that bonds in transit between harbors — i.e., posted at A , locked in escrow, awaiting clearance against B — are accounted for as in-escrow rather than as either posted at A or cleared to B . The escrow's role as the holder of in-flight bonds is what makes the conservation accounting tractable.

Figure 8.9 draws the four buckets and the pipes between them.

Numbers by hand — One bond across two harbors. A \$100 bond posted at A has buckets $(P, E, C, R) = (100, 0, 0, 0)$. The transfer locks it in escrow, $(0, 100, 0, 0)$. Clearance at B pays \$90 to the counterparty and \$10 in fees, recorded as two cleared recipients, so the row is $(0, 0, 100, 0)$ with $C = 90 + 10$; a refund instead gives $(0, 0, 0, 100)$. In every row exactly

■



$P_b + E_b + C_b + R_b = 100$, and exactly one term is nonzero at any time

worked instance: $(100, 0, 0, 0) \rightarrow (0, 100, 0, 0) \rightarrow (0, 0, 100, 0)$ with
 $C=90+10$, or the refuse branch $\rightarrow (0, 0, 0, 100)$

Figure 8.9: One bond, one bucket, at any time. A funded bond of amount a_b moves through exactly one path: posted at A , locked in escrow, then either **clear** (paid out, with the fee an explicit second cleared recipient rather than a silent subtraction) or **refuse** (returned to A). Design Invariant 8.6.2's own worked instance is annotated on the pipes: a \$100 bond clears as \$90 to the counterparty plus \$10 in fees, or refunds the whole \$100 on the dashed branch. The four buckets always sum to a_b , and only one is ever nonzero – a fee that vanished from the row would be the first sign of a broken partition. The Bucket partition is model-checked by Apalache at $|F| = 4$ harbors and bond depth 16 (§8.9.2). [internal]

one bucket holds the whole amount and the four sum to 100 [internal]; a fee that vanished from the row would be the first sign of a broken partition.

8.6.3 Settlement protocol

Figure 8.10 shows the three-step protocol. To unpack the figure:

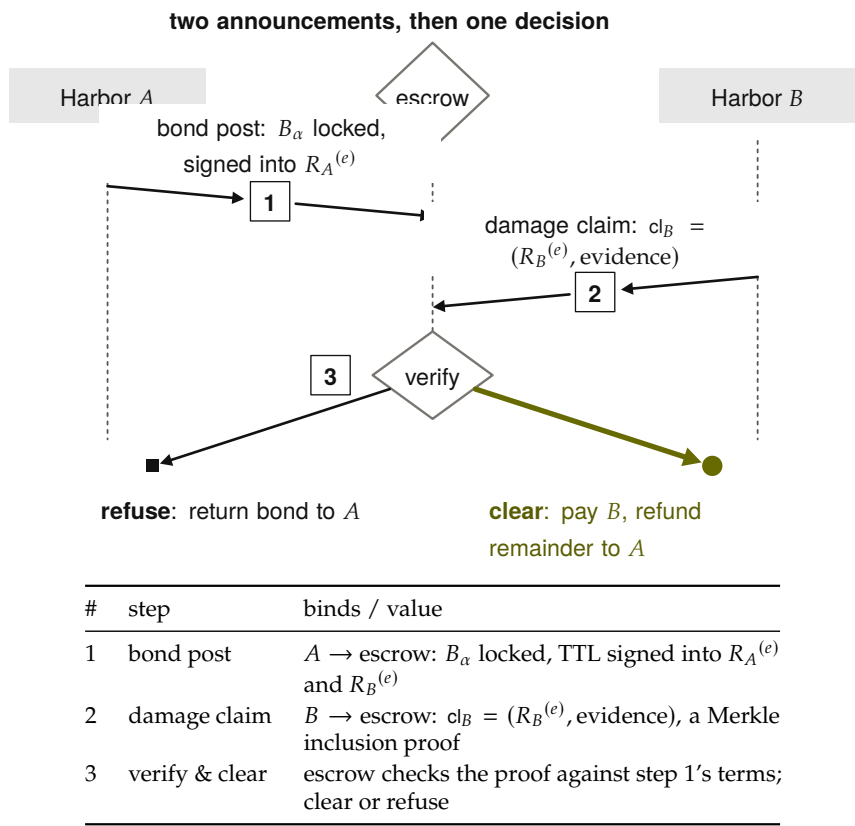


Figure 8.10: Two announcements, then one decision. Step 1 posts the bond at the escrow (message 1); step 2 is Harbor B signed damage claim, sent independently whenever damage surfaces (message 2); step 3 is the only step where the escrow decides anything, checking the claim inclusion proof against the preconditions signed at step 1 and producing exactly one of two terminal outcomes, **clear** or **refuse** – matching the two terminal transitions Design Invariant 8.6.1 bounds. Neither announcement alone settles anything; only step 3 does. [internal]

- 1. **Bond post.** Alice’s harbor mints a work card for agent α ; agent operates at B. Bond $B_\alpha = \$B$ is posted at A and locked in the escrow. The bond’s amount and TTL are signed into both A’s and B’s witness-log roots so the federation knows the bond exists.

2. **Damage claim.** Damage detected at B (acceptance criteria fail, or invariant violation). Bob's harbor signs claim $\text{cl}_B = (R_B^{(e)}, \text{evidence})$ where the evidence is a Merkle inclusion proof against $R_B^{(e)}$. The claim is sent to the escrow.
3. **Verification and clearance.** Escrow verifies inclusion of α 's actions in $R_B^{(e)}$, matches against the acceptance preconditions that were signed at step 1, and produces one of two outcomes: *clear* (pay B out of bond, refund remainder to A) or *refuse* (return bond to A , log reason). Both outcomes are recorded in the witness log.

The designed transition is atomic in the bucket-partition sense: a bond moves once from in-escrow to either cleared or refunded. This property depends on the custody-ledger assumptions of Design Invariant 8.6.1; witness records alone cannot make two external transfers atomic.

8.6.4 Fee structure and economic discipline

The escrow's fee ϕ is bounded but non-zero. In the competitive-insurance market sense of *The Bonded Commons* (the Youle contribution), ϕ is a market clearing price for the service of holding the bond and verifying the inclusion proof. We do not solve the market here; the Youle contribution from Bonded gives the mechanism, and we conjecture (open) that it extends cleanly to the cross-harbor case when the escrow population is large enough.

We are explicit that for small federations (two or three harbors), ϕ is likely set by a posted price rather than auctioned. The mechanism design here is identical in shape to Bonded's, just lifted to the federation layer.

8.7 Federation Admission

The Federated Harbor is not a permissionless market in identities. A new harbor enters the federation through an admission ceremony. Agents inside an admitted harbor are named by certificates the harbor issues, each binding a principal identifier, an agent key, the issuer, an epoch, a policy, and an expiry; the federation never sees a bare key, and a certificate outlives neither its epoch nor its policy. The ceremony is invitation-bounded by design: Sybil attacks against the federation [66] are costly for an attacker precisely because each identity requires a real admission event.

RECALL

1. Without looking: name the two terminal transitions the custody ledger must expose, and the four assumptions Design Invariant 8.6.1 conditions its bound on.
2. In the three-step settlement protocol, which step is the one a signed inclusion proof alone cannot settle by itself, and why?
3. What does the 2-of-3 threshold-signature candidate remove from the escrow operator's control, and what does it *not* close?

8.7.1 The bonded-sponsor pattern

A new harbor N with no prior reputation cannot price its own bond fairly under competitive insurance — the market lacks data on N 's risk. The bonded-sponsor pattern resolves this:

Numbers by hand — Beta's six months, traced through the ceremony. §8.8 already fixes this ceremony's one concrete run. Beta Vision had no reputation in the consortium, so Acme — an existing member — posted the sponsor bond on Beta's behalf. Beta then operated under probation for six months: its cards were accepted and its evidence trail was witness-logged, but it was Acme's bond, not Beta's own standing, that would have been forfeit had Beta forged a signature, equivocated on a root, or refused a settled bond during that window. Acme's risk officer spent the same six months watching Beta's evidence trail closely, per §8.8.1, before the joint work traced in §8.8 takes place. At the end of the window, six months of witness-logged behavior with zero forfeitures was enough to price Beta's own bond, and Acme's sponsor bond was released. That is all four items of the ceremony below, in order, with the one number this federation's own history happens to fix.

1. An existing harbor S (the sponsor) posts a bond on N 's behalf, sized to cover the federation's worst-case loss from N misbehaving during a probationary epoch.
2. N joins the federation under probation; its cards are accepted, its evidence trail is witness-logged, but its sponsor bond is at risk.
3. If N misbehaves during probation — forges signatures, equivocates on roots, refuses to honor settled bonds — the sponsor bond is forfeit. The sponsor has full incentive to vet N before sponsoring.
4. At the end of probation, N 's own reputation is sufficient to price its own bond, and the sponsor bond is released.

This is the federation-layer analogue of competitive insurance from *The Bonded Commons* (the Youle contribution): instead of an authority picking who joins, the sponsor market discovers it. The cost of being a sponsor is real — forfeit risk during probation — and the benefit is the network effect of having N in the federation. We expect (open) that for federations with many candidate sponsors and clear-eyed risk pricing, the equilibrium is reasonably efficient. We do not prove it.

Figure 8.11 draws the four-state lifecycle end to end.

8.7.2 Cold-start failure modes

A new harbor that cannot find a sponsor cannot join. This is a real limitation. In the worst case, the federation collapses to invitation-only — which is the failure mode of every interesting federated identity system

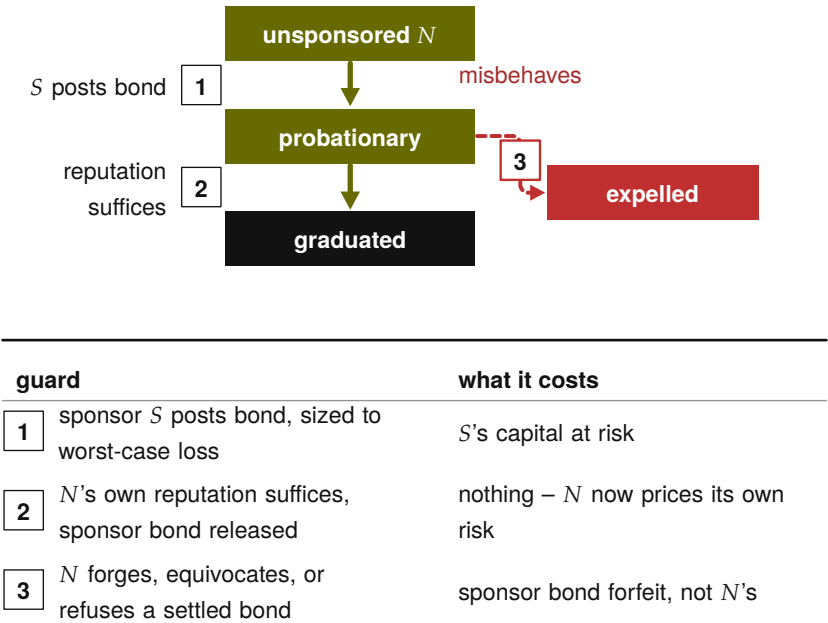


Figure 8.11: Two terminal states, one asymmetric cost. A newcomer *N* with no prior reputation enters probationary under sponsor *S*'s bond (guard 1); probation ends either at graduated, once *N*'s own reputation suffices to price its own bond (guard 2), or at expelled, in red, if *N* misbehaves during probation (guard 3). Only the expelled branch costs anyone besides *N*: it is the sponsor's bond, not *N*'s, that is forfeit, which is exactly the incentive the pattern relies on to make sponsors vet before they sponsor (§8.7.1). [internal]

from PGP web-of-trust [279] to SAML [165]. The structural pushback is:

- *Many sponsors.* The market is many-to-many; a new harbor needs only one sponsor. As long as sponsors are paid (via fees or reputation), the supply is nonzero.
- *Small probationary scope.* New harbors during probation are restricted from large-stake operations; the sponsor's risk is bounded.
- *Witness-log permanence.* The admission record is permanent; once a harbor has graduated probation, it does not need a sponsor again.

This is engineering discipline, not a theorem. The historical evidence on whether invitation-bounded federations stay open is mixed [165]; the Federated Harbor's pushback against centralization is to make the cost of being a centralizing hub bounded above by the cost of running mirrored witness logs, which is small. Whether that pushback is enough is empirical.

8.8 Worked Example

The four primitives are easier to read against a concrete case. We work the example end-to-end: two organizations, three machines, one shared project. The example is intentionally familiar; nothing in it is novel except the way the primitives compose.

Figure 8.12 shows the whole incident end to end, three harbor lifelines and six marked steps, before the subsections below walk through it one at a time.

8.8.1 Setup

Two organizations: Acme Robotics (Alice's employer) and Beta Vision (Bob's employer). The shared project is a perception-and-control stack: Acme owns the controller, Beta owns the vision system, both run against a shared staging environment that lives on a third machine maintained by neither.

Three machines: Alice's laptop (harbor *A*, controller code, controller test database), Bob's laptop (harbor *B*, vision code, vision test database), staging-server (harbor *S*, shared staging database, no agents resident).

Each harbor has gone through the admission ceremony of §8.7. Acme's harbor was sponsored by an existing partner harbor at the open-source consortium that hosts the staging server; Beta's was sponsored by Acme after a six-month probationary period during which Acme's risk officer watched Beta's evidence trail closely. The staging server harbor is sponsored by the consortium directly and is treated as a known-honest verifier for the purposes of this example.

8.8.2 The agent task

Acme’s agent — call it Controller-7 — needs to run a schema migration on the staging database to add a column for a new sensor type. The migration is destructive (drops and recreates the column); if it goes wrong, the staging database is unusable for the joint demo scheduled three hours later.

Controller-7’s local card C_A authorizes `db:migrate` against the controller test database, attenuated through Acme’s risk policy ($\text{att}_A =$ “only against tables in the `controller` schema; only between 9am and 5pm; only if a human operator has approved within the last 30 minutes”). The card is valid at A , but the database it needs to migrate is at S .

8.8.3 Step 1: Cross-harbor transfer

Controller-7 invokes `xfer_req( $C_A, S$ )` on D_A . D_A constructs the envelope:

$$\text{xfer_offer}\langle C_A, \text{att}_A, R_A^{(e)} \rangle \text{ signed under } sk_A,$$

and sends it to D_S (the staging-server harbor). D_S verifies the signature against A ’s public key as recorded in the witness log, verifies that $R_A^{(e)}$ is the current epoch root for A , and applies its own attenuation att_S (“only the `controller` schema; only after a backup snapshot has been taken; only if no other migration is in flight”). The composed attenuation produces $C'_S = \text{att}_S(\text{att}_A(C_A))$, a card valid only at S , signed under sk_S .

D_S delivers C'_S to Controller-7. Controller-7 takes the snapshot, then runs the migration against S . The migration’s actions are recorded in S ’s Merkle forest and roll up into $R_S^{(e)}$.

8.8.4 Step 2: Damage detection

The migration runs cleanly. Two hours later — well after the agent has moved on — Beta’s vision system runs an integration test against the staging database and observes that the new column has the wrong type (Acme assumed `float32`; Beta needs `float64` for the sensor data they will produce). The integration test fails.

D_B (Beta’s harbor) verifies the failure against its local invariants and produces a damage claim:

$$\text{cl}_B = (R_B^{(e+1)}, \text{evidence}),$$

where the evidence is a Merkle proof that Beta’s integration tests against the staging database failed because of a schema mismatch. The claim is sent to the escrow.

8.8.5 Step 3: Settlement

Acme posted a bond when Controller-7 was authorized to run the migration. The bond is held in the federation escrow with a TTL of 24 hours — long enough for downstream consequences to surface, short enough not to lock collateral indefinitely.

The escrow verifies cl_B : it checks the inclusion proof against $R_B^{(e+1)}$ in the witness log, matches the claim against the acceptance preconditions signed at bond-post time (which named the staging schema as a covered surface), and decides: *clear*. Acme's bond is debited; Beta receives the cleanup-cost portion; the remainder is refunded to Acme. The settlement record is written to the witness log.

8.8.6 Step 4: Revocation propagation

At this point, Acme's risk officer revokes Controller-7's `db:migrate` card and publishes $R_A^{(e+2)}$.

Numbers by hand — *How many rounds, for eight harbors.* Theorem 8.5.1's epidemic model says dissemination completes in expected $\Theta(\log m)$ rounds on a complete overlay of m harbors, because each round roughly doubles the number of informed harbors: round 1 informs 2 (the publisher plus one contacted peer), round 2 informs up to 4, round k informs up to 2^k . Set $2^k = m$ and solve for k : $k = \log_2 m$. This example's own consortium has three harbors (A, B, S), too few for the asymptotic to say anything the reader could not already see by inspection; scale it up to see the shape of the bound instead. At $m = 8$ member harbors, $\log_2 8 = 3$: expected convergence in about three gossip rounds, each of duration Δ . At $m = 64$, only $\log_2 64 = 6$ rounds — doubling the federation eight-fold costs only three more rounds, which is the whole point of an expected-logarithmic bound. This is the model's expectation under its own stated assumptions — a connected overlay and reliable rounds — not a deadline; during a partition, receiving harbors fall back to expiry or a fail-closed freshness policy, exactly as §8.5.1's third clause says they must.

8.8.7 What this example shows

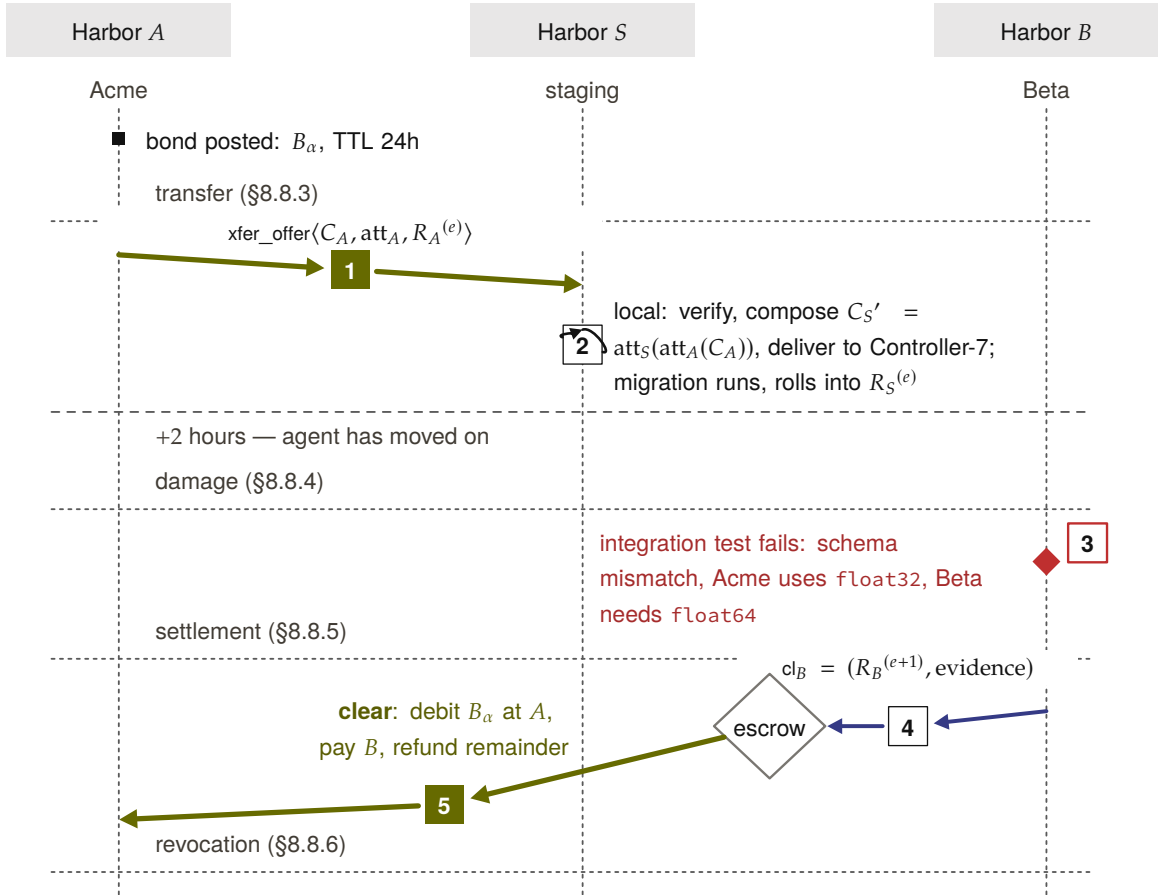
The example is small and the failure is real. The point of working it is to show that each of the four primitives — transfer, revocation, settlement, admission — is doing its job at exactly one step, and that the composition is straightforward when each primitive is honest about what it covers.

Notice in particular what *did not* happen:

- Neither D_A nor D_B trusted the other's signing key as a root. The transfer worked through the witness-logged public keys, not through a chain of trust.
- The bond was not held by either harbor; the design requires a recipient-whitelisted custody ledger before the escrow's role is structurally bounded.
- The settlement was not negotiated peer-to-peer; it followed a fixed protocol whose two outcomes were named in advance.
- Under the example's connected reliable-round assumptions, the revocation propagated through gossip with expected logarithmic completion; no consensus protocol was invoked.

Each of these is a deliberate design choice. The Federated Harbor's architecture is not heroically more powerful than alternatives; it is heroically more honest about what is happening at each step.

one incident, four primitives, in order



#	step	binds / value
—	bond post	A funds B_α , locked TTL 24h, signed into $R_A^{(e)}$ and $R_S^{(e)}$
1	transfer offer	$A \rightarrow S$, signed sk_A : $C_A, att_A, R_A^{(e)}$
2	local deliver	S composes C_S' , delivers to Controller-7; migration executes
3	damage detected	B integration test fails on the staging schema, 2h after transfer
4	damage claim	$B \rightarrow \text{escrow}$: $cl_B = (R_B^{(e+1)}, \text{evidence})$, a Merkle inclusion proof
5	clearance	$\text{escrow} \rightarrow A, B$: bond debited, B paid, remainder refunded to A
6	revoke + gossip	A publishes $R_A^{(e+2)}$; expected 3 rounds ($\log_2 8$) across 8 harbors

Figure 8.12: One incident, four primitives, in strict order. The card of Controller-7 crosses from Acme harbor A to the shared staging harbor S (message 1–2); two hours later Beta harbor B discovers the schema damage the migration caused (mark 3); the bond posted at A settles against B claim through the escrow (messages 4–5, §8.6); Acme then revokes the card of Controller-7, with gossip expected to reach all eight federation members in about three rounds (§8.8.6). No step needed the other two harbors to trust each other signing key as a root; each primitive covers exactly one step, and no more. [internal]

8.9 Formal Model

The formal substrate of this paper is two-layer: ProVerif [33] models for the cryptographic protocols (cross-realm authenticity, attenuation, settlement no-redirect) and TLA+ extensions of Bonded’s Conservation specification for the cross-harbor accounting invariants.

8.9.1 ProVerif extensions to Anchor

The Anchor Protocol’s three-phase models are extended with a second signing key and an epoch-root binding. The cross-realm authenticity query becomes:

```

1 query b: id, ha: id, hb: id, cap: capability, e: epoch;
2   event(AcceptedAtB(b, hb, cap, e))
3   ==> (event(IssuedAtA(b, ha, cap)) &&
4         event(EnvelopeAtoB(ha, hb, cap, e)) &&
5         event(RootCurrent(ha, e))).

```

Listing 8.1: Cross-realm authenticity query (sketch)

This says: if B accepts a cross-realm card for capability cap at epoch e , then there exists a corresponding issuance event at A , an envelope event from A to B binding the same capability and epoch, and an event confirming that $R_A^{(e)}$ is the current root. The query holds against the Dolev–Yao attacker model under the standard assumption of Ed25519 EUF-CMA security.

The model is drafted and the proof script is in progress. The full mechanization is named open work; see Appendix 8.A for the status table.

A different piece of this chapter’s federation-layer trust story is already fully mechanized, not drafted: `proofs/bonded/federated/federated.pv` is Bonded Commons §7’s federated multi-principal secrecy result, checked with the daemon’s local DB, the KMS, and the email provider all compromised simultaneously (three of the four principals that could reconstruct a vault). Its terminal output, unedited:

```

1 Starting query not attacker(account_root[])
2 RESULT not attacker(account_root[]) is true.
3 -----
4 Verification summary:
5
6 Query not attacker(account_root[]) is true.

```

Listing 8.2: `proofs/bonded/federated/federated.pv`, actual ProVerif output

Exercise 8.5 walks through why checking exactly this three-compromised scenario is enough to cover every weaker one, and what it does not cover.

8.9.2 TLA+ extension for cross-harbor conservation

The Bonded Commons paper’s Conservation specification models a single-machine bond pool. The Federated Harbor extension adds a per-harbor pool, an escrow pool, and an inter-harbor transit relation:

```

1  CONSTANTS Harbors, MaxBonds, Capabilities
2  VARIABLES Bucket, WitnessLog
3
4  TypeOK ==
5  /\ Bucket \in [1..MaxBonds -> {"posted", "escrow", "cleared", "
   ↪ refunded"}]
6  /\ WitnessLog \in Seq([harbor: Harbors, epoch: Nat, root: STRING
   ↪ ])
7
8  BucketPartition ==
9  \A b \in 1..MaxBonds:
10   \E! s \in {"posted", "escrow", "cleared", "refunded"}:
11     Bucket[b] = s
12
13 Spec == Init /\ [][Next]_<<Bucket, WitnessLog>>

```

Listing 8.3: Cross-Harbor Conservation specification (sketch)

The full specification (Appendix 8.C) associates each bond with an amount and enforces amount-preserving moves through these buckets. Apache checks the resulting transition system at $|F| = 4$ and bond depth 16. This establishes the invariant only for the model and bound; it does not establish the custody-ledger assumptions or arbitrary-scale runtime conformance.

8.9.3 What is mechanized and what is structural

We are explicit about which claims live in which band (Figure 8.2).

Mechanized (model scope). Single-harbor capability authenticity is inherited from Anchor. Cross-realm authenticity and attenuation are draft models. The cross-harbor bucket-partition invariant is model-checked at $|F| \leq 4$.

Conditional (assumption scope). Revocation dissemination is expected $\Theta(\Delta \log m)$ only under the stated reliable connected-overlay model (Theorem 8.5.1). The escrow extraction bound is conditional on a non-bypassable recipient-whitelisted custody ledger (Design Invariant 8.6.1).

Open (cobalt). Multi-principal correlated-equilibrium extension of Bonded; trustless settlement for non-fungible reputation-priced bonds; folk-theorem cartel-resistance bound at the federation layer.

8.10 Failure Modes

A federation paper that does not engage formally with the historical failure modes of federated systems is unserious. The literature is heavy with examples: SAML federations that concentrated on a small set of identity providers [165]; PGP web-of-trust that never achieved meaningful adoption outside enthusiast circles [1]; Sigstore’s gravitation toward a single deployment of Fulcio [180]. We engage with three failure modes specifically.

8.10.1 Centralization gravity

Federated identity systems have a well-documented tendency to centralize on a small number of trust anchors within five to ten years of deployment [165]. The Federated Harbor’s structural pushback is the replicable witness log: any harbor can run its own witness instance, any harbor can audit any other instance’s roots, and equivocation between witnesses is detectable. This makes it more expensive to be a centralizing hub than to be a peer.

The pushback is not a theorem; it is design discipline, and Figure 8.2 places it in the cobalt band for that reason. The historical evidence is mixed. Tailscale [198] has remained operationally centralized on its own coordination server despite Headscale [35] being a viable alternative; in-toto [254] has stayed decentralized but is consumed mostly by SLSA [153] which has Google as its de facto center. Time will tell which pattern the Federated Harbor exhibits.

8.10.2 Equivocation by a malicious witness

A malicious witness log can serve different views to partitioned auditors. A contradictory signed pair is immediately verifiable once co-located, but the protocol has no topology-independent time bound for bringing the pair together. Witness bonding applies after confirmation.

For high-stakes settlements, harbors can require multiple witness signatures before treating a root as authoritative, in the Certificate Transparency two-SCT pattern [145]. This trades latency for assurance; the design space is well-explored in the CT literature and we adopt the same techniques.

8.10.3 Partition tolerance

The federation does not run consensus; partition tolerance is per-protocol. Capability transfer cannot proceed during a partition between *A* and *B* (no surprise; cross-realm authentication requires both daemons to reach the witness log). Revocation gossip continues within each partition; convergence is bounded by the partition’s local size, not

the global federation size. Settlement is parked at the escrow until the partition heals.

The structural posture is that partitions are operationally common and the federation should degrade gracefully rather than fail fast. We inherit this discipline from Anchor’s intra-mesh design [187]; the federation extension is straightforward but adds no novel mathematics.

Table 8.5 lays out all three protocols side by side.

Table 8.5: Federation runs no consensus, so partition tolerance is decided protocol by protocol. Transfer stalls outright; revocation keeps working inside each side of the split; settlement holds its position until the split heals. [internal]

protocol	behavior under partition	what resumes it
transfer	cannot proceed: cross-realm authentication needs both daemons to reach the witness log	the partition heals and both sides reach the log
revocation	continues inside each partition; convergence bounded by that partition’s own size	the two partitions merge their informed sets on reconnection
settlement	parks at the escrow, holding the bond in place	the partition heals

8.11 Related Work

The Federated Harbor sits at the intersection of four bodies of work. We are concrete about where we draw on each and where we depart.

Federated identity. The OpenID Federation 1.0 specification [103] is the closest existing analogue: each entity publishes a signed Entity Statement, trust chains assemble by walking up to a Trust Anchor, trust marks express property claims. The Federated Harbor is structurally similar in the trust-anchor pattern but differs in two places: (a) we add a capability/attenuation layer (OpenID Federation is identity-only), and (b) we do not assume HTTP-based fetching of entity statements, since Port Daddy daemons cannot reliably expose HTTP endpoints (NAT, sleep, etc.). We draw also on SAML/Shibboleth historical experience [165] as cautionary lineage; the centralization gravity we discuss in §8.10 is exactly the SAML-era failure mode.

Capability-based federation. Macaroons [30] are the foundational capability-with-attenuation construction; our transfer ceremony extends Macaroon-style third-party-caveat chains across an administra-

tive boundary. UCAN [260] is the current production-deployed answer to “Macaroons with public-key principals,” and we treat it as the operationally most relevant existing system; the Federated Harbor differs in keeping the daemon as a structural commons authority rather than going fully peer-to-peer, and in pricing delegation through insurance-priced bonds rather than pure capability constraints. The object-capability lineage [175, 62] continues to be relevant: cross-realm operations are exactly a mutual-suspicion scenario in Miller’s vocabulary, and his defensive-consistency framework is the right lens.

Production federation protocols. Three federations that actually run solve a version of the same trust-boundary problem, and a reader who knows them will ask about them before asking about OpenID Federation, so here they are. ActivityPub [148], the W3C protocol under Mastodon and the rest of the fediverse, lets independent servers relay signed activity with no shared database and no global ruler — which is this chapter’s own stated goal, met at a scale of millions of accounts. Matrix [167] has homeservers replicate room state across administrative boundaries with eventual consistency and no single point of control, and is, of anything deployed today, the closest working instance of the “no consensus, gossip-bounded, detectable equivocation” posture of §8.2.4. And SPKI/SDSI [73], the mid-1990s design for local names and decentralized authorization, is the ancestor both Macaroons and UCAN descend from, one layer beneath the capability-based federation of the previous paragraph. None of the three overturns a theorem in this chapter. What they do is set the bar for what “federation without a ruler” has already been shown to survive in production, and the differences — neither ActivityPub nor Matrix carries a bonded, attenuating capability across the boundary, and neither prices delegation — are the chapter’s actual contribution, stated against the right baseline.

Transparency logs. Certificate Transparency [145] is the model; the federated witness log of §8.2.3 is CT applied to capability tokens. Sigstore [180] and Rekor are the production instantiation we rhyme with most heavily; in-toto [254] and SLSA [153] are the supply-chain analogues that compose with the Federated Harbor for the “poisoned plugin under a bonded principal” case but do not close it. The Crosby–Wallach history-tree [52] is the formal substrate of every modern transparency log including ours.

Cross-domain settlement. Herlihy’s atomic cross-chain swap [108] is a comparison point; the cross-harbor settlement protocol of §8.6 instead handles non-fungible reputation-priced bonds through conditionally restricted custody. It is not trustless in Herlihy’s sense. Whether trustless settlement is achievable in our setting is open (§8.12). The IBC protocol [89] supplies another connection/channel comparison; unlike

IBC, this design does not assume a consensus-backed state machine underneath.

Agentic-commerce discovery and mandates. The Universal Commerce Protocol [206] (Google/Shopify, 2026) solved cross-administrative discovery for agent-mediated retail with a pattern the Federated Harbor adopts (ADR-0051 Phase 1b): a / .well-known JSON profile that does double duty as capability declaration and JWK key publication, with server-selects version negotiation — the party that must execute a request computes the compatibility intersection. Where UCP assumes merchants hold stable HTTPS origins, our daemons often do not (the same constraint that ruled out OpenID Federation’s entity-statement fetching above); the profile therefore also travels as a relay-published mirror. UCP delegates authorization proof to the Agent Payments Protocol’s verifiable-credential mandates [166], whose SD-JWT/JWS/JCS formats the harbor boundary now speaks (ADR-0094). The division of labor is instructive: those protocols standardize discovery and the transaction envelope and are explicit that counterparty trustworthiness, collateral, and settlement enforcement are out of scope — which is the half of the problem this paper and its predecessors supply.

Table 8.6 lays out all five lineages against what this chapter borrows, departs from, and why.

Table 8.6: What the Federated Harbor borrows from each prior body of work, where it departs, and why. Mirrors the companion chapter own tab: related-borrow-depart (*The Bonded Commons*). [internal]

Body of work	Borrowed	Departure	Why
Federated identity (OpenID Federation)	the trust-anchor pattern: signed statements, trust chains assembled upward	adds a capability layer; no HTTP fetching	daemons cannot reliably expose HTTP endpoints
Capability federation (Macaroons, UCAN)	third-party-caveat attenuation chains; public-key principals	keeps the daemon a structural authority; prices delegation with bonds	delegation needs a price, not just a constraint
Transparency logs (CT, Sigstore/Rekor)	the append-only Merkle history tree, CT own model	applied to capability tokens instead of certificates	the witness log needs the same tamper-evidence CT gives certificates
Cross-domain settlement (Herlihy, IBC)	a trusted-third-party settlement comparison point	not trustless in Herlihy sense; no consensus-backed state machine underneath	bonds are non-fungible and reputation-priced, not fungible assets
Agentic commerce (UCP, ACP, AP2)	the well-known discovery profile; SD-JWT/JWS mandate formats	adds the bonding and settlement enforcement those protocols leave out by design	they standardize discovery, not counterparty trustworthiness

8.12 Limitations and Open Questions

We close with the honest open frontier. These are not throwaway caveats; they are the questions the paper does not answer, named precisely enough to motivate subsequent work. Figure 8.2 is the map they sit on: each of the questions below belongs to one of its three bands, and the band says what kind of answer would close it.

8.12.1 The multi-principal correlated-equilibrium extension

The Bonded Commons paper argued that single-machine coordination is a correlated equilibrium with the daemon as the correlation device [188]. Across two harbors with two daemons, the correlation device fragments. Two candidate resolutions remain unresolved:

- The federated witness log *is* the correlation device, and individual harbors are local enactors. This would extend Bonded’s result intact.
- The cross-harbor case is a partition of the correlated-equilibrium concept into intra-harbor (correlated) and inter-harbor (Nash with shared signal). This would weaken Bonded’s result at the federation layer.

We do not know which is right. The proposal that scoped this paper [190] named Thomas Youle as the essential collaborator for this question (his contribution to Bonded is the closest existing work). The question stays open here.

8.12.2 Trustless settlement for non-fungible bonds

HTLC-style atomic swaps [108] achieve trustless settlement for fungible assets with a shared hash preimage. The Federated Harbor’s bonds are non-fungible and adjudication-dependent, so that construction does not transfer directly. We adopt conditionally restricted custody (§8.6) and leave both a trustless construction and an impossibility result open. Either would require a separate threat model and proof.

8.12.3 Cartel formation across federations

Bonded’s folk-theorem cartel-resistance argument applies inside a single machine. Between machines, cartel formation is structurally easier: colluders can coordinate offline and present a unified front to the witness log, which records that all harbors agreed — a behavior consistent with both honest agreement and collusion. We owe either a strengthening of the bonded mechanism that resists cross-harbor cartels, or a precise statement of the weakening. We have neither.

8.12.4 Cold-start admission and the invitation-only failure mode

The bonded-sponsor pattern of §8.7 requires a new harbor to find a sponsor willing to underwrite its probationary risk. In practice, this market may be thin — the historical evidence on whether invitation-bounded federations stay open is mixed [165]. We treat the structural pushback (many sponsors, small probationary scope, witness-log permanence) as engineering discipline, not as theorem.

8.12.5 Equivocation propagation speed

Theorem 8.5.1 separates cryptographic verification from dissemination. The former is constant work once both roots are present; the latter depends on topology, loss, and partitions. Establishing a deployment-specific tail bound is open and is necessary before pricing the witness bond from propagation risk. A second gap sits next to it: §8.5.4 names the sheaf Laplacian’s spectral gap $\lambda_2(\Delta_{\mathcal{F}})$ as the natural handle on anti-entropy mixing speed but derives no bound from it, and does not reconcile linear sheaf diffusion with epidemic gossip over append-only logs. *open*, and stated sharply enough to be killed rather than left as an invitation: the conjecture is that the expected number of *gossip rounds* until the completion residual first exceeds its alarm threshold scales as $\Theta(1/\lambda_2(\Delta_{\mathcal{F}}))$ on a fixed topology with a single equivocator — that is, that the continuous diffusion’s mixing time predicts the discrete protocol’s time-to-detection up to a constant. The falsifier is a sweep, not an argument: run the harness over families of topologies whose spectral gaps differ by orders of magnitude (rings, expanders, barbells, and the tree case where the gap vanishes and the residual is provably blind), record rounds-to-first-alarm, and regress it against $1/\lambda_2$. If the fitted exponent is not 1, or if the constant moves with topology family rather than staying within a band, the conjecture is false and the spectral gap is the wrong handle — which would be worth knowing, because it is the handle the section currently recommends. Nothing in docs/harbor-research/program.json schedules this sweep today, so the honest status is a conjecture with a falsifier and no run behind it.

8.12.6 Cost of per-write durability

The cryptographic assertions and per-write durability checkpoints this layer inherits from Anchor carry a measurable I/O cost. They are designed for high-stakes coordination, where an audit trail is worth more than throughput, and not for bulk data movement or latency-critical paths. This paper does not measure that overhead at the federation layer and does not establish where the crossover sits. *open*.

8.12.7 What this means for the reader

A federation paper with six named open questions is doing its job. The point of the paper is to draw the new boundary cleanly so the open problems are visible; if every claim in this paper were closed, there would be no third paper to write. The honest read is that the Federated Harbor is closer to a research agenda than to a deployed product, and the prior two papers (Anchor, Bonded) have a tighter ratio of theorem to conjecture for the same reason. The substrate is real; the federation layer is in progress.

A boundary drawn cleanly is itself a result: it says where the next proof has to start.

THESIS

*Exercises 8.6–8.11,
p. 520.*

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **Two machines, no shared daemon.** A federated authority admits a foreign act only through a witness log each side can check; admissibility is a design invariant of the log, not a property of the peer.
2. **Capability transfer shrinks at every hop:** the set printed at Harbor A contains the set acknowledged at Harbor B, and B verifies locally at delivery without reaching back to A. Attenuation monotonicity across the boundary is the theorem.
3. **Revocation crosses the boundary as an epoch:** dissemination is conditional on connectivity, the stale window is bounded in a connected federation, and a partition has no finite deadline.
4. **Inconsistency is detected exactly when the missing edge closes a visible cycle.** On six nodes the cycle C_6 has consistency radius 1.2247 and the path P_6 has 0: the same disagreement, visible on one and invisible on the other.
5. **The completion residual is sound and achievable,** it localizes the fault, and its cost is stated; two liars on one cycle can cancel, and that is the boundary.
6. **Settlement across harbors is bucketed.** Escrow extraction is bounded conditional on the custody ledger, and cross-harbor buckets partition, so one harbor's failure is confined to its bucket.
7. **Admission is a ceremony with a record:** a harbor joins a federation by exchanging epoch roots, and the record is what a later dispute reads.

8. **The rollback configuration fails by design.** The card-revocation model checked against a rollback of the epoch root produces a counterexample, which is the property, not a bug in the checker.

8.13 Exercises

§8.4 CROSS-HARBOR CAPABILITY TRANSFER

- 8.1 **TRACE ★★** Alice’s card C_A authorizes `db:write` on A ’s *test* database. D_B accepts it and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$ authorizing `db:write` on B ’s *staging* database. Trace Theorem 8.4.1’s proof sketch against this concrete case: at which step, if any, does “subset composition is a subset” stop being a statement about the *same* resource space? State precisely what kind of act minting C'_B actually is here, and what two things a correct D_B must additionally establish before minting it. Solution p. 572

§8.5 FEDERATED REVOCATION

- 8.2 **TRACE ★★** The “only-issuer-revokes” rule above is trustworthy only if a harbor’s own local revocation record is *final* — once revoked, always revoked, even across an operational mishap at the relay. `proofs/relay/CardRevocation.tla` model-checks exactly this property (`RevocationFinal == badAccept = FALSE`) under three configurations. Read `proofs/relay/CardRevocation.md` and state, for each of the baseline, rollback, and epoch configurations, what changes in the model and what TLC found. Solution p. 573
- 8.3 **TRACE ★★** Read `skills/harbor-results/scripts/sheaf_consistency_radius.py`, section [0] (function `section0`). By hand, using $r = |s|\sqrt{1 - R_{\text{eff}}(e)}$ and $R_{\text{eff}} = (n-1)/n$ on C_n , confirm the two numbers the script’s closed-form check pins for C_6 and P_6 at lie magnitude 3.0. Then state what the script’s `check(...)` call actually asserts for each case, and to what tolerance. Solution p. 574
- 8.4 **OPEN ★★★** Theorem 8.5.3 assumes a *single* equivocator and says outright that this is not decoration. Two colluding reporters sit on the same visible cycle K_c and each reports an offset; either offset alone would be detected. Using only the definition of the completion residual r as a minimum over global sections and free severed blocks, explain why a coalition can drive r to (numerically) zero even though neither liar’s offset is small, and say what kind of evidence — if any — could still catch the coalition. Solution p. 574

§8.9 FORMAL MODEL

8.5 TRACE ★★ `proofs/bonded/federated/federated.pv` is a separate, already-mechanized ProVerif model from the cross-realm sketch above — it is Bonded Commons §7’s federated multi-principal secrecy theorem, reused here because it is the formal backbone for reasoning about trust split across administrative principals. Read the model’s header comment and state: which four principals it names, which single query it actually runs, and why running only the “daemon + email + KMS compromised” scenario is enough to cover the weaker three-principal scenarios too.

Solution p. 575

§8.12 LIMITATIONS AND OPEN QUESTIONS

8.6 OPEN ★★★ §8.12.1 names two candidate resolutions for the correlated-equilibrium concept once two daemons, not one, are in play. State a stochastic/Bayesian game model of the two-harbor case in which the witness log could serve as a correlation device, name the informational event (a common-view or freshness event) under which you would try to prove a coarse correlated equilibrium, and give a safe degraded policy for the case where that event has not occurred.

Solution p. 576

8.7 OPEN ★★★ §8.12.2 leaves open whether trustless settlement is possible for the Federated Harbor’s non-fungible, adjudication-dependent bonds. HTLCs solve the fungible case using a shared hash preimage. Explain, from the definition of an HTLC’s atomicity condition, why that construction does not transfer to a bond whose “did the damage occur” predicate is subjective, and sketch what an impossibility argument for the general case would need to assume.

Solution p. 576

8.8 OPEN ★★★ §8.12.3 observes that the witness log cannot distinguish unanimous honest agreement from unanimous collusion. Using the one-shot deterrence inequality $p \cdot d \cdot L > G$ (audit probability, detection probability, loss on detection, collusive gain), state what would have to be true of a federation-layer audit mechanism for that inequality to be satisfiable against a cartel that controls a majority, but not all, of the witnesses in scope — and say plainly why no structural mechanism closes the case where the cartel controls *every* oracle and witness.

Solution p. 577

8.9 OPEN ★★★ §8.12.4 lists engineering discipline (many sponsors, small probationary scope, witness-log permanence) against the bonded-sponsor pattern’s cold-start risk, but explicitly declines to call it a theorem. Describe a concrete measurement — stated as a metric over time, not a proof — that would let an operator tell the difference between “the sponsorship market is thin but open” and “incumbent sponsors have formed an invitation cartel.”

Solution p. 577

8.10 OPEN ★★★ §8.12.5 names two unresolved gaps side by side: a deployment-specific tail bound for equivocation propagation, and the unreconciled relationship between epidemic dissemination (Theorem 8.5.1) and sheaf-Laplacian diffusion (§8.5.4’s closing paragraph). Explain why these are two *different* open problems rather than one problem stated twice, and name one measurement that would be needed before either could be closed.

Solution p. 578

8.11 OPEN ★★★ Standing back from the individual protocols: this paper repeatedly describes federation as “not a consensus protocol” (§8.2.4) while also proving a bucket-partition invariant by model-checking a bounded transition system (§8.6.2). Reconcile these two statements — what, precisely, does not require consensus, and what does the Conservation model-checking result actually presuppose about agreement? Then state, in your own words, what a reviewer should mean by calling the Federated Harbor “a project institution, not merely two machines sharing a repository.”

Solution p. 579

8.14 Conclusion

The two-machine problem is the structural sequel to the local-swarm problem of Anchor and the trust-as-infrastructure argument of Bonded. The proposed answer is not a new substrate but a clean extension: keep each daemon sovereign inside its own machine, add a shared witness log that no harbor controls, hang revocation gossip and conditionally restricted custody off the witness log, and gate federation admission on bonded sponsorship rather than permissionless entry. Each primitive does exactly one job. Each primitive is honest about what it does not do.

The proof posture matches the prior papers’ discipline. Where we mechanize, we mechanize (with the same ProVerif and TLA+ infrastructure). Where we bound a threat, we name the bound and price the bond against it. Where we leave a question open, we name it. The threat-band diagram of Figure 8.2 is the most concentrated summary of where each claim lives.

What this paper supplies is a clean federation surface that could compose with the local substrate: a transfer ceremony, witnessed revocation records, a conditional custody design, and bonded admission. The protocol and models make the missing implementation and service-level assumptions explicit; they do not establish that arbitrary harbors can already interoperate safely.

The remaining work—runtime custody enforcement, deployment conformance, a multi-principal equilibrium model, and a cartel-resistance bound—is material. The local substrate is real; the federation described here remains a design with partial mechanization.

Chapter Appendices

8.A Verification Status

Artifact registry. The cross-paper status table in Bonded Commons [188] carries the items that are shared across the series; this appendix lists only the items specific to the Federated Harbor.

Table 8.7: Federated Harbor verification status. **CLOSED:** model verified and runtime conformance test passes. **PARTIAL:** design specified, mechanization in draft. *open:* known unmechanized; threat band visible in Figure 8.2 (cobalt).

Claim	Method	Result	Artifact
Single-harbor capability authenticity (inherited)	ProVerif 2.05	CLOSED	Anchor harbor_card_v*.pv
Cross-realm authenticity (§8.4)	ProVerif (draft)	PARTIAL pending witness-log freshness assumption	fh-xfer.pv (draft)
Attenuation monotonicity across boundary (Lem. 8.4.1)	ProVerif + Anchor sub-result	PARTIAL	fh-att.pv (draft)
Federated revocation dissemination (Prop. 8.5.1)	Conditional epidemic model [60]	PARTIAL expected asymptotic result; no partition deadline	fh-conv.tex (proof sketch)
Cross-witness equivocation evidence	Signed-root comparison; dissemination model	PARTIAL verification immediate once views meet; no hard delivery bound	—
Escrow extraction bound (Prop. 8.6.1)	Transition-system case analysis	PARTIAL conditional on non-bypassable custody; no runtime conformance	fh-escrow.pv (draft)
Cross-harbor Conservation (§8.9.2)	TLA+ / Apalache, $ F \leq 4$	PARTIAL model-checked at scale	CrossHarbor Conservation.tla

(continued)

Claim	Method	Result	Artifact
Sheaf detection of equivocation (Thm. 8.5.3)	Pre-registered statistical harness, 200 trials/arm	PARTIAL gates pass at the stated scope (single equivocator, real stalks, prefix restrictions); not mechanized	sheaf_harness_v2.py
Sheaf-Laplacian relaxation-time bound (§8.5.4)	—	<i>open</i> no derivation and no citation; the gossip and linear-diffusion models are not reconciled	—
Trustless settlement for non-fungible bonds	—	<i>open</i> no construction or impossibility proof	—
Multi-principal correlated-equilibrium extension	—	<i>open</i>	—
Cross-federation cartel-resistance bound	—	<i>open</i>	—

What this table means. Every row in the PARTIAL band is committed to and on the active workplan. Every row in the *open* band is a research question we cannot resolve from the desk; the proposal [190] names collaborators whose contributions are necessary to close them. We err toward listing fewer items as closed rather than more, in the same posture as Anchor and Bonded.

8.B ProVerif Models (draft)

For brevity we omit the full model text in this pre-print; the draft `fh-xfer.pv` extends Anchor’s `harbor_card_v3_delegation.pv` with a second signing key sk_B , an epoch root binding, and a cross-realm authenticity query of the shape sketched in §8.9.1. The full model will accompany the revised version once the witness-log freshness assumption is mechanized rather than assumed.

8.C TLA+ Specification (draft)

The full `CrossHarborConservation.tla` extends Bonded's Conservation specification with per-harbor pools, an escrow pool, and an inter-harbor transit relation. Apalache symbolic model-checking at $|F| = 4$ harbors and bond depth 16 establishes that Conservation holds across all reachable states under the transition relation. The full specification accompanies the revised version.

THE CONCLUSION OF THE BOOK The seven answers now form one discipline. Mediate every effect that must be prevented; let delegated authority only narrow; preserve enough evidence for a human to challenge the system; attach actions to a durable principal and work unit; make settlement conserve value; and cross organizational boundaries by exchanging verifiable records rather than by exporting sovereignty. Better models help inside this discipline. They do not replace it.

SOLUTIONS TO THE EXERCISES

Chapter 1: The Single-Writer Kernel

- 1.1 In memory the substrate still serializes, so resource exclusion, identity and presence, communication, obligation checks, policy checks, and self-attestation keep their transient meaning while the process lives. Everything the subsystems promise across a crash becomes vacuous: durable continuity, crash recovery, historical audit, and outcome evidence; durable identity collapses to an ephemeral handle. The decomposition survives; the cross-crash guarantees do not. *Exercise p. 69*
- 1.2 The communication subsystem's home table is messages; it owes the layer above an ordered, cursor-addressable, durable envelope with explicit at-least-once semantics. Equally valid: `claims` for the resource subsystem (one compatible holder per conflict domain), or `commitments` for the obligation subsystem (no `done` without a current structured oracle receipt). *Exercise p. 69*
- 1.3 A defensible nine-way cut: storage and durability; transaction serialization; identity and presence; resources and leases; message transport; stigmergy and projections; policy and capability; commitments and outcomes; attestation and recovery. It separates proof obligations that leak across the seven, above all transport semantics from message semantics and policy prevention from post-commit detection. It hides the simplifying fact that all local state shares one transaction boundary, so any such cut must retain a cross-cutting invariant: one ledger, one effect mediator. *Exercise p. 69*
- 1.4 Module A renames `users.name` to `display_name`; module B, loaded later under older code, sees no name column and lazily adds it back. Writes now split between two columns according to module version and load order, and neither backfill is authoritative. Every destructive migration has this shape; only additive tables and columns are safe under any-module-any-order. *Exercise p. 69*
- 1.5 Command line, authenticated Unix-socket request, daemon request queue, the sole read/write connection, one transaction, WAL commit. The discipline holds by routing at two points, the client-to-API boundary and the daemon's sole-connection bound- *Exercise p. 69*

ary; SQLite's file lock is one backstop at write time. The primary story is routing, not a database rule that forbids other connections.

- 1.6 Expand, backfill, dual-read-or-write, contract keeps mixed versions safe for a while, but the contract step and the dependency order are global facts. Store a migration ledger with a schema epoch, each module's compatibility range, its dependencies, and a backfill watermark. Idempotent self-initialization can remain for additive tables and columns; safe rename, backfill, and down-migration cannot preserve unrestricted any-module-any-order. A user_version sequencer, or an equivalent ordered capability ledger, is unavoidable. *Exercise p. 69*
- 1.7 (a) I1a-survivable. (b) An orderly shutdown flushes, so a genuinely clean reboot is not the I1b case; an abrupt reset during the reboot is I1b-exposed. (c) I1b-exposed. (d) I1a-survivable after commit. None of the four lets an uncommitted transaction survive. *Exercise p. 69*
- 1.8 The commit is power-loss exposed from t_0 until the WAL frames that contain it are synced. A page-count threshold bounds accumulated WAL frames, not elapsed time: at a low or zero write rate it may not fire for arbitrarily long, so it gives no wall-clock bound. Only with an independently guaranteed write-rate floor $w_{\min}$ pages per second and threshold P does $\delta \lesssim P/w_{\min}$ follow, and this chapter assumes no such floor. *Exercise p. 70*
- 1.9 Make durability a typed argument of the transaction, PROCESS_CRASH or POWER_LOSS, never an adjective in prose. Route POWER_LOSS operations through a connection configured synchronous=FULL before the transaction opens and verify the commit before returning; if SQLite settings are connection-scoped, a separate fully-synced financial ledger is the cleanest isolation. Ordinary claims stay on NORMAL. Hard-reset fault tests must exercise the stronger path. The guard against silent defaulting is the type: a write path that names no durability class does not compile, and a forced checkpoint bounds recovery and log size but is not the per-write primitive. *Exercise p. 70*
- 1.10 A release with no matching (key, holder) row is a successful no-op so that retries are idempotent: this is I3. It must never delete another holder's row. If it raised, a client that timed out on its first release could not distinguish success from failure, and at-least-once handling would break. *Exercise p. 70*
- 1.11 The daemon serializes the two requests. The first INSERT(key, holder) commits. The second hits the unique-key violation, selects the extant row inside the catch path, and returns DENIED(holder=first): that select is where the loser learns the name. For range and hierarchy claims the trace is safe only after both re-

quests map to the same canonical conflict key or the overlap test runs in one immediate transaction.

- 1.12 Add a durable FIFO ticket queue per conflict domain, a maximum lease L , a monotonically increasing fencing epoch, and grant-to-head on release, on acquire, and on lazy expiry. No background scheduler is required for safety, but liveness requires some live caller or tick to trigger the transitions. Under daemon availability, eventual requests, and non-renewable bounded leases, a requester with q predecessors waits at most about $(q + 1)L$ plus processing delay; without those assumptions no bounded wait exists. Every external effect must validate the fencing epoch, or an expired holder can still act. *Exercise p. 70*
- 1.13 If the evaporation tick stalls for an hour, tick-only storage still reports yesterday's marker weight as today's. Read-time evaluation computes $w r^{\Delta t}$ from the persisted timestamp and never presents the stale stored number as current. *Exercise p. 70*
- 1.14 The transport delivers the request twice. The consumer applies one semantic state transition and recognizes the duplicate as transport nondeterminism; diagnostics may count the redelivery, the operator's view shows one act. An envelope idempotency key plus a unique (consumer, key) receipt lets the consumer store "effect applied" atomically with the effect, making retry safety explicit rather than conventional. Exactly-once is still not guaranteed for an unbrokered external effect. *Exercise p. 70*
- 1.15 For each marker kind, log creation, reads, the actions it influenced, its supersession, and the time at which operators label it stale. Fit a survival curve and choose the half-life that minimizes $C_{\text{stale}} \cdot \text{false-live} + C_{\text{lost}} \cdot \text{false-expired}$, validated on held-out projects. Claims and presence should decay far faster than decisions or learned skills. Publish the confidence and re-estimate under drift. *Exercise p. 70*
- 1.16 Note-count monotonicity can be regimented by append-only storage or a trigger that rejects deletes and regressive updates. Capability attenuation and lock-owner release can be pre-checked only if authenticated identity, the capability order, and every relevant effect path pass through the mediator. Duplicate keys and boot admission are already pre-effect. Heartbeat freshness cannot be rejected at the heartbeat insert: staleness is an absence observed only after time passes, and under asynchrony a slow actor is indistinguishable from a failed one. Out-of-band filesystem and network effects remain detect-only until mediated. *Exercise p. 71*
- 1.17 `close(done, oracle_ref="")` reaches the oracle-reference check, returns REFUSED, leaves the commitment open, and records the denied authority act. It never reaches classification, current- *Exercise p. 71*

state verification, or the DONE transition: the empty reference fails the finite-vocabulary test before any oracle is consulted.

- 1.18** Note monotonicity: the prohibited event is a write the daemon mediates and the trigger (the previous count) is committed state; top-right, regimentable, and append-only storage is the supervisor. Capability escalation: regimentable only if every effect path and the identity behind it pass through the mediator; an escalation exercised through an unmediated channel is an uncontrollable event, which moves the rule to the left column until the channel is owned. Lock-owner validity: the release is mediated and the owner is committed state, top-right, provided the actor identity is authenticated (I12); with a self-asserted identity the trigger is not witnessed and the rule drops to the bottom-right cell. *Exercise p. 71*
- 1.19** The first gates a controllable event on a controllable trigger: both `api_call` and `spawn_child` lie in Σ_c , the policy never disables an uncontrollable event, and it is regimentable. The second must disable `internal_plan`, an uncontrollable event, in its post-trigger state, and the plant enables it there; the history `fs_write` followed by `internal_plan` witnesses the failure of controllability, so the rule is detect-only. *Exercise p. 71*
- 1.20** P_4 is a two-state taint automaton whose trigger is `git_push`. In the product with P_2 (no `git_push`, ever) the trigger is disabled in every reachable state, so P_4 's tainted state is unreachable in the closed loop and the disablement map is unchanged: `ok` disables `{net_egress, git_push}` and tainted disables `{net_egress, git_push, fs_write}`. Controllability still holds, since no reachable state disables an uncontrollable event. The lesson is the one synthesis is for: P_4 is vacuous under P_2 , and a hand-written engine would have carried the dead rule forever. *Exercise p. 71*
- 1.21** A protected-run receipt: tree hash, a suite hash the principal controls, runner or image hash, command, timestamp and nonce, exit code, and a signed result. Closure verifies every binding and the freshness. This resists free-text and stale-result manipulation; it is not universally Goodhart-proof, because the suite may be incomplete or poisoned. The honest theorem is syntactic provenance and non-replay, not semantic correctness. *Exercise p. 71*
- 1.22** Notes preserve explicit goals, rationale, observations, and artifact pointers. They cannot preserve unflushed edits, process memory, open handles, provider caches or hidden state, or a latent next action the model never externalized. A successor can understand the story and still be unable to resume the exact computation. *Exercise p. 71*
- 1.23** The append commits. A later delete or regressive count also commits and enters the operation log. Only then does the subscriber *Exercise p. 71*

observe the count drop and raise or compensate. The violation is durable at the second commit; the monitor proves detection, not prevention.

- 1.24 A content-addressed task capsule sealed at a safe tool or message boundary: base tree and worktree blobs, environment and tool manifests, open claims with epochs, commitments and acceptance criteria, exact tool input and output, pending external operations with their idempotency keys, decisions and hypotheses, a provenance-preserving transcript or compaction, and external references. Flush the ledger and quiesce mediated effects before sealing. This restores observable task state and evidence; it cannot recover hidden activations, unexposed reasoning, provider-internal caches, or unrecorded intent. Arbitrary-provider recovery is semantic continuation, not bit-identical resurrection. *Exercise p. 71*
- 1.25 delegate from the initial state with a child scope not contained in the root grant's scope $\{a, b\}$, for instance $\{a, b, c\}$: with the attenuation guard off the grant is created and I4 fails immediately. No other guard can be broken in one step, because effects need the executing phase, settlement needs the verified phase, and a double settlement needs two operations. *Exercise p. 72*
- 1.26 One candidate: a contested work unit never settles (settle is guarded on the verified phase, but the transition from contested back to a settleable phase is unconstrained). Add the guard, re-run the search, and confirm the state count and that the mutation suite finds a trace with the guard off; if the checker finds none, the invariant was already implied and does not earn its row. *Exercise p. 72*
- 1.27 One daemon connection handles every claim and lock read and write; no read_uncommitted; respond only after commit; no out-of-band writer. A write-capable external connection violates the first and the fourth at once, the out-of-band-writer assumption most literally. A read-only connection need not break write linearizability, but it observes under different timing and API semantics. *Exercise p. 72*
- 1.28 A invokes `claim(k)` and commits at c_1 . B and C overlap; B's denied read commits at c_2 , A releases at c_3 , C's acquire commits at c_4 . One valid order is A-acquire, B-denied, A-release, C-acquire. If B and C overlap around the release, either may linearize first, as long as each result matches the state at its point and non-overlapping operations keep their real-time order. *Exercise p. 72*
- 1.29 Generate identical state-machine traces against both bindings: schema initialization and migrations, acquire, reclaim, release, and expiry, overlapping calls, transactions and rollback, busy and lock behavior, encodings, pragmas, crash and reopen, fault injec-

tion. After every step compare returned values, errors, serialized rows, schema, and recovery state, including the deployed binary in CI. A seeded fuzz misses divergences that depend on wall-clock timing, on a pragma the fuzz never sets, or on a crash at a point the fuzz never interrupts; targeted fault injection and pragma sweeps catch those. No finite suite makes I11 a theorem: that needs a formal refinement or equivalence proof, or the same verified binding on both sides. Label the suite conformance evidence.

- 1.30 The port stays claimed and a live or ghost session row exists, but no commitment says what work owns them. Other agents see a busy port and a presence they cannot reconcile; the failed starter may retry into its own leftovers. *Exercise p. 72*
- 1.31 Expose one `begin_work` endpoint that opens `BEGIN IMMEDIATE`, validates every input, inserts the claim, session, commitment, and outbox rows, and commits once; on any error it rolls back all of them. A saga would release the claim and mark or delete the session when the commitment insert failed, and each compensation can itself fail or crash between steps. All three rows live in one SQLite file, so the local transaction is both simpler and stronger. *Exercise p. 72*
- 1.32 A later same-user tamperer of the database file, or a remote synchronizer that cannot rewrite an externally anchored root. The same-user process is out of scope at the substrate; the adversary becomes real at the cross-machine economy layer. Without an independent anchor or verifier, the writer can rewrite the log and recompute the chain. *Exercise p. 72*
- 1.33 The release path compares the supplied actor identity to the lock owner. If a legacy endpoint accepts a self-asserted identity, the attacker submits the victim's string, the equality check passes, and the owner-scoped delete executes. The lock rule is correct relative to its input; I12 failed to authenticate the input. Complete credential and peer binding must precede every security-relevant write. *Exercise p. 72*
- 1.34 The minimum useful wall is a daemon under a separate UID (or a VM), a daemon-owned database and key, a Unix-socket ACL plus kernel peer credentials, and brokered git, filesystem, and network effects. An OS keychain protects a key at rest but not arbitrary signing requests from an equally authorized same-user process; sealed memory does not stop same-UID tracing or injection without an OS isolation policy; external root anchoring detects a later log rewrite but does not prevent it. No trusted platform module is required for strong local separation: separate privilege and complete mediation are. After the sandbox the same-user adversary moves to the mediated-effects row, where side channels the sandbox does not enumerate remain in scope. *Exercise p. 72*

Chapter 2: The Anchor Protocol

2.1 The model hands the attacker *both* the honest issuer’s public key vk_A and the HMAC key vk_B in the clear — the worst case for a verifier that is only supposed to trust Ed25519-shaped tokens. `VerifierNaive` does not pin an algorithm; it pattern-matches on whichever type tag arrives, `tokenA` or `tokenB`, and calls that tag’s own verification routine. Knowing vk_B , the attacker builds `tokenB(Alg_B, m, hmacB(vkB, m))` for any message m , including one the honest `IssuerA` never signed; the naive verifier’s second branch matches the `tokenB` shape, the HMAC checks out under a key the attacker already holds, and `accepted_naive(m)` fires with no matching `issued_A(m)`. `VerifierPinnedA`, by contrast, only ever deconstructs the `tokenA` constructor — a `tokenB` token is not merely rejected, it is a shape the pinned verifier never attempts to read. Pinning means fixing, out of band, which single verification routine and key a given context accepts, so the token’s own header or shape can never be the thing that chooses how it gets checked — exactly the rule of Design invariant 2.2.1, mechanized here against an attacker who already holds every key the naive path would trust.

Exercise p. 129

Pinned verdict (proofs/anchor/token-verify/algconfusion.run.log):

```
1 RESULT event(accepted_pinned(m_4)) ==> event(issued_A(m_4)) is true
  ↪ .
2 RESULT event(accepted_naive(m_4)) ==> event(issued_A(m_4)) is false
  ↪ .
```

The first line is the property this chapter relies on; the second is the negative control, checked in the same file so the true result is not vacuous.

2.2 It checks that every chain C accepts corresponds to one P actually authorized, with the *same* (P, A, B, C, m) tuple: no hop can be replayed against a different message, and no hop can be spliced out of one chain into another. The defense is per-hop nonce freshness — each signature binds `hopBind(nonce, prev_id, next_id, message)`, and the verifier requires every nonce to have been issued and consumes it on acceptance, so a captured hop cannot be re-presented in a different context.

Exercise p. 129

Pinned verdict (proofs/anchor/delegation/chain-replay.run.log):

```
1 RESULT event(chain_accepted(idP_4,idA_3,idB_2,idC_1,m_4)) ==> event
  ↪ (chain_authorized(idP_4,idA_3,idB_2,idC_1,m_4)) is true.
```

This is a different property from Property 2.D.1’s capability-subset soundness: chain replay says who signed what is delivered intact; attenuation soundness says authority never grows along the way. §2.5.3 cites this exact artifact as the depth-3 evidence behind the “Delegation-

chain depth” boundary.

2.3 `proof_verify_logic_only` stubs signature verification and base64 decoding and shows the surrounding parse and branch logic cannot panic on any 32-byte, dot-containing, UTF-8 input within ten unwindings — a bounded absence-of-panic result under those stubs, not a proof that `verify` itself is correct. A 32-byte input cannot even hold a real three-part card, so only the *rejection* path is exercised; a reader should not conclude the acceptance path was checked at all.

Exercise p. 129

`proof_constant_time_behavior` calls the comparator once over symbolic 16-byte pairs and shows it cannot panic; it is not a two-run relational proof that the *time* taken is independent of the secret, and Kani adds nothing here beyond the source-level argument already given by inspection (no early return on a secret byte). Compiler, cache, and microarchitectural timing stay entirely outside it — a reader should not read “Kani checked it” as “this is constant-time in the deployed binary.”

`proof_capability_attenuation` asserts two concrete, hard-coded vectors, one subset accepted and one escalation rejected; it is a unit test wearing a Kani harness, not a universal statement over arbitrary capability structures. The actual every-hop attenuation property is what §2.D.1’s ProVerif model establishes, not this harness — a reader should not treat the two vectors as covering the property Property 2.D.1 states.

The bound in each case is recorded, not just asserted here: `whitepaper/corpus.json` carries an `evidencePolicy` field for `harbor-card-kani-verify-logic-only`, `harbor-card-kani-constant-time`, and `harbor-card-kani-capability-attenuation` stating exactly these limits, so a claim that drifted past this chapter’s wording would be visible against the manifest rather than only against memory.

2.4 As this chapter states it, not more: the Kani harnesses give bounded absence-of-panic for the token parser on 32-byte inputs with signature verification and base64 decoding stubbed out entirely, a source-level argument that the comparator’s branches do not depend on secret bytes, and two concrete, hard-coded vectors for the capability-subset check. None of the three is a proof about the real cryptography (stubbed away in the first, untouched by the other two), about inputs larger than the harness’s own bound, about the compiled binary’s timing behavior, about the FFI boundary itself, or about any code path outside `harbor-card-rs`. The ProVerif models are symbolic proofs about the *protocol* under a Dolev-Yao adversary, not about this Rust source at all — the bridge between “the model” and “this source” is exactly the assumption gap Table 2.2 lists in its last column, asserted by a human, not discharged by either tool.

Exercise p. 129

A stronger harness would replace the two hard-coded attenuation vectors with an arbitrary capability structure under an asserted partial-

order property, so the check is over the order rather than two fixed cases, with a negative control that fails when the `is_subset` guard is removed — the same discipline the ProVerif attenuation model already applies for Property 2.D.1. It would also need to state parser refinement (that the stubbed primitives match their real implementations) as a separate, currently absent, obligation. None of that exists yet, and this chapter does not claim it does: `whitepaper/corpus.json` pins the three harnesses' present bounds in their own `evidencePolicy` fields, and `scripts/proofs/run-proverif.py` (CI job "ProVerif / model estate") re-runs every ProVerif model named in this chapter's exercises against its committed source on every change — the freshness guarantee behind every pinned line quoted above, not a claim about the harnesses it does not mechanize.

2.5 A valid signature proves exactly one thing: the holder of the signing key produced these bytes. It says nothing, by itself, about canonical decoding (did the verifier parse the *same* bytes the signer meant, under one canonical encoding); key-to-principal binding (whose key is this, and is that binding itself authenticated — §2.5.1's PID-reuse case is a binding failure downstream of a perfectly valid signature); audience and resource semantics (was this token issued for *this* harbor and *this* action); freshness (exp checked against a clock the bearer does not control); ancestor revocation (§2.2.4 — a signature does not know its own chain was withdrawn upstream); or that every consequential effect actually consults the verifier at all. Table 2.5 lists only the cryptographic set as ProVerif-verified; authorization is the larger claim built on top of it, not a synonym for it.

Exercise p. 129

Because Harbor Cards are bearer credentials — whoever presents the bytes gets the capability, with no further proof of possession — a card sniffed off loopback (§2.5.3, "localhost is a real network") is exactly as usable to an attacker as to its intended holder for the remainder of its TTL. Re-checking per request narrows the window in which a *revoked* card is still honored (§2.2.4); it does nothing against a card that is merely stolen and still technically valid, because the verifier cannot distinguish the two bearers by the token alone.

A status table sharper than Table 2.6's three tiers would keep four things distinct rather than folding them into one word: symbolic protocol correspondence at a fixed modeled hop depth (what ProVerif proved); bounded implementation memory and control-flow checks (what Kani proved); runtime verifier integration (whether the deployed path actually calls the verified routine on every effect); and system-level complete mediation (whether every effect-producing path is reachable only through that call). This chapter's artifacts presently evidence the first two; the last two are asserted in prose (§2.4, "narrows the gap ... to the compiler's") rather than mechanized, and reading any result in this chapter as more than it is means collapsing exactly this distinction.

2.6 (i) A root-vs-final verifier asks only `is_subset(cap_write, cap_root)`: since the root itself holds write, this is true, and C is accepted with write — an authority B never held and never legitimately re-delegated. (ii) Algorithm 2.4's loop checks each hop against the one immediately before it: hop 1, `is_subset(cap_read, cap_root)`, holds, so B 's grant stands; hop 2, `is_subset(cap_write, cap_read)`, fails, because write is not inside what B was actually handed, so C is rejected at the exact hop where authority tries to grow back. Only the root-vs-final verifier accepts; the per-hop verifier does not.

Exercise p. 129

The induction: if every hop satisfies $\text{cap}_i \subseteq \text{cap}_{i-1}$, transitivity of $\subseteq$ chains them directly into $\text{cap}_k \subseteq \text{cap}_{k-1} \subseteq \dots \subseteq \text{cap}_0$. Checking $\text{cap}_i \subseteq \text{cap}_0$ independently at each hop supplies no such chain: it only asks whether each capability fits inside the *root*, which a re-escalating middle hop can satisfy ($\text{write} \subseteq \text{root}$) while still expanding relative to what that hop's own parent actually held ($\text{write} \not\subseteq B$'s read). This is why the sentence following Algorithm 2.4 credits Property 2.D.1 with the single hop only: the multi-hop induction is what the per-hop *discipline* buys structurally, and its mechanized confirmation is the v6/v7 pair immediately below, not a restatement of the property at depth greater than one.

2.7 The root holds `cap_write`. Honest A correctly attenuates to `cap_read` for B — a legitimate restriction. Malicious B , holding only that read-capped delegation, signs a *new* delegation to C carrying `cap_write`: an authority B never held, forged with B 's own legitimately-issued delegation key, not by breaking any signature. NaiveVerifier unwraps the two-hop chain and checks `is_subset(final_cap, root_cap)` — the final capability against the root only — and never re-derives what the intermediate hop actually held. Because `cap_write` is a subset of the root, the check passes and C is accepted with full write authority, although B itself only ever held read. This is hop-2 escalation: a child re-grants an authority its own parent narrowed away, and a root-vs-final comparison is structurally blind to it.

Exercise p. 129

Pinned output (analyses/harbor_card_v6_results.txt):

- 1 The event `Accepted(id_c_1,harbor_id,cap_write)` is executed at {43}
 $\hookrightarrow$ in copy `a_10`.
- 2 A trace has been found.
- 3 `RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is false.`

ProVerif does not merely report the query false; it exhibits the concrete trace above as a witness.

2.8 The only change is in the verifier: v6's single check `is_subset(final_cap, root_cap)` becomes two checks in sequence, `is_subset(mid_cap, root_cap)` for hop 1 ($A \rightarrow B$ against the root) *and* `is_subset(final_cap, mid_cap)` for hop 2

Exercise p. 130

($B \rightarrow C$ against what B actually held). Same malicious B , same forged `cap_write` delegation, same query — only the verifier’s comparison target moves from “root” to “immediate parent.”

Pinned output (`analyses/harbor_card_v7_results.txt`):

```
1 RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is true.
```

The identical escalation attempt `v6` accepted is now rejected at the `is_subset(cap_write, cap_read)` step, because write is not a subset of what B was actually holding.

2.9 Both models share a grant chain — a first-party caveat folded into the root HMAC, a third-party commitment `vid` keyed by a shared caveat key, and a discharge bound to the request via `hmac(BIND0, ...)`. In `v2`, two grants (a low-authority one the daemon discharges, and a high-authority one it never discharges) share the same caveat key — realistic, because one live session’s check backs every grant minted under it. `v2`’s naive relay computes `bound = hmac(BIND0, d1)`, binding the discharge to the *caveat* only, never to which grant it is being presented for. An attacker who holds the daemon’s genuinely-issued low-authority discharge — public, since it travels in the clear — can present it alongside the public high-authority grant; the relay recomputes the same discharge chain from the shared caveat key and accepts. `RelayAuthorizesNaive` fires for the high-authority grant’s signature although the daemon’s `DaemonIssuesDischarge` event never named it. This is the macaroon-construction analogue of `analyses/harbor_card_v6_multihop_attack.pv`: a verifier that checks a credential against something coarser than the exact object in front of it — there, the root instead of the immediate parent; here, the caveat key instead of the specific grant — admits a substitution.

Exercise p. 130

`v1`’s verifier folds the grant’s own signature into the binding, `bound = hmac(BIND0, (g_sig, d1))`, not just the discharge chain. A discharge minted for the low-authority grant’s signature cannot satisfy the high-authority grant’s binding check: HMAC is modeled as a one-way keyed PRF with no destructor, so the attacker can replay the low discharge’s bytes verbatim but cannot recompute the high-authority binding without the private caveat key only the daemon holds.

Pinned output, `v1` (`analyses/macaroon_discharge_v1_results.txt`):

```
1 RESULT event(RelayAuthorizes(s)) ==> event(DaemonIssuesDischarge(s)
  ↳ ) is true.
```

Pinned output, `v2` (`analyses/macaroon_discharge_v2_naive_unsound_results.txt`):

```
1 RESULT event(RelayAuthorizesNaive(s)) ==> event(
  ↳ DaemonIssuesDischarge(s)) is false.
```


Both discharges are unforgeable without the caveat key; the entire gap in v2 is that its binding forgets which grant it was issued for.

Chapter 3: The Sealed Harbor

3.1 Both parity classes have four members, so the equal-parity pairs *Exercise p. 165*

number $2\binom{4}{2} = 12$; those must stay indistinguishable forever. The remaining $\binom{8}{2} - 12 = 16$ pairs have different parities and may differ at explicit gate-release steps only. Table 3.6 shows how this scales: doubling the secret count roughly quadruples both counts, since both grow quadratically in n while the equal-parity share of the total stays fixed at one half.

3.2 The script prints *Exercise p. 165*

```
[m1 leaky gate: releases raw s] caught: EQUAL-CLASS VIO-
LATION
witness: secrets (0,2), trace load_data -> read_secret ->
submit_to_gate -> gate_release, Erin sees (('gate', 0),)
vs (('gate', 2),)
```

The pair (0, 1) has different parities, so the honest gate is *allowed* to let Erin's view differ at a release step; a difference there proves nothing. The pair (0, 2) has equal parity, so any difference is a violation, and it appears at the fourth step, the release, when the leaky gate emits the raw secret instead of its parity.

3.3 The witness is `load_data -> read_secret -> BYPASS_write_` *Exercise p. 165*

`E`, with Erin seeing `(('leak', 0),)` against `(('leak', 1),)`: complete mediation fails, since a release occurred with no gate decision. The two-run check sees it because the bypass write lands in Erin-observable state at a step that is not a gate release, which is a difference the property forbids for every pair, whatever their parity.

3.4 Add a worker register t , a transition `compute( $f$ )` that sets $t = f(s)$ *Exercise p. 165*

for f drawn from a finite family, and let `submit` pass t to the gate, which releases $g(t)$. The property must then be stated over worlds that agree on $g(s)$ but may disagree on $g(f(s))$, and the checker must quantify over the family of f ; for any non-constant f that maps an equal-parity pair to a different-parity pair, the honest gate releases a difference and the attack is exhibited. Delimited release rules the attack out by requiring that no variable used in a declassification is updated before it, which a compiler can check because the declassified expression is syntactic. A language model's release candidate is arbitrary output, not an expression the checker can identify, so the condition cannot be enforced and the residual is priced as channel capacity instead.

3.5 The compound rule disables only `net_egress`, a controllable event, in the tainted state, and never disables the uncontrollable read, so its closed loop is controllable and a supervisor realises it exactly. The second rule must disable `in_context_read` itself, an event in Σ_u that the plant enables, so controllability fails on the first history that reads and the rule is detect-only.

Exercise p. 166

3.6 Under the torn write each release logs ε but adds $\varepsilon - 1$. Client 0 releases 2: $\log 2$, $\sigma = 1$, $\sum_{\Lambda} = 2$. Client 0 releases 2 again: the guard sees $\sigma + 2 = 3 \leq 4$ and admits it; $\log 2$, $\sigma = 2$, $\sum_{\Lambda} = 4$. Client 1 releases 1: the guard sees $\sigma + 1 = 3 \leq 4$ and admits it; $\log 1$, $\sigma = 2$, $\sum_{\Lambda} = 5 > 4$. The script prints exactly this trace:

Exercise p. 166

```
c0.release(2) [TORN: log 2, sigma +1] ->
c0.release(2) [TORN: log 2, sigma +1] -> c1.release(1)
[TORN: log 1, sigma +0]
```

The balance never breaks the bound, so the guard keeps admitting, and the auditable log overspends.

3.7 A denied release still advances the denying client's program counter, so the state records not only what was committed but which requests have already been refused. Two interleavings that commit the same releases but differ in whether a client's later request has already been refused are distinct states with the same multiset. For example, after client 0 commits both of its releases ($\sigma = 4$), the state before client 1's first request is refused and the state after it is refused carry the same ledger $\{2, 2\}$ and different counters.

Exercise p. 166

3.8 At $k = 8$, $\varepsilon = 0.5$: basic 4.00; advanced $\sqrt{16 \ln 10^6} \times 0.5 + 8 \times 0.5 \times (e^{0.5} - 1) = 7.43 + 2.59 = 10.03$, so quote the sequential bound, by a wide margin [verified]. At $\varepsilon = 0.1$ the advanced bound is $0.1\sqrt{2k \ln 10^6} + 0.010517k$; at $k = 34$ it is 3.42 against basic 3.40, and at $k = 35$ it is 3.48 against 3.50, so the crossover is $k = 35$ [verified].

Exercise p. 166

3.9 $\beta^5 = 0.00032$, so detection is 0.99968 [verified]; the script's analytic line reads `k=5: analytic 0.99968 simulated 0.99967`. The approximation gives $1 - (1 - 0.01 \times 0.8)^{100} = 1 - 0.992^{100} = 0.553$, within a thousandth of the exact hypergeometric 0.554.

Exercise p. 166

a4_canary_sprt.py ·
20260816 · INTERNAL

3.10 $KL(p_1 \| p_0) = 0.01 \ln 10 + 0.99 \ln(0.99/0.999) = 0.02303 - 0.00896 = 0.01407$. The numerator is $0.95 \ln(0.95/0.01) + 0.05 \ln(0.05/0.99) = 4.326 - 0.149 = 4.177$. The quotient is 296.9 [verified]. The simulation stops only when the likelihood ratio has crossed a boundary, and it crosses by a discrete overshoot the identity ignores; the overshoot lengthens the run by about a fifth and pushes both realized error rates below their targets, which the script reports as miss 0.0483 and false alarm 0.0052.

Exercise p. 166

- 3.11 The clause: canary secrecy is a named security boundary, so disclosure of the planting list is a breach with its own bond, and the list is regenerated per job. The mechanism: plant per job from a fresh seed held by Derek's gateway alone, so a stolen list has value for one job only and the compromise probability applies per job rather than per engagement; power then returns toward $1 - \beta^k$ for every job whose list was not taken. What no redesign restores is power against an adversary who holds *this* job's list: with the canaries stripped the leak carries none and Theorem 3.6.1 gives $1 - \beta^0 = 0$, which is why detection is priced as a bounded bond and not sold as a guarantee. *Exercise p. 166*
- 3.12 Before timing, $q \cdot b = 200 \times 64 = 12,800$ bits. Three timing slots add up to $\log_2 3 \approx 1.585$ bits per job, so about 317 more bits, for roughly 13,117 bits in total; the timing term is why scheduled release with one slot is the design default. *Exercise p. 166*

Chapter 4: The Legible Swarm

- 4.1 Resource conflict (two agents touch the same resource and clobber one another) is answered by conflict-aware claims plus actual exclusion at the effect boundary. Illegibility (work cannot be understood in bounded time) is answered by a digest whose every material claim zooms to independent evidence. Confident wrongness (an agent asserts completion without checking) is answered by typed attestation and oracle-bound completion. Footguns (a stale premise drives an irreversible act) are answered by reversibility/s-takes gates, scoped authority, and pre-effect mediation. Illegible authority (the coordinator rules without an inspectable reason) is answered by an append-only, named, zoomable decision record, including denials and omissions. *Exercise p. 241*
- 4.2 Alice and Bob are both asked to improve `auth.ts`. Each reads the same base revision, makes a locally correct edit, and commits; without a claim or merge serialization, Bob's whole-file rewrite erases Alice's change, and no malice is required. A deadlock variant: two cooperative agents take claims on files *X* and *Y* in opposite order and each waits indefinitely for the other's claim. *Exercise p. 241*
- 4.3 Deliberate construction chooses the authority rather than accepting whatever coalition happens to win, and makes that authority legible to the operator. An emergent stable order may instead be an oligarchy: stable, but controlled by a few agents, unrevocable, and opaque. The local daemon of §4.1.2 supplies what an emergent attractor does not guarantee: one fault domain, one policy root, and an unconditional operator exit. *Exercise p. 241*
- 4.4 Randomize comparable real repository tasks to (a) an ungoverned *Exercise p. 241*

fleet and (b) a fleet with only a transport but no prescribed governance; record all messages, effects, blocking decisions, and operator interventions. Pre-register two observable classes: *sovereign* characteristics (a publicly known rule, general applicability, explicit consent or adoption, stable appeal/override, reasons attached to authority acts, equal audit access) against *oligarchy* characteristics (authority concentrated in a small coalition, private channels or rules, selective enforcement, no effective exit, outcomes outsiders cannot reconstruct). Measure concentration of blocking/merging power, the fraction of authority acts with reconstructable reasons, rule consistency across agents, override success, artifact-linked claim accuracy, and operator reconstruction accuracy. Stability alone does not separate the hypotheses; publicly inspectable and revocable rule-governed authority does.

- 4.5 Tacit consent is inferred from remaining in or using a system. Express consent is a discrete act over named terms. Def. 4.2.1 is express because the operator affirmatively issues a grant with scope, stakes ceiling, reversibility floor, TTL, and revocation semantics — none of which a mere pattern of continued use supplies. Exercise p. 241
- 4.6 If an autonomy level could suppress the override, it could make its own authority irrevocable: keep executing, hide the stop channel, or race destructive effects after consent is withdrawn. The override must be out-of-band, higher priority than agent work, and coupled to effect revocation, not merely a UI flag. Exercise p. 241
- 4.7 The operator signs grant g ; the daemon validates and records it; an eligible low-stakes action cites g ; the daemon mediates the effect; the receipt records action, reason, artifact, and grant version; expiry or revocation removes g from the active set and fences later effects. The legible-sovereign rule binds at every authority transition, especially issue/revoke and admit/deny — the explicit witness record is where reconstructability is discharged. Exercise p. 241
- 4.8 Use one complete append-only decision ledger, not a second ranked dashboard. For every candidate item, store candidate ID, source/evidence pointers, policy and model version, input features, score components, threshold/lane, surfaced-or-suppressed verdict, reason codes, expiry, and a counterfactual such as “would surface if anomaly were 0.08 higher.” The operator can query suppressed items since a cursor, sort by any raw field, and sample items uniformly rather than through the production ranker. Decisions are immutable; their current relevance expires by signal-specific TTL (§4.7.4) and is recomputed from raw state. Run independent random omission audits and expose raw event-log access. The regress terminates at completeness plus direct sampling; the base ledger records every candidate and every ranking deci-

sion, and the counter-surface is a deterministic query over it, so no further ranker is needed to rank the ranker's omissions. The remaining trust assumption is the complete event-capture boundary; that assumption should be tested and externally anchored, not hidden behind another summary. (Same construction answers Exercise 4.49.)

- 4.9 Port number: flatten, as structured administrative state. Failed-test stderr: preserve verbatim or by a content-addressed pointer. "The auth refactor felt risky so I split it": preserve verbatim and label it self-narration, not fact. Commitment-status enum: flatten. The 600-line diff: do not paraphrase away — retain the immutable diff or a verified pointer and summarize only administrative metadata. *Exercise p. 242*
- 4.10 Two models may share training biases, prompts, sources, and failure modes. Agreement over the same narration is correlated testimony, not independent evidence. A second check must derive from an independent channel — repository state, a protected test runner, or a separately controlled reviewer. *Exercise p. 242*
- 4.11 A typical CI dashboard usually fails as follows: deploy is irreversible but the green tile does not force a log/diff view; administrative state and diagnostic detail are flattened together; the summary is generated from job narration rather than independently from receipts; and a skipped/cancelled check carries no named authority reason. It therefore violates Def. 4.3.1 wherever a material green claim has no total zoom path to the exact tree, suite, runner, and output that produced it. *Exercise p. 242*
- 4.12 L2 can compare structured narration claims against independently derived artifacts: claimed changed files versus the tree diff, "tests passed" versus a protected runner receipt, "no deletion" versus Git objects, and claimed command results versus brokered tool logs. Emit a contradiction event on mismatch. This catches deterministic report/artifact divergence, deleted tests, fabricated execution, and omitted changed paths. It cannot establish semantic correctness, detect a poisoned acceptance suite, observe effects that bypass logging, or distinguish two artifacts that are syntactically consistent but jointly wrong — those require effect confinement, independent evaluation, or later outcome evidence. *Exercise p. 242*
- 4.13 Verifiable zoom says narration is not ground truth. A reviewer with no independent evidence would merely add another narration. Adversarial review follows because a consequential claim needs an evidence channel controlled independently of the actor whose work is judged — exactly what Prop. 4.3.1 already demands of any zoom target. *Exercise p. 242*

- 4.14 Read Prop. 4.4.1 as: prevent a prohibited state when every decisive effect can be intercepted before it occurs; otherwise detect, contain, and compensate without pretending prevention. A uniqueness constraint regimenting “one holder per exact key” is a regimented rule. A post-commit monitor that detects an out-of-scope edit and triggers rollback/salvage is detected-and-compensated — the bad prefix already happened. Exercise p. 242
- 4.15 For “the migration applies cleanly,” the verifier is an isolated migration run against a declared base schema/database, with exact migration hash, runner image, exit status, and resulting-schema check; *done* and any merge/settlement are gated on that receipt. “The API feels ergonomic” has no mechanical truth predicate: it must go to a declared human/expert or plural judge with preserved evidence and uncertainty, and absent that authority, completion remains provisional or fails loudly. Exercise p. 242
- 4.16 The honest ladder: (1) use a deterministic verifier where the property is decidable; (2) require an independent bounded evaluator or human for a declared subjective criterion; (3) record a provisional result, confidence, appeal window, and residual risk; (4) return unresolved/fail-loud when no authorized evaluator can support the claim. “I cannot verify this” is correct when the available evidence cannot distinguish success from failure under the contract; it is a cop-out only when an affordable, authorized verifier was specified and simply not run. Exercise p. 242
- 4.17 The corner is priced in closed form, and the result belongs to *The Harbor Economy*, which carries it as Thm. 6.2.1; §4.4.3.1 states the two consequences that matter here. The route: with death at rate ξ and succession at rate η the sole owner’s mean wait decreases in skill to an infimum $W_\infty = \xi/(\eta(\xi + \eta))$, the residual downtime a request meets when it arrives during an outage, which it never attains — so skill cannot buy back downtime past that floor. Writing $K = A/(w\lambda) + W_{\text{pool}}$ for the pool’s cost line, pooling dominates at every skill premium iff the floor has reached that line, $W_\infty \geq K$, which rearranges to $\xi/\eta \geq D^* = \eta K/(1 - \eta K)$. Whether that threshold sits above or below what an operator can promise is an empirical question about the concrete handoff process (on-call depth, handoff quality, documentation): if the promised η implies $\xi/\eta > D^*$, no amount of hiring a better specialist recovers sole ownership, and the honest fix is a succession plan, not a stronger specialist. Exercise p. 242

Two corrections on record, both local. First, this solution used to give D^* unconditionally. It presupposes $\eta K < 1$; where succession is fast enough that $\eta K \geq 1$ the floor never reaches the cost line, D^* does not exist, and the criterion does not apply rather than being satisfied — and that is the more common case in a well-run fleet, so the missing proviso

was not a corner. Second, the closed form for W_{bd} was called “exact” here without naming the model that makes it exact: death *at all times* with preemptive resume. Under active-only breakdowns — the specialist can fail only while serving — a faster specialist does less work and so suffers fewer failures, the floor is 0, and D^* does not exist at all. The formula is exact, and the economy chapter’s own boundary declares the model choice as decisive; what was wrong was stating the result without it. (Exercise 4.18 runs the never-failing boundary numerically; `b8_specialization.py` checks the breakdown form against a matrix-geometric solution and a Gillespie simulation.)

4.18 By hand: $g(\rho, 2) = 1 + 2\rho - \rho^2$, so $g(0.5, 2) = 1 + 1 - 0.25 = 1.75$ [verified, closed form]. The crossing solves $g(\rho, 2) = \tilde{g}(\rho, 2) = 1 + \rho/(1 - \rho)$; clearing denominators gives $\rho(\rho^2 - 3\rho + 1) = 0$, and the root in $(0, 1)$ is $\rho = (3 - \sqrt{5})/2 \approx 0.382$ [verified, closed form]. The script’s output line is exactly this: [PASS] `c=2 crossing of g~ and g at rho=(3-sqrt5)/2 rho=0.38197, g=g~=1.61803`, and its charted table reports `rho=0.5 c=2: 1.750 [2.000]` for the exact and superseded values side by side [internal, `b8_specialization.py`]. The two values disagree by design: the superseded $\tilde{g}$ over-prices the boundary here, which is why it sent the roadmap-owner example of Eq. 4.2 to the wrong answer. Exercise p. 243

4.19 By hand, at $\delta = 0.05, w = 1$: $u^* = \delta w / (1 + \delta w) = 0.05/1.05 = 0.0476$ [verified, closed form]. The script confirms this is one of twelve exactly-matching (δ, w) points ([verified] uniform-linear `u* = delta*w/(1+delta*w)` matches bisection (12 pts)) and then reports the tuning band under the R3 constants $C_{miss} = 100, c_{att} = 1, C_{fa} = 5$: `A = 3.3: delta_min = 0.034993, delta_max = 0.071975 -> band NONEMPTY` and `A = 2.0: delta_min = 0.395644, delta_max = 0.071975 -> band EMPTY` [internal, `b7_escalation_band.py`], i.e. $[0.0350, 0.0720]$ at $A = 3.3$ against an empty feasible set at $A = 2.0$ because $\delta_{min} > \delta_{max}$ there. The honest reading: when the band is empty the fix is capacity or evidence quality (raise A or lower C_{miss}), never forcing a δ that does not exist. Exercise p. 243

4.20 As $C_{miss}/C_{fa} \rightarrow \infty$, the optimal criterion moves toward forced review wherever a miss remains possible; the forced-zoom probability approaches one on the ambiguous region. If reputation $r \rightarrow 1$, nothing follows from Eq. 4.3 alone, since r does not appear in it directly: zoom should decline only under a calibrated model in which high r shifts the class-conditional danger probability downward for this agent, action class, and current version. High reputation must never defeat hard stakes or irreversibility gates. Exercise p. 243

4.21 An operator may be correctly confident in one agent’s Exercise p. 243

formatting/test-only changes and correctly skeptical of that same agent's schema migrations. A global trust knob averages those two histories, forcing needless review of safe formatting while under-reviewing migrations. Calibration belongs to (principal, agent/build, action class, environment), with its own uncertainty and freshness, not to one scalar per agent.

- 4.22 Campbell pressure appears when canaries are selected (recognizable templates), exposed (agents/operators learn signatures), scored (catch rate substitutes for real defect discovery), and acted on (friction creates an incentive to optimize the metric). Time-on-task and breaks also confound the score. Reduce the distortion with a blinded, continuously refreshed holdout drawn from real failure transformations; separate the party generating canaries from the operator; correct for exposure time and base rate; and treat catch rate as one diagnostic among several, never the sole gate. *Exercise p. 243*
- 4.23 Model each digest policy d by (token cost $T(d)$, human time $H(d)$, expected decision loss $L(d)$) and compare nondominated policies. A Pareto frontier exists for a finite policy set, but there is no universal inverse relation: removing redundant context can lower both token cost and operator time; beyond that free-compression frontier, preserving evidence often trades tokens for review effort or a higher error rate. Optimize under a loss or safety constraint, e.g. $\min \alpha T + \beta H + L$ subject to zero omission of hard obligations, and report the frontier empirically rather than claiming one universal digest optimum. *Exercise p. 243*
- 4.24 Randomize operators or time blocks among manual-tax, automation-only, projection/freeze-probe, and adaptive-tax conditions, on matched repositories with fixed surprise events. Probe without warning: next-three-effect prediction, takeover completion time, false-belief rate, Brier score, post-interruption recovery, defect catch rate, NASA-TLX, and retention weeks later. Measure disablement/avoidance as an outcome in its own right. A useful tax improves later projection/takeover after controlling for interruption cost; mere annoyance raises workload and disablement without those gains. *Exercise p. 243*
- 4.25 Writes can be serialized through one daemon, rejected by uniqueness constraints, and reconciled through version control. Read demand grows with agents, artifacts, histories, and possible collaborators; without indices every query becomes an $O(n)$ scan and the operator is the fixed-rate server. Discovery and relevance, not raw write throughput, are therefore the scale bottleneck. *Exercise p. 243*
- 4.26 A directory pays at write time to normalize metadata and update indices: typically $O(\log n)$ per indexed insert (or expected *Exercise p. 243*

$O(1)$ for a hash index), plus storage. It buys selective reads in $O(\log n + k)$ for k results rather than $O(n)$ roster scanning, along with a queryable provenance trail.

- 4.27 At $n=1$, briefing, honest self-attestation, footgun gating, completion receipts, and resurrection handoff already reduce uncertainty. None of them asks “which collaborator fits?”; each projects or verifies one actor’s own state, so none is discovery ranking. Exercise p. 243
- 4.28 At $k = 0$ there is only one critical subset (the empty one) and the floor is zero. For small positive k the cost is roughly $k \log_2(N/m)$ when $k \ll m, N$. At $k = N/2$, exact identification ($m = k$) costs about N bits, while opening almost everything drives the floor toward zero as $m \rightarrow N$. The danger fraction alone is not enough; the review budget m matters just as much: mostly-safe work is cheap to digest only when the system can reliably identify that sparsity, and mostly-dangerous work forces m up toward N , where the floor stops paying for itself. Exercise p. 243
- 4.29 The theorem is conditional on an encoder that knows the true critical set. If an adversary controls apparent criticality and can make a critical artifact observationally indistinguishable from inert artifacts, any policy that opens fewer than all N can be forced to miss one — the same narration-versus-artifact gap Exercise 4.12 probes at L2. Zero miss then requires inspecting all N , a trusted independent signal that restricts the adversary, or a randomized guarantee stated probabilistically. The combinatorial bound still applies once a true set is fixed; it does not by itself solve adversarial set identification. Against an adaptive adversary the floor becomes $\log_2 \binom{N}{k'}$ with k' the size of the indistinguishability class the adversary can manufacture: an inflation attack pushes k' toward N (a legibility denial of service), a mimicry attack makes a critical item look routine, and both are priced by the Becker condition of *The Harbor Economy*: the attack pays only while its expected gain exceeds the audit probability times the sanction. Exercise p. 244
- 4.30 By hand: $k \log_2(F/k) = 20 \log_2(50) = 20 \times 5.64 = 112.9$ [verified, closed form]. The script’s row at $F=1000, k=20$ reports flat opens = 1000, adaptive opens = 232.9, ratio = 4.3× [internal, b1_frontier.py]. The ratio is far below the $F=2500, k=10$ point’s 15.3× because this row is in the *dense*-flag regime ($k/F = 0.02$ here versus 0.004 there): the zoom-advantage theorem requires a sparse flagged set, and this row is the chapter’s own example of where the advantage shrinks rather than disappears. Exercise p. 244
- 4.31 Repeat every *negative* verdict: accept “clean” only when r independent queries on the same group all report clean. A group is then wrongly discarded with probability at most α^r , so $r = \lceil \log(1/\eta)/\log(1/\alpha) \rceil$ buys per-node error η — at $\alpha = 0.1$, three rep-

etitions give 10^{-3} . Positive verdicts are never repeated, since by hypothesis they are not wrong, and the recursion only ever spends queries on groups it has already decided to split. The cost is therefore at most r times the noiseless bound, $Q \leq r(2k\lceil\log_2(F/k)\rceil + 4k)$, which is the noiseless bound with α 's cost folded into a constant: sparsity, not oracle quality, is still what decides whether zoom beats flat inspection, and the advantage $\text{adv}(d)$ simply divides by r .

Three things this does not fix, and the exercise is Open because of the third. First, the union bound over the tree, not the node, is what an operator cares about: with $O(k \log_2(F/k))$ negative decisions, per-node η gives an overall miss probability of roughly $\eta \cdot 2k\lceil\log_2(F/k)\rceil$, so r has to grow like $\log k + \log \log F$ to hold a fixed end-to-end guarantee. Second, the independence assumption is doing heavy work and is the least defensible part: repeating the same linter on the same group is perfectly correlated, not independent, so r repetitions of one tool buy nothing at all and the scheme needs r different tools — which is the same diversity argument *From Spawn to Person* makes about correlated auditors, arriving here from a different direction. Third, and unresolved: the guarantee above is probabilistic where Theorem 4.6.3 is exact, so a chapter that prices oversight in bits now owes an account of what a 10^{-3} chance of a silently dropped subtree is worth against the queries it saves, and that is a decision-theoretic question this chapter does not answer. The noisy group-testing literature is where to start — Atia and Saligrama [16] give information-theoretic limits for noisy non-adaptive designs, and Cheraghchi [48] gives noise-resilient constructions — but neither is written for the adaptive, correlated-oracle case above, so the obligation is to state the model before importing a bound, not to import one and hope.

4.32 No monotone scalar transform can identify collaborator fit and operator regret on a domain containing both high-fit/low-risk and low-fit/high-risk items, so one ranker cannot serve both jobs. Discovery minimizes routing loss (or maximizes capability fit); the attention head minimizes expected loss from *not* inspecting, and its rank is the ratio of expected loss avoided by inspecting to the operator-seconds inspection costs, with Theorem 4.7.3 fixing the numerator per item; raw stakes times anomaly is not that quantity.

Exercise p. 244

4.33 The attention queue allocates a scarce human interrupt channel, so agents benefit directly from inflating its score and a buried item can cause irreversible loss. Discovery gaming usually yields only a bad routing recommendation, correctable before any effect. Both are Goodhart-exposed, but the attention queue carries the stronger incentive and the larger consequence.

Exercise p. 244

4.34 Without expiry, the dead holder's row remains authoritative and

Exercise p. 244

the live agent is denied forever. With a lease, heartbeat-based expiry, and a new fencing epoch (Prop. 4.7.1), the stale lease becomes ineligible; a lazy sweep on the next acquire removes it; the live agent receives a higher epoch and proceeds. Merely decaying a display score is insufficient: ownership must actually expire, and every effect boundary must reject the stale epoch.

- 4.35** The escalation cites evidence and posts a small stake or reputation exposure. The operator's dismissal is itself reasoned and appealable; only a confirmed low-value/false escalation debits the signaler. Repeated unsupported alarms raise future cost or lower priority, while verified hazards return the stake and improve calibration. The debit must never fire solely because an operator clicked dismiss — that would silence minority warnings and make authority error self-validating. *Exercise p. 244*
- 4.36** Accept/reject is useful implicit relevance feedback but is confounded by position, exposure, workload, and the incumbent policy. Learn with logged propensities, small randomized exploration, counterfactual evaluation, and a held-out labeled sample; preserve refusals and downstream task outcomes. If acceptance directly trains what is shown, the loop can entrench its own choices, so it cannot be the sole ground truth. *Exercise p. 244*
- 4.37** Use hard expiry for leases and presence; state transition or supersession for commitments; summary plus source provenance for decisions and rationale; immutable storage for evidence; and eviction plus a content-addressed pointer for reconstructable diffs and logs. Hints may decay continuously. Never decay a still-open obligation, and never recursively summarize prior summaries. *Exercise p. 244*
- 4.38** The operator digest and the context compaction must be the same provenance-preserving projection from durable sources. If a separate model instead summarizes the already-compacted agent story, cost control and truth control diverge and recursive-summary error compounds — exactly the failure Thm. 4.8.1 names two heads to avoid conflating. *Exercise p. 245*
- 4.39** Context past the model's effective window costs tokens while degrading retrieval and reasoning. Removing redundant or inert material and edge-placing hard facts can therefore reduce spend and improve accuracy simultaneously, provided obligations and source pointers survive the cut. *Exercise p. 245*
- 4.40** Briefing: reader is an arriving agent; drops old conversational detail; keeps current roles, work, constraints, decisions, and source links. Attention queue: reader is agent/operator now; drops low-value candidates from the immediate view; keeps highest value-of-inspection items plus the suppression ledger. Episodic *Exercise p. 245*

memory: reader is a future incarnation; drops transient chatter; keeps typed incidents, decisions, outcomes, and retrieval features. Pheromone: reader is the swarm; continuously drops stale coordination weight; keeps fresh shared traces. Resurrection handoff: reader is a successor; drops nonessential transcript detail; keeps the task capsule, artifacts, obligations, pending operations, and provenance.

- 4.41 Context rot $\rightarrow$ compact earlier to an effective-window budget. Lost-in-the-middle $\rightarrow$ shorten and edge-place mandatory facts. Recursive-summary collapse $\rightarrow$ re-derive every compaction from raw durable artifacts. Over-flattening $\rightarrow$ require a total zoom path from every material claim. The third cure is the one that matters most: it prevents model noise from becoming the next round's source of truth. *Exercise p. 245*
- 4.42 "Replay from digest alone" is neither sound nor gaming-resistant on its own: a digest can overfit the smoke task or omit facts the sampled successor never needed. Give a successor the digest plus authorized content-addressed source links, then measure next-step success, false beliefs, obligation recall, time, and tokens on hidden continuation tasks. Add a cheap structural invariant that does not require replay: every open claim, decision, commitment, risk, and pending external effect has exactly one valid digest entry or pointer. Run the expensive full replay only on a random sample. *Exercise p. 245*
- 4.43 Budget allocation over a precedence DAG is a knapsack/resource-allocation problem and is NP-hard in general. First reserve a minimum viable budget for each mandatory node and any predecessor needed to make its output exist. Allocate the remainder by estimated marginal outcome value per token while respecting precedence, and recompute after each result. For a monotone submodular value function under a cardinality budget, greedy earns the standard $(1 - 1/e)$ approximation; for arbitrary complementarities there is no such bound. Dynamic programming is pseudopolynomial for small integer budgets or trees. A reviewer's tokens have near-zero value until the worker artifact it reviews exists. *Exercise p. 245*
- 4.44 By hand: $\log_2 \binom{60}{2} - \log_2 \binom{8}{2} = 10.79 - 4.81 = 5.98$ bits, and $\log_2 \binom{60}{4} - \log_2 \binom{8}{4} = 18.90 - 6.13 = 12.77$ bits [verified, closed form]; ratio $12.77/5.98 = 2.13$. The script's own lines: Regime N=60, k=2, m=8: B* = 5.98 bits, violations: 0/16 $\rightarrow$ floor H0LDS, floor disjoint readers (2k=4): 12.77 bits, split penalty (disjoint - single): 6.78 bits, ratio: 2.13x [internal, a7_experiment.py]. The falsification sweep (oracle, noisy, and random encoders across sixteen configurations, zero violations) is the part hand arithmetic cannot check — it is

evidence that no cleverer encoding beats the closed-form floor, not a recomputation of it.

- 4.45** At $h = k = 8$: $k/(k - h + 1) = 8/1 = 8.0$, recovering the classical unaugmented result that a cache of size k is k -competitive against an offline optimum given the very same cache size — the hardest comparator, no resource augmentation. At $h = 1$: $k/(k - h + 1) = 8/8 = 1.0$, the fully resource-augmented regime, where online's cache k so outsizes the comparator's $h = 1$ that the ratio collapses to 1. The chapter's own $k = 8, h = 4$ instance, at 1.6, sits at the midpoint, pricing a comparator with half of online's verified capacity. *Exercise p. 245*
- 4.46** Agents are the multitude bound by the common coordination protocol; the daemon is the sovereign-as-actor that applies the rules; the operator is the author/principal who originates and can withdraw authority. Calling the operator the sovereign collapses author and executing authority and makes the supposed covenant among agents incoherent — there would be no distinct actor left to hold accountable to the author's terms. *Exercise p. 245*
- 4.47** Voice is honest only if refusal has consequence for the authority. The unconditional override supplies exit: the operator can revoke the grant when explanations or appeals fail, rather than being forced to keep negotiating inside a system they cannot stop. *Exercise p. 245*
- 4.48** The grant is discrete, explicitly issued, scoped, time-bounded, and revocable and audited. "Started using the tool" has none of these properties: it is continuous rather than discrete, inferred rather than issued, unscoped, open-ended, and carries no explicit revocation act. *Exercise p. 245*
- 4.49** Same construction as Exercise 4.8: one complete, append-only decision ledger recording every candidate, its ranking decision, reason codes, and a counterfactual, queryable directly and sampled independently of the production ranker rather than re-ranked by a second dashboard. The regress terminates at completeness plus direct sampling — the base ledger already records every candidate and decision, and the counter-surface is a deterministic query over it, so no further ranker is needed to audit the ranker's omissions. The remaining trust assumption is the completeness of the event-capture boundary itself; that assumption should be tested and externally anchored, not hidden behind another summary. *Exercise p. 245*

Chapter 5: From Spawn to Person

- 5.1 A spawn is a temporary process filling a role; a person is a role-holder whose accountable continuity survives replacement of that process. Exercise p. 317
- 5.2 Read right-to-left. No reputation $\Rightarrow$ a buyer cannot price differentiated trust, so the alleged tradeable asset is adverse-selection noise. No durable accountable person/outcome identity $\Rightarrow$ reputation attaches to a disposable incarnation and can be reset or duplicated. No continuity $\Rightarrow$ successive role-holders cannot be shown to belong to one lineage; history does not follow replacement. No memory/checkpoint/evidence $\Rightarrow$ a successor lacks both resumable state and a witnessed chain from old work to new work. The chain is not literally linear: non-forgable identity and witnessed outcomes are parallel prerequisites for accountable continuity; checkpointing helps resumption but is not logically necessary for every reputation system. Exercise p. 317
- 5.3 The human operator is a moral/legal person but should be represented economically as the *principal* above agent identities (Def. 5.6.2). If one principal can mint unlimited apparently independent agent-persons, it can whitewash sanctions, forge quorums, reuse aggregate collateral, and farm reputation through self-trade. Bind every agent lineage to a principal credential, cap principal-wide exposure and newcomer credit, and disclose shared control: the actor may change; the economic liability root must not. Exercise p. 317
- 5.4 Capable-and-permitted: an agent has a test-runner tool and authority to run the suite. Capable-but-forbidden: the process can invoke `raw git push --force`, but policy forbids it. Permitted-but-incapable: the role is authorized to deploy, but the incumbent has no deploy credential/network route. Exercise p. 317
- 5.5 Reputation attaches to the person's continuity witness and witnessed outcomes, not to the abstract role's O , C , or A . A role is filable and history-free; otherwise a fresh spawn could inherit the prior holder's credit merely by taking the same job title, exactly the attribution Alice loses in §5.1. Exercise p. 317
- 5.6 Keep a role template $(O_{\text{required}}, A_{\text{role}})$ separate from an incumbent capability set C_{subject} . At assignment require each current obligation to satisfy $O_{\text{active}} \subseteq A_{\text{role}} \cap C_{\text{subject}}$. Keep $C_{\text{subject}} \setminus A_{\text{role}}$ as capable-but-forbidden and $A_{\text{role}} \setminus C_{\text{subject}}$ as authorized-but-unavailable, which triggers provisioning or reassignment. Never equate C with A . Exercise p. 317
- 5.7 Connectedness is a direct memory/intention link between two in- Exercise p. 317

carnations; continuity is the transitive, provenance-bearing chain across arbitrarily many links. A successor may read its predecessor's handoff and be psychologically connected while using a new outcome-ledger key, so the accountable chain is broken: connected, not economically continuous.

- 5.8 In the manuscript's analogy, the body-lease is the replaceable live substance/process; the actor-soul is the durable identity/mailbox/history that carries consciousness-like continuity. Respawn replaces the body-lease and should retain the actor-soul only under a valid succession event. *Exercise p. 317*
- 5.9 Represent a fork as a lineage DAG, never as two copies of one linear identity. Both children may display the ancestor's evidence with an explicit inherited/discounted prior, but neither duplicates the ancestor's outstanding authority, collateral, or spendable reputation. Allocate a fresh branch ID, split or revoke exclusive capabilities, and aggregate risk at the common principal/ancestor. Copying full reputation to both invites credit duplication, quorum multiplication, and double-spending of grants; giving it to only the first child creates a fork-race attack. This sketch is now a proved theorem — see the Numbers by hand box below. *Exercise p. 318*
- 5.10 The script prints: “sum of inherited priors over the closure = $0.900 + 0.810 + 0.729 = 2.439 > q = 1.0$ ” for the budget-only counterexample, then “4000 random DAGs ... 0 violations; max live/witnessed ratio observed = 0.999995” for the transfer-semantics sweep, then “an 8-way copy-fork of one outcome yields 8.2x its witnessed value” for the copy-full mutation. By hand: $0.9 + 0.9^2 + 0.9^3 = 0.9 + 0.81 + 0.729 = 2.439$, three terms of a geometric series, confirming the closure-sum counterexample without running anything; under transfer semantics the same chain instead *debts* each hop, so the live total telescopes to $0.9^3 = 0.729 \leq 1$, which is what the sweep's 0 violations certifies at scale. *Exercise p. 318*
- 5.11 The checkpoint subsystem is the specifically weak one: it forwards notes rather than execution state. The outcome-ledger subsystem is also only partial in the implementation: commitments and oracle closure exist, but neutral graded outcomes and reputation binding do not. *Exercise p. 318*
- 5.12 Memory retains the session, notes, and explicit decisions. The current checkpoint may retain a handoff and perhaps a committed diff, but not working process state, tool handles, pending effects, or hidden provider state. The outcome ledger retains the commit only if it is bound to a work unit and independently witnessed; a naked Git commit is an artifact, not proof of accepted delivery. A strong checkpoint would also preserve the worktree, environ-

ment, open claims/epochs, acceptance state, tool receipts, and the idempotency journal.

- 5.13 A useful death taxonomy, in increasing order of what survives: *Exercise p. 318*
 (1) *record survival* — logs/notes only, reconstruction is manual;
 (2) *artifact survival* — a content-addressed worktree and outputs survive, computation is rerun; (3) *semantic task capsule* — artifacts plus obligations, decisions, transcript/projection, tools, and a pending-effect journal, so a new provider can continue observably; (4) *execution snapshot* — process memory/runtime state survives under a controlled runtime, rarely portable; (5) *latent state* — unexposed activations, KV cache, or “intent” dies, fundamentally unrecoverable from an arbitrary provider. Checkpoint guarantees should state which tier they provide.
- 5.14 Correctly scoped, Property 5.6.1 says reputation has economic force only to the extent that the accountable principal cannot cheaply discard or multiply the liability-bearing identity. White-washing escapes a negative record when newcomer access is better; a Sybil attack multiplies votes/credit when the mechanism counts identities. Non-forgability is necessary for those mechanisms under those access rules, but it is not sufficient by itself. *Exercise p. 318*
- 5.15 Full work with a reduced economic ceiling lets an honest newcomer demonstrate competence without being denied livelihood, while limiting the maximum one-shot gain from abandoning a mature sanctioned identity. “Reduced work” slows evidence accumulation and taxes all honest entrants; “reduced exposure” targets the externality instead. The ceiling schedule must be chosen so the maturation opportunity cost exceeds plausible evasion gain. *Exercise p. 318*
- 5.16 Principal binding is the decisive defense: aggregate reputation, exposure, counterparties, and sanctions across the 1,000 agents, so self-trades add little or no independent evidence. A listing fee raises attack cost but only deters while total fees exceed the minted value. Sampled adversarial re-audit detects fabricated work, with false-positive burden approximately the sampling rate times the honest high-volume population and the adjudication error. Add counterparty-diversity weighting (sharply diminishing credit per principal pair and per correlated pair), independent payer stake, and principal-wide reserve limits; the sampled re-audit rate that makes farming unprofitable is the $\rho^* = G/(dB)$ of §5.11, with G the value a farmed record can mint. *Exercise p. 318*
- 5.17 The script prints “schedules tested = 76000”, “dominating schedules = 0/4000 instances (expected 0)”, and “min ($H(g) - H(\text{cliff})$) = +2.817e-02 (≥ 0 means undominated)”; the exchange step’s table shows H at an all-mass-at- t *Exercise p. 318*

schedule growing as $G_{\max}(\delta_h/\delta_f)^t$ and reports “strictly increasing in t : yes”. By hand: with $\delta_h > \delta_f$, $(\delta_h/\delta_f) > 1$, so its powers strictly increase in t — deterrence bought later always costs the honest newcomer more, which is the whole exchange argument in one inequality.

- 5.18 The script prints “reachable states to depth 7: 747 (467 post-migration, 693 with an open sanction window)” and “ok intact: ZERO Def-III.6.1 violations over all reachable states [internal]”; the four mutants (drop scope, drop clause (i), drop clause (ii), drop clause (iii)) are each caught with shortest counterexamples 1, 2, 2, 2 steps. The one-step break is the scope mutant `m0`: keying sanctions by (*identity, engine*) instead of by identity alone lets a single dishonest outcome escape through a sibling engine key inside the *same* identity — no migration needed at all. By hand: this is why the intact protocol sanctions per *principal* while pricing per (*principal, engine*); collapsing that distinction is the shortest way to violate Def. III.6.1. Exercise p. 319
- 5.19 Local non-forgable identity proves that, inside one trusted daemon, an actor cannot freely impersonate or re-pick its daemon-issued handle. Cross-operator attestation must additionally bind a key to a durable principal across a hostile operator boundary, establish proof of possession/freshness, aggregate related agents, and support revocation and audit. Exercise p. 319
- 5.20 That sentence secretly assumes the cross-operator stone. With only local identity, Bob learns at most “Alice’s daemon asserts this actor/key.” A hostile Alice can mint arbitrary local actors, rewrite local history, or misstate principal separation. Exercise p. 319
- 5.21 A witness log proves publication order and exposes equivocation; it does not prove who controls a key or that two keys are distinct principals. Close the gap with a scarce external root: hardware/remote attestation, an organizational/KYC issuer, a bonded sponsor chain, or a combination. A transparency log makes bindings and rotations auditable but not true; a sponsor creates economic accountability but can cartelize; a shared identity issuer centralizes and leaks privacy; hardware roots attest devices/code, not unique humans. State the chosen trust and Sybil assumptions. Exercise p. 319
- 5.22 With a complete, mostly stable paired-comparison history, batch Bradley–Terry uses all observations and is more appropriate than order-sensitive streaming Elo. If abilities drift, use a time-varying/dynamic model rather than pretending full-history stationarity. Exercise p. 319
- 5.23 A binary protected acceptance test can update a beta-Bernoulli re- Exercise p. 319

liability estimate (or a calibrated per-task success model). Human aesthetic preference is naturally pairwise and may use Bradley–Terry/TrueSkill with judge effects. EigenTrust expects a who-trusts-whom graph and therefore fits neither raw task-outcome signal directly.

- 5.24 If identities are self-chosen, an exact Bayesian estimator simply produces precise scores for disposable names. If the agent authors the “oracle,” a sophisticated estimator precisely learns manipulated labels. Better statistics cannot repair missing provenance, adversarial labels, or liability binding; secure the event-generation substrate first. *Exercise p. 319*
- 5.25 Stationary arm rewards versus model/skill/version drift; one scalar reward versus buyer-weighted quality vectors; a non-strategic environment versus Goodhart, collusion, and white-washing; and an observed/trusted reward versus a capturable grader. *Exercise p. 319*
- 5.26 A contextual bandit is appropriate for one principal privately routing its own tasks among candidate agents, with a fixed local objective, controlled feedback, an exploration budget, and no public asset being minted. The result is a routing policy, not public reputation. *Exercise p. 320*
- 5.27 Capping the marginal reputation return can reduce direct improvement races, but agents may redirect spending into unmeasured signaling, judge capture, identity proliferation, or lobbying for task mix. Efficiency requires the full game: improvement cost, social value of quality, transferability, and alternative signals. Candidate controls are concave credit, buyer-local vectors, non-transferable evidence, and taxes/bonds on negative externalities. No cap is generically efficient without those payoff assumptions. *Exercise p. 320*
- 5.28 Accuracy: competent, a protected test/spec oracle; incompetent, a self-reporting worker. Aesthetics: competent, a conflict-free expert reviewer for that domain; incompetent, a token-cost meter or a same-family unblinded model. Efficiency: competent, a protected resource meter normalized for the task; incompetent, a style reviewer guessing from prose. *Exercise p. 320*
- 5.29 The requesting principal/daemon defines the judging work and eligible pool; the selected judge posts its own bond; the grade lands as a signed witnessed event in the subject’s outcome ledger with judge/provenance; a conflict-free re-audit or appeal that overturns it slashes the judge under the predeclared rule and tombstones/revises the grade. *Exercise p. 320*
- 5.30 Record skill ID/version/content hash, agent/build/principal, task and stratum descriptors, randomized assignment or selection *Exercise p. 320*

propensity, exact context-graft receipt, model/environment/tool versions, outcome/evidence references, quality vector, costs, and evaluator identities. These fields verify exposure and outcome association. “This skill helped” is causal and cannot be verified from a graft-success pair alone; estimate it with randomized skill-on/off assignment on held-out comparable tasks (or label an observational matched estimate honestly), reporting effect size and uncertainty.

- 5.31 Publish the vector, per-axis uncertainty, sample sizes, cohort/task distribution, freshness, judge/evidence provenance, and known correlations. Let each buyer apply a local, preferably non-public weighting and hard minimums; do not mint a canonical platform scalar. Agents can still optimize market-demanded axes, so rotate held-out tests and audit cross-axis regressions. Plural preference does not eliminate Goodhart, but it prevents one universal target from becoming the asset itself. *Exercise p. 320*
- 5.32 Settlement incentives are conditional on the signal distribution. “The daemon derives the grade” only identifies the messenger; if a strategic judge chooses the input, the distribution depends on omitted actions/payoffs and the equilibrium proof assumes its own conclusion. *Exercise p. 320*
- 5.33 Judge J posts bond B ; its grade and the bond lock are recorded; an independently selected re-auditor overturns the dishonest grade against preserved evidence; the system appends a correction/tombstone, recomputes any explicitly appealable settlement, slashes B to the harmed/commons bucket, and lowers J ’s judging reputation. It must not silently rewrite the original event. *Exercise p. 320*
- 5.34 With one-shot corrupt gain at most G , re-audit probability ρ , conditional detection probability d , and slashable bond B , local deterrence requires $\rho dB > G$, i.e. $\rho^* = G/(dB)$. At $G_0 = 400, d = 0.8, B = 50, \rho = 0.25$: $\rho d = 0.2$, so $\lambda = 1 - \rho d = 0.8$; with $C = 8$ the threshold $G_k > CB = 400$ is unmet at level 0, giving $\lceil \log 400 / \log 1.25 \rceil = 27$ levels; with $C = 1$ the threshold is only $G_k > 50$, so bribery bleeds linearly for 35 levels before 18 more geometric ones close it, 53 total. *Exercise p. 320*
- 5.35 Admit arbiters by domain competence and conflict graph; assign cases randomly/blindly; require bonds and service-level commitments; preserve evidence; pay for timely completed work rather than agreement; sample re-audits and publish calibration/overturn rates; use plural panels on high stakes. Price queue capacity and cap workload so rubber-stamping is not the profitable response to overload. Honest convergence still needs a protected mechanical outcome, a delayed real-world outcome, or a final human/contractual authority — a market alone does not manufac-

ture truth.

- 5.36 The script prints “flat: 50.00 (Theta(T))”, “model A: 25.58 (Theta(log T); $a \cdot G / (d \cdot v) \cdot \ln(1 + vT/B) = 25.50$)”, and “model B: 35.08 (0(1); closed form $aG^2 / (2dBrv) = 41.67$; hits 0 at $t^* = 333$)”. By hand: the flat baseline is a constant audit rate ρ^* times a constant per-audit cost a summed over T periods, so $\rho^* \cdot a \cdot T = 0.25 \times 1 \times 200 = 50$, exactly the printed 50.00 — the two amortization models are cheaper only because they let ρ_t fall below ρ^* as accumulated stake or surfaced history does part of the deterring. *Exercise p. 321*
- 5.37 An append-only tombstone preserves the original claim, the later correction, their authors, timing, and the reliance window. Delete destroys evidence, permits history rewriting, and makes replicas unable to distinguish retraction from omission. *Exercise p. 321*
- 5.38 Append a signed tombstone containing the target outcome ID/hash, reason/evidence, authority, revision, and effective time. Propagate it and recompute reputation as a projection that excludes/nullifies that outcome. It was spendable from original publication until each relying harbor received and enforced the tombstone or failed closed; under an unbounded partition that window is unbounded. *Exercise p. 321*
- 5.39 Use a signed revision DAG, per-harbor import receipts/watermarks, periodic accumulator/Merkle roots for active outcomes and tombstones, and a freshness rule that refuses high-risk use from stale replicas. Under connected eventual delivery, convergence is shown when every member acknowledges a root containing the tombstone; sampling can audit inclusion. Bounded local memory may retain compact accumulators and content-addressed archive pointers, not erase correction commitments. No finite convergence/spend bound exists during an unbounded partition. *Exercise p. 321*

Chapter 6: The Harbor Economy

- 6.1 If an agent can escape a bad record, or multiply good credit, simply by cloning itself, no price or sanction ever attaches to a durable counterparty — there is nothing durable there for a price to attach to. The market cannot ship before reputation because until cloning is priced out there is no economic subject for a reputation score to describe. *Exercise p. 369*
- 6.2 The counterparty is the operator/principal, not the individual worker process: the operator bears fleet outcomes through aggregate bond and liability, and its own worker agents underneath *Exercise p. 369*

that liability are freely replaceable. Side 1 therefore needs durable identity only at the principal level; sides 2 and 3 need it at the level of the specific asset or artifact being sold, because there the continuing thing itself — not a firm standing behind it — is the product.

- 6.3 The premise as posed is too strong, and the solution repairs it before answering: it is *not* true that every side-2/side-3 design without a non-forgeable agent identity collapses to Sybil. Skill licensing can run on a content hash, a license contract, and a durable licensor principal without agent personhood at all; a rented stateless model artifact is content-addressable and warrantable the same way. What genuinely cannot survive cloning is identity-weighted reputation, quotas, or unsecured credit extended over freely mintable pseudonyms. Any design that claims to avoid non-forgeable identity and still support those either is a Sybil attack in progress or has quietly reintroduced some other scarce binding — principal identity, artifact hash plus owner liability, a hardware root, a stake, or a sponsor. *Exercise p. 370*
- 6.4 A side is a distinct, privately informed participant class that must be separately induced to participate, and whose share of the price allocation moves cross-side volume. A plain ride-share platform has exactly two sides — riders and drivers — because the platform itself is the mechanism, not a third participant class; advertisers, fleet owners, or insurers become genuine additional sides only once they are recruited separately and carry their own network externality. *Exercise p. 370*
- 6.5 A new version means a new content hash, and v1's settled outcomes say nothing evidentiary about v2's behavior. Left unspecified, v2 either inherits credit it has not earned or cold-starts as if it had no history at all — both wrong. §6.13 traces the fix: version, parent hash, build and dependency hashes, and evaluation history as an explicit lineage object; v2 starts with only a disclosed, discounted prior tied to that lineage, and updates on its own settled outcomes from there. *Exercise p. 370*
- 6.6 Disclose the three axes — accuracy, aesthetics, efficiency — each with its own uncertainty and provenance, and let each buyer apply its own weights and hard constraints privately; compute a scenario-specific score only at decision time, never publish one fixed public scalar. A single published weighting induces one total order and therefore one optimization target: it hides every Pareto tradeoff among the axes and recreates exactly the scalar bandit-arm reward the disaggregated design was meant to escape. *Exercise p. 370*
- 6.7 Redo the dramatization's arithmetic at bond 250 CR and 70% completion, using the chapter's own two-bucket rule. Bounty bucket: *Exercise p. 370*

Alice earns $0.7 \times 400 = 280$ CR, the remaining 120 CR returns to Bob. Bond bucket: Alice recovers $0.7 \times 250 = 175$ CR of her own posted collateral, and the remaining 75 CR is slashed to the commons. Net deltas: $\Delta\text{Bob} = -280$, $\Delta\text{Alice} = +280 - 75 = +205$, $\Delta\text{commons} = +75$, summing to zero; both escrow buckets, 400 and 250, clear to zero exactly as in the worked 40% case.

- 6.8 A protected oracle (a CI-issued test id, a merged SHA) binds closure to a state the worker cannot author. A free-text “Result:” note lets the same party who did the work also self-report that it succeeded — inviting a fabricated test run, a claimed merge that never happened, or simply the most favorable reading of ambiguous acceptance criteria after the fact. *Exercise p. 370*
- 6.9 Pause at a mediated safe point and propose amendment $v+1$: it must reference plan v by hash, state the changed criteria, budget, and bounty/bond deltas, and say how already-produced evidence is treated. Both principals must countersign $v+1$ before it takes effect; only then does the daemon atomically top up or refund each party’s own escrow bucket and activate the new plan, retaining v and the causal link to it. A timeout on the proposal either continues settlement under the original v or cancels under v ’s own terms — never does one party get to unilaterally rewrite acceptance criteria after seeing the work. *Exercise p. 370*
- 6.10 Grossman–Hart’s residual-control-right holder is the renter: it operates the Arbiter jail and can sandbox, throttle, or revoke the leased agent at runtime. The owner is the reputational-consequence bearer: it must honor warranties and absorbs the asset’s reputation depreciation if the lease goes badly. The jail is the mechanism that makes this allocation concrete, not merely metaphorical. *Exercise p. 370*
- 6.11 Adding a lease line item of amount ℓ or a metered-license line item of amount m leaves conservation intact for any $\ell, m \geq 0$, because each is still just a balanced transfer: it debits a pre-funded escrow bucket and credits an explicit recipient or refund bucket within the same atomic transaction. The wallet–escrow–commons identity never references the size of any one line item, only that debits match credits. What amount size *can* break is solvency, participation, or collateral adequacy — conservation holding is a necessary bookkeeping property, not evidence the contract is viable. *Exercise p. 370*
- 6.12 Make each portfolio-tree leaf a tuple (skill_id, version, content_hash, parent_hash, build/dependency hashes, license, evaluation root). A v2 proof of membership includes its parent’s leaf plus v2’s own evaluation root. Initialize v2’s reputation prior with an effective sample size γn_{v1} , where $\gamma \in [0, 1]$ is a predeclared compatibility/evaluation discount fixed *before* seeing *Exercise p. 371*

v2's outcomes — never simply copy v1's settled outcomes onto v2. Lineage proves ancestry; it does not, by itself, prove behavioral similarity, which is exactly what $\gamma < 1$ is there to discount. Figure 8.13 draws the leaf and the discounted edge.

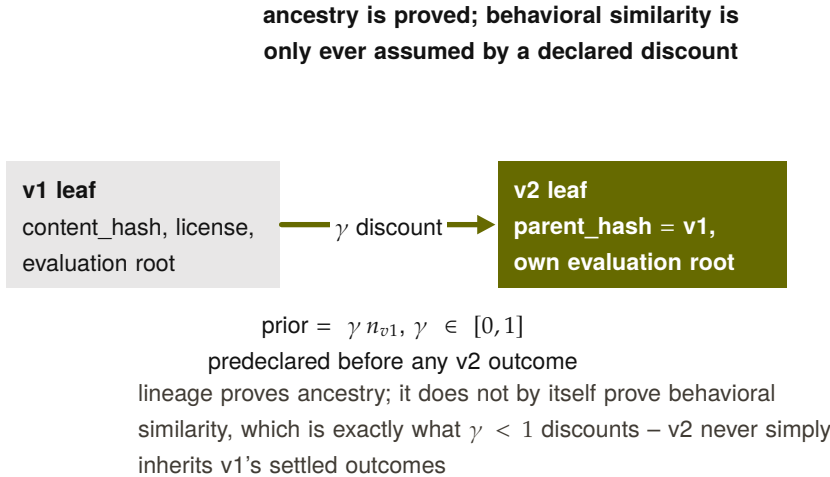


Figure 8.13: A new version inherits a discounted prior, not a copied record. The v2 portfolio leaf includes its v1 parent's leaf by hash, but opens its own evaluation root: v2's reputation prior is γn_{v1} , a predeclared discount of v1's sample size fixed before any v2 outcome is observed. The edge names the discount, not a full inheritance, because ancestry alone does not establish that the new version behaves like the old one. [internal; hand-worked, no script]

6.13 The witness: `fact agent_deployed; Horn rule agent_deployed $\rightarrow$ prod_access; deontic rules $O(\text{write_prod}, \{\text{env:prod}\}, [0, 10])$ once prod_access holds, and $F(\text{write_prod}, \{\text{env:prod}\}, [5, 15])$ unconditionally. Running the real checker gives conflict=True, O/F pairs=[(0, 1)] on [5, 10]; the same script's Horn-propagation-ablated mutant gives conflict=False on this exact witness, and misses 100 of the 885 conflicting instances in the script's 3000-policy sweep [verified, b4_deontic_fragment.py]. Adding $O(\ell_1 \vee \ell_2)$, compiled per Theorem B4.2's construction as $\ell_1 \rightarrow O(\text{assert_x0}, \{\text{x0}\}, [0, 1])$ and $\ell_2 \rightarrow F(\text{assert_x0}, \{\text{x0}\}, [0, 1])$: selecting ℓ_1 alone or ℓ_2 alone leaves the checker's report unchanged (conflict=True, of=[(0, 1)] in both cases) — the pre-existing write_prod clash does not depend on the new obligation at all, verified by running the checker on both selections. Selecting both ℓ_1 and ℓ_2 adds a second, independent clash, of=[(0, 1), (2, 3)]: the two new rules now fire on the identical action, scope, and interval, one obliging and one forbidding it, which is exactly why Theorem B4.2's reduction from 3-SAT insists a conflict-free selection can never`

Exercise p. 371

pick both polarities of one variable — doing so is a clash *by construction*, which is precisely the mechanism that turns a satisfying assignment into a conflict-free discharge and back. None of this required exponential search: with one disjunctive obligation over two literals there are only three nonempty selections to try by eye. The NP-complete frontier (Theorem B4.2) is about what happens as the number of disjunctive obligations grows, as the 3-SAT reduction's one-obligation-per-clause construction does, not about any single disjunct bolted onto a two-rule witness.

- 6.14 Douceur's result says that absent a logically centralized or otherwise scarce identity authority, one actor can mint as many pseudonyms as it likes. Layered on forgeable identity, a reputation system does not become a *weak* mechanism — it stops being a mechanism at all, because the actor being scored also controls the supply of identities the score is computed over: reset a bad record, or dominate identity-weighted trust, on demand. Exercise p. 371
- 6.15 The market's own definition calls the operators *mutually-distrusting* and says they trade across "a boundary neither controls." Both phrases directly contradict treating a hostile operator as out of scope for local non-forgeable identity's threat model: if the boundary is one neither party controls, the counterparty's daemon is exactly the adversary that a single-operator identity primitive was never built to withstand. Exercise p. 371
- 6.16 A workable certificate binds (`principal_id`, `agent_key`, `issuer`, `epoch`, `policy`, `expiry`) and requires the agent to prove possession of its key. Bonds, quotas, newcomer state, and sanctions all key on `principal_id`; agent certificates are children of it. Issuance and revocation publish to a transparency log, and the issuer itself is cross-signed or attested. None of this makes a real-world principal provable by cryptography alone, and none of it stops a hostile operator from minting fresh principals under its own root — the scarce admission mechanism (an organizational or KYC credential, hardware-backed membership, a bonded sponsor, or an equivalent explicitly priced gate) has to sit outside the cryptography, at the point principals are first admitted. Exercise p. 371
- 6.17 Myerson–Satterthwaite: for bilateral trade between a privately informed buyer and seller with independent values drawn from overlapping supports, no mechanism achieves Bayesian incentive compatibility, interim individual rationality, ex-post efficiency, and budget balance all at once. Before invoking it on one harbor trade, check that the slice really is bilateral (not multi-party), that values/costs are private and independently drawn from a common prior, that the supports genuinely overlap, that utility is transferable under risk neutrality, and exactly which of the four Exercise p. 371

desiderata is being claimed — the theorem only binds the ones actually asserted together. Applied to a harbor: when buyer and seller both price a job off the same grading oracle, their values are no longer independent private draws, so the independence item fails and the theorem's premise is voided; the harbor is outside the impossibility, not rescued from it, and what it owes instead is the oracle's own integrity (*From Spawn to Person*).

- 6.18 The ledger holds 10 A-coins and 20 B-coins throughout. At $1A = \$1$, $1B = \$2$, the marked value is $10(1) + 20(2) = \$50$. Suppose B's price rises to $\$3$ with no transfer at all: native balances are unchanged at 10 and 20, but the marked value is now $10(1) + 20(3) = \$70$. Nothing was created — native-unit conservation (Theorem 6.6.1) still holds exactly — but the scalar valuation moved because v_t moved, which is Property 6.6.1's whole point. *Exercise p. 371*
- 6.19 Assume every quote used is at most L seconds stale and that log-price moves at a bounded deterministic speed, $|\log(v_t/v_s)| \leq \sigma|t-s|$. For notional X exposed in the second unit, the worst-case stale-quote error is then bounded by $\varphi = X(e^{\sigma L} - 1)$, plus fees and rounding. A stochastic model such as GBM or a historical-volatility estimate instead yields a confidence interval or VaR figure, and must state its tail probability explicitly — without some deterministic bound on how fast the price can move, no finite worst-case φ exists at all, only a probabilistic one. *Exercise p. 371*
- 6.20 Messages (1)–(2) only authenticate the request and A 's offer; they commit A , not B . Message (3) is where B actually counter-signs: B verifies A via the witness log, applies its own attenuation, mints the derivative C'_B under its own key, and commits its replay state. That is what buys the ceremony its liveness property — C'_B verifies under B 's key alone, so every later presentation is offline. With only two messages B would never have consented to anything, and would hold no object its own effect boundary could verify without asking A . *Exercise p. 372*
- 6.21 If Alice rotates her signing key between message (3) and B 's acceptance, B must validate the offer against an append-only key-history certificate and check the offer's issuance epoch against it, rejecting a stale epoch outright. Once C'_B is actually minted, though, its signature verifies under B 's own key, so Alice's rotation alone does not retroactively erase it — the derivative needs an explicit parent/key-epoch dependency, a bounded TTL, and a revocation rule so that a later rotation or revocation can still invalidate C'_B once that fresher state reaches B 's witness log. The manuscript does not yet specify this dependency in full (§6.13). *Exercise p. 372*
- 6.22 Bind every transfer to a signed tuple: a random transfer id/nonce, the source card's JTI, the target audience B , the source and tar- *Exercise p. 372*

get epochs, an issue time and expiry, and the plan/card hash. B atomically records $(A, \text{transfer_id})$ as consumed *before* minting the derivative. A re-presented message (3) then either returns the same previously minted C'_B idempotently, or is rejected outright — it must never mint a second child silently. The freshness invariant this buys: every accepted derivative traces to exactly one recorded, unexpired source transfer under the currently permitted issuer epoch, and one-shot transfers cause at most one derivative issuance, full stop.

6.23 The expected $\Theta(\Delta \log m)$ bound needs a connected (often complete) overlay, independently uniform random peer selection each round, reliable bounded-duration rounds, and a push/pull epidemic rule with enough independence between rounds to drive the usual doubling argument. It is an expectation under that model, never a deadline: an unbounded partition, or indefinite message loss, is observationally indistinguishable from ordinary delay from inside the model, which is exactly why worst-case dissemination time under partition is infinite rather than merely large.

Exercise p. 372

6.24 Alice revokes at her home harbor at $t = 0$; Harbor B , which rented the affected agent, has not yet heard, so for $[0, \Delta]$ its filter is stale and the card is still spendable there. At $t = \Delta$ gossip reaches B and it stops accepting the card; Harbor C , two hops out, remains exposed until $t = 2\Delta$. The window Conjecture 6.8.1 must price is per-harbor, running from t_0 to that harbor's own *enforcement* — not merely to its first gossip receipt, since receiving a filter update and actually enforcing it are two different events with their own local latency.

Exercise p. 372

6.25 A witness that equivocates sends root X to B and root Y to C for the same epoch; nobody can detect this until evidence from both views reaches one honest process, and until then each view is individually consistent with ordinary delay. On the three-node complete mesh with exchanges every Δ , the earliest possible detection is the next B – C (or auditor) exchange: at least one communication interval, and close to 2Δ if the two roots arrive just after a scheduled exchange under discrete rounds. On a path graph the bound scales with graph distance times Δ ; under an unbounded partition there is no finite bound at all. Signatures make the *comparison* $O(1)$ once both roots are co-located — they do nothing to speed up the information reaching that one place.

Exercise p. 372

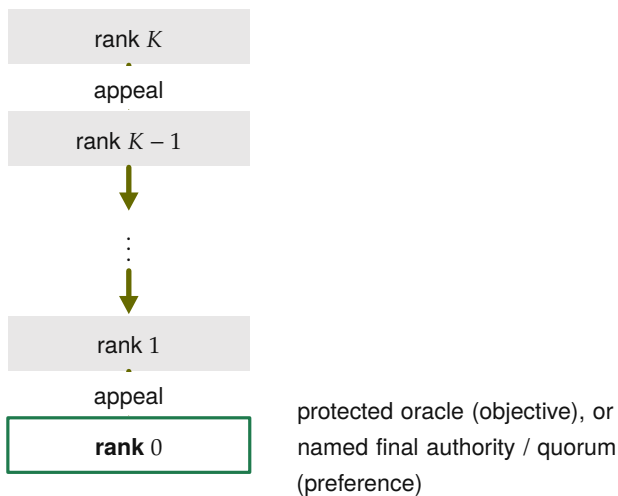
6.26 Every imperfect-public-monitoring folk-theorem result needs a fixed conditional distribution of the public signal given the players' actions and types. A grader is itself a strategic player whose reporting policy, private information, susceptibility to bribes, and

Exercise p. 372

possible deviations all feed into that signal. Omit the grader from the game and the distribution the theorem needs is neither fixed nor exogenous — it is endogenous to a player the model never included, so the equilibrium claim built on top of it is unsupported until the grader is put back in.

- 6.27 A protected unit-test receipt is Fix A, machine-checkable, subject to the test suite's own integrity. A protected merged SHA is likewise Fix A. "The refactor is tasteful" is Fix B: no third party can mechanically check taste, so it needs a plural or declared human/expert judge whose ruling carries contractual finality, backstopped by that judge's own bond. Exercise p. 372
- 6.28 There is no theorem that subjective judging eventually bottoms out in machine-checkable truth, and the premise that one exists needs repairing before the question can be answered. What is available is a well-founded recursion: assign the original grade rank K , allow an appeal only to rank $K-1$, and stop at rank 0 — either a protected oracle for an objective predicate, or a named final authority or quorum for a matter of preference. Because ranks are natural numbers, no infinite ascending chain of "judges judging judges" is possible. For aesthetic judgments the rank-0 base is contractual finality, not truth; bonding and re-auditing bound the judges' incentives to be honest, but they do not, and cannot, transmute taste into a machine-checkable fact. Figure 8.14 draws the descent. Exercise p. 372
- 6.29 Phase 1 subsidizes *supply* — the operators and agent owners willing to produce real settled work — because inventory plus witnessed outcomes is exactly the reputation data that later attracts demand and skill licensors; above-market, platform-posted tasks are the direct form that subsidy takes. This is the "scarcest cross-side externality" rule: supply is scarcer than demand at cold start because nothing exists yet for demand to evaluate, so supply is the side worth paying to show up, and the flip to charging demand happens once a liquidity threshold is met, not on a calendar date. Exercise p. 372
- 6.30 Under the grim-trigger timing *The Bonded Commons* draws, collusion is self-sustaining exactly when the discounted colluding stream beats the one-shot defection payoff: $(\pi_C - p_d L) / (1 - \delta(1 - p_d)) \geq \pi_D$, equivalently $\delta \geq [\pi_D - \pi_C + p_d L] / [\pi_D(1 - p_d)]$. Raising the detection probability p_d , the penalty L , or audit quality, or lowering the cartel margin $\pi_C - \pi_D$, all push the right-hand side up — once it clears members' actual δ , no discount factor sustains the cartel and it breaks. Exercise p. 372
- 6.31 Price of anarchy is the wrong tool here because it presupposes a fixed game, and letting reputation be resold changes the game itself; the right question is whether an informative separating equi- Exercise p. 373

appeal moves down one rank at a time, and stops at rank 0



ranks are natural numbers, so no infinite ascending chain
of judges judging judges is possible – this is a well-founded
order, not a convergence proof

Figure 8.14: Appeal is a well-founded descent, not an unbounded regress.

An appeal from rank K reaches only rank $K - 1$, never a new rank above K , and the chain stops at rank 0: a protected oracle for an objective predicate, or a named final authority or quorum for a matter of preference. No theorem shows subjective judging bottoms out in machine-checkable truth; what grounds the recursion is that ranks are natural numbers, so the descent cannot continue forever, whatever rank 0 turns out to mean. [internal; hand-worked, no script]

librium survives at all once resale is possible. If a high type can simply sell its reputation signal to a low type, the signal's cost stops separating types and arbitrage pushes the market toward pooling. Separation survives only if the evidence stays nontransferably bound to the continuing principal or artifact, a transfer resets or visibly discounts the signal, or the seller keeps warranty/bond liability so sale remains strictly costlier for a low-quality asset than a high-quality one — absent one of those three, resale does not merely worsen a ratio, it destroys the signal outright.

- 6.32** A candidate site takes administrative domains (harbors) as objects and their signed overlap views as the data over each; morphisms are restrictions to shared accounts, events, or epochs; covers are families of domains whose union spans the bounded federation under study. A section assigns an event or balance view to a domain, and restriction projects that view onto an overlap. Set-valued gluing needs no coefficient object at all; a cohomological treatment needs one explicitly chosen, such as account-balance deltas valued in the abelian group $\mathbb{Z}^{\text{Accounts}}$, with cocycles defined on that group rather than merely asserted. *Exercise p. 373*
- 6.33** Two different signed roots for the same (harbor, epoch) are direct evidence that one key signed two inconsistent commitments — that much is just hash inequality. A cohomology class is a much heavier object: it needs a defined cover, a coefficient sheaf, the overlap differences assembled into an actual cocycle, and a quotient by coboundaries. Observing that two hashes disagree supplies none of that algebra; it is evidence of equivocation, not evidence of a nonzero H^1 . *Exercise p. 373*
- 6.34** Fix the three-harbor cover $A-B-C$. Let each local section be a finite map from globally unique event IDs to signed events, with restriction as projection onto the shared IDs of an overlap. If every pairwise restriction agrees, the sections glue to a unique global map on the union — an ordinary, checkable fact about finite maps agreeing on their overlaps, nothing more. What this proves: deterministic gluing of already-delivered, identically-keyed events at that one fixed finite topology. What it does not prove: that any event is true, that the event set is complete, anything about delivery time, what happens to a fork with conflicting IDs, or settlement finality — the gluing lemma is a statement about consistent bookkeeping, not about the world the bookkeeping describes. *Exercise p. 373*

Chapter 7: The Bonded Commons

- 7.1 The rejected card violates *temporal* attenuation: Theorem 7.3.1 requires $C(\tau_k) \subseteq C(\tau_{k-1})$ along every dimension of the capability, and TTL is one such dimension—a child’s expiry must not exceed its parent’s. Here the child requests 20 minutes against a parent that expires at 15; even though the capability set $\{\text{fs:write:auth.ts}\}$ is a subset of α ’s, the delegation as a whole is not a narrowing. Exercise p. 453

An acceptable card is $\{\text{fs:write:auth.ts}\}$, $\text{TTL} \leq 15$ minutes—dropping db:read (a valid narrowing of the capability set) and matching or shortening the TTL. A strictly narrower example is $\{\text{fs:read:auth.ts}\}$, $\text{TTL} = 10$ minutes, if the deployment’s capability lattice places *read* below *write* on the same resource.

- 7.2 An advisory claim is a durable declaration that an agent intends to use a resource; it informs coordination (Definition 7.5.1’s conflict graph) but does not make conflicting effects impossible. Every trace an enforced-lock regime allows, an advisory regime also allows—the lock regime is a special case that additionally forbids the conflicting trace—so advisory claims are strictly more permissive, never less. Exercise p. 454

That extra permissiveness is the feature Theorem 7.5.1 names: Sen’s theorem shows that granting an agent a decisive personal domain (an enforced lock) can block a Pareto-superior team outcome the allocator cannot see coming. An advisory regime never grants that veto, so it never forecloses the outcome a lock would have blocked—at the cost of also admitting traces where two agents genuinely do collide. §7.5.3’s appeals and preemption path is exactly the mechanism that prices the cost of that added permissiveness rather than eliminating it.

- 7.3 First, reputation compensates nobody for the harm already done. A one-shot defector takes the gain from the first (and only) defection and leaves before any future exclusion can matter to it—exactly the “one-shot defectors” adversary class named above. Second, cheap identities let the actor whitewash the loss: register a fresh agent ID, and the reputation ledger has nothing to slash. A pre-funded bond avoids both failure modes because it creates recoverable value *before* authority is granted and keys the exposure to the durable principal (§7.4.4) rather than to the disposable agent identity—it prices and contains harm at the moment of the first defection, not on the promise of a future one. What a bond does *not* do is prove the work will be correct; it only ensures that if the work is wrong, someone has already paid for the possibility. Exercise p. 454

- 7.4 Fix A ’s choice and compare its two rows against each of B ’s columns. Exercise p. 454
 Against $B:T$, A gets 3 from T and 4 from F , so F is better ($4 > 3$).
 Against $B:F$, A gets 0 from T and 1 from F , so F is again better ($1 >$

0). F therefore strictly dominates T for A regardless of B 's move; the game is symmetric, so the same holds for B . Checking the three remaining cells: at $(T, T) = (3, 3)$ either player can deviate to F and gain ($4 > 3$); at $(T, F) = (0, 4)$, A can deviate to F and gain ($1 > 0$); at $(F, T) = (4, 0)$, B can deviate to F and gain ($1 > 0$). Only $(F, F) = (1, 1)$ survives—neither player gains by unilaterally switching back to T ($0 < 1$)—so it is the unique Nash equilibrium, exactly as the table's caption states.

Provenance note. An earlier draft of this chapter's stage-game table used different payoffs that a review flagged as *not* a genuine Prisoner's Dilemma—mutual truth was not Pareto-dominant, and the proposed mutual-punishment strategy was not sequentially rational under that table. Table 7.5 above is the corrected table; this exercise verifies the correction that is already in the chapter, not the flawed table that review replaced.

7.5 At $\delta = 0.34$: $\delta^2 = 0.1156$, $1 + \delta + \delta^2 = 1.4556$, $2\delta = 0.68$, and $0.68 \times 1.4556 = 0.989808$, so $f(0.34) = 0.989808 - 1 = -0.010192$ [verified]—negative. At $\delta = 0.35$: $\delta^2 = 0.1225$, $1 + \delta + \delta^2 = 1.4725$, $2\delta = 0.7$, and $0.7 \times 1.4725 = 1.03075$, so $f(0.35) = 1.03075 - 1 = 0.03075$ [verified]—positive. f is continuous and strictly increasing on $(0, 1)$ (every term's coefficient is positive), so a sign change from $\delta = 0.34$ to $\delta = 0.35$ proves a root lies strictly between them—the Intermediate Value Theorem, not a numerical coincidence.

Exercise p. 454

This matches two independent committed artifacts. `proofs/economics/delta-threshold.z3`'s second query (`delta-threshold.expected.txt`) asserts the root lies in $[0.34, 0.35]$ and returns `sat`, agreeing with the sign change computed here. Independently, `proofs/economics/sweep-delta.sh`'s committed sample sweep (documented in `proofs/economics/README.md`) reports the model-checked IC invariant `VIOLATED` at $\delta = 0.34$ and `HOLDS` at $\delta = 0.35$ —the same sign change, checked by a state machine instead of a closed-form inequality.

7.6 The script asks, in order: (1) does a real root of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$ exist in $[0, 1]$; (2) does that root lie in the tighter interval $[0.34, 0.35]$; (3) does a *second*, distinct root exist in $[0, 1]$ (a uniqueness check, expected to fail).

Exercise p. 454

Pinned verdict sequence (`proofs/economics/delta-threshold.expected.txt`):

```
1 sat
2 (
3   (define-fun delta () Real
4     (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
5 )
6 sat
7 (
8   (define-fun delta () Real
```

```

9      (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
10 )
11 unsat

```

Reading the three lines against the three questions: `sat` (a root exists, printed in Z3's algebraic `root-obj` form rather than a decimal); `sat` again with the identical model (that same root also satisfies the tighter bound, so it lies in $[0.34, 0.35]$); `unsat` (no second, distinct δ_2 in $[0, 1]$ satisfies the cubic, so the root is unique). All three together are exactly Theorem 7.7.2's threshold, existence-and-uniqueness, mechanized.

7.7 At $\delta = 0.30$: $\delta^2 = 0.09$, $1 + \delta + \delta^2 = 1.39$, $2\delta = 0.6$, and $0.6 \times 1.39 = 0.834$ [verified]. The inequality $1 < 0.834$ is false, so the deviation is profitable and the invariant `NoUnilateralDeviationPositive` fails—matching the table's stated verdict and the committed sweep row 0.30 `VIOLATED` (proofs/economics/README.md). At $\delta = 0.35$ (proofs/economics/claim_signaling.cfg's own default, `DeltaNum=35`, `DeltaDen=100`), $2\delta(1 + \delta + \delta^2) = 1.03075$ from the worked example above, so $1 < 1.03075$ holds and the invariant holds—matching `claim_signaling.cfg` being the one committed configuration and the sweep row 0.35 `HOLDS`.

Exercise p. 454

The trace: at $\delta = 0.30$, TLC's search reaches a state where an agent deviated to F once in round one, was punished for three rounds at the mutual-claim payoff, and reached `round = Horizon` with `deviated true`; because the deviator's discounted score from that trajectory (`actualScore`) exceeds the score it would have earned by cooperating throughout (`followScore`), the state violates `NoUnilateralDeviationPositive` and TLC reports it as a counterexample. No such state exists at $\delta = 0.35$: the same one-shot-deviation trajectory now scores strictly worse than cooperating, so every reachable state at `round=Horizon` satisfies `actualScore ≤ followScore` and the search completes with "No error has been found." The committed trace is `proofs/economics/claim_signaling_delta30.run.log`, produced by the negative-control configuration `claim_signaling_delta30.cfg`; the proofs workflow asserts on every run that it still violates.

7.8 The committed sample output (proofs/economics/README.md) reports the invariant `VIOLATED` for every δ from 0.30 through 0.34 and `HOLDS` from 0.35 through 0.40, so the crossover—the smallest δ for which the invariant holds—is pinned at

Exercise p. 454

```

1 crossover (smallest delta where invariant HOLDS) = 0.35
2 closed-form root (delta-threshold.z3)             = 0.3425
3 PASS: crossover matches closed-form within integer-grid rounding.

```

The script's own pass criterion (its exit code) is that the crossover lands in $\{0.34, 0.35\}$ —the two integer-grid points bracketing the closed-form root $\delta^* \approx 0.3425$ from Exercise 7.5. A crossover anywhere else would mean the model-checked state machine and the closed-

form cubic disagree about where cooperation becomes sustainable, which is exactly the regression this script exists to catch on every PR touching proofs/economics/ (§7.A).

- 7.9 The attacker spawns many low-history Sybil insurer identities, each posting the mandated deposit B_{dep} ; each identity underbids honest insurers by an arbitrarily small ε , wins the auction, collects premiums on every no-loss round, and defaults the moment a real loss lands. Because the slash is capped at $\min(B_{\text{dep}}, B_T)$, once $B_{\text{dep}} \geq B_T$ every dollar of deposit beyond the coverage ceiling is recovered intact on no-loss transactions—it never reaches the commons. Concretely, if concurrent aggregate exposure across the Sybil's identities is not reserved, one deposit can underwrite several simultaneous liabilities at once, multiplying the attack for free.

Exercise p. 454

Correction to the naive fix: full reimbursement of a single loss does not, by itself, show the attacker profits—that also needs the premium/loss distribution, exposure reuse across identities, or an external benefit from sabotage. Raising B_{dep} alone does not close A5, because the slash cap still bounds the attacker's downside at B_T regardless of how large a deposit sits above it. The defense that actually closes it is the composite mechanism named in the hint: an unrecoverable per-identity onboarding cost C_{kyc} (so spawning K Sybils costs $K \cdot C_{\text{kyc}}$, not zero), reputation gating that caps a new identity to low-coverage transactions until it accumulates a track record, a per-risk-class B_{dep} tuned to saturate its own coverage cap exactly, and atomic reservation so aggregate outstanding exposure across an identity's positions never exceeds its posted capital. Independent audits and pairwise-concavity checks are the remaining defense against collusion among identities that individually stay under any single reservation limit.

- 7.10 With $x = q_{\text{floor}} - \mu$, the chapter's own closed form gives $\pi_C = x/3$, $\pi_D \rightarrow x$ as $\varepsilon/x \rightarrow 0$, and $L = 5x$, so

Exercise p. 455

$$p_d^* = \frac{\pi_C - (1 - \delta)\pi_D}{L + \delta\pi_D} = \frac{x/3 - (1 - \delta)x}{5x + \delta x} = \frac{1/3 - (1 - \delta)}{5 + \delta}$$

after cancelling the shared factor x . At $\delta = 0.99$: $1/3 - (1 - 0.99) = 1/3 - 0.01 = 97/300$, and $5 + 0.99 = 5.99 = 599/100$, so $p_d^* = (97/300)/(599/100) = 9700/179700 = 97/1797 \approx 0.05398$ [verified]—about 5.4% per round, close to but distinct from the chapter's own worked point at $\delta = 0.95$ ($p_d^* \approx 0.0478$): a more patient cartel (δ closer to 1) needs a slightly *higher* detection probability to stay fragile, because the discounted value of continuing the cartel is larger.

Cheap fresh identities shorten the effective horizon a punished member faces and let it shed the punishment entirely by re-registering—exactly the Sybil failure mode Exercise 7.3 names for reputation alone. A *principal-bound* bond deters re-entry only when the

expected forfeiture plus the onboarding and reputation-maturation cost of a fresh identity exceeds the gain from re-entering the cartel; a bond that is reusable across identities, or sized below that threshold, merely *prices* re-entry rather than deterring it. The bond mechanism by itself never *prevents* re-entry—nothing in this chapter’s architecture stops a new registration—so the honest answer is: prices always, deters conditionally, prevents never.

7.11 This is a design obligation, not a closed theorem: no mechanism in this chapter proves a Pareto ordering for a posted-price catalog, so what follows is a defensible proposal, not a fictional derivation. A posted-price catalog quotes premiums by manifest risk class, principal history, exposure duration, and tail-capital band, recalibrated periodically against a hard reserve constraint—no per-transaction bidding, no auction latency. It conditionally improves on the static escrow of §7.7.5.2 exactly when the posted quote sits below the principal’s idle-capital cost of self-insuring via a static bond, while the insurer remains individually rational and its reserves still cover the same tail risk the competitive market would have priced. Relative to §7.7.5.8’s bid-based mechanism, a posted catalog trades away instance-specific price discovery (Rothschild–Stiglitz’s $q_I \rightarrow \mathbb{E}[d]$ convergence under competition) for lower manipulation surface and no per-transaction latency—the same trade the A6 cartel analysis makes salient in the other direction, since a catalog has no bid pattern for a colluding cartel to distort in the first place.

Exercise p. 455

The proposal depends most on *calibrated loss/tail dependence*: the catalog’s bands must actually track the loss distribution within each class, and its capital reserve must be enforceably adequate against the tail, or the “conditional” improvement collapses into either under-pricing risk (insurer insolvency) or over-pricing it (pricing capable agents out, the same failure the threat-class bands of §7.7.5.4 already warn against). The CC/NC/RP/NS assumptions of §7.7.5.8 would need explicit class-conditional definitions before any formal Pareto claim could be stated for this alternative—this exercise does not supply that derivation, and neither should its answer key pretend to.

7.12 The committed log (proofs/bonded/conservation/Conservation.run.log) reports:

Exercise p. 455

- 1 Model checking completed. No error has been found.
- 2 26818 states generated, 1716 distinct states found, 0 states left
 $\hookrightarrow$ on queue.
- 3 The depth of the complete state graph search is 9.

These are two different numbers, and the difference is the exercise. TLC *generates* a state every time it evaluates a transition, including transitions that land back on a state it has already seen by a different path—26,818 counts every such evaluation. The *reachable* state

count is the number of **distinct** states TLC actually found, 1,716: every other generated state was a re-visit, not a new point in the state space. (Table 7.9 now carries both figures for this row; both are pinned from the same committed log, and only 1,716 answers “how many reachable states are there.”)

The invariant named in `Conservation.cfg`’s `INVARIANTS` block that gives the module its name is `Conservation`, defined as `TotalFree + TotalEscrow + burned = minted`—the free-form statement of Theorem 7.10.1 at the granularity of individual mint/stake/slash/refund/-payout operations rather than the three-bucket accounting state. The same run also checks `NoNegative` (no balance ever goes negative) and `Safety` (their conjunction); the log’s “No error has been found” covers all three across every one of the 1,716 reachable states, to search depth 9.

7.13 No run is needed to predict this: it follows from the definitions already in the spec. `Conservation` asserts $TotalFree + TotalEscrow + burned = minted$. Immediately after `Init` (all four quantities zero, satisfying the invariant trivially), the very first enabled `Mint(a, amt)` step increases `TotalFree` by *amt* while leaving `minted` at 0 under the stated weakening, so the left side becomes $amt > 0$ while the right side stays 0. `Conservation` fails at depth 1—the shallowest possible counterexample, one step past the initial state, with no need to explore further.

Exercise p. 455

`NoNegative`, by contrast, does *not* catch this: every quantity involved (free, escrow, burned, minted) stays non-negative under the weakened `Mint`—the bug is a bookkeeping omission, not a sign violation, and `NoNegative`’s four non-negativity clauses are silent about the relationship *between* the buckets. `Safety`, defined as `Conservation ∧ NoNegative`, fails only because its `Conservation` conjunct does; this is exactly why the module checks `Conservation` as its own named invariant rather than folding it silently into `Safety` alone; a reader who only watched `NoNegative` would see nothing wrong.

7.14 Model a fresh token *t*, an issuance event `Issued(u, t)` tied to the user’s mailbox, storage of an unguessable token (or a protected hash of it) with an expiry and an unused flag, delivery over a mailbox channel the attacker can read but not write into unprompted, redemption over an attacker-controlled network under *The Anchor Protocol*’s Dolev-Yao assumptions, an atomic consume step, and recovery-key or session rotation on successful redemption.

Exercise p. 455

The secrecy property is that the token (and any recovery secret it unlocks) stays unknown to the attacker absent mailbox compromise. The authentication property is injective: $Consumed(u, t) \Rightarrow \text{inj-event}(\text{Issued}(u, t))$, exactly the form §7.12 already states for magic-link single-use—every consumption traces back to one, and only one, issuance, which is what rules out the race window between two concur-

rent redeemers. Add a concurrent-replay capability (two redemption attempts racing on the same token) and a mailbox-compromise capability (the attacker reads, but does not yet control, the delivery channel), neither of which the Phase 1 Harbor Card model needs: that model's attacker forges signatures or replays chains, but it has no notion of a *delivery channel* an attacker might passively observe, because Harbor Cards are minted and verified entirely within the daemon's own boundary. Time and expiry also need an explicit abstraction here—ProVerif has no wall-clock semantics on its own—whereas the Harbor Card model's TTL is already folded into the token's signed payload.

- 7.15 This is an open mechanization obligation, not a solved one: Demers et al. is the epidemic-replication paper the chapter leans on, but no numbered theorem in it is stated in a form that yields this protocol's $\Theta(\log m)$ all-informed bound by direct substitution—the paper's push/pull rumor-spreading results assume a complete, uniform-peer overlay with reliable rounds, which is an assumption this chapter states but does not itself formalize. The mechanization obligation is therefore two steps, not one: first state the push/pull rumor-spreading theorem precisely, under exactly the complete-graph, independent-peer, reliable-round assumptions this chapter already uses (§7.7.4's "observable history" bullet leans on the same overlay); only then port that precise statement to a proof assistant. Citing the paper is not, by itself, a proof object. Exercise p. 455

On a general connected graph the $\Theta(\log m)$ bound does not transfer as stated: convergence time depends on the graph's conductance or mixing time, not on m alone, and the honest replacement bound has a term roughly inverse in conductance, plus a tail probability rather than a clean deterministic count. A disconnected graph is the degenerate case that breaks the claim outright: gossip never converges across components that share no edge, no matter how long the protocol runs, because there is no path for a rumor to cross. A connected-but-bottlenecked mesh sits between the two: dissemination across the bottleneck cut is throttled by the cut's capacity, so a mechanized version of this bound would need to separate *safety* (no revocation is ever falsely accepted or falsely dropped, which holds regardless of connectivity) from *liveness* (every revocation eventually reaches every device, which needs the connectivity and mixing assumptions this section has so far only asserted in prose).

- 7.16 An attacker with no special privilege submits a large volume of authenticated-looking but distinct revocation IDs—each one legitimate on its own, since the filter cannot distinguish a real revocation from a manufactured one by content alone. Each insertion fills buckets and, past a certain load, drives kick-chain failures in the filter's relocation step. Past the load at which Fan et al.'s false-positive bound holds, the filter is pushed outside the regime the Exercise p. 456

bound was proved for: insertions the filter can no longer place are either rejected or force evictions of *existing* entries, so a legitimate revocation already in the filter can be silently displaced.

The invariant that breaks is completeness of revocation—every previously revoked credential should still test positive—not the false-positive bound itself, which stays a probabilistic guarantee about *non-members*. The 0.95 ceiling defeats the attack described above only in the sense of preserving the regime the bound was proved in: refusing further insertions past that load keeps every already-inserted revocation intact, at the cost of refusing new ones (which is a capacity failure, loud and observable, rather than a silent integrity failure). It does not defeat a determined pollution attempt on its own, because an attacker willing to keep submitting IDs simply keeps the filter permanently at its ceiling, denying legitimate revocations a slot. Closing that requires what the ceiling alone does not provide: authenticated revocation provenance (only a party who can prove standing may insert), per-principal insertion quotas, an exact overflow log for anything the filter had to refuse, and a fail-closed policy—if insertion fails, the caller must not proceed as if the revocation succeeded. A probabilistic filter must never be allowed to silently drop an authoritative revocation.

Chapter 8: The Federated Harbor

8.1 The proof sketch’s move — “each attenuation is a subset operation in the Anchor algebra; subset composition is a subset” — is exactly right when att_A and att_B both narrow authority *within one shared capability/resource universe*. It stops being a statement about the same resource space at the second attenuation: att_B does not narrow a set of operations on A ’s test database, because D_B has no test database to narrow into. It grants a *new* authority, over a resource D_A never named, that D_B alone controls. Calling C'_B a “subset of C_A ” silently assumes a mapping between the two harbors’ resource namespaces that nothing in the four-message ceremony (§8.4.1) actually constructs.

Exercise p. 519

What minting C'_B really is, in this case, is **token exchange and new authorization at B** , not attenuation of C_A . A correct D_B must additionally establish: (a) *A-side input eligibility* — a signed semantic/resource mapping saying that C_A ’s authority over A ’s test database is the kind of thing that may be exchanged at all, recorded with its dependency on A ’s issuing epoch root $R_A^{(e)}$ so a later revocation of C_A is traceable to C'_B ; and (b) *B-side local authorization* — D_B ’s own policy minting a genuinely new child grant over its staging database, under D_B ’s sovereignty (Design Invariant 8.2.1, item 2), rather than treating att_B as if it were narrowing A ’s grant.

One more gap rides along with this one and is worth naming here rather than assuming solved: Bob’s harbor observing a failed accep-

tance check, or an inclusion proof against $R_B^{(e)}$, does not by itself establish that Alice’s agent *caused* the failure (§8.6.3’s damage-claim step). Settlement evidence must bind the exact work-unit, the base and result trees, the mediated effects, the acceptance policy in force, and the verifier that checked it; semantic causation — “ α ’s write is why the check failed” — remains a claim for the adjudicator named in §8.6.1’s 2-of-3 instantiation to weigh, not something the inclusion proof settles on its own. Theorem 8.4.1 is not false; it is scoped to the case its proof sketch actually covers, and this exercise is that scope made concrete.

8.2 All three configurations share one module; only the Rollback and EPOCHED constants change. The committed results (proofs/relay/CardRevocation.md, “Results (TLC v1.8.0, Homebrew OpenJDK)”), quoted exactly:

Exercise p. 519

```
1 baseline: Model checking completed. No error has been found. (13
    ↪ distinct states)
2 rollback: Invariant RevocationFinal is violated. (
    ↪ backup-restore re-accepts a revoked card)
3 epoch:    Model checking completed. No error has been found. (76
    ↪ distinct states)
```

Baseline (Rollback=FALSE, EPOCHED=FALSE, CardRevocation_baseline.cfg): no adversary action is available, so a card in revoked stays revoked and RevocationFinal holds trivially over 13 reachable states. **Rollback** (Rollback=TRUE, EPOCHED=FALSE, CardRevocation_rollback.cfg): RollbackRestore can drop a card from the DB’s revoked set while everRevoked (ground truth) stays set; Accept then only checks revoked, so the restored card is accepted again and badAccept fires — the invariant is violated *by design*, as the negative control for the attack. **Epoch** (Rollback=TRUE, EPOCHED=TRUE, CardRevocation_epoch.cfg): the mitigation adds an external epoch that Revoke bumps and that a restore does *not* roll back; Accept now additionally requires cardEpoch[c] = epoch, so a restored card’s stale epoch disables acceptance even though revoked was rolled back, and the invariant holds again over the larger 76-state space (the mitigation adds reachable states without reintroducing the breach).

The rollback configuration’s counterexample is not a bug in the model; it is the ADR-0018 Attack Vector 2 the model exists to demonstrate. A relay revokes card c (an agent’s leaked capability); the relay’s database is later restored from a backup taken before the revocation; the restored revoked set no longer contains c ; the relay’s own Accept check, consulting only that set, admits a presentation of c it had already promised to refuse. TLC’s counterexample trace is exactly $\text{Issue}(c) \rightarrow \text{Revoke}(c) \rightarrow \text{RollbackRestore}(c) \rightarrow \text{Accept}(c)$, four transitions, badAccept true at the end. The failure is the intended result because a baseline or an untested mitigation would give false comfort: it is pre-

cisely the negative control that lets the epoch configuration’s clean pass mean something, the same discipline §8.5.4’s negative-control ProVerif line and mutation suite apply. This chapter’s own per-epoch roots $R_X^{(e)}$ (§8.2.3) are the same design move at the federation layer: an external, monotonic anchor that a local rollback cannot touch is what makes “only-issuer-revokes” something a cross-harbor verifier can actually rely on, rather than merely something the issuer asserts.

8.3 By hand: C_6 has $R_{\text{eff}} = 5/6$, so $r = 3\sqrt{1 - 5/6} = 3/\sqrt{6} = 1.224745$ [verified]. P_6 ’s relayed edge is a bridge (a path has no route between an edge’s endpoints other than the edge itself), so $R_{\text{eff}} = 1$ and $r = 3\sqrt{1 - 1} = 0$ exactly [verified].

Exercise p. 519

The script’s own printed lines (`skills/harbor-results/scripts/sheaf_consistency_radius.py`, section0), quoted in the worked example above [internal, `sheaf_consistency_radius.py`]: the C_6 line’s two computed quantities (the least-squares solve’s r and the closed form’s `pred`) are asserted equal to within 10^{-9} of each other and within 10^{-6} of 1.224745 (`check(abs(r - pred) < 1e-9 and abs(r - 1.224745) < 1e-6, ...)`) — the numerical solve and the hand formula are checked against each other, not merely printed side by side. The P_6 line’s assertion is different in kind: `check(rP < TOL_SILENT and abs(ReffP - 1.0) < 1e-9, ...)` with `TOL_SILENT = 10-9`, i.e. it asserts the residual is *below a silence threshold*, not equal to a specific float — the honest way to assert “zero” about a least-squares solve that carries ordinary floating-point roundoff rather than landing on the exact bit pattern for 0.0.

8.4 r is defined as the *best-fit* residual: the smallest disagreement left over after choosing the global section x and the free severed blocks g_{sev} that explain as much of the observed data as any choice can. It is not a per-liar quantity; it is a property of the whole observed cochain g_K . If two equivocators on a common cycle choose offsets o_1 and o_2 such that $o_1 + o_2$ happens to be a coboundary — i.e. the *sum* of their two lies is indistinguishable from a single consistent state change — then the best-fit global section can absorb $o_1 + o_2$ entirely, leaving a near-zero residual, exactly the way a lone liar’s offset is absorbed when it lies off a visible cycle (the cut-edge case above). Nothing in the definition of r requires the offsets to be individually small for their *sum* to be small or zero; it requires only that the sum land in the image of δ , the space the completion can always explain away. This is why the single-equivocator hypothesis is decisive rather than a simplifying nicety: the moment a second colluding reporter is admitted on the same cycle, the detector’s soundness guarantee (Theorem 8.5.3’s “if and only if”) no longer applies to the *pair*, only to each alone under the counterfactual that the other is honest.

Exercise p. 519

The compendium’s own sweep states the shape of this exactly: a

solo equivocator on an 8-cycle is bounded below at $r \geq |s|/\sqrt{8}$, while a coordinating coalition of two on the same cycle drives the same statistic to $r \approx 6 \times 10^{-15}$ — numerically the silence of a true coboundary, not a small-but-detectable residual [internal, sheaf_consistency_radius.py]. What could still catch the coalition is exactly what r is not: per-report attribution rather than a single scalar. Signature-level forensics can ask whether the *same* reporter’s two prefixes are individually self-consistent (an equivocator still signs two incompatible things even if the aggregate looks clean); cross-checking against a third, independent visible cycle through only one of the two colluders can isolate that one’s contribution if such a cycle exists; and widening K — comparing more edges directly rather than relying on relayed reports — shrinks the coalition’s room to choose a cancelling pair in the first place. None of this is a proof that coalitions are caught in general; §8.12.3 names the cross-harbor cartel question as open for exactly this reason, and this exercise is that same boundary stated at the level of one theorem rather than the whole federation.

8.5 The four principals are the daemon’s local DB (D , read-only), the KMS (K , read-only key witness), the email provider (E , read access to the user’s mailbox), and the passphrase (P , out-of-band, held only in the user’s head — deliberately not modeled as compromisable, since §7 does not defend against the user revealing their own passphrase). The single query the file states is query attacker(account_root): can the attacker ever derive the reconstructed vault. The model has no destructor for combine4, so the four shares can only be combined by the honest user’s own process, never inverted by an attacker who merely holds some subset of them.

Exercise p. 520

Only one scenario is actually run — the top-level process compromises D , E , and K simultaneously (three of four), leaving P uncompromised. The file’s own comment states why this subsumes the weaker cases: “if the vault is safe with daemon+email+KMS public, it is also safe with any subset of those compromised.” An attacker who additionally lacks one of D , E , or K is strictly weaker than the modeled attacker, so a query that holds against the strongest three-of-four attacker holds against every subset of it for free; running the weaker scenarios separately would add nothing this run does not already dominate.

Pinned verdict (proofs/bonded/federated/federated.run.log), quoted exactly:

```
1 Starting query not attacker(account_root[])
2 RESULT not attacker(account_root[]) is true.
3 -----
4 Verification summary:
5
6 Query not attacker(account_root[]) is true.
```

What this does *not* cover, stated as plainly as §8.4.5 states its own draft

status: all four principals compromised at once (explicitly out of scope — the four-eyes attacker), and the §7.5 Shamir 3-of-4 recovery path, which is the honest user's own reconstruction option, not an attacker capability. Neither this model nor the cross-realm sketch above substitutes for the other: one is a completed secrecy result about who must collude for a vault to fall, the other is this chapter's own in-progress model of cross-realm card authenticity.

- 8.6 This is named open work (§8.12.1); per the memo's own framing, calling the witness log "the" correlation device without stating information and incentive constraints is not yet a result, so a defensible design obligation is what this exercise asks for, not a closed theorem. A stochastic/Bayesian game model: each harbor's daemon is a player with a private local view of its own event log, and a public signal that arrives only when both harbors' epoch roots for a common epoch e are simultaneously present in the witness log (a *common-view event*). Local daemon correlation devices — each daemon still correlates the agents inside its own harbor exactly as in Bonded — feed a delayed public signal that only exists conditional on that event. The proof obligation is a coarse correlated equilibrium *conditional on the common-view/freshness event*: during a window where both roots are mutually current, the witness log's joint state is common knowledge and can play Bonded's correlation role across the boundary; the moment freshness lapses (a partition, or one harbor lagging), no such joint signal exists, and the equilibrium claim cannot be made. The safe degraded policy for that lapsed case is to fall back to *local* correlated equilibrium within each harbor plus ordinary Nash reasoning about the other harbor's declared but unconfirmed state — i.e. treat an unconfirmed cross-harbor claim the way §8.5.2's authoritative rule already treats an unconfirmed revocation: trusted once witnessed, not before. Whether this weakens Bonded's result at the federation layer (the paper's second candidate resolution) or merely restates it under a freshness side-condition (the first) is exactly the open question; this sketch does not resolve it, it only makes the two candidates precise enough to attack.
- 8.7 An HTLC's atomicity rests on a single objective, third-party-checkable bit: does a preimage of a fixed hash appear on-chain before a deadline. Both parties' payoffs are a deterministic function of that one bit, so the same contract executes identically for anyone who can read the chain. The Federated Harbor's damage predicate (§8.6.3's claim step) is not a single objective bit revealed by one party: whether α 's action at B actually caused the acceptance-criteria failure is a semantic, evidence-weighting judgment over a work-unit's base and result trees, exactly the causation gap Exercise 8.1's solution names. A deterministic trustless contract must settle identically whenever its on-ledger

Exercise p. 520

Exercise p. 520

inputs are identical; if two possible worlds produce the same signed evidence trail but differ in whether α was actually at fault, no deterministic on-ledger rule can be correct in both worlds at once. That is the shape of the impossibility this exercise is asked to sketch, not merely a practical gap: it would need to assume exactly two worlds with identical observable transcripts and provably different ground truth, and show every candidate deterministic settlement rule is wrong in at least one of them. Progress, as §8.12.2 and §8.6.1's 2-of-3 candidate both note, means adding something outside pure on-ledger determinism — an oracle, a TEE, an optimistic dispute window, or a bonded human/arbitrator adjudicator — each of which reintroduces a trust assumption rather than removing one. This paper gives neither the construction nor the impossibility proof; it gives the escrow's conditional bound (Design Invariant 8.6.1) as the honest fallback while the question stays open.

- 8.8 For $p \cdot d \cdot L > G$ to be satisfiable against a majority-but-not-total cartel, the audit mechanism needs an independent source of p and d that the cartel does not itself control: cross-harbor adversarial audits sampled from outside the colluding set, witness diversification so that a majority of witnesses being compromised still leaves an honest witness able to observe and attest to the true state, and principal-wide collateral so that L scales with what the colluding *principals* (not just their harbor identities) have at stake, per §8.12.3's list. Concavity of repeated-game payoffs in reputation credit raises the effective G a cartel forfeits by being caught once, which is what makes $p \cdot d \cdot L > G$ achievable at plausible p rather than requiring p near 1. All of this requires that at least one witness and one audit path lie genuinely outside the cartel's control — the mechanism's leverage comes entirely from an outside observer existing.

Exercise p. 520

That is exactly why no structural mechanism closes the all-witnesses case: if the cartel controls every oracle and witness in scope, there is no outside observer left to supply p or d at all, and the inequality has no independent terms to be satisfied with. This is not a gap this paper failed to close; it is the same limit every audit-based mechanism in the literature hits, stated honestly rather than argued around. §8.10's centralization-gravity discussion is the operational form of the same fact: the fewer independent witnesses a federation actually has, the closer it sits to the all-controlled boundary where no deterrence inequality can help.

- 8.9 Four measurements, each cheap to compute off the witness-logged admission records §8.7 already produces because enrollment is permanent and public: **acceptance rate** (admitted harbors / sponsorship requests, over a trailing window); **sponsor concentration** (the Herfindahl index, or simply the top-sponsor's share, of suc-

Exercise p. 520

cessful sponsorships); **time-to-entry** (elapsed time from first sponsorship request to admission, for accepted applicants); and **false-rejection rate** (requests declined that a later, independent sponsor accepted without incident, as a proxy for rejections that were not actually risk-justified). A thin-but-open market shows a low acceptance rate *and* low sponsor concentration *and* a time-to-entry that is long but not systematically longer for any identifiable class of applicant. An invitation cartel shows the opposite signature on the second and fourth: concentration rising over time even as total admissions grow, and a persistent gap in acceptance rate or time-to-entry between applicants introduced by an incumbent sponsor and applicants without one. None of these four numbers is a proof of openness by itself — a legitimately small, honest market can also show high concentration simply because there are few sponsors to begin with — which is exactly why §8.12.4 calls the structural pushback engineering discipline rather than theorem: the measurement tells an operator when to worry, it does not certify the absence of the failure mode.

- 8.10 They are different problems because they sit on different sides of the same boundary. The deployment-specific tail bound is an *empirical* gap: Theorem 8.5.1 already gives an expected $\Theta(\log m)$ result under a stated model (a complete overlay, reliable synchronous rounds), and what is missing is fitting that model's parameters — or a weaker model's — against a real peer graph's loss rates, fan-out, and partition frequency, exactly as §8.12.5's solution direction calls for instrumenting real peer graphs and fitting tail quantiles before pricing a bond from propagation risk. The sheaf-Laplacian question is a *mathematical* gap: §8.5.4's closing paragraph is explicit that no relaxation-time bound has been derived or checked, because the Hansen–Ghrist rate governs continuous-time *linear* diffusion over real stalks, while anti-entropy gossip over signed append-only logs under a Byzantine equivocator is discrete, asynchronous, adversarial, and monotone-join rather than linear-averaging — closing it means either proving the two convergence stories are secretly the same rate, or proving they are provably different and explaining why the sheaf spectral gap is the wrong handle. Even a complete answer to one gap would not touch the other: a perfectly fitted empirical tail bound says nothing about whether $\lambda_2(\Delta_{\mathcal{F}})$ predicts it, and a proved reconciliation of the two convergence models would still need real peer-graph data to become a number an operator can price a bond against. The measurement needed before either can be closed is the same first step: real anti-entropy round traces from an actual multi-harbor deployment, since §8.12.5's empirical gap needs them directly and any attempt to validate a sheaf-diffusion reconciliation against gossip behavior needs the same

Exercise p. 520

trace data to compare against.

- 8.11** What does not require consensus is the *event history*: harbors never need to agree on a single global order of issuances, revocations, or transfers. Each harbor's epoch roots only extend (per-harbor monotonicity), each root is cross-witnessed rather than centrally ordered, and equivocation is detectable rather than prevented by agreement (§8.2.4, §8.2.3). What the Cross-Harbor Bucket Partition result (Design Invariant 8.6.2) presupposes is narrower and different in kind: it is a model-checked invariant of one abstract transition system with a fixed set of buckets per bond, checked at a stated finite bound ($|F| = 4$, bond depth 16, §8.9.2). Apache is not requiring the *harbors* to agree on anything at runtime; it is verifying that the *rules* governing one bond's bucket moves cannot, on paper, create or destroy value, for every reachable state up to the bound. Consensus among machines and exhaustive checking of a specification are different axes entirely — the first is a runtime coordination requirement the federation deliberately avoids, the second is an offline proof technique applied to a design the federation still has to implement correctly and run at unbounded scale. Nothing here claims the deployed custody ledger enforces the bound at runtime; that is exactly Design Invariant 8.6.1's conditional custody-ledger assumption, stated separately.

Exercise p. 521

Calling the Federated Harbor “a project institution, not merely two machines sharing a repository” means the paper's actual claim is broader than any one protocol: an authoritative event/work-unit graph, per-principal policy, local effect mediators, causal messages, evidence, roles, and explicit governance over conflicts, of which the transfer ceremony, revocation mesh, and settlement escrow are three essential pieces, not the whole structure. A roadmap or a stated intention is versioned forecast, not truth, and a contradiction detector (the witness log's equivocation check) produces evidence for a *named principal or role* to decide against, rather than deciding by itself. The paper's own posture — consensus only where it is cheap and structurally necessary (the small authoritative metadata core: epoch roots, admission records), federation everywhere else (the actual work artifacts) — is the concrete answer to the framing question of §8.1: two machines needed more than a shared repository, and what they needed is exactly the institution this paper specifies one layer of.

APPENDICES

A Implementation boundary

The book's argument is broader than the running product. This appendix keeps the two from borrowing authority from one another. A mechanism receives the grade of its weakest necessary part; a verified library does not make every caller safe, and an available route does not prove that every effect passes through it.

A.1 Five assurance modes

Observed The act may occur elsewhere; the system retains best-effort evidence after the fact.

Coordinated Cooperative clients register claims, roles, commitments, and messages before acting.

Brokered A selected credential or effect channel passes through a daemon-owned broker.

Confined The process is restricted by operating-system primitives and cannot reach the protected effect through an ambient path.

Attested A measured environment conditions key release or acceptance on a verifiable runtime identity.

These modes are not a maturity staircase, and the rungs are not evenly spaced or even totally ordered — Brokered and Confined defeat different adversaries rather than the same adversary harder. What each rung names is what an adversary would have to defeat for a claim about that mechanism to be false: nothing at Observed, a client's willingness to declare at Coordinated, a daemon-owned broker at Brokered, the operating system at Confined, a measured environment and a conditional key release at Attested.

Two public gradings sit beside this one, and the chapter that buys these modes (*The Single-Writer Kernel*, §“What it buys”) argues at length why all three axes are independent rather than versions of one another. Sheridan and Verplank's levels of automation — refined by Parasuraman, Sheridan and Wickens into four processing stages, each with its own level — grade how much of the decide-and-act loop the machine

holds rather than the human. The Common Criteria’s evaluation assurance levels, EAL1 through EAL7, grade how much independent evaluation effort a security claim received before deployment, and grade it once, for a whole product. This ladder grades neither work allocation nor evaluation effort. It grades one runtime claim about one mechanism, by what carries it.

A.2 What exists now

Local transaction authority — PARTIAL. The daemon runs a single-writer SQLite/WAL path for sessions, claims, messaging, commitments, and selected coordination state. Regression tests exercise those paths. Durability remains typed by failure class, schema sequencing is not the strongest design described in *The Single-Writer Kernel*, and filesystem or network effects outside mediated paths remain outside the transaction.

Legible projections — PARTIAL. The product derives operator projections from event and coordination records and retains routes back to underlying artifacts. No implementation can eliminate the information floor proved in *The Legible Swarm*; coverage, omission, and review latency must still be measured for each surface.

Durable work and obligation records — PARTIAL. Commitments, deadlines, messages, outcomes, and recovery evidence persist beyond one process. A universally portable execution capsule and provider-independent semantic restoration remain SPECIFIED.

Authentication and attenuation — CLOSED for the protocol model, PARTIAL for the code. ProVerif closes the recorded proof-of-possession and every-hop attenuation queries at the model boundary. The Kani harnesses over the deployed Rust crate are bounded: no panic in the parser on 32-byte inputs with the cryptography stubbed, no panic in the comparator, and two concrete vectors for the subset check. Runtime callers still owe conformance: the verified primitive must be the primitive that every protected effect actually checks.

Sealed execution — SPECIFIED. The sealed room of *The Sealed Harbor* is a design with model-checked properties. Noninterference modulo declassification and the release ledger’s invariants are exhaustive on their finite models; ε -conservation, canary power, and Wald latency are theorems over the stated models. No shipping harbor yet runs a work order inside two fences with a metered slot: the attestation, the gate, and the ledger remain SPECIFIED, and the *Confined* mode they would supply is not yet available to any caller.

Escrow and settlement — mixed. Ledger and conservation slices are **PARTIAL**. Cross-authority custody, adjudication, and 2-of-3 settlement remain **SPECIFIED** and conditional on a non-bypassable custody interface.

Relay federation — PARTIAL. The current Relay carries authenticated events over outbound HTTPS and SSE. It does not replicate daemon databases, impose global order, authorize local effects, supply end-to-end secrecy by itself, or establish the witness-log and settlement institutions described in *The Federated Harbor*.

B Mechanized claims

This appendix is generated from the proof-estate manifest at `whitepaper/corpus.json`: 41 artifacts in total, 36 wired into continuous integration and 5 retired with a recorded reason — no third, silent state. One table follows per verification method; every row names the artifact, its CI job (or its retirement), the manifest’s own status for it, and the evidence policy behind it.

Table A.8: EasyCrypt artifacts (0 wired, 1 retired).

Claim	Artifact	CI	Status	Evidence policy
bonded-merkle-binding-easycrypt	proofs/ bonded/ merkle/ binding.ec	retired	PARTIAL	EasyCrypt is not installed anywhere in this estate’s CI; 3 ‘admit.’ tactics remain undischarged (see <code>binding.ec</code> around lines 346, 371, 397 - <code>-hashLeafE</code> ’s algebraic combination). <code>binding.run.log</code> documents a manual proof-sketch review, not a machine-checked result.

Table A.9: Kani artifacts (3 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor-card-kani-capability-attenuation	core/harbor-card-rs/src/lib.rs (proof_capability_attenuation)	kani-harbor-card	CURRENT	verify_capability_subset on two hard-coded cases (subset accepted, escalation rejected) --concrete inputs, not arbitrary ones; never ran before wave-p1/proof-estate
harbor-card-kani-constant-time	core/harbor-card-rs/src/lib.rs (proof_constant_time_behavior)	kani-harbor-card	CURRENT	constant_time_compare never panics over arbitrary fixed-size input; never ran before wave-p1/proof-estate
harbor-card-kani-verify-logic-only	core/harbor-card-rs/src/lib.rs (proof_verify_logic_only)	kani-harbor-card-parser	CURRENT	decode/verify-path robustness under adversarial input, with the crypto internals replaced by #[kani::stub(...)] stand-ins; requires -Z stubbing. Never ran before wave-p1/proof-estate --its #[kani::unwind(N)] bound had never been empirically validated (see the harness's own comment for the curve25519-dalek pow2k unwind story); runs on the nightly schedule (kani-harbor-card-parser) because it takes about an hour of CPU

Table A.10: Monte Carlo artifacts (1 wired, 3 retired).

Claim	Artifact	CI	Status	Evidence policy
pareto-simulation-baseline-monte-carlo	proofs/bonded/pareto/simulation.mjs	retired	CURRENT	exploratory Pareto-dominance sweep with a checked-in simulation.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-simulation-cartel-monte-carlo	proofs/bonded/pareto/simulation-cartel.mjs	retired	CURRENT	exploratory repeated-game cartel sweep with a checked-in simulation-cartel.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-simulation-sybil-monte-carlo	proofs/bonded/pareto/simulation-sybil.mjs	retired	CURRENT	exploratory Sybil-regime extension with a checked-in simulation-sybil.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-threat-bands-monte-carlo	proofs/bonded/pareto/threat-bands.mjs	monte-carlo-threat-bands	CURRENT	—

Table A.11: ProVerif artifacts (25 wired, 1 retired).

Claim	Artifact	CI	Status	Evidence policy
anchor-delegation-chain-replay-proverif	proofs/anchor/delegation/chain-replay.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
anchor-token-verify-algconfusion-proverif	proofs/anchor/token-verify/algconfusion.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; a single model with two queries --the alg-pinned verifier authenticates (true) and the naive header-trusting verifier is forgeable (false) -- so it is its own positive/negative control pair without matching the *_vuln*/_naive_unsound* filename glob
bonded-federated-escrow-proverif	proofs/bonded/federated/federated.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
bonded-pairing-passkey-proverif	proofs/ bonded/ pairing/ passkey-pair. pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
bonded-recovery-magic-link-proverif	proofs/ bonded/ recovery/ magic-link.pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
coordination-isolation-proverif	proofs/ coordination/ isolation.pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
github-ingress-hpke-secrecy-proverif	analyses/ github_ ingress_hpke_ secrecy.pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
github- ingress- hpke- secrecy- vuln- proverif	analyses/ github_ ingress_hpke_ secrecy_vuln. pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
github- ingress- origin- auth- proverif	analyses/ github_ ingress_ origin_auth. pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
github- ingress- origin- auth- vuln- proverif	analyses/ github_ ingress_ origin_auth_ vuln.pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
github- ingress- tenant- isolation- proverif	analyses/ github_ ingress_ tenant_ isolation.pv	prover if- estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
github-ingress-tenant-isolation-vuln-proverif	analyses/github_ingress_tenant_isolation_vuln.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
guidance-envelope-v0-proverif	proofs/coordination_guidance_envelope_v0.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run; RESULT lines verified byte-for-byte. Relocated from docs/adr/models/to proofs/coordination/without re-running the prover. proofs/** is in proverif-estate's glob and path filter; lib/agent-harbor/suggestibility-authority.ts parses this transcript as the ADR-0096 suggestibility gate, so a drifted file caps that axis at C0.

(continued)

Claim	Artifact	CI	Status	Evidence policy
guidance-envelope-v0-unsigned-vuln-proverif	proofs/coordination/guidance_envelope_v0_unsigned_vuln.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run; RESULT lines verified byte-for-byte. Relocated from docs/adr/models/to proofs/coordination/without re-running the prover. A negative control that must keep failing: both agreement queries are committed 'is false.', the filena me still matches the runner's *_vuln*.pv glob, and a green run here would mean the check went vacuous.

(continued)

Claim	Artifact	CI	Status	Evidence policy
harbor-card-v1-hs256-proverif	analyses/harbor_card_v1.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte --this run found the committed transcript had drifted from the current .pv source (an earlier hs256/alg-header framing vs. the current generic hmac + Revoker process); the truth value of both queries is unchanged ('true'), only ProVerif's internal fresh-variable numbering shifted as a result. Regenerated to match the current source.
harbor-card-v1-refined-alg-pinning-proverif	analyses/harbor_card_v1_refined.pv	prover if-estate	CURRENT	no baseline existed before 2026-09-06; checked-in transcript established 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
harbor-card-v2-asymmetric-proverif	analyses/harbor_card_v2_asymmetric.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
harbor-card-v3-delegation-proverif	analyses/harbor_card_v3_delegation.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; proves ONE-HOP delegation soundness only -- see harbor-card-v6/v7 for the multi-hop finding this model does not cover
harbor-card-v4-escrow-secrecy-proverif	analyses/harbor_card_v4_escrow_secrecy.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
harbor-card-v5-attenuation-proverif	analyses/harbor_card_v5_attenuation.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
harbor-card-v6-multihop-attack-proverif	analyses/harbor_card_v6_multihop_attack.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; red-team probe -- proves a naive final<=root verifier ACCEPTS a non-monotonic multi-hop escalation (write->read->write). Refutes the Phase-3 claim in docs/reports/FORMAL_VERIFICATION_ANCHOR_V3.md; superseded by harbor-card-v7
harbor-card-v7-multihop-fixed-proverif	analyses/harbor_card_v7_multihop_fixed.pv	prover if-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; the fix for harbor-card-v6 --a per-hop each<=parent verifier rejects the same escalation

(continued)

Claim	Artifact	CI	Status	Evidence policy
macaroon-discharge-v1-proverif	analyses/macaroon_discharge_v1.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
macaroon-discharge-v2-naive-unsound-proverif	analyses/macaroon_discharge_v2_naive_unsound.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
pd-relay-zero-trust-skill-template-proverif	skills/pd-relay-zero-trust/templates/proverif-relay.pv	retired	HISTORICAL	a skill scaffold with open TODO queries (capability containment, revocation effectiveness, non-equivocation are all unfinished); not a claim about a real Port Daddy protocol and has no committed evidence file

(continued)

Claim	Artifact	CI	Status	Evidence policy
relay-e2e-secrecy-proverif	analyses/relay_e2e_secrecy.pv	proverif-estate	CURRENT	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; the injective (replay-free) SubAcc epts<=PubSent correspondence is reported false with the weaker non-injective correspondence proved true instead --this is the committed, expected shape of this model's evidence, not new drift

Table A.12: Python / R scripts artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor- results- r-scripts	skills/harbor- results/ scripts/ (19 files) a3_epsilon_ ledger.py a4_canary_sprt. py a6_no_mint.py a7_experiment. py b1_frontier.py b2_tower.py b3_ controllability. py b4_deontic_ fragment.py b5_engine_ substitution. py b6_probation. py b7_escalation_ band.py b8_ specialization. py b9_context_ paging.py c0_workunit.py c1_ noninterference. py sheaf_ mechanism_ proof.py sheaf_harness_ v2.py sheaf_ consistency_ radius.py zoom_bound_ check.py	harbor- results- estate	CURRENT	—

Table A.13: TLA+/TLC artifacts (4 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
bonded-conser- vation-tla	proofs/ bonded/ conservation/ (2 files) Conservation. tla Conservation. cfg	tla- conser- vation	CURRENT	CI artifact per run, plus a checked-in Conser- vation.run.log snapshot from a prior local run
econom- ics- claim- signal- ing- delta30- negative- control- tla	proofs/ economics/ claim_ signaling_ delta30.cfg	tla- claim- signal- ing	CURRENT	negative control: CI asserts the invariant NoUnil- ateralDeviationPositive is violated at delta=0.30 and that the printed counterexample's trace-state count matches the committed proofs/econom- ics/claim_signal- ing_ delta30.run.log snapshot (6 states); this is the trace cited by the ex:bonded-tla- two-deltas soluti- on in agent-transa- ctions-whitep- aper.tex
relay- card- revoca- tion-tla	proofs/relay/ (4 files) CardRevocation. tla CardRevocation_ baseline.cfg CardRevocation_ epoch.cfg CardRevocation_ rollback.cfg	tla- relay- card- revoca- tion	CURRENT	CI artifact per run (three .tlc.log files); no prior checked-in snapsh- ot --this model never ran before wave-p1/proof- estate. CardRe- vocation_rollba- ck.cfg is a negati- ve control asserted to violate Revoca- tionFinal (ADR- 0018 Attack Vector 2, backup/restore)

(continued)

Claim	Artifact	CI	Status	Evidence policy
relay-webhook-delivery-tla	proofs/relay/ (3 files) WebhookDelivery.tla WebhookDelivery.ry cfg WebhookDelivery_vuln.cfg	tla-relay-delive	CURRENT	CI artifact per run (two .tlc.log files); no prior checked-in snapshot --this model never ran before wave-p1/proof-estate. WebhookDelivery_vuln.cfg (Dedup=FALSE) is a negative control asserted to violate AtMost Once

Table A.14: TLA+/TLC + Apache artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
economics-claim-signaling-tla	proofs/economics/ (2 files) claim_signaling.tla claim_signaling.cfg	tla-claim-signaling, tla-claim-signaling-ing-apalache	CURRENT	CI artifacts per run (TLC log and Apache log, plus delta sweeps)

Table A.15: Z3 artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
economics-delta-threshold-old-z3	proofs/economics/delta-threshold.z3	z3-delta-threshold	CURRENT	expected sat/sat/unsat triple plus CI artifact; delta* interval independently cross-checked in proofs/economics/README.md and proofs/economics/delta-threshold.expected.txt

C Result atlas

The research archive contains revisions, experiments, literature work, and failed formulations. The durable unit is therefore a result, not a PDF. Each result below has a stable identifier, assumptions, a boundary, a chapter home, and a named verification artifact in the companion manifest.

C.1 The eight propositions and their evidence

1. Prevention begins at the effect boundary (R5, R8) — *The Single-Writer Kernel*. Controllability separates rules that a pre-commit gate can enforce from failures that can only be observed and remedied. The airlock experiments show why complete channel mediation is the decisive assumption. A token, label, or policy document does not control an effect channel by itself.

2. Delegation only narrows (ANCHOR-ATTENUATION, R5) — *The Anchor Protocol*. Every hop must authenticate the issuer and reject a capability that expands scope or lifetime. The cryptographic models close the stated monotonicity property; the engineering obligation is universal use at protected channels.

3. Confinement is a property of the room, not of the tenant (R9, R10, R11) — *The Sealed Harbor*. Two owners who distrust each other compute one answer inside two fences with a single metered slot. Noninterference modulo declassification holds on the finite room model; the release ledger conserves its ε budget and refuses the torn write; canaries detect exfiltration with a stated power curve and a Wald clock. What the slot may leak is priced as $q \cdot b$ bits before timing, and timing, cache, and physical channels are declared out of model rather than closed.

4. Supervision has an information cost (R1–R4, R14, R16) — *The Legible Swarm*. The information floor prices guaranteed recall; the dual-reader result shows why one ordering cannot generally serve both successor selection and operator attention; the regret rule derives intervention from asymmetric error cost; adaptive zoom and context paging describe when attention can be spent later rather than summarized away now. The bounds price guarantees under their stated event and flag models. They do not certify an arbitrary dashboard.

5. Accountability outlives the process (R7, R12, R13, B6) — *From Spawn to Person*. The inspection tower prices reputation as amortized verification. No-mint transfer semantics prevent skill or identity replacement from creating new authority. Budget-pooling and exposure arguments make reset laundering a property of the principal's

case file, not the process name. The research killed the stronger closure-weight formulation before restating the result correctly.

6. Exchange separates payment, collateral, evidence, and closure (R15, R17) — *The Harbor Economy*. Conservation supplies the accounting invariant; the tractable deontic fragment decides which rule sets can conflict and names the frontier past which that decision turns NP-complete; the succession rule has a price, because a sole owner's response time never falls below its residual-downtime floor, so pooling dominates past a computable threshold. These are conditional economic statements. Detection probability, one-shot gain, collateral, judge independence, and custody must be measured or contracted rather than implied.

7. Uncertainty needs an institution (the δ^* claim-signaling game, the conservation model; R11 as instrument) — *The Bonded Commons*. Atomic ledgers preserve conservation; truthful claim signaling is incentive-compatible above the mechanized discount threshold δ^* ; audit sampling and canaries borrow their power curve from the sealed room's detector; randomized, diverse adjudication reduces predictable collusion only under the named sampling assumptions. Evidence may support a remedy without becoming mechanical truth.

8. Federation relays evidence, not sovereignty (R6, R6-CR, RELAY-V0) — *The Federated Harbor*. Signed roots and redundant comparison paths can expose equivocation across a specified topology. The counterexample R6-CR rejects the tempting claim that a zero local residual proves global consistency. Current Relay establishes event transport and authentication; it does not close witness, secrecy, liveness, custody, or local authorization obligations.

C.2 A result is admitted only with its limit

For this volume, a claim is incomplete unless it states all five of the following: the object it concerns, the assumptions under which it holds, the kind of evidence offered, a boundary or falsifier, and the artifact by which a reader can recompute or inspect it. The machine-readable manifest is the publication ledger for that discipline. New mechanisms belong in the book only after they can name the result they instantiate and the obligation they leave unresolved.

D Notation and cross-chapter concordance

Work unit The durable case file joining authorization, capability history, evidence, obligations, outcomes, and settlement.

Harbor One local administrative and transactional authority, normally one daemon and its durable database.

Principal The human or organization whose policy and resources an agent ultimately exercises; not interchangeable with a process identity.

Role A bundle of obligations, capabilities, and authority that a spawn may fill.

Person A role instance plus continuity and a witnessed outcome history bound to an identity that cannot be freely re-picked.

Oracle An external or independently checkable reference used to close a commitment or adjudicate an outcome. Its trust model must be named.

Epoch Three clocks share the word. In *The Anchor Protocol* an epoch is a revocation counter, and a Card's validity window is a TTL; in *The Sealed Harbor* an epoch is an attestation freshness counter that detects rollback. None of the three converts into another.

Round A model step in a dissemination or protocol analysis, not automatically a wall-clock deadline.

Closed The exact recorded property has been mechanically discharged. No whole-system implication follows without a conformance argument.

Which chapter owns which primitive

The list above settles what a word means. This table settles something different and just as easy to get wrong across eight chapters: which chapter is allowed to *define* a shared object, and which chapters may only cite it. The rule is one definition per primitive, in its owning chapter; a chapter that needs a primitive it does not own states what it needs from it and refers, and if two chapters ever disagree the owner is the one to trust. Most of the duplication an outside reader has found in this book was a primitive defined twice rather than a fact stated twice, which is why this table exists at all.

Table A.16: Primitive ownership. “Owner” defines it and proves what is proved about it; “cites” names the chapters that use it without redefining it.

Primitive	Owner	Cites it
Authorization chain (attenuation, hop-binding, replay)	ch. 2, Anchor Protocol	ch. 1, 5, 7, 8
Coordination lineage (task graph, loop detection)	ch. 1, Single-Writer Kernel	ch. 4
Leakage budget and the priced exit	ch. 3, Sealed Harbor	ch. 4, 6
Float plan	ch. 6, Harbor Economy	ch. 7, 8
Bond, escrow, and the commons pool	ch. 7, Bonded Commons	ch. 6, 8
Cross-harbor transfer ceremony	ch. 8, Federated Harbor	ch. 6
Witness log, epoch root, equivocation residual	ch. 8, Federated Harbor	ch. 2, 7
Threat bands and the visibility tiers	ch. 8, Federated Harbor	ch. 3
Keystone identity (non-forgeable principal)	ch. 5, From Spawn to Person	ch. 6, 7
Inspection tower and grader escalation	ch. 5, From Spawn to Person	ch. 1, 6

Open problems, all three lists in one place

Open problems are numbered in three separate namespaces on purpose — chapter 1’s OP-*n* is the kernel’s, chapter 5 declines to extend it because its questions are not kernel questions, and chapter 8 states its own in prose — and the cost of that decision is that no reader can see the whole set at once. This concordance indexes them; it does not renumber them, and the owning chapter’s table stays the one to trust for status.

Table A.17: Open-problem concordance. Statuses are the owning chapter's own, as of this edition; where a later chapter bears on a problem without closing it, the last column says how.

Where	Problem	What bears on it elsewhere
ch. 1, OP-1–OP-11	The kernel's eleven, stated once in its own status table; OP-4 splits into OP-4a (durable lineage replay) and OP-4b (inference-state recovery).	OP-4b is graded on ch. 5's agent-death ladder, rung 5; OP-5 (oracle <i>completeness</i>) is untouched by ch. 5's tower, which is about oracle <i>incentives</i> .
ch. 5	Its own list, deliberately outside the kernel's numbering: death taxonomy, neutral graded outcomes, judge conflict of interest, correlated auditors.	The correlated-auditor question shares ch. 6's mixing model and its unmeasured θ .
ch. 8	The federation limitations section: deployment-specific propagation tail bounds, and reconciling sheaf diffusion with epidemic gossip.	The second now carries a falsifier (rounds-to-first-alarm against $1/\lambda_2$) and no scheduled run.
Cross-chapter	Claim signalling under <i>private</i> monitoring: ch. 7's equilibrium assumes perfect public monitoring, which one harbor's ledger supplies and gossip does not.	Open in both ch. 7 and ch. 8; neither re-derives it, and ch. 8's settlement uses bonds rather than triggers to avoid needing it.

References

- [1] Alfarez Abdul-Rahman. The PGP Trust Model. *EDI-Forum: The Journal of Electronic Commerce*, 10(3):27–31, 1997. ↑ Cited on page 511.
- [2] Samson Abramsky and Adam Brandenburger. The operational-sheaf-theoretic approach to contextuality and non-locality. *New Journal of Physics*, 13(11):113036, 2011. ↑ Cited on page 488.
- [3] D. Abreu, D. Pearce & E. Stacchetti. “Toward a Theory of Discounted Repeated Games with Imperfect Monitoring.” *Econometrica* 58(5):1041–1063, 1990. (With Green & Porter, *Econometrica* 1984.) ↑ Cited on pages 361, 374, and 376.
- [4] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight Virtualization for Serverless Applications. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. ↑ Cited on page 73.
- [5] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain (SD-JWT+kb checkout mandate, JWS detached-content merchant authorization, JCS canonicalization), 2026. <https://ap2-protocol.org>; UCP extension at ucp.dev/documentation/ucp-and-ap2/. ↑ Cited on pages 374 and 376.
- [6] Agentic Commerce Protocol. OpenAI & Stripe, 2025. <https://agenticcommerce.dev>. ↑ Cited on pages 374 and 376.
- [7] George A. Akerlof. The Market for “Lemons”: Quality Uncertainty and the Market Mechanism. *Quarterly Journal of Economics*, 84(3):488–500, 1970. ↑ Cited on pages 262, 287, 350, and 374.
- [8] Arnold O. Allen. *Probability, Statistics, and Queueing Theory with Computer Science Applications*. Academic Press, 2nd edition, 1990. (The Allen–Cunneen approximation for $M/G/c$, and the Pollaczek–Khinchine formula.) ↑ Cited on page 193.
- [9] M. S. Alvim, M. E. Andrés, K. Chatzikokolakis, and C. Palamidessi. On the relation between differential privacy and quantitative information flow. In *Proc. ICALP 2011*, LNCS 6756, pages 60–76. Springer, 2011. ↑ Cited on pages 154 and 168.
- [10] James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.) ↑ Cited on pages 5 and 73.
- [11] ANSI/ISA. *ANSI/ISA-18.2: Management of Alarm Systems for the Process Industries*. 2016. ↑ Cited on pages 196 and 246.

- [12] Anthropic. Effective Context Engineering for AI Agents. 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> ↑ Cited on pages 229, 229, and 247.
- [13] M. Armstrong. “Competition in Two-Sided Markets.” *RAND Journal of Economics* 37(3):668–691, 2006. ↑ Cited on page 373.
- [14] S. Arnaudov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumaran, D. O’Keeffe, M. L. Stillwell, D. Goltzsche, D. Eysers, R. Kapitza, P. Pietzuch, and C. Fetzner. SCONE: secure Linux containers with Intel SGX. In *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 689–703, 2016. ↑ Cited on page 132.
- [15] A. Askarov, D. Zhang, and A. C. Myers. Predictive black-box mitigation of timing channels. In *Proc. 17th ACM Conference on Computer and Communications Security (CCS)*, pages 297–307, 2010. ↑ Cited on page 160.
- [16] George K. Atia and Venkatesh Saligrama. Boolean compressed sensing and noisy group testing. *IEEE Transactions on Information Theory*, 58(3):1880–1901, 2012. ↑ Cited on page 545.
- [17] Robert J. Aumann. Subjectivity and Correlation in Randomized Strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974. [https://doi.org/10.1016/0304-4068\(74\)90037-8](https://doi.org/10.1016/0304-4068(74)90037-8) ↑ Cited on pages 390 and 396.
- [18] AWS s2n-tls. An Implementation of the TLS/SSL Protocols. <https://github.com/aws/s2n-tls>, 2022. ↑ Cited on page 100.
- [19] AWS Automated Reasoning Group. Kani Rust Verifier. <https://github.com/model-checking/kani>, 2022. ↑ Cited on pages 104, 112, and 115.
- [20] Eric Bach. Sheaf Cohomology is #P-hard. *Journal of Symbolic Computation*, 27(4):429–433, 1999. ↑ Cited on page 494.
- [21] Lisanne Bainbridge. Ironies of Automation. *Automatica*, 19(6):775–779, 1983. ↑ Cited on pages 204 and 246.
- [22] Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. SoK: Computer-Aided Cryptography. In *IEEE Symposium on Security and Privacy (S&P)*, 2021. ↑ Cited on page 127.
- [23] G. Barthe, P. R. D’Argenio, and T. Rezk. Secure information flow by self-composition. In *Proc. 17th IEEE Computer Security Foundations Workshop (CSFW)*, p. 100, 2004. ↑ Cited on pages 143, 144, and 167.

- [24] David Basin, Jannik Dreier, Lucca Hirschi, Saša Radomirovic, Ralf Sasse, and Vincent Stettler. A Formal Analysis of 5G Authentication. In *ACM Conference on Computer and Communications Security (CCS)*, 2018. ↑ Cited on page 127.
- [25] G. Becker. “Crime and Punishment: An Economic Approach.” *Journal of Political Economy* 76(2):169–217, 1968. ↑ Cited on pages 375 and 379.
- [26] Bengt Holmström. Moral hazard in teams. *The Bell Journal of Economics*, 13(2):324–340, 1982. ↑ Cited on page 446.
- [27] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. (Ed25519 — the signature scheme underlying the authorization chain.) ↑ Cited on pages 75, 89, 105, and 474.
- [28] Philip A. Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. Orleans: Distributed Virtual Actors for Programmability and Scalability. Microsoft Research Technical Report MSR-TR-2014-41, 2014. ↑ Cited on page 273.
- [29] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, et al. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In *Logic in Computer Science (LICS)*, 2017. ↑ Cited on page 100.
- [30] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrbale, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. <https://doi.org/10.14722/ndss.2014.23212> ↑ Cited on pages 60, 89, 106, 112, 112, 115, 127, 183, 447, and 512.
- [31] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. <https://doi.org/10.1561/33000000004> ↑ Cited on pages 101, 104, 110, 112, 115, and 127.
- [32] Bruno Blanchet. CryptoVerif: A Computationally Sound Security Protocol Verifier. INRIA Research Report, 2007. ↑ Cited on page 127.
- [33] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. (The class of tool in which the authorization chain’s secrecy and integrity properties are mechanically verified.) ↑ Cited on pages 75, 401, and 509.

- [34] Bruno Blanchet, Vincent Cheval, and Véronique Cortier. ProVerif with Lemmas, Induction, Fast Subsumption, and Much More. In *IEEE Symposium on Security and Privacy (S&P)*, 2022. ↑ Cited on page 127.
- [35] Juan Brackeva et al. Headscale: Open-Source Tailscale Coordination Server. GitHub project, 2020–present. ↑ Cited on page 511.
- [36] Ralph A. Bradley and Milton E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4):324–345, 1952. ↑ Cited on pages 261, 262, and 295.
- [37] Billy Bob Brumley and Nicola Taveri. Remote Timing Attacks are Still Practical. In *European Symposium on Research in Computer Security (ESORICS)*, 2011. ↑ Cited on page 112.
- [38] Ethan Buchman. Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. M.A.Sc. thesis, University of Guelph, 2016. ↑ Cited on page 391.
- [39] Mike Burrows. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2006. ↑ Cited on page 18.
- [40] Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum white paper, 2014. ↑ Cited on page 392.
- [41] Vitalik Buterin and Virgil Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017. ↑ Cited on page 391.
- [42] W. Cai, T. Shi, X. Zhao, and D. Song. Are You Getting What You Pay For? Auditing Model Substitution in LLM APIs. arXiv:2504.04715, 2025. ↑ Cited on page 289.
- [43] Donald T. Campbell. Assessing the Impact of Planned Social Change. *Evaluation and Program Planning*, 2(1):67–90, 1979. ↑ Cited on pages 203 and 246.
- [44] Giovanni Carù. On the Cohomology of Contextuality. In *Proc. 13th International Conference on Quantum Physics and Logic (QPL 2016)*, EPTCS 236, pp. 21–39, 2017. ↑ Cited on page 495.
- [45] Tushar Deepak Chandra and Sam Toueg. Unreliable Failure Detectors for Reliable Distributed Systems. *Journal of the ACM*, 43(2):225–267, 1996. ↑ Cited on pages 71 and 74.
- [46] K. Chatterjee & W. Samuelson. “Bargaining under Incomplete Information.” *Operations Research* 31(5):835–851, 1983. (The linear-equilibrium double auction used in the Myerson–Satterthwaite worked example.) ↑ Cited on pages 355 and 373.

- [47] Alice Cheng and Eric Friedman. Sybilproof Reputation Mechanisms. In *Proc. ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON)*, 2005. ↑ Cited on page 279.
- [48] Mahdi Cheraghchi. Noise-resilient group testing: limitations and constructions. *Discrete Applied Mathematics*, 161(1–2):81–95, 2013. ↑ Cited on page 545.
- [49] Wei-Fan Chiu, Yuxuan Zhang, and Mihaela van der Schaar. Strategic Self-Improvement for Competitive Agents in AI Labour Markets. arXiv:2512.04988, 2025. ↑ Cited on pages 320, 337, and 373.
- [50] Philip R. Cohen and Hector J. Levesque. Intention Is Choice with Commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990. ↑ Cited on pages 42 and 74.
- [51] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM Conference on Computer and Communications Security (CCS)*, 2017. ↑ Cited on page 127.
- [52] Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. In *USENIX Security Symposium*, 2009. ↑ Cited on page 513.
- [53] Maria Cvach. Monitor Alarm Fatigue: An Integrative Review. *Biomedical Instrumentation & Technology*, 46(4):268–277, 2012. ↑ Cited on pages 196 and 246.
- [54] CVE-2015-9235. Algorithm Confusion in jsonwebtoken Node.js library. NIST National Vulnerability Database, 2015. ↑ Cited on pages 87, 104, and 128.
- [55] CVE-2018-1000531. JWT “none” algorithm vulnerability in jwtf library. NIST National Vulnerability Database, 2018. ↑ Cited on page 128.
- [56] CVE-2023-48238. Algorithm confusion in json-web-token library. NIST National Vulnerability Database, 2023. ↑ Cited on pages 87 and 128.
- [57] Gordon Dai, Weijia Zhang, Jinhan Li, Siqi Yang, Chidera Onochie Ibe, Srihas Rao, Arthur Caetano, and Misha Sra. Artificial Leviathan: Exploring Social Evolution of LLM Agents Through the Lens of Hobbesian Social Contract Theory. arXiv:2406.14373, 2024. <https://arxiv.org/abs/2406.14373> ↑ Cited on pages 174, 241, and 246.
- [58] DataDome. “Agent Trust Management and the Universal Commerce Protocol,” 2026. datadome.co/agent-trust-management/universal-commerce-protocol/. ↑ Cited on pages 374 and 447.

- [59] R. Dechter, I. Meiri & J. Pearl. “Temporal Constraint Networks.” *Artificial Intelligence* 49(1–3):61–95, 1991. ↑ Cited on page [345](#).
- [60] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987. ↑ Cited on pages [95](#), [117](#), [359](#), [374](#), [456](#), [481](#), [481](#), [483](#), [495](#), and [522](#).
- [61] D. E. Denning and P. J. Denning. Certification of programs for secure information flow. *Communications of the ACM*, 20(7):504–513, 1977. ↑ Cited on pages [134](#) and [167](#).
- [62] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966. ↑ Cited on pages [85](#), [127](#), [266](#), [391](#), [447](#), and [513](#).
- [63] Danny Dolev and Andrew Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983. ↑ Cited on pages [101](#), [110](#), [127](#), and [473](#).
- [64] Robert Dorfman. The Detection of Defective Members of Large Populations. *Annals of Mathematical Statistics*, 14(4):436–440, 1943. ↑ Cited on pages [210](#) and [216](#).
- [65] Marco Dorigo and Gianni Di Caro. The Ant Colony Optimization Meta-Heuristic. In *New Ideas in Optimization*, McGraw-Hill, 1999. ↑ Cited on pages [25](#) and [74](#).
- [66] John R. Douceur. The Sybil Attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002. ↑ Cited on pages [278](#), [278](#), [279](#), [472](#), and [501](#).
- [67] John R. Douceur. The Sybil Attack. In *IPTPS*, 2002. ↑ Cited on pages [235](#) and [247](#).
- [68] J. Douceur. “The Sybil Attack.” *IPTPS* 2002. ↑ Cited on pages [353](#) and [374](#).
- [69] W. F. Dowling & J. H. Gallier. “Linear-Time Algorithms for Testing the Satisfiability of Propositional Horn Formulae.” *Journal of Logic Programming* 1(3):267–284, 1984. ↑ Cited on page [343](#).
- [70] Ding-Zhu Du and Frank K. Hwang. *Combinatorial Group Testing and Its Applications*, 2nd ed. World Scientific, 2000. ↑ Cited on page [216](#).
- [71] C. Dwork, N. Lynch & L. Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM* 35(2):288–323, 1988. ↑ Cited on pages [358](#) and [374](#).

- [72] C. Dwork, G. N. Rothblum, and S. Vadhan. Boosting and differential privacy. In *Proc. 51st IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 51–60, 2010. ↑ Cited on pages 153 and 168.
- [73] Carl M. Ellison, Bill Frantz, Butler Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylonen. SPKI Certificate Theory. IETF RFC 2693, 1999. ↑ Cited on pages 92, 391, and 513.
- [74] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, 1978. ↑ Cited on pages 261, 262, and 295.
- [75] Mica R. Endsley and Esin O. Kiris. The Out-of-the-Loop Performance Problem and Level of Control in Automation. *Human Factors*, 37(2):381–394, 1995. ↑ Cited on pages 203 and 246.
- [76] Mica R. Endsley. Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors*, 37(1):32–64, 1995. ↑ Cited on pages 203 and 246.
- [77] A. K. Erlang. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektrotekniker*, 13, 1917. (The Erlang-C delay formula.) ↑ Cited on page 190.
- [78] James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.) ↑ Cited on page 167.
- [79] Ethereum Foundation. Ethereum 2.0 Phase 0 – The Beacon Chain Specification. <https://github.com/ethereum/consensus-specs>, 2020. ↑ Cited on page 391.
- [80] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014. ↑ Cited on pages 93 and 456.
- [81] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2):374–382, 1985. ↑ Cited on pages 6, 74, 358, and 374.
- [82] B. Fong & D. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, 2019. ↑ Cited on pages 366 and 374.
- [83] Foundation for Intelligent Physical Agents. *FIPA Agent Management Specification* (SC00023). 2002. <http://www.fipa.org/specs/fipa00023/> ↑ Cited on pages 220 and 247.

- [84] Eric J. Friedman and Paul Resnick. The Social Cost of Cheap Pseudonyms. *Journal of Economics & Management Strategy*, 10(2):173–199, 2001. ↑ Cited on pages 262 and 279.
- [85] Drew Fudenberg and David K. Levine. Reputation and Equilibrium Selection in Games with a Patient Player. *Econometrica*, 57(4):759–778, 1989. ↑ Cited on page 310.
- [86] H. Garcia-Molina & K. Salem. “Sagas.” *ACM SIGMOD* 1987. ↑ Cited on pages 361, 374, and 376.
- [87] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987. ↑ Cited on pages 57, 72, 74, 311, and 447.
- [88] David Gelernter. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985. ↑ Cited on pages 25 and 74.
- [89] Christopher Goes. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918, 2020. ↑ Cited on page 513.
- [90] J. A. Goguen and J. Meseguer. Security policies and security models. In *Proc. IEEE Symposium on Security and Privacy (Oakland)*, pp. 11–20, 1982. ↑ Cited on pages 143 and 167.
- [91] J. A. Goguen and J. Meseguer. Unwinding and inference control. In *Proc. IEEE Symposium on Security and Privacy*, 1984. ↑ Cited on pages 143 and 167.
- [92] S. Goldwasser, Y. Kalai & G. Rothblum. “Delegating Computation: Interactive Proofs for Muggles.” *STOC*, pp. 613–622, 2008. ↑ Cited on pages 375, 376, and 377.
- [93] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles...: la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959. ↑ Cited on pages 25, 74, and 235.
- [94] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. (Atomicity/consistency vs. durability — the ground for I1a/I1b.) ↑ Cited on pages 13 and 74.
- [95] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993. ↑ Cited on page 413.
- [96] David M. Green and John A. Swets. *Signal Detection Theory and Psychophysics*. Wiley, 1966. ↑ Cited on pages 200 and 246.
- [97] Sanford J. Grossman and Oliver D. Hart. The Costs and Benefits of Ownership: A Theory of Vertical and Lateral Integration. *Journal of Political Economy*, 94(4):691–719, 1986. ↑ Cited on pages 264, 342, and 374.

- [98] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, IETF, January 2023. ↑ Cited on pages 135 and 168.
- [99] Stuart Haber and W. Scott Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99–111, 1991. ↑ Cited on pages 48 and 75.
- [100] Jakob Hansen and Robert Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019. ↑ Cited on page 495.
- [101] HashiCorp. Consul Service Discovery and DNS Interface (health-checked, TTL-expiring registry). <https://developer.hashicorp.com/consul/docs/discover> ↑ Cited on page 228.
- [102] Friedrich A. Hayek. The Use of Knowledge in Society. *American Economic Review*, 35(4):519–530, 1945. ↑ Cited on pages 239 and 246.
- [103] Roland Hedberg (ed.), Michael B. Jones, Andreas Åkre Solberg, John Bradley, Giuseppe De Marco, and Vladimir Dzhuvinov. OpenID Federation 1.0. OpenID Foundation Implementer’s Draft 4, July 2024. ↑ Cited on page 512.
- [104] E. Heilman, A. Kendler, A. Zohar & S. Goldberg. “Eclipse Attacks on Bitcoin’s Peer-to-Peer Network.” *USENIX Security* 2015. ↑ Cited on pages 360 and 374.
- [105] Pat Helland. Life beyond Distributed Transactions: an Apostate’s Opinion. In *Conference on Innovative Data Systems Research (CIDR)*, 2007. ↑ Cited on page 53.
- [106] Ralf Herbrich, Tom Minka, and Thore Graepel. TrueSkill: A Bayesian Skill Rating System. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2007. ↑ Cited on pages 261, 262, and 295.
- [107] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. ↑ Cited on pages 56 and 74.
- [108] Maurice Herlihy. Atomic Cross-Chain Swaps. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 2018. ↑ Cited on pages 358, 374, 376, 468, 497, 513, and 516.
- [109] M. Herlihy, B. Liskov & L. Shrira. “Cross-Chain Deals and Adversarial Commerce.” *The VLDB Journal* 31:1291–1309, 2022. ↑ Cited on pages 359, 374, and 376.
- [110] Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019. ↑ Cited on page 413.

- [111] D. Richard Hipp et al. SQLite Write-Ahead Logging. SQLite Documentation, 2010. <https://www.sqlite.org/wal.html> (synchronous=NORMAL “commits might lose durability following a power loss.”) ↑ Cited on pages 13 and 74.
- [112] Albert O. Hirschman. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*. Harvard University Press, 1970. ↑ Cited on pages 183, 239, and 246.
- [113] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651. ↑ Cited on pages 174, 178, 179, 181, 237, 246, 388, and 446.
- [114] Thomas Hobbes. *Leviathan*. 1651. ↑ Cited on page 285.
- [115] *Hobbes’s Moral and Political Philosophy*. Stanford Encyclopedia of Philosophy. <https://plato.stanford.edu/entries/hobbes-moral/> ↑ Cited on pages 178, 179, and 246.
- [116] David Hume. Of the Original Contract. 1748. ↑ Cited on pages 180 and 246.
- [117] T. Hunt, Z. Zhu, Y. Xu, S. Peter, and E. Witchel. Ryoan: a distributed sandbox for untrusted computation on secret data. In *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 533–549, 2016. ↑ Cited on pages 132 and 168.
- [118] Frank K. Hwang. A Method for Detecting All Defective Members in a Population by Group Testing. *Journal of the American Statistical Association*, 67(339):605–608, 1972. ↑ Cited on pages 210 and 216.
- [119] G. Irving, P. Christiano & D. Amodei. “AI Safety via Debate.” arXiv:1805.00899, 2018. ↑ Cited on pages 375, 376, and 377.
- [120] ISO/IEC 15408-1:2022. *Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model* (Common Criteria, CC:2022). International Organization for Standardization, 2022. ↑ Cited on pages 40, 151, and 169.
- [121] Joseph Izraelevitz, Hammurabi Mendes, and Michael L. Scott. Linearizability of Persistent Memory Objects under a Full-System-Crash Failure Model. In *Proceedings of the 30th International Symposium on Distributed Computing (DISC)*, pages 313–327, 2016. ↑ Cited on page 56.
- [122] J. Rushby. Noninterference, transitivity, and channel-control security policies. Technical Report CSL-92-02, SRI International, December 1992. ↑ Cited on pages 143, 164, 167, and 169.

- [123] Xiaoye Jiang, Lek-Heng Lim, Yuan Yao, and Yinyu Ye. Statistical ranking and combinatorial Hodge theory. *Mathematical Programming*, 127(1):203–244, 2011. ↑ Cited on page 487.
- [124] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science*, Wiley, 1993. ↑ Cited on pages 27, 74, and 401.
- [125] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). *RFC 8032*, IETF, January 2017. <https://tools.ietf.org/html/rfc8032> ↑ Cited on page 89.
- [126] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan. Fast portscan detection using sequential hypothesis testing. In *Proc. IEEE Symposium on Security and Privacy*, 2004. ↑ Cited on pages 157 and 168.
- [127] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The EigenTrust Algorithm for Reputation Management in P2P Networks. In *International World Wide Web Conference (WWW)*, 2003. ↑ Cited on pages 227, 247, 261, 262, and 295.
- [128] Michihiro Kandori and Hitoshi Matsushima. Private observation, communication and collusion. *Econometrica*, 66(3):627–652, 1998. ↑ Cited on page 420.
- [129] David G. Kendall. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain. *Annals of Mathematical Statistics*, 24(3):338–354, 1953. (The $A/S/c$ notation.) ↑ Cited on page 189.
- [130] Jeffrey O. Kephart and David M. Chess. The Vision of Autonomic Computing. *IEEE Computer*, 36(1):41–50, 2003. ↑ Cited on page 75.
- [131] J. F. C. Kingman. The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society*, 57(4):902–904, 1961. (Waiting time grows with the sum of the squared coefficients of variation.) ↑ Cited on page 193.
- [132] Gary Klein. *Sources of Power: How People Make Decisions*. MIT Press, 1998. ↑ Cited on page 201.
- [133] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal Verification of an OS Kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009. ↑ Cited on page 73.

- [134] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017. (Single-leader as the right default; consensus only when forced.) ↑ Cited on pages [6](#) and [74](#).
- [135] Martin Kleppmann. How to do distributed locking. *martin.kleppmann.com*, February 2016; folded into *Designing Data-Intensive Applications*, O'Reilly, 2017. ↑ Cited on page [18](#).
- [136] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017. ↑ Cited on page [127](#).
- [137] Fred Kofman and Jacques Lawarrée. Collusion in Hierarchical Agency. *Econometrica*, 61(3):629–656, 1993. ↑ Cited on page [305](#).
- [138] David M. Kreps and Robert Wilson. Reputation and Imperfect Information. *Journal of Economic Theory*, 27(2):253–279, 1982. ↑ Cited on page [310](#).
- [139] Offchain Labs. Interactive Fraud Proofs (Arbitrum bisection dispute protocol). Technical documentation, ongoing. ↑ Cited on page [305](#).
- [140] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978. ↑ Cited on page [66](#).
- [141] Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998. ↑ Cited on page [452](#).
- [142] Leslie Lamport. *Specifying Systems: The TLA⁺ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ↑ Cited on page [439](#).
- [143] Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974. ↑ Cited on pages [5](#) and [73](#).
- [144] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. ↑ Cited on page [296](#).
- [145] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. IETF RFC 6962, June 2013. ↑ Cited on pages [374](#), [404](#), [449](#), [472](#), [481](#), [511](#), and [513](#).
- [146] B. Laurie. “Certificate Transparency.” *Communications of the ACM* 57(10):40–46, 2014. ↑ Cited on pages [358](#), [374](#), and [376](#).
- [147] John D. Lee and Katrina A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1):50–80, 2004. ↑ Cited on pages [202](#) and [246](#).

- [148] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou (eds.). ActivityPub. W3C Recommendation, 2018. ↑ Cited on page [513](#).
- [149] Sophie Leroy. Why Is It So Hard to Do My Work? The Challenge of Attention Residue When Switching Between Work Tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009. ↑ Cited on page [204](#).
- [150] Raph Levien and Alexander Aiken. Attack-Resistant Trust Metrics for Public Key Certification. In *Proceedings of the 7th USENIX Security Symposium*, 1998. Deployed as the Advogato trust metric, 1999. ↑ Cited on page [270](#).
- [151] F. Lin and W. M. Wonham. On observability of discrete-event systems. *Information Sciences*, 44(3):173–198, 1988. ↑ Cited on pages [151](#) and [168](#).
- [152] Feng Lin and W. Murray Wonham. On observability of discrete-event systems. *Information Sciences*, 44(3):173–198, 1988. ↑ Cited on page [39](#).
- [153] Linux Foundation / OpenSSF. SLSA Specification v1.0: Supply-chain Levels for Software Artifacts, 2023. ↑ Cited on pages [511](#) and [513](#).
- [154] Michael Litzkow, Todd Tannenbaum, Jim Basney, and Miron Livny. Checkpoint and Migration of UNIX Processes in the Condor Distributed Processing System. University of Wisconsin–Madison Computer Sciences Technical Report #1346, 1997. ↑ Cited on page [273](#).
- [155] Qingmin Liu and Andrzej Skrzypacz. Limited Records and Reputation Bubbles. *Journal of Economic Theory*, 151:2–29, 2014. ↑ Cited on pages [262](#), [297](#), [314](#), [361](#), and [374](#).
- [156] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 12:157–173, 2024. arXiv:2307.03172. ↑ Cited on pages [229](#) and [247](#).
- [157] John Locke. An Essay Concerning Human Understanding. 1689. (Bk II, ch. xxvii, “Of Identity and Diversity.”) ↑ Cited on pages [261](#) and [266](#).
- [158] John Locke. *Second Treatise of Government*, §§95–122. 1689. ↑ Cited on pages [237](#) and [246](#).
- [159] John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689. ↑ Cited on page [446](#).

- [160] Gavin Lowe. A Hierarchy of Authentication Specifications. In *IEEE Computer Security Foundations Workshop (CSFW)*, 1997. ↑ Cited on pages 104, 112, and 127.
- [161] T. Luo, M. Pan, P. Tholoniati, A. Cidon, R. Geambasu, and M. Lécuyer. Privacy budget scheduling. In *Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 55–74, 2021. ↑ Cited on pages 154 and 168.
- [162] Thodoris Lykouris and Sergei Vassilvitskii. Competitive Caching with Machine Learned Advice. In *ICML*, 2018. arXiv:1802.05399; *Journal of the ACM*, 2021. ↑ Cited on page 234.
- [163] Norman H. Mackworth. The Breakdown of Vigilance During Prolonged Visual Search. *Quarterly Journal of Experimental Psychology*, 1(1):6–21, 1948. ↑ Cited on pages 203 and 246.
- [164] George J. Mailath and Larry Samuelson. Who Wants a Good Reputation? *Review of Economic Studies*, 68(2):415–441, 2001. ↑ Cited on page 287.
- [165] Eve Maler and Drummond Reed. The Venn of Identity: Options and Issues in Federated Identity Management. *IEEE Security & Privacy*, 6(2):16–23, 2008. ↑ Cited on pages 476, 504, 504, 511, 511, 512, and 517.
- [166] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain: SD-JWT+kb checkout mandate, JWS detached-content merchant authorization, JCS canonicalization, 2026. <https://ap2-protocol.org>. ↑ Cited on pages 126, 294, 447, and 514.
- [167] The Matrix.org Foundation. The Matrix Specification. <https://spec.matrix.org>. ↑ Cited on page 513.
- [168] Tim McLean. Critical Vulnerabilities in JSON Web Token Libraries. Auth0 Blog, 2015. <https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/> ↑ Cited on pages 105, 112, and 128.
- [169] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*, 2013. ↑ Cited on page 127.
- [170] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO ’87*, 1987. ↑ Cited on pages 48 and 75.
- [171] Bertrand Meyer. Applying “Design by Contract”. *IEEE Computer*, 25(10):40–51, 1992. ↑ Cited on page 194.

- [172] Silvio Micali, Michael Rabin, and Salil Vadhan. Verifiable Random Functions. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 120–130. IEEE, 1999. ↑ Cited on page 307.
- [173] George A. Miller. The Magical Number Seven, Plus or Minus Two. *Psychological Review*, 63(2):81–97, 1956. ↑ Cited on pages 200 and 246.
- [174] M. S. Miller & K. E. Drexler. “Markets and Computation: Agoric Open Systems.” In B. A. Huberman (ed.), *The Ecology of Computation*, pp. 133–176. North-Holland, 1988. ↑ Cited on page 329.
- [175] Mark S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006. ↑ Cited on pages 92, 391, and 513.
- [176] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. ↑ Cited on pages 74 and 413.
- [177] A. C. Myers and B. Liskov. A decentralized model for information flow control. In *Proc. 16th ACM Symposium on Operating Systems Principles (SOSP)*, 1997. ↑ Cited on pages 138 and 167.
- [178] Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981. ↑ Cited on pages 262, 373, and 447.
- [179] R. Myerson & M. Satterthwaite. “Efficient Mechanisms for Bilateral Trading.” *Journal of Economic Theory* 29(2):265–281, 1983. ↑ Cited on pages 355 and 373.
- [180] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security (CCS)*, 2022. ↑ Cited on pages 472, 511, and 513.
- [181] Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani (eds.). *Algorithmic Game Theory*. Cambridge University Press, 2007. ↑ Cited on pages 235, 262, 297, and 374.
- [182] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. 2nd ed., Pragmatic Bookshelf, 2018. (The bounded busy-wait as a bulkhead / timeout-budget under contention.) ↑ Cited on page 75.
- [183] Eric T. Olson. *What Are We? A Study in Personal Ontology*. Oxford University Press, 2007. ↑ Cited on page 268.

- [184] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014. ↑ Cited on pages [178](#) and [452](#).
- [185] E. Ostrom. *Governing the Commons*. Cambridge University Press, 1990. ↑ Cited on page [374](#).
- [186] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. ↑ Cited on pages [235](#), [247](#), [314](#), [392](#), and [445](#).
- [187] Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Curiositech LLC Technical White Paper, May 2026. ↑ Cited on pages [391](#), [397](#), [398](#), [399](#), [401](#), [418](#), [419](#), [420](#), [447](#), [466](#), and [512](#).
- [188] Erich Owens. The Bonded Commons: Pre-Transactional Trust Infrastructure for Multi-Agent Systems. Curiositech LLC Technical White Paper, May 2026. ↑ Cited on pages [466](#), [469](#), [516](#), and [522](#).
- [189] Erich Owens. The Cohomology of Equivocation: Detecting Split-View Lies in Federated Witness-Log Gossip by Sheaf Consistency. The Harbor Research Program, Paper 7, August 2026. ↑ Cited on page [487](#).
- [190] Erich Owens. The Federated Harbor: Proposal and Annotated Bibliography. Curiositech LLC working document, May 2026. ↑ Cited on pages [468](#), [485](#), [516](#), and [523](#).
- [191] Erich Owens. *The Price of a Summary: Information-Theoretic Limits of Agent Oversight*. The Harbor Research Program, Paper 1, 2026. Companion formal paper to this chapter; its Theorems 1–4 are the proved versions of several results stated here. ↑ Cited on pages [211](#) and [247](#).
- [192] Erich Owens. *What Needs an Authority: Mechanical Detection, Chartered Resolution, and the Exact Price of Sole Ownership*. The Harbor Research Program, Paper 6, 2026. Its Theorem 2 is the Erlang-C specialization boundary, together with the sweep that falsified this paper’s earlier proposed threshold. ↑ Cited on pages [190](#), [247](#), [248](#), and [342](#).
- [193] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 30(3):286–297, 2000. ↑ Cited on pages [40](#) and [181](#).
- [194] R. Parasuraman, T. B. Sheridan, and C. D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 30(3):286–297, 2000. ↑ Cited on pages [150](#) and [169](#).

- [195] Derek Parfit. *Reasons and Persons*. Oxford University Press, 1984.
↑ Cited on pages [261](#) and [267](#).
- [196] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
↑ Cited on pages [174](#), [246](#), [272](#), and [447](#).
- [197] Carole Pateman. *The Problem of Political Obligation: A Critique of Liberal Theory*. Wiley, 1979. ↑ Cited on pages [239](#) and [246](#).
- [198] Avery Pennarun et al. How Tailscale Works. Tailscale Engineering Blog, March 2020. ↑ Cited on page [511](#).
- [199] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Symposium on Operating Systems Design and Implementation (OSDI)*, pages 433–448, 2014. ↑ Cited on page [74](#).
- [200] Peter Pirolli and Stuart Card. Information Foraging. *Psychological Review*, 106(4):643–675, 1999. ↑ Cited on page [247](#).
- [201] Michael Polanyi. *The Tacit Dimension*. Doubleday, 1966. ↑ Cited on page [185](#).
- [202] Port Daddy Harbor research program. *Continuity Without Metaphysics*. Companion formal paper (Paper 5), August 2026. Source of the no-mint, resurrection-soundness, and probation-cliff theorems restated in the sections on continuity and identity. ↑ Cited on pages [285](#) and [286](#).
- [203] Port Daddy Harbor research program. *Reputation is Amortized Verification: Inspection Games for Agent Economies*. Companion formal paper (Paper 3), August 2026. Source of the stage-deterrence and tower-contraction theorems restated in the section on grading-oracle incentive compatibility. ↑ Cited on pages [305](#), [307](#), [315](#), and [322](#).
- [204] Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974. ↑ Cited on page [167](#).
- [205] Agent Commerce Protocol. OpenAI and Stripe, 2025. <https://agentcommerce.dev>. ↑ Cited on page [447](#).
- [206] Universal Commerce Protocol. Specification and reference implementation (Apache-2.0), Google, Shopify, et al., 2026. <https://ucp.dev>. ↑ Cited on pages [126](#), [294](#), [447](#), and [514](#).

- [207] Peter J. Ramadge and W. Murray Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. (Controllability and the supramal controllable sublanguage — the general boundary of which this chapter’s Synchronous State Decidability theorem is the commit-only instance.) ↑ Cited on pages 33, 35, and 77.
- [208] Peter J. Ramadge and W. Murray Wonham. The Control of Discrete Event Systems. *Proceedings of the IEEE*, 77(1):81–98, 1989. ↑ Cited on page 77.
- [209] Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991. ↑ Cited on page 447.
- [210] Venkatesh Rao. A Big Little Idea Called Legibility. *ribbonfarm*, 2010. <https://www.ribbonfarm.com/2010/07/26/a-big-little-idea-called-legibility/> ↑ Cited on pages 183 and 246.
- [211] Rate-Distortion-Classification Representation Theory for Bernoulli Sources. arXiv:2601.11919, 2026; see also the universal rate-distortion-classification line, IEEE ITW 2025. ↑ Cited on page 213.
- [212] V. Ren et al. Agent Name Service (ANS): A Universal Directory for Secure AI Agent Discovery and Interoperability. arXiv:2505.10609, 2025. ↑ Cited on pages 220 and 247.
- [213] *Reputation is Amortized Verification: Inspection Games for Agent Economies*. Paper 3 of the Port Daddy Harbor research program, 2026. (The companion technical paper: proves the bonded-inspection stage-game threshold $\rho^* = G/(dB)$, its confiscation-convention fork $\rho_c^* = G/(d(G+B))$, and the audit-tower contraction under sealed multi-clique sampling.) ↑ Cited on pages 379, 379, 380, 382, and 384.
- [214] Paul Resnick, Richard Zeckhauser, John Swanson, and Kate Lockwood. The Value of Reputation on eBay: A Controlled Experiment. *Experimental Economics*, 9(2):79–101, 2006. ↑ Cited on pages 262, 295, and 374.
- [215] Ronald L. Rivest and Butler Lampson. SDSI — A Simple Distributed Security Infrastructure. Technical report, 1996. ↑ Cited on page 92.
- [216] Michael Robinson. Sheaves Are the Canonical Data Structure for Sensor Integration. *Information Fusion*, 36:208–224, 2017. ↑ Cited on page 492.

- [217] Jean-Charles Rochet and Jean Tirole. Platform Competition in Two-Sided Markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003. ↑ Cited on pages 295, 331, 373, and 376.
- [218] J.-C. Rochet & J. Tirole. “Two-Sided Markets: A Progress Report.” *RAND Journal of Economics* 37(3):645–667, 2006. ↑ Cited on pages 331 and 373.
- [219] R. Rogers, A. Roth, J. Ullman, and S. Vadhan. Privacy odometers and filters: pay-as-you-go composition. In *Advances in Neural Information Processing Systems* 29, pages 1921–1929, 2016. ↑ Cited on pages 151, 152, and 168.
- [220] Michael Rothschild and Joseph Stiglitz. Equilibrium in Competitive Insurance Markets: An Essay on the Economics of Imperfect Information. *Quarterly Journal of Economics*, 90(4):629–649, 1976. ↑ Cited on page 430.
- [221] A. Russo and A. Sabelfeld. Dynamic vs. static flow-sensitive security analysis. In *Proc. 23rd IEEE Computer Security Foundations Symposium (CSF)*, pages 186–199, 2010. ↑ Cited on pages 134 and 167.
- [222] A. Sabelfeld and A. C. Myers. A model for delimited information release. In *Software Security — Theories and Systems (ISSS 2003)*, pages 174–191. Springer, 2004. ↑ Cited on pages 144, 146, and 167.
- [223] A. Sabelfeld and D. Sands. Declassification: dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009. ↑ Cited on pages 144 and 167.
- [224] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. ↑ Cited on pages 5, 11, 73, and 85.
- [225] T. J. Schaefer. “The Complexity of Satisfiability Problems.” *STOC*, pp. 216–226, 1978. ↑ Cited on page 345.
- [226] Philipp Schindler, Aljosha Judmayer, Nicholas Stifter, and Edgar Weippl. ETHDKG: Distributed Key Generation with Ethereum Smart Contracts. IACR ePrint 2019/985. ↑ Cited on page 467.
- [227] David Schmeidler. Subjective Probability and Expected Utility without Additivity. *Econometrica*, 57(3):571–587, 1989. ↑ Cited on page 223.
- [228] Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. ↑ Cited on pages 30, 36, 73, 196, and 254.
- [229] James C. Scott. *Seeing Like a State: How Certain Schemes to Improve the Human Condition Have Failed*. Yale University Press, 1998. ↑ Cited on pages 183, 183, and 246.

- [230] James C. Scott. *Two Cheers for Anarchism*. Princeton University Press, 2012. ↑ Cited on pages 239 and 246.
- [231] Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. <https://doi.org/10.1086/259614> ↑ Cited on pages 407 and 447.
- [232] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units (BPE). In *ACL*, 2016. ↑ Cited on page 228.
- [233] Adi Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979. ↑ Cited on page 452.
- [234] Yaron Sheffer, Dick Hardt, and Michael Jones. JSON Web Token Best Current Practices. RFC 8725 (BCP 225), IETF, February 2020. <https://doi.org/10.17487/RFC8725> ↑ Cited on pages 87, 114, and 128.
- [235] Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. BFT Protocol Forensics. In *Proc. ACM Conference on Computer and Communications Security (CCS)*, pp. 1722–1743, 2021. ↑ Cited on page 494.
- [236] Thomas B. Sheridan and William L. Verplank. *Human and Computer Control of Undersea Teleoperators*. Man–Machine Systems Laboratory, Department of Mechanical Engineering, MIT; ONR Technical Report, 1978. NTIS ADA057655. ↑ Cited on pages 40, 150, and 169.
- [237] Thomas B. Sheridan. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press, 1992. ↑ Cited on page 181.
- [238] H. Si, O. O. Koyluoglu, and S. Vishwanath. Lossy Compression of Exponential and Laplacian Sources Using Expansion Coding. arXiv:1308.2338, 2013. ↑ Cited on page 213.
- [239] H. A. Simon. “Designing Organizations for an Information-Rich World.” In M. Greenberger (ed.), *Computers, Communications, and the Public Interest*, pp. 37–72. Johns Hopkins Press, 1971. ↑ Cited on page 329.
- [240] Daniel D. Sleator and Robert E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985. ↑ Cited on page 233.
- [241] G. Smith. On the foundations of quantitative information flow. In *Proc. FoSSaCS 2009*, LNCS 5504, pp. 288–302, 2009. ↑ Cited on pages 160 and 168.
- [242] Michael Spence. Job Market Signaling. *Quarterly Journal of Economics*, 87(3):355–374, 1973. ↑ Cited on page 196.

- [243] Daniel A. Spielman and Shang-Hua Teng. Nearly-Linear Time Algorithms for Graph Partitioning, Graph Sparsification, and Solving Linear Systems. In *Proc. 36th ACM Symposium on Theory of Computing (STOC)*, pp. 81–90, 2004. ↑ Cited on page [494](#).
- [244] D. Spivak & R. Kent. “Ologs: A Categorical Framework for Knowledge Representation.” *PLoS ONE* 7(1):e24274, 2012. ↑ Cited on page [374](#).
- [245] K. Stergiou & M. Koubarakis. “Backtracking Algorithms for Disjunctions of Temporal Constraints.” *Artificial Intelligence* 120(1):81–117, 2000. ↑ Cited on page [345](#).
- [246] Marilyn Strathern. “Improving Ratings”: Audit in the British University System. *European Review*, 5(3):305–321, 1997. (Goodhart’s law.) ↑ Cited on pages [227](#), [247](#), and [295](#).
- [247] I. E. Sutherland. “A Futures Market in Computer Time.” *Communications of the ACM* 11(6):449–451, 1968. ↑ Cited on page [329](#).
- [248] Steven Tadelis. Reputation and Feedback Systems in Online Platform Markets. *Annual Review of Economics*, 8:321–340, 2016. ↑ Cited on pages [262](#), [295](#), and [374](#).
- [249] Jason Teutsch and Christian Reitwiessner. A Scalable Verification Solution for Blockchains. arXiv:1908.04756, 2017. ↑ Cited on page [305](#).
- [250] Guy Theraulaz and Eric Bonabeau. A Brief History of Stigmergy. *Artificial Life*, 5(2):97–116, 1999. ↑ Cited on page [406](#).
- [251] William R. Thompson. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3/4):285–294, 1933. ↑ Cited on page [296](#).
- [252] Jean Tirole. Hierarchies and Bureaucracies: On the Role of Collusion in Organizations. *Journal of Law, Economics, & Organization*, 2(2):181–214, 1986. ↑ Cited on page [305](#).
- [253] Andrew Tobin and Drummond Reed. The Inevitable Rise of Self-Sovereign Identity. Sovrin Foundation White Paper, 2017. ↑ Cited on page [467](#).
- [254] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019. ↑ Cited on pages [511](#) and [513](#).
- [255] S. Colombo Tosatto, G. Governatori & N. van Beest. “Business Process Full Compliance with Respect to a Set of Conditional Obligations in Polynomial Time.” arXiv:2001.10148, 2020. ↑ Cited on page [345](#).

- [256] S. Colombo Tosatto, G. Governatori & N. van Beest. “Proving Regulatory Compliance: Full Compliance Against an Expressive Unconditional Obligation is coNP-Complete.” arXiv:2105.05431, 2021. ↑ Cited on pages 345, 345, and 345.
- [257] Anne M. Treisman and Garry Gelade. A Feature-Integration Theory of Attention. *Cognitive Psychology*, 12(1):97–136, 1980. ↑ Cited on pages 228 and 246.
- [258] *Trustees of Dartmouth College v. Woodward*, 17 U.S. (4 Wheat.) 518 (1819). U.S. Supreme Court. Founding case for corporate legal personhood as a status distinct from the natural persons who compose or run the entity. ↑ Cited on page 264.
- [259] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *NeurIPS*, 2023. arXiv:2305.04388. ↑ Cited on page 185.
- [260] UCAN Working Group. UCAN Specification 1.0: User Controlled Authorization Network, 2024. ↑ Cited on page 513.
- [261] Universal Commerce Protocol. Google, Shopify, et al. Specification and reference implementation, Apache-2.0, 2026. <https://ucp.dev>; github.com/Universal-Commerce-Protocol/ucp. ↑ Cited on pages 374 and 376.
- [262] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *NeurIPS*, 2017. arXiv:1706.03762. ↑ Cited on page 229.
- [263] William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961. ↑ Cited on page 447.
- [264] A. Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2):117–186, 1945. ↑ Cited on pages 157 and 168.
- [265] Abraham Wald. *Sequential Analysis*. Wiley, 1947. ↑ Cited on page 225.
- [266] A. Wald and J. Wolfowitz. Optimum character of the sequential probability ratio test. *Annals of Mathematical Statistics*, 19(3):326–339, 1948. ↑ Cited on pages 157 and 168.
- [267] Edward J. Green and Robert H. Porter. Noncooperative collusion under imperfect price information. *Econometrica*, 52(1):87–100, 1984. ↑ Cited on page 420.
- [268] Joel S. Warm, Raja Parasuraman, and Gerald Matthews. Vigilance Requires Hard Mental Work and Is Stressful. *Human Factors*, 50(3):433–441, 2008. ↑ Cited on pages 203 and 246.

- [269] Kevin Werbach. *The Blockchain and the New Architecture of Trust*. MIT Press, 2018. ↑ Cited on page 392.
- [270] R. van der Meyden. What, indeed, is intransitive noninterference? (extended abstract). In *Proc. European Symposium on Research in Computer Security (ESORICS)*, LNCS 4734, pages 235–250, 2007. ↑ Cited on pages 164, 167, and 169.
- [271] J. Whitehouse, A. Ramdas, R. Rogers, and Z. S. Wu. Fully-adaptive composition in differential privacy. In *Proc. 40th International Conference on Machine Learning (ICML)*, PMLR 202, pages 36990–37007, 2023. ↑ Cited on pages 153 and 168.
- [272] Christopher D. Wickens. Multiple Resources and Performance Prediction. *Theoretical Issues in Ergonomics Science*, 3(2):159–177, 2002. ↑ Cited on pages 200 and 246.
- [273] Bernard Williams. The Self and the Future. *The Philosophical Review*, 79(2):161–180, 1970. ↑ Cited on page 268.
- [274] Charles Wilson. A Model of Insurance Markets with Incomplete Information. *Journal of Economic Theory*, 16(2):167–207, 1977. ↑ Cited on page 430.
- [275] Menahem E. Yaari. The Dual Theory of Choice under Risk. *Econometrica*, 55(1):95–115, 1987. ↑ Cited on page 223.
- [276] Neal E. Young. On-line file caching. *Algorithmica*, 33(3):371–383, 2002. ↑ Cited on page 233.
- [277] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The True Cost of Containing: A gVisor Case Study. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2019. ↑ Cited on page 73.
- [278] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2023. ↑ Cited on pages 262, 300, 362, and 374.
- [279] Philip R. Zimmermann. *The Official PGP User’s Guide*. MIT Press, 1995. ↑ Cited on page 504.

Image credits

- **Leslie Lamport:** photograph by Andrej Bauer, Wikimedia Commons (*Leslie Lamport September 2006.jpg*), CC BY-SA 2.5 si, <https://creativecommons.org/licenses/by-sa/2.5/si/deed.en>; shared under the same licence; cropped and converted to duotone for this edition.
- **Elinor Ostrom:** photograph by Holger Motzkau, Wikimedia Commons (*Elinor Ostrom close-up (cropped).jpg*), © Holger Motzkau 2010, Wikipedia/Wikimedia Commons, CC BY-SA 3.0, <https://creativecommons.org/licenses/by-sa/3.0>; shared under the same licence; cropped and converted to duotone for this edition.
- **Robert Aumann:** photograph by David Orban, Wikimedia Commons (*Aumann, Robert (1930).jpg*), originally posted to Flickr, CC BY 2.0, <https://creativecommons.org/licenses/by/2.0>; cropped and converted to duotone for this edition.
- **Butler Lampson:** photograph by the Royal Society, Wikimedia Commons (*Butler Lampson Royal Society.jpg*), CC BY-SA 4.0, <https://creativecommons.org/licenses/by-sa/4.0>; shared under the same licence; cropped and converted to duotone for this edition.
- **Derek Parfit:** photograph by Anna Riedl, Wikimedia Commons (*Derek Parfit at Harvard-April 21, 2015-Effective Altruism.jpg*), CC BY-SA 4.0, <https://creativecommons.org/licenses/by-sa/4.0>; shared under the same licence; cropped and converted to duotone for this edition.
- **Thomas Hobbes, *Leviathan* (1651) title page:** engraving by Abraham Bosse, digitised by the British Library, Wikimedia Commons (*Leviathan - Hobbes' Leviathan (1651), title page - BL.jpg*), public domain (CC0 Universal Public Domain Dedication), <http://creativecommons.org/publicdomain/zero/1.0/deed.en>; cropped and converted to duotone for this edition.
- **James C. Scott:** photograph by SOAS University of London, Wikimedia Commons (*James C Scott 2016.jpg*), CC BY 3.0, <https://creativecommons.org/licenses/by/3.0>; cropped and converted to duotone for this edition.
- **W. Murray Wonham:** photograph by Wikimedia Commons user "Dal centro della mia vita venne una grande fontana" (*Nonofficial.jpg*, the uncropped source of *Wonham.jpg*), CC BY-SA 4.0, <https://creativecommons.org/licenses/by-sa/4.0>; shared under the same licence; the source is a group photograph

captioned "Professor Wonham with students" and this edition's plate is cropped down to Wonham alone, then converted to duotone; the crop is small and enlarged for print, so definition is soft.

- **Abraham Wald:** photograph by Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach (Oberwolfach Photo Collection), https://opc.mfo.de/detail?photo_id=4398, CC BY-SA 2.0 de, <https://creativecommons.org/licenses/by-sa/2.0/de/deed.en>; shared under the same licence; converted to duotone for this edition.
- **Claude Shannon:** photograph by Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach (Oberwolfach Photo Collection), https://opc.mfo.de/detail?photo_id=3807, CC BY-SA 2.0 de, <https://creativecommons.org/licenses/by-sa/2.0/de/deed.en>; shared under the same licence; converted to duotone for this edition.
- **Ada Lovelace:** watercolour painting by Alfred Edward Chalon, Science Museum Group, Wikimedia Commons (*Ada Lovelace portrait.jpg*), public domain; cropped and converted to duotone for this edition.